Mastering Ethereum: 第2版

本書は「Mastering Ethereum」の第2版です。以下のリンクから書籍を閲覧・購入できます。

- Amazonで書籍を購入
- オンライン版を読む

著者

- アンドレアス・M・アントノプロス
- ギャヴィン・ウッド
- カルロ・パリージ
- アレッサンドロ・マッツァ
- ニッコロ・ポッツォリーニ

ライセンス

本書はクリエイティブ・コモンズ・ライセンスに基づいて提供しています。出版後最初の12ヶ月間はCC BY-NC-ND 4.0ライセンス（表示-非営利-改変禁止）で利用できます。12ヶ月経過後はCC BY-SA 4.0ライセンス（表示-継承）で利用できるようになります。

はじめに

本書は、私（カルロ・パリージ、別名Blackie）、アレッサンドロ・マッツァ、ニッコロ・ポッツォリーニの共同作業です。初版は2016年から2019年にかけてアンドレアス・M・アントノプロスとギャヴィン・ウッド博士が執筆し、もちろん本書にも大きな影響を与えました。

2023年11月、いくつかの幸運な偶然が重なり、私とアンドレアスはグラスゴーで会いました。ビールを飲みながらサインをもらった後、『Mastering Ethereum』の第2版を執筆する予定があるか尋ねました。初版は傑作ですが、時間の経過とともに内容が古くなっていたからです。2019年の出版当時、イーサリアムはまだproof-of-workを使用しており、ロードマップも大きく異なっていました。

アンドレアスの返答は、第2版を執筆する予定はないというものでした。ただし、この会話がきっかけとなり、私自身がプロジェクトを引き受けるアイデアが生まれました。グラスゴーでの会合から1日も経たないうちに、私はオライリーと第2版執筆の可能性について話し合っていました。

これが大きく重要な仕事になることは、すぐにわかりました。光栄に思う一方で、十分な仕事ができるか不安でもありました。長年アンドレアスの作品のファンであり、2014年から2015年にかけてビットコインを深く理解できたのもアンドレアスのおかげです。だからこそ、助けが必要だとわかっていました。

最初に頭に浮かんだのはアレッサンドロでした。話が現実味を帯びるとすぐにテキストメッセージを送り、一緒に参加しないかと誘いました。アレッサンドロは有償かどうかも含め条件を何も聞かずに、即座に喜んで引き受けてくれました。

ニッコロを説得するのは少し大変でした。参加を承諾してもらうまでに1ヶ月かかりました。幸い、私は非常に頑固で、ノーという返事を受け入れるつもりはありませんでした。1ヶ月後、ニッコロはついに同意してくれました。こうしてチーム全員が揃い、『Mastering Ethereum: Second Edition』プロジェクトが正式に始まりました。

本書を読んだすべての読者が、少しでも知識を得てくれることを願っています。『Mastering Ethereum』は私や何千人もの人々に多くのことを教えてくれた本です。同じレベルの品質を維持するために懸命に努力しました。完成までに2年間の調査と執筆を要し、簡単だったとは言えません。

また、全員がイタリア人のチームでこのプロジェクトを実現できたことを非常に誇りに思っています。イタリアの暗号通貨コミュニティにとっても誇りとなることを願っています。

本書の利用方法

本書は、イーサリアムのリファレンスマニュアルとしても、一からイーサリアムを学ぶための入門書としても使えるように構成されています。最初の2章は初心者向けのやさしい入門で、少し技術的なスキルがあれば誰でも例を試すことができます。この2章でイーサリアムの基本を理解し、基本的なツールを使えるようになります。第3章以降はプログラマを対象としており、技術的なトピックやプログラミングの例を多く含んでいますが、大部分は誰でも理解できるはずです。

リファレンスマニュアルであり、イーサリアムに関する包括的な解説書でもあるため、本書には必然的に重複する内容が含まれています。たとえばガスなどのトピックは、他のトピックを理解するために早い段階で導入する必要がありますが、専用のセクションでも詳しく解説しています。

また、本書の索引を使えば、キーワードから具体的なトピックや関連セクションを簡単に見つけることができます。

対象読者

本書は主にすべての人を対象としています。スマートコントラクトやブロックチェーンの仕組み、その使い方、そしてスマートコントラクトやDAppの開発方法を学ぶことができます。最初の数章は、イーサリアム初心者向けの入門としても最適です。

コード例

例はSolidity、Vyper、JavaScriptで記述し、UnixライクなOSのコマンドラインを使って説明しています。各言語のコンパイラ、インタープリタ、ライブラリを最小限インストールすれば、ほとんどのOSでコードスニペットを再現できます。必要に応じて、基本的なインストール手順とその出力例をステップごとに示します。

すべてのコードスニペットでは可能な限り実際の値と計算を使用しています。そのため、例を順に追いながら、同じコードで同じ結果を確認できます。たとえば、秘密鍵とそれに対応する公開鍵やアドレスはすべて実在するものです。サンプルのトランザクション、コントラクト、ブロック、ブロックチェーン参照もすべて実際のイーサリアムブロックチェーン上に存在し、公開台帳の一部として確認できます。

本書におけるイーサリアムアドレスとトランザクション

本書で使用しているイーサリアムアドレス、トランザクション、鍵、QRコード、ブロックチェーンデータは、ほとんどが実在するものです。そのため、ブロックチェーンを閲覧したり、例として提供しているトランザクションを確認したり、独自のスクリプトやプログラムで取得したりできます。

ただし、本書に掲載されているアドレスの秘密鍵は「焼却」済みです。これらのアドレスに送金すると、資金は永遠に失われるか、むしろ高い確率で流用されます。本書を読んだ人なら誰でも、掲載されている秘密鍵を使って資金を引き出せるからです。

> **警告**
>
> 本書に記載されているいずれのアドレスにも送金しないでください。あなたのお金は他の読者に奪われるか、永遠に失われます。

本書で使用されている表記規則

本書では以下の表記規則を使用します。

イタリック体

新しい用語、URL、メールアドレス、ファイル名、ファイル拡張子を示す。

`等幅フォント`

プログラムリスト、および段落内で変数名や関数名、データベース、データ型、環境変数、ステートメント、キーワードなどのプログラム要素を示す。

`太字等幅フォント`

ユーザーが文字通り入力すべきコマンドやその他のテキストを示す。

`イタリック体等幅フォント`

ユーザーが指定する値、またはコンテクストに応じて決まる値に置き換えるテキストを示す。

> **ヒント**
>
> この要素はヒントまたは提案を示す。

注記

この要素は一般的な注記を示す。

警告

この要素は警告または注意を示す。

コード例の使用

コラボレーションへの取り組みの一環として、オライリー・メディアと協力し、本書をクリエイティブ・コモンズ・ライセンスの下で公開しました。

技術的な質問やコード例の使用に関する問題は、support@oreilly.comまでメールでお問い合わせください。

本書は、皆様の仕事を支援するためにあります。本書のサンプルコードは、プログラムやドキュメントで自由に使用できます。コードの大部分を複製しない限り、許可は不要です。たとえば、本書のコードを一部使ってプログラムを作成する場合、許可は不要です。一方、オライリーの書籍のサンプルを販売または配布する場合は許可が必要です。本書を引用してサンプルコードで質問に答える場合、許可は不要です。本書のサンプルコードの大部分を製品のドキュメントに組み込む場合は許可が必要です。

帰属表示は歓迎しますが、必須ではありません。帰属表示には通常、タイトル、著者、出版社、ISBNを含めます。例えば、「*Mastering Ethereum*、第2版、アンドレアス・M・アントノプロス、ギャヴィン・ウッド、カルロ・パリージ、アレッサンドロ・マッツァ、ニッコロ・ポッツォリーニ著（オライリー）。Copyright 2026 カルロ・パリージ、アレッサンドロ・マッツァ、ニッコロ・ポッツォリーニ、978-1-098-16842-1。」

コード例の使用が公正使用または上記の許可の範囲外と思われる場合は、permissions@oreilly.comまでお気軽にお問い合わせください。

オライリー・オンライン・ラーニング

注記

40年以上にわたり、オライリー・メディアはテクノロジーとビジネスのトレーニング、知識、洞察を提供し、企業の成功を支援してきた。

専門家とイノベーターによるユニークなネットワークが、書籍、記事、オンライン学習プラットフォームを通じて知識と専門性を共有しています。オライリーのオンライン学習プラットフォームでは、ライブトレーニングコース、詳細な学習パス、インタラクティブなコーディング環境に加え、オライリーと200以上の出版社によるテキストやビデオの膨大なコレクションにオンデマンドでアクセスできます。詳細はoreilly.comをご覧ください。

お問い合わせ

本書に関するご意見やご質問は、出版社までお寄せください。

O'Reilly Media, Inc.

141 Stony Circle, Suite 195

Santa Rosa, CA 95401

800-889-8969（米国またはカナダ）

707-827-7019（国際または現地）

707-829-0104（ファックス）

support@oreilly.com

oreilly.com/about/contact.html

本書のウェブページでは、正誤表や追加情報を掲載しています。
https://oreil.ly/MasteringEthereum2eからアクセスできます。

書籍やコースに関するニュースや情報は、https://oreilly.comをご覧ください。

LinkedInで見つけてください

YouTubeでご覧ください

Carloへの連絡先

CarloのYouTubeチャンネルを購読する

CarloをTwitter/Xでフォローする

CarloとLinkedInで繋がる

メール: carlo.parisi01234@gmail.com

Alessandroへの連絡先

AlessandroをGitHubでフォローする

AlessandroとLinkedInで繋がる

AlessandroをTwitter/X (イタリア語プロフィール)でフォローする

AlessandroをTwitter/X (英語プロフィール)でフォローする

AlessandroのYouTubeチャンネルを購読する

Niccolòへの連絡先

NiccolòをTwitter/Xでフォローする

NiccolòとLinkedInで繋がる

Carloからの謝辞

私のキャリアや人生の多くは、アンドレアスの作品のおかげです。『Mastering Bitcoin』の初版によって、2014年から2015年にかけてビットコインの可能性を真に理解できました。その結果、暗号通貨の分野でキャリアを築く決意をしました。当時、包括的なリソースはほとんどなく、『Mastering Bitcoin』はコミュニティ全体にとって計り知れない贈り物でした。あの素晴らしい作品がなければ、今のキャリアも人生も大きく違っていたでしょう。『Mastering Ethereum』についても同様です。アンドレアス、本当にありがとうございます。

高校のコンピュータサイエンスの先生であるニコラ・ルイージ・グリエルモ・ディ・ナンナ先生にも感謝します。先生がいなければ、プログラミングやブロックチェーンへの情熱を見つけることはできなかったでしょう。良い先生は生徒の人生を変えると言いますが、まさにそのとおりでした。コンピュータサイエンスへの情熱を分かち合い、素晴らしい手本を示してくださったことに感謝します。

また、本書の執筆に多大な協力をしてくれたアレッサンドロとニッコロ、そして素晴らしいテクニカルレビュアの皆様にも感謝します。

あらゆる段階でサポートしてくださったオライリーのミシェル・スミスとシーラ・エヴァンスに感謝します。

イタリアのコミュニティ、特にDiscord、YouTube、Twitterのコミュニティの皆様に感謝します。このプロジェクトをはじめ、多くのプロジェクトに挑戦する自信を与えてくれました。

素晴らしいイーサリアムコミュニティに感謝します。皆様の継続的なサポートと貢献がなければ、これほど深い調査と執筆はできませんでした。

最後に、すべての活動を応援し、情熱と興味を自由に追求させてくれた素晴らしい家族に感謝します。

Alessandroからの謝辞

感謝すべきことがたくさんあります。まず、カルロが連絡をくれ、本書を執筆するという素晴らしい機会を与えてくれたことに感謝します。イタリアのカタンツァーロで初めて直接会った時のことを今でも覚えています。

大学の教授であるルカ・ジュッツィ博士に感謝します。ビットコインとシュノアデジタル署名アルゴリズムをテーマにした論文の執筆を許可してくださり、暗号通貨分野でのキャリアが始まりました。

自信を与え、間接的に今の私を導いてくれたイタリアのコミュニティに心からの感謝を。このコミュニティのサポートがなければ、おそらくカルロに会うこともなかったでしょう。

オライリー、特にこの旅を通して導いてくれたシーラ・エヴァンスとミシェル・スミスに感謝します。細やかな調整とサポート、そしてすべてのレビュアの協力が、本書の実現を可能にしました。

最後に、常に私を支え、愛してくれている家族とガールフレンドのアレッサンドラに感謝します。

Niccolòからの謝辞

カルロに、一緒に本書を執筆する機会を与えてくれたことに感謝します。この2年間、多くのプロジェクトで協力し、たくさんの笑い、海外旅行、そしてこの本を共有してきました。カルロと一緒に仕事ができたことで、この旅は楽しくやりがいのあるものになりました。

『Mastering Ethereum』の初版を書いてくれたアンドレアスとギャヴィンに感謝しています。あの本がブロックチェーンの世界への扉を開いてくれました。多くの人々にとっても同様でしょう。アンドレアスとギャヴィンの仕事は、今日私たちが属するこの素晴らしいブロックチェーンコミュニティの構築と成長に貢献しました。

最初から最後までサポートしてくれたオライリーのミシェル・スミスとシーラ・エヴァンスに心から感謝します。書類作成を手伝い、スケジュール管理を行い、本書を実現するまでの全プロセスを導いてくれました。

家族、友人、そしてガールフレンドのジュディッタ、いつもそばにいてくれてありがとう。執筆の数ヶ月間、忙しいスケジュールとストレスに耐えてくれました。アイデアを話したいときには耳を傾け、締め切りで集まりに参加できないときには理解してくれ、困難なときには励まし続けてくれました。その絶え間ないサポートが、続ける強さと集中力を与えてくれました。この本は私だけの作品ではありません。いつも示してくれた愛とサポートのおかげで存在しています。

貢献者

テクニカルレビュアの皆様に感謝します。

- ベン・エジントン
- カレブ・レント
- ブライアン・ウー
- ゴンサロ・マガリャンイス

本書の品質向上に多大な貢献をしてくださいました。心から感謝します。

第1章　イーサリアムとは何か

イーサリアムはしばしば「ワールドコンピュータ」と呼ばれます。ではそれはどういう意味でしょうか。まずコンピュータサイエンスの観点から説明し、次にイーサリアムの機能や特徴をより実践的に分析します。その上で、ビットコインをはじめとする非中央集権的な情報交換プラットフォーム（いわゆる*ブロックチェーン*）と比較してみましょう。

コンピュータサイエンスの観点から見ると、イーサリアムは決定論的でありながら事実上無制限のステート空間を持つステートマシンです。グローバルにアクセス可能なシングルトンステート（世界中で単一の状態）と、そのステートに変更を適用する仮想マシンで構成されています。

より実践的な観点から見ると、イーサリアムは*スマートコントラクト*と呼ばれるプログラムを実行する、オープンソースかつグローバルな非中央集権型コンピューティング基盤です。ブロックチェーンを使用してシステムの状態変更を同期・保存し、*ether*と呼ばれる暗号通貨を使用して実行リソースのコストを計測・制限します。

イーサリアムプラットフォームを使えば、開発者は経済的な機能が組み込まれた強力な分散アプリケーションを構築できます。高可用性や監査可能性、透明性、中立性を提供しつつ、検閲を削減または排除し、特定のカウンターパーティリスク（取引相手の不履行リスク）を軽減します。

イーサリアムとビットコインの比較

多くの人は暗号通貨、特にビットコインの経験を経てイーサリアムに接します。イーサリアムは他のオープンブロックチェーンと多くの要素を共有しています。具体的には以下のようなものです：

- 参加者を接続するピアツーピア（P2P）ネットワーク
- 状態更新の同期のためのビザンチン耐障害性コンセンサスアルゴリズム（一部の参加者が不正行動をとっても合意形成できる仕組み）
- デジタル署名やハッシュ値などの暗号技術の基本要素（暗号プリミティブ）の使用
- デジタル通貨（ether）

一方で、イーサリアムの目的と構造はビットコインを含む先行するオープンブロックチェーンとは大きく異なります。

イーサリアムの主な目的は、デジタル通貨の決済ネットワークになることではありません。デジタル通貨であるetherはイーサリアムの運用に不可欠ですが、ワールドコンピュータとしてのイーサリアムプラットフォームの利用料を支払うための*ユーティリティ通貨*として設計されています。

非常に限定されたスクリプト言語を持つビットコインとは異なり、イーサリアムは任意かつ無制限の複雑さを持つコードを実行できる仮想マシンを備えた、汎用的でプログラム可能なブロックチェーンとして設計されています。ビットコインのScript言語が送金条件（使用条件）の単純な真偽評価に意図的に制約されているのに対し、イーサリアムの言語は*チューリング完全*です。つまり、イーサリアムは汎用コンピュータとして機能できます。

2022年9月、イーサリアムはThe Mergeアップグレードによりコンセンサスモデルをproof-of-work（PoW）からproof-of-stake（PoS）に移行し、ビットコインとの差別化をさらに進めました。この変更は環境負荷の削減に対するイーサリアムの取り組みを示すだけでなく、汎用コンピューティングプラットフォームとしての革新的なビジョンに沿ったものであり、スケーラビリティとセキュリティの機能も強化しています。

ブロックチェーンの構成要素

オープンなパブリックブロックチェーンの構成要素は（通常）以下の通りです：

- 参加者を接続し、標準化された「ゴシップ」プロトコル（情報をネットワーク全体に伝搬させる通信方式）に基づいてトランザクションと、検証済みトランザクションから成るブロックを伝播するP2Pネットワーク
- 状態遷移を表すメッセージ（トランザクションの形式）
- トランザクションの構成要件と有効な状態遷移の条件を規定するコンセンサスルールの集合
- コンセンサスルールに従ってトランザクションを処理するステートマシン
- 検証・承認されたすべての状態遷移のジャーナルとして機能する、暗号学的に保護されたブロックの連鎖
- コンセンサスルールの執行に参加者の協力を強制することで、ブロックチェーンに対する制御を非中央集権化するコンセンサスアルゴリズム
- オープンな環境でステートマシンを経済的に保護するための、ゲーム理論に基づいたインセンティブ設計（例：PoWコスト＋ブロックリワード）
- これらの構成要素の1つ以上のオープンソースソフトウェア実装（「クライアント」）

これらの構成要素のすべて、または大部分は、通常は単一のソフトウェアクライアントに統合されています。例えばビットコインでは、リファレンス実装はビットコインコアのオープンソースプロジェクトが開発し、ビットコインクライアントとして実装しています。PoSへの移行前、イーサリアムも単一のクライアントを必要としていました。ただし現在はコンセンサス用と実行用の2つの異なるクライアントを利用しています。

リファレンス実装の代わりに、イーサリアムはリファレンス仕様に依拠しています。これは「イエローペーパー」に詳述された数学的記述であり、イーサリアムの開発を通じて一貫して更新されてきました。現在、イーサリアムコミュニティはコンセンサスと実行の両クライアントについて、Pythonで記述したリファレンス仕様への移行を進めています。このリファレンス仕様

に基づいて多数のクライアントが構築されています。このトピックについては第3章で詳しく掘り下げます。

図1-1はブロックチェーンの構成要素をグラフィカルに表現したものです。

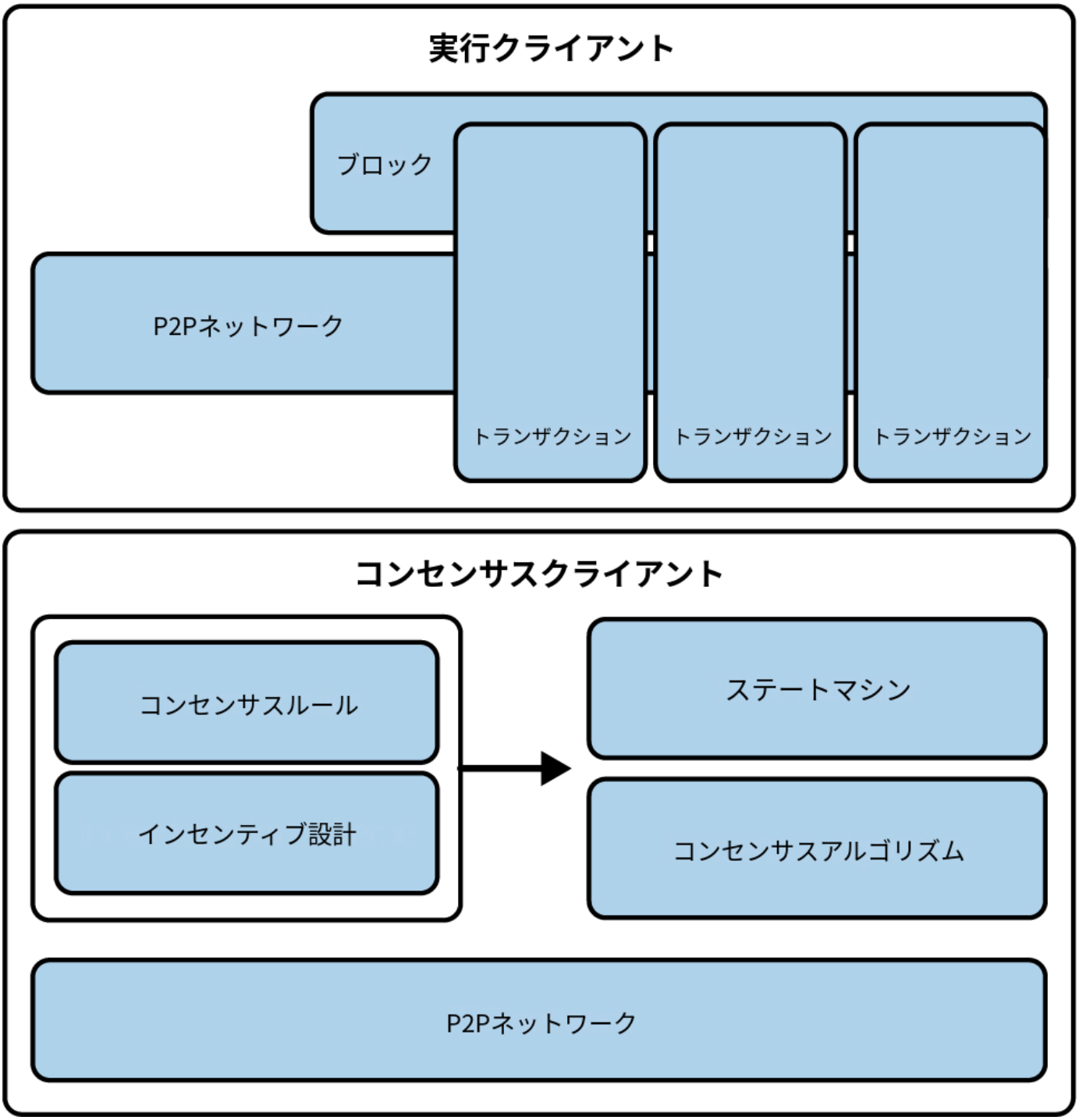


図1-1. ブロックチェーンの構成要素

過去には、*ブロックチェーン*という用語を、上記の構成要素すべてを包含する技術の組み合わせを指す簡略表現として使用していました。ただし今日では、異なる特性を持つ非常に多様なブロックチェーンが存在します。対象のブロックチェーンの特徴を理解するには修飾語が必要です。*オープン*、*パブリック*、*グローバル*、*非中央集権*、*中立*、*検閲耐性*といった言葉を使い、こ

れらの構成要素が実現する「ブロックチェーン」システムの重要な創発的特性（個々の要素には存在しないが、全体として現れる性質）を特定します。

すべてのブロックチェーンが同じように作られているわけではありません。多くの特性を共有していますが、ブロックチェーンはパーミッションレスかパーミッションドか、パブリックかプライベートかで大きく分類できます：

パーミッションレス

ビットコインやイーサリアムのようなパーミッションレスブロックチェーンは誰でもアクセスできる。これらの非中央集権的なネットワークでは誰でも参加でき、コンセンサスプロセスに関与し、データの読み書きを行い、透明性を通じて信頼を促進している。

パーミッションド

パーミッションドブロックチェーンはアクセスを制限し、認可された参加者だけがネットワークに参加して特定のアクションを実行できる。

パブリック

パブリックブロックチェーンは非中央集権的であり、すべての人に開放されている。ネットワーク活動への幅広い参加を可能にし、広範な分散とコンセンサスメカニズムによって透明性を確保している。

プライベート

プライベートブロックチェーンは、特定の参加者グループ（多くの場合、組織内または信頼できるパートナー間）にアクセスを制限している。

イーサリアムの誕生

すべての偉大なイノベーションは現実の問題を解決するものであり、イーサリアムも例外ではありません。イーサリアムは、人々がビットコインモデルの力を認識し、暗号通貨のアプリケーションを超えようとしていた時期に着想されました。ただし開発者たちはあるジレンマに直面していました。ビットコインの上に構築するか、新しいブロックチェーンを始めるかという選択です。

ビットコインの上に構築するということは、ネットワークの意図的な制約を受け入れた上で、その回避策を模索することを意味しました。限定されたトランザクションタイプやデータタイプ、データストアのサイズは、ビットコイン上で直接実行できるアプリケーションの種類を制限していました。直接実行できないアプリケーションにはオフチェーンレイヤーが必要となり、パブリックブロックチェーンの利点の多くが失われてしまいます。オンチェーンで自由度と柔軟性を求めるプロジェクトにとって、新しいブロックチェーンが唯一の選択肢でした。とはいえそれ

は、すべてのインフラ要素のブートストラップ、徹底的なテストなど、多大な作業を意味していました。

2013年末、若いプログラマーでビットコイン愛好家のヴィタリック・ブテリンは、ビットコインとMastercoin（初歩的なスマートコントラクトを提供するためにビットコインを拡張するオーバーレイプロトコル）の機能をさらに拡張することを考え始めました。同年10月、ヴィタリックはMastercoinチームに対し、Mastercoinの専用コントラクト言語を柔軟でスクリプト可能な（ただしチューリング完全ではない）コントラクトに置き換える、より一般化されたアプローチを提案しました。Mastercoinチームは感銘を受けましたが、この提案は開発ロードマップに組み込むにはあまりにも急進的な変更でした。

2013年12月、ヴィタリックはイーサリアムの背後にあるアイデアを概説したホワイトペーパーを共有し始めました。チューリング完全な汎用ブロックチェーンというアイデアです。数十人がこの初期草稿を目にしてフィードバックを提供し、ヴィタリックの提案を進化させる手助けをしました。

本書の原著者であるアンドレアス・M・アントノプロスとギャヴィン・ウッド博士の両名は、ホワイトペーパーの初期草稿を受け取りコメントしました。アントノプロスはこのアイデアに興味を持ち、スマートコントラクトの実行に関するコンセンサスルールを強制するために別のブロックチェーンを使用することや、チューリング完全言語の意味について、ヴィタリックに多くの質問をしました。アントノプロスは大きな関心を持ってイーサリアムの進捗を追い続けましたが、著書*Mastering Bitcoin*の執筆初期段階にあったため、イーサリアムへの直接参加はずっと後のことになりました。一方、ギャヴィンはヴィタリックに連絡を取り、C++プログラミングのスキルで協力を申し出た最初の人物の一人でした。ギャヴィンはイーサリアムの共同創設者、共同設計者、CTOとなりました。

ヴィタリックは「A Prehistory of the Ethereum Protocol」の投稿で次のように回想しています：

> これはイーサリアムプロトコルが完全に自分自身の創作物であった時期であった。しかし、ここから新たな参加者が加わり始めた。プロトコル側で最も傑出していたのはギャヴィン・ウッドである。

ギャヴィンは、イーサリアムをプログラム可能なマネーのプラットフォーム（デジタル資産を保持し、あらかじめ設定されたルールに従って転送するブロックチェーンベースのコントラクト）から、汎用コンピューティングプラットフォームへとビジョンを転換する上で大きな功績があります。この変化は強調点と用語の微妙な変化から始まり、やがて「Web3」構想への注目の高まりとともに影響が強くなっていきました。Web3はイーサリアムをWhisperとSwarmという他の2つの非中央集権的な技術とともに、一連の非中央集権的な技術の一部として捉えるものです（なお、Whisperプロトコルはその後廃止され、後継として分散型メッセージングプロトコルのWakuが開発されました）。2013年12月から、ヴィタリックとギャヴィンはそのアイデアを洗練・発展させ、イーサリアムのプロトコル層を共に構築していきました。

イーサリアムの創設者たちは、特定の目的を持たないブロックチェーンを考えていました。*プログラム*することで幅広い種類のアプリケーションをサポートできるものです。イーサリアムのような汎用ブロックチェーンを使えば、開発者はP2Pネットワーク、ブロックチェーン、コンセンサスアルゴリズムなどの基盤メカニズムを実装せずに、自身のアプリケーションをプログラムできます。イーサリアムはこれらの詳細を抽象化し、分散アプリケーションを記述するための決定論的で安全な環境を提供します。この考え方の転換は開発を容易にしただけでなく、ブロックチェーンにできることを根本的に拡大しました。分散金融やNFT、分散自律組織（DAO）といった、従来の単一目的ブロックチェーンでは実現不可能だった全く新しいセクターの基盤を築いたのです。

ビットコインの偽名の開発者であるサトシ・ナカモトと同様に、ヴィタリックとギャヴィンは単に新しい技術を発明しただけではありません。新しい発明と既存の技術を斬新な方法で組み合わせ、自らのアイデアを世界に証明するプロトタイプコードを提供しました。

創設者たちはビジョンの構築と洗練に何年も費やしました。そして2015年7月30日、最初のイーサリアムブロックがマイニングされました。ワールドコンピュータは世界にサービスを提供し始めたのです。

注記

ヴィタリック・ブテリンの記事「A Prehistory of the Ethereum Protocol」は2017年9月に公開され、イーサリアムの最初期の時代について魅力的な一人称視点の記述を提供しています。

イーサリアムの開発段階

イーサリアムの開発は4つの段階にわたって計画され、各段階で大きな変更が行われました。各段階には、後方互換性のない方法で機能を変更する*ハードフォーク*と呼ばれるサブリリースが含まれることがあります。

4つの主要な開発段階には、フロンティア、ホームステッド、メトロポリス、セレニティというコードネームが付けられています。執筆時点では最後の段階であるセレニティにあります。セレニティはさらに6つのサブステージに分けられ、The Merge、The Surge、The Scourge、The Verge、The Purge、The Splurgeというコードネームが付けられています。

それでは4つの開発段階について、その主な目的を説明しましょう：

フロンティア（2015年7月30日）

フロンティアはジェネシス（最初のイーサリアムブロックがマイニングされた時）にローンチされました。マイニングリグのセットアップやETHトークンの取引開始、最小限のネットワーク環境でのDAppテストを可能にし、マイナーと開発者のための基盤を準備しました。当初はブロ

ックのガスリミットが5000に設定されていましたが、2015年9月にこの制限が解除されてトランザクションが可能となりました。またこの時期に「ディフィカルティボム」が導入されました。イーサリアムの*ディフィカルティボム*は、時間の経過とともにマイニングのディフィカルティを指数関数的に増加させ、最終的にマイニングを実行不可能にするメカニズムです。これにより、元のPoWコンセンサスからエネルギー効率の高いPoSモデルへの移行が促進されました。

ホームステッド（2016年3月14日）

ブロック1,150,000で開始されたホームステッドは、主要なプロトコル更新（EIP-2、EIP-7、EIP-8）によりイーサリアムをより安全で安定させました。これらのアップグレードは開発者の利便性を向上させ、さらなるプロトコル改善への道を開きましたが、ネットワークはベータフェーズのままでした。

メトロポリス（2017年10月16日）

ブロック4,370,000から始まったメトロポリスは、ネットワーク機能の向上を目指し、DAppの作成とネットワーク全体のユーティリティを促進しました。このフェーズではビザンチウム、コンスタンティノープル、イスタンブールなどの重要なフォークにより、ガスコストの最適化やセキュリティの強化、レイヤー2（L2）スケーリングソリューションの導入が行われました。

ビザンチウムフォークはマイニングリワードを削減し、暗号学的な仕組みを実装しました。コンスタンティノープルフォークはガスコストをさらに最適化し、未作成のアドレスとの相互作用を可能にしました。イスタンブールは分散型サービス拒否（DDoS）攻撃に対するネットワークの耐性を高め、スケーラビリティとプライバシーを向上させるゼロ知識証明（zk-SNARKsとzk-STARKs）を導入しました。これらの強化は総合的にEthereum 2.0への道を整え、Ethereum 1.0の最終フェーズを表しています。

セレニティ（2022年9月15日）

セレニティは一般に*Ethereum 2.0*として知られ、イーサリアムをPoWからPoSコンセンサスメカニズムに変革する大規模なアップグレードです。イーサリアムをより持続可能にし、増大するユーザーとアプリケーションに対応できるようにすることに焦点を当てています。高いエネルギー消費とネットワーク輻輳に対処し、より堅牢で効率的なブロックチェーンへの道を開きます。

セレニティのアップグレードは、ネットワークの進化の特定の側面に対処するいくつかのサブステージに分かれています。主要な4つの開発段階は順次実装されてきましたが、セレニティの6つのサブステージは同時に開発されています。この並行アプローチは従来の開発手法からの戦略的な転換です。最初の4つの段階で築かれた基盤がセレニティのサブステージの同時開発を可能にしました。各サブステージはイーサリアムチェーンの異なる側面を改善するものであり、互いに独立しているため、より柔軟でダイナミックなアップグレードプロセスが可能になっています：

The Merge

The Mergeはイーサリアムのメインネットとビーコンチェーン（PoSコンセンサスを処理するサイドチェーン）を統合しました。これによりネットワークは正式にPoSに移行し、エネルギー

消費を大幅に削減しました。

The Surge

The Surgeはシャーディングを導入し、ネットワークをより小さく管理可能な部分に分割してイーサリアムのスケーラビリティを向上させ、1秒あたりのトランザクション数を増やします。

The Scourge

The Scourgeは中央集権化と検閲耐性の問題に取り組み、イーサリアムが非中央集権的でオープンなネットワークであり続けることを目指します。

The Verge

The VergeはVerkleツリーを実装し、ノードに必要なデータストアを削減してネットワークの効率性とスケーラビリティを向上させます。

The Purge

The Purgeはイーサリアムに保存される履歴データを削減し、ノード運用を簡素化し、ネットワークの輻輳を低減することを目指します。

The Splurge

The Splurgeは、すべての主要な変更の実装後にイーサリアムがスムーズかつ効率的に動作するよう、さまざまなマイナーアップグレードと最適化を含みます。

イーサリアム：汎用ブロックチェーン

元祖ブロックチェーンであるビットコインのブロックチェーンは、ビットコインの単位とその所有権のステートを追跡します。ビットコインは分散コンセンサス*ステートマシン*と考えることができ、トランザクションがコインの所有権を変更するグローバルな*状態遷移*を引き起こします。状態遷移はコンセンサスルールによって制約され、数ブロックのマイニング後にすべての参加者が（最終的には）システムの共通ステートに収束します。

イーサリアムも分散ステートマシンですが、通貨の所有権のステートだけを追跡するのではなく、汎用データストアの状態遷移を追跡します。つまり*キーバリューペア*として表現可能な任意のデータを保持できるデータストアです。キーバリューデータストアは、何らかのキーによって参照される任意の値を保持します。例えば「Book Title」というキーで参照される「Mastering Ethereum」という値です。これはほとんどの汎用コンピュータが使用するランダムアクセスメモリ（RAM）のデータストアモデルと同じ目的を果たします。

イーサリアムはコードとデータの両方を格納するメモリを持ち、このメモリの変化をイーサリアムブロックチェーンで追跡します。汎用のストアドプログラムコンピュータと同様に、イーサリアムは自身のステートマシンにコードをロードして*実行*し、結果として生じる状態変更をブロ

ックチェーンに保存できます。ほとんどの汎用コンピュータとの重要な違いは2つあります。イーサリアムの状態変更がコンセンサスルールに従うことと、ステートがグローバルに分散されていることです。イーサリアムは「任意のステートを追跡し、ステートマシンをプログラムして、コンセンサスの下で動作するワールドコンピュータを作れたらどうだろうか」という構想を実現したものです。

イーサリアムの構成要素

イーサリアムでは、ブロックチェーンシステムの構成要素（「ブロックチェーンの構成要素」で説明したもの）は、より具体的には以下の通りです：

P2Pネットワーク

イーサリアムは*イーサリアムメインネット*上で動作し、TCPポート30303で到達可能であり、*ÐΞVp2p*と呼ばれるプロトコルを実行します。

コンセンサスルール

イーサリアムの元のコンセンサスプロトコルは、リファレンス仕様である「イエローペーパー」で定義されたPoWモデルのEthashでした。その後、2022年9月のThe Mergeアップグレードの際にPoSに移行しました（第15章参照）。

トランザクション

イーサリアムのトランザクションは、（とりわけ）送信者、受信者、バリュー、データペイロードを含むネットワークメッセージです。

ステートマシン

イーサリアムの状態遷移は*イーサリアム仮想*マシン（EVM）が処理します。EVMはバイトコード（機械語命令）を実行するスタックベースの仮想マシンです。*スマートコントラクト*と呼ばれるEVMプログラムは、高水準言語（例：Solidity）で記述され、EVM上で実行するためにバイトコードにコンパイルされます。

データ構造

イーサリアムのステートは各ノードにローカルな*データベース*（通常はGoogleのLevelDB）として保存されます。トランザクションとシステムステートは*マークルパトリシアツリー*と呼ばれるシリアライズされたハッシュデータ構造に格納されています。

コンセンサスアルゴリズム

イーサリアムはエネルギー効率とスケーラビリティの向上のため、PoWからPoSコンセンサスメカニズムに移行しました。PoSではバリデータが暗号通貨をステークし、トランザクションの検証、新しいブロックの作成、ネットワークセキュリティの維持を行う権利を獲得します。イー

サリアムのPoSはCasper FFG（Casper the Friendly Finality Gadget）とLMD-GHOST（Latest Message Driven GHOST）という2つの異なるアルゴリズムを融合した設計です（詳細は第15章で説明します）。

経済的セキュリティ

イーサリアムはGasperと呼ばれるPoSアルゴリズムでブロックチェーンに経済的セキュリティを提供します。Gasperの仕組みについてはファイナリティとバリデータの協調における役割を含め、第15章で詳しく説明します。

クライアント

イーサリアムには、実行クライアントとコンセンサスクライアントの相互運用可能な複数の実装があり、最も著名なものとして、実行用の*go-ethereum*（Geth）とNethermind、コンセンサス用のPrysmとLighthouseがあります。

ここで言及した技術の詳細については、以下を参照してください：

- イーサリアム「イエローペーパー」
- コンセンサスクライアントのPython仕様
- 実行クライアントのPython仕様

イーサリアムとチューリング完全性

イーサリアムについて読み始めるとすぐに*チューリング完全*という用語に出会うでしょう。イーサリアムはビットコインとは異なりチューリング完全であると言われます。これは正確にはどういう意味でしょうか。

この用語はコンピュータサイエンスの父と見なされるイギリスの数学者アラン・チューリングにちなんでいます。1936年、チューリングはシーケンシャルメモリ（無限長の紙テープに似たもの）上のシンボルを読み書きして操作するステートマシンで構成されるコンピュータの数学的モデルを作成しました。この構成によりチューリングは*普遍的計算可能性*、すなわちすべての問題が解決可能かどうかという問いに数学的基盤を提供しました（答えは否定的でした）。チューリングは計算不可能な問題のクラスが存在することを証明しました。具体的には*停止問題*（任意のプログラムとその入力が与えられた場合に、そのプログラムが最終的に停止するかどうかを判定できるか）が解決不能であることを証明しました。

チューリングはさらに、任意のチューリングマシンをシミュレートするために使用できるシステムを*チューリング完全*と定義しました。このようなシステムは*ユニバーサルチューリングマシン*（UTM）と呼ばれます。

EVMと呼ばれるステートマシンでストアドプログラムを実行し、メモリにデータを読み書きできることが、イーサリアムをチューリング完全なシステム、すなわちUTMにしています。イーサ

リアムは有限メモリの制約はあるものの、任意のチューリングマシンで計算可能なあらゆるアルゴリズムを計算できます。

イーサリアムの画期的な革新は、ストアドプログラムコンピュータの汎用コンピューティングアーキテクチャと非中央集権的なブロックチェーンを組み合わせ、分散されたシングルトンステートのワールドコンピュータを作り出したことです。イーサリアムプログラムは「あらゆる場所で」実行されながら、コンセンサスルールで保護された共通のステートを生成します。

「機能」としてのチューリング完全性

イーサリアムがチューリング完全であると聞くと、チューリング不完全なシステムには欠けている*機能*のように聞こえるかもしれません。しかし実際にはその逆です。チューリング完全性は非常に容易に達成できます。既知の最も単純なチューリング完全ステートマシンは4つのステートと6つのシンボルを持ち、そのステート定義はわずか22命令です。実のところ、システムが「偶然にチューリング完全」だと判明する場合もあります（そのようなシステムの面白い参考資料はこちら）。

一方でチューリング完全性は非常に危険でもあります。特に前のセクションで説明した停止問題があるため、パブリックブロックチェーンのようなオープンアクセスシステムでは問題になります。例えば現代のプリンターはチューリング完全であり、プリンターをフリーズさせるファイルを印刷させることができます。イーサリアムがチューリング完全であるということは、任意の複雑さのプログラムを計算できることを意味します。ただしその柔軟性は厄介なセキュリティとリソース管理の問題をもたらします。応答しないプリンターは電源を切って入れ直せば済みますが、パブリックブロックチェーンではそのようなことは不可能です。

チューリング完全性の意味するもの

チューリングは、プログラムをコンピュータ上でシミュレートしても終了するかどうかを予測できないことを証明しました。簡単に言えば、プログラムを実行せずにその経路を予測することはできません。チューリング完全なシステムは*無限ループ*に陥る可能性があります。無限ループとは、終了しないプログラムを指す用語です。終了しないループを実行するプログラムの作成は簡単です。ただし意図しない無限ループは、開始条件とコードの複雑な相互作用により警告なく発生することがあります。

イーサリアムではこれが課題になります。すべての参加ノード（クライアント）はすべてのトランザクションを検証し、呼び出されたスマートコントラクトを実行しなければなりません。チューリングが証明したように、スマートコントラクトが終了するかどうか、どれだけの時間実行されるかを、実際に実行せずに予測することはできません。偶然にせよ意図的にせよ、ノードが検証を試みる際に永遠に実行されるスマートコントラクトを作成できます。これは事実上のDoS攻撃と同じことです。

もちろん、検証に1ミリ秒かかるプログラムと永遠に実行されるプログラムの間には、リソースを浪費しメモリ使用量を増大させ、CPUを過熱させる厄介なプログラムが無数に存在します。ワールドコンピュータでは、リソースを濫用するプログラムは世界のリソースを濫用することになります。リソース使用を事前に予測できないのに、イーサリアムはスマートコントラクトが使用するリソースをどのように制限するのでしょうか。

この課題に対処するため、イーサリアムは*ガス*と呼ばれる計量メカニズムを導入しました。EVMがスマートコントラクトを実行する際、すべての命令（計算、データアクセスなど）を慎重に計上します。各命令にはガス単位で事前に決められたコストがあります。トランザクションがスマートコントラクトの実行をトリガーする際は、実行中に消費できるガス量の上限を含める必要があります。消費されるガス量がトランザクションで利用可能なガスを超えると、EVMは実行を終了します。ガスは、イーサリアムがチューリング完全な計算を可能にしつつ、プログラムが消費できるリソースを制限するメカニズムです。

次の疑問は、イーサリアムワールドコンピュータでの計算の対価としてガスをどう入手するかです。ガスは取引所では購入できません。トランザクションの一部としてのみ購入でき、etherでしか購入できません。トランザクション送信時にetherを添付し、ガスプライス（1ガス単位あたりの価格）を指定してガスの購入に充てる必要があります。ガソリンスタンドと同様にガスの価格は固定されていません。ガスはトランザクション用に購入され、計算が実行され、未使用のガスはトランザクションの送信者に返金されます。

汎用ブロックチェーンからDAppへ

イーサリアムは、さまざまな用途にプログラムできる汎用ブロックチェーンを目指して始まりました。そのビジョンは非常に速く拡大し、DApp向けプラットフォームへと発展しました。DAppはスマートコントラクト単体よりも広い概念を指します。最低限、スマートコントラクトとウェブユーザーインターフェースで構成されます。より広い意味では、オープンで非中央集権的なP2Pインフラサービスの上に構築されたウェブアプリケーションです。

DAppは少なくとも以下で構成されます：

- ブロックチェーン上のスマートコントラクトである
- ウェブフロントエンドユーザーインターフェースである

さらに、多くのDAppには以下のような他の非中央集権的なコンポーネントが含まれます：

- 非中央集権的な（P2P）ストレージプロトコルとプラットフォームである
- 非中央集権的な（P2P）メッセージングプロトコルとプラットフォームである

実践的な観点から見ると、JavaScriptライブラリのweb3.jsはブラウザで実行されるJavaScriptアプリケーションとイーサリアムブロックチェーンを橋渡しします。当初はSwarmと呼ばれるP2Pストレージネットワークと*Whisper*と呼ばれるP2Pメッセージングサービスを含んでおり、完全に非中央集権的なWeb3 DAppの開発を可能にするツールでした。この完全に非中

央集権的なDApp設計は魅力的でしたが、その後数年間あまり普及しませんでした。ユーザーエクスペリエンスの向上とユーザーの採用促進のために妥協が必要となり、スマートコントラクトとやり取りする中央集権型のWeb2ウェブサイトが現在ではDAppの標準となっています。

インターネットの第三の時代

2004年、*Web 2.0*という用語が、ユーザー生成コンテンツ、レスポンシブなインターフェース、インタラクティビティへのウェブの進化を示す呼び名として注目を集めました。Web 2.0は技術仕様ではなく、ウェブアプリケーションの新たな方向性を表す用語です。

DAppの概念は、ウェブアプリケーションのあらゆる側面にP2Pプロトコルによる非中央集権化を導入し、ウェブを次の自然な進化段階に進めることを目的としています。この進化を表す用語が*Web3*であり、ウェブの第三の「バージョン」を意味します。ギャヴィン・ウッドが最初に提案したWeb3は、ウェブアプリケーションの新しいビジョンと焦点を表しています。中央で所有・管理されるアプリケーションから、非中央集権的なプロトコル上に構築されたアプリケーションへの転換です。

イーサリアムの開発文化

ここまで、イーサリアムの目標と技術が、ビットコインのような先行するブロックチェーンとどのように異なるかについて話してきました。イーサリアムはまた、非常に異なる開発文化を持っています。

ビットコインでは、開発は保守的な原則に導かれています。すべての変更は既存のシステムを壊さないよう慎重に検討されます。ほとんどの場合、変更は後方互換性がある場合にのみ実装されます。既存のクライアントはオプトインが許可されており、アップグレードしなくても引き続き動作します。この慎重なアプローチはビットコインのガバナンスモデルと整合しており、変更はBitcoin Improvement Proposal（BIP）プロセスを経ます。BIPプロセスは安定性を重視して意図的にゆっくりと進む、コンセンサス駆動のパイプラインとして設計されています。

対照的に、イーサリアムコミュニティの開発文化は過去ではなく未来に焦点を当てています。（完全に本気ではありませんが）「素早く動いて壊せ」というマントラがあります。変更が必要な場合、それまでの仮定を無効にしたり、互換性を破壊したり、クライアントに更新を強制することになっても実装されることがあります。イーサリアムのガバナンスはこの実践的なスタイルを反映しています。研究者、クライアントチーム、エコシステムの利害関係者が今後の変更を議論・協議する頻繁なAllCoreDevs会議を通じて公開的に調整が行われます。これは安定性の一部を犠牲にして、より速いイノベーションのペースを得るアジャイルで反復的なプロセスです。

開発者にとってこれは、基盤となる前提が変更されたときにインフラを再構築する準備を常に持ち、柔軟性を維持する必要があるということです。イーサリアムの開発者が直面する大きな課

題の一つは、一度デプロイすると変更できないシステムへのコードデプロイと、まだ進化中の開発プラットフォームとの間にある本質的な矛盾です。スマートコントラクトは単純にはアップグレードできません。新しいコントラクトをデプロイし、ユーザー、アプリ、資金を移行して最初からやり直す準備が必要です。

皮肉なことに、これはより自律的で中央集権的な制御の少ないシステムを構築するという目標が、まだ完全には実現されていないことも意味します。自律性と非中央集権化を真に実現するには、今後数年のイーサリアムが提供できる以上のプラットフォームの安定性が必要です。プラットフォームを「進化」させるには、スマートコントラクトを破棄して再スタートする準備が必要であり、そのためスマートコントラクトに対して一定程度の制御を維持し続けなければなりません。

とはいえ良い面もあります。イーサリアムは非常に速く前進しています。*バイクシェディング*の余地はほとんどありません。バイクシェディングとは、原子力発電所の裏の自転車置き場をどう建てるかといった些細な議論で開発が止まってしまう現象です。バイクシェディングを始めると、気を取られている間に他の開発メンバーが計画を変更し、自転車を捨てて自律型ホバークラフトに乗り換えていたことに突然気づくかもしれません。

最終的に、イーサリアムプラットフォームの開発は減速し、インターフェースは固定されるでしょう。ただしそれまでの間、イノベーションが指導原則です。開発の速度は誰も落としてくれないので、遅れずについていきましょう。

なぜイーサリアムを学ぶのか

ブロックチェーンは学習コストが非常に高い分野です。プログラミングや情報セキュリティ、暗号学、経済学、分散システム、P2Pネットワークなど複数の分野を1つの領域に組み合わせているためです。イーサリアムはこの学習コストをかなり下げてくれるので、すぐに始めることができます。ただし一見シンプルな環境の表面のすぐ下には多くの複雑さが隠れています。学び、より深く見ていくと、常に別の複雑さと驚きのレイヤーが存在します。

イーサリアムはブロックチェーンを学ぶための素晴らしいプラットフォームであり、他のどのブロックチェーンプラットフォームよりも速く膨大な開発者コミュニティを構築しています。他のどのプラットフォームよりもイーサリアムは*開発者のためのブロックチェーン*です。開発者による、開発者のための構築です。JavaScriptアプリケーションに精通した開発者であれば、イーサリアムに飛び込んですぐに動作するコードを書き始めることができます。イーサリアムの最初の数年間はわずか5行のコードでトークンを作成できると書かれたTシャツをよく見かけました。もちろんこれは諸刃の剣で、コードを書くのは簡単ですが、*良い*コードや*安全*なコードを書くのは非常に難しいのです。

L2をはじめ多くのブロックチェーンプロジェクトはイーサリアムをベースにしています。イーサリアムを学ぶことでこれらのプロジェクトをより深く理解し、ブロックチェーン世界のさらなる発展を探求するツールを得ることができます。この知識は最新のブロックチェーン技術に関わりたいすべての人にとって鍵となるものです。

まとめ

イーサリアムはブロックチェーンの世界で画期的なプラットフォームとして傑出した存在です。チューリング完全なシステムとして設計されたことで、ビットコインのシンプルな機能を超えた、洗練されたプログラム可能なロジックを持つ分散アプリケーションを作成できます。

開発者や技術者にとって、イーサリアムを理解することはブロックチェーン技術とその潜在的なアプリケーションのより深い理解への扉を開きます。イーサリアムを習得することで、この分野の最先端に立ち、インターネットの継続的な進化に参加・貢献するためのツールを手にすることができます。

第2章　イーサリアムの基礎

この章ではイーサリアムの探索を始めます。ウォレットの使い方やトランザクションの作成方法、基本的なスマートコントラクトの実行方法について説明します。

etherの通貨単位

イーサリアムの通貨単位は*ether*と呼ばれ、ETHや記号Ξ（ギリシャ文字の*クサイ（Xi）*で、*装飾的な大文字のE*に似た形をしている）、またはあまり一般的ではありませんが♦でも表記します。例えば1 ether、1 ETH、Ξ1、♦1のように表記します。

> **ヒント**
>
> ΞにはUnicode文字U+039Eを、♦にはU+2666を使用する。

etherはより小さな単位に細分化でき、最小単位は*wei*と呼ばれます。1 etherは100京wei（1 × 10^18^、つまり1,000,000,000,000,000,000）です。通貨を「Ethereum」と呼ぶ人もいますが、これはよくある初心者の間違いです。Ethereumはシステムであり、etherが通貨です。

etherの値はイーサリアム内部で常にwei建ての符号なし整数値として表現します。1 etherのトランザクションを行う場合、トランザクションは値として1,000,000,000,000,000,000 weiをエンコードします。

etherのさまざまな単位には、国際単位系（SI）に基づく科学的な名称と、コンピュータ科学や暗号学の偉大な先人たちに敬意を表す通称の両方があります。表2-1にさまざまな単位とその通称（一般名）およびSI名を示します。値の内部表現に合わせて、この表ではすべての単位をwei（最初の行）で表示し、etherは7番目の行で10^18^ weiとして示しています。

表2-1. etherの単位と単位名

値（wei）	指数	通称	SI名
1	1	Wei（ウェイ）	Wei
1,000	10^3^	Babbage（バベッジ）	Kilowei or femtoether
1,000,000	10^6^	Lovelace（ラブレス）	Megawei or picoether
1,000,000,000	10^9^	Shannon（シャノン）	Gigawei or nanoether
1,000,000,000,000	10^12^	Szabo（サボ）	Microether or micro
1,000,000,000,000,000	10^15^	Finney（フィニー）	Milliether or milli
1,000,000,000,000,000,000	*10^18^*	*Ether（イーサ）*	*Ether*
1,000,000,000,000,000,000,000	10^21^	Grand（グランド）	Kiloether
1,000,000,000,000,000,000,000,000	10^24^		Megaether

イーサリアムウォレットの選択

*ウォレット*という用語はさまざまな意味を持つようになりましたが、定義はすべて関連しており、日常的にはほぼ同じものを指しています。本書では、イーサリアムアカウントの管理を支援するソフトウェアアプリケーションを*ウォレット*と呼びます。簡単に言えば、イーサリアムウォレットはイーサリアムシステムへの入口であり、秘密鍵を保持してユーザーに代わってトランザクションを作成・ブロードキャストします。イーサリアムウォレットにはさまざまな機能やデザインのものがあり、どれを選ぶか迷うことがあります。初心者向けのものもあれば上級者向けのものもあります。イーサリアムプラットフォーム自体がまだ発展途上にあるため、「最良の」ウォレットとは、プラットフォームのアップグレードに柔軟に対応し続けるものであることが多いです。

でも心配は無用です。ウォレットを選んで使い心地が気に入らなくても、最初は気に入っていたが後で別のものを試したくなっても、簡単にウォレットを変更できます。古いウォレットから新しいウォレットに資金を送るトランザクションを作成するか、秘密鍵をエクスポートして新しいウォレットにインポートするだけです。

ウォレットアプリケーションは秘密鍵にアクセスする必要があるため、信頼できるソースからのみダウンロードして使用することが重要です。幸い、ウォレットアプリケーションは人気が高いほど信頼性も高い傾向にあります。とはいえ「卵を一つのかごに盛るな」というベストプラク

ティスに従い、イーサリアムアカウントを複数のウォレットとシードフレーズに分散させることをお勧めします。

以下は初心者に適したウォレットの例です。ここで紹介するウォレットは品質やセキュリティを保証するものではなく、デモやテストの出発点として挙げています。以下のウォレットはすべてブラウザ拡張ウォレットとモバイルウォレットの両方に対応しています：

MetaMask

MetaMaskは使いやすく便利で、さまざまなイーサリアムノードやテストブロックチェーンに接続できるため、テストに適しています。

Rabby Wallet

Rabbyはシンプルさと使いやすさを重視して設計されており、新規ユーザーにとって良い選択肢です。セキュリティ機能も多く組み込まれています。

Phantom

Phantomは非EVMチェーンを中心としつつ、EVMチェーンにも対応したウォレットです。イーサリアムをはじめ、現在はPolygon、Baseなど複数のEVMチェーンもサポートしています。

管理と責任

イーサリアムのようなオープンブロックチェーンが重要なのは、*非中央集権*的なシステムとして運営されるからです。非中央集権性はさまざまなことを意味しますが、その決定的な側面の一つは、イーサリアムの各ユーザーが自身の*秘密鍵*を管理でき、また管理すべきであるという点です。秘密鍵は資金やスマートコントラクトへのアクセスを制御します。資金とスマートコントラクトへのアクセスの組み合わせを*アカウント*や*ウォレット*と呼ぶことがあります。これらの用語の厳密な定義はやや複雑なため、後で詳しく説明します。ただし基本原則としては、一つの秘密鍵が一つの「アカウント」に対応するという単純なものです。一部のユーザーはオンラインの中央集権型取引所などのサードパーティカストディアンに秘密鍵の管理を委ねています。本書では秘密鍵を自分で管理する方法を説明します。

管理には大きな責任が伴います。秘密鍵を失えば、資金やコントラクトへのアクセスも失います。アクセスの回復を助けてくれる人はいません。資金は永久にロックされます。この責任を果たすためのヒントをいくつか紹介します：

- 即興的なセキュリティ対策は避け、実績のある標準的なアプローチを使用すること
- アカウントが重要であるほど（例えば管理する資金が総資産に占める割合が高いほど）、より高いセキュリティ対策を講じること

- 最高レベルのセキュリティの一つはハードウェアウォレットデバイスから得られるが、すべてのアカウントにこのレベルが必要なわけではない
- 秘密鍵を暗号化せずそのままの形（平文）で保存してはならない（特にデジタル形式で）。幸い、現在のほとんどのユーザーインターフェースでは、複数の警告を表示せずに生の秘密鍵を見られるようにはなっていない
- 鍵をニーモニックワードシーケンスとしてバックアップするよう求められた場合は、ペンと紙を使って物理的なバックアップを作成する。この作業を「後で」にしてはならない。忘れてしまうからである
- バックアップはシステムに保存されたすべてのデータを失った場合や、パスワードを忘れたり紛失した場合に秘密鍵を再構築するために使用できる。ただし攻撃者が秘密鍵を入手するためにも使用できるため、デジタルで保存せず、少なくとも1部の物理的なコピーを安全に保管すること
- 高額（特に新しいアドレスへの）の送金の前に、まず少額のテストトランザクション（例えば1ドル未満の価値）を行い、受領の確認を待つこと
- 新しいアカウントを作成する際は、まず新しいアドレスに少額のテストトランザクションのみを送信する。テストトランザクションを受け取ったら、そのアカウントから再び送り返してみる。アカウント作成がうまくいかない理由はたくさんあり、問題があった場合は少額の損失で済む方がよい。テストが成功すれば問題ない（テストネットを使用すればいかなる損失も避けられる）
- パブリックブロックエクスプローラーは、トランザクションがネットワークに受け入れられたかどうかを独立して確認する簡単な方法である。この情報はすでにブロックチェーン上で公開されているが、ブロックエクスプローラーを使えばアクセスが非常に容易になる。ただしアドレスをブロックエクスプローラーに公開すると追跡される可能性があるため、プライバシーへの影響に注意すること
- パブリックブロックエクスプローラーは通常信頼性があるが、常にそうとは限らない。盲目的に信頼してはならない
- 本書に表示されているアドレスにお金を送らないこと。秘密鍵は本書に記載されており、誰でもすぐにそのお金を取り出せてしまいます

基本的な鍵管理とセキュリティのベストプラクティスについて説明したので、MetaMaskを使って作業を始めましょう。

MetaMaskを使い始める

Google Chromeブラウザを開き、拡張機能に移動します。「MetaMask」を検索し、キツネのロゴをクリックします。図2-1のような画面が表示されるはずです。

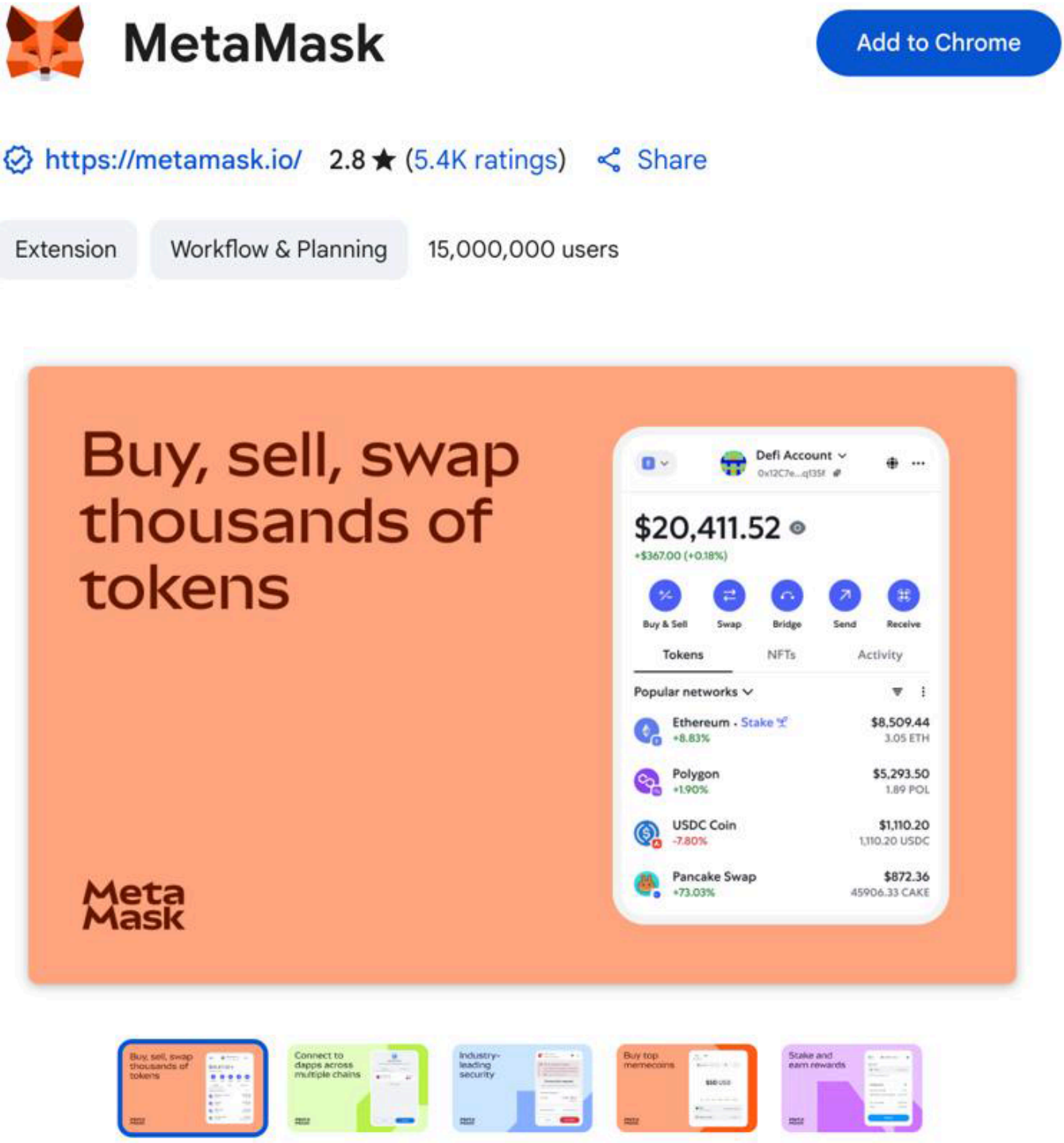


図2-1. MetaMask Chrome拡張機能の詳細ページ

本物のMetaMask拡張機能をダウンロードしているか確認することが重要です。悪意のある拡張機能がGoogleのフィルターをすり抜けることがあるからです。本物の拡張機能には以下の特徴があります：

- アドレスバーにID `nkbihfbeogaeaoehlefnkodbefgpgknn` が表示される
- *https://metamask.io* が提供元として表示される
- 5,400件以上のレビューがある

- 1,500万人以上のユーザーがいる

※これらの数字は原書執筆時点のものです。現在の数字はChrome Web Storeで確認してください。

　正しい拡張機能であることを確認したら、「Chromeに追加」をクリックしてインストールします。

ウォレットの作成

　MetaMaskをインストールすると、ブラウザのツールバーに新しいアイコン（キツネの顔）が表示されます。クリックして開始します。利用規約への同意を求められ、その後パスワードを入力して新しいイーサリアムウォレットを作成します（図2-2を参照）。

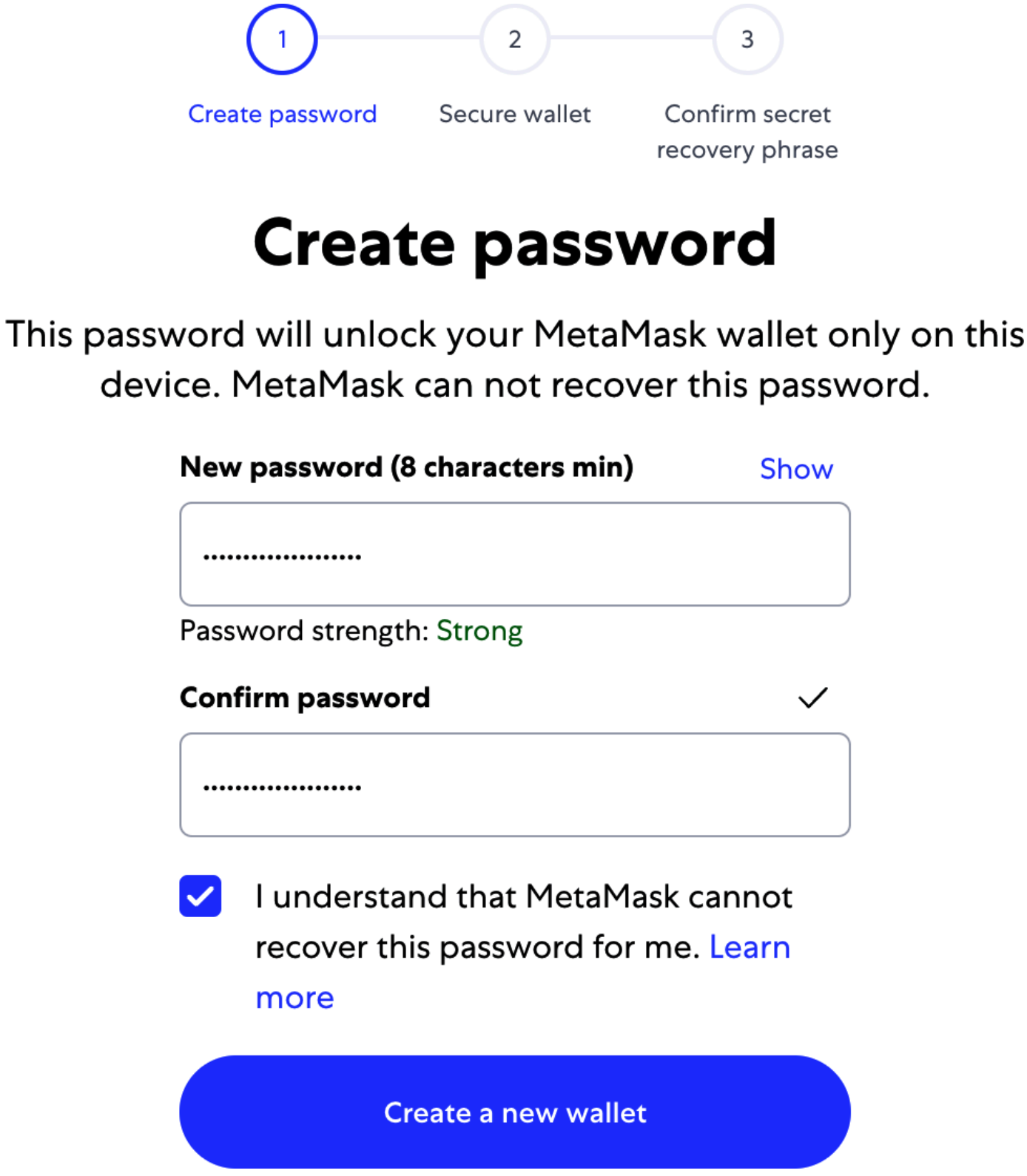


図2-2. MetaMask Chrome拡張機能のパスワードページ

ヒント

パスワードはMetaMaskへのアクセスを保護し、ブラウザを共有する第三者がMetaMaskを操作できないようにするためのものである。このパスワードはローカルデバイス上でのみ

有効である。秘密鍵やシードフレーズを入手した攻撃者は、パスワードがなくてもアドレスの資金に直接アクセスできてしまう点に注意が必要である。

パスワードを設定すると、MetaMaskはウォレットを生成し、12個の英単語からなるニーモニックバックアップを表示します（図2-3を参照）。MetaMaskやコンピュータに問題が起きた場合、互換性のある任意のウォレットでこの単語列を使って資金へのアクセスを回復できます。回復にパスワードは不要で、12個の単語だけで十分です。

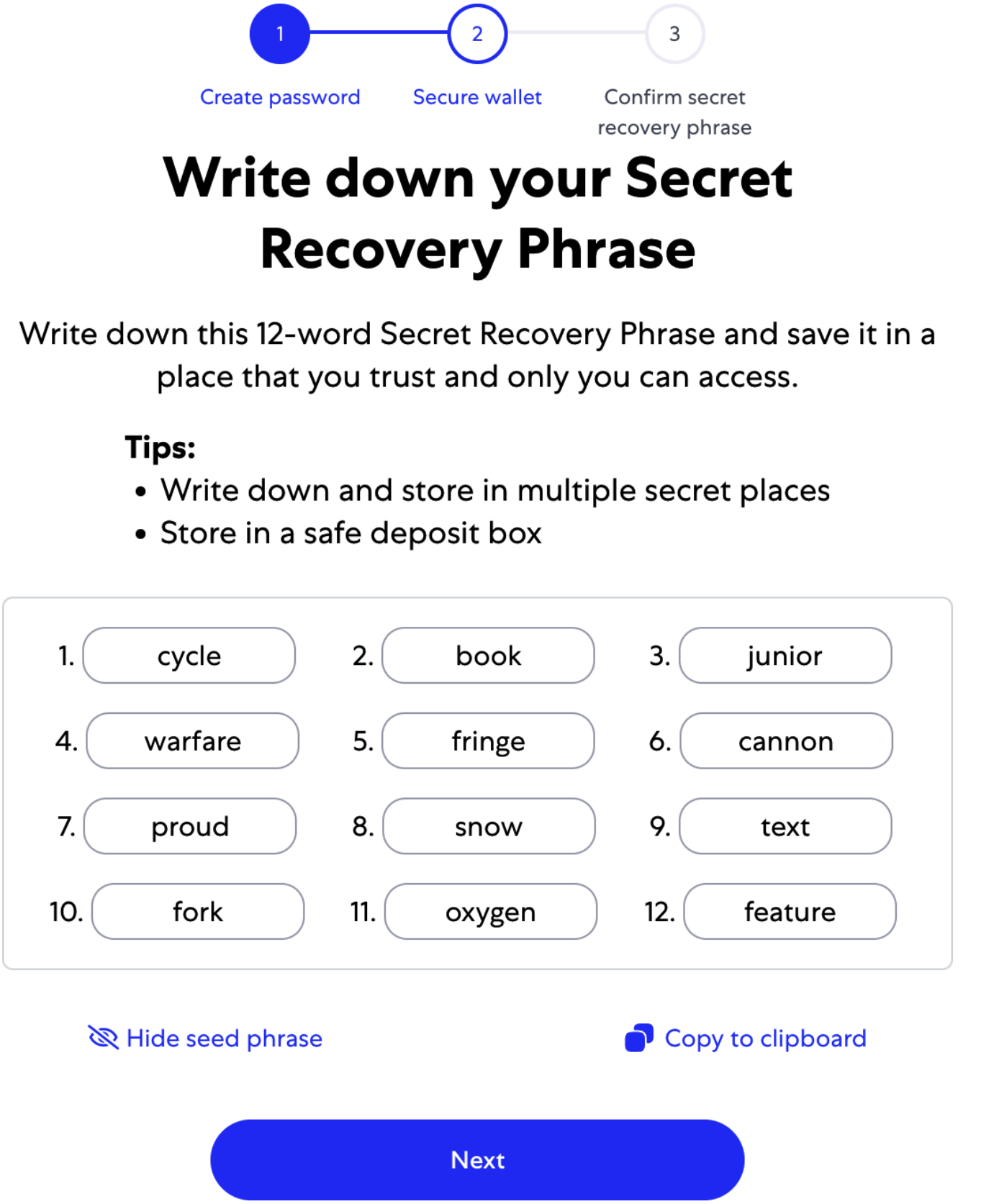


図2-3. MetaMaskが作成したウォレットのニーモニックバックアップ

ヒント

ニーモニック（12単語）を紙に2部バックアップする。2つの紙のバックアップを、耐火金庫、施錠された引き出し、貸金庫など、2つの異なる安全な場所に保管する。紙のバック

アップは、イーサリアムウォレットに保管している資金と同等の価値がある現金のように扱うこと。これらの単語にアクセスできる人は誰でも、資金にアクセスして盗むことができる。シードフレーズを安全に保つ方法については、第5章で詳しく説明する。

ニーモニックを安全に保管したことを確認すると、図2-4のようにイーサリアムアカウントの詳細が表示されます。

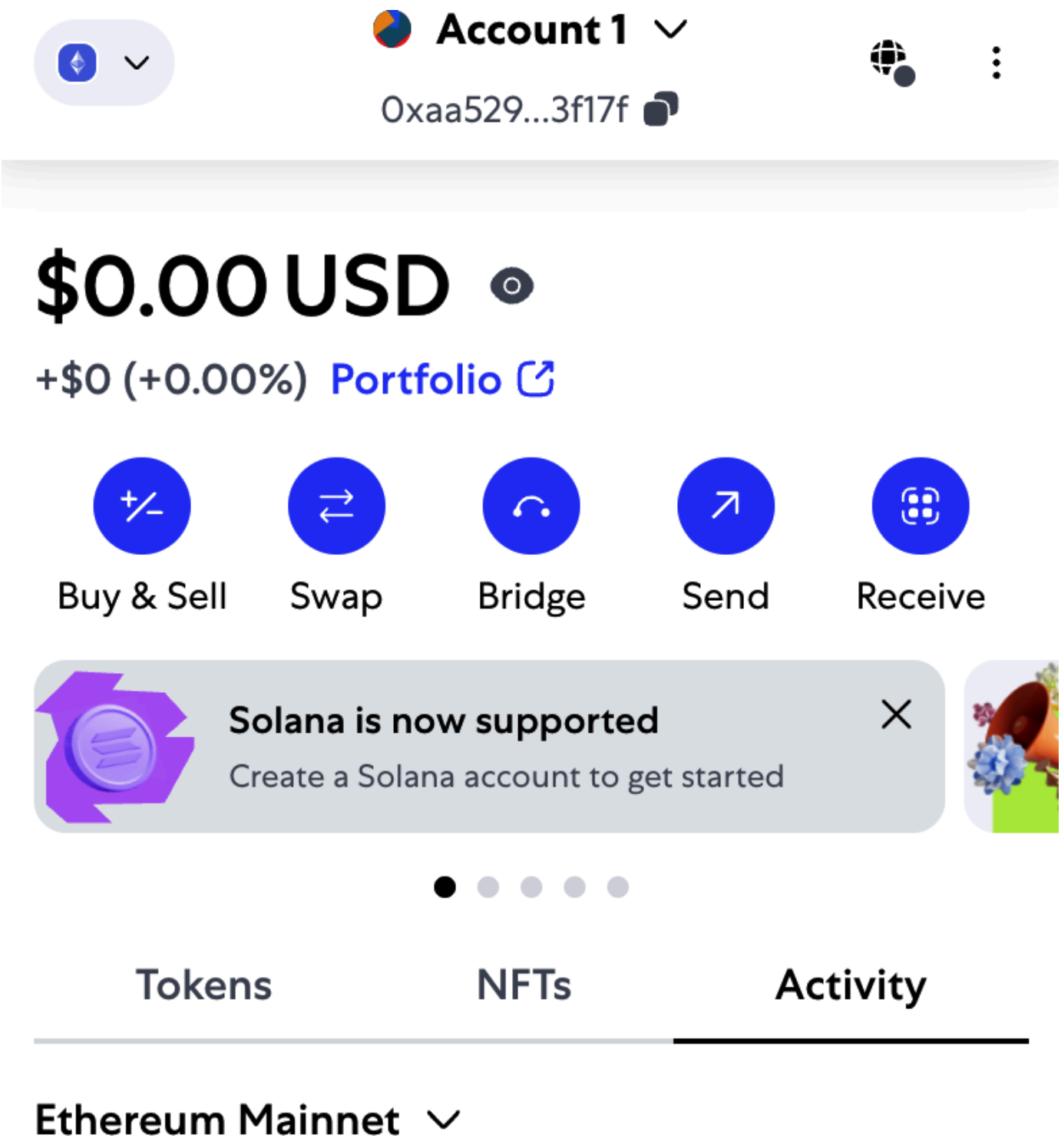


図2-4. MetaMaskのイーサリアムアカウント

注記

本書に表示されているアドレスにアセットを送らないこと。シードフレーズは教育目的で公開されており、これらのアドレスに送られたすべてのアセットはおそらく失われる。

アカウントページには、アカウント名（デフォルトでは「Account 1」）、イーサリアムアドレス（この例では`0xaa529···f17f`）、およびアカウントを視覚的に区別するためのカラフルなアイコンが表示されます。アカウントページの上部には、現在使用しているイーサリアムネットワーク（この例では「Main Network」）が表示されます。

おめでとうございます。最初のイーサリアムウォレットのセットアップが完了しました。

ネットワークの切り替え

MetaMaskのアカウントページに表示されているように、複数のイーサリアムネットワークから選択できます。デフォルトではMetaMaskはメインネットへの接続を試みます。その他の選択肢はパブリックテストネット、任意のイーサリアムノード、またはローカルコンピュータ（localhost）上でプライベートブロックチェーンを実行するノードです：

メインイーサリアムネットワーク

メインのパブリックイーサリアムブロックチェーン。本物のETH、本物の価値を扱い、実際の結果が生じる。

Sepoliaテストネット

2021年10月にイーサリアムのコア開発者がproof-of-authority（PoA）ネットワークとして立ち上げ、その後PoSコンセンサスに移行したテストネット。イーサリアムのメインネット環境を反映している。

Holeskyテストネット

Holeskyテストネットは、ステーキング、インフラ、プロトコル開発のためのイーサリアムの高度なテスト環境である。※Holeskyテストネットは2025年9月に廃止されました。後継はHoodiテストネットです。

Localhost 8545

ブラウザと同じコンピュータ上で実行されているノードに接続する。このノードはパブリックブロックチェーン（メインまたはテストネット）やプライベートテストネットの一部である場合がある。

カスタムRPC

Geth互換のリモートプロシージャコール（RPC）インターフェースを持つ任意のノードにMetaMaskを接続できる。このノードはパブリックでもプライベートでも任意のブロックチェーンの一部である場合がある。

注記

MetaMaskウォレットは接続するすべてのネットワークで同じ秘密鍵とイーサリアムアドレスを使用する。ただし各イーサリアムネットワーク上のイーサリアムアドレスの残高は異なる。例えばSepoliaテストネットでetherを送信するために鍵を使用しても、他のネットワーク上の残高には影響しない。

テスト用etherの入手

手始めにウォレットへ入金しましょう。ただし最初はメインネットではなくテストネットを使います。本物のetherには実際のお金がかかるうえ、取り扱いに慣れておく必要があるためです。

図2-5のように、左上のイーサリアムアイコンをクリックしてMetaMaskをSepoliaテストネットに切り替えます。「テストネットを表示」オプションを切り替えて、Sepoliaをクリックします。

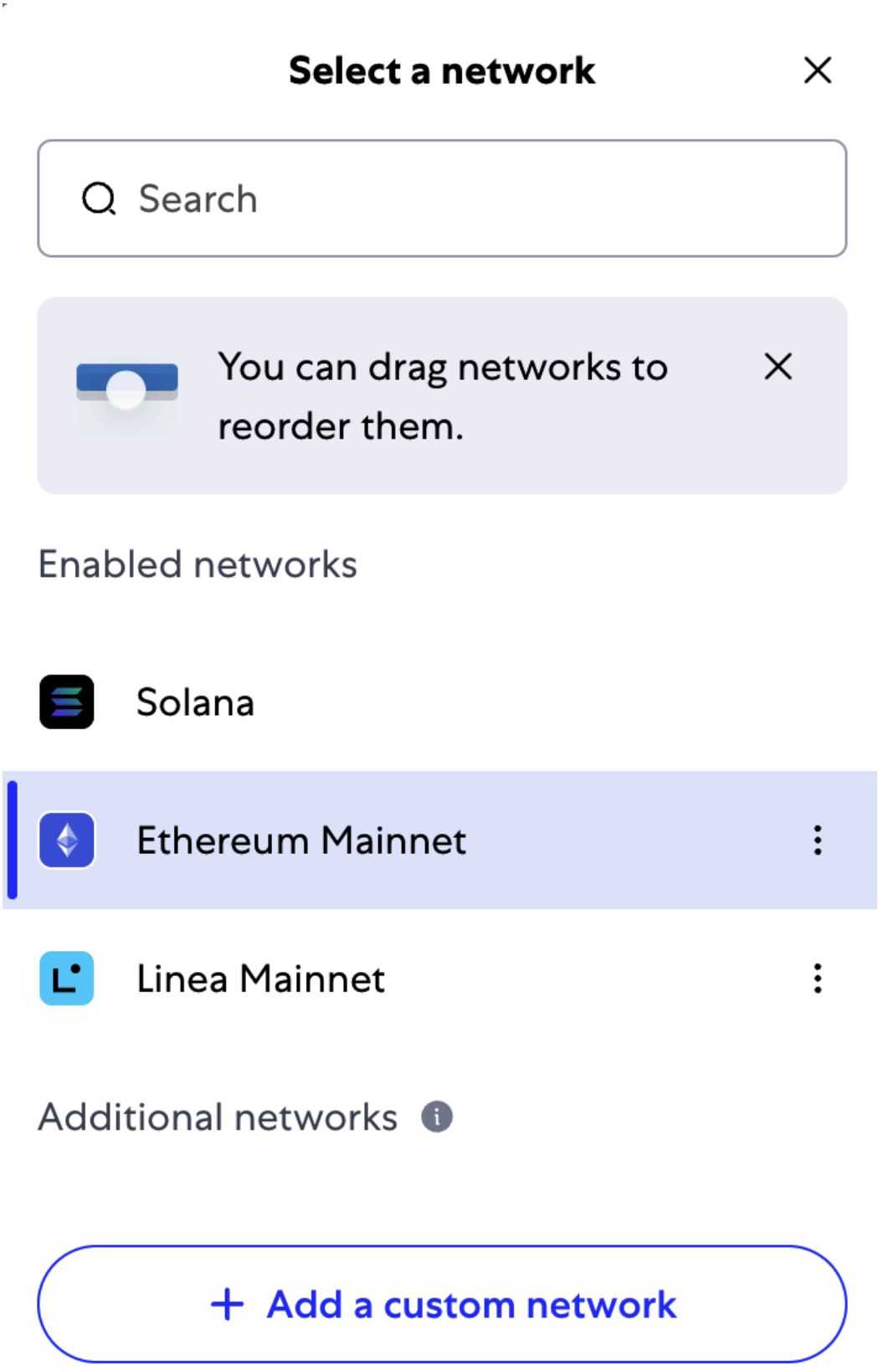


図2-5. MetaMaskのネットワーク

「購入」をクリックし（※MetaMaskのUIは更新されている場合があります。「フォーセット」や「テストネット用ETHを取得」などのボタンから同様に操作できます）、このリストのフォーセットの一つに移動します。使用するフォーセットを決めたら（どれもほぼ同等です）、図2-6のようにテストネット上でetherをリクエストします。

図2-6. MetaMask Sepoliaテストフォーセット

テストネットetherのリクエストのトランザクションIDは以下のようになります：

```
0x471273d9417e98e7f1adaae61e53a353b2d2313de2e71fc4b6184bf5a63fa0ae
```

数秒後、新しいトランザクションがSepoliaネットワークで処理され、MetaMaskウォレットに0.05 ETHの残高が表示されます（フォーセットが送信するetherの量によって異なります）。次にブラウザ拡張機能で最初のトランザクションをクリックし、図2-7のように「ブロックエクスプローラーで表示」をクリックします。*ブロックエクスプローラー*に移動します。ブロックエクスプローラーはブロック、アドレス、トランザクションを可視化して探索できるウェブサイトです。

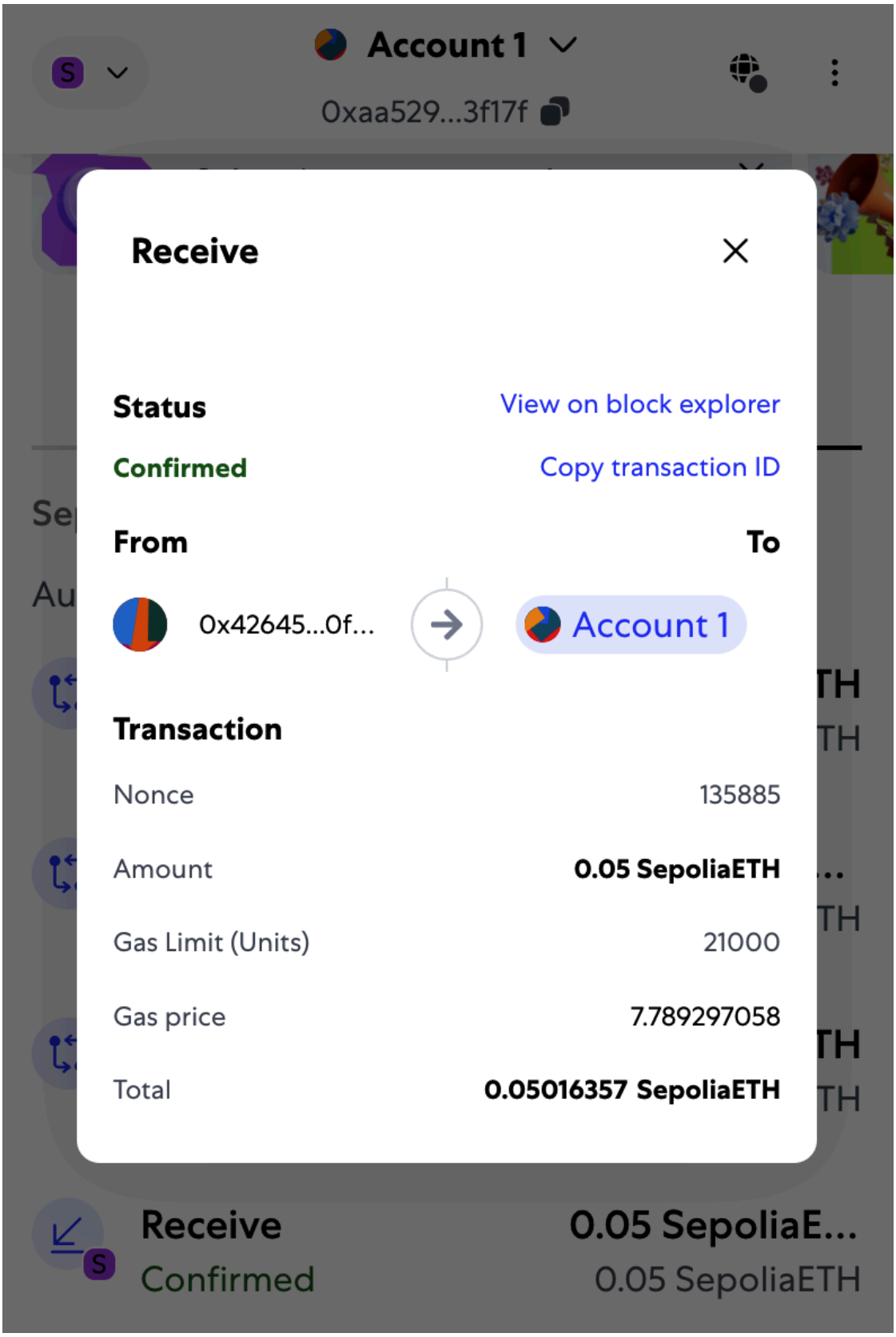


図2-7. MetaMaskから確認したトランザクション

MetaMaskは人気のあるイーサリアムブロックエクスプローラーの一つであるEtherscanブロックエクスプローラーを使用します。Sepoliaテストフォーセットからの支払いを含むトランザクションを図2-8に示します。

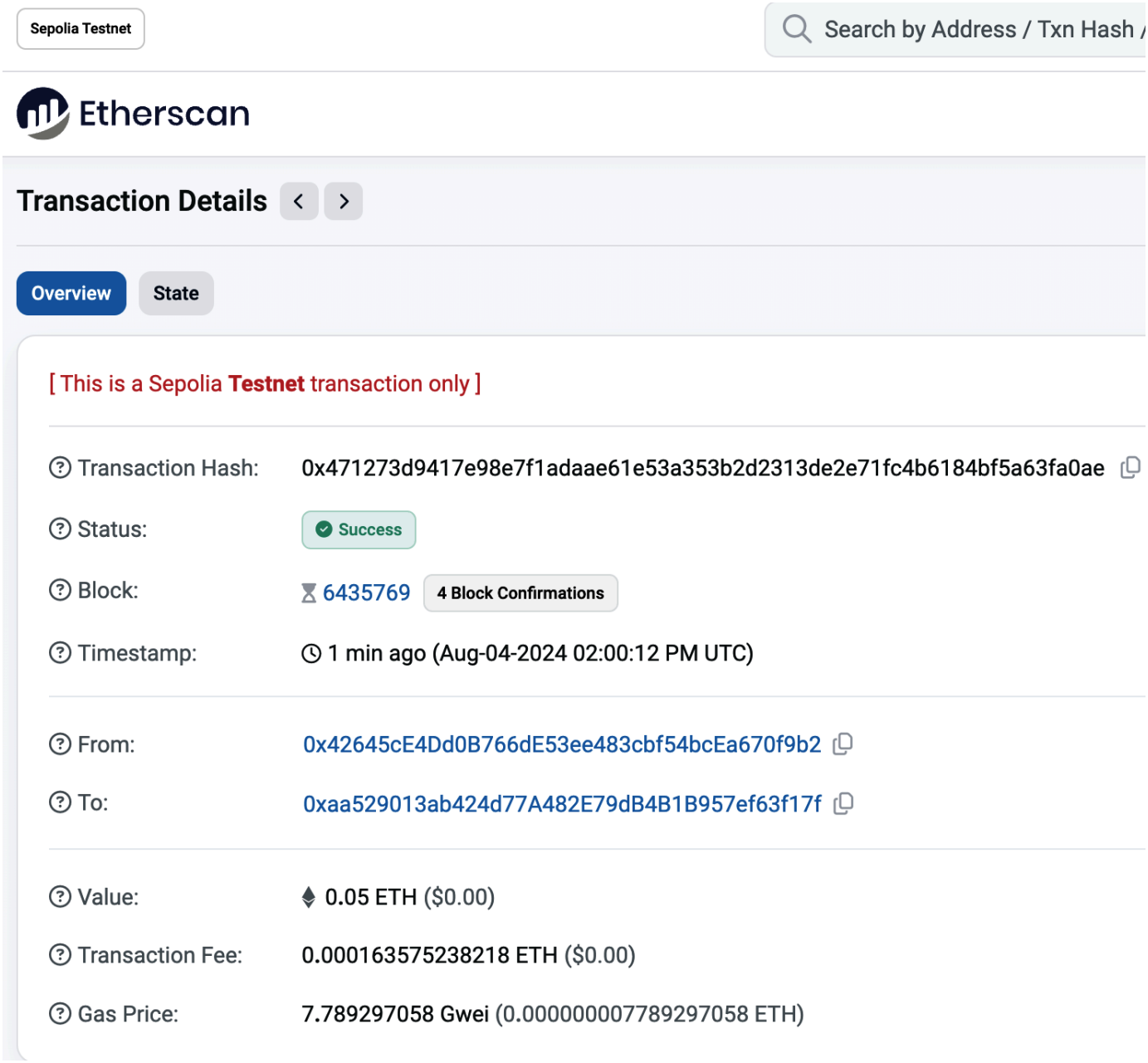


図2-8. Etherscan Sepoliaブロックエクスプローラー

トランザクションはSepoliaブロックチェーンに記録されており、トランザクションIDを検索するだけでいつでも誰でも閲覧できます。*sepolia.etherscan.io*ウェブサイトにトランザクションハッシュを入力して自分で確認してみてください：

```
0x471273d9417e98e7f1adaae61e53a353b2d2313de2e71fc4b6184bf5a63fa0ae
```

MetaMaskからetherを送信する

Sepoliaテストフォーセットから最初のテスト用etherを受け取ったら、フォーセットに返送したり他のアドレスに送信したりしてetherの送信を試すことができます。この例では図2-9のようにテストネットのetherをヴィタリック・ブテリンに送信してみます。

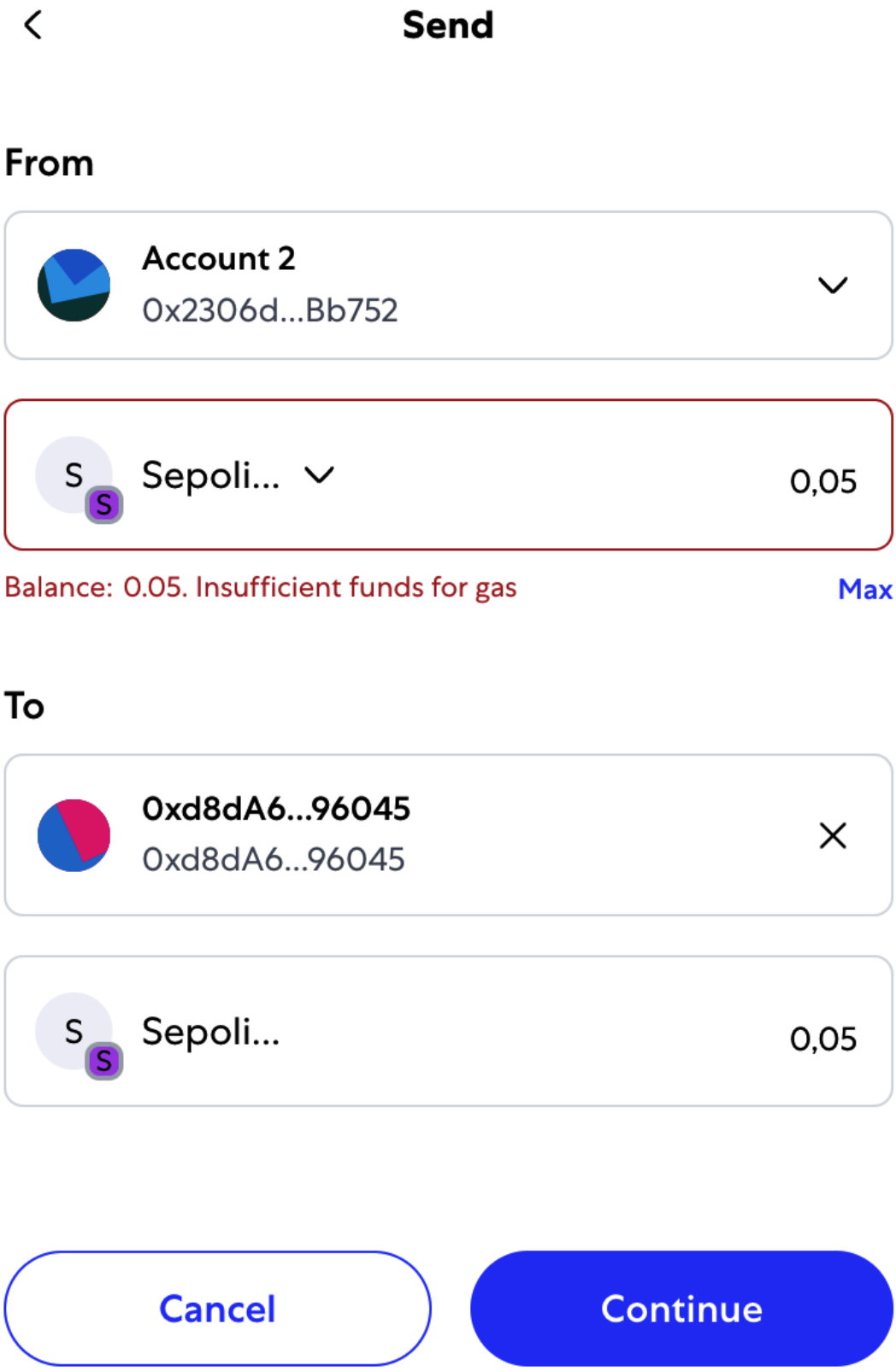


図2-9. 0.05 etherをアドレスに送信する

トランザクションを完了できないことに気づいたかもしれません。MetaMaskが残高不足だと表示しています。0.05 ETHを持っていて0.05 ETHを送りたいのに、なぜ残高不足なのでしょうか。

その答えはガスのコストにあります。すべてのイーサリアムトランザクションには手数料の支払いが必要で、ネットワークがトランザクションを検証するために徴収します。イーサリアムの手数料は*ガス*と呼ばれる仮想的な通貨単位で課金されます。ガスの支払いにはetherを使用し、トランザクションの一部として支払います。

> **注記**
>
> テストネットでも手数料が必要である。手数料がなければテストネットはメインネットとは異なる挙動をし、適切なテストプラットフォームにならない。手数料はメインネットと同様にテストネットもDoS攻撃や不適切に構築されたコントラクト（例：無限ループ）から保護する。

トランザクションを送信する際、MetaMaskは最近成功したトランザクションの平均ガス価格を計算します。例えば3 gwei（gigaweiの略）です。weiはether通貨の最小の細分化単位であり、「etherの通貨単位」で説明した通りです。ガスリミットは基本的なトランザクションを送信するコストに設定されます。これは21,000ガス単位で、トランザクションの送信に使用できるガスの最小量です。

つまり使用するETHの最大量は以下の通りです：

3 gwei（ガス価格）× 21,000ガス単位 = 63,000 gwei = 0.000063 ETH

なお平均ガス価格は変動する可能性があります。後の章では、トランザクションに優先順位を付けるためにガスリミットを増減させる方法を説明します。

つまり、0.05 ETHのトランザクションを行うには0.050063 ETHが必要ということです。「拒否」をクリックしてこのトランザクションをキャンセルしましょう。今度は0.01 ETHを送信してみましょう。

アドレスのトランザクション履歴の確認

ここまでで、MetaMaskを使ってテスト用etherを送受信するエキスパートになりました。ウォレットで支払いの受領と送信を行いました。これらすべてのトランザクションは*sepolia.etherscan.ioブロックエクスプローラー*で確認できます。ウォレットアドレスをコピーしてブロックエクスプローラーの検索ボックスに貼り付けるか、MetaMaskにページを開いてもらうことができます。MetaMaskのアカウントアイコンの横に3つの点を示すボタンがあります。クリックするとアカウント関連のオプションメニューが表示されます（図2-10を参照）。

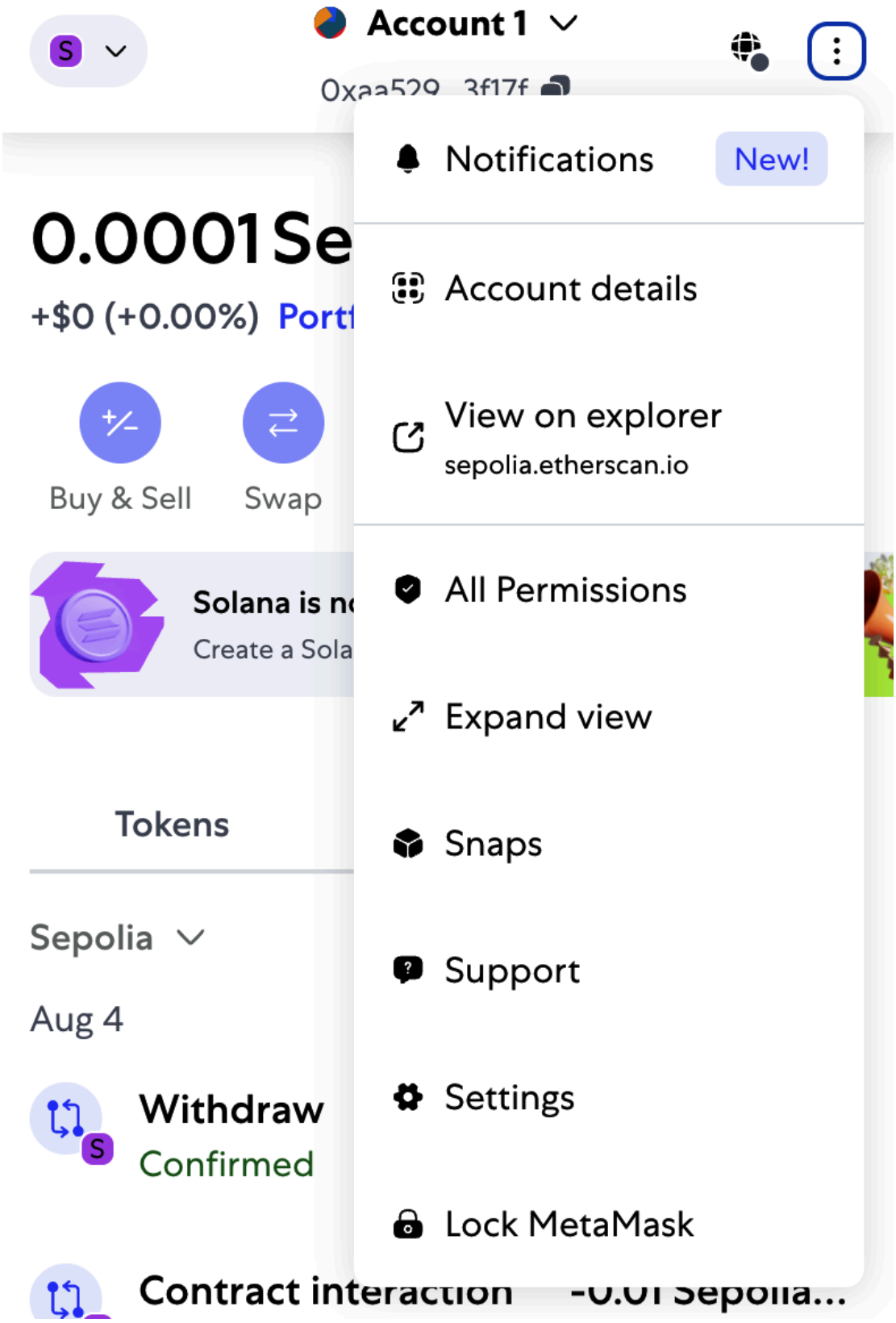


図2-10. MetaMaskアカウントコンテクストメニュー

ヒント

MetaMaskのデフォルト設定はあまりプライバシー重視ではない。「設定」→「セキュリティとプライバシー」にある設定を慎重に確認することをお勧めする。MetaMaskの仕組み

に慣れたら、イーサリアムメインネットの接続先を、デフォルト設定のものから自分の要件に合ったプライバシー設定を持つRPCを使うものへ変更することも推奨する。最もプライベートな方法は、接続可能な独自のRPCを持つ独自のノードを運用することである。第3章でその方法を説明する。

「Etherscanでアカウントを表示」を選択すると、図2-11のようにブロックエクスプローラーでアカウントのトランザクション履歴を表示するウェブページが開きます。

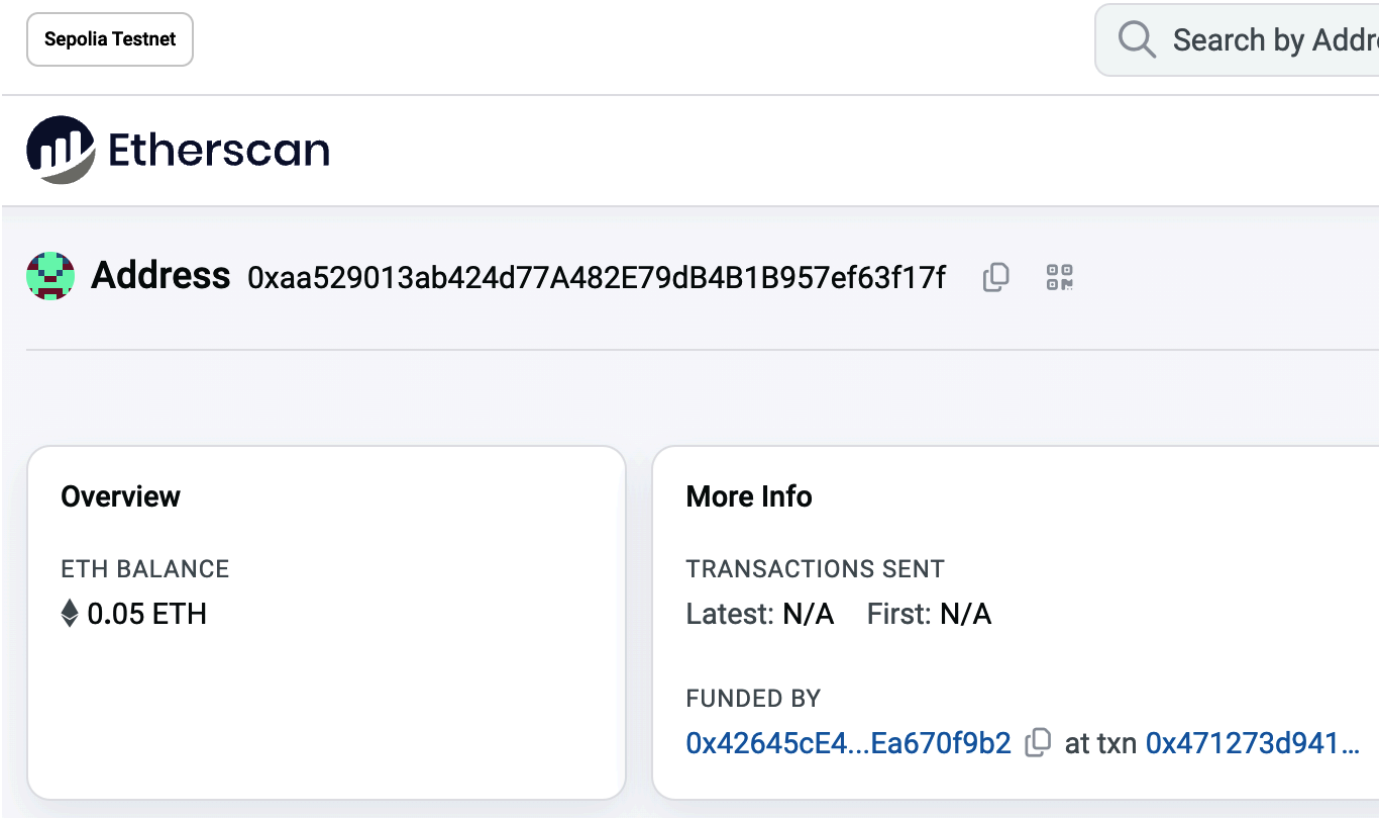


図2-11. Etherscanでのアドレストランザクション履歴

ここではイーサリアムアドレスの全トランザクション履歴を確認できます。Sepoliaブロックチェーンに記録された、自分のアドレスが送信者または受信者であるすべてのトランザクションが表示されます。いくつかのトランザクションをクリックして詳細を確認してみてください。

警告

*アドレスポイズニング*と呼ばれる既知の攻撃があり、ブロックエクスプローラー上で偽装されたアドレスのトランザクションを表示させることができる。ブロックエクスプローラーは簡易的な確認に使用するべきであるが、表示される情報が正確でない場合もある。

任意のアドレスのトランザクション履歴を調べることができます。Sepoliaテストフォーセットのアドレスのトランザクション履歴を見てみましょう（ヒント：自分のアドレスへの最も古い支払いに記載されている「送信者」アドレスです）。フォーセットから自分や他のアドレスに送られたすべてのテスト用etherを確認できます。表示されるすべてのトランザクションはさらに

多くのアドレスやトランザクションにつながります。やがて相互接続されたデータの迷宮の中で迷子になるでしょう。パブリックブロックチェーンには膨大な量の情報が含まれており、今後の例で見るようにすべてプログラムで探索できます。

ワールドコンピュータの紹介

ウォレットを作成し、etherの送受信を行いました。ここまではイーサリアムを暗号通貨として扱ってきました。しかしイーサリアムはそれ以上のものです。実際、暗号通貨の機能は非中央集権的なワールドコンピュータとしてのイーサリアムの機能に従属するものです。etherは*スマートコントラクト*と呼ばれるコンピュータプログラムの実行費用として使用されます。スマートコントラクトは*EVM*と呼ばれるエミュレートされたコンピュータ上で実行されます。

EVMはグローバルシングルトンです。つまり、あらゆる場所で動作するグローバルな単一インスタンスコンピュータであるかのように動作します。イーサリアムネットワーク上の各ノードはEVMのローカルコピーを実行してコントラクトの実行を検証し、イーサリアムブロックチェーンはトランザクションとスマートコントラクトを処理する際のワールドコンピュータの変化する*状態*を記録します。これについては第14章でさらに詳しく説明します。

外部所有アカウントとコントラクト

MetaMaskウォレットで作成したアカウントの種類は*外部所有アカウント*（EOA）と呼ばれます。EOAは秘密鍵を持つアカウントであり、秘密鍵を持つことで資金やコントラクトへのアクセスを制御できます。

お気づきかもしれませんが、アカウントにはもう一種類あります。それは*コントラクトアカウント*です。コントラクトアカウントはスマートコントラクトコードを持っていますが、単純なEOAにはそれがありません。さらにコントラクトアカウントは秘密鍵を持ちません。代わりにスマートコントラクトコードのロジックによって所有（および制御）されます。スマートコントラクトコードはコントラクトアカウントの作成時にイーサリアムブロックチェーン上に記録され、EVMが実行するソフトウェアプログラムです。

コントラクトにはEOAと同様にアドレスがあります。コントラクトもEOAと同様にetherを送受信できます。ただしトランザクションの宛先がコントラクトアドレスの場合、そのコントラクトはEVM上で*実行*され、トランザクションとそのデータが入力として使用されます。etherに加えて、トランザクションにはコントラクト内のどの関数を実行するか、その関数にどのパラメーターを渡すかを示す*データ*を含めることができます。このようにしてトランザクションはコントラクト内の関数を*呼び出す*ことができます。

コントラクトアカウントは秘密鍵を持たないため、トランザクションを*開始*できない点に注意してください。トランザクションを開始できるのはEOAのみですが、コントラクトは他のコン

トラクトを呼び出すことでトランザクションに反応し、複雑な実行パスを構築できます。

次のいくつかのセクションで最初のコントラクトを作成します。その後MetaMaskウォレットとSepoliaテストネット上のテスト用etherを使って、コントラクトの作成、資金の提供、使用方法を学びます。

シンプルなコントラクト：テスト用etherフォーセット

イーサリアムにはいくつかの高水準言語があり、いずれもコントラクトの作成とEVMバイトコードの生成に使用できます。最も著名で興味深い言語については第7章で説明します。スマートコントラクトプログラミングで圧倒的に主流の高水準言語はSolidityです。Solidityはギャヴィン・ウッドが作成し、イーサリアム（およびそれ以外）で最も広く使用されている言語になりました。Solidityを使って最初のコントラクトを作成します。

最初の例（例2-1）として、フォーセットを制御するコントラクトを作成します。Sepoliaテストネットでテスト用etherを取得するためにすでにフォーセットを使用しました。*フォーセット*は比較的シンプルなもので、要求するアドレスにetherを配布し、補充できます。

例2-1. *Faucet.sol*：フォーセットを実装するSolidityコントラクト

```
pragma solidity 0.8.26;
// SPDX-License-Identifier: GPL-3.0

// 最初のコントラクトはフォーセット！
contract Faucet {

    // 要求する人にetherを配布する
    function withdraw(uint256 _withdrawAmount, address payable _to) public {

        // 引き出し額を制限する
        require(_withdrawAmount <= 1000000000000);

        // リクエストしたアドレスに金額を送信する
        _to.transfer(_withdrawAmount);
    }

    // Etherを受け取る関数。msg.dataは空でなければならない
    receive() external payable {}

    // msg.dataが空でない場合に呼び出されるフォールバック関数
    fallback() external payable {}
}
```

これは非常にシンプルなコントラクトで、これ以上シンプルにすることはほとんどできません。また、多くの悪い慣行やセキュリティの脆弱性を示す*欠陥のある*コントラクトでもあります。後のセクションでこれらの欠陥を一つひとつ掘り下げていきます。ただし今のところはこのコントラクトが何をし、どのように動作するかを一行ずつ見ていきましょう。Solidityの多くの要素がJavaScript、Java、C++などの既存のプログラミング言語に似ていることにすぐ気づくでしょう。

最初の行は`pragma`文です：

```
pragma solidity 0.8.26;
```

Solidityソースファイルの先頭にこの行を含めることで、コードがSolidityコンパイラのバージョン0.8.26でコンパイルされることを保証し、互換性を確保して異なるコンパイラバージョンの使用から生じる問題を回避します。

次に、スマートコントラクトがGPL-3.0ライセンスの下でライセンスされていることを示すコメントがあります：

```
// SPDX-License-Identifier: GPL-3.0
```

これは法的およびコンプライアンス上の理由から重要です。コードの使用と配布に関するユーザーや開発者の権利と義務を明示するからです。

コメントは人間が読むためのものであり、実行可能なEVMバイトコードには含まれません。通常は説明しようとしているコードの前の行か同じ行に配置します。コメントは2つのスラッシュ`//`で始まります。最初のスラッシュからその行の終わりまでのすべてが空行と同じように扱われ、無視されます。

これもコメントです：

```
// 最初のコントラクトはフォーセット！
```

次の行は実際のコントラクトの開始です：

```
contract Faucet {
```

この行は`contract`オブジェクトを宣言します。他のオブジェクト指向言語の`class`宣言に似ています。コントラクトの定義は、他の多くのプログラミング言語と同様に*スコープ*を定義する中括弧（`{}`）の間のすべての行を含みます。

次に、`Faucet`コントラクトの最初の関数を宣言します：

```
function withdraw(uint256 _withdrawAmount, address payable _to) public {
```

この関数は`withdraw`という名前で、`_withdrawAmount`という名前の符号なし整数（`uint256`）と`_to`という名前の`address payable`を引数に取ります。public関数として宣言されており、他のコントラクトから呼び出せます。関数の定義は中括弧の間に続きます。`withdraw`関数の最初の部分は引き出しに制限を設けています：

```
require(_withdrawAmount <= 1000000000000);
```

Solidityの組み込み関数`require`を使って前提条件をテストしています。`_withdrawAmount`が1,000,000,000,000 wei以下、つまり0.000001 ether以下であることを確認しています（weiはetherの最小単位です。表2-1を参照）。`withdraw`関数がこの金額を超える`withdraw_amount`で呼び出された場合、`require`関数はコントラクトの実行を停止し、*例外*で失敗させます。なおSolidityでは文の末尾にセミコロンが必要です。

コントラクトのこの部分がフォーセットの主要なロジックです。引き出しに制限を設けることで、コントラクトからの資金の流れを制御しています。非常にシンプルな制御ですが、プログラマブルブロックチェーンの力を垣間見ることができます。お金を制御する非中央集権的なソフトウェアです。

ここにコントラクトの最初の設計上の欠陥があります。セキュリティ上の欠陥ではありませんが、`require`文には常にエラーメッセージを追加する方が良いでしょう。こうすれば`require`文によってユーザーのトランザクションが失敗した場合に理由が明確になります。

修正された`require`文は以下の通りです：

```
require(_withdrawAmount <= 1000000000000, "The requested amount is too much, try a smaller amount!");
```

次に実際の引き出しが来ます：

```
_to.transfer(_withdrawAmount);
```

ここでいくつか興味深いことが起きています。`transfer`関数は現在のコントラクトから別の指定されたアドレス（この場合は`_to`アドレス）にetherを転送する組み込み関数です。`transfer`関数は唯一の引数として金額を取ります。数行前に宣言した`withdraw`関数のパラメーターである`_withdrawAmount`の値を渡します。

`_to`アドレスがpayableとして定義されているため、これが可能です。組み込み関数`transfer`と`send`はpayableアドレスに対してのみ呼び出せます。ここにコードの2番目の欠陥があります。`transfer`はEOAが`withdraw`関数を呼び出す場合には問題ありませんが、別のコントラクトがこの関数をトリガーする場合には問題になります。`transfer`関数は最大2,300ガスしか利用できず、複数のコントラクト呼び出しではこの制限を超える可能性が高いため、トランザクションが失敗することがあります。この問題を修正するには`transfer`や`send`の代わりに組み込みの`call`

関数を使用します。ただし`call`関数はセキュリティ上の欠陥を避けるために適切に扱う必要があります。今のところは組み込みの`transfer`関数のままにしておきます。

次の行は閉じ中括弧で、`withdraw`関数の定義の終了を示しています。

次に、さらに2つの関数を宣言します：

```
receive() external payable {}
fallback() external payable {}
```

これらは`fallback`関数と`receive`関数です。コントラクトをトリガーしたトランザクションが、コントラクト内の宣言された関数を指定しなかった場合、またはデータを含まなかった場合に呼び出されます。コントラクトはこれらの関数を持つことができ、通常はetherを受け取るために使います。*msg.data*が空の場合は`receive`関数がトリガーされ、*msg.data*が空でない場合は`fallback`関数がトリガーされます。

`fallback`関数のすぐ下にある最後の閉じ中括弧は、`Faucet`コントラクトの定義を閉じるものです。以上です。

Faucetコントラクトのコンパイル

最初のサンプルコントラクトができたので、Solidityコンパイラを使ってSolidityコードをEVMバイトコードに変換する必要があります。これにより、そのコードをブロックチェーン上でEVMに実行させられるようになります。

Solidityコンパイラはスタンドアロンの実行ファイルとして、さまざまなフレームワークの一部として、またIDEにバンドルされて提供されています。ここでは簡単にするために、最も人気のあるIDEの一つであるRemixを使用します。

Chromeブラウザ（先ほどインストールしたMetaMaskウォレットを使用）でRemix IDEに移動します。

Remixを最初に読み込むと*Storage.sol*というサンプルコントラクトが開きます。これは不要なので、図2-12のようにタブの隅にある`x`をクリックして閉じます。

図2-12. デフォルトのサンプルタブを閉じる

　次に、図2-13のように新しいファイルを作成します。新しいファイルの名前を*Faucet.sol*にします。

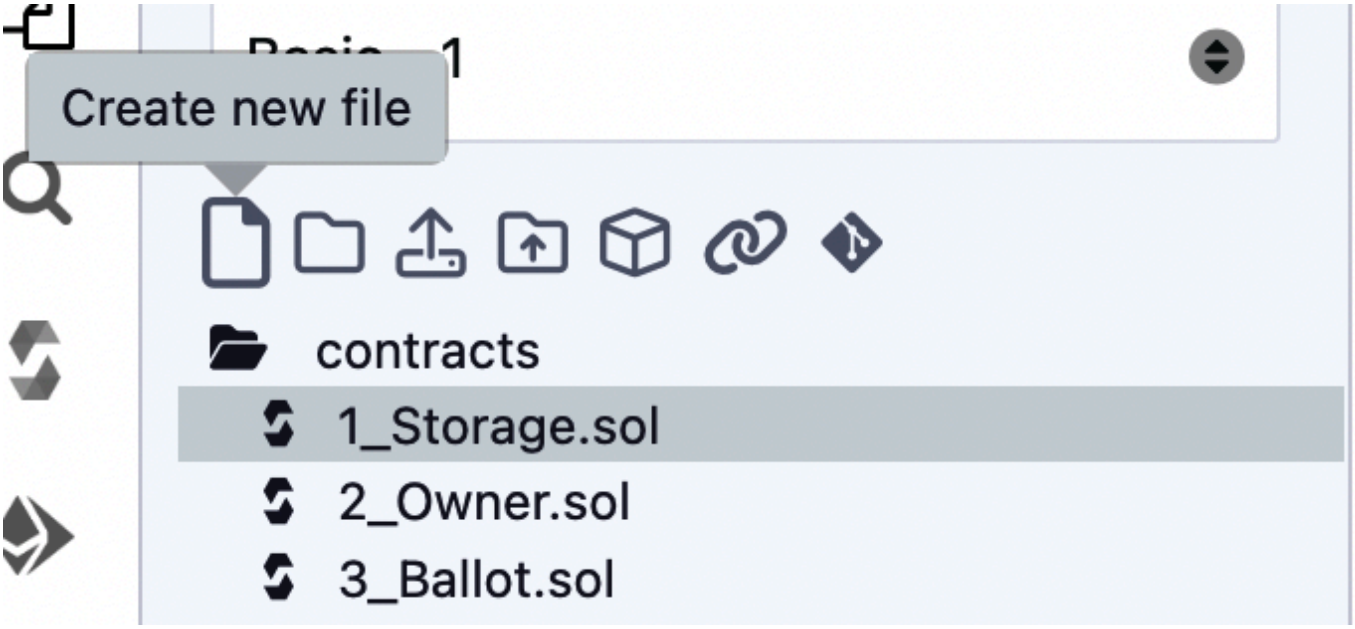


図2-13. 新しいコントラクトを作成する

　新しいタブが開いたら、図2-14のようにサンプルの*Faucet.sol*のコードをコピーして貼り付けます。

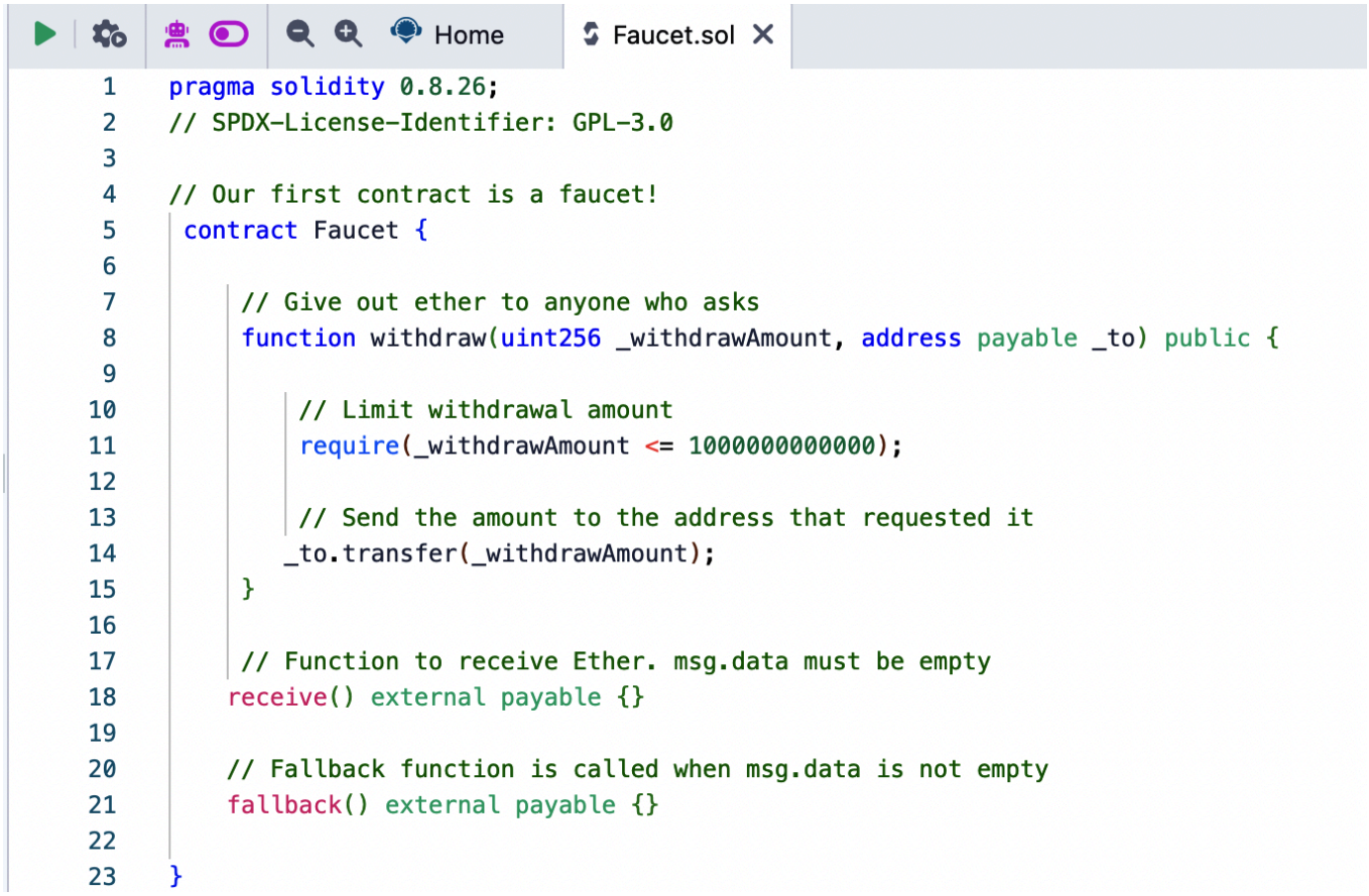


図2-14. Faucetのサンプルコードを新しいコントラクトにコピーする

*Faucet.sol*コントラクトをRemix IDEに読み込んだら、Remixのコンパイルセクションに移動し「Compile *Faucet.sol*」をクリックします。すべて順調であれば緑色のボックスが表示されます（図2-15を参照）。

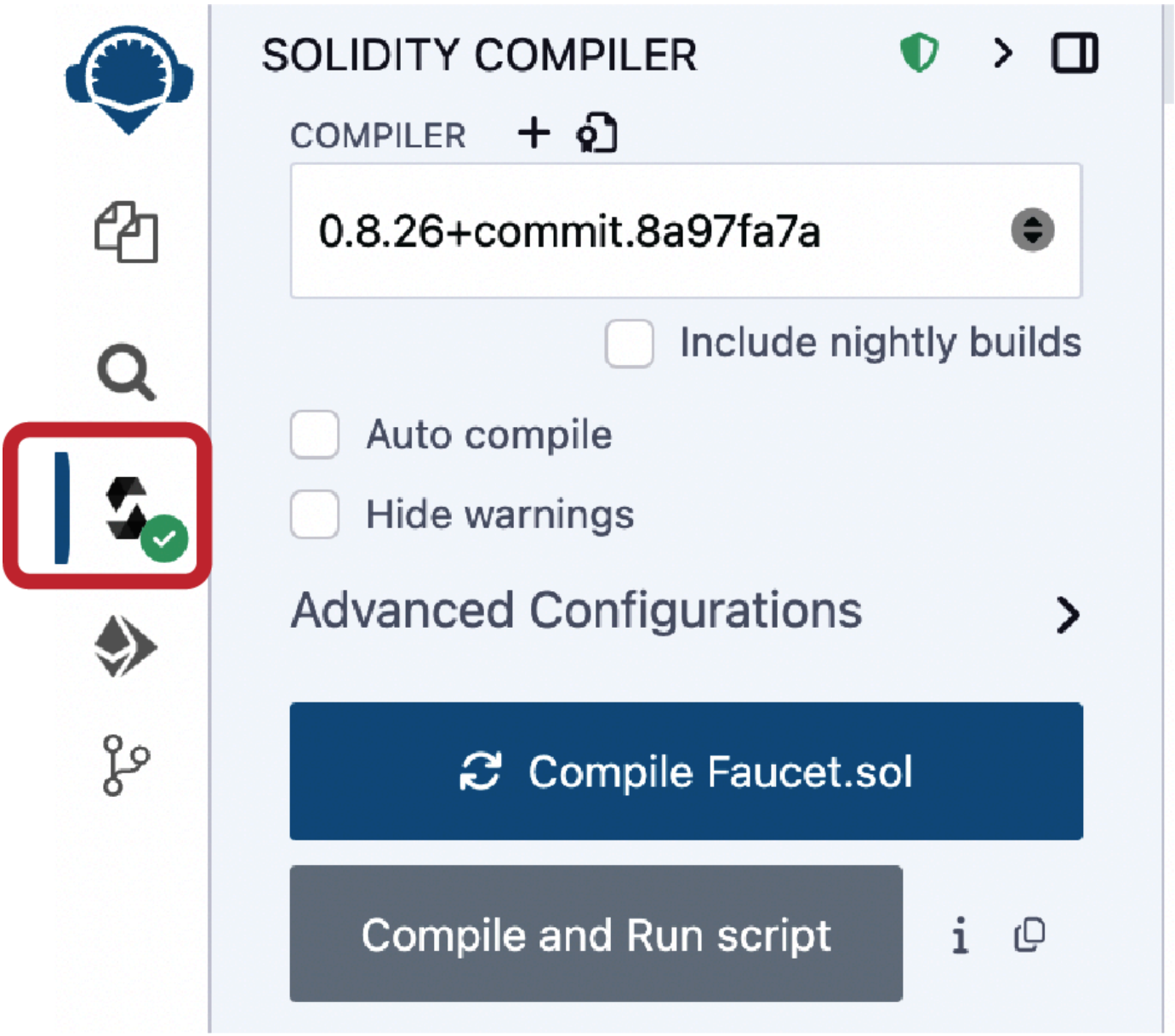


図2-15. Remixが*Faucet.sol*コントラクトのコンパイルに成功した

何か問題が発生した場合、最も可能性の高い原因はRemix IDEが0.8.26とは異なるバージョンのSolidityコンパイラを使用していることです。その場合`pragma`ディレクティブにより*Faucet.sol*のコンパイルが妨げられます。コンパイラタブに移動してバージョンを0.8.26に設定し、再試行してください。

SolidityコンパイラがFaucet.solをEVMバイトコードにコンパイルしました。参考までにバイトコードは以下の通りです：

```
608060405234801560e575f80fd5b506101af8061001c5f395ff3fe608060405260043610610020575f3560
e01c8062f714ce1461002957610027565b3661002757005b005b348015610034575f80fd5b5061004f600480
360381019061004a919061013b565b610051565b005b64e8d4a510008211156100625f80fd5b8073ffffff
ffffffffffffffffffffffffffffffffffff166108fc8390811502906040515f604051808303818588888f19350
5050501580156100a5573d5f803e3d5ffd5b505050565b5f80fd5b5f81905091905056 5b6100c0816100ae56
5b81146100ca575f80fd5b50565b5f813590506100db816100b7565b92915050565b5f73ffffffffffffffff
ffffffffffffffffffffffff82169050919050565b5f61010a826100e1565b9050919050565b61011a816101
00565b8114610124575f80fd5b50565b5f8135905061013581610111565b92915050565b5f80604083850312
```

```
156101515761015061006100aa565b5b5f61015e858286016100cd565b925050602061016f858286016101275565b
915050925092905056fea2646970667358221220 7de2f4d88c747c9332dceef5dcd739f3380ec8a8c2167a29
2ba64ee24fa32a8a64736f6c634300081a0033
```

EVMバイトコードを直接プログラミングする代わりに、Solidityのような高水準言語を使えてよかったと思いませんか。私たちもそう思います。

ブロックチェーン上へのコントラクトの作成

コントラクトができ、バイトコードにコンパイルしました。次にイーサリアムブロックチェーンにコントラクトを「登録」する必要があります。コントラクトのテストにはSepoliaテストネットを使用するので、そこに送信します。

ブロックチェーンへのコントラクトの登録は、`to`フィールドが空（null）の特別なトランザクションを作成して行います。この空の宛先は、既存のアドレスにトランザクションを送信するのではなく新しいコントラクトをデプロイしたいことをイーサリアムブロックチェーンに知らせます。トランザクションの`data`フィールドにはコンパイル済みのコントラクトバイトコードが含まれます。幸いRemix IDEがこれらすべてを処理し、トランザクションをMetaMaskに送信してくれます。

まず「Run」タブに切り替え、「Environment」ドロップダウン選択ボックスで「Injected Web3」を選択します。Remix IDEがMetaMaskウォレットに接続され、MetaMaskを通じてSepoliaテストネットに接続されます。接続すると「Environment」の下にSepoliaが表示されます。また「Account」選択ボックスにはウォレットのアドレスが表示されます（図2-16を参照）。

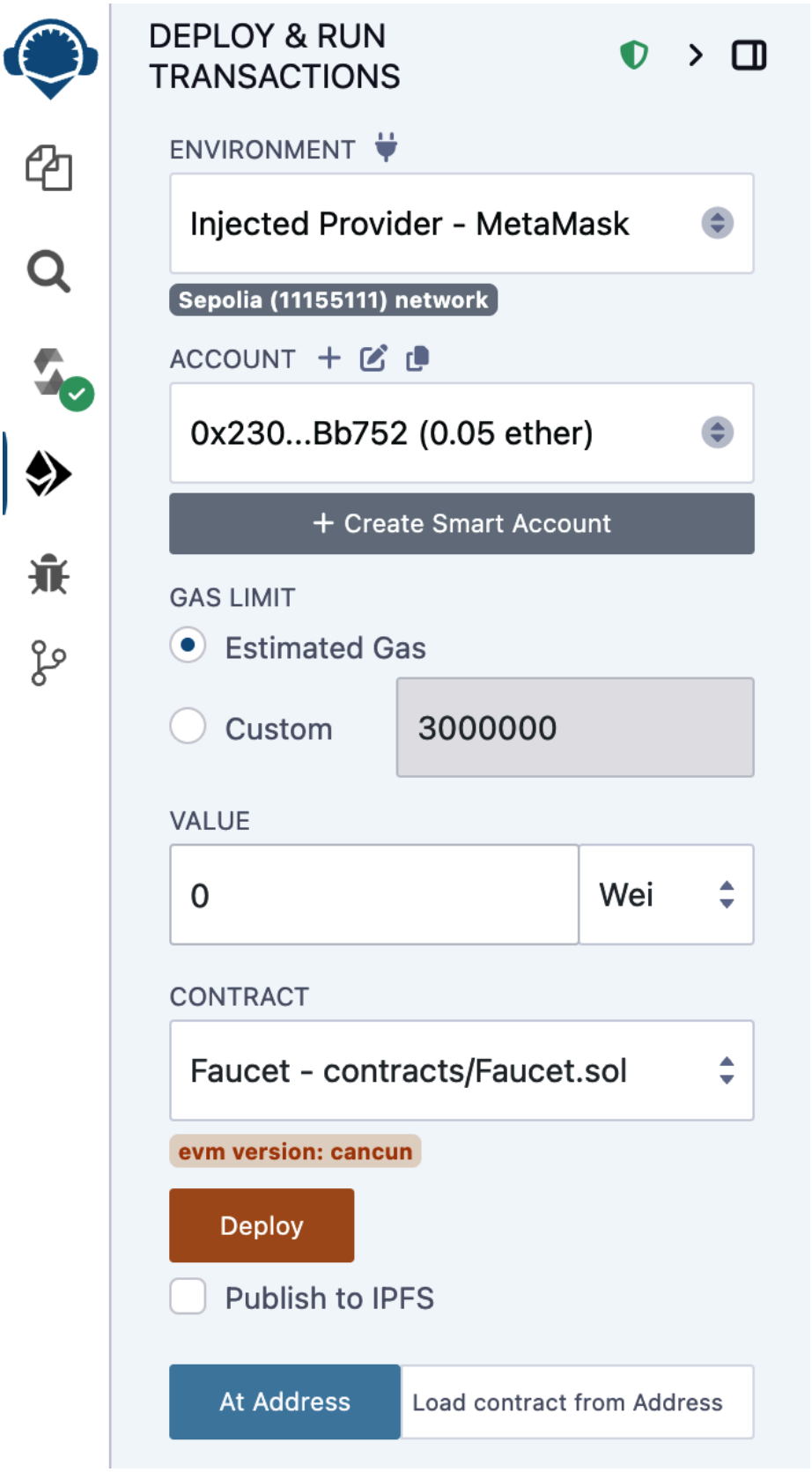


図2-16. Injected Web3環境を選択したRemix IDEのRunタブ

先ほど確認したRun設定のすぐ下に、作成準備が整った`Faucet`コントラクトがあります。図2-16に表示されているDeployボタンをクリックします。

Remixが特別な「作成」トランザクションを構築し、MetaMaskが図2-17のように承認を求めます。コントラクト作成トランザクションにはetherが含まれていませんが、データのバイト

（コンパイル済みコントラクト）が含まれておりガスとしていくらかのgweiを消費します。「送信」をクリックして承認します。

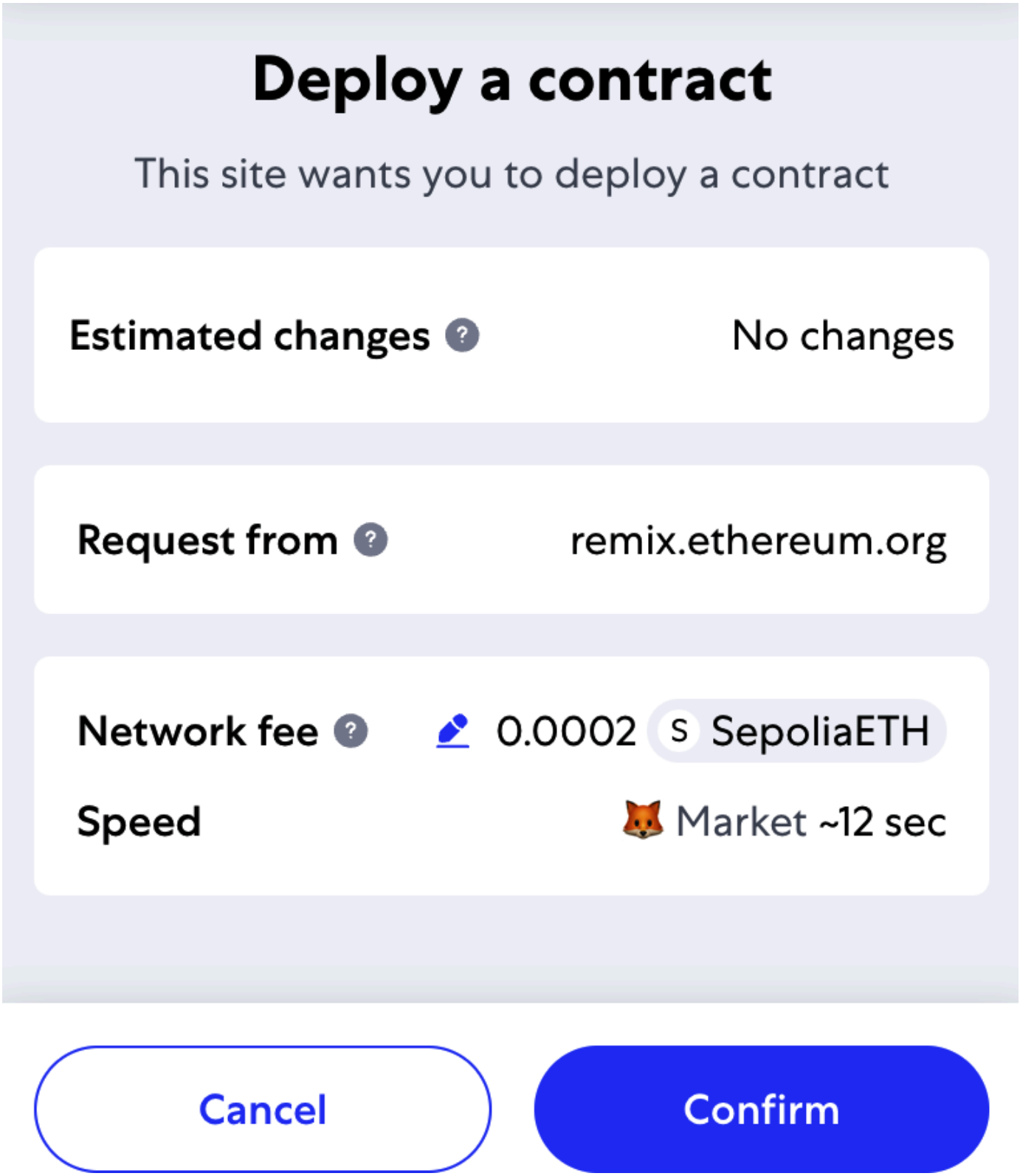


図2-17. コントラクト作成トランザクションを表示するMetaMask

ここで少し待つ必要があります。Sepoliaでコントラクトが処理されるまで約15〜30秒かかります。Remixはあまり動きがないように見えますが、しばらく待ってください。

コントラクトが作成されると図2-18のようにRunタブの下部に表示されます。

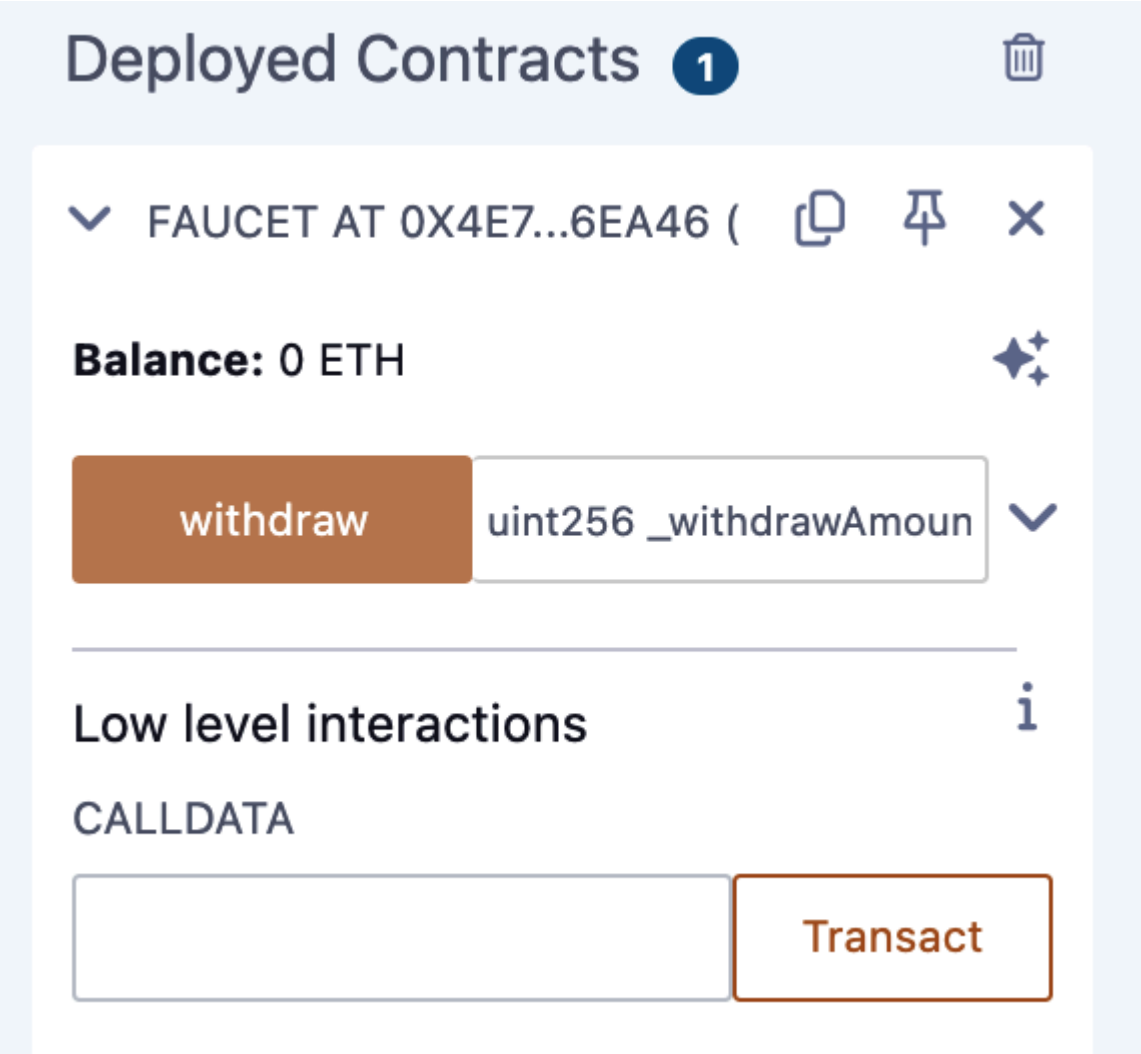


図2-18. Faucetコントラクトが稼働している

`Faucet`コントラクトには独自のアドレスがあることに注目してください。Remixでは「Faucet at 0x4E7…6EA46」と表示されています（あなたのアドレス、つまりランダムな文字と数字は異なります）。

コントラクトとのやり取り

ここまで学んだことを振り返りましょう。イーサリアムのコントラクトはEVM内部で実行されるお金を制御するプログラムです。バイトコードをブロックチェーンに記録する特別なトランザクションで作成します。ブロックチェーン上に作成されるとウォレットと同様にイーサリアムアドレスを持ちます。コントラクトアドレスにトランザクションを送信するたびに、そのトランザクションを入力としてコントラクトがEVM上で実行されます。コントラクトアドレスに送信するトランザクションにはetherまたはデータ、あるいはその両方を含めることができます。etherが含まれている場合はコントラクトの残高に「入金」されます。データが含まれている場合はコントラクト内の名前付き関数を指定してその関数を呼び出し、引数を渡せます。

ブロックエクスプローラーでコントラクトアドレスを確認する

コントラクトがブロックチェーンに記録され、イーサリアムアドレスを持つことを確認できました。*sepolia.etherscan.io*ブロックエクスプローラーでコントラクトがどのように見えるか確認してみましょう。Remix IDEでコントラクト名の横にあるアイコンをクリックしてコントラクトのアドレスをコピーします（図2-19を参照）。

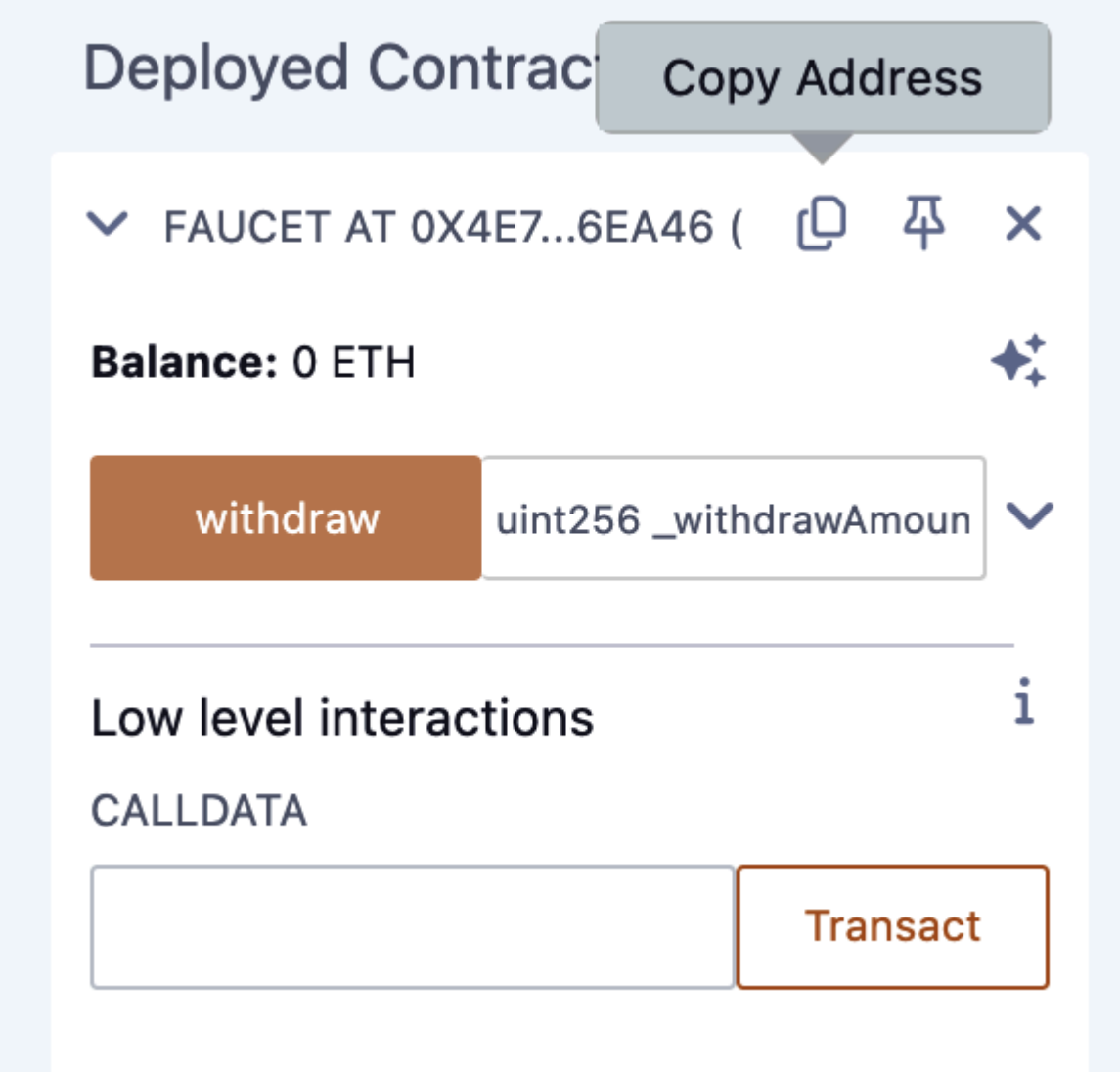


図2-19. Remixからコントラクトアドレスをコピーする

Remixは開いたままにしておいてください。後で戻ってきます。次にブラウザで*sepolia.etherscan.io*に移動し、検索ボックスにアドレスを貼り付けます。図2-20のようにコントラクトのイーサリアムアドレスの履歴が表示されるはずです。

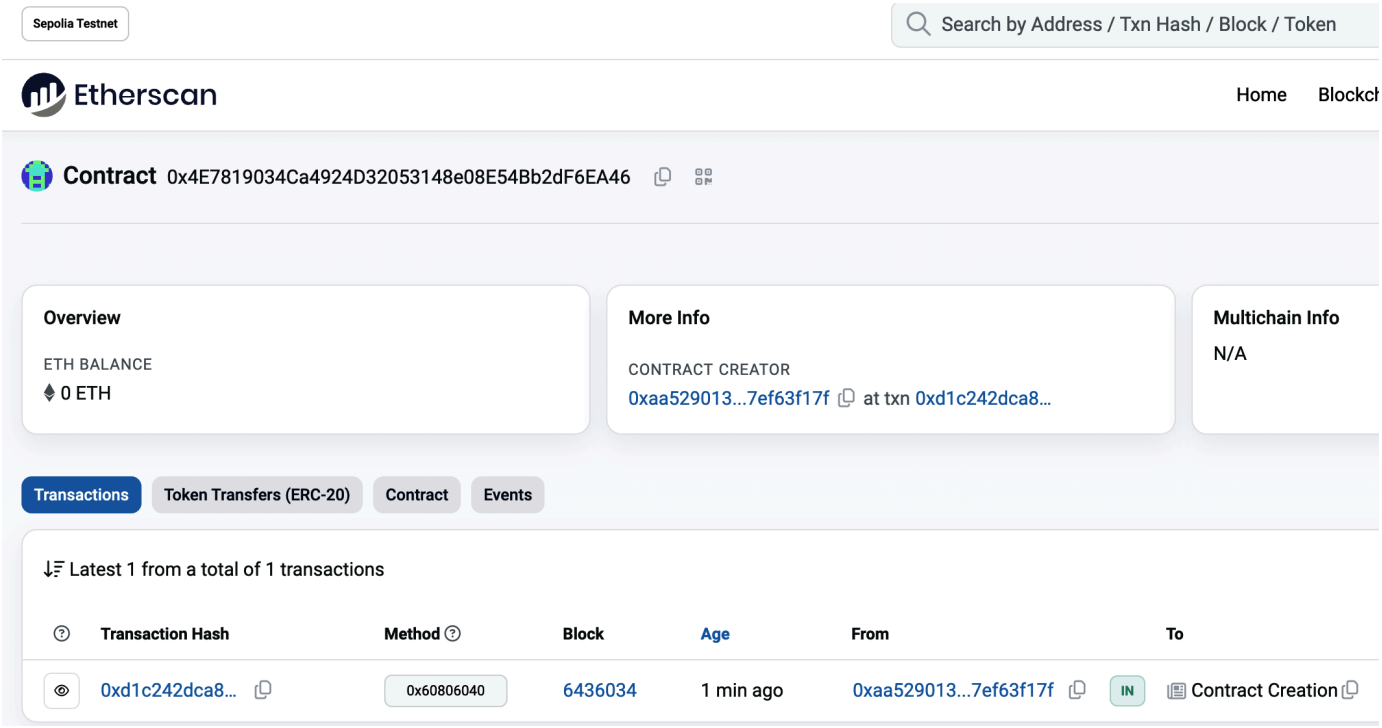


図2-20. EtherscanブロックエクスプローラーでFaucetコントラクトアドレスを表示する

コントラクトへの資金提供

現時点ではコントラクトの履歴にはコントラクト作成トランザクションの1件のみがあります。ご覧の通りコントラクトにはether（残高ゼロ）もありません。作成トランザクションでetherを送信しなかったからです（送信することもできました）。

フォーセットには資金が必要です。最初のプロジェクトとしてMetaMaskを使ってコントラクトにetherを送信します。クリップボードにまだコントラクトのアドレスがあるはずです（なければRemixから再度コピーしてください）。MetaMaskを開き、他のイーサリアムアドレスに送信するのとまったく同じように0.01 etherをコントラクトに送信します（図2-21を参照）。

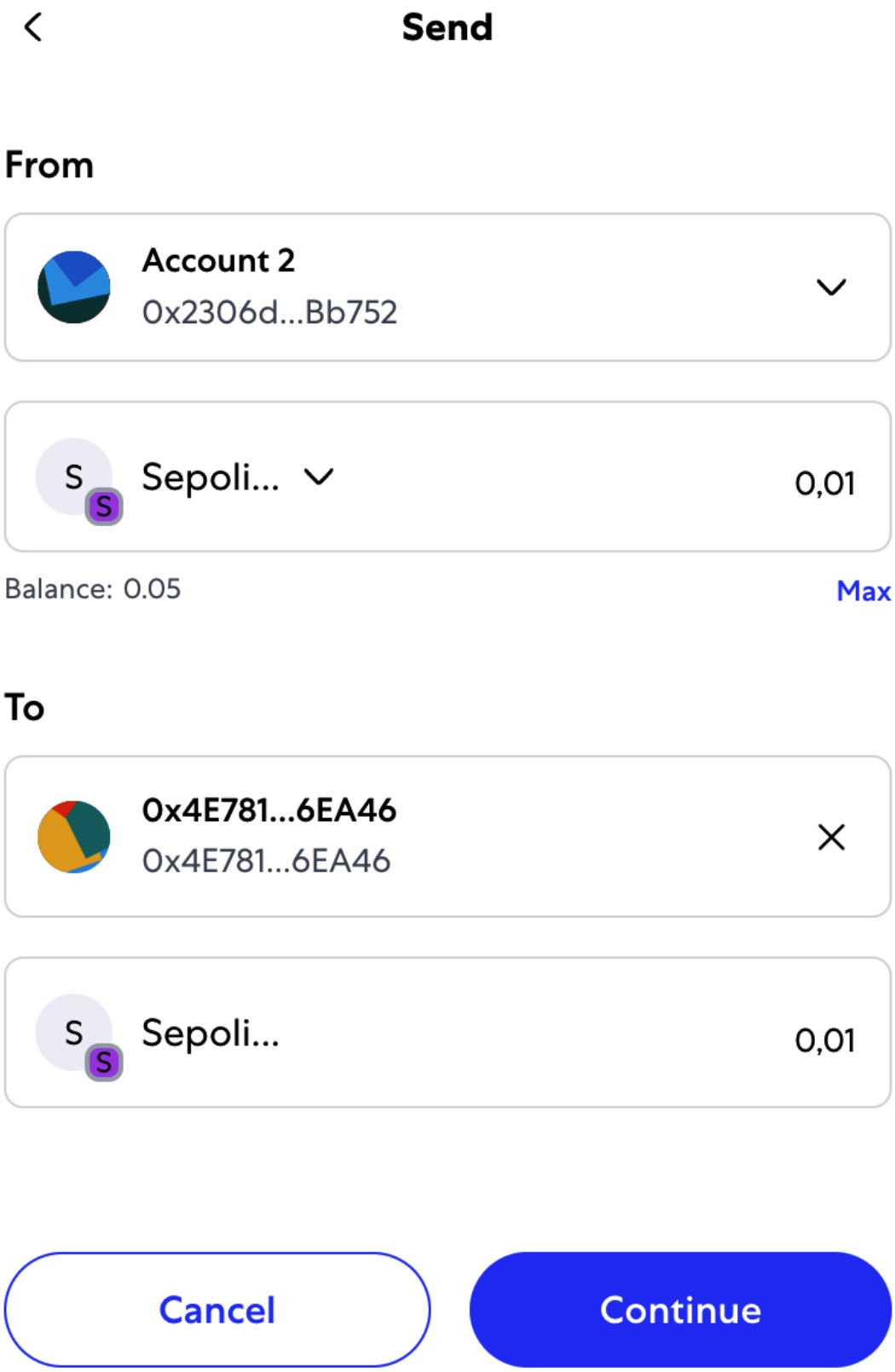


図2-21. コントラクトアドレスに0.01 etherを送信する

1分後にEtherscanブロックエクスプローラーを再読み込みすると、コントラクトアドレスへの別のトランザクションと0.01 etherに更新された残高が表示されます。

*Faucet.sol*コードの`receive`関数を思い出してください。以下のようでした：

```
receive() external payable {}
```

コントラクトアドレスにトランザクションを送信し、呼び出す関数を指定するデータがない場合にこの`receive`関数が呼び出されます。トランザクションによってコントラクトがEVM上で実行され、残高が更新されました。フォーセットに資金を提供したのです。

コントラクトからの引き出し

次にフォーセットから資金を引き出しましょう。引き出しには`withdraw`関数を呼び出し、`_withdrawAmount`と`_to`の引数を渡すトランザクションを構築する必要があります。ここでは簡単に済ませるために、Remixにトランザクションを構築してもらい、MetaMaskで承認します。

Remixのタブに戻り、「Run」タブのコントラクトを確認します。「withdraw」というラベルの付いた赤いボックスと、「uint256 _withdrawAmount, address _to」というラベルの付いたフィールド入力が表示されるはずです（図2-22を参照）。

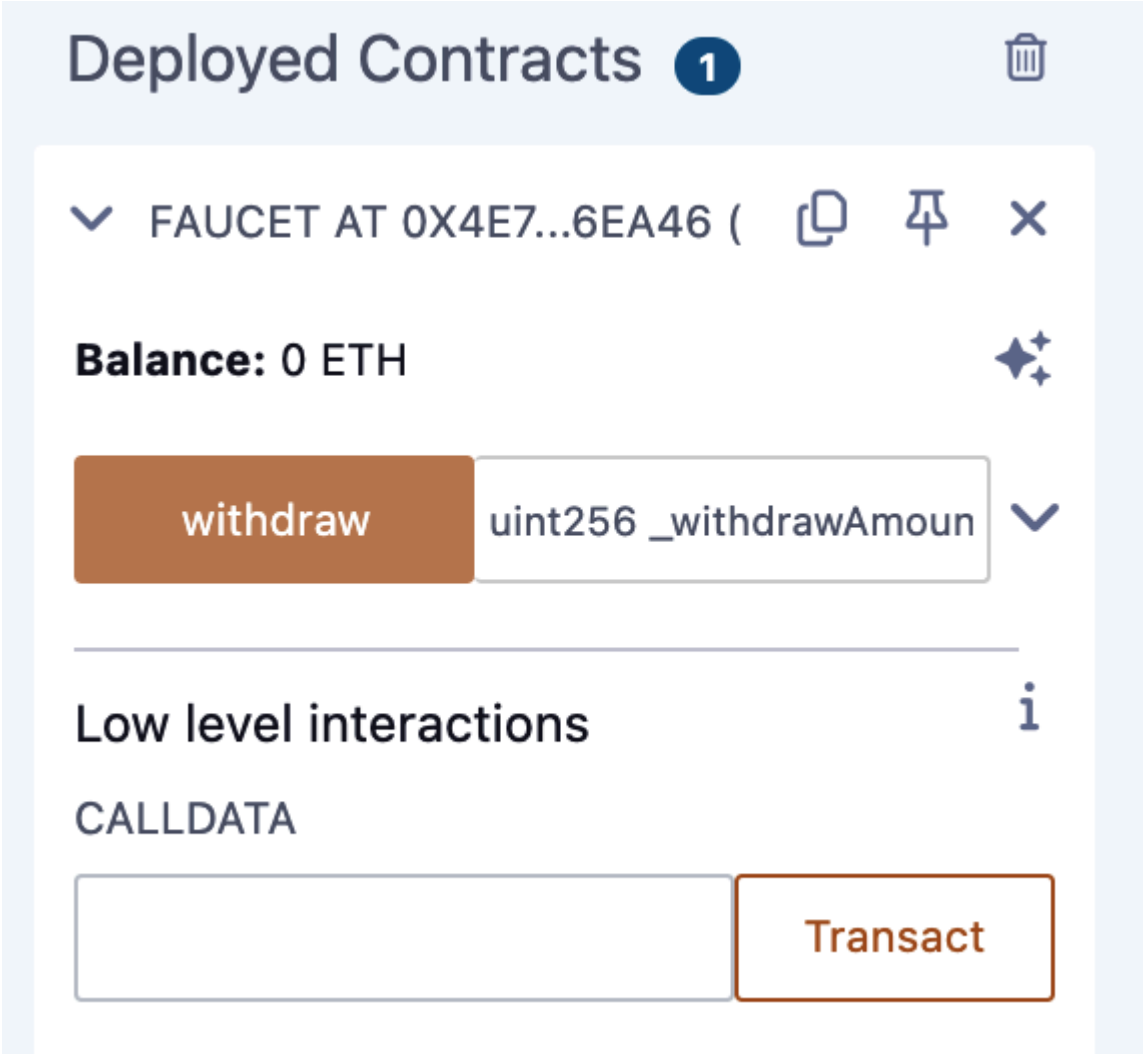


図2-22. Remix上の*Faucet.sol*のwithdraw関数

これはコントラクトへのRemixインターフェースです。コントラクトに定義された関数を呼び出すトランザクションを構築できます。`_withdrawAmount`と`_to`アドレスを入力し、withdrawボタンをクリックしてトランザクションを生成します。

まず`_withdrawAmount`を決めましょう。コントラクトで許可されている最大量である0.000001 etherを引き出してみます。イーサリアムのすべての通貨値は内部的にweiで表記されており、`withdraw`関数も`_withdrawAmount`をwei建てで期待していることを忘れないでください。引き出したい金額は0.000001 etherで、これは1,000,000,000,000 wei（1の後に12個のゼロ）です。

`_to`アドレスにはMetaMaskのAccount 1を使用します。

> **ヒント**
>
> JavaScriptの制限により10^17^を超えるような大きな数値はRemixで処理できない。代わりに引用符で囲み、RemixがBigNumberとして文字列を受け取り操作できるようにする。引用符で囲まないとRemix IDEは処理に失敗し「Error encoding arguments: Error: Assertion failed.」と表示される。

`_withdrawAmount`ボックスに`"1000000000000"`（引用符付き）と入力し、MetaMaskからAccount 1のアドレスをコピーして貼り付けてtransactボタンをクリックします（withdrawボタンとして表示される場合もあります。図2-23は関数の展開ビューです。ビューが展開されていない場合はボタンが「withdraw」と表示されます）。

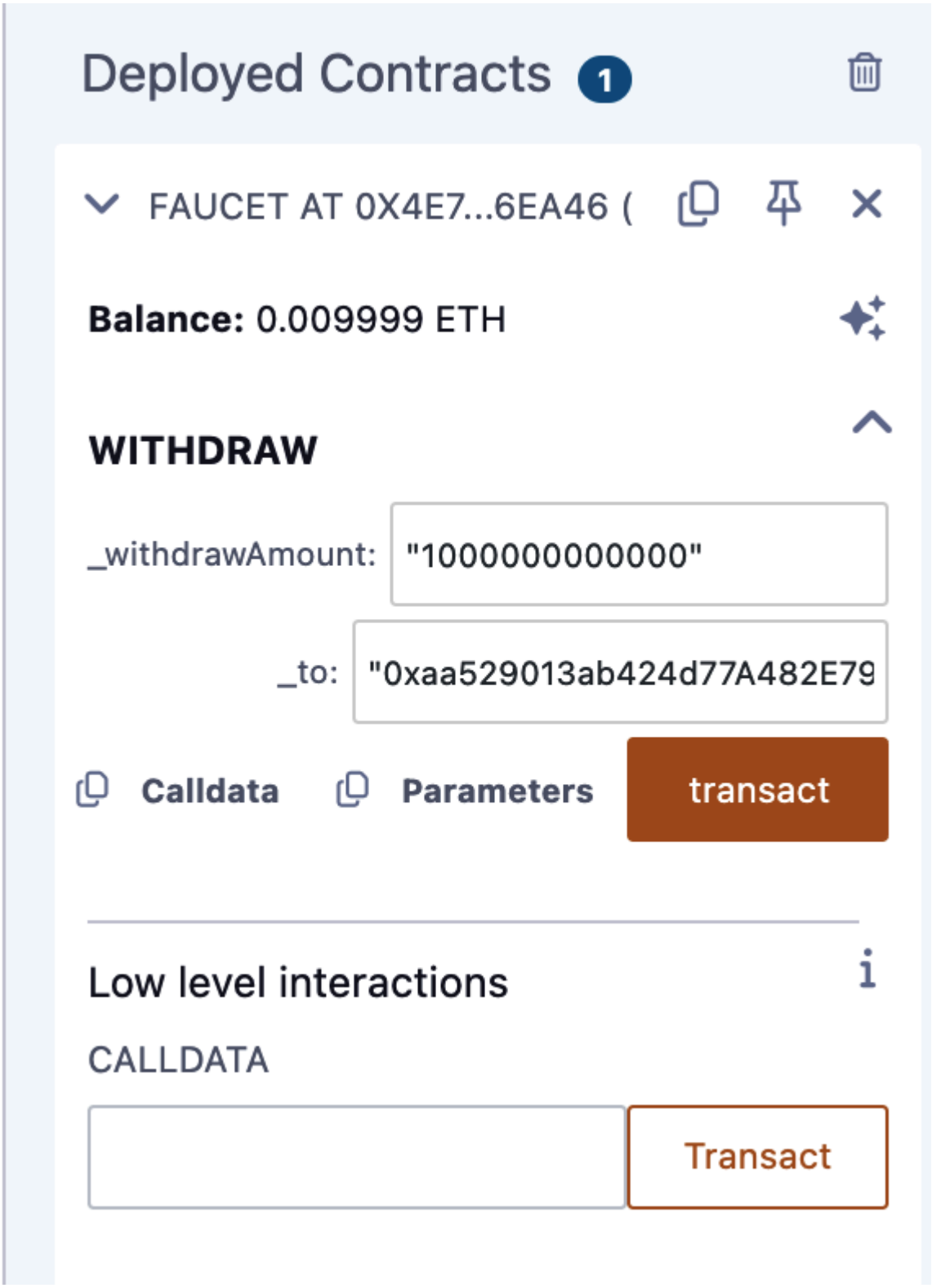


図2-23. Remixでtransactをクリックして引き出しトランザクションを作成する

MetaMaskがトランザクションウィンドウをポップアップして承認を求めます。「送信」をクリックして引き出し操作の呼び出しをコントラクトに送信します（図2-24を参照）。

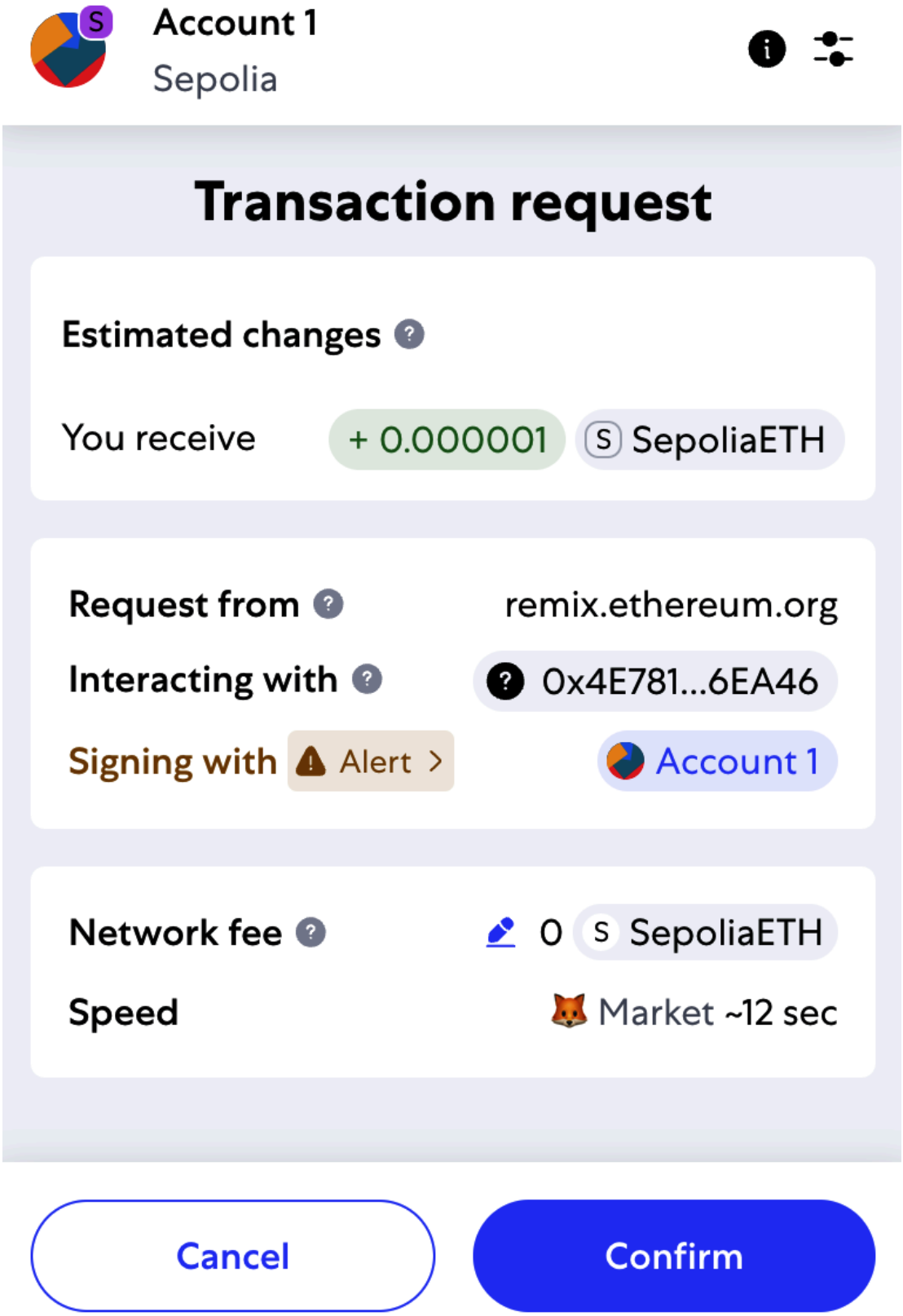


図2-24. withdraw関数を呼び出すMetaMaskトランザクション

1分ほど待ってからEtherscanブロックエクスプローラーを再読み込みすると、`Faucet`コントラクトアドレスの履歴にトランザクションが反映されていることを確認できます（図2-25を参照）。

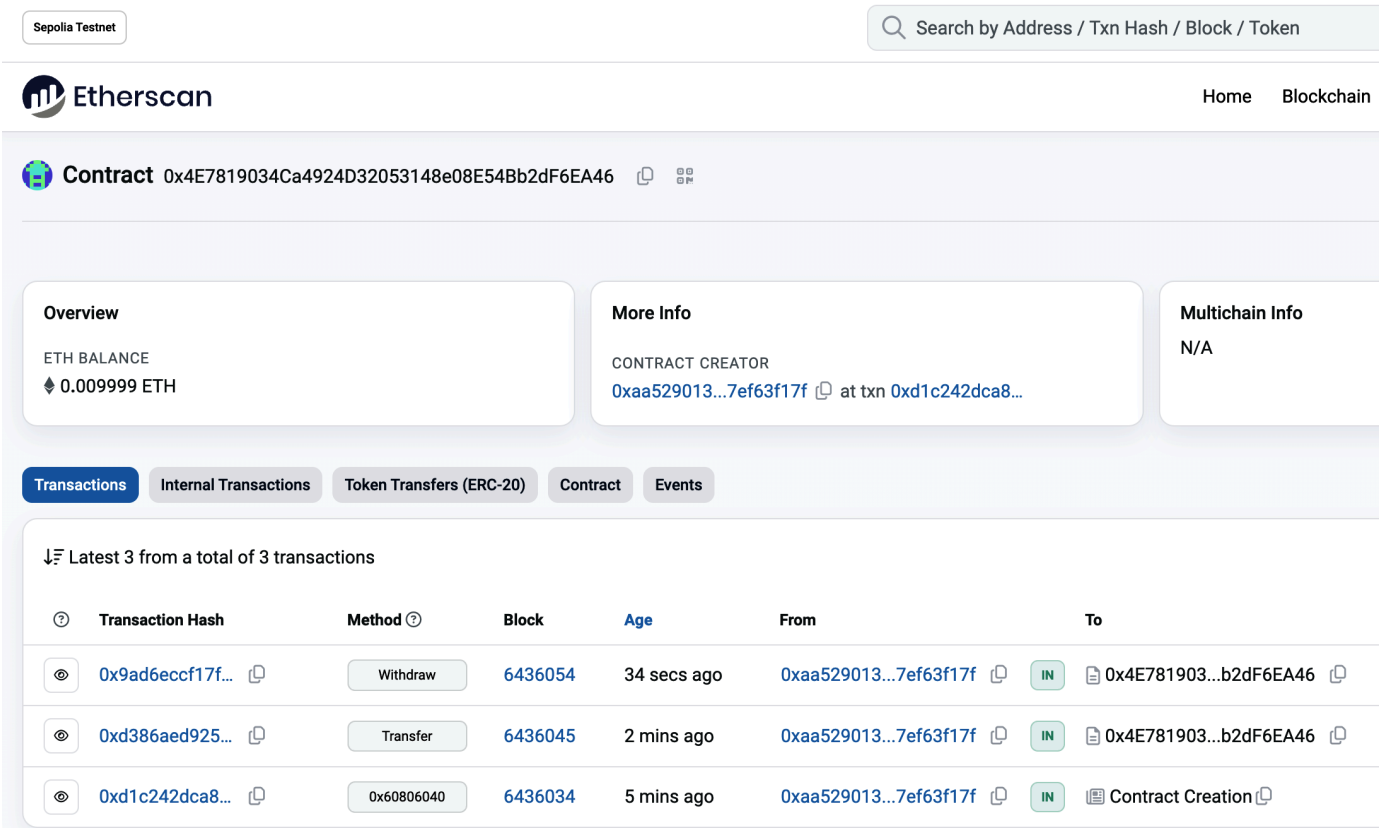


図2-25. withdraw関数の呼び出しを示すEtherscan

コントラクトアドレスを宛先とし値が0 etherの新しいトランザクションが確認できます。コントラクトの残高が変化し、要求通り0.000001 etherが送られたため0.009999 etherになりました。

ただしコントラクトアドレスの履歴には「OUT」トランザクションが見当たりません。送信された引き出しはどこにあるのでしょうか。コントラクトのアドレス履歴ページに「Internal Transactions」という新しいタブが表示されています。0.000001 etherの送金はコントラクトコードから発生したため、インターナルトランザクション（メッセージとも呼ばれる）です。そのタブをクリックして確認しましょう（図2-26を参照）。

この「インターナルトランザクション」は、コントラクトの以下のコード行（*Faucet.sol*の`withdraw`関数内）によって送信されました：

```
_to.transfer(_withdrawAmount);
```

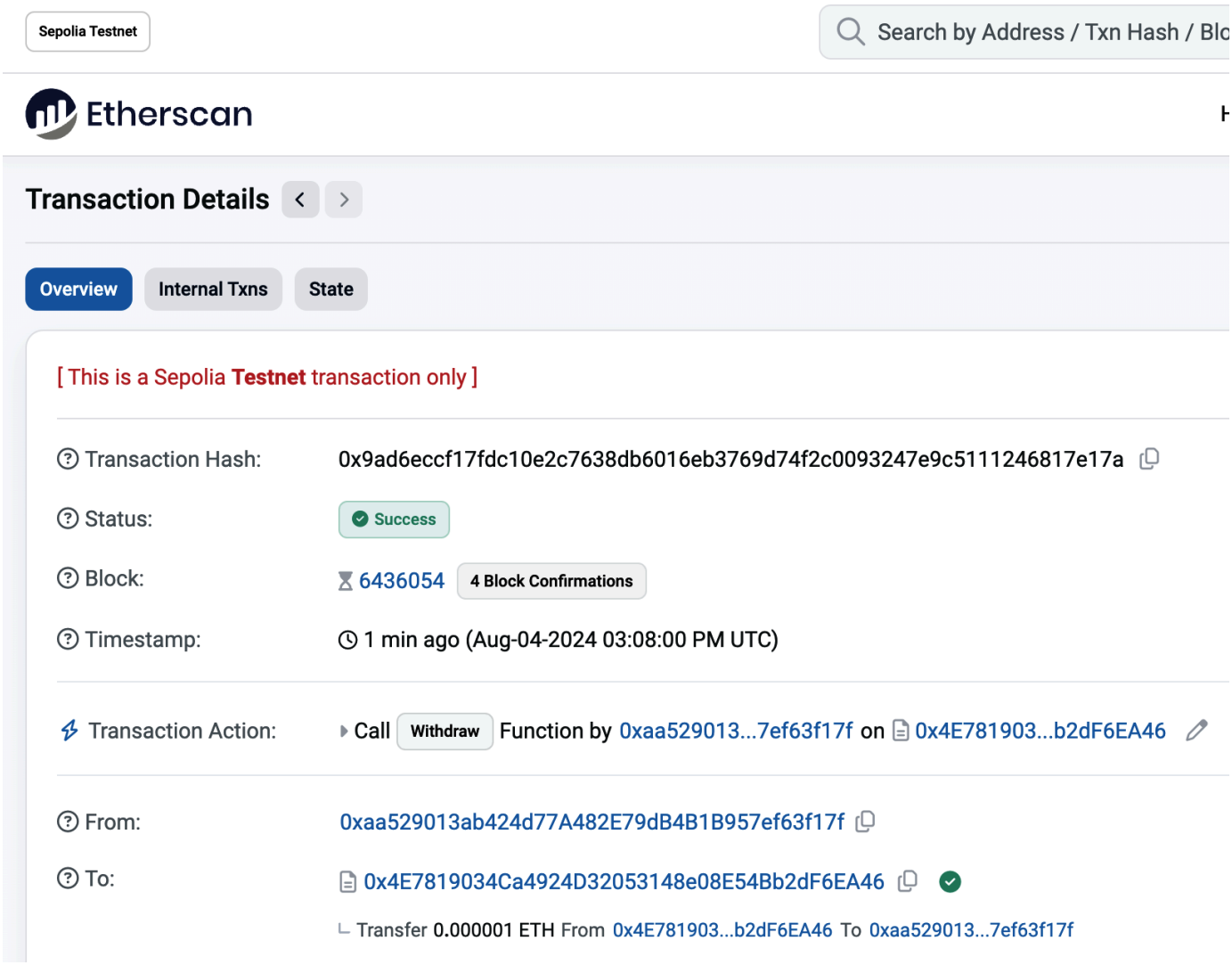


図2-26. コントラクトからetherを転送するインターナルトランザクションを表示するEtherscan

要約すると、まずMetaMaskウォレットから、`withdraw`関数を`_withdrawAmount`引数0.000001 etherとアドレスを指定して呼び出すデータを含むトランザクションを送信しました。このトランザクションによりコントラクトがEVM内で実行されました。EVMは`withdraw`関数を実行し、最初に`require`で引き出し額が0.000001 ether以下であることを検証した後、`transfer`でetherを送信しました。この`transfer`の実行により、コントラクト残高からウォレットアドレスへ0.000001 etherを送るインターナルトランザクションが生成されました。これがEtherscanの「Internal Transactions」タブに表示されているものです。

まとめ

この章ではMetaMaskを使ってウォレットをセットアップし、Sepoliaテストネット上のフォーセットで資金を調達しました。ウォレットのイーサリアムアドレスにetherを受け取り、別のアドレスにetherを送信しました。

次にSolidityで`Faucet`コントラクトを作成しました。Remix IDEでコントラクトをEVMバイトコードにコンパイルし、Remixでトランザクションを作成してSepoliaブロックチェーン上に`Faucet`コントラクトを作成しました。作成されたFaucetコントラクトにはイーサリアムアドレスがあり、etherを送信しました。最後に`withdraw`関数を呼び出すトランザクションを構築し、0.000001 etherの引き出しに成功しました。コントラクトはリクエストを確認し、インターナルトランザクションで0.000001 etherを送信しました。

大したことないように見えるかもしれませんが、非中央集権的なワールドコンピュータ上でお金を制御するソフトウェアとやり取りすることに成功したのです。

第7章でさらに多くのSolidityスマートコントラクトプログラミングを行い、第9章でベストプラクティスとセキュリティに関する考慮事項を学びます。

第3章　イーサリアムノード

イーサリアムノードとは、イーサリアムの仕様を実装し、P2Pネットワークを通じて他のイーサリアムノードと通信するソフトウェアアプリケーションです。

当初、ノードはイーサリアムエコシステムに参加するために必要なすべての要件を満たす単一のクライアントを実行するだけで十分でした。しかし2022年9月15日にThe Merge〔訳注：イーサリアムがPoWからPoSへ移行した重要なアップグレード〕ハードフォークが行われ、コンセンサスプロトコルがPoWベースの方式から新しいPoSベースのプロトコルに変更されました。この新しいプロトコルはGasperと呼ばれ、CasperFFGとLMD-GHOSTを組み合わせたPoSコンセンサスプロトコルです。この変更によりコンセンサスと実行の責務が分離され、新しいタイプのイーサリアムクライアントであるコンセンサスクライアントが誕生しました。

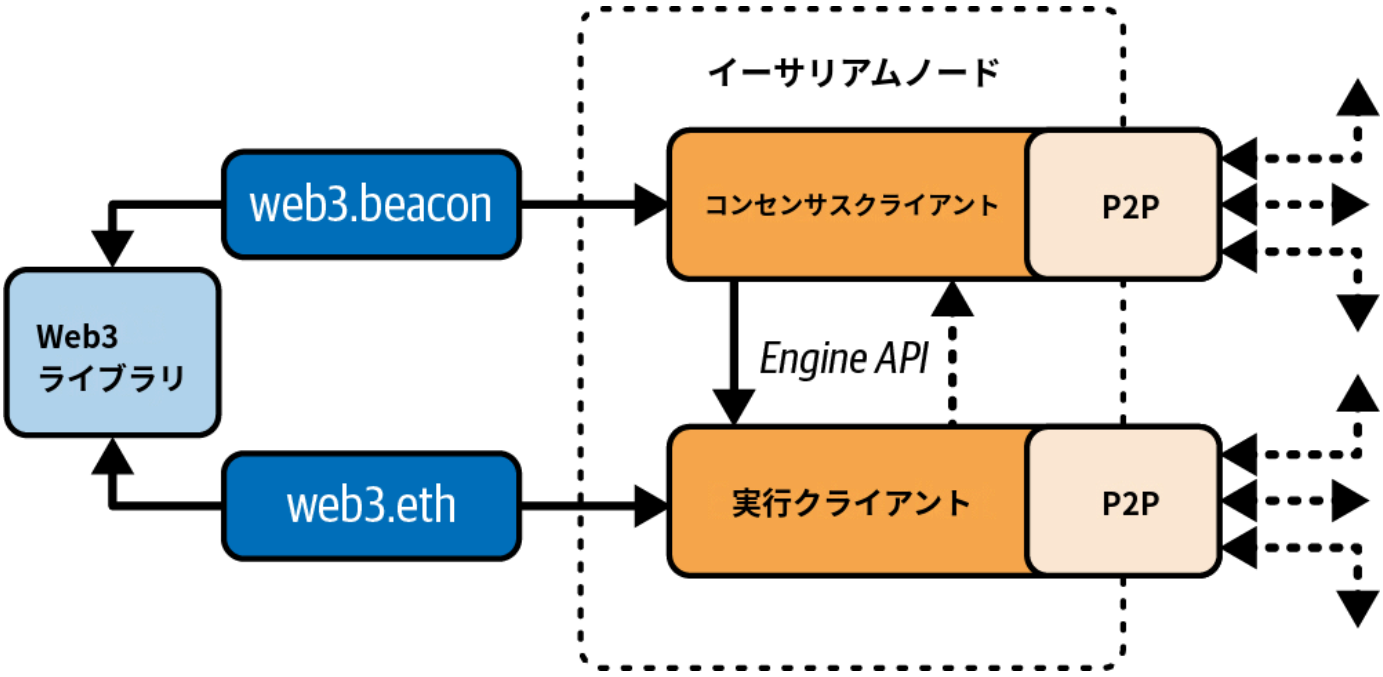
そのため本書執筆時点では、図3-1に示すようにイーサリアムノードは最新の仕様に準拠するために2つのソフトウェアを同時に実行する必要があります。それぞれの定義は以下のとおりです。

コンセンサスクライアント

すべてのノードがブロックチェーンの単一の履歴に合意できるようにするコンセンサスプロトコルを担当する新しいソフトウェアです。

実行クライアント

ネットワーク上で発生するすべてのブロックとトランザクションを受信し、EVM内で実行して正しさを検証することに特化したソフトウェアです。



異なるイーサリアムクライアント（実行クライアントとコンセンサスクライアントの両方）は、リファレンス仕様と標準化された通信プロトコルに準拠していれば相互運用が可能です。各クライアントは異なるチームが異なるプログラミング言語で実装していますが、すべて同じプロトコルを「話し」、同じルールに従います。そのため、どのクライアントでも同じイーサリアムネットワークを操作し、やり取りできます。

イーサリアムはオープンソースプロジェクトであり、すべての主要クライアントのソースコードはオープンソースライセンス（LGPL v3.0など）の下で提供されています。自由にダウンロードでき、いかなる目的にも利用できます。ただオープンソースとは単に無料で使えるという意味だけではありません。イーサリアムはボランティアのオープンコミュニティによって開発されており、誰でも変更を加えることができます。多くの人の目に触れることで、より信頼性の高いコードが実現します。

イーサリアムは当初、本書の原著者の1人であるギャヴィン・ウッドが執筆したイエローペーパーによって定義されていました。イーサリアムに大きな変更が加えられるたびにこの仕様は更新されてきましたが、現在は実行クライアント用とコンセンサスクライアント用の2つのリファレンス実装に向けた明確な方向性があります。これらのリファレンス実装はPythonで書かれており、可読性とシンプルさを重視しています。

> **注記**
>
> これらの仕様はフルノードの実装を意図したものではありません。実行可能な擬似コード仕様として機能するものです。

このように形式仕様で定義されている点は、たとえばビットコインとは対照的です。ビットコインは形式的な方法で定義されておらず、ビットコインの「仕様」はリファレンス実装であるビットコインコアです。一方イーサリアムの実行仕様は、英語による説明と数学的（形式的）な定義を組み合わせた論文に記述されています。この形式仕様はさまざまなEIP（Ethereum Improvement Proposals）やPythonで書かれた新しいコンセンサス仕様とともに、イーサリアムノードの標準的な動作を定義しています。

イーサリアムには明確な形式仕様があるので、独立して開発されながらも相互運用可能なイーサリアムクライアントの実装が多数存在します。イーサリアムは他のどのブロックチェーンよりも多様な実装がネットワーク上で動作しており、これは一般的に良いことだと考えられています。たとえば特定のクライアントの実装戦略を悪用するネットワーク攻撃に対する優れた防御手段となります。あるクライアントの実装が悪用されても、そのクライアントの開発者はパッチ対応に追われますが、他のクライアントはほぼ影響を受けずにネットワークを稼働させ続けるからです。

イーサリアムネットワーク

イーサリアムの「イエローペーパー」で定義された形式仕様に概ね準拠するイーサリアムベースのネットワークは多数存在しますが、相互運用できるとは限りません。

Ethereum Classic、BNB Chain、PolygonなどのEVM互換チェーンは実行仕様の大部分を共有していますが、コンセンサスやパラメーターにおいて異なる点も多くあります。プロトコルレベルではほぼ互換性があるものの、各ネットワークをサポートするためにクライアントソフトウェアのメンテナーが小さな変更を加える必要があることもよくあります。そのためイーサリアムクライアントソフトウェアのすべてのバージョンが、すべてのイーサリアムベースのブロックチェーンで動作するわけではありません。

2025年6月時点で、4つの言語で書かれたイーサリアム実行プロトコルの5つの主要な実装と、5つの言語で書かれたイーサリアムコンセンサスプロトコルの5つの実装があります。

実行クライアントは以下のとおりです。

- Geth（Goで実装）
- Nethermind（C#で実装）
- Besu（Javaで実装）
- Erigon（Goで実装）
- Reth（Rustで実装）

コンセンサスクライアントは以下のとおりです。

- Lighthouse（Rustで実装）
- Lodestar（TypeScriptで実装）
- Nimbus（Nimで実装）
- Prysm（Goで実装）
- Teku（Javaで実装）

本セクションでは、以下の2つの実行クライアントについて見ていきます。

Geth

イーサリアムファンデーションが保守する、最も古く最も広く使われている実行クライアントです。

Reth

Parity/OpenEthereumの開発終了後にParadigmが開発した、新しいRustベースの実行クライアントです。

そして、以下の2つのコンセンサスクライアントについて見ていきます。

Prysm

初期に最も使用されたコンセンサスクライアントであり、現在はOffchain Labsが保守しています。

Lighthouse

最も使用されているコンセンサスクライアントであり、Sigma Primeが保守しています。

各クライアントを使用してノードをセットアップする方法を紹介します。具体的には、Geth-PrysmとReth-Lighthouseの組み合わせを使用し、コマンドラインオプションやAPIについても探っていきます。

> **注記**
>
> これらのペアはあくまで例にすぎません。イーサリアムノードを実行するには、好みの実行クライアントとコンセンサスクライアントを自由に組み合わせることができます。

フルノードを実行すべきか

ブロックチェーンの健全性、レジリエンス、検閲耐性は、独立して運用され地理的に分散した多数のフルノードに依存しています。フルノードとはブロックチェーン全体をダウンロードし、データを無期限に保持するノードです。各フルノードは新しいノードがブロックデータを取得して運用を開始する手助けをするとともに、オペレーターはすべてのトランザクションとコントラクトを確実かつ独立した方法で自ら検証できます。

> **注記**
>
> 正確に言えば、これらのノードには以下の区別があります。
>
> **アーカイブノード** すべてのデータを無期限に保持するイーサリアムノード
>
> **フルノード** 過去のステートとレシートを破棄するイーサリアムノード。通常、ノードを起動する際のデフォルトオプションである

ただしフルノードの運用にはハードウェアリソースと帯域幅のコストが発生します。フルノードは少なくとも2 TBのデータをダウンロードし（2025年6月時点、クライアントの設定による）、ローカルのハードディスクに保存しなければなりません。このデータ量は新しいトランザクションやブロックが追加されるたびに急速に増加します。このトピックについては後のセクション「フルノードのハードウェア要件」で詳しく説明します。

メインネットでフルノードを実行することはイーサリアム開発に必須ではありません。テストネットノード（小規模なパブリックテストブロックチェーンの1つに接続するノード）、Anvilの

ようなローカルのプライベートブロックチェーン、InfuraやAlchemyのようなサービスプロバイダが提供するホスティッドノードAPIを使えば、必要なことのほぼすべてを実行できます。

リモートクライアントを実行するという選択肢もあります。リモートクライアントはブロックチェーンのローカルコピーを保存せず、ブロックやトランザクションの検証も行いません。ウォレットの機能を提供し、トランザクションの作成とブロードキャストが可能です。以下のような既存のネットワークへの接続に使用できます：

- 自分自身のフルノード
- パブリックブロックチェーン
- パブリックまたは許可型（パーミッションド）テストネット（proof-of-authority＝PoA、特定の承認済み参加者のみが検証を行うネットワーク）
- プライベートなローカルブロックチェーン

実際にはMetaMask、Rabby Wallet、Coinbase Walletなどのリモートクライアントを、さまざまなノードオプションを切り替える便利な手段として使うことが多いでしょう。

リモートクライアントとウォレットという用語は同じ意味で使われますが、いくつかの違いがあります。通常リモートクライアントはウォレットのトランザクション機能に加えてAPI（web3.js APIなど）を提供します。

イーサリアムにおけるリモートクライアントの概念を、軽量クライアント（ビットコインにおけるSimplified Payment Verification〔SPV〕クライアントに相当するもの）と混同しないでください。軽量クライアントはブロックヘッダーを検証し、マークル証明（ハッシュツリーを利用してデータの存在を効率的に証明する手法）を使ってブロックチェーンへのトランザクションの包含を検証してその影響を判定します。これによりフルノードに近いセキュリティレベルを提供します。

一方イーサリアムのリモートクライアントはブロックヘッダーやトランザクションを検証しません。フルノードに完全に依存してブロックチェーンにアクセスするため、セキュリティと匿名性の保証が大幅に低下します。自分自身でフルノードを運用すれば、これらの問題を軽減できます。

フルノードの利点と欠点

フルノードを実行すると接続先のネットワークの運用に貢献できますが、軽微から中程度のコストが発生します。利点と欠点を見てみましょう。

利点

- イーサリアムベースのネットワークのレジリエンスと検閲耐性をサポートする
- すべてのトランザクションを自ら確実に検証する
- 仲介者なしにパブリックブロックチェーン上の任意のコントラクトとやり取りできる
- 仲介者なしにパブリックブロックチェーンにコントラクトを直接デプロイできる

- ブロックチェーンの状態（アカウント、コントラクトなど）をオフラインで照会（読み取り専用）できる
- 第三者に読み取っている情報を知らせることなくブロックチェーンを照会できる

欠点

- 大量かつ増加し続けるハードウェアと帯域幅のリソースが必要である
- 最初の起動時に完全な同期に数日かかる場合がある
- 同期を維持するためにメンテナンス、アップグレード、オンラインの維持が必要である

パブリックテストネットの利点と欠点

フルノードを実行するかどうかにかかわらず、パブリックテストネットのノードを実行したくなることが多いでしょう。利点と欠点を見てみましょう。

利点

- テストネットノードの同期と保存に必要なデータ量は少なく、ネットワークにもよるが約100〜300 GB程度（2025年6月時点）。
- テストネットノードは数時間で完全に同期できる。
- コントラクトのデプロイやトランザクションの実行にはテスト用etherが必要だが、テスト用etherは価値がなく、いくつかの「フォーセット」から無料で取得できる。
- テストネットは多くのユーザーやコントラクトが存在し、「ライブ」で動作するパブリックブロックチェーンである。

欠点

- テストネットでは「本物の」お金は使えず、テスト用etherで動作する。経済的な利害関係がないため、実際の攻撃者に対するセキュリティをテストできない。
- パブリックブロックチェーンの一部の側面はテストネットでは現実的にテストできない。たとえばトランザクション手数料はトランザクションの送信に必要だが、テストネットではガスが無料のため（テストネットのETHには実際の経済的価値がないため）気にする必要がない。またテストネットでは、パブリックメインネットで時折発生するようなネットワーク混雑が起きない。
- 特定の目的のために設計されたテストネットもあり、イーサリアムメインネットとわずかに異なることがある。

ローカルブロックチェーンシミュレーションの利点と欠点

多くのテスト目的において、最良の選択肢はシングルインスタンスのプライベートブロックチェーンを起動することです。Anvilは最も人気のあるローカルブロックチェーンシミュレーションの1つであり、他の参加者なしに実行してやり取りできます。

利点

- 同期が不要で、ディスク上のデータもほぼゼロ。最初のブロックを自分自身で生成する。
- テスト用etherを入手する必要がない。テストに使用できるブロック報酬を自分自身に「付与」する。
- 他のユーザーがいない。自分だけである。
- 他のコントラクトがない。起動後にデプロイするもののみ。

欠点

- 他のユーザーがいないため、ローカルチェーンはパブリックブロックチェーンと同じようには動作しない。トランザクションスペースの競争やトランザクションの順序付けもない。
- 自分以外のブロックプロデューサーがいないため、ブロック生成がより予測可能で、パブリックブロックチェーンで発生する一部のシナリオをテストできない。AnvilやHardhatなどのツールではブロック生成モードを設定してメインネットに近い挙動を再現できるが、イーサリアムメインネット上にいるのとは同じではない。
- 他のコントラクトがないため、テストしたいものをすべてデプロイする必要がある（依存関係やコントラクトライブラリを含む）。幸いAnvilのようなツールではイーサリアムメインネットのチェーンを任意のブロックでフォークし、メインネットに近い状態でスマートコントラクトを試せる。

イーサリアムノードの実行

時間とリソースがあるならば、仕組みを学ぶためだけでもフルノードの実行を試みるべきです。本セクションではイーサリアムクライアントであるGeth-PrysmおよびReth-Lighthouseのダウンロード、コンパイル、実行方法について説明します。OSのコマンドラインインターフェース（CLI）の使用にある程度精通している必要があります。フルノード、テストネットノード、ローカルのプライベートブロックチェーンのいずれのクライアントとして実行する場合でも、これらのクライアントをインストールする価値はあります。

フルノードのハードウェア要件

始める前に、イーサリアムフルノードを実行するのに十分なリソースを持つコンピュータがあることを確認してください。イーサリアムブロックチェーンの完全なコピーを保存するには少なくとも2 TBのディスクスペースが必要です。イーサリアムテストネットのフルノードも実行したい場合は、追加で少なくとも100〜400 GBが必要になります。2 TBのブロックチェーンデータのダウンロードには長い時間がかかるため、高速なインターネット接続で作業することを推奨します。

イーサリアムブロックチェーンの同期はI/O（入出力）集約型の処理です。ソリッドステートドライブ（SSD）の使用が最適です。ハードディスクドライブ（HDD）を使用する場合はキャッシュとして少なくとも8 GBのRAMが必要です。さもないとシステムが遅すぎて完全な同期に追いつけない可能性があります。

イーサリアムベースのブロックチェーンの完全なコピーを同期するための最小要件の概要は以下のとおりです。

- 2コア以上のCPU
- 少なくとも2 TBの空きストレージ
- SSD使用時は最低8 GBのRAM、HDD使用時は8 GB以上（SSDを強く推奨）
- 7 Mbps以上のダウンロード速度

妥当な時間で同期し、本書で説明するすべての開発ツール、ライブラリ、クライアント、ブロックチェーンを保存したい場合はより高性能なコンピュータが必要です。推奨スペックは以下のとおりです。

- 4コア以上の高速CPU。コア数よりもクロック速度が重要
- 16 GB以上のRAM
- 少なくとも2 TBの空き容量がある高速NVMe SSD
- 24 Mbps以上のダウンロードインターネットサービス

ブロックチェーンのサイズがどのくらいのペースで増加し、いつ追加のディスクスペースが必要になるかを予測するのは困難です。同期を開始する前にブロックチェーンの最新サイズを確認することを推奨します。

注記

ここに記載されているディスクサイズの要件は、古いステートデータを「プルーニング」するデフォルト設定でノードを実行することを前提としています。すべてのステートをディスク上に保持する完全な「アーカイブ」ノードを実行する場合、クライアントによっては2 TB以上（最大12〜15 TB）のディスクスペースが必要になることがあります。ノードを実行する前に、公式クライアントのウェブサイトで最新のハードウェア要件を必ず確認してください。

クライアントのビルドと実行のソフトウェア要件

本セクションではGeth-PrysmとReth-Lighthouseのクライアントソフトウェアについて説明します。Unix系のコマンドライン環境を使用していることを前提としています。以下の例ではBashシェル（コマンドライン実行環境）を実行するmacOSでのコマンドと出力を示しています。ほとんどのLinuxディストリビューションでも同じ手順が変更なしで動作します。Windowsユーザーは Windows Subsystem for Linux（WSL2）を使用できます。

ヒント

この章の多くの例ではターミナルアプリケーションからアクセスできるOSのCLI（シェルとも呼ばれる）を使用します。シェルはプロンプトを表示し、コマンドを入力するとテキストで応答し、次のコマンドのための新しいプロンプトを表示します。プロンプトはシステムによって異なりますが、以下の例では$記号で示しています。$記号の後に表示されているコマンド（太字で示されています）を入力し、Enterキーを押して実行してください（$記号自体は入力しません）。各コマンドの下の行はOSの応答です。次の$プレフィックスが表示されたら、新しいコマンドの始まりです。同じ手順を繰り返します。

始める前にいくつかのソフトウェアのインストールが必要になる場合があります。現在使用しているコンピュータでソフトウェア開発をしたことがなければ、基本的なツールのインストールが必要でしょう。以降の例ではソースコード管理システムであるgit、Goプログラミング言語と標準ライブラリであるgolang、システムプログラミング言語であるRustのインストールが必要です。

この例で使用する4つのクライアントのドキュメントページは以下のとおりです。

- Geth
- Prysm
- Reth
- Lighthouse

各クライアントのアーキテクチャの詳細やインストール中のトラブルシューティングには、これらのウェブサイトを参照してください。

準備フェーズ

ホームディレクトリから始めて、コンピュータ上に*ethereum-node1*というフォルダを作成し、その中に*execution*と*consensus*の2つのサブフォルダを作成します。

```
$ mkdir ethereum-node1
$ cd ethereum-node1
$ mkdir execution
$ mkdir consensus
```

これで以下のようなフォルダ構造になっているはずです。

```
ethereum-node1
├──── consensus
└──── execution
```

前のステップを*ethereum-node2*という新しいフォルダで繰り返します。

```
$ cd .. # このコマンドはホームディレクトリに戻るために使用する
$ mkdir ethereum-node2
$ cd ethereum-node2
$ mkdir execution
$ mkdir consensus
```

最終的に、2つのルートフォルダ（*ethereum-node1*と*ethereum-node2*）があり、それぞれのルートフォルダ内に*execution*と*consensus*の2つのサブフォルダが存在するはずです。

また、GoとRustのインストールも必要です。インストール方法については各公式ウェブサイトのガイドを参照してください。

Geth-Prysm

新しく作成した*ethereum-node1*フォルダに移動します。

```
$ cd ethereum-node1
```

Geth

まずソースコードからビルドしてGethをインストールします。GethはGo言語による実行仕様の実装であり、イーサリアムファンデーションが活発に開発しているためイーサリアムクライアントの「公式」実装と見なされています。通常、イーサリアムベースのブロックチェーンにはそれぞれ独自のGeth実装があります。Gethを使用する場合は以下のレポジトリリンクのいずれかから、自分のブロックチェーンに適したバージョンを取得してください。

- イーサリアム
- BNB Chain
- Polygon PoS

> **注記**
>
> これらの手順をスキップして、プラットフォームに合ったコンパイル済みバイナリをインストールすることもできます。コンパイル済みのリリースはインストールがはるかに簡単で、ここに記載されているレポジトリの「releases」セクションから入手できます。ただしソフトウェアを自分でダウンロードしてコンパイルすれば、より多くのことを学べるでしょう。

レポジトリのクローン。最初のステップはGitレポジトリをクローンしてソースコードのコピーを取得することです。executionサブフォルダ内で以下のようにgitコマンドを使用してレポジトリのローカルクローンを作成します。

```
$ cd execution
$ git clone https://github.com/ethereum/go-ethereum.git
```

レポジトリがローカルシステムにコピーされる際に進捗レポートが表示されるはずです。

```
Cloning into 'go-ethereum'...
remote: Enumerating objects: 130745, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 130745 (delta 1), reused 6 (delta 0), pack-reused 130734
Receiving objects: 100% (130745/130745), 204.15 MiB | 6.13 MiB/s, done.
Resolving deltas: 100% (80729/80729), done.
```

Gethのローカルコピーが取得できたので、プラットフォーム用の実行可能ファイルをコンパイルできます。

ソースコードからのGethのビルド。Gethをビルドするにはソースコードがダウンロードされたディレクトリに移動し、最新リリースを選択した後にmakeコマンドを使用します。現時点ではv1.14.3ですが、最新版は公式リポジトリで随時確認できます。

```
$ cd go-ethereum
$ git checkout v1.14.3
$ make geth
```

すべてがうまくいけば、Gethの実行可能ファイルが完成するまでGoコンパイラが各コンポーネントをビルドしている様子が表示されます。

```
go run build/ci.go install ./cmd/geth
go: downloading golang.org/x/crypto v0.22.0
go: downloading golang.org/x/net v0.24.0
[...]
github.com/ethereum/go-ethereum/cmd/utils
github.com/ethereum/go-ethereum/beacon/blsync
github.com/ethereum/go-ethereum/cmd/geth
Done building.
Run "./build/bin/geth" to launch geth.
```

実際に起動せずにGethが動作するか確認してみましょう。

```
$ ./build/bin/geth version

GethVersion: 1.14.3-stable
```

```
Git Commit: ab48ba42f4f34873d65fd1737fabac5c680baff6
Architecture: arm64
Go Version: go1.22.2
Operating System: darwin
[...]
```

`geth version`コマンドの出力はわずかに異なるかもしれませんが、上記のようなバージョンレポートが表示されるはずです。

まだGethを実行しないでください。チェーンの先端まで同期するために、コンセンサスクライアントをインストールする必要があります。

Prysm

次はコンセンサスクライアントの番です。Prysmはコンセンサス仕様のGo言語実装であり、Offchain Labsが活発に開発しています。当初はThe Merge後のコンセンサスクライアントとして最も多く使われていました。現在はクライアントの多様性を高めるコミュニティの取り組みにより、市場シェアは大幅に減少して37%となっています。

バイナリのインストール。PrysmはGethと同様にソースコードからビルドすることもできますが、少し複雑です。推奨されるインストール方法は以下のとおりです。まず*consensus*フォルダに移動します。

```
$ cd ../.. # このコマンドはethereum-node1フォルダに戻るために使用する
$ cd consensus
```

次に、以下のコマンドを実行します。

```
$ curl https://raw.githubusercontent.com/prysmaticlabs/prysm/master/prysm.sh --output prysm.sh
&& chmod +x prysm.sh
```

JWT（JSON Web Token）シークレットの生成。イーサリアムノードを構成する実行クライアントとコンセンサスクライアントは2つの別々のソフトウェアですが、常に相互にやり取りする必要があります。そのため両方のクライアントが接続の認証に使用するパスワードに相当するものがあります。ここではそのパスワードを生成します。

```
$ ./prysm.sh beacon-chain generate-auth-secret
```

*jwt.hex*ファイルが生成されるはずです。そのファイルを親フォルダに移動しましょう。

```
$ mv jwt.hex ../jwt.hex
```

ノードの実行

実行クライアントとコンセンサスクライアントの両方をインストールし、JWTシークレットを正しく生成できたので、クライアントを起動してイーサリアムフルノードを稼働させましょう。

実行クライアントの実行。まず実行クライアントであるGethを実行します。*execution*フォルダに戻り、以下のコマンドを実行します。

```
$ cd .. # このコマンドはethereum-node1フォルダに戻るために使用する
$ cd execution
$ ./go-ethereum/build/bin/geth --mainnet \
    --http \
    --http.api eth,net,engine,admin,web3 \
    --authrpc.jwtsecret=../jwt.hex
```

以下のような表示が出れば、正常に動作しています。

```
INFO [06-08|17:56:38.738] Starting Geth on Ethereum mainnet...
INFO [06-08|17:56:38.738] Bumping default cache on mainnet         provided=1024 updated=4096
INFO [06-08|17:56:38.740] Maximum peer count                       ETH=50 total=50
INFO [06-08|17:56:38.745] Set global gas cap                       cap=50,000,000
INFO [06-08|17:56:38.752] Initializing the KZG library             backend=gokzg
INFO [06-08|17:56:38.771] Allocated trie memory caches             clean=614.00MiB
dirty=1024.00MiB
INFO [06-08|17:56:38.772] Using pebble as the backing database…
```

コンセンサスクライアントの実行。次にコンセンサスクライアントであるPrysmを実行します。実行クライアントが動作しているターミナルタブは閉じないでください。新しいターミナルウィンドウまたはタブを開き、*consensus*フォルダに移動します。

```
$ cd ethereum-node1
$ cd consensus
$ ./prysm.sh beacon-chain \
    --execution-endpoint=http://localhost:8551 \
    --mainnet \
    --jwt-secret=../jwt.hex \
    --checkpoint-sync-url=https://beaconstate.info \
    --genesis-beacon-api-url=https://beaconstate.info
```

Prysmの利用規約への同意を求められることがあります。その場合は**accept**と入力すれば完了です。

```
Prysm Terms of Use
By downloading, accessing or using the Prysm implementation ("Prysm"), you (referenced
herein as "you" or the "user") certify that you have read and agreed to the terms and
conditions below.
```

```
TERMS AND CONDITIONS: https://github.com/prysmaticlabs/prysm/blob/develop/TERMS_OF_SERVICE.md
Type "accept" to accept this terms and conditions [accept/decline]: (default: decline):
```

以上で完了です。実行クライアントとコンセンサスクライアントの両方がターミナルに大量のデータをログ出力し始めるはずです。

実行クライアント：

```
INFO [06-08|18:08:49.039] Forkchoice requested sync to new head    number=20,048,206
hash=8df21a..4afb49 finalized=unknown
INFO [06-08|18:08:52.507] Syncing beacon headers                   downloaded=322,560
left=19,725,577 eta=42m4.183s
INFO [06-08|18:08:57.515] Looking for peers                        peercount=1 tried=42
static=0
INFO [06-08|18:09:00.508] Syncing beacon headers                   downloaded=370,688
left=19,677,449 eta=43m35.827s
INFO [06-08|18:09:01.637] Forkchoice requested sync to new head    number=20,048,207
hash=d99dab..0293c9 finalized=unknown
```

コンセンサスクライアント：

```
[2024-06-08 18:09:24]  INFO blockchain: Called new payload with optimistic block
payloadBlockHash=0xd44520a09a7a slot=9253245
[2024-06-08 18:09:24]  INFO blockchain: Called fork choice updated with optimistic block
finalizedPayloadBlockHash=0x38916be8a559 headPayloadBlockHash=0xd44520a09a7a headSlot=9253245
[2024-06-08 18:09:24]  INFO blockchain: Synced new block block=0xec930e7c...
epoch=289163finalizedEpoch=289161 finalizedRoot=0xb8065a78... slot=9253245
[2024-06-08 18:09:24]  INFO blockchain: Finished applying state transition attestations=123
payloadHash=0xd44520a09a7a slot=9253245 syncBitsCount=510 txCount=212
[2024-06-08 18:09:24]  INFO p2p: Peer summary activePeers=64 inbound=0 outbound=63
[2024-06-08 18:09:28]  INFO sync: Subscribed to
topic=/eth2/6a95a1a9/beacon_attestation_35/ssz_snappy[2024-06-08 18:09:36]  INFO blockchain:
Called new payload with optimistic block payloadBlockHash=0xff879102f29e slot=9253246
```

これでチェーンの先端に向けて同期中のイーサリアムフルノードが稼働しています。同期にはかなりの時間がかかる場合があります（ハードウェアとインターネット接続速度に応じて数時間から数日）。

> **注記**
>
> この例で使用した具体的なコマンドやCLIフラグについて詳しく知りたい場合は、Gethと Prysmの公式ドキュメントが最適な情報源です。

Reth-Lighthouse

同じことをRethとLighthouseという2つの異なるクライアントで行ってみましょう。Rethを実行クライアント、Lighthouseをコンセンサスクライアントとして使用します。

Reth

まずRethをインストールします。*ethereum-node2*フォルダに移動し、executionフォルダに入ります。

レポジトリのクローン。最初のステップはGitレポジトリをクローンしてソースコードのコピーを取得することです。ホームディレクトリに戻り、以下のコマンドを入力します。

```
$ cd ethereum-node2
$ cd execution
$ git clone https://github.com/paradigmxyz/reth
```

Rethのローカルコピーが取得できたので、プラットフォーム用の実行可能ファイルをコンパイルできます。

ソースコードからのRethのビルド。Rethをビルドするには以下のコマンドを実行します。

```
$ cd reth
$ cargo install --locked --path bin/reth --bin reth
```

インストールの完了までに10分以上かかることがあります。完了したら以下を実行してRethが正しくインストールされたか確認できます。

```
$ reth --version
```

以下のような表示がされるはずです（バージョンは変わることがあります）。

```
reth Version: 0.2.0-beta.6-dev
Commit SHA: ac29b4b73
Build Timestamp: 2024-04-22T17:29:01.000000000Z
Build Features: jemallocBuild Profile: maxperf+
```

Lighthouse

次にコンセンサスクライアントであるLighthouseをインストールします。*ethereum-node2*フォルダに戻り、*consensus*フォルダに入ります。

```
$ cd .. # このコマンドはethereum-node2フォルダに戻るために使用する
$ cd consensus
```

まずいくつかの依存関係をインストールする必要があります。macOSの場合は、以下を実行します。

```
$ brew install cmake
```

別のOSを使用している場合はLighthouseの公式ドキュメントを参照してください。

レポジトリのクローン。最初のステップはGitレポジトリをクローンしてソースコードのコピーを取得することです。

```
$ git clone https://github.com/sigp/lighthouse.git
```

Lighthouseのローカルコピーが取得できたので、プラットフォーム用の実行可能ファイルをコンパイルできます。

ソースコードからのLighthouseのビルド。Lighthouseをビルドするには以下のコマンドを実行します。

```
$ cd lighthouse
$ git checkout stable
$ make
```

完了までに10分以上かかる場合があります。

ノードの実行

ここでもまず実行クライアントであるRethを実行します。

実行クライアントの実行。*execution*フォルダに戻り、以下のコマンドを実行します。

```
$ cd ../.. # このコマンドはethereum-node2フォルダに戻るために使用する
$ cp ../ethereum-node1/jwt.hex ./jwt.hex # 以前生成したjwt.hexファイルを再利用する
$ cd execution
$ reth node --full --http --http.api all --authrpc.jwtsecret=../jwt.hex
```

以下のような表示が出れば、正常に動作しています。

```
2024-06-08T16:58:43.498297Z  INFO Starting reth version="0.2.0-beta.6-dev (ac29b4b73)"
2024-06-08T16:58:43.498434Z  INFO Opening database
path="/Users/alessandromazza/Library/Application Support/reth/mainnet/db"
2024-06-08T16:58:43.514141Z  INFO Configuration loaded
```

```
path="/Users/alessandromazza/Library/Application Support/reth/mainnet/reth.toml"
2024-06-08T16:58:43.514778Z  INFO Database opened
2024-06-08T16:58:43.514917Z  INFO Pre-merge hard forks (block based):…
```

コンセンサスクライアントの実行。次にコンセンサスクライアントであるLighthouseを実行します。実行クライアントが動作しているターミナルタブは閉じないでください。新しいターミナルウィンドウまたはタブを開き、*consensus*フォルダに移動します。

```
$ cd ethereum-node2
$ cd consensus
$ lighthouse bn \
    --network mainnet \
    --checkpoint-sync-url https://mainnet.checkpoint.sigp.io \
    --execution-endpoint http://localhost:8551 \
    --execution-jwt ../jwt.hex \
    --http
```

以上で完了です。実行クライアントとコンセンサスクライアントの両方がターミナルに大量のデータをログ出力し始めるはずです。

実行クライアント：

```
2024-06-08T17:03:03.355648Z  INFO Received headers total=10000 from_block=18458372
to_block=18448373
2024-06-08T17:03:04.792262Z  INFO Received headers total=10000 from_block=18448372
to_block=18438373
2024-06-08T17:03:04.800043Z  INFO Received headers total=10000 from_block=18438372
to_block=18428373
2024-06-08T17:03:04.913377Z  INFO Received headers total=10000 from_block=18428372
to_block=18418373
```

コンセンサスクライアント：

```
Jun 08 17:03:24.929 INFO New block received                      root:
0xa49c057026cea3190df38548d49963e271ebdc4d6f93d2301adc4034d6563113, slot: 9253515
Jun 08 17:03:29.001 WARN Head is optimistic                      execution_block_hash:
0x5a14bfcb9e74c5b3a5121f99ef461ae066262200c269b5d11475274eb78aa7a5, info: chain not fully
verified, block and attestation production disabled untilexecution engine syncs, service:
slot_notifier
Jun 08 17:03:29.001 INFO Synced                                  slot: 9253515, block: 0xa49c…
3113, epoch: 289172, finalized_epoch: 289170, finalized_root: 0xca35…2b06, exec_hash: 0x5a14…
a7a5 (unverified), peers: 31, service: slot_notifier
```

これでチェーンの先端に向けて同期中のイーサリアムフルノードが稼働しています。同期にはかなりの時間がかかる場合があります（ハードウェアとインターネット接続速度に応じて数時間から数日）。

注記

この例で使用した具体的なコマンドやCLIフラグについて詳しく知りたい場合は、Rethと Lighthouseの公式ドキュメントが最適な情報源です。

次のセクションでは、イーサリアムブロックチェーンの初回同期の課題について説明します。

ヒント

これらの手順は複雑すぎてわかりにくいと感じましたか？それでもなお、信頼できる第三者に頼ることなく、自分のイーサリアムフルノードを立ち上げてネットワークに貢献したいと思いますか？

ぴったりの解決策があります。BuidlGuidl Clientというプロジェクトで、1行のコマンドでイーサリアムノードを実行できるようにしたものです。信じられませんか？ご自身でご確認ください。

もう1つの選択肢はDappnodeを使用することです。2つのソリューションから選択できます。

- イーサリアムフルノードが組み込まれたプラグアンドプレイデバイスを購入する。
- Dappnode Coreソフトウェアをインストールして、イーサリアムフルノードを非常に簡単に起動できるようにする。

イーサリアムベースのブロックチェーンの初回同期

通常イーサリアムブロックチェーンを同期する際、クライアントはジェネシスブロックから始めてすべてのブロックとトランザクションをダウンロードして検証します。この方法でブロックチェーンを完全に同期できますが、非常に長い時間がかかり大量のリソースが必要です（大量のRAMが必要になり、高速なストレージがなければ完了までさらに時間がかかります）。

2016年末にイーサリアムで発生したDoS攻撃の影響は、多くのイーサリアムベースのブロックチェーンにも及びました。影響を受けたブロックチェーンはフル同期を行うと同期が遅くなる傾向があります。たとえばイーサリアムでは新しいクライアントはブロック2,283,397まで急速に進みます。このブロックは2016年9月18日にマイニングされ、DoS攻撃の始まりを示しています。このブロックからブロック2,700,031（2016年11月26日）まで、トランザクションの検証が極端に遅くなり、メモリ集約型かつI/O集約型になります。2016年当時のハードウェアではブロックあたりの検証時間が1分を超えました。

イーサリアムはハードフォークを通じて一連のアップグレードを実施し、DoS攻撃で悪用された根本的な脆弱性に対処しました。これらのアップグレードによりスパムトランザクションで作成された約2,000万の空アカウントのクリーンアップも行われました。

完全な検証で同期する場合、クライアントの処理速度は低下し、DoS攻撃の影響を受けたブロックの検証には数日、場合によってはそれ以上かかることがあります。幸いほとんどのイーサリアムクライアントには、ブロックチェーンの先端への同期が完了するまでトランザクションの完全な検証をスキップし、チェーンの新しい先端から完全な検証を再開する「高速」同期オプションがあります。実行クライアントでは、高速同期を有効にするオプションは一般にsnap syncと呼ばれます。コンセンサスクライアントではcheckpoint syncです。

このチュートリアルでは実行クライアントではsnap sync、コンセンサスクライアントではcheckpoint syncを使用し、デフォルトで高速同期を行ってきました。ただしRethは例外で、2025年6月時点ではまだsnap syncをサポートしていません。

JSON-RPCインターフェース

イーサリアムクライアントはJSONでエンコードされたAPIとRPC（Remote Procedure Call、遠隔手続き呼び出し）コマンドのセットを提供します。これをJSON-RPC APIと呼びます。要するに、JSON-RPC APIは、イーサリアムクライアントを介してイーサリアムネットワークやブロックチェーンにアクセスするプログラムを書くためのインターフェースです。

通常RPCインターフェースはポート8545上のHTTPサービスとして提供されます。セキュリティ上の理由からデフォルトではlocalhost（自分のコンピュータのIPアドレスである127.0.0.1）からの接続のみを受け付けるよう制限されています。

JSON-RPC APIにアクセスするには、各RPCコマンドに対応する「スタブ」関数呼び出しを提供する専用ライブラリ（好みのプログラミング言語で書かれたもの）を使うか、手動でHTTPリクエストを構築してJSONエンコードされたリクエストを送受信します。curlのような汎用のコマンドラインHTTPクライアントでRPCインターフェースを呼び出すこともできます。試してみましょう。まず実行クライアントが設定され実行中であることを確認します。次に新しいターミナルウィンドウに切り替えて以下のコマンドを入力します。

```
$ curl -X POST -H "Content-Type: application/json" --data \
    '{"jsonrpc":"2.0","method":"web3_clientVersion","params":[],"id":1}' \
    http://localhost:8545

{"jsonrpc":"2.0","id":1,"result":"Geth/1.14.3-stable/darwin-arm64/go1.22.2"}
```

この例ではcurlを使って`http://localhost:8545`へのHTTP接続を行っています。すでに実行クライアントを起動しており、ポート8545上でJSON-RPC APIをHTTPサービスとして提供しています。curlにHTTP POSTメソッドを使うよう指示し、コンテンツタイプを**application/json**として指定しています。最後にHTTPリクエストのデータコンポーネントとしてJSONエンコードされ

たリクエストを渡しています。このコマンドラインの大部分は、curlがHTTP接続を正しく行うための設定にすぎません。注目すべきなのは実際に発行するJSON-RPCコマンドです。

```
{"jsonrpc":"2.0","method":"web3_clientVersion","params":[],"id":1}
```

JSON-RPCリクエストはJSON-RPC 2.0仕様に従ってフォーマットされます。各リクエストには4つの要素が含まれます。

jsonrpc

JSON-RPCプロトコルのバージョン。正確に「2.0」でなければならない。

method

呼び出すメソッドの名前。

params

メソッドの呼び出し時に使用されるパラメーター値を保持する構造化された値。このメンバーは省略可能である。

id

クライアントが設定する識別子であり、含まれる場合は文字列、数値、またはNULL値でなければならない。含まれる場合サーバーはレスポンスオブジェクトで同じ値を返さなければならない。このメンバーは2つのオブジェクト間のコンテクストを関連付けるために使用する。

> **ヒント**
>
> idパラメーターは主に単一のJSON-RPC呼び出しで複数のリクエストを行う場合に使います。この方法はバッチングと呼ばれます。バッチングはリクエストごとに新しいHTTPおよびTCP接続を行うオーバーヘッドを避けるために使います。イーサリアムではたとえば1つのHTTP接続で数千のトランザクションを取得したい場合にバッチングを使います。バッチングでは各リクエストに異なるidを設定し、JSON-RPCサーバーからのレスポンスのidと照合します。最も簡単な実装方法は、カウンターを用意してリクエストごとにインクリメントすることです。

受信するレスポンスは以下のとおりです。

```
{"jsonrpc":"2.0","id":1,"result":"Geth/1.14.3-stable/darwin-arm64/go1.22.2"}
```

これは、JSON-RPC APIがGethクライアントバージョン1.14.3-stableによって提供されていることを示しています。

もう少し面白いことを試してみましょう。次の例では、JSON-RPC APIに現在のガス価格をweiで問い合わせます。

```
$ curl -X POST -H "Content-Type: application/json" --data \
    '{"jsonrpc":"2.0","method":"eth_gasPrice","params":[],"id":4213}' \
    http://localhost:8545

{"jsonrpc":"2.0","id":4213,"result":"0x1B1717FC7"}
```

レスポンスの0x1B1717FC7は現在のガス価格が7.27 gwei（gigaweiまたは10億wei）であることを示しています。16進数を10進数に変換したい場合は、コマンドライン上でBashのテクニックを使えます。

```
$ echo $((0x1B1717FC7))7271972807
```

完全なJSON-RPC APIはイーサリアム wikiで調べることができます。

ヒント

このセクションではイーサリアムのJSON-RPCインターフェースを示すために生のcurlリクエストを使いました。実際にはより優れたプログラマティックな方法でアクセスしたいと思うでしょう。ここでライブラリの出番です。最も有名で広く使われている3つのライブラリを自由に探索してみてください。

- ethers.js
- web3.py
- alloy

リモートイーサリアムクライアント

リモートクライアントはフルクライアントの機能のサブセットを提供します。完全なイーサリアムブロックチェーンを保存しないため、セットアップが速く必要なデータストレージもはるかに少なくて済みます。

これらのクライアントは通常、以下のような機能を提供します。

- ウォレット内で秘密鍵とイーサリアムアドレスを管理する
- トランザクションの作成、署名、ブロードキャストを行う
- データペイロードを使用してスマートコントラクトとやり取りする
- DAppを閲覧してやり取りする

- ブロックエクスプローラーなどの外部サービスへのリンクを提供する
- etherの単位変換と外部ソースからの為替レートの取得を行う
- Web3インスタンスをJavaScriptオブジェクトとしてウェブブラウザに注入する
- 別のクライアントによって提供または注入されたWeb3インスタンスを使用する
- ローカルまたはリモートのイーサリアムノード上のRPCサービスにアクセスする

リモートクライアントは通常、ローカルのマシン上やウェブサーバー上、あるいは第三者のサーバー上で動作しているフルノードに接続します。これによりイーサリアムブロックチェーンのローカルコピーを同期せずに、フルノードイーサリアムクライアントの一部の機能を提供します。

最も人気のあるリモートクライアントと、提供される機能について見てみましょう。

モバイル（スマートフォン）ウォレット

ほとんどの本番環境向けモバイルウォレットは、スマートフォンにはフルのイーサリアムクライアントを実行するのに十分なリソースがないため、リモートクライアントとして動作します。軽量クライアントは開発中ですがイーサリアムではまだ一般的に使われていません。最も有名なものはHeliosで、まだ実験的なソフトウェアです。

人気のあるモバイルウォレットには以下のものがあります（これらは単なる例であり、セキュリティや機能を推奨・保証するものではありません）。

Coinbase Wallet

イーサリアム（およびすべてのL2）、EVM互換L1、ビットコイン、Solana、Litecoin、Dogecoinなど、多数のチェーンをサポートするモバイルウォレットです。Coinbaseアカウントとの接続も可能です。

Phantom

イーサリアム、Solana、ビットコイン、Polygonに対応するもう1つのマルチチェーンウォレットです。

Trust Wallet

100を超えるブロックチェーンをサポートするモバイルマルチチェーンウォレットです。Trust WalletはiOSとAndroidで利用可能です。

Uniswap Wallet

イーサリアムとEVM互換のL2およびL1のみをサポートするモバイルウォレットです。Uniswapチームによって開発されました。比較的新しく、iOSとAndroidの両方で利用可能です。

ブラウザウォレット

ChromeやFirefoxなどのウェブブラウザのプラグインまたは拡張機能として、さまざまなウォレットやDAppブラウザが利用できます。これらはブラウザ内で動作するリモートクライアントです。人気のあるものには以下があります。

MetaMask

MetaMaskは第2章で紹介した、多機能なブラウザベースのウォレット、RPCクライアント、および基本的なコントラクトエクスプローラーです。Chrome、Firefox、Opera、Brave Browserで利用可能です。

Phantom

Phantomにはウェブブラウザウォレットもあり、非常に洗練されたきれいなUIを備えています。

Rabby Wallet

Rabbyは100を超える異なるブロックチェーン（EVM互換チェーン）をサポートする新しいマルチチェーンウェブブラウザウォレットです。

Coinbase Wallet

Coinbase Walletにはウェブブラウザウォレットもあります。モバイル版と同じ機能を備えています。

ハードウェアウォレット

大半のモバイルおよびブラウザウォレットはハードウェアウォレットの高いセキュリティと組み合わせることができます。ハードウェアウォレットはインターネットに接続しないよう設計されたオフラインデバイスで、改ざんやその他の物理的攻撃に耐えるように作られており、より高いレベルのセキュリティを提供します。このようなデバイスを製造する企業は複数ありますが、最も広く使われているのはLedgerとTrezorの2社です。

まとめ

この章ではイーサリアムクライアントについて理解を深めました。クライアントのダウンロード、インストール、同期を行い、イーサリアムネットワークの参加者となりました。自分のコンピュータにブロックチェーンを複製することで、システムの健全性と安定性に貢献できます。

イーサリアムに関する研究開発は非常に盛んであり、将来的に新しいタイプのイーサリアムクライアントが登場するでしょう。興味深い分野には以下のものがあります。

履歴のプルーニング

　フルノードのストレージ要件を低減するために過去のデータを削除する

Verkleツリーとステートレス性

　イーサリアムの完全なステートを保持することなくブロックを検証できるようにする

zk-EVM

　ブロック内のすべてのトランザクションを再実行することなく、ゼロ知識証明を検証することでブロックの正しさを検証する

　これらの概念は以降の章で探っていきますが、まずはこのすべてを可能にする真の魔法、すなわち暗号学について明らかにする必要があります。

第4章 暗号技術

イーサリアムの基盤技術のひとつが*暗号技術*です。暗号技術はコンピュータセキュリティで広く使われる数学の一分野です。ギリシャ語で「秘密の書き物」を意味しますが、研究対象は*暗号化*と呼ばれる秘密の書き物だけにとどまりません。たとえば、秘密を明かさずにその秘密の知識を証明したり（*デジタル署名*など）、データの真正性を証明したり（デジタルフィンガープリント、いわゆるハッシュなど）する用途にも使えます。こうした暗号学的証明は、イーサリアムプラットフォーム（実際にはすべてのブロックチェーンシステム）の運用に不可欠な数学的ツールであり、イーサリアムアプリケーションでも広く利用されています。

なお、本書の出版時点では、イーサリアムプロトコルに暗号化は含まれていません。つまり、イーサリアムプラットフォームとの通信やノード間の通信（トランザクションデータを含む）はすべて暗号化されておらず、誰でも（必然的に）読めます。これは、すべての人が状態更新の正しさを検証し、コンセンサスに到達できるようにするためです。将来的にはゼロ知識証明や準同型暗号などの高度な暗号ツールが利用可能になり、一部の暗号化された計算をブロックチェーン上に記録しつつコンセンサスを可能にするでしょう。こうした暗号ツールの基礎技術は研究段階で整いつつありますが、イーサリアムプロトコルへの本格的な組み込みはまだ進んでいません。

この章では、イーサリアムで使われる暗号技術のうち、*公開鍵暗号*（PKC）を紹介します。公開鍵暗号は、秘密鍵とアドレスという形で資金の所有権を管理するために使われています。

鍵とアドレス

本書の前の章で学んだように、イーサリアムにはEOAとコントラクトの2種類のアカウントがあります。EOAによるEtherの所有権は、デジタル*秘密鍵*、イーサリアム*アドレス*、デジタル*署名*によって確立されます。秘密鍵はユーザーがイーサリアムと対話するすべての操作の中核にあり、アカウントアドレスは秘密鍵から直接導出されます。ひとつの秘密鍵が、アカウントとも呼ばれる単一のイーサリアムアドレスを一意に決定します。

秘密鍵はイーサリアムシステムで直接使われることはありません。ネットワークに送信されるメッセージに含まれることも、チェーン上に保存されることもありません。秘密鍵は秘密のまま保たれるべきであり、イーサリアムシステム上で送信・保存されるのはアカウントアドレスとデジタル署名のみです。秘密鍵の安全な保管方法については「ウォレットのベストプラクティス」を参照してください。

ユーザーは秘密鍵を使ってデジタル署名を作成し、この署名によって資金へのアクセスと管理を行います。イーサリアムのトランザクションがブロックチェーンに含まれるには、有効なデジタル署名が必要です。秘密鍵のコピーを持つ人は誰でも、対応するアカウントとそのアカウントが保有するEtherを管理できます。ユーザーが秘密鍵を安全に保管している限り、イーサリアム

のトランザクションにおけるデジタル署名は秘密鍵の所有権を証明し、ひいては資金の真の所有者を証明します。

イーサリアムが使用するPKCベースのシステムでは、鍵は秘密鍵（シークレットキー）と公開鍵のペアで構成されます。公開鍵は銀行の口座番号、秘密鍵は秘密のPINに似たものと考えてください。アカウントを管理するのが秘密鍵で、他者に対してアカウントを識別するのが公開鍵です。秘密鍵自体はイーサリアムユーザーの目に触れることはほとんどなく、通常は暗号化された形式で特別なファイルに保存され、イーサリアムウォレットソフトウェアが管理します。

イーサリアムトランザクションの支払い部分では、受取人はイーサリアムアドレスで表されます。イーサリアムアドレスは銀行振込の受取人口座情報と同じように使われます。後ほど詳しく見るように、EOAのイーサリアムアドレスは鍵ペアの公開鍵部分から生成されます。ただし、すべてのイーサリアムアドレスが公開鍵と秘密鍵のペアを表すわけではありません。コントラクトを表すアドレスもあり、第7章で見るようにコントラクトは秘密鍵に裏付けられていません。

この章の残りでは、以下の内容を取り上げます。

- 暗号技術の基礎とイーサリアムにおける数学的基盤を深く掘り下げる
- 鍵の生成、保存、管理のプロセスを検討する
- 秘密鍵、公開鍵、アドレスに使われるさまざまなエンコーディング形式を確認する
- バリデータ鍵の暗号技術とKZGコミットメントスキームを調査する。これらはイーサリアムの暗号基盤に対する最新の更新である

PKCと暗号通貨

PKC（*非対称暗号*とも呼ばれる）は、現代の情報セキュリティの中核をなす技術です。1970年代にマーティン・ヘルマン、ウィットフィールド・ディフィー、ラルフ・マークルが最初に発表した*鍵交換プロトコル*は、暗号技術分野への一般の関心を大きく喚起した画期的な発明でした。1970年代以前、強力な暗号技術の知識は各国政府が機密情報として厳重に管理していました。

PKCは固有の鍵を使って情報を保護します。この鍵は、順方向の計算は容易でも逆関数の計算は困難であるという特別な性質を持つ数学関数に基づいています。PKCはこの性質を利用することで、デジタルの秘密と偽造不可能なデジタル署名を作成できます。その安全性は数学の法則によって保証されます。

たとえば、2つの大きな素数を掛け合わせることは簡単です。しかし、2つの大きな素数の積が与えられたとき、素因数を見つけることは非常に困難です（この問題は*素因数分解*と呼ばれます）。8,018,009という数を示し、これが2つの素数の積であると伝えたとしましょう。この2つの素数を見つけることは、掛け合わせて8,018,009を求めるよりもはるかに難しいでしょう。

こうした数学関数の中には、ある秘密の情報を知っていれば容易に逆算できるものがあります。先ほどの例で素因数のひとつが2,003であると教えれば、もうひとつは単純な割り算で簡単

に見つかります。8,018,009 ÷ 2,003 = 4,003です。このような関数は*トラップドア関数*と呼ばれます。逆算のショートカットとなる秘密情報がなければ、逆算が非常に困難だからです。

暗号技術で役立つより高度な数学関数は、*楕円曲線*上の算術演算に基づいています。楕円曲線の算術では、素数を法とする乗算は単純ですが、除算（逆演算）は実質的に不可能です。これは*離散対数問題*と呼ばれ、現在のところ既知のトラップドアは存在しません。この性質を応用した*楕円曲線暗号*は現代のコンピュータシステムで広く使われており、イーサリアムをはじめとする暗号通貨における秘密鍵とデジタル署名の基盤となっています。

注記

暗号技術と現代の暗号技術で使われる数学関数についてさらに読みたい場合は、以下のリソースを参照してください。

- Cryptography
- Trapdoor function
- Prime factorization
- Discrete logarithm
- Elliptic curve cryptography

イーサリアムでは、PKCを使ってこの章で説明してきた公開鍵と秘密鍵のペアを作成します。公開鍵は秘密鍵から導出されるため「ペア」と見なされます。鍵ペアはイーサリアムアカウントを表します。公開鍵は誰でもアクセスできるアカウントハンドル（アドレス）を提供し、秘密鍵はアカウント内のEtherへのアクセスやスマートコントラクト使用時の認証を秘密裏に制御します。秘密鍵はデジタル署名の作成に必要な唯一の情報であり、これがアクセスを制御します。デジタル署名はアカウント内の資金を使うトランザクションの署名に必要であり、第7章で見るようにコントラクトの所有者やユーザーの認証にも使われます。

ヒント

ほとんどのウォレット実装では、秘密鍵と公開鍵は利便性のために鍵ペアとして一緒に保存される。ただし、公開鍵は秘密鍵から容易に計算できるため、秘密鍵のみを保存することも可能である。

デジタル署名はあらゆるメッセージに対して作成できます。イーサリアムのトランザクションでは、トランザクション自体の詳細がメッセージとして使われます。暗号技術の数学（この場合は楕円曲線暗号）は、メッセージ（つまりトランザクションの詳細）を秘密鍵と組み合わせて、秘密鍵を知らなければ生成できないコードを作成する方法を提供します。そのコードがデジタル署名です。

イーサリアムのトランザクションは、特定のイーサリアムアドレスを持つアカウントへのアクセス要求です。資金を移動したりスマートコントラクトと対話したりするためにイーサリアムネ

ットワークへトランザクションを送信する場合、そのイーサリアムアドレスに対応する秘密鍵で作成したデジタル署名と一緒に送信する必要があります。誰でもこのデジタル署名を、トランザクションの詳細やアクセス対象のイーサリアムアドレスと照合することで、楕円曲線の数学にもとづいてトランザクションの有効性を検証できます。この検証プロセスでは秘密鍵がまったく関与しないため、秘密鍵は秘密のまま保たれます。それでも検証プロセスは、そのトランザクションがイーサリアムアドレスの背後にある公開鍵に対応する秘密鍵を持つ人からしか送信できなかったことを疑いなく判定します。これがPKCの「マジック」です。

ヒント

イーサリアムプロトコルの一部として暗号化は行われない。イーサリアムネットワークの運用の一環として送信されるすべてのメッセージは、（必然的に）誰でも読むことができる。したがって、秘密鍵はトランザクション認証のためのデジタル署名の作成にのみ使われる。

秘密鍵

秘密鍵はランダムに選ばれた数値です。秘密鍵の所有と管理は、ユーザーによるアカウント制御の根幹です。これにより、対応するイーサリアムアドレスに関連するすべての資金を管理し、そのアドレスを認可するコントラクトにアクセスできます。秘密鍵はトランザクションで使う資金の所有権を証明し、Etherの使用に必要な署名を作成するために使われます。秘密鍵は常に秘密にしておかなければなりません。第三者に秘密鍵を明かすことは、その秘密鍵で保護されているEtherとコントラクトの管理権を渡すことと同義です。また、秘密鍵はバックアップを取り、偶発的な紛失から保護する必要もあります。秘密鍵を紛失した場合は回復できず、その秘密鍵で保護されていた資金も永久に失われます。

ヒント

イーサリアムの秘密鍵は単なる数値である。秘密鍵をランダムに選ぶひとつの方法は、コイン、鉛筆、紙を使うことである。コインを256回投げれば、イーサリアムウォレットで使えるランダムな秘密鍵の2進数が得られる（おそらく――以下の段落を参照）。公開鍵とアドレスはその秘密鍵から生成できる。

鍵を生成する最初で最も重要なステップは、安全な*エントロピー*（ランダム性）の源を見つけることです。イーサリアムの秘密鍵の作成は、本質的に1から2^256の間の数値を選ぶことです。予測可能でも決定論的でもなければ、その数値を選ぶ方法は何でも構いません。イーサリアムソフトウェアは、基盤となるOSの*乱数生成器*（RNG）を使って256ビットのランダムビットを生成します。通常、OSのRNGは人間由来のランダム性で初期化されるため、マウスを数秒間動かしたり、キーボードでランダムなキーを押したりするよう求められることがあります。別の方

法として、コンピュータのマイク入力から得られる環境ノイズ（宇宙放射線の影響を含む）を利用することもできます。

より正確に言えば、秘密鍵は1以上かつ「2^256よりわずかに小さい非常に大きな数」以下の任意の整数です。この数は78桁の巨大な数で、約1.158 × 10^77です。この正確な数値は2^256と最初の38桁を共有しており、イーサリアムで使用される楕円曲線の位数として定義されています。秘密鍵を作成するには、256ビットの数値をランダムに選び、有効な範囲内にあるか確認します。プログラミングでは通常、暗号学的に安全なランダム性のソースから収集したさらに大きなランダムビット列を、Keccak-256やSHA-256などの256ビットハッシュアルゴリズムに入力します。これらのアルゴリズムはちょうど256ビットの数値を生成します。結果が有効範囲内であれば適切な秘密鍵が得られ、そうでなければ別のランダムな数値で再試行します。

注記

イーサリアムの秘密鍵空間の大きさ——2^256——は、想像を絶するほど大きな数である。10進数では約10^77——つまり78桁の数である。比較のために、可視宇宙に存在する原子の数は10^80と推定されている。したがって、宇宙のすべての原子にイーサリアムアカウントを割り当てられるほどの秘密鍵が存在する。秘密鍵をランダムに選べば、誰かがそれを推測したり同じものを選んだりすることは考えられない。

秘密鍵の生成プロセスはオフラインで行われます。イーサリアムネットワークとの通信は一切不要で、実際のところ誰とも通信する必要はありません。そのため、他の誰も選ばない数値を選ぶには本当にランダムでなければなりません。自分で数値を選んだ場合、誰かが同じ数値を試す（そしてEtherを持ち去る）可能性が極めて高くなります。質の悪いRNG（ほとんどのプログラミング言語の擬似ランダムな`rand`関数など）を使うのはさらに危険で、出力パターンが予測されやすく再現も容易だからです。オンラインアカウントのパスワードと同様に、秘密鍵は推測不可能でなければなりません。幸い、秘密鍵を人間が暗記する必要はまったくないため、最善のアプローチである真のランダム性をそのまま採用できます。

警告

自分でランダムな数値を生成するコードを書いたり、プログラミング言語が提供する「単純な」RNGを使ったりしないこと。ブラウザウォレットにおけるJavaScriptベースのRNGは、OSのエントロピーに裏付けられていない限り安全でない場合がある。暗号学的に安全な擬似乱数生成器（CSPRNGなど）を、十分なエントロピーのソースからのシードとともに使用することが不可欠である。選択したRNGライブラリのドキュメントを調べて、暗号学的に安全であることを確認すること。CSPRNGライブラリの正しい実装は、鍵のセキュリティにとって極めて重要である。

以下は、ランダムに生成された秘密鍵を16進数形式で示したものです（256ビットを64桁の16進数で表示、各桁は4ビット）。

```
f8f8a2f43c8376ccb0871305060d7b27b0554d2cc72bccf41b2705608452f315
```

公開鍵

イーサリアムの公開鍵は楕円曲線上の点で、楕円曲線方程式を満たすxとyの座標の組です。

より簡単に言えば、イーサリアムの公開鍵は2つの数値を結合したものです。これらの数値は秘密鍵から一方向にしか計算できない演算によって生成されます。つまり、秘密鍵があれば公開鍵の計算は簡単ですが、公開鍵から秘密鍵を逆算することはできません。

> **警告**
>
> これから数学の話が始まる。慌てる必要はない。以下の段落で迷子になったら、次のいくつかのセクションを飛ばしても構わない。計算を代わりにやってくれるツールやライブラリがたくさんある。

公開鍵は*楕円曲線乗算*を使って秘密鍵から計算されます。この演算は実質的に不可逆です。`K = k * G`において、`k`は秘密鍵、`G`は*生成点*と呼ばれる定数の点、`K`は結果の公開鍵、`*`は特別な楕円曲線の「乗算」演算子です。楕円曲線乗算は通常の乗算とは異なり、機能的な属性は共有していますがそれだけです。たとえば逆演算（通常の数値では除算に当たるもの）は「離散対数を見つける」ことに相当します。つまり`K`がわかっている場合に`k`を計算する操作です。この計算は`k`のすべての可能な値を試す総当たり探索と同じくらい困難であり、この宇宙が許容する以上の時間がかかります。

より簡単に言えば、楕円曲線上の算術は「通常の」整数算術とは異なります。点（`G`）を整数（`k`）で乗算して別の点（`K`）を得ることができます。しかし除算は存在しないため、公開鍵`K`を点`G`で単純に「割って」秘密鍵`k`を計算することは不可能です。これは、前のセクション「PKCと暗号通貨」で説明した一方向性の数学関数です。

> **注記**
>
> 楕円曲線乗算は、暗号技術者が「一方向」関数と呼ぶタイプの関数である。一方向（乗算）の計算は容易だが、逆方向（除算）の計算は不可能である。秘密鍵の所有者は公開鍵を容易に作成して世界中と共有でき、誰も公開鍵から秘密鍵を逆算できないことを確信できる。この数学的トリックが、イーサリアムの資金の所有権とコントラクトの管理を証明する、偽造不可能で安全なデジタル署名の基盤となる。

秘密鍵から公開鍵を生成する方法を示す前に、楕円曲線暗号をもう少し詳しく見てみましょう。

楕円曲線暗号の解説

楕円曲線暗号は、楕円曲線上の点の加算と乗算で表される離散対数問題に基づく非対称暗号（公開鍵暗号）の一種です。図4-1は、イーサリアムで使用されるものに似た楕円曲線の例です。

> **注記**
>
> イーサリアムはビットコインとまったく同じ楕円曲線 `secp256k1` を使用している。これにより、ビットコインの楕円曲線ライブラリやツールの多くを再利用できる。

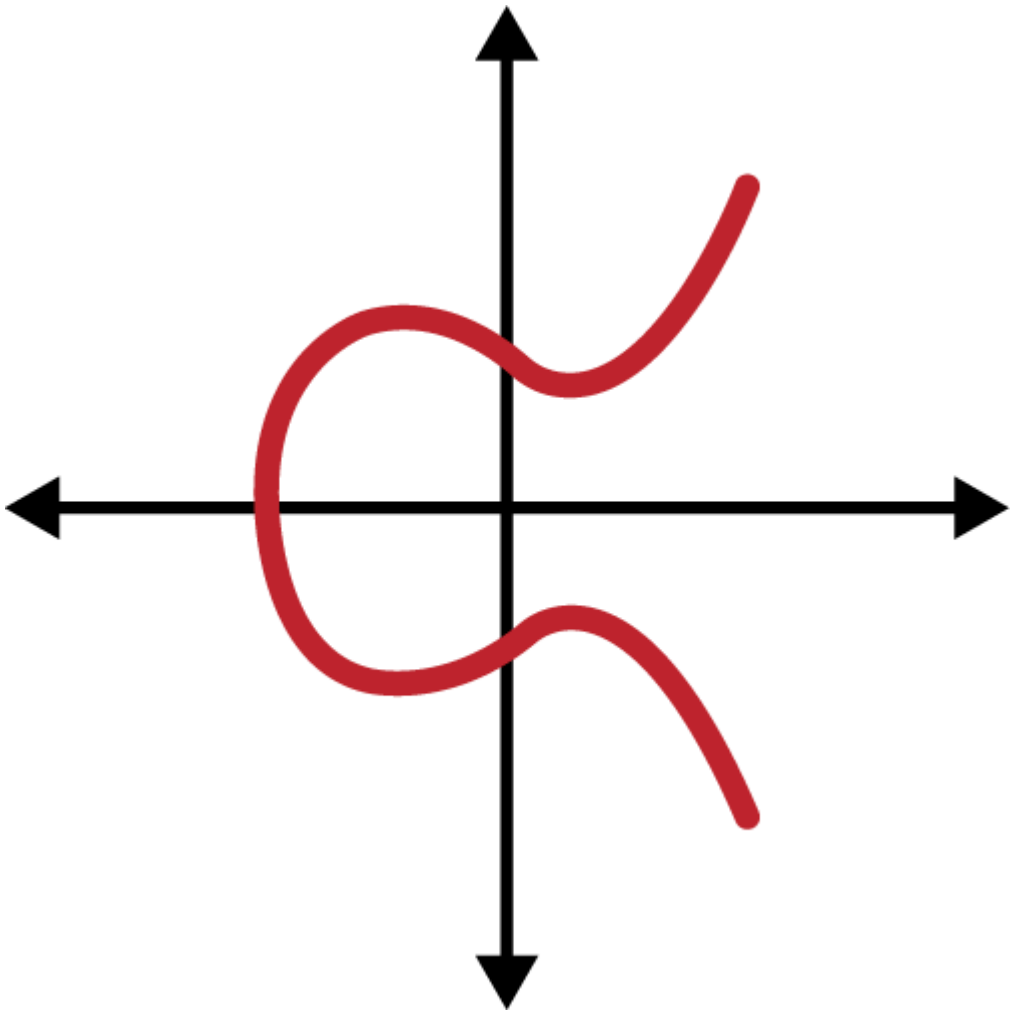


図4-1 楕円曲線

イーサリアムは、米国国立標準技術研究所（NIST）が確立した`secp256k1`という標準で定義された特定の楕円曲線と数学定数のセットを使用します。`secp256k1`曲線は、以下の関数で定義されます。

```
y^2 = (x^3 + 7) over (Fp)
```

または：

```
y^2 mod p = (x^3 + 7) mod p
```

`mod p`（素数pを法とする）は、この曲線がFpとも書かれる素数位数pの*有限体*上に定義されていることを示します。ここでp = 2^256 – 2^32 – 2^9 – 2^8 – 2^7 – 2^6 – 2^4 – 1であり、非常に大きな素数です。

この曲線は実数上ではなく素数位数の有限体上に定義されているため、2次元に散らばった点のパターンのように見え、視覚化が困難です。しかし数学は実数上の楕円曲線と同一です。例として図4-2は、同じ楕円曲線をはるかに小さな素数位数17の有限体上で示したもので、グリッド上の点のパターンを表しています。`secp256k1`のイーサリアム楕円曲線は、想像を絶するほど大きなグリッド上のはるかに複雑な点のパターンと考えられます。

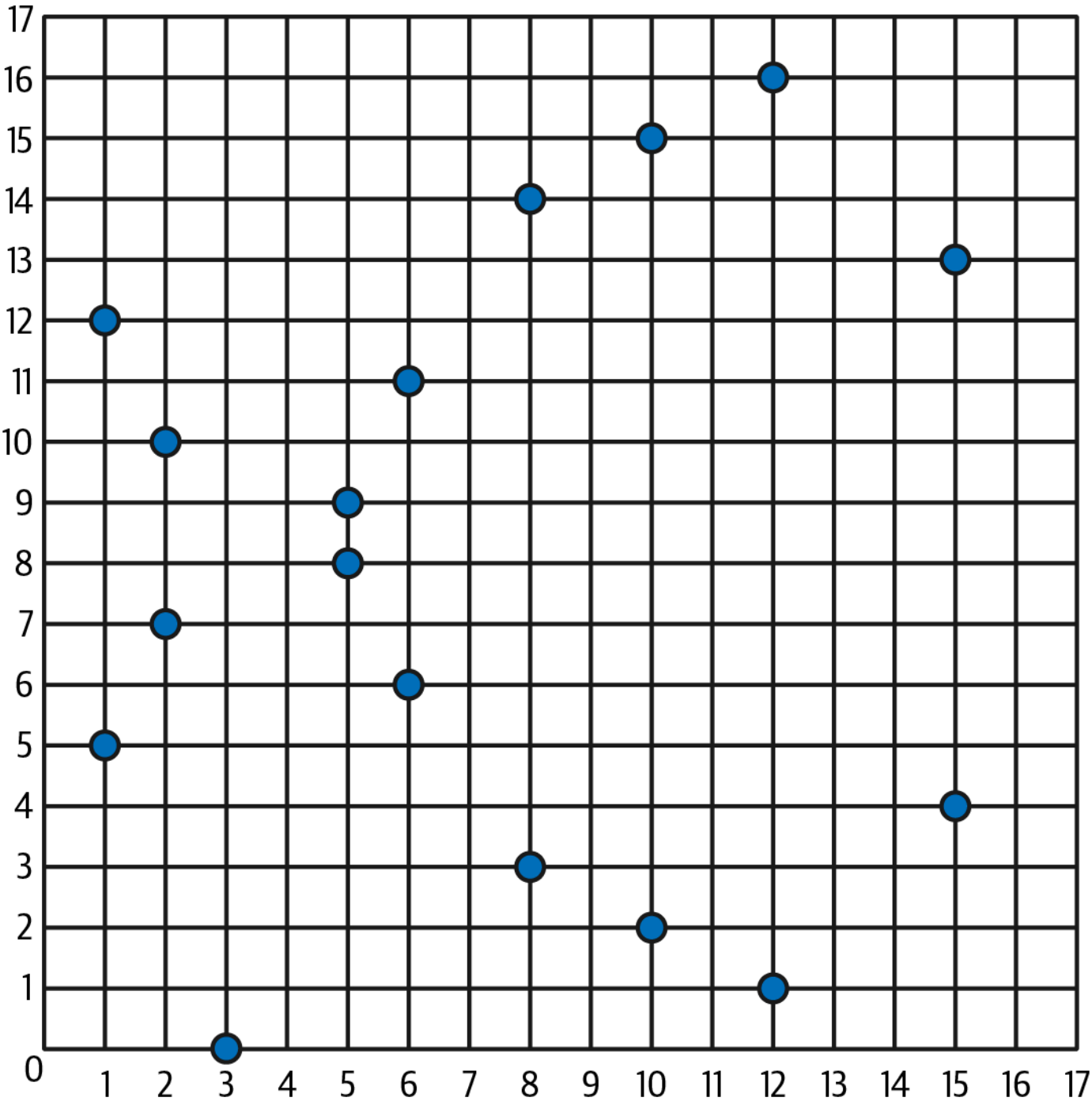


図4-2 楕円曲線暗号：F(p)上の楕円曲線の可視化（p=17）

たとえば、以下は座標(x, y)を持つ点Qで、`secp256k1`曲線上の点です。

```
Q = (49790390825249384486033144355916864607616083520101638681403973749255924539515,
595741321618999000458620864939210157800321752917558073992840077210503412973 60)
```

例4-1は、Pythonを使ってこれを自分で確認する方法を示しています。変数`x`と`y`は前述の例の点Qの座標です。変数`p`は楕円曲線の素数位数（すべての剰余演算に使われる素数）です。Pythonの最後の行は楕円曲線方程式です（Pythonの`%`演算子は剰余演算子です）。`x`と`y`が確かに楕円曲線上の点の座標であれば、方程式を満たし、結果はゼロになります。コマンドラインで`python`（または`python3`）と入力し、リストの各行（プロンプト`>>>`の後）をコピーして試してみてください。

例4-1 Pythonを使ってこの点が楕円曲線上にあることを確認する

```
Python 3.12.4 (main, Jun  6 2024, 18:26:44) [Clang 15.0.0 (clang-1500.3.9.4)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> p = 115792089237316195423570985008687907853269984665640564039457584007908834671663
>>> x = 49790390825249384486033144355916864607616083520101638681403973749255924539515
>>> y = 595741321618999000458620864939210157800321752917558073992840077210503412973 60
>>> (x ** 3 + 7 - y**2) % p
0
```

楕円曲線の算術演算

楕円曲線の数学の多くは、学校で習う整数算術と表記も振る舞いもよく似ています。具体的には、数直線上をジャンプする代わりに曲線上の他の点にジャンプする加算演算子を定義できます。加算演算子を定義すれば、点と整数の乗算も定義でき、これは加算の繰り返しに相当します。

楕円曲線の加算は、楕円曲線上の2点P1とP2が与えられたとき、同じく楕円曲線上にある第3の点P3 = P1 + P2が存在するように定義されます。

幾何学的には、第3の点P3はP1とP2を通る直線を引くことで求められます。驚くべきことに、この直線は楕円曲線と別の1点でちょうど交わります。この点をP3′ = (x, y)と呼び、x軸に関して反転させてP3 = (x, –y)を得ます。図4-3に示す通りです。

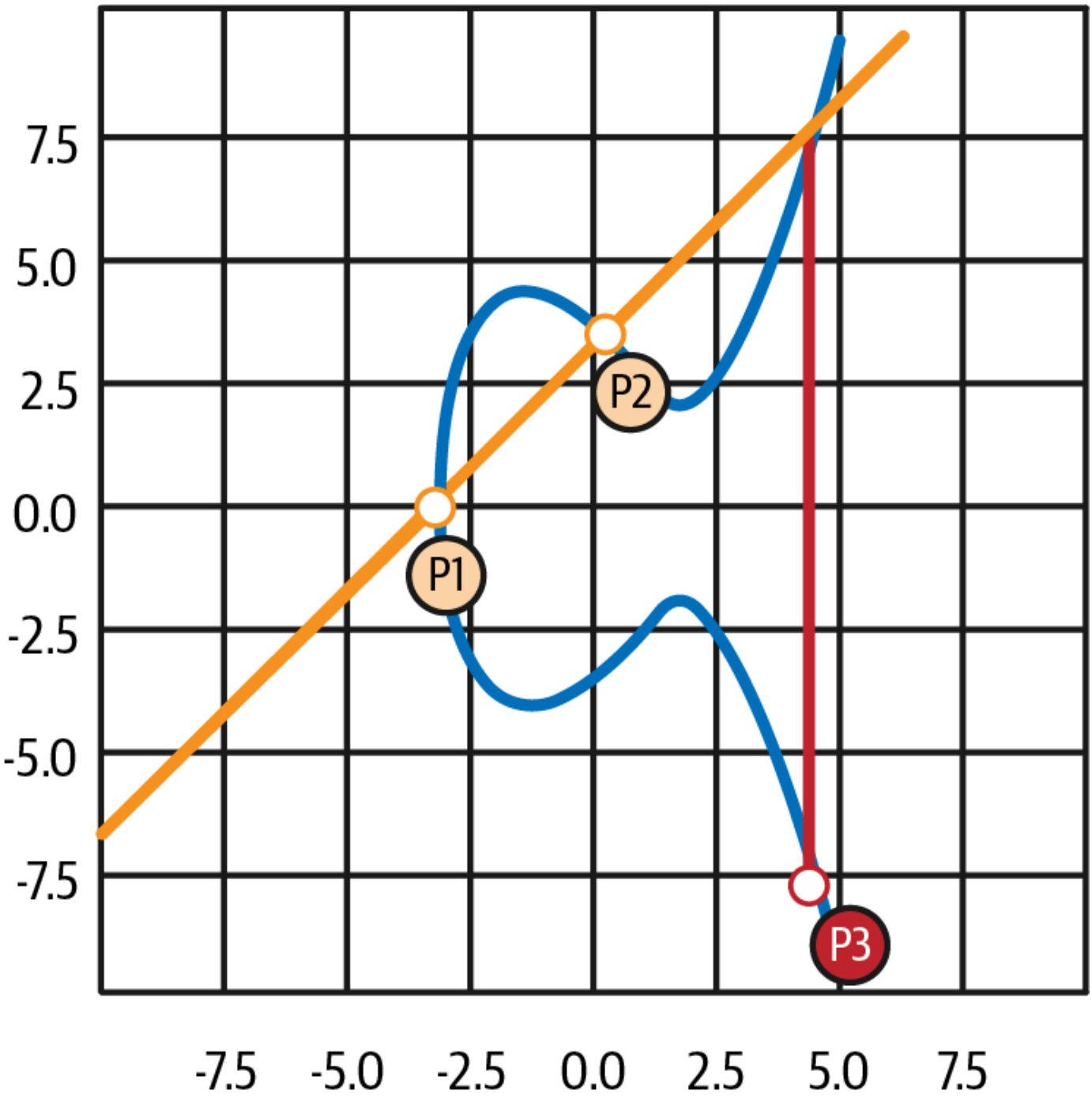


図4-3 楕円曲線の加算：楕円曲線上の2点を加算する

　P1とP2が同じ点の場合は、P1とP2の「間」の直線の代わりに、P1における接線を引きます。この接線は、図4-4に示すように曲線と別の1点でちょうど交わります。微積分のテクニックを使えば接線の傾きを求められます。興味深いことに、曲線上の整数座標を持つ点のみに関心を限定しても、このテクニックは機能します。

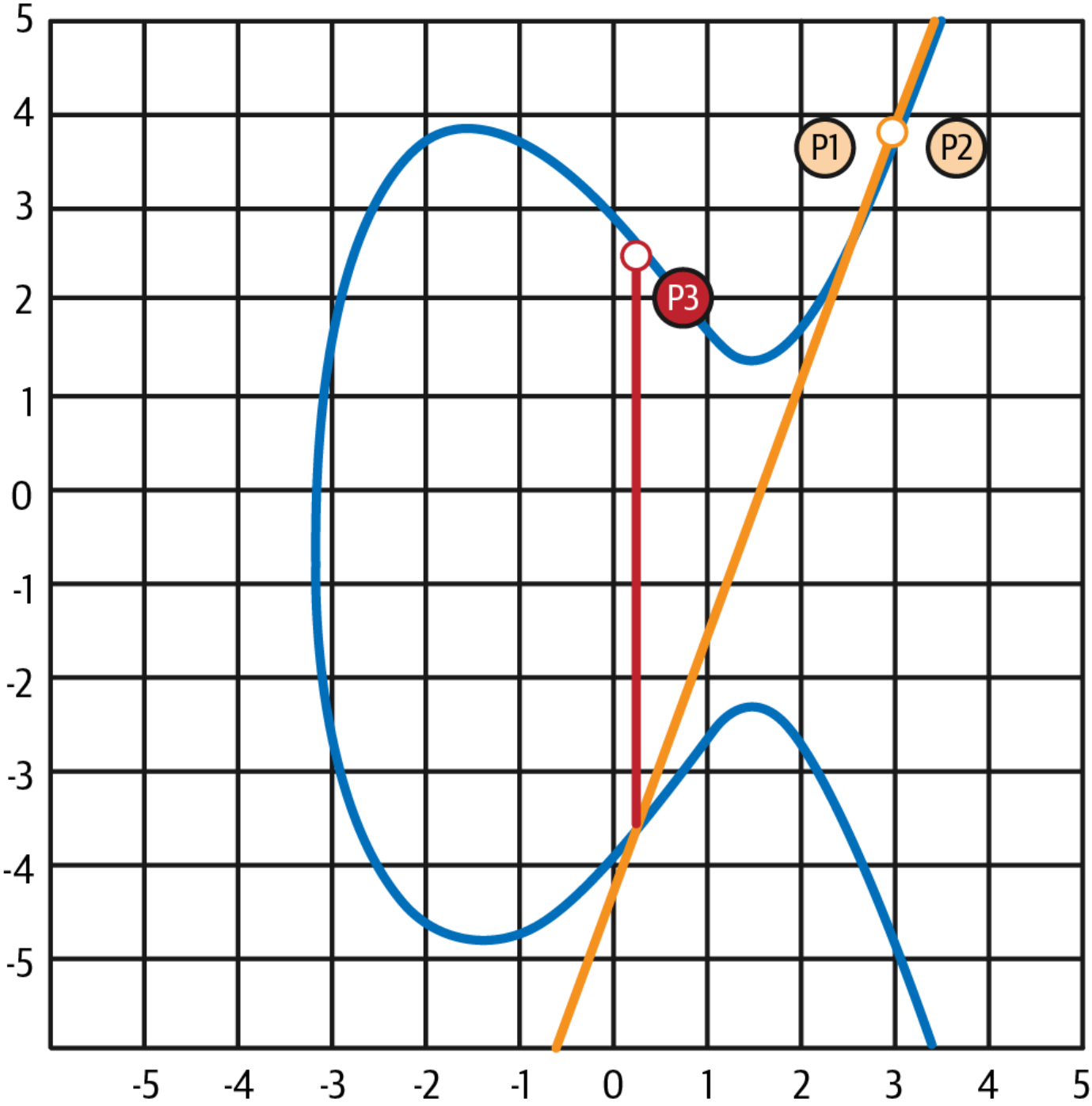


図4-4 楕円曲線の加算：点をそれ自体に加算する

楕円曲線の数学には、加算における数値0の役割にほぼ対応する*無限遠点*と呼ばれる点も存在します。コンピュータ上では`x = y = 0`で表されることがあります（これは楕円曲線方程式を満たしませんが、容易にチェックできる別のケースです）。無限遠点が必要となる特殊なケースがいくつかあります。

場合によっては（たとえば図4-5のようにP1とP2が同じx値で異なるy値を持つ場合）、直線は完全に垂直になり、P3＝無限遠点となります。

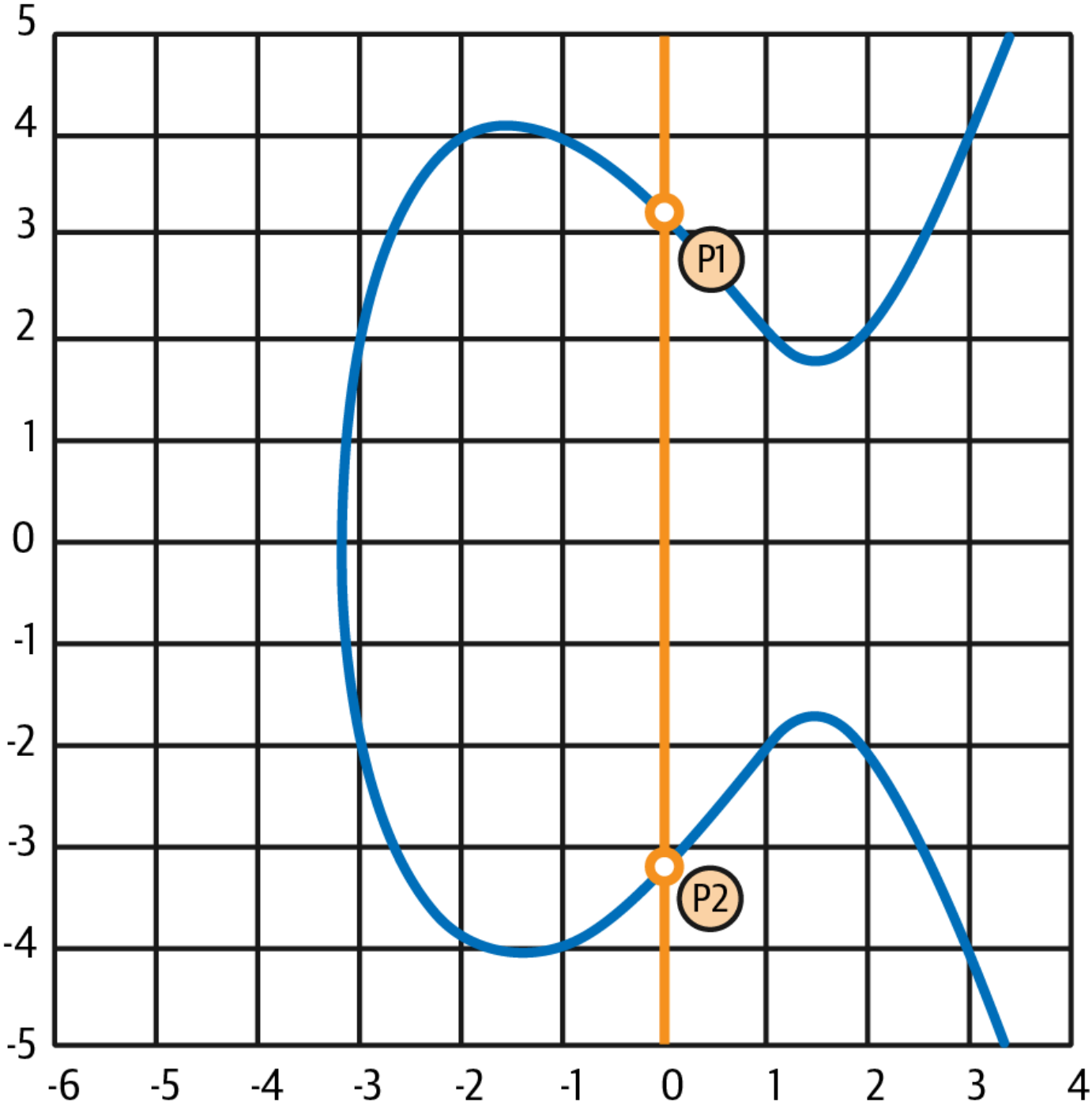


図4-5 楕円曲線の加算：無限遠点が結果となる特殊なケース

P1が無限遠点の場合、P1 + P2 = P2となります。同様に、P2が無限遠点の場合、P1 + P2 = P1となります。これは、無限遠点が「通常の」算術でゼロが果たす役割を担うことを示しています。

`+`は結合法則を満たすことがわかっています。つまり`(A + B) + C = A + (B + C)`です。これは`A + B + C`と（括弧なしで）曖昧さなく書けることを意味します。

加算を定義したので、加算の拡張としての標準的な方法で乗算を定義できます。楕円曲線上の点Pに対して、kが整数であれば`k * P = P + P + P + … + P`（k回）です。この場合、kは（紛らわしいかもしれませんが）指数と呼ばれることがあります。

公開鍵の生成

ランダムに生成された数値`k`を秘密鍵とします。これに、曲線上の所定の点である生成点`G`を乗算すると、曲線上の別の点が得られます。これが対応する公開鍵`K`です。

```
K = k * G
```

生成点は`secp256k1`標準の一部として指定されており、すべての実装で同じです。この曲線から導出されるすべての鍵は同じ点`G`を使います。生成点はすべてのイーサリアムユーザーで常に同じであるため、秘密鍵`k`に`G`を乗算すると常に同じ公開鍵`K`になります。`k`と`K`の関係は固定されていますが、`k`から`K`への一方向にしか計算できません。そのためイーサリアムアドレス（`K`から導出される）は誰とでも共有でき、ユーザーの秘密鍵（`k`）が明かされることはありません。

前のセクションで説明したように、`k * G`の乗算は加算の繰り返しに相当するため、`G + G + G + ⋯ + G`をk回繰り返すことになります。要約すると、秘密鍵`k`から公開鍵`K`を生成するには、生成点`G`をk回自分自身に加算します。

> **ヒント**
>
> 秘密鍵は公開鍵に変換できるが、公開鍵を秘密鍵に逆変換することはできない。数学が一方向にしか機能しないからである。

この計算を「秘密鍵」のセクションで示した特定の秘密鍵に適用して公開鍵を求めてみましょう。

```
K = f8f8a2f43c8376ccb0871305060d7b27b0554d2cc72bccf41b2705608452f315 * G
```

暗号ライブラリを使えば、楕円曲線乗算で`K`を計算できます。結果の公開鍵`K`は次の点として定義されます。

```
K = (x, y)
```

ここで：

```
x = 6e145ccef1033dea239875dd00dfb4fee6e3348b84985c92f103444683bae07b
y = 83b5c38e5e2b0c8529d7fa3f64d46daa1ece2d9ac14cab9477d042c84c32ccd0
```

イーサリアムでは、公開鍵が130文字の16進数（65バイト）のシリアライゼーションとして表されることがあります。これは業界コンソーシアムであるStandards for Efficient Cryptography Group（SECG）が提案した標準シリアライゼーション形式に基づき、Standards for Efficient

Cryptography (SEC1)に文書化されています。この標準では、楕円曲線上の点を識別するための4つのプレフィックスが定義されています（表4-1）。

表4-1 シリアライズされた楕円曲線公開鍵のプレフィックス

プレフィックス	意味	長さ（プレフィックスを含むバイト数）
0x00	無限遠点	1
0x04	非圧縮の点	65
0x02	偶数yの圧縮された点	33
0x03	奇数yの圧縮された点	33

イーサリアムは非圧縮の公開鍵のみを使用するため、関連するプレフィックスは（16進数の）`04`のみです。公開鍵のx座標とy座標を連結してシリアライズします。

```
04 + x座標（32バイト / 64桁の16進数） + y座標（32バイト / 64桁の16進数）
```

したがって、先ほど計算した公開鍵は次のようにシリアライズされます。

```
046e145ccef1033dea239875dd00dfb4fee6e3348b84985c92f103444683bae07b83b5c38e5e2b0c8529d7fa3f64d46daa1ece2d9ac14cab9477d042c84c32ccd0
```

楕円曲線ライブラリ

暗号通貨関連のプロジェクトで使われている`secp256k1`楕円曲線の実装がいくつかあります。

OpenSSL

OpenSSLライブラリは`secp256k1`の完全な実装を含む包括的な暗号プリミティブのセットを提供します。たとえば公開鍵の導出には`EC_POINT_mul`関数を使用できます。

libsecp256k1

Bitcoin Coreの`libsecp256k1`は、`secp256k1`楕円曲線およびその他の暗号プリミティブのC言語実装です。Bitcoin Coreソフトウェアの OpenSSLを置き換えるためにゼロから書かれたもので、パフォーマンスとセキュリティの両面で優れていると見なされています。

暗号学的ハッシュ関数

暗号学的ハッシュ関数はイーサリアム全体で使われています。実際、ほぼすべての暗号システムで広く使われています。暗号技術者のブルース・シュナイアーは「暗号化アルゴリズムよりも

はるかに、一方向ハッシュ関数が現代の暗号技術の主力である」と述べています。

このセクションではハッシュ関数について説明し、その基本的な性質を探り、現代の暗号技術の多くの分野でいかに有用であるかを見ていきます。ハッシュ関数はイーサリアムの公開鍵からアドレスへの変換の一部であるため、ここで取り上げます。また、データの検証を助けるデジタルフィンガープリントの作成にも使えます。

簡単に言えば、*ハッシュ関数*は任意のサイズのデータを固定サイズのデータにマッピングする関数です。入力は*プリイメージ*、*メッセージ*、または単に入力データと呼ばれ、出力は*ハッシュ*と呼ばれます。暗号学的ハッシュ関数は、イーサリアムのような安全なプラットフォームに有用な特定の性質を持つ特別なサブカテゴリーです。

暗号学的ハッシュ関数は、任意のサイズのデータを固定サイズのビット列にマッピングする*一方向ハッシュ関数*です。「一方向」とは、出力ハッシュだけを知っていても入力データを再現することが計算上不可能であることを意味します。可能な入力を決定する唯一の方法は総当たり探索であり、出力が一致する各候補を確認していくことになります。探索空間が事実上無限であることを考えると、このタスクが実質的に不可能であることは容易に理解できます。仮にハッシュが一致する入力データを見つけられたとしても、それが元の入力データと同じとは限りません。ハッシュ関数は複数の入力を同じ出力に写す「多対一」の関数だからです。同じ出力にハッシュされる2つの異なる入力データセットを見つけることは*ハッシュ衝突*の発見と呼ばれます。大まかに言えば、ハッシュ関数が優れているほどハッシュ衝突は稀になります。イーサリアムにおいてハッシュ衝突は事実上不可能です。

暗号学的ハッシュ関数の主な性質を詳しく見てみましょう。

決定性

与えられた入力メッセージは常に同じハッシュ出力を生成する。

検証可能性

メッセージのハッシュ計算は効率的である（線形の計算量）。

無相関性

メッセージへの小さな変更（たとえば1ビットの変更）は、元のメッセージのハッシュとは相関できないほどハッシュ出力を大幅に変えるべきである。

不可逆性

ハッシュからメッセージを計算することは不可能であり、すべての可能なメッセージの総当たり探索に相当する。

衝突耐性

同じハッシュ出力を生成する2つの異なるメッセージを計算することは不可能であるべきである。

ハッシュ衝突への耐性は、イーサリアムにおけるデジタル署名の偽造を避けるために特に重要です。

これらの性質の組み合わせにより、暗号学的ハッシュ関数は幅広いセキュリティアプリケーションに有用なものとなっています。

- データのフィンガープリンティング
- メッセージの完全性（エラー検出）
- プルーフ・オブ・ワーク
- 認証（パスワードのハッシュ化と鍵ストレッチング）
- 擬似乱数生成器
- メッセージコミットメント（コミット・リビールメカニズム）
- 一意の識別子

システムのさまざまなレイヤーを進むにつれて、イーサリアムでこれらの多くを見ることになります。

イーサリアムの暗号学的ハッシュ関数：Keccak-256

イーサリアムは多くの場所でKeccak-256暗号学的ハッシュ関数を使用しています。Keccak-256は、2007年にNISTが開催したSHA-3暗号学的ハッシュ関数コンペティションの候補として設計されました。Keccakが優勝アルゴリズムとなり、2015年に連邦情報処理標準（FIPS）202として標準化されました。

しかしイーサリアムが開発されていた時期には、NISTの標準化はまだ最終決定されていませんでした。NISTは標準化プロセスの完了後、効率向上のためとしてKeccakのパラメーターを調整しました。同時期に、内部告発者エドワード・スノーデンが示唆的な文書を公開しました。その内容は、NISTが国家安全保障局（NSA）から不正に影響を受け、Dual_EC_DRBG RNG標準を意図的に弱体化させて標準RNGにバックドアを仕込んだ可能性があるというものでした。この論争の結果、提案された変更への反発とSHA-3の標準化の大幅な遅延が生じました。当時、イーサリアム財団はNISTが修正したSHA-3標準ではなく、発明者が提案したオリジナルのKeccakアルゴリズムを実装することを決定しました。

警告

イーサリアムのドキュメントやコードで「SHA-3」という表記を見かけるかもしれないが、それらの多く――すべてではないにしても――は実際にはKeccak-256を指しており、最終版のFIPS-202 SHA-3標準ではない。実装の違いはパディングパラメーターに関する軽微なものだが、Keccak-256は同じ入力に対してFIPS-202 SHA-3とは異なるハッシュ出力を生成するため、その違いは重要である。

どのハッシュ関数を使っているのか

使用しているソフトウェアライブラリがFIPS-202 SHA-3とKeccak-256のどちらを実装しているのか、両方が「SHA-3」と呼ばれている可能性がある場合、どう判別できるでしょうか。

簡単な方法は*テストベクトル*を使うことです。テストベクトルとは、与えられた入力に対する期待される出力のことです。ハッシュ関数で最もよく使われるテストは空の入力です。空の文字列を入力としてハッシュ関数を実行すると、以下の結果が得られるはずです。

```
Keccak256("") = c5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470
SHA3("") = a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b49fa82d80a4b80f8434a
```

関数の名称に関係なく、このシンプルなテストを実行することで、オリジナルのKeccak-256なのか最終的なNIST標準FIPS-202 SHA-3なのかを確認できます。イーサリアムはKeccak-256を使用していることを忘れないでください。コードでSHA-3と呼ばれていても同様です。

> **注記**
>
> イーサリアムで使用されるハッシュ関数（Keccak-256）と最終標準（FIP-202 SHA-3）の違いによる混乱のため、すべてのコード、オペコード、ライブラリにおける `sha3` のすべてのインスタンスは `keccak256` に改名された。詳細はERC-59を参照。

次に、イーサリアムにおけるKeccak-256の最初の応用として、公開鍵からイーサリアムアドレスを生成する方法を見てみましょう。

イーサリアムアドレス

イーサリアムアドレスは、Keccak-256一方向ハッシュ関数を使って公開鍵またはコントラクトから導出される一意の識別子です。

前の例では、秘密鍵から始めて楕円曲線乗算を使って公開鍵を導出しました。

秘密鍵 k：

```
k = f8f8a2f43c8376ccb0871305060d7b27b0554d2cc72bccf41b2705608452f315
```

公開鍵 K（x座標とy座標を連結し、16進数で表示）：

```
K = 6e145ccef1033dea239875dd00dfb4fee6e3348b84985c92f103444683bae07b83b5c38e5e...
```

注記

アドレスの計算時には、公開鍵にプレフィックス（16進数の） `04` が付与されないことに注意する価値がある。

Keccak-256を使ってこの公開鍵のハッシュを計算します。

```
Keccak256(K) = 2a5bc342ed616b5ba5732269001d3f1ef827552ae1114027bd3ecf1f086ba0f9
```

次に、最後の20バイト（最下位バイト）のみを保持します。これがイーサリアムアドレスです。

```
001d3f1ef827552ae1114027bd3ecf1f086ba0f9
```

イーサリアムアドレスは通常、16進数エンコードであることを示す `0x` プレフィックス付きで表示されます。

```
0x001d3f1ef827552ae1114027bd3ecf1f086ba0f9
```

イーサリアムアドレスの形式

イーサリアムアドレスは16進数であり、公開鍵のKeccak-256ハッシュの最後の20バイトから導出される識別子です。

ビットコインのアドレスは、どのクライアントのユーザーインターフェースでもチェックサム付きでエンコードされ、入力ミスを検出できるようになっています。一方、イーサリアムアドレスはチェックサムなしの生の16進数として表示されます。この設計判断の根拠は、イーサリアムアドレスは最終的にシステムの上位レイヤーの抽象化（ネームサービスなど）の背後に隠され、必要であればチェックサムは上位レイヤーで追加すべきだということでした。

実際には上位レイヤーの開発は非常にゆっくりとしか進みませんでした。そのためこの設計上の選択はエコシステムの初期にいくつかの問題を引き起こし、誤入力されたアドレスや入力検証エラーによって資金が失われる事例も発生しました。さらにイーサリアムネームサービスの開発も当初の予想より遅かったため、ウォレット開発者による代替エンコーディングの採用も大きく遅れました。次にいくつかのエンコーディングオプションを見てみましょう。

注記

2017年にアレックス・ヴァン・デ・サンデとニック・ジョンソンによって導入されたイーサリアムネームサービス（ENS）について触れておく価値がある。ENSは、

`masteringethereum.eth` のような人間が読める名前をイーサリアムアドレスに変換するオンチェーンソリューションを提供する。

大文字小文字によるチェックサムを持つ16進数エンコーディング（ERC-55）

ネームサービスの展開が遅かったため、ERC-55によって標準が提案されました。ERC-55は16進数アドレスの大文字小文字を変更することで、イーサリアムアドレスに後方互換性のあるチェックサムを提供します。イーサリアムアドレスは大文字小文字を区別せず、すべてのウォレットは大文字でも小文字でも表されたイーサリアムアドレスを解釈の違いなく受け入れるべきです。アドレス中のアルファベット文字の大文字小文字を変更することで、タイプミスや読み間違いからアドレスの完全性を保護するチェックサムを伝達できます。ERC-55チェックサムに未対応のウォレットは、大文字小文字の混在を無視するだけです。対応済みのウォレットはアドレスを検証し、99.986%の精度でエラーを検出できます。

大文字小文字混在のエンコーディングは微妙であり、最初は気づかないかもしれません。この例のアドレスは：

```
0x001d3f1ef827552ae1114027bd3ecf1f086ba0f9
```

ERC-55の大文字小文字混在チェックサムを適用すると：

```
0x001d3F1ef827552Ae1114027BD3ECF1f086bA0F9
```

違いがわかりますか。16進数エンコーディングのアルファベット（A〜F）の一部が大文字になり、残りは小文字のままです。

ERC-55の実装は非常にシンプルです。小文字の16進数アドレスのKeccak-256ハッシュを取得します。このハッシュはアドレスのデジタルフィンガープリントとして機能し、便利なチェックサムを提供します。入力（アドレス）への小さな変更が結果のハッシュ（チェックサム）に大きな変化をもたらすため、効果的にエラーを検出できます。アドレスのハッシュはアドレス自体の大文字小文字にエンコードされます。ステップごとに分解してみましょう。

1. `0x` プレフィックスなしの小文字アドレスをハッシュ化する：

   ```
   Keccak256("001d3f1ef827552ae1114027bd3ecf1f086ba0f9") =
   23a69c1653e4ebbb619b0b2cb8a9bad49892a8b9695d9a19d8f673ca991deae1
   ```

2. ハッシュの対応する16進数の桁が `0x8` 以上の場合、アドレスの各アルファベット文字を大文字にする。アドレスとハッシュを並べると分かりやすくなる：

   ```
   Address: 001d3f1ef827552ae1114027bd3ecf1f086ba0f9
   ```

```
Hash    : 23a69c1653e4ebbb619b0b2cb8a9bad49892a8b9...
```

アドレスの4番目の位置にアルファベット文字`d`があります。ハッシュの4番目の文字は`6`で、8より小さいです。したがって、`d`は小文字のままにします。アドレスの次のアルファベット文字は6番目の位置の`f`です。16進数ハッシュの6番目の文字は`c`で、8より大きいです。したがって、アドレスの`F`を大文字にします。同様に続けます。ご覧の通り、チェックサムとしてハッシュの最初の20バイト（40桁の16進数文字）のみを使用します。アドレスの20バイト（40桁の16進数文字）を適切に大文字化するために使います。

結果の大文字小文字混在アドレスを自分で確認し、どの文字が大文字化され、アドレスハッシュのどの文字に対応しているか見てみてください。

```
Address: 001d3F1ef827552Ae1114027BD3ECF1f086bA0F9
Hash    : 23a69c1653e4ebbb619b0b2cb8a9bad49892a8b9...
```

ERC-55エンコードアドレスのエラー検出

ここで、ERC-55アドレスがエラーの発見にどう役立つかを見てみましょう。ERC-55でエンコードされたイーサリアムアドレスを印刷したとします。

```
0x001d3F1ef827552Ae1114027BD3ECF1f086bA0F9
```

このアドレスを読み取る際に基本的なミスを犯したとしましょう。最後から2番目の文字は大文字のFです。この例では、大文字のEと読み間違えて、以下の（間違った）アドレスをウォレットに入力したとします。

```
0x001d3F1ef827552Ae1114027BD3ECF1f086bA0E9
```

幸いなことに、ウォレットはEIP-55に準拠しています。大文字小文字の混在に気づき、アドレスの検証を試みます。小文字に変換してチェックサムハッシュを計算します。

```
Keccak256("001d3f1ef827552ae1114027bd3ecf1f086ba0e9") =
5429b5d9460122fb4b11af9cb88b7bb76d8928862e0a57d46dd18dd8e08a6927
```

ご覧の通り、アドレスが1文字しか変わっていなくても（実際には1ビットだけ、`e`と`f`は1ビット違い）、アドレスのハッシュは劇的に変化しています。これがハッシュ関数をチェックサムに非常に有用にする性質です。

2つを並べて大文字小文字を確認してみましょう。

```
001d3F1ef827552Ae1114027BD3ECF1f086bA0E9
5429b5d9460122fb4b11af9cb88b7bb76d892886...
```

すべてが間違っています。いくつかのアルファベット文字の大文字小文字が正しくありません。大文字小文字は正しいチェックサムのエンコーディングであることを思い出してください。

入力したアドレスの大文字小文字は計算されたばかりのチェックサムと一致しないため、アドレスに何かが変更され、エラーが導入されたことがわかります。

バリデータの暗号技術

このセクションでは、新しいPoSベースのコンセンサスプロトコルでバリデータが使う暗号技術を探ります。デジタル署名を作成しメッセージを検証するという核心的なアイデアは常に同じですが、いくつかの興味深い要件があります。その結果、トランザクションやアドレスに使われる暗号技術とは設計上の選択も最終的な実装も異なります。

はじめに

イーサリアムが*Ethash*（PoWコンセンサスアルゴリズム）を使っていた時代には、ブロック提案者（当時はマイナー）が生成するブロックに署名すべき理由はありませんでした。PoWベースのコンセンサスアルゴリズムは、正しく機能するために誰がブロックを作成しているかを知る必要がないからです。ブロック提案者が不正を行った場合、プロトコルは電気エネルギーと時間、つまりお金を無駄にさせることで暗黙的に罰します。

二重支払いを試みるためにマイナーが同時に2つのブロックを作成する例を考えてみましょう。この「悪用」の核心的なアイデアをよりよく理解するために、マイナーが車を購入するにあたり、ディーラーにETHで支払うことで合意したとします（10 ETHとしましょう）。マイナーは同時に2つのブロックを作成します。両方が有効だとして、最初のブロックにはマイナーがカーディーラーに10 ETHを送る支払いトランザクションを追加し、もう一方には追加しません。マイナーはチェーンを2つに分割しようとしています。

（ほぼ）ランダムな要因により、カーディーラーはマイナーが10 ETHを支払ったブロックを先に受信する可能性があります。そのためカーディーラーは支払いが完了したと想定できます。しかしまだ車をマイナーに渡しません。カーディーラーはPoWシステムの仕組みを知っており、トランザクションが確実に決済されたとみなすまでに通常いくつかのブロックを待つ必要があるためです。

約12秒後、別のマイナーが10 ETHの支払いを含まないブロックの上に新しいブロックを生成します。次のマイナーがどのブロックを先に受信するかに通常左右されるため、これは十分にあり得ます。こうしてより長いブロックチェーンが存在するようになり、*最重チェーンルール*（少し不正確ですが*最長チェーンルール*とも呼ばれる）のおかげで、そのチェーンがすべてのノードに唯一の有効なものと見なされます。

カーディーラーは、有効なブロックチェーンに以前受信した10 ETHの支払いを含むブロックが含まれていないことを確認し、支払いが完了していないと見なして、マイナーに車を渡しませ

ん。

PoWシステムでは、マイナーはPoWチェックをパスする有効なブロック（つまりブロックハッシュが動的な閾値より低いブロック）を生成するために実際のエネルギーと時間を費やさなければなりません。エネルギーと時間をかけて作ったブロックをネットワーク全体から拒否されること自体が、マイナーへの暗黙の罰となります。有効なブロックのみを作成してすべてのルールを遵守し、不正を試みないことに使えたはずの貴重なエネルギーと時間を失ったのです。

PoSシステムには、不正を行うブロック生成者（つまりバリデータ）に対する同じ暗黙的な罰がありません。代わりに、明示的な方法である*スラッシング*を使用します。

*スラッシング*とは、ルールに従わなかったバリデータに対し、ステークしたETHの一部（またはすべて）を差し引く形で罰を与える行為です。バリデータがアクティブなバリデータになるために最低限のETHをステークしなければならないのはこのためです。ステークされたETHがなければ、悪意ある行動を始めたバリデータをプロトコルは罰せません。

ルールに従わないバリデータを明示的に罰するには、すべてのバリデータが何をしているかを正確に知る必要があります。そのためバリデータが送信するすべてのメッセージ（ブロックを含む）はデジタル署名で認証されなければなりません。

先ほどマイナーで使った例を再現してみましょう。今度は同時に2つのブロックを提案するバリデータがいます。PoWではマイナーが同時に2つの有効なブロックを生成するために2倍のエネルギーと時間を費やす必要がありましたが、PoSでは（ほぼ）完全に無料です。

しかしここでスラッシングが登場します。*二重署名*（同時に2つのブロックを提案すること）はスラッシング対象のイベントであり、バリデータは（この具体例では）ステークしたETHのすべてを破壊されることで即座に罰せられます。

バリデータがすべてのメッセージを認証しなければならない理由がわかったので、そのために使われる暗号技術——BLS署名——について深く掘り下げることができます。次のセクションでは、BLS暗号技術を使用する最終的な決定につながった要件を見ていきます。

要件

この問題（つまりバリデータが互いに送信するすべてのメッセージを認証する必要性）に対する最初のソリューションとして考えられるのは、イーサリアムがすべてのトランザクションへのデジタル署名に既に使っている*楕円曲線デジタル署名アルゴリズム*（ECDSA）を適用することです。

しかし、100万以上のバリデータを抱えるイーサリアムの規模では、ECDSAは現実的な選択肢ではありません。署名アルゴリズムには満たすべき基本要件があるからです。具体的には、ブロック内で署名が占めるスペースを削減し、ノードがすべてのバリデータの署名を検証する時間を短縮するために、署名を集約できる必要があります。

現在、すべてのバリデータは*エポック*ごと（つまり32*スロット*ごと）に1回投票できます。この投票こそ認証が必要なメッセージです（詳しくは第15章で説明します）。これらのメッセージはP2Pゴシップレイヤーを通じて他のバリデータやノードに到達し、すべてのノードが署名の有効性を検証してブロックに挿入する必要があります。

ここでボトルネックを見つけるのは容易です。ネットワークに参加するバリデータが増えるほど、ノードが処理すべきメッセージは増え、バリデータ全体の署名が占めるスペースも大きくなり、結果としてブロックも大きくなります。

注記

アクティブなバリデータの数に基づいて、ノードが処理する必要があるメッセージ数を簡単に計算できる：

```
N = アクティブなバリデータの数
1 msg / 1 epoch × N =
= 1 msg / 32 slots × N = ← 1エポックは32スロットで構成される
= 1 msg / 32 × 12 seconds × N = ← 各スロットは12秒かかる
= 1 msg / 384 seconds × N =
= 0.0026041667 msg / second × N
```

したがって、毎秒約0.0026メッセージにN（ネットワーク内のアクティブなバリデータ数）を乗じることになる：

```
N = 1,000 → 毎秒約2.6メッセージ
N = 10,000 → 毎秒約26メッセージ
N = 100,000 → 毎秒260メッセージ
N = 1,000,000 → 毎秒2,600メッセージ
```

現在、イーサリアムのPoSプロトコルは毎秒約2,600メッセージを処理しています。

こうした理由から、ほとんどのPoSブロックチェーンのバリデータセットは非常に小さく、最大でも数十から数百です。イーサリアムの当初の提案（EIP-1011参照）でさえ最大900バリデータをターゲットとし、アクティブなバリデータセットに選出されるための最低預入額は1,500 ETHでした。

この仕様がイーサリアムのPoSプロトコルの最終形になっていたでしょう——イーサリアム財団の研究者ジャスティン・ドレイクが2018年5月にethresearchウェブサイトで公開した長い記事で、BLS署名の集約というアイデアを提案していなければ。

BLSデジタル署名

BLS署名はその考案者の名前にちなんで名付けられています。BLSはボネ・リン・シャチャム（Boneh–Lynn–Shacham）の略で、2001年に論文"Short Signatures from the Weil Pairing"でこの署名方式を発表した3人の暗号技術者、ダン・ボネ、ベン・リン、ホヴァヴ・シャチャムを指します。

ECDSA（第6章でさらに詳しく説明します）と同様に、BLS署名も楕円曲線暗号に基づいています。イーサリアムはショーン・ボウが2017年にZcashプロトコルのために設計した曲線*BLS12-381*を使用しています。BLS12-381は以下の関数で定義されます。

```
y^2 = x^3 + 4
```

qを法とする整数上で定義されます。ここでqは115桁の10進数です：`0x1a0111ea397fe69a4b1ba7b6434bacd764774b84f38512bf6730d2a0f6b0f6241eabfffeb153ffffb9feffffffffaaab`。

仕組み

核心的なアイデアはECDSAの仕組みとよく似ています。秘密鍵があり、そこから公開鍵が導出されます。すべてのメッセージは秘密鍵で署名され、他の人は対応する公開鍵を使って完全性を検証できます。

秘密鍵（`sk`）は1からr – 1の間の整数です。ここでrはこの巨大な数：`0x73eda753299d7d483339d80809a1d80553bda402fffe5bfeffffffff00000001`です。77桁の10進数です。

公開鍵（`pk`）は`sk * g1`（楕円曲線点のスカラー乗算）によって得られます。ここで`g1`はG1と呼ばれる群の生成元です。`pk`は圧縮・シリアライズされた形式で表現され、48バイトの文字列となります。

署名されるメッセージ（`m`）は常に楕円曲線の点にマッピングされます。この点はG2と呼ばれる別の群のメンバーです。このマッピングは、メッセージ`m`——つまり実際のバリデータのアテステーション——を受け取り、G2の点`H(m)`を出力する一種のハッシュ関数と考えることができます。圧縮・シリアライズされた形式では96バイトの文字列として表現されます。

最後に、メッセージ`m`の署名（σ）は`sk * H(m)`（G2における新しい楕円曲線の点）によって得られます。

ECDSAとBLSの主な違いは、2つのプロトコルの低レベルの詳細（つまり署名の正しさを検証する数学的テクニック）にあります。ECDSAは楕円曲線上のスカラー乗算や加算などの線形計算を伴いますが、BLS署名はより複雑な*楕円曲線の双線形ペアリング*の算術に依存しています。

実際、署名σが有効であるのは、以下の等式が成り立つ場合に限ります。

```
e(g1,σ) = e(pk, H(m))
```

ここで`e`は楕円曲線の双線形ペアリングです。

楕円曲線の性質を探る

前の等式を完全に理解するには、楕円曲線、ペアリング、およびそれらの広大な深淵に関する深い知識が必要です。しかし、ここでの手順を追うだけでも理解の助けになります。ペアリングがその性質を持つ理由と仕組みのすべての詳細は把握できませんが、親しみを持つ助けにはなるでしょう。

```
e(pk, H(m)) = e(sk * g1,H(m)) = ← pk = sk * g1
= e(g1, H(m))^sk = ← ペアリングの性質による
= e(g1, sk * H(m)) = ← 同じ性質による
= e(g1,σ) ← σ = sk * H(m)
```

ヴィタリック・ブテリンがMediumの記事で指摘したように：

> 楕円曲線の点を一方向に暗号化された数値と見なすと―― `encrypt(p) = p * G = P`（ここでGは生成点）――従来の楕円曲線の数学では数値の線形制約を検証できる（例：`P = G * p`、`Q = G * q`、`R = G * r`の場合、`5 * P + 7 * Q = 11 * R`の検証は実際には`5 * p + 7 * q = 11 * r`の検証である）。一方、ペアリングでは二次制約を検証できる（例：`e(P, Q) * e(G, G * 5) = 1`の検証は実際には`p * q + 1 * 5 = 0`の検証である）。

想像できるように、より複雑な算術は署名の生成と検証により多くの時間がかかることを意味します。イーサリアムがバリデータにBLS署名を使っている唯一の理由は、*署名の集約*という極めて重要な性質です。

実際、2つのECDSA署名を足し合わせても、意味のある結果は得られません。その結果を使って最初に署名されたメッセージの完全性を証明しようとしても、テストにはパスしません。

一方、2つのBLS署名を足し合わせると、単純に楕円曲線の加算を行っていることになります。結果はG2の新しい楕円曲線の点であり、2つの対応する公開鍵の和――G1の楕円曲線の点のまま――に対して使用することで、最初に署名されたメッセージの完全性を正しく検証できます。

> **注記**
>
> 集約された署名と集約された公開鍵は、単一の署名と単一の公開鍵と区別がつかないため、集約された署名の正しさを検証するためにまったく同じアルゴリズムを使用できる。
>
> **集約された署名と公開鍵：**

```
σagg = σ1 + σ2 + σ3 + ⋯ + σn
pkagg = pk1 + pk2 + pk3 + ⋯ + pkn
```

集約された署名の検証：

```
e(pkagg,H(m)) =
= e(pk1 + pk2 + pk3 + ... + pkn,H(M)) =
= e((sk1 + sk2 + sk3 + ... + skn) * g1,H(m)) =
= e(g1,H(m))^(sk1 + sk2 + sk3 + ... + skn) =
= e(g1,(sk1 + sk2 + sk3 + ... + skn) * H(m)) =
= e(g1,σ1 + σ2 + σ3 + ... + σn) =
= e(g1,σagg)
```

まとめ

暗号技術は本当に難しく、深い数学的背景が必要です。この本は暗号技術者向けではないので、前のセクションで簡単に触れた内容を要約しましょう。BLS署名の仕組みと、バリデータが新しいPoSベースのコンセンサスプロトコルで使うデジタル署名アルゴリズムとして選ばれた理由を説明しました。集約の性質により、複数のデジタル署名をひとつにまとめられ、保存に必要なスペースと検証にかかる時間を削減しつつセキュリティを犠牲にしません。

バリデータが「実際の運用」でBLSアルゴリズムをどう使い、署名の集約がどう機能するかを示す簡単な例を見てみましょう。図4-6は、3人のバリデータがブロックAに対して投票を表明したいシナリオを示しています。投票を行い、署名し、互いに共有します。

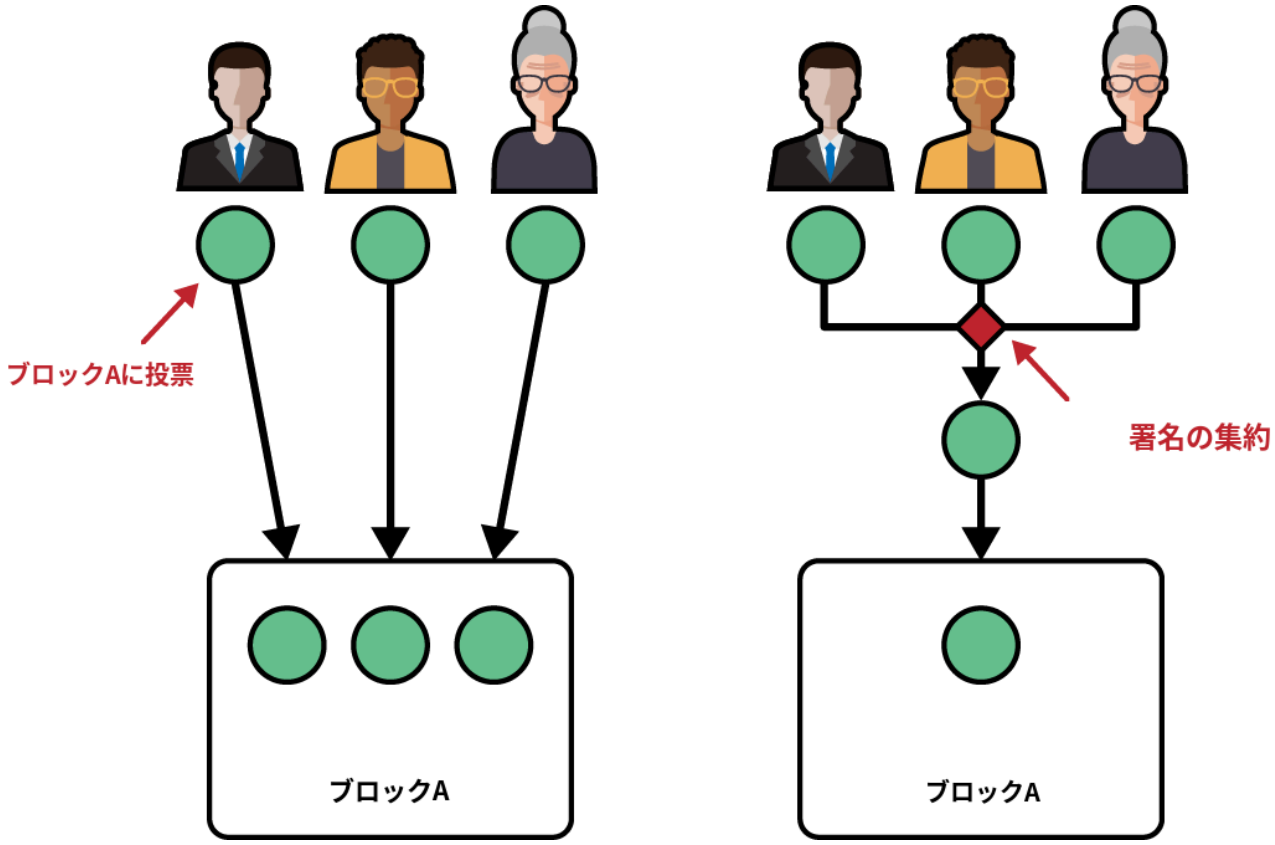


図4-6 BLS署名の集約：3人のバリデータが投票し署名する

BLSがなければ、3つの署名付き投票すべてをブロックに保存して永続化する必要があります。BLSを使えば、3つの署名付き投票すべてをひとつの新しい集約投票に集約し、それだけをブロックに保存できます。

これはスペースを節約するだけでなく、すべてのイーサリアムノードが署名付き投票を検証する際に必要な時間と計算量を劇的に削減します。個々の署名付き投票ごとに検証する代わりに、集約された投票を直接検証できるからです。BLS暗号技術のマジックは、集約された結果が通常の署名と変わらないことです。つまり集約された署名の有効性を検証することは、単一の署名の有効性を検証するのと同じ難しさです。検証すべき署名付き投票の数は大幅に減りますが、各検証に必要な計算量は同じです。そのため、フルノードが実行すべき計算量と時間の合計は、BLS集約署名を使わない場合よりはるかに少なくなります（図4-7参照）。

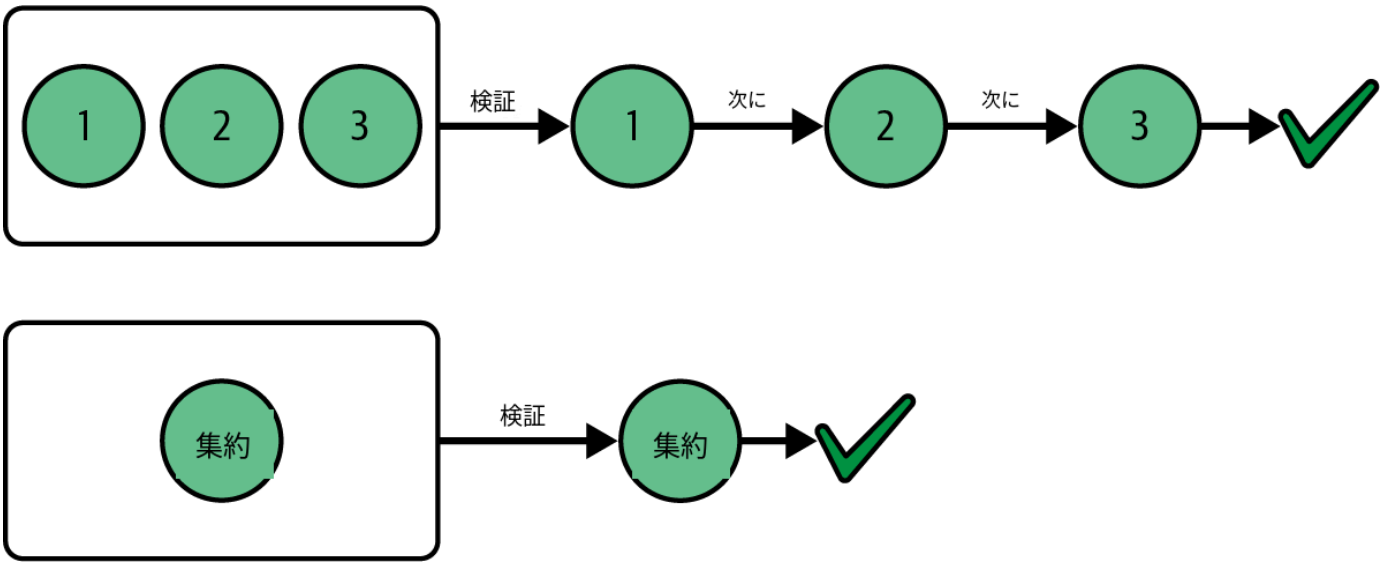


図4-7 BLS署名の集約による検証時間の削減

バリデータがルールに従わなかったらどうなるでしょうか。バリデータが悪意を持って行動した場合（たとえば二重署名、つまり同時に2つの異なるブロックに投票した場合）、プロトコルはこの行動を検出し、図4-8に示すようにバリデータを適切に罰せます。

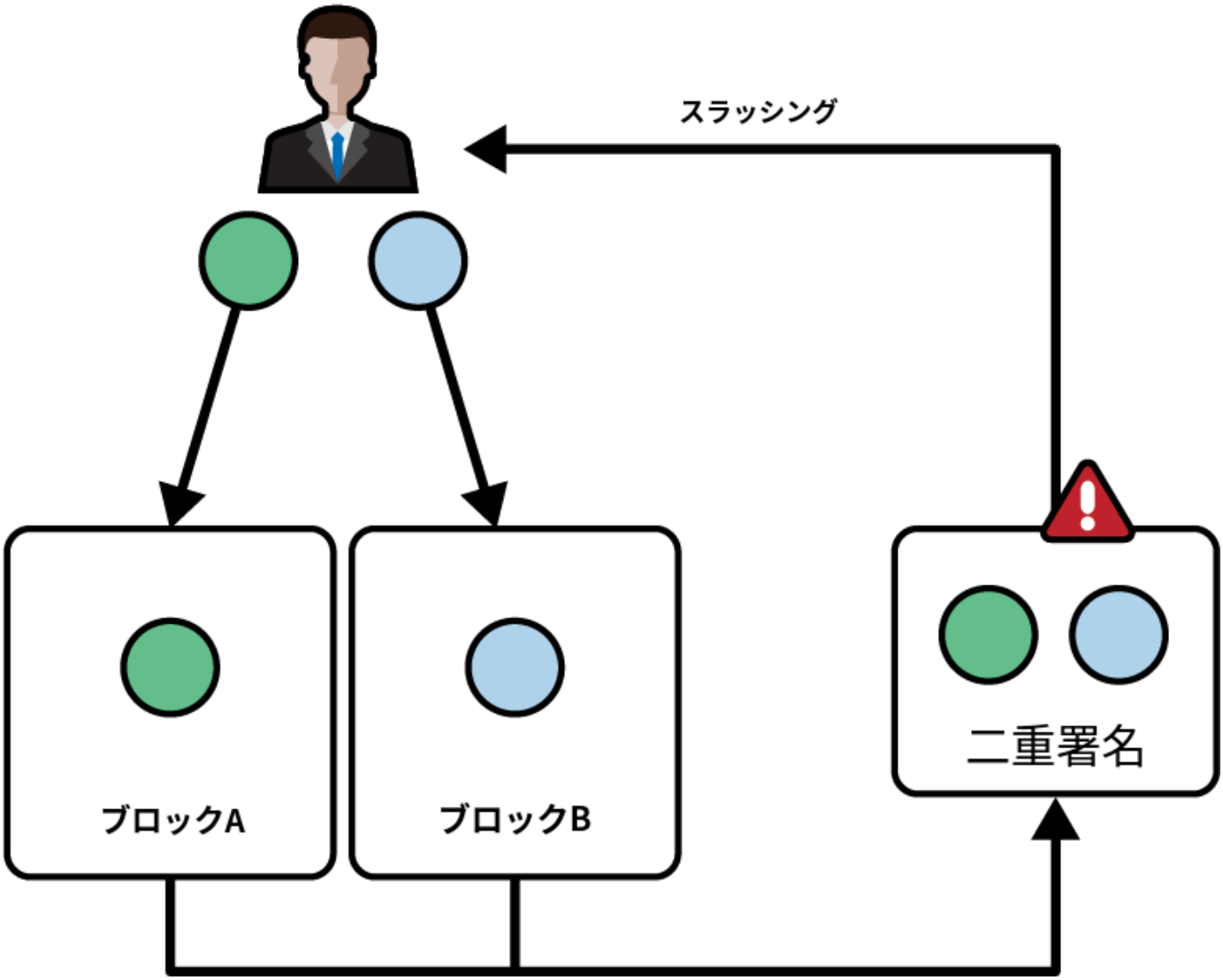


図4-8 BLS署名により悪意あるバリデータの検出が可能になる

すべての投票がBLS署名方式でデジタル署名されているため、不正行為の原因となったバリデータを特定し、適切にスラッシングすることは容易です。

KZG暗号技術

2024年3月13日、イーサリアムはCancun-Deneb（Dencun）ハードフォークにアップグレードしました。このアップグレードには多くの変更が含まれていましたが、最も重要なものは間違いなく新しいタイプのトランザクション、EIP-4844 blobトランザクションの導入でした。

blobトランザクションがイーサリアムのロードマップにとってなぜ重要なのかは第16章ではるかに詳しく説明するため、ここでは深入りしません。代わりに次のセクションでは、この新しいタイプのトランザクションの基盤となる暗号技術（*多項式コミットメントスキーム*、より具体的には*KZGコミットメントスキーム*）を探ります。

はじめに

多項式コミットメントスキームを見る前に、直面している問題と達成したい最終目標を理解することが重要です。

> **注記**
>
> このセクションでは、レイヤー2（L2）という用語に簡単に触れる。L2については第16章で適切に説明・展開するが、ここでは簡単な紹介をしておくと役立つだろう。L2はイーサリアムの新しいスケーリングソリューションである。独自の履歴と状態を持つ実際のブロックチェーンであり、定期的にすべての更新とデータをイーサリアムのメインネットに投稿する。その考え方は、イーサリアムのメインネットのみを見ることで、L2ブロックチェーン全体を導出できなければならない、というものである。

詳細は省きますが、L2は毎日大量のデータをイーサリアムのメインチェーンに投稿します。このデータはネットワーク内のすべてのノードがダウンロードして検証しなければなりません。この状況は通常ガス代の急上昇につながり、深刻なシナリオでは単一のETH送金だけで10〜20ドルに達することがあります。

最終的には、すべてのデータを直接オンチェーンに投稿する代わりに、データへのコミットメント（つまり暗号学的ハッシュ）のみを保存し、完全なデータはイーサリアムチェーンの外に残したいと考えています。しかしハッシュのみを投稿すると新たな問題が生じます。ネットワーク内のすべてのノードが、コミットメントの指すデータがどこかに存在することを確認しなければなりません。そうでなければ重要なデータが失われている可能性があります。

最初の（明白な）ソリューションは、すべてのノードに完全なデータの一時的なダウンロードを義務付けることです。これによりコミットメントが有効であること（つまり自分が持っている完全なデータを指し示していること）を容易に検証できます。本章の執筆時点（2025年6月）ではこれが現在の状況です。

長期的なソリューションは、すべてのノードに完全なデータのダウンロードを要求するのではなく、ごく一部のみをダウンロードさせる方式です。このときノードは、データが完全に利用可能であることを保証する暗号学的証明を利用できます。ただしこの証明は、サイズが小さく迅速に検証できるものでなければなりません。基本的な考え方は、データが利用可能であることを証明・検証するほうが、データ全体をダウンロードして直接検証するよりもはるかに高速かつ軽量だというものです。この戦略は*データ可用性サンプリング*（DAS）と呼ばれ、イーサリアムコミュニティにおける最も重要な研究分野のひとつです。

DASの最終目標を達成するために、すべてのノードが以下のことを可能にする暗号システムが必要です。

- あるデータ（存在し、完全に利用可能であることを証明したいデータ）へのコミットメントを作成する
- 以前作成したコミットメントに関連するデータの小さな証明を生成する

- これらの証明を容易に検証する

この暗号システムは、参照するデータに関する情報をあまり明かさないことが（任意ですが）理想的です。

次のセクションでは、そのような暗号システム——多項式コミットメントスキーム、より具体的にはKZGコミットメントスキーム——を探ります。

> **注記**
>
> なぜデータコミットメントスキームではなく多項式コミットメントスキームの話をしているのか、と疑問に思うかもしれない。それはとても良い質問である。目標はデータにコミットすることであり、多項式にコミットすることではない。しかし、ここにトリックがある。データは多項式として表現できる。したがって、多項式コミットメントスキームを作成できれば、それで十分である。

多項式コミットメントスキーム

*多項式コミットメントスキーム*は、証明者が多項式へのコミットメントを計算できるようにするものです。このコミットメントは後で任意の位置で「開く」（評価する）ことができる性質を持ちます。証明者はある位置における多項式の値が主張された値と等しいことを示し、その主張に対する証明を生成できます。検証者はコミットメントと証明の両方を持ち、証明者が不正をしていない場合にのみ（つまりコミットされた多項式が選択された位置で実際に主張された値に評価される場合にのみ）証明が有効であることを検証できます。

多項式入門

このセクションでは多項式について多く語ることになるため、多項式が何であるかを全員が理解していることが重要です。学校で学んでから時間が経っているかもしれません。

多項式は次のような数学的な式です。

```
x^3 + 3x^2 – 9
```

ここで`x`は変数であり、すべての数値（1, 3, 0, –9）は係数です。3番目の係数を0としたのは、この多項式が実際には以下のものだからです。

```
x^3 + 3x^2 + 0x – 9
```

このセクションを通じて、多項式を`p(x)`と表記します。

```
p(x) = x^3 + 3x^2 + 0x - 9
```

多項式の次数`n`も必要です。次数は多項式における最高次の冪の数です。この例の多項式の次数は3です。したがって`n = 3`です。

多項式を記述する一般的な公式は以下の通りです。

$$p(x) = p_0 + p_1x + p_2x^2 + ... + p_nx^n$$

またはより形式的に：

$$p(x) = \Sigma\ p_ix^i \quad (\text{for } i = 0 \text{ to } n)$$

ここで`n`は多項式の次数、`pi`はすべての係数です。

マークルツリーの使用

古典的なマークルツリーデータ構造を使った多項式コミットメントスキームを見てみましょう。マークルツリーの詳細については、第14章で詳しく説明します。

マークルツリーの深さ`d`に対して次数`n = 2^d - 1`の多項式にコミットできます。ツリーのすべてのリーフ要素を、コミットしたい多項式のすべての係数`pi`に等しく設定します。

以下の例を見てみましょう。

```
p(x) = x^7 + 5x^5 - 2x^4 + 3
n = 7 ← p(x)の次数
d = 3 ← 必要なマークル木の深さ
```

図4-9は、マークルルートを核となる暗号プリミティブとして使い、この例に多項式コミットメントスキームを適用した様子を示しています。

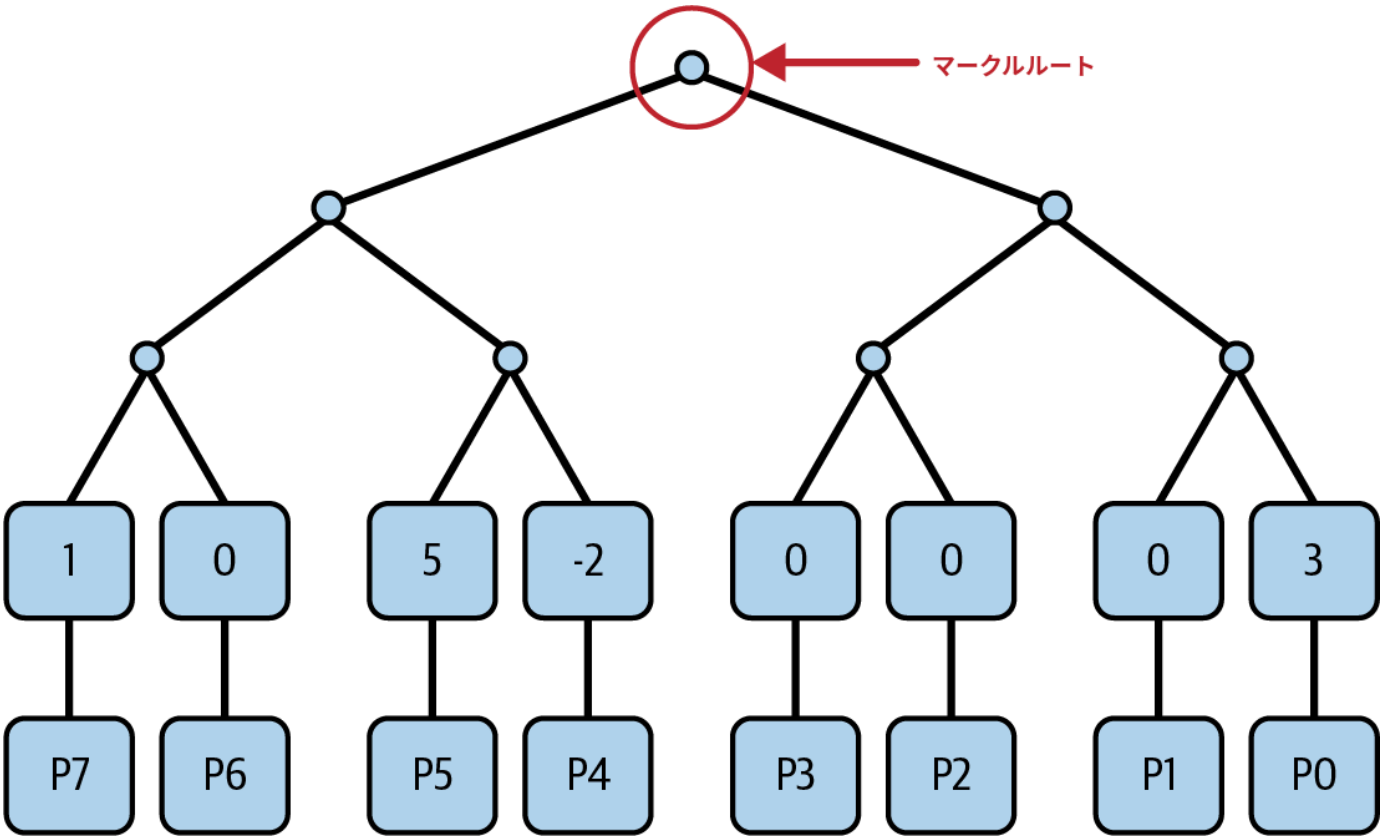


図4-9 マークルツリーを使った多項式コミットメント

最終的なコミットメントは、完全なマークルツリーを構築して得られるマークルルートです。

ここで、`p(1) = 7`であることを（検証者に）証明したいとしましょう。この主張は正しいです。なぜなら、`p(x)`に`x = 1`を代入すると、多項式がその位置で7に評価されることを容易に計算できるからです。

`p(1) = 7`という主張を証明するには、検証者に以下を送信する必要があります。

- 証明したい主張：`p(1) = 7`
- マークルルート——つまり多項式`p(x)`へのコミットメント
- 多項式のすべての係数`pi`

こうすることで、検証者はすべての係数`pi`を取得し、`x = 1`のときの多項式の値を計算できます。これにより、主張が実際に正しいことを検証します。次に、すべての係数`pi`からマークルツリーを計算し、自分のマークルルートと提供されたものを比較します。こうして検証者は、最初にコミットした多項式が実際に送信されたものと同じであることを検証します。

マークルツリーを使った多項式コミットメントの性質をまとめると、以下の通りです。

コミットメントのサイズ

コミットメントはマークルルートであり、単一のハッシュで通常32バイトの長さである。

証明のサイズ

評価を証明するには、多項式のすべての係数`pi`を送信する必要がある。つまり、証明のサイズは多項式の次数`n`に対して線形である。

検証者の計算量

検証者は（多項式の次数に対して）線形の作業を行う必要がある。実際、検証者はすべての係数`pi`を持ち、マークルツリーの計算と主張された点での多項式の評価の両方を計算しなければならない。

スキームのプライバシー

証明者がすべての係数`pi`を送信するため、スキームは多項式に関する情報を何も隠さない。

これらの性質は理想的ではありません。実際のイーサリアムプロトコルでは多項式の次数`n`が非常に大きくなる可能性があり、すべての係数をネットワーク上で送信するために必要な帯域幅が過大になります。また、証明者がコミットした多項式に関する情報をできるだけ少なく明かすプロトコルが望ましいです。

幸いなことに、すべての要件を満たすスキームが存在します。ここでKZGが登場します。

KZGコミットメント

KZGはケイト、ザヴェルチャ、ゴールドバーグの頭文字で、3人の著者の名前です。この3人の暗号技術者は、2010年の論文"Constant-Size Commitments to Polynomials and Their Applications"でこのコミットメントスキームを発表しました。

トラステッドセットアップ

KZGコミットメントスキームには*トラステッドセットアップ*が必要です。トラステッドセットアップは、暗号プロトコルのすべての参加者（証明者と検証者の両方）が共有する共通の知識ベースと見なせます。「トラステッド」と呼ばれる理由は次の手順にあります。まず一部の参加者がランダムな数値（秘密）を生成し、それを暗号化して最終的な共通知識ベースを作成します。その後、プロトコルの安全性を確保するために、参加者は生成した秘密を破棄しなければなりません。各参加者が本当に秘密を破棄したと信頼する必要があるため、このセレモニー全体がトラステッドセットアップと呼ばれます。

現代のセットアップは通常、1-of-Nの信頼モデルを持つ*Powers-of-Tau*セットアップを使用します。つまり、トラステッドセットアップ全体が安全と見なされるために必要な正直な参加者は1人だけです。

イーサリアムのKZGトラステッドセットアップセレモニーには、図4-10に示すように、14万人以上の参加者が関与しました。

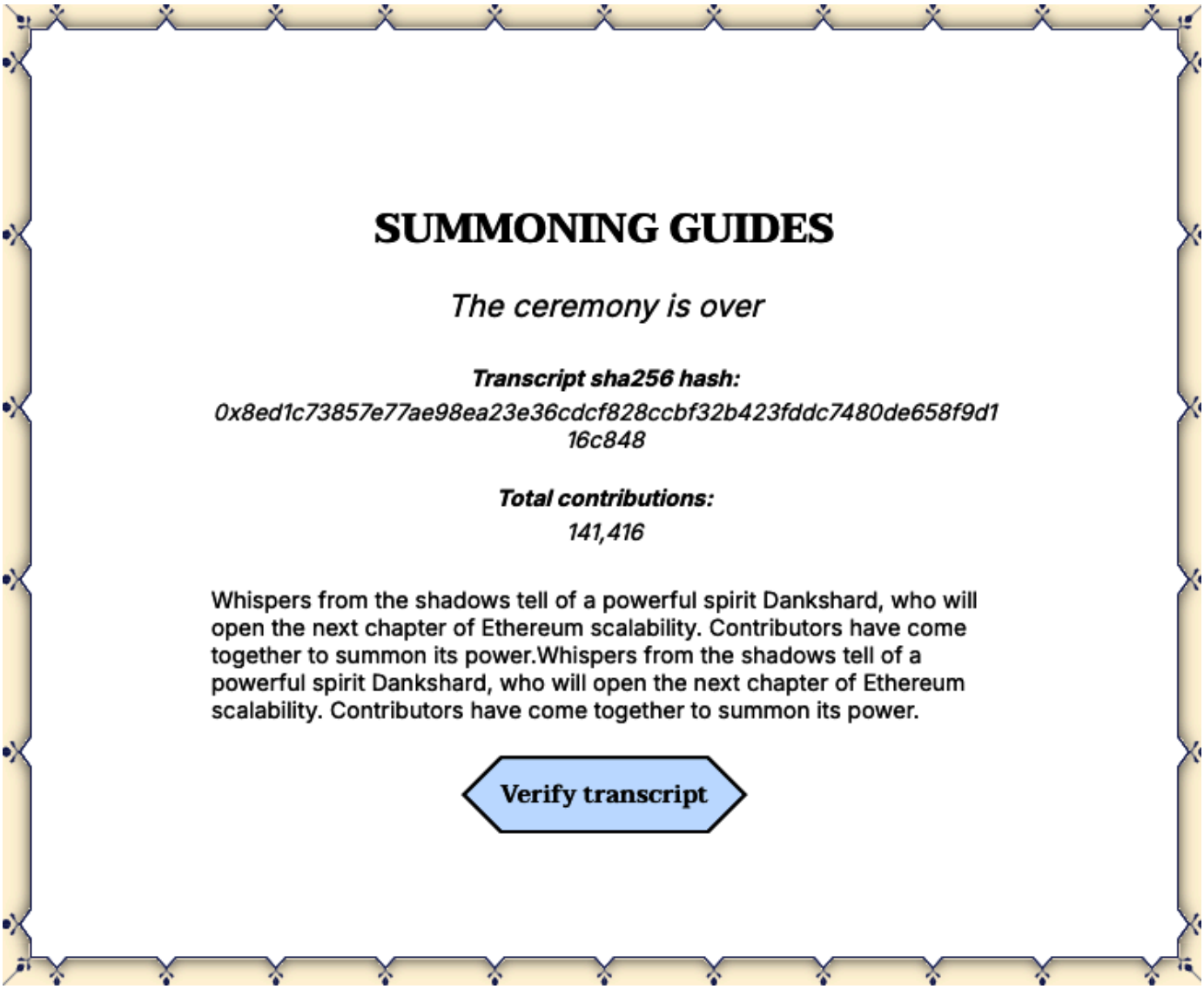


図4-10 イーサリアムのKZGトラステッドセットアップセレモニーの参加者

より具体的には、トラステッドセットアップは`i = 0, 1, ⋯ , n – 1`に対して要素`[s^i]₁`と`[s^i]₂`を生成します。ここで：

- `s`は誰も知らないトラステッドセットアップの秘密（各参加者が生成した秘密の合計で構成される）
- `[s^i]₁`と`[s^i]₂`は実際には楕円曲線の点（それぞれ曲線G1とG2上）
- `n`は多項式の次数

ヒント

角括弧の中にあるものは楕円曲線の点を表す。

コミットメント

ご覧の通り、KZGコミットメントスキームは（再び）楕円曲線とペアリングを伴います。BLSデジタル署名で既に見たものです。

楕円曲線では、秘密の数値 `a` がある場合、楕円曲線乗算によって楕円曲線の点 `[a]` を得ることができることを思い出してください。

```
[a]₁ = aG₁
```

逆方向の計算は計算上不可能です。したがって、 `[a]` のみを持っている場合、秘密 `a` を得ることはできません。

証明者も検証者もトラステッドセットアップの秘密 `s` を知りませんが、それに対して特定の演算を行うことはできます。

```
c[s^i]₁ = cs^iG₁ = [cs^i]₁  ← 楕円曲線乗算；cは整数
c[s^i]₁ + d[s^i]₁ = (cs^i + ds^i)G₁ = [cs^i + ds^i]₁  ← 楕円曲線の点の加算
```

したがって、 `p(x) = p₀ + p₁x + p₂x² + ... + pₙxⁿ` が多項式である場合、証明者は以下を計算できます。

```
[p(s)]₁ = [p₀ + p₁s₁ + p₂s²₁ + ... + pₙsⁿ₁] = p₀ + p₁[s]₁ + p₂[s²]₁ + ... + pₙ[sⁿ]₁
```

`[s^i]₁` はトラステッドセットアップの一部であり共通の知識であるため、証明者は誰も知らない秘密の点 `s` で多項式を評価できます。魔法のように聞こえるでしょう。ただし重要な点があります。前の演算の出力は整数ではなく楕円曲線の点です。楕円曲線の点 `[p(s)]₁` が、多項式 `p(x)` に対するKZGコミットメントです。

KZGの健全性：証明者がコミットメントを偽造できない理由

証明者は秘密 `s` を知らずに、同じKZGコミットメントを持つ異なる多項式 `q(x) != p(x)` を見つけられるでしょうか。 `[q(s)]₁ = [p(s)]₁` のように。もしそうなら、証明者は多項式 `p` を知っていると検証者に信じさせられます。

できると仮定しましょう。すると次数 `n` の定数でない多項式 `r(x) = p(x) - q(x)` が存在します。 `q(x) != p(x)` であるため、 `r(x)` は最大 `n` 個のゼロ（つまり `r(x) = 0` となる位置）を持ちます。これは代数の基本的な性質です。学校で学んだすべての基本的な幾何学的形状について正しいことを容易に確認できます。直線（1次の多項式）は1つのゼロを持ち、放物線（2次の多項式）は最大2つ持つ、といった具合です。

証明者が `q(s) = p(s)` を達成する唯一の方法は、 `r(x) = p(x) + q(x) = 0` をできるだけ多くの場所で成り立たせることです。しかし、前述のように最大 `n` 個のゼロしか選べません。

証明者はsを知らないため、推測できる可能性は極めて低いです。実際、n（多項式の次数）<< p（楕円曲線の位数）です。

$n = 228$で$p = 2^{256}$の場合、sが選択されたゼロのひとつである確率は2×10^{-69}です。

したがって、同じコミットメント$C = [p(s)]_1$を持つ多項式は多数存在するものの、それを計算することは不可能と言えます。これは*計算的束縛*です。

ここまでで秘密の点sで多項式$p(x)$を評価してコミットできるようになりましたが、検証者にすべての係数piを送信せずに証明者の主張$p(z) = y$が正しいことを証明する方法がまだ欠けています。ここでも楕円曲線のペアリングが役立ちます。

KZG証明

最終的なKZG証明を理解するには、多項式のいくつかの重要な性質を思い出す必要があります。

多項式$p(x)$がzでゼロを持つ場合、$(x - z)$で割り切れます。これは非常に理解しやすいです。多項式$p(x) = x^2 - 4$を取り上げましょう。$p(x) = (x + 2)(x - 2)$とも表現できます。$p(2) = 4 - 4 = 0$であるため、$p(x)$は$(x - 2)$で割り切れます。

逆も成り立ちます。多項式$p(x)$が$(x - z)$で割り切れる場合、$p(x)$はzでゼロを持ちます。先ほどと同じ例を使えば容易に示せます。

（証明者として）$p(z) = y$を証明したいことを思い出してください。ここで多項式$p(x) - y$を取り上げます。この多項式は$x = z$でゼロに評価されるため、先ほど説明した性質——特に、$p(x) - y$がzでゼロを持つので$(x - z)$で割り切れるという事実——を使用できます。

したがって、商多項式$q(x)$を計算できます。

```
q(x) = [p(x) - y] / (x - z)
```

これは以下の等価な形式で書くこともできます。

```
q(x)(x - z) = p(x) - y
```

> **注記**
>
> ここで非常に重要な点は、商多項式$q(x)$は$p(x) - y$が$x - z$で割り切れるからこそ計算可能であるということである。そうでなければ、常に余りが生じるため不可能である。
>
> これをよりよく理解するために、例を見てみよう。再び多項式$p(x) = (x + 2)(x - 2)$を取り上げる。この多項式は$(x + 2)$で割り切れるため、商多項式を計算できる：

```
q(x) = p(x) / (x + 2) = [(x + 2)(x - 2)] / (x + 2) = x - 2
```

`(x - 3)` で割ろうとすると、商多項式を得ることはできない。前の例を使って試してみてほしい…

これで証明を生成する準備が完全に整いました。KZG証明は主張 `p(z) = y` に対して `π = [q(s)]₁` です。基本的に、秘密の点 `s` で評価された（楕円曲線の点の）商多項式です。

検証者は以下の等式を計算してこの証明の有効性を確認します。

```
e(π, [s - z]₂) = e(C - [y]₁, H)
```

ここで：

- `π = [q(s)]₁` はKZG証明
- `C = [p(s)]₁` は多項式 `p(x)` へのコミットメント
- `H` はG2群の生成元であり、証明者と検証者の両方に既知

この暗号スキームが以下の両方であることを証明しましょう。

正当性

証明者がすべてのルールに従えば、検証者が正しく検証できる有効な証明を生成できなければならない。

健全性

証明者は偽の証明を計算することで検証者を騙すことができない——つまり、証明者は `y' != y` となる `p(z) = y'` であると検証者に信じさせることができない。

正当性

検証者の等式をペアリング群に書き下す——基本的にペアリング演算を計算する——と、以下が得られます。

```
[q(s)(s - z)]ₜ = [p(s) - y]ₜ
```

ここで `T` は有限体拡大の乗法部分群です。

この等式は、先ほど計算した等式とまったく同じであることがわかります。

```
q(x)(x - z) = p(x) - y
```

秘密の点`s`で評価されたものです。唯一の違いは、数値の代わりに楕円曲線の点があることです。

この等式は構成上常に成り立ちます。正当性の部分はこれで完了です。

健全性

証明者は偽造できるでしょうか。つまり、証明者は`p(z) = y'`が真でないにもかかわらずそう主張し、検証者のチェックをパスできるでしょうか。

それを試みるために、検証者には2つの選択肢があります〔訳注：以下（項目1・2）に登場する「検証者」は、原文では "the verifier" だが、文脈上おそらく「証明者（prover）」が正しいと思われる〕。

1. 検証者は通常の手順に従い、主張する値`y'`を変更するだけでよい。そのため、`p(x) - y'`を`(x - z)`で割って商多項式を計算する必要がある。しかし、`p(z) - y' != 0`であるため不可能である——多項式の除算はここでは実行できない。証明者が`p(x) - y' = 0`となるような`y' != y`を選べるのではないかと思うかもしれない。実際、前の注で既に回答を示しており、確率はほぼゼロである。証明者はそのような`y'`を計算することができない。この最初のソリューションは現実的な選択肢にならない。

2. 検証者は証明を構築する際に楕円曲線上で直接作業できる。実際、以下の証明を生成できれば：

   $$\pi' = [C - y'] \ / \ [s - z]_2$$

   何でも証明できる：

   $$e(\pi', [s - z]_2) = e(C - [y']_1, H) \rightarrow$$
   $$\rightarrow e(\pi', [s - z]_2) = e(C - [y']_1, H) \rightarrow$$
   $$\rightarrow (C - y`)/(s - z)*(s - z) = C - y` \rightarrow$$
   $$\rightarrow C - y` = C - y` \quad \leftarrow \text{これは常に真。}$$

   しかし、この証明の計算は証明者にとって計算上不可能である。なぜなら`s`の知識が必要であり、`s`はトラステッドセットアップの秘密で誰もその値を知らないからである。

KZGの性質

KZG証明システムを使った多項式コミットメントの性質をまとめると、以下の通りです。

コミットメントのサイズ

コミットメントはペアリングを許容する楕円曲線の単一の群要素である。たとえば、曲線BLS12_381では、圧縮形式で48バイトの長さである。

証明のサイズ

証明のサイズはコミットする多項式`p(x)`のサイズに依存しない。常に楕円曲線のひとつの群要素である。

検証者の計算量

検証も多項式のサイズに依存せず、2つの（楕円曲線の）群乗算と2つのペアリングのみが必要である。

スキームのプライバシー

スキームは、証明者がコミットする多項式`p(x)`をほぼ隠すことができる。実際、マークルツリーでは証明者がすべての係数を送信する必要があったが、KZGでは秘密の点`s`で評価された商多項式のみを送信すればよい。ただし、それは整数ではなく楕円曲線の点である。

マルチプルーフ

多項式`p(x)`の単一の評価を証明できましたが、さらに多くのことが可能です。多項式を任意の数の点で評価し、同じ証明サイズで証明できます。

その仕組みを見てみましょう。すべてのz_iに対して`p(x)`がy_iに評価されることを証明したいとします。

$$(z_0, y_0), (z_1, y_1), \cdots, (z_{k-1}, y_{k-1})$$

これらのすべての点を通る`k`未満の次数の多項式を常に見つけることができます。これは代数的性質であり、その多項式を生成する数学的公式も存在します。*補間多項式*`I(x)`と呼ばれます。

$$I(x) = \Sigma\ y_i \cdot \prod\ (x - z_j)/(z_i - z_j)\ \ \text{(for i,j = 0 to k-1, j} \neq \text{i)}$$

難しく聞こえるかもしれませんが、実際にはかなり理解しやすいです。2つの異なる点があるとします（図4-11参照）。

```
A = (1, 0)
B = (2, 1)
```

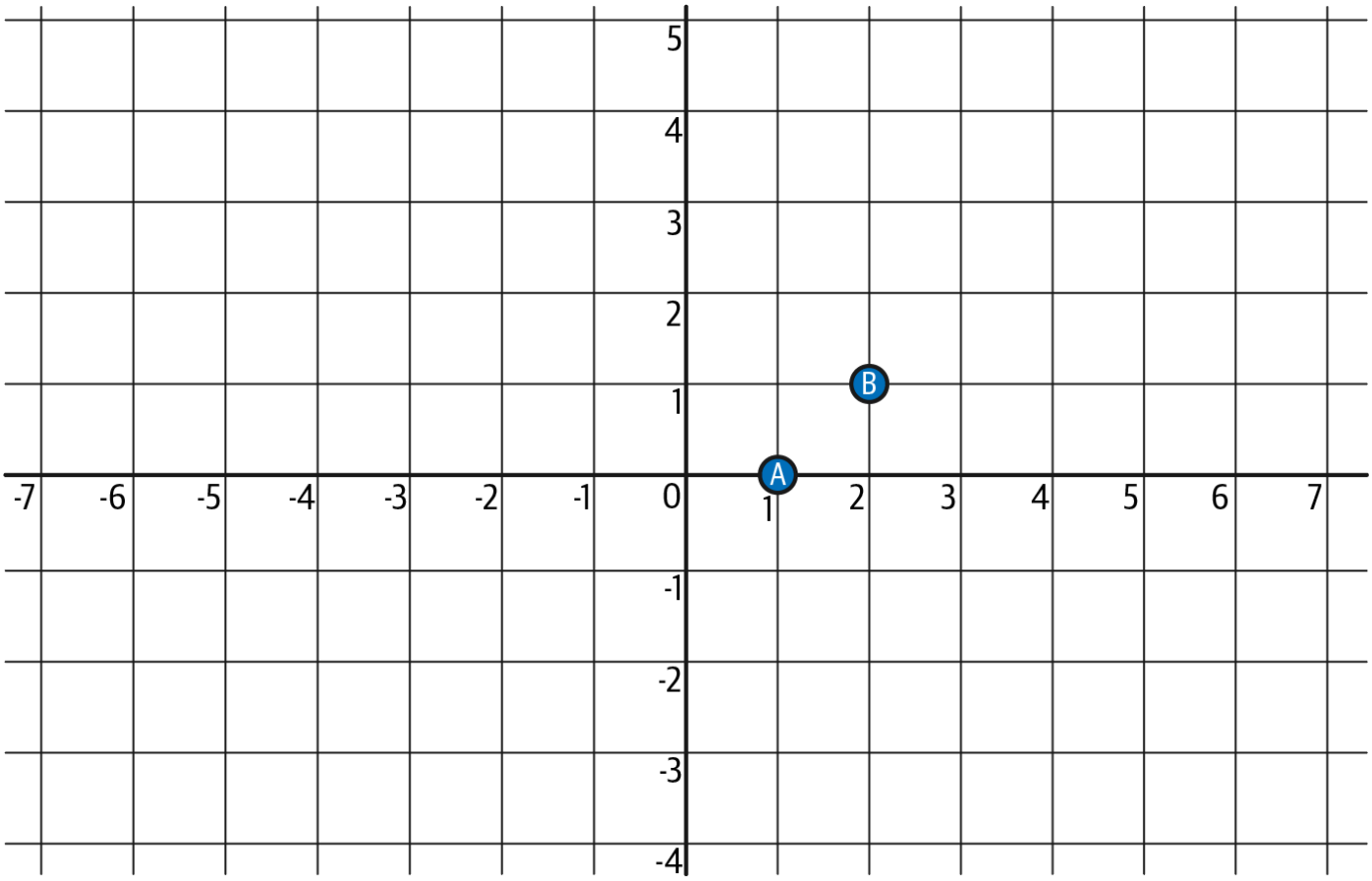


図4-11 2つの異なる点AとB

AとBの両方を通る直線——1次の多項式（次数が点の数 2 より小さい）——が存在することは容易に理解できます。

その直線は：

```
I(x) = x – 1
```

図4-12はこれをよりよく示しています。

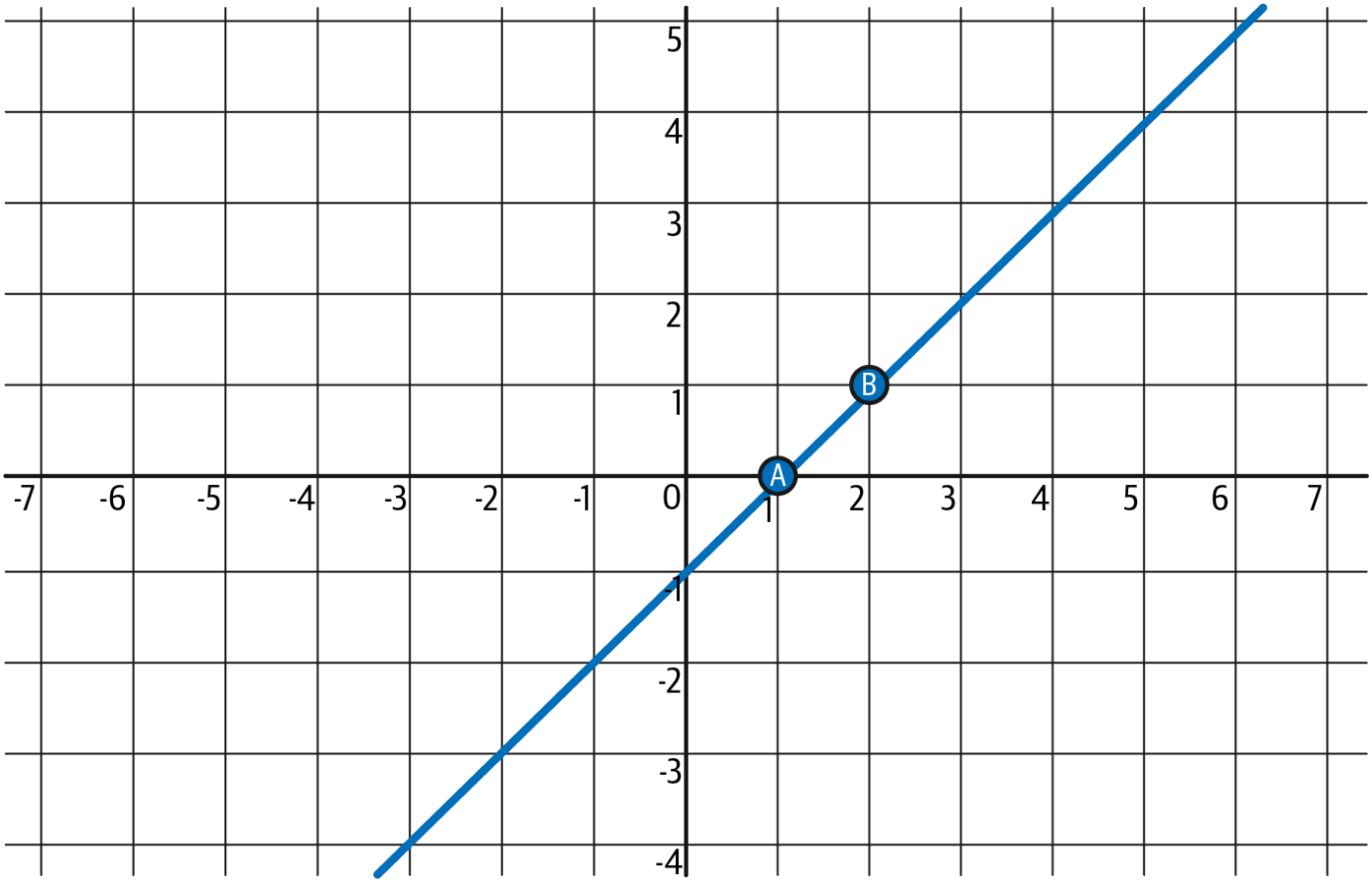


図4-12 点AとBを通る補間多項式（直線）

ここでは直感を使いましたが、公式を使って同じ結果を証明できます。$k = 2$であることを思い出してください。

$$
\begin{aligned}
I(x) &= y_0 \cdot (x - z_1)/(z_0 - z_1) + y_1 \cdot (x - z_0)/(z_1 - z_0) \\
&= 0 \cdot (x - 2)/(1 - 2) + 1 \cdot (x - 1)/(2 - 1) \\
&= 0 + (x - 1) = x - 1
\end{aligned}
$$

証明者は$p(x)$がこのセクションの冒頭で述べたすべての点を通ることを証明しようとしています。この主張が正しい（つまり証明者が信頼できる）と仮定すると、$p(x) - I(x)$は各$z_0, z_1, \cdots, z_{k-1}$の点でゼロになります。

つまり、$(x - z_0)(x - z_1)...(x - z_{k-1})$で割り切れます。この多項式を*ゼロファイア*$Z(x)$と呼びます。

通常の方法で進めます。証明者は商多項式$q(x)$を計算します。

$$q(x) = [p(x) - I(x)] / Z(x)$$

そして証明$\pi = [q(s)]_1$を生成します。この証明は通常の（単一の楕円曲線の点の）KZG証明であることに注意してください。

この証明の有効性を確認するために、検証者は以下を行う必要があります。

1. 先ほど説明した公式を適用して $I(x)$ を計算し、次に $[I(s)]_1$ を計算する。
2. $Z(x)$ を計算し、次に $[Z(s)]_2$ を計算する。なお検証者は、証明者が証明したいと主張するすべての点（と値）を知っているため、$Z(x)$ を計算できる。
3. そして通常の等価性チェックを適用する：

$$e(\pi, [Z(s)]_2) = e(C - [I(s)]_1, H)$$

この等式をペアリング群に書き下すと：

$$q(s)Z(s) = p(s) - I(s)$$

これは、証明者が以前に商多項式を計算したときと同じ等式であり、唯一の違いは秘密の点 s で評価されていることです。

まとめ

KZGコミットメントスキームにおける証明者↔検証者のフロー全体をまとめましょう。

まず証明者から始めます。証明者はあるデータの知識を持っています。証明者はこのデータを多項式にエンコードします（図4-13参照）。

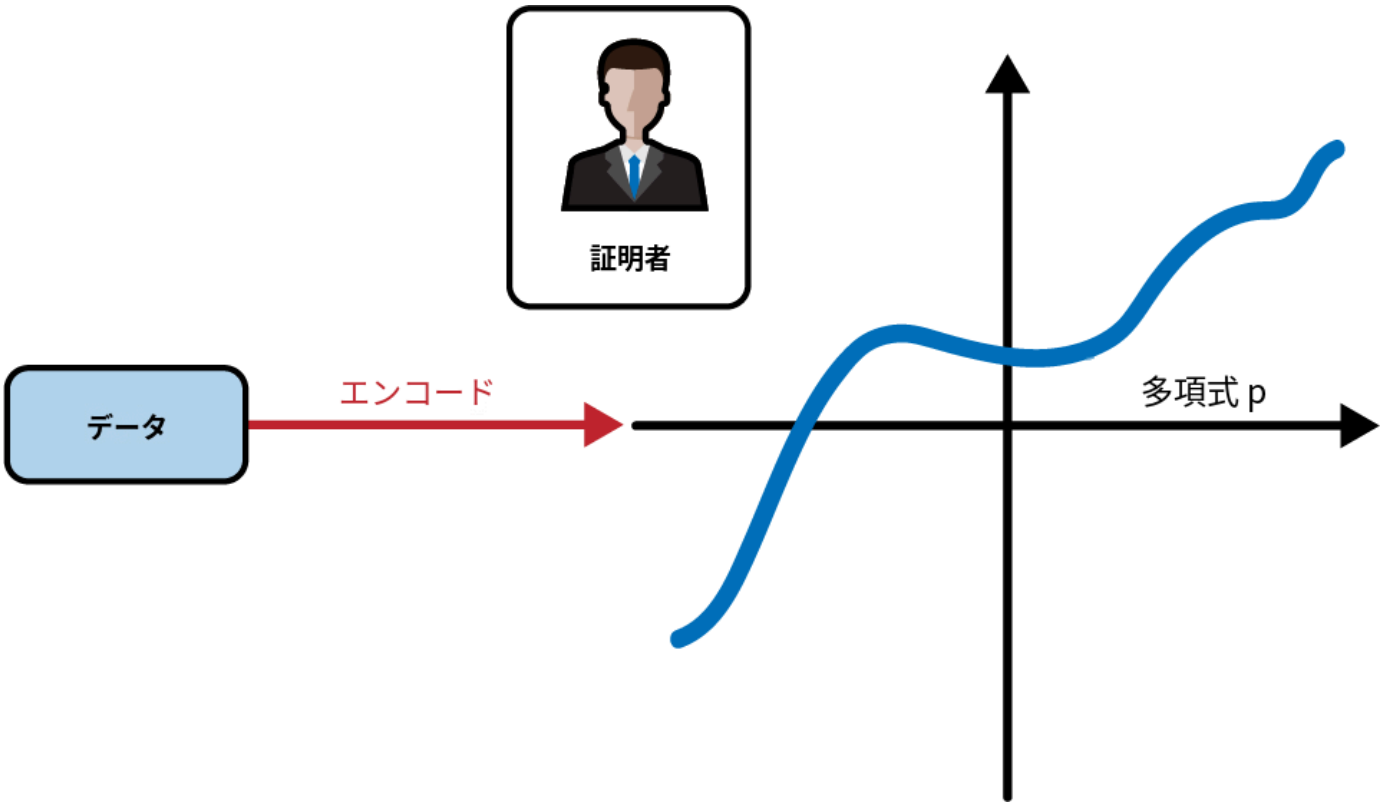


図4-13 証明者がデータを多項式pにエンコードする

証明者の目標は、ある特定のデータ（現在は多項式としてエンコードされている）を知っていることを検証者に納得させることです。しかし証明者はデータセット全体を明かしたくありませ

ん。代わりに、検証者がデータに関する特定の主張を検証できるようにしたいと考えています。特に、多項式が特定の点で期待される値をとることを検証者に確認してもらいたいと考えています（図4-14参照）。

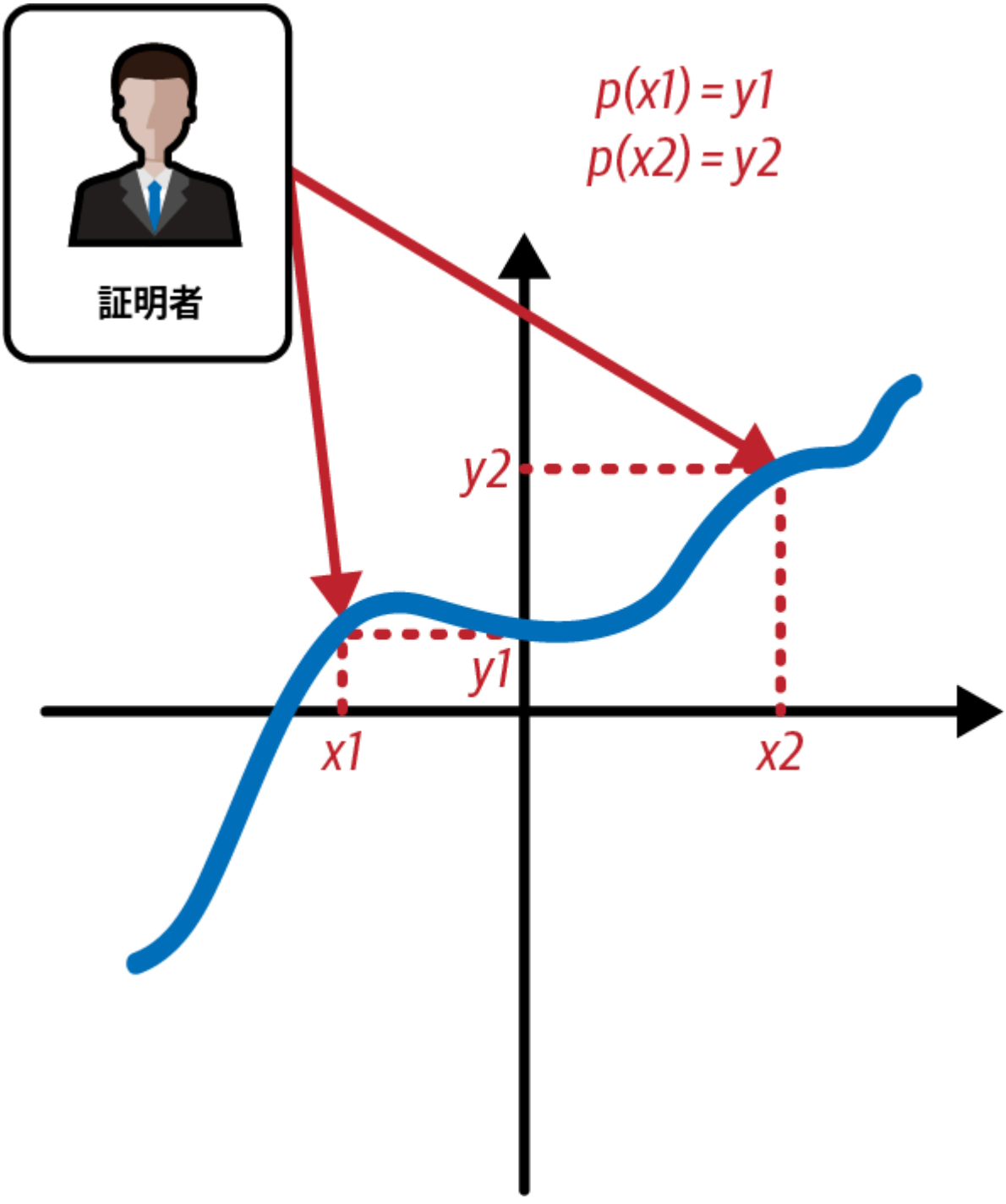


図4-14 証明者が特定の点での多項式の評価を証明したい

この目標を達成するために、証明者は多項式のKZGコミットメントを計算し、検証者に送信します（図4-15参照）。

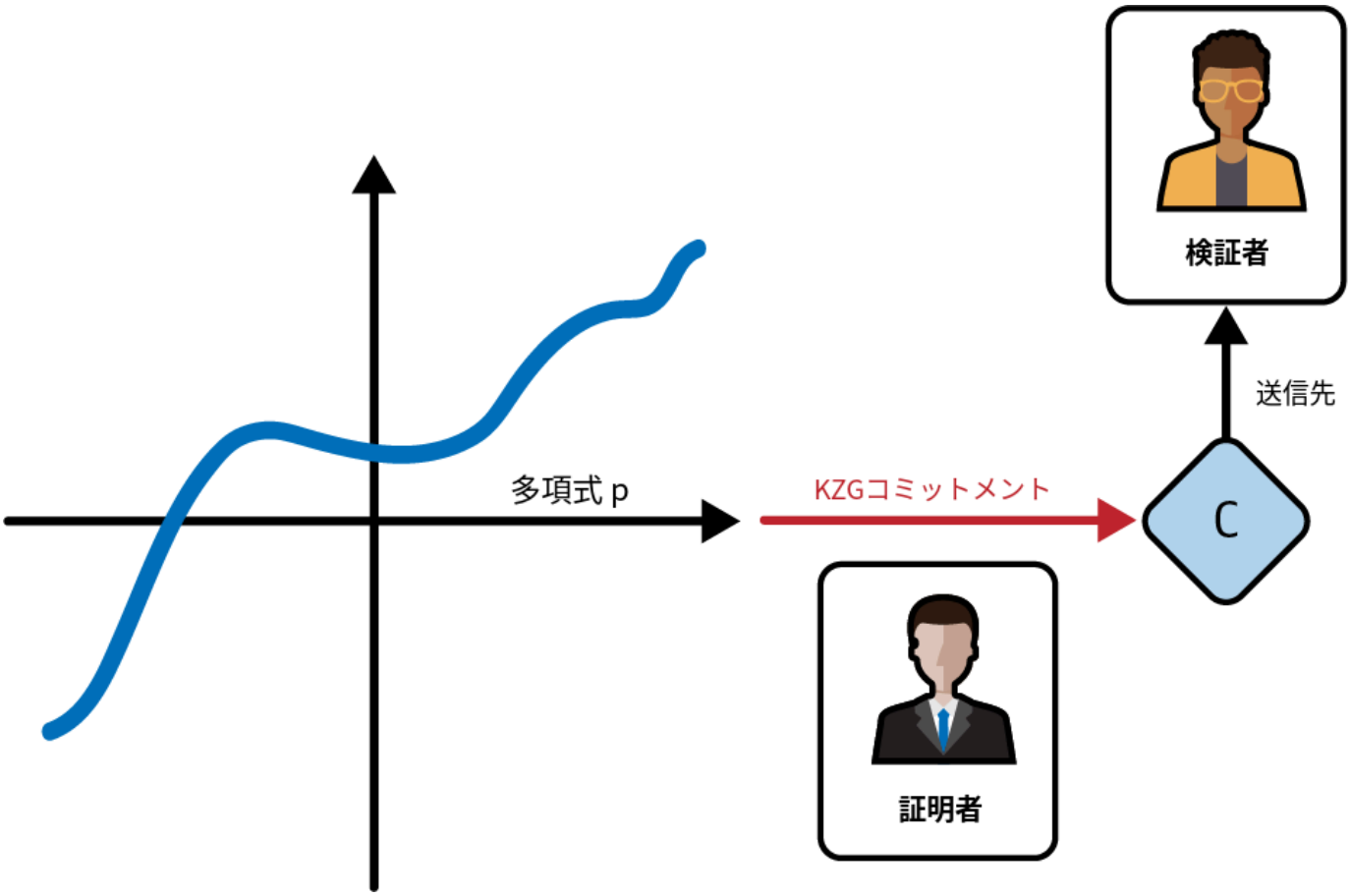


図4-15 証明者がKZGコミットメントCを計算し、検証者に送信する

次に、証明者は検証者に証明したいすべての評価に対するKZG証明を計算する必要があります（図4-16参照）。

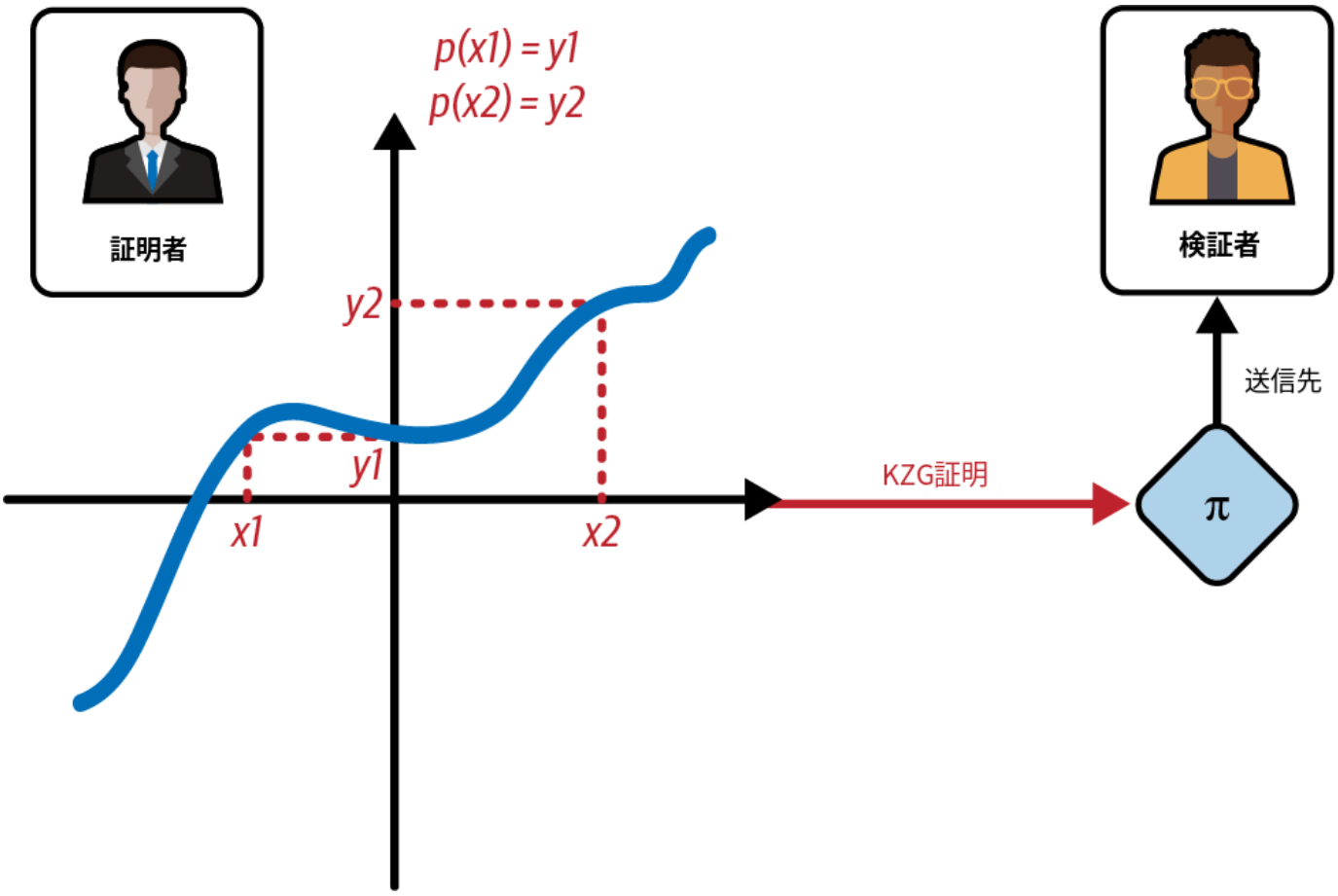


図4-16 証明者が多項式のさまざまな評価のKZG証明を計算し、検証者に送信する

　ここで検証者の番です。証明者が正直であることを確認するために、検証者は楕円曲線のペアリングチェックを計算する必要があります。このとき、証明者が以前に送信した情報とトラステッドセットアップを使います（図4-17参照）。

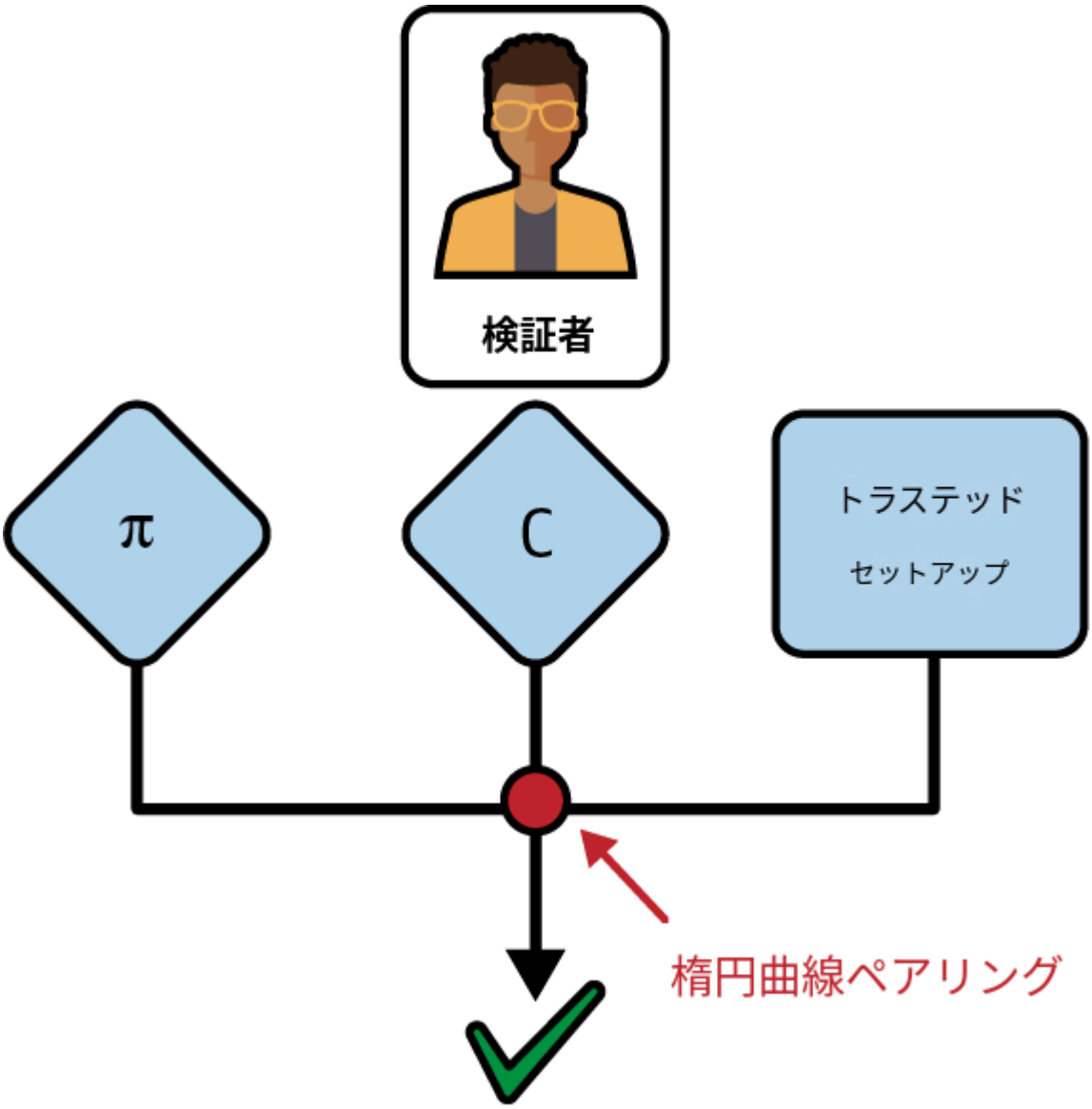


図4-17 検証者が楕円曲線のペアリング演算を通じて評価の有効性を確認する

結論

PKCの概要を紹介し、イーサリアムにおける公開鍵と秘密鍵の使用、そしてイーサリアムアドレスの作成と検証で用いられるハッシュ関数などの暗号ツールに焦点を当てました。また、デジタル署名が秘密鍵を明かさずにその所有を示す方法についても見ました。第5章ではこれらの考えを総合し、ウォレットを使って複数の鍵をまとめて管理する方法を見ていきます。

第5章：ウォレット

「ウォレット」という用語は、イーサリアムにおいていくつかの異なるものを表すために使われます。広義には、ウォレットとはイーサリアムへの主要なユーザーインターフェイスとして機能するソフトウェアアプリケーションのことです。ウォレットはユーザーの資金へのアクセスを制御し、鍵とアドレスの管理、残高の追跡、トランザクションの作成と署名を行います。さらに、一部のイーサリアムウォレットはERC-20トークンなどのコントラクトとやり取りすることもできます。

プログラマーの観点からより狭い意味で言えば、「ウォレット」という用語はユーザーの鍵を保管・管理するためのシステムを指します。すべてのウォレットは鍵を管理する機能を備えていますが、中には鍵の管理機能しか持たないウォレットもあります。他のウォレットはブラウザというはるかに広いカテゴリーに属しています。ブラウザとはイーサリアムベースのDAppへのインターフェイスのことで、第12章で詳しく解説します。「ウォレット」という用語でまとめられているさまざまなカテゴリーの間には、明確な線引きはありません。

この章では、秘密鍵を保存するコンテナとして、また鍵を管理するシステムとして、ウォレットを解説します。

ウォレット技術の概要

この節では、利便性、安全性、柔軟性を兼ね備えたイーサリアムウォレットを実現するために使われているさまざまな技術の概要を説明します。

ウォレット設計における重要な検討事項の1つは、利便性とプライバシーのバランスをとることです。最も使いやすいイーサリアムウォレットとは、常に1つの秘密鍵とアドレスを再利用するものでしょう。残念ながら、そのような設計はプライバシーの面では悪夢のようなものです。誰もがトランザクションを簡単に追跡し、関連付けられるからです。すべてのトランザクションに対して新しい鍵を使うことがプライバシーの観点からは最も望ましいのですが、管理は非常に難しくなります。適切なバランスをとることは困難な課題であり、だからこそ優れたウォレット設計が重要になります。

イーサリアムに関するよくある誤解は、「イーサリアムのウォレットはイーサやトークンを保管している」というものです。非常に厳密に言えば、ウォレットは鍵しか保管していません。イーサや他のトークンはイーサリアムブロックチェーンに記録されています。ユーザーは自身のウォレットにある鍵でトランザクションに署名することで、ネットワーク上のトークンを制御します。ある意味、イーサリアムのウォレットは*キーチェーン*なのです。とはいえ、ウォレットに保管されている鍵はほかの人にイーサやトークンを転送するために必要な唯一のものであるため、実際にはこの区別はほとんど意味がありません。

この違いが重要になるのは、考え方を従来の銀行の中央集権システムからブロックチェーンプラットフォームの分散型システムへ転換するときです。従来の銀行では、本人と銀行だけが口座の資金を確認でき、取引のために資金を移動するには銀行を説得するだけでよいものでした。一方、ブロックチェーンの分散型システムでは、誰もがアカウントのイーサ残高を見られますが、アカウントの所有者はおそらくわからず、トランザクションを実行するには所有者が資金の移動を望んでいることを全員が納得する必要があります。実際にはこれは、ウォレットを必要とせずにアカウントの残高を確認する独立した方法があることを意味します。さらに、最初に使い始めたウォレットアプリが気に入らない場合は、現在のウォレットから別のウォレットにアカウント管理を移行することもできます。

注記

イーサリアムのウォレットには、イーサやトークンではなく、鍵が保管されています。ウォレットは秘密鍵と公開鍵のペアを含むキーチェーンのようなものです。ユーザーは秘密鍵を使ってトランザクションに署名し、それによってイーサを所有していることを証明します。イーサはブロックチェーン上に保管されています。

ウォレットには、主に2つの種類があります。両者の区別は、ウォレットが保持している複数の鍵が互いに関連しているかどうかによります。

1つ目のタイプは*非決定性ウォレット*であり、それぞれの鍵は異なる乱数から独立に生成されます。鍵は互いに関連していません。このタイプのウォレットは「just a bunch of keys（鍵の束）」というフレーズから*JBOKウォレット*とも呼ばれています。

2つ目のタイプは*決定性ウォレット*です。すべての鍵は*シード*と呼ばれる1つのマスター鍵から導出されます。決定性ウォレット内のすべての鍵は互いに関連しており、元のシードがあれば再び生成することができます。決定性ウォレットではいくつかの異なる鍵導出方法が使用されています。最も一般的な導出方法は、「階層的決定性ウォレット（BIP-32/BIP-44）」で説明しているように、ツリー状の構造を用いたものです。

携帯電話の盗難や水没などのデータ消失に備えて、決定性ウォレットの安全性を少しでも高めるために、シードを単語リスト（英語など）にエンコードして書き留めておくことがよくあります。この単語のリストはウォレットの*ニーモニックコードワード*として知られています。もちろん、第三者がニーモニックコードワードを入手した場合、ウォレットを再生成してイーサやスマートコントラクトにアクセスすることもできます。したがって、リカバリー用の単語リストの取り扱いには細心の注意を払ってください！電子的に保存してはいけません。コンピュータや携帯電話のファイルに保管してはいけません。紙に書き留め、安全で確実な場所に保管してください。

次項以降では、それぞれのウォレット技術について概説します。

非決定性（ランダム）ウォレット

最初のイーサリアムウォレットはイーサリアムの先行販売のために作られ、それぞれのウォレットファイルにはランダムに生成された単一の秘密鍵が格納されていました。このような「古いスタイル」の非決定性ウォレットは多くの点で不備や欠点があり、決定性ウォレットで置き換えられつつあります。たとえば、イーサリアムを使用する際は、プライバシーを最大限に保護するためにも、イーサリアムアドレスの再利用は避けたほうがよいでしょう。資金を受け取る場合は、都度新しいアドレス（新しい秘密鍵が必要）を使うようにしてください。あるいは、各トランザクションに対して新しいアドレスを使用することもできますが、トークンを大量に取り扱う場合にはコストが高くなります。この方法を採用する場合、非決定性ウォレットは定期的に鍵のリストを増やす必要があります。つまり、定期的にバックアップを取る必要があります。ウォレットのバックアップを取る前にデータを失った場合（ディスクの故障、飲み物をこぼす、携帯電話の盗難）、資金とスマートコントラクトにアクセスできなくなります。

「Type-0」の非決定性ウォレットは、「使い捨て」のような方法で新しいアドレスごとに新しいウォレットファイルを作成するため、取り扱いがかなり面倒になります。

それにもかかわらず、多くのイーサリアムクライアント（Gethを含む）は*キーストアファイル*を使用しています。キーストアファイルはJSONエンコードされたファイルであり、ランダムに生成された秘密鍵が1つ含まれ、セキュリティを強化するためにパスフレーズで暗号化されています。JSONファイルの内容は次のようになります：

```
{
    "address": "001d3f1ef827552ae1114027bd3ecf1f086ba0f9",
    "crypto": {
        "cipher": "aes-128-ctr",
        "ciphertext":
            "233a9f4d236ed0c13394b504b6da5df02587c8bf1ad8946f6f2b58f055507ece",
        "cipherparams": {
            "iv": "d10c6ec5bae81b6cb9144de81037fa15"
        },
        "kdf": "scrypt",
        "kdfparams": {
            "dklen": 32,
            "n": 262144,
            "p": 1,
            "r": 8,
            "salt":
                "99d37a47c7c9429c66976f643f386a61b78b97f3246adca89abe4245d2788407"
        },
        "mac": "594c8df1c8ee0ded8255a50caf07e8c12061fd859f4b7c76ab704b17c957e842"
    },
    "id": "4fcb2ba4-ccdb-424f-89d5-26cce304bf9c",
    "version": 3
}
```

キーストア形式では、*パスワードストレッチングアルゴリズム*とも呼ばれる*鍵導出関数*（KDF）を使用しており、ブルートフォース（総当たり）攻撃、辞書攻撃、レインボーテーブル攻撃から保護します。簡単に言えば、秘密鍵はパスフレーズで直接暗号化はされません。その代わり、パスフレーズに対して繰り返しハッシュ関数を実行するストレッチングが行われます。ハッシュ関数は262,144ラウンド繰り返されており、キーストアJSONでは `crypto.kdfparams.n` として確認できます。パスフレーズをブルートフォース攻撃しようとする攻撃者は、試行するパスフレーズごとに262,144回のハッシュを適用しなければなりません。この仕組みにより攻撃は十分に遅くなります。結果として、十分な複雑さと長さを持つパスフレーズは実質的に破られなくなります。

> **ヒント**
>
> 非決定性ウォレットの使用は、単純なテスト以外の用途では推奨されません。ごく基本的な状況以外では、バックアップして使うには扱いにくすぎます。代わりに、バックアップのためにニーモニックシードを用いる業界標準ベースの階層的決定性ウォレットを使用してください。

決定性（シード）ウォレット

決定性ウォレット、または「シード（seeded）」ウォレットは、すべて1つのマスター鍵もしくはシードから導出された秘密鍵が入ったウォレットです。シードとは、ランダムに生成された数値で、インデックス番号または「チェーンコード」（「拡張公開鍵と拡張秘密鍵」を参照）といった他のデータと組み合わせて複数の秘密鍵を生成します。決定性ウォレットでは、シードがあれば生成したすべての鍵を復元できます。そのため、シードの生成時にバックアップを1回取るだけで、ウォレット内のすべての資金とスマートコントラクトを保護できます。このシードはウォレットのエクスポートとインポートに適しており、異なる実装のウォレット間において鍵の移行を容易にしてくれます。

この設計では、シードのセキュリティが最も重要になります。シードだけあればウォレット全体へのアクセスを得ることができるからです。一方で、セキュリティ対策を1つのデータに集中できるという点は利点とも言えます。

階層的決定性ウォレット（BIP-32/BIP-44）

決定性ウォレットは、1つのシードから多くの鍵を生成しやすくするために開発されました。現在、決定性ウォレットの最も進んだ形式は、ビットコインのBIP-32標準で定義されている階層的決定性（HD）ウォレットです。HDウォレットには、ツリー構造から導出された鍵が含まれています。この構造においては、親鍵が子鍵群を生成し、子鍵がさらに孫鍵群を生成する、という連鎖が続きます。このツリー構造を図5-1に示します。

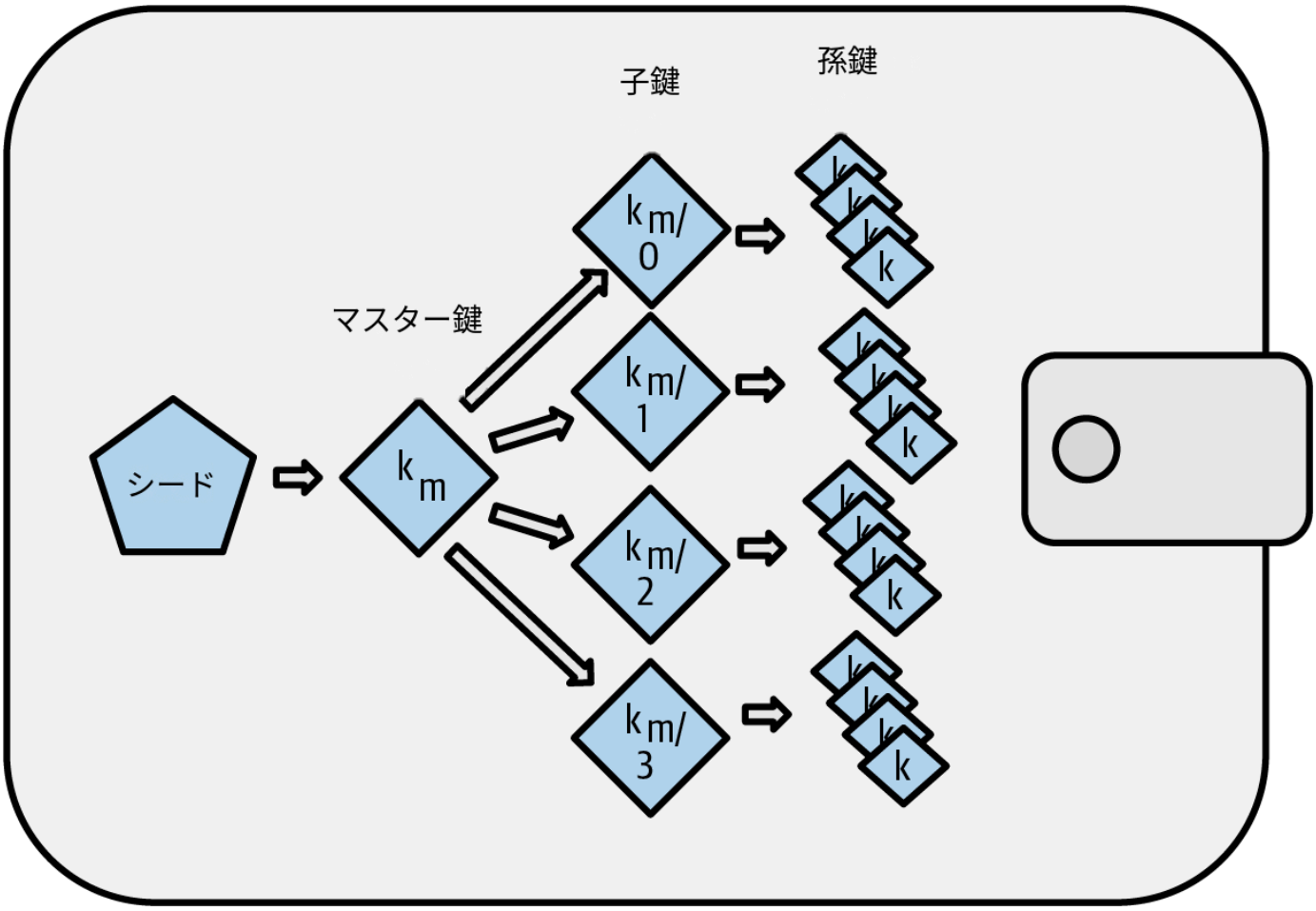


図5-1. HDウォレット：1つのシードから生成された鍵ツリー

HDウォレットには、単純な決定性ウォレットに比べて、いくつかの大きな利点があります。1つ目の利点は、ツリー構造を用いることで用途を使い分けられることです。たとえば、ある特定の鍵の「ブランチ」を支払いの受け取りとして利用したり、他の「ブランチ」を支払いのお釣りの受け取りとして使ったりできます。他にも、会社内での使い分けとして、部や課などの部署ごと、特定の機能ごと、口座の種別ごとにブランチを割り当てることもできます。

HDウォレットのもう1つの利点は、対応する秘密鍵にアクセスすることなく公開鍵群を生成できることです。これにより、秘密鍵を持たないサーバー（安全でないサーバーや閲覧・受信専用サーバー）でもHDウォレットを利用できます。

シードとニーモニックコード（BIP-39）

安全なバックアップと復元のために秘密鍵をエンコードする方法はたくさんあります。現在推奨されている方法は、正しい順序でまとめると秘密鍵を一意に再生成できる一連の単語を使用することです。これはニーモニックと呼ばれることがあり、BIP-39で標準化されています。今日では、多くのイーサリアムウォレット（および他の暗号通貨ウォレット）がこの標準を使用しており、相互運用可能なニーモニックを用いてバックアップおよびリカバリー用のシードをインポート・エクスポートすることができます。

このアプローチが普及した理由を理解するために、以下の例を見てみましょう：

```
FCCF1AB3329FD5DA3DA9577511F8F137 ← 16進数表記のシード
wolf juice proud gown wool unfair
wall cliff insect more detail hub ← BIP-39表記のシード
```

実際の問題として、16進数の文字列を書き留める際に転記ミスが発生する確率は容認できないほど高くなります。その一方で、既知の単語のリストは扱いがとても簡単です。主な理由は、単語（特に英語の単語）の書き取りには高い冗長性があるためです。もし誤って*inzect*が記録された場合、ウォレットのリカバリーの際に*inzect*は正しい英単語ではなく、代わりに*insect*が使用されるべきだとすぐに判断できます。HDウォレット管理時にはシードを書き留めることが推奨されています。データ消失（事故や盗難）時のリカバリーに必要だからです。バックアップを取っておくことは非常に賢明です。ただし、シードは極めて秘密に保たなければならないため、デジタルバックアップは慎重に避けるべきです。ペンと紙でバックアップすることをおすすめします。

要約すると、HDウォレットのシードをエンコードしたリカバリーワードリストを使用することが、秘密鍵のセットを安全にエクスポートし、転記し、紙に記録し、エラーなく読み取り、別のウォレットにインポートするための最も簡単な方法です。

ウォレットのベストプラクティス

暗号通貨ウォレットの技術が成熟するにつれて、共通の業界標準が現れました。これらの標準により、幅広い相互運用性と使いやすさ、安全性、柔軟性を備えたウォレットを設計できるようになりました。これらの標準により、ウォレットは、1つのニーモニックから複数の異なる暗号通貨の鍵を生成することもできます。共通標準として次のものがあります。

- ニーモニックコードワード（BIP-39準拠）
- HDウォレット（BIP-32準拠）
- 多目的HDウォレット構造（BIP-43準拠）
- 多通貨、複数アカウントウォレット（BIP-44準拠）

これらの標準は将来の開発によって変わるかもしれませんし、また時代遅れになるかもしれません。しかし、今のところこれらの標準は、それぞれに連動した技術を形成しており、ほとんどのブロックチェーンプラットフォームとそれらの暗号通貨のデファクトスタンダードになっています。

幅広いソフトウェアウォレットおよびハードウェアウォレットがこれらの標準を採用しており、すべて相互運用が可能です。ユーザーは、これらのウォレットの1つで生成されたニーモニックをエクスポートし、別のウォレットにインポートすることで、すべての鍵とアドレスをリカバリーできます。

これらの標準をサポートするソフトウェアウォレットの例としては、Rabby Wallet、MetaMask、Phantomがあります。ハードウェアウォレットの例としては、LedgerおよびTrezor

の各種モデルがあります。

以下では、これらの技術のそれぞれの詳細について説明します。

ヒント

もしイーサリアムウォレットを実装する場合は、以下で説明するBIP-32、BIP-39、BIP-43、BIP-44の標準に従い、バックアップのニーモニックコードとしてエンコードされたシードを持つHDウォレットとして構築する必要があります。

ニーモニックコードワード（BIP-39）

ニーモニックコードワードとは、決定性ウォレットを生成する際にシードとして使用される乱数をエンコードした単語のリストです。この単語のリストがあればシードを再生成できます。そして、再生成したシードを用いて、ウォレットとすべての鍵を再生成できます。ニーモニックワード方式の決定性ウォレットを実装するアプリケーションは、初期設定時に12〜24個の単語リストをユーザーに表示します。この単語のリストはウォレットのバックアップであり、同一もしくは互換性のあるウォレットにおいて、すべての鍵を復元・再生成するために使うことができます。前節で説明したように、ニーモニックワードのリストは、可読性が高く、正確に転記できます。そのためユーザーは簡単にウォレットをバックアップすることができます。

注記

ニーモニックワードはよく*ブレインウォレット*と混同されるが、両者は同じものではない。主な違いは、ブレインウォレットがユーザーの選んだ単語で構成されるのに対し、ニーモニックワードはウォレットによってランダムに生成されてユーザーに提示される点である。この違いにより、ニーモニックワードははるかに安全である。人間はランダム性の生成源として非常に乏しいためである。さらに重要な点として、*ブレインウォレット*という用語は単語を暗記すべきであることを示唆している。これは悪い考えであり、必要なときにバックアップが手元にないという事態を招く。

ニーモニックコードはBIP-39で定義されています。ただし、BIP-39はニーモニック標準の実装方式の1つでしかないことに注意してください。ビットコインウォレットのElectrumは、BIP-39に先行して、別の単語群を持った標準を採用しています。BIP-39は、Trezorハードウェアウォレットを製造している会社によって提案されたもので、Electrumとの実装には互換性はありません。しかしながらBIP-39は、現在、数多くの相互運用可能な実装によって幅広い業界のサポートを得ており、デファクトスタンダードとみなせます。さらに、BIP-39はイーサリアム対応の多通貨ウォレット実装に使えます。一方でElectrumのシードは使えません。

BIP-39は、ニーモニックコードとシードの生成手順を定義しています。ここでは9ステップで説明します。理解しやすくするために、生成手順を2つの部分に分けています。ステップ1〜6は

「ニーモニックワードの生成」に、ステップ7〜9は「ニーモニックからシードへ」に記載しています。

ニーモニックワードの生成

ウォレットはBIP-39で定義された標準化されたプロセスを使用して、ニーモニックワードを自動的に生成します。ウォレットはエントロピー源から開始し、チェックサムを追加した後、以下のステップに従ってエントロピーを単語リストにマッピングします：

1. 128〜256ビットの暗号論的にランダムなシーケンス*S*を生成する
2. *S*のSHA-256ハッシュの先頭（*S*のビット長÷32）ビットを取り、*S*のチェックサムを生成する
3. チェックサムをランダムなシーケンス*S*の末尾に追加する
4. シーケンスとチェックサムの連結を11ビットずつのセクションに分割する
5. 各11ビットの値を、あらかじめ定義された2,048語を持つ辞書の中の単語にマッピングする
6. 順序を維持しながら、一連の単語からニーモニックコードを生成する

図5-2は、エントロピーを使用してニーモニックワードを生成する方法を示しています。

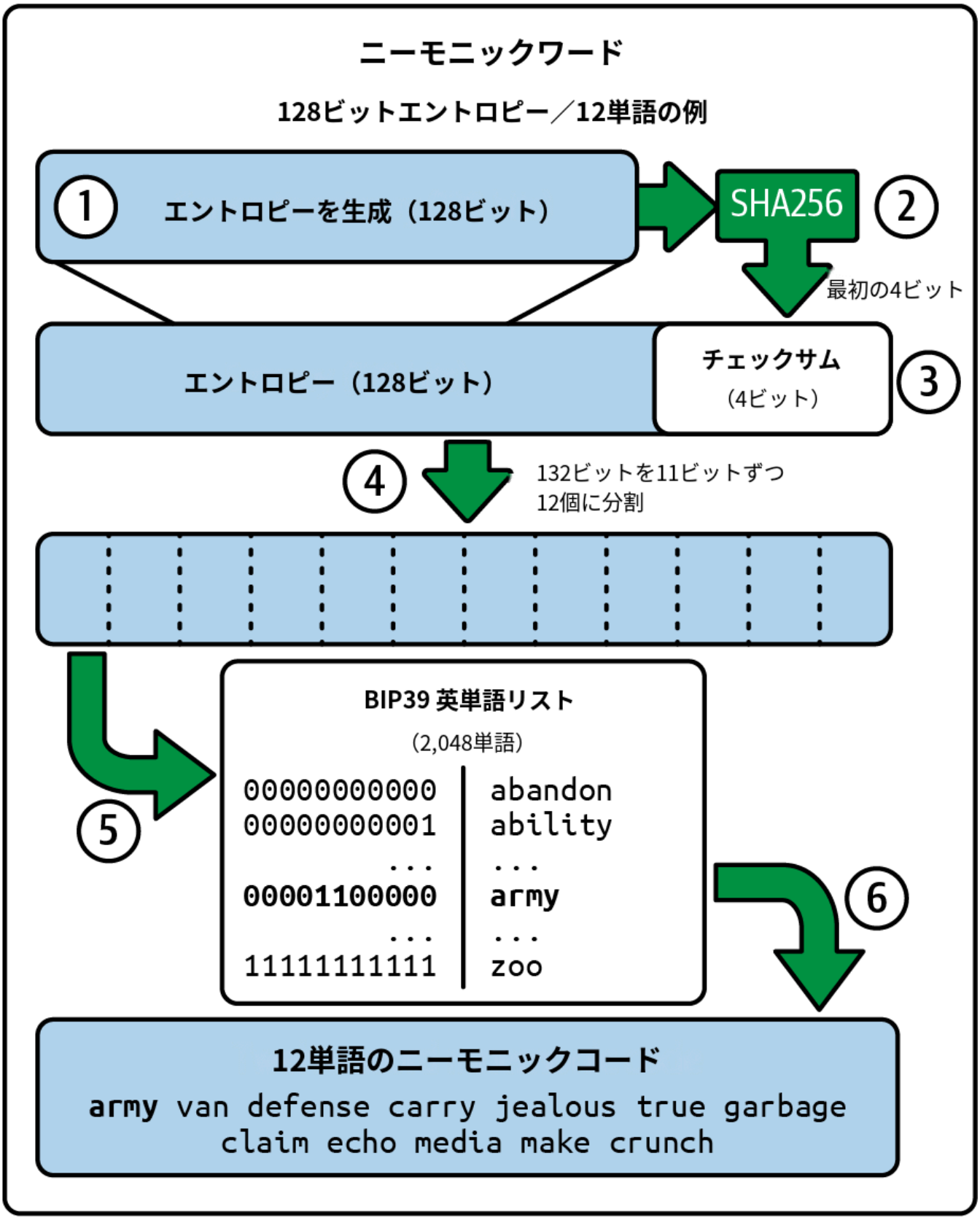


図5-2. ニーモニックワードの生成

　表5-1は、エントロピーデータのサイズとニーモニックコードの長さ（単語数）の関係を示しています。

表5-1 ニーモニックコード：エントロピーと単語長

エントロピー（ビット）	チェックサム（ビット）	エントロピー＋チェックサム（ビット）	ニーモニックの長さ（単語数）
128	4	132	12
160	5	165	15
192	6	198	18
224	7	231	21
256	8	264	24

ニーモニックからシードへ

ニーモニックワードは、128〜256ビットの長さを持つエントロピーを表しています。このエントロピーは、鍵ストレッチ関数PBKDF2を用いてより長い（512ビット）シードを導出するために使われます。生成されたシードは、決定性ウォレットを構築し、そのウォレットの鍵を生成するために使われます。

鍵ストレッチ関数には2つの引数があります。ニーモニックとソルトです。鍵ストレッチ関数におけるソルトの目的は、ブルートフォース攻撃を可能にするルックアップテーブルの構築を困難にすることです。BIP-39標準では、ソルトにはもう1つの目的があります。それは、シードを保護する追加のセキュリティ要素として機能するパスフレーズの導入を可能にすることで、次のセクションで詳しく説明します。

プロセスは前のセクションから続き、以下のステップで進みます：

7. 鍵ストレッチ関数PBKDF2の最初の引数は、ステップ6で生成したニーモニックである
8. 鍵ストレッチ関数PBKDF2の2つ目の引数は*ソルト*である。このソルトは、文字列定数`"mnemonic"`とユーザーが提供するオプションのパスフレーズの連結で構成される
9. PBKDF2は、HMAC-SHA512アルゴリズムで2,048ラウンドのハッシュ化を行い、ニーモニックとソルトのパラメータを拡張し、最終出力として512ビットの値を生成する。その512ビットの値がシードとなる

図5-3にニーモニックを使ってシードを生成する方法を示します。

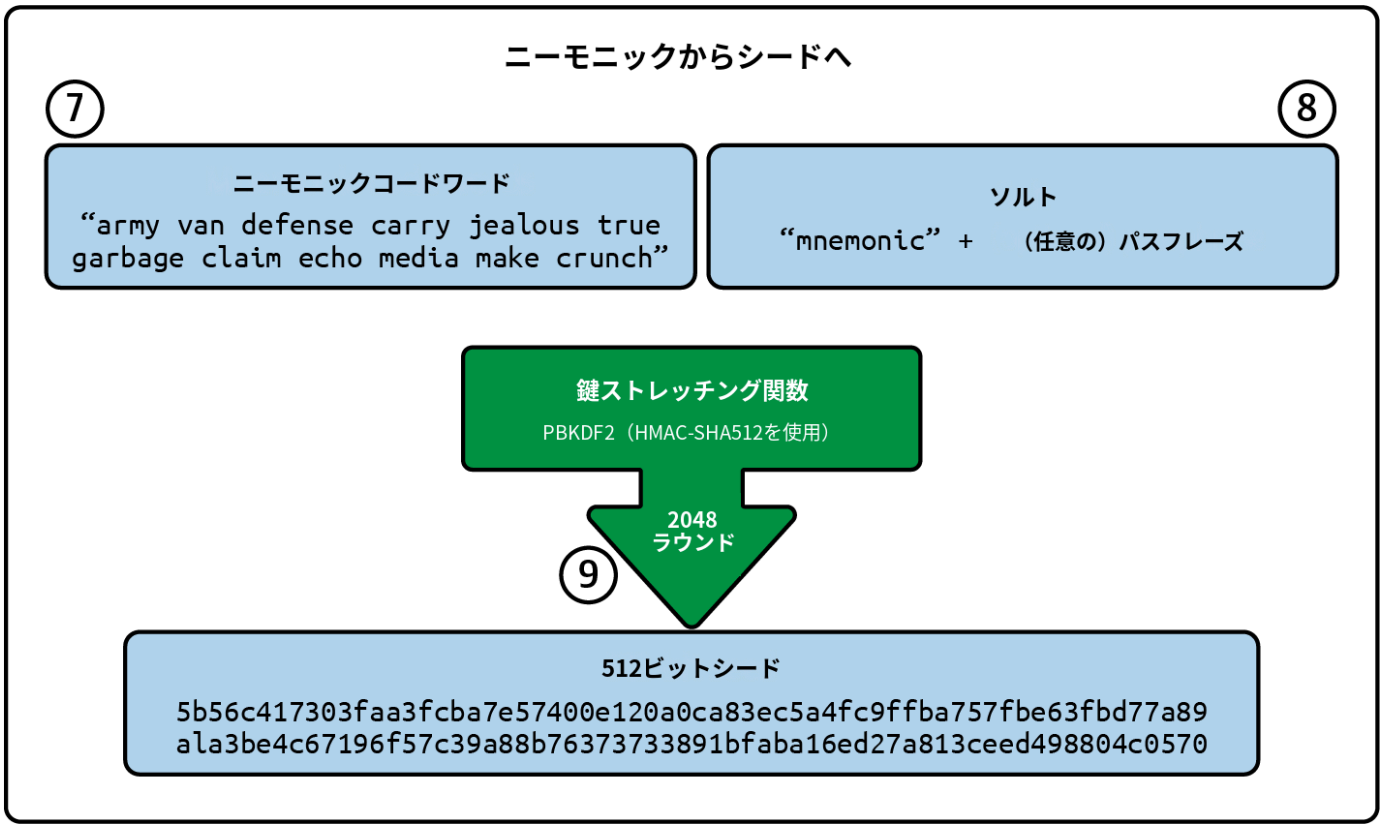


図5-3. ニーモニックからシードへ

注記

2,048ラウンドのハッシュ化を行う鍵ストレッチ関数は、ニーモニックやパスフレーズへのブルートフォース攻撃に対していくらか効果的な保護手段です。数千ものパスフレーズやニーモニックの組み合わせを試すのにはかなりの計算量となりコストがかかります。一方で、取り得るシードの数は膨大です（2^512^、すなわち約10^154^）。これは可視宇宙に存在する原子の数（約10^80^）よりもはるかに多いのです。

表5-2から表5-4にニーモニックコードとそれらが生成するシードの例を示します。

表5-2 128ビットエントロピーのニーモニックコード（パスフレーズなし）と生成されるシード

エントロピー	0c1e24e5917779d297e14d45f14e1a1a
ニーモニック	army van defense carry jealous true garbage claim echo media make crunch
パスフレーズ	（なし）
シード	5b56c417303faa3fcba7e57400e120a0ca83ec5a4fc9ffba757fbe63fbd77a89a1a3be4

表5-3 128ビットエントロピーのニーモニックコード（パスフレーズあり）と生成されるシード

エントロピー	0c1e24e5917779d297e14d45f14e1a1a
ニーモニック	army van defense carry jealous true garbage claim echo media make crunch
パスフレーズ	SuperDuperSecret
シード	3b5df16df2157104cfdd22830162a5e170c0161653e3afe6c88defeefb0818c793dbb28

表5-4 256ビットエントロピーのニーモニックコード（パスフレーズなし）と生成されるシード

エントロピー	2041546864449caff939d32d574753fe684d3c947c3346713dd8423e74abcf8c
ニーモニック	cake apple borrow silk endorse fitness top denial coil riot stay wolf luggage oxygen measure invite love trap field dilemma oblige
パスフレーズ	（なし）
シード	3269bce2674acbd188d4f120072b13b088a0ecf87c6e4cae41657a0bb78f5315b33b3

BIP-39のオプションのパスフレーズ

BIP-39標準では、シードの導出にオプションのパスフレーズを使用できます。パスフレーズが使用されない場合、ニーモニックワードは定数文字列「mnemonic」からなるソルトでストレッチ（引き伸ばし）され、任意のニーモニックワードから特定の512ビットのシードが生成されます。パスフレーズが使用されている場合、ストレッチ関数は同じニーモニックから別のシードを生成します。実際には、1つのニーモニックがあれば、ありとあらゆるパスフレーズから異なるシードが生成されます。基本的に「間違った」パスフレーズというものは存在しません。すべてのパスフレーズは有効であり、それらはすべて異なるシードを生成します。そして、膨大な数の未初期化ウォレットを作り出すことができます。生成可能なウォレットの数は2^512^と膨大です。パスフレーズの複雑さと長さが十分であれば、ブルートフォース攻撃で破られたり偶然推測されたりすることは事実上ありません。

ヒント

> BIP-39には「間違った」パスフレーズはありません。あらゆるパスフレーズはウォレットを生成することができます。つまり以前に使用されていない限り、新しいウォレットを生成できます。

オプションのパスフレーズは、2つの重要な機能があります。

- 第2の要素（暗記したもの）、つまりオプションのパスフレーズによって、ニーモニックを単独では使えないようにできる。これで、泥棒による情報漏洩からニーモニックのバックアップを保護できるようになる
- 一見もっともらしい形態を取ったウォレット、または「脅迫ウォレット（duress wallet）」を作ることができる。オプションのパスフレーズを使用することで、少額の資産しか入っていないウォレットも作れる。これは大半の資産が入っている「本物」のウォレットを攻撃からそらすために使える

しかしながら、パスフレーズを使うことには紛失の危険性があることに注意しなければなりません。

- もしウォレットの所有者が、活動することができないか死亡しており、誰もパスフレーズを知らない場合、そのシードは役に立たない。ウォレットに保存されているすべての資産は永遠に失われる
- 逆に、所有者がシードと同じ場所にパスフレーズをバックアップすると、シード生成のための第2の要素としての役割を果たさない

パスフレーズは非常に便利ですが、一方で、相続人が暗号通貨資産をリカバリーできるようにしておくなど、バックアップとリカバリーのために慎重に計画したプロセスと組み合わせてパスフレーズを使用する必要があります。

ニーモニックコードの活用

BIP-39は数多くのプログラミング言語でライブラリとして実装されています。以下に例を示します。

python-mnemonic

BIP-39を提案したSatoshiLabsチームによるリファレンス実装（Python）

ConsenSys/eth-lightwallet

ノードとブラウザ向けの軽量JavaScriptイーサリアムウォレット（BIP-39対応）

npm/bip39

ビットコインBIP-39のJavaScript実装

スタンドアロンのウェブページとして実装されたBIP-39ジェネレーターもあり（図5-4）、テストや実験に非常に便利です。Mnemonic Code Converterはニーモニック、シード、拡張秘密鍵

を生成します。ブラウザでオフラインで使用することも、オンラインでアクセスすることもできます。

> **注記**
>
> この章を通じて繰り返し強調されますが、シードのセキュリティにおいて最も重要なルールです：いかなる状況においても、シードフレーズをデジタル形式で保存してはいけません。

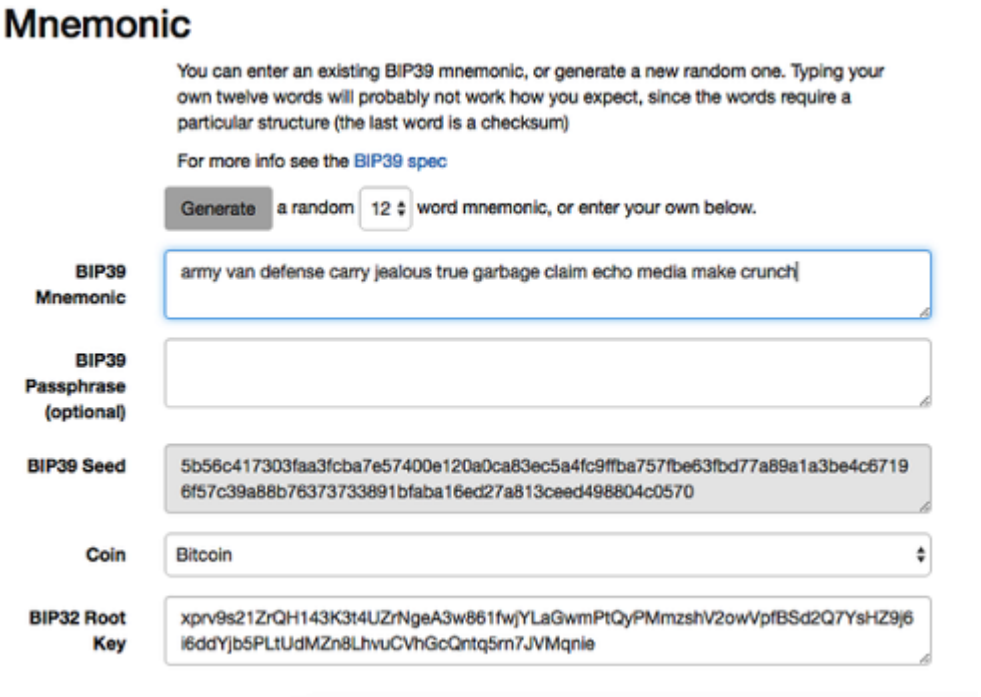


図5-4. BIP-39ジェネレーターのウェブページ

シードからのHDウォレットの生成

HDウォレットは、128ビットあるいは256ビット、512ビットの乱数である単一のルートシードから生成されます。通常、このシードは、前節で説明したニーモニックから生成されます。

HDウォレットにあるすべての鍵は、ルートシードから決定論的に導出されます。この仕組みにより、互換性のあるHDウォレットであればシードからウォレット全体を再生成できます。ルートシードを生成したニーモニックを転送するだけで、HDウォレット（数千から数百万の鍵を含むもの）のバックアップ・復元・エクスポート・インポートを容易に行えます。

HDウォレット（BIP-32）

ほとんどのHDウォレットはBIP-32標準に準拠しており、決定論的な鍵生成のデファクトスタンダードとなっています。ここではBIP-32のすべての詳細を議論するのではなく、ウォレットで

の使われ方を理解するために必要なコンポーネントのみを説明します。最も重要な点は、図5-1で見られるように、導出された鍵がツリー状の階層的な関係を持てることです。拡張鍵と強化鍵の概念を理解することも重要であり、これらは以降の節で説明します。

多くのソフトウェアライブラリで、相互運用可能なBIP-32の実装が数多く提供されています。これらの多くはビットコインウォレット向けに設計されており、アドレスの実装方法は異なりますが、イーサリアムのBIP-32互換ウォレットと同じ鍵導出の実装を共有しています。イーサリアム向けに設計されたものを使用するか、ビットコイン向けのものにイーサリアムアドレスのエンコーディングライブラリを追加して適用してください。

スタンドアロンのウェブページとして実装されたBIP-32生成器もあり、BIP-32のテストや実験に非常に便利です。

> **警告**
>
> スタンドアロンのBIP-32生成器はHTTPSサイトではありません。ここでHTTPSサイトでないことを強調しているのは、このツールの使用の安全性が保証されていないことを知ってもらうためです。あくまでもこのツールはテスト専用です。このサイトで生成した鍵は、本物の資金の運用で使うべきではありません。

拡張公開鍵と拡張秘密鍵

BIP-32の用語では、鍵は「拡張」することができます。適切な数学的演算を用いることで、これらの拡張された「親」鍵から「子」鍵を導出し、前述した鍵とアドレスの階層構造を生成できます。親鍵はツリーの最上位にある必要はなく、ツリー階層のどこからでも選択できます。鍵を拡張するとは、鍵自体に特別な*チェーンコード*を付加することです。チェーンコードは256ビットのバイナリ文字列で、各鍵と混合されて子鍵を生成します。

鍵が秘密鍵である場合、プレフィックス`xprv`で識別される*拡張秘密鍵*になります：

```
xprv9s21ZrQH143K2JF8RafpqtKiTbsbaxEeUaMnNHsm5o6wCW3z8ySyH4UxFVSfZ8n7ESu7fgir8i...
```

拡張公開鍵は、プレフィックス`xpub`によって識別されます：

```
xpub661MyMwAqRbcEnKbXcCqD2GT1di5zQxVqoHPAgHNe8dv5JP8gWmDproS6kFHJnLZd23tWevhdn...
```

HDウォレットには、秘密鍵を使わずに親の公開鍵から子の公開鍵を生成できるという、とても有用な特徴があります。このことから、子の公開鍵を生成する方法が2種類あることがわかります。子の秘密鍵から直接生成するか、または親の公開鍵から生成するかです。したがって、拡張公開鍵を用いることで、HDウォレットの持つブランチにあるすべての公開鍵（公開鍵のみ）を取得できます。

この仕組みにより、拡張公開鍵のコピーを持っているものの秘密鍵は一切持たないサーバーやアプリケーションにおいて、非常に安全に公開鍵のみがデプロイできます。このようなデプロイにおいては、制限なく公開鍵とイーサリアムアドレスが生成できますが、それらのアドレスに送金された資産を使うことはできません。一方、より安全な別のサーバーにおいて、拡張秘密鍵から対応するすべての秘密鍵を生成し、トランザクションに署名して資産を使うことができます。

この方法の応用として一般的なのは、eコマースを提供するウェブサーバーに拡張公開鍵をインストールすることです。このウェブサーバーは、公開鍵の導出関数を使って、すべてのトランザクションのために新しいイーサリアムアドレスを生成することができます（たとえば顧客のショッピングカートごとに）。このウェブサーバーには盗難に脆弱な秘密鍵は保管されません。HDウォレットを使わない場合、別の安全なサーバーで何千ものイーサリアムアドレスを事前生成し、eコマースサーバーに読み込んでおく必要があります。このアプローチは煩雑であり、eコマースサーバーが鍵を使い果たさないようにするためには、定期的なメンテナンスが必要になります。そのためHDウォレットの拡張公開鍵を使用することが好ましいでしょう。

もう1つの一般的な応用としては、コールドストレージもしくはハードウェアウォレットを用いることです。この場合は、拡張秘密鍵はハードウェアウォレットに保管されますが、拡張公開鍵はオンラインで保管することができます。ユーザーは自由に「受信」アドレスを作成できますが、一方で秘密鍵は安全にオフラインで保存されます。資産を使うには、ユーザーは拡張秘密鍵をイーサリアムのオフライン署名クライアントで使用するか、ハードウェアウォレットデバイス上でトランザクションに署名する必要があります。

強化子鍵導出

拡張公開鍵もしくは`xpub`から公開鍵のブランチを生成する機能は非常に便利ですが、これには潜在的なリスクが伴います。`xpub`へアクセスできても、子秘密鍵へのアクセスはできません。しかしながら、`xpub`にはチェーンコード（親公開鍵から子公開鍵の導出に使用）が含まれています。そのため、子秘密鍵が1つでも漏洩すると、チェーンコードを用いて他のすべての子秘密鍵を生成できてしまいます。親チェーンコードとともに、1つの子秘密鍵が漏洩してしまえば、すべての子の秘密鍵が明らかになってしまいます。さらに悪いことに、親チェーンコードとともに子秘密鍵を使うことで、親秘密鍵を推測することもできてしまいます。

このリスクに対処するため、HDウォレットは、親の公開鍵と子のチェーンコードの関係を「破壊」する*強化導出*（hardened derivation）と呼ばれる鍵導出関数を代わりに使用します。この強化鍵導出関数は、親公開鍵ではなく、親秘密鍵を用いて子チェーンコードを導出します。親または兄弟秘密鍵を危険にさらすことのないチェーンコードを使って、親と子の間に「ファイアウォール」が作られるのです。

チェーンコードの漏洩リスクにさらされることなく、`xpub`の利便性を利用して公開鍵のブランチを導出したいのであれば、通常の親（親公開鍵）ではなく、強化された親（親秘密鍵）から導出する必要があります。マスター鍵の第1レベルの子は、マスター鍵への不正アクセスを防ぐために常に強化導出を通じて生成するというのがベストプラクティスになります。

通常導出と強化導出のインデックス番号

与えられた親鍵から複数の子鍵を導出できることは明らかに望ましいことです。これを管理するために、インデックス番号が使用されます。各インデックス番号は、特別な子鍵導出関数を使って親鍵と組み合わされると、異なる子鍵を生成します。BIP-32の親から子への導出関数で使用されるインデックス番号は32ビット整数です。このインデックス番号は2つの範囲に分割されています。通常（非強化）の鍵導出関数で導出された鍵と、強化導出関数で導出された鍵を簡単に区別するためです。0から2^31^-1（0x0から0x7FFFFFFF）のインデックス番号は、通常の導出にのみ使用されます。2^31^から2^32^-1（0x80000000から0xFFFFFFFF）のインデックス番号は強化導出にのみ使用されます。したがって、インデックス番号が2^31^未満であれば子は通常の鍵導出関数によって導出されており、2^31^以上であれば子は強化導出によって導出されています。

インデックス番号を読みやすく表示するために、強化導出によって導出された子のインデックス番号はゼロから始まり、プライム記号（'）付きで表示されます。したがって、最初の通常導出による子鍵は0として表示され、最初の強化導出による子鍵（インデックス0x80000000）は0'として表示されます。2番目の強化導出鍵はインデックス0x80000001を持ち、1'として表示されます。HDウォレットのインデックス番号*i'*は、2^31^ + *i*を意味します。

HDウォレットの鍵識別子（パス）

HDウォレット内の鍵は「パス」の命名規則を使用して識別され、ツリーの各レベルはスラッシュ（`/`）文字で区切られます（表5-5参照）。マスター秘密鍵から得られた秘密鍵は`m`で始まります。マスター公開鍵から得られた公開鍵は`M`で始まります。したがって、マスター秘密鍵の最初の子秘密鍵は`m/0`です。最初の子公開鍵は`M/0`です。最初の子の2番目の孫は`m/0/1`であり、以下同様に続いていきます。

右から左に進むほど「祖先」に近づき、最終的にはマスター鍵にたどり着きます。たとえば、識別子`m/x/y/z`は、鍵`m/x/y`の*z*番目の子である鍵を示します。また鍵`m/x/y`は鍵`m/x`の*y*番目の子です。さらに`m/x`は`m`の*x*番目の子です。

表5-5. HDウォレットのパスの例

HDパス	鍵の説明
m/0	マスター秘密鍵（m）の最初（0）の子秘密鍵
m/0/0	最初の子秘密鍵（m/0）の最初（0）の孫秘密鍵
m/0'/0	最初の強化された子秘密鍵（m/0'）の最初（0）の通常の孫秘密鍵
m/1/0	2番目の子秘密鍵（m/1）の最初（0）の孫秘密鍵
M/23/17/0/0	24番目の子公開鍵の、18番目の孫公開鍵の、最初（0）の曽孫公開鍵の、最初（0）の玄孫公開鍵

HDウォレットのツリー構造をたどる

HDウォレットのツリー構造はとても柔軟性があります。しかし、その柔軟性の裏返しとして、際限のない複雑さが挙げられます。それぞれの親拡張鍵は40億個の子鍵を持つことができます。そのうちの20億個は普通の子鍵で、そのほかの20億個は強化導出された子鍵です。そして、子鍵のそれぞれが、さらに40億の子鍵を持つことができます。ユーザーの要望に応じてツリーの階層を深くすることができ、無限の世代の鍵を作ることができます。ただし、際限なく続くツリーのパスをたどるのは非常に困難です。

このためBIP-43とBIP-44という2つのBIPは、HDウォレットのツリー構造に関する標準を作ることで、この複雑さを解消しようとしています。BIP-43では、ツリー構造の「目的」を示す特別な識別子として、最初に強化子鍵のインデックス番号を使用することを提案しています。BIP-43によると、BIP-32で定義されているHDウォレットの階層（パス）構造の1階層目を「目的」として使うことにしています。ウォレットの目的を定義するインデックス番号が1階層目に示されており、その目的に応じて2階層目以下の構造と名前空間が決定されることになります。より具体的には、階層`m/i'/...`のブランチのみを使用するHDウォレットは、特定の目的に沿った使用を想定されており、その目的はインデックス番号*i*によって識別されます。

その仕様を拡張して、BIP-44は「目的」番号を`44'`に設定することで示されるマルチ通貨、マルチアカウント構造を提案しています。BIP-44の構造に従うすべてのHDウォレットは、ツリーの1つのブランチ`m/44'/*`のみを使用することで識別されます。BIP-44は、5つの事前定義されたツリーレベルで構成される構造を規定しています：

```
m / purpose' / coin_type' / account' / change / address_index
```

第1レベルの`purpose'`は常に`44'`に設定されます。第2レベルの`coin_type'`は暗号通貨の種類を指定し、各通貨が第2レベルの下に独自のサブツリーを持つマルチ通貨HDウォレットを可能にします。標準文書SLIP-0044にはいくつかの通貨が定義されています。例えば、イーサリアムは`m/44'/60'`、イーサリアムクラシックは`m/44'/61'`、ビットコインは`m/44'/0'`、すべての通貨のテストネットは`m/44'/1'`です。

ツリーの第3レベルは`account'`で、ユーザーが会計や組織の目的に合わせてウォレットを独立した論理的なサブアカウントに細分化できるようにします。例えば、HDウォレットには2つのイーサリアムアカウントが含まれることがあります：`m/44'/60'/0'`と`m/44'/60'/1'`。各アカウントはそれぞれのサブツリーのルートとなります。

BIP-44はもともとビットコイン用に作成されたため、イーサリアムの世界では関係のない「癖」が含まれています。パスの第4レベルの`change`では、HDウォレットは2つのサブツリーを持ちます。1つは受信アドレスの作成用、もう1つはお釣りアドレスの作成用です。イーサリアムではビットコインのようなお釣りアドレスの必要がないため、「受信」パスのみが使用されます。前のレベルが強化導出を使用していたのに対し、このレベルでは通常導出が使用されることに注意してください。これは、安全でない環境で使うためにアカウントレベルの拡張公開鍵をエクスポートできるようにする目的です。

利用可能なアドレスは、HDウォレットによって第4レベルの子として導出され、ツリーの第5レベルが`address_index`になります。例えば、プライマリアカウントでのイーサリアム支払い用の3番目の受信アドレスは`M/44'/60'/0'/0/2`となります。表5-6にさらにいくつかの例を示します。

表5-6. BIP-44 HDウォレット構造の例

HDパス	説明
M/44'/60'/0'/0/2	最初（0'）のイーサリアムアカウントの3番目の受信用公開鍵
M/44'/0'/3'/1/14	4番目のビットコインアカウントの15番目のお釣り用公開鍵
m/44'/2'/0'/0/1	トランザクションに署名するためのLitecoinメインアカウントの2番目の秘密鍵

セキュリティ

セキュリティとリカバリーのしやすさはトレードオフの関係にあります。極端な話、最も安全なシードとは何でしょうか？それは誰もリカバリーできないシード、自分自身でさえもリカバリーできないシードです。つまり、シードが作成され、すべてのコピー（物理的・デジタル的）が破壊されたなら、それは世界で最も安全なシードです。リカバリーできないので実用的ではありませんが、技術的には最も安全です。

逆に、リカバリーのしやすさについて言えば、最もリカバリーしやすいシードとは何でしょうか？シードフレーズがあらゆるところに書かれている、例えば体に刺青として見える場所に入れてあるようなものです。そのシードはリカバリーが非常に簡単ですが、まったく安全ではありません。

セルフカストディにおいてOpSec（運用セキュリティ）のために行う選択は、シードフレーズのセキュリティ強化とリカバリーのしやすさ、どちらかを向上させる方向に働きます。セキュリティを高めると通常はリカバリーのしやすさが低下し、その逆も同様です。場合によっては両方が低下してしまうこともあり、これは避けるべきです。

高セキュリティのシステムであっても、100%安全な設定は存在しません。完璧なシステムを作るためにできる限りのことをしても、脆弱性は依然として存在します。中でも興味深い脆弱性として秘密鍵の衝突があります。これは本質的には、すでに使われているシードと同じものを偶然生成してしまうことによって発生します。ハードウェアウォレットのシードはオフラインで生成され、生成済みシードのデータベースは存在しないため、技術的には同じウォレットを2回生成することが可能です。これが起こる確率は天文学的に小さいですが（宝くじに2回連続で当たる確率よりも低い）、可能性としては存在します。

警告

シードのセキュリティについては、多くの場合、急いで対処する必要はありません。シードが盗まれていればすでに手遅れですし、盗まれていなければ何も問題はないからです。これは特にビットコインウォレットに当てはまります。イーサリアムでも基本的には同様ですが、危険なスマートコントラクトへの承認を取り消すなど、迅速に行動すれば資金を守れるケースもあります。シードについて緊急の対応を求めるメッセージを受け取った場合、それはおそらく詐欺です。

シードを隠すための複雑なセキュリティメカニズムはベストプラクティスではありません。一見完璧に見えるかもしれませんが、その分野の専門家は適切なツールを使えばこの種のシードをリカバリーできることが多いです。そして、専門家でもリカバリーできないほど複雑であれば、必要なときに自分自身でもリカバリーできない可能性が高いです。

もう1つの避けるべきプラクティスは、セキュリティを隠蔽に頼ることです。システムには「秘密」のコンポーネントをできるだけ少なくし、秘密にすべきものはシード自体に限定すべきです。システムの一部を公開するとシステムが崩壊するのであれば、それは安全ではありません。

ベストプラクティスは、他の多くの人がやっていることに従うことです。新しいシステムを発明しようとしたり、さらに悪いことにすべてをゼロから作ろうとするのは間違いです。安全ではありませんし、ほとんどの人にはそれを行う技術的スキルがありません。シンプルに保ちましょう。シンプルさは間違いなくベストプラクティスです。

シードのバックアップを作成することは、おそらく最も重要なベストプラクティスです。ハードウェアウォレットであれブラウザウォレットであれ、バックアップは不可欠です。バックアップの保管場所は、(a) 自然災害に耐えられること、(b) ハードウェアウォレットとは地理的に離れていること、(c) 安全な場所であること、(d) 改ざんの痕跡が分かるパッケージであること、が求められます。シードを複数の紙に分割することはベストプラクティスではありません。誤った安心感を与えるだけです。

ヒント

シードを保管する安全な場所がない場合は、パスフレーズの使用を検討してください。パスフレーズは鍵生成プロセス中にシードと組み合わされることで、追加の保護層を提供します。パスフレーズはシードなしでは無意味なので、デジタルで保存することもできます。誰かがシードを入手してもパスフレーズを入手しなければ、一定期間は資金にアクセスできません。ただし、攻撃者がシードを持っていればパスフレーズは時間をかけてブルートフォースで解読される可能性があります。

人によってニーズは異なり、セキュリティ要件もそれに応じて変わります。しかし、常にベストプラクティスに従うべきです。ニーズは時間とともに変化する可能性があるため、常に注意を払い、必要に応じてOpSecを調整してください。

ハードウェアウォレットをセットアップした後、シードをリカバリーする能力をテストしてください。少なくとも12か月に1回はこれを行い、バックアップが機能し、アクセス可能であることを確認してください。

安全なシステムとはどのようなものでしょうか？信頼できるハードウェアウォレット（Trezorなど）、少なくとも1つの安全なバックアップ（デジタルで保存しないこと）、そしてあなたに何かあった場合の相続計画です。

> **注記**
>
> 相続は非常に複雑なトピックです。パメラ・モーガン著『*Cryptoasset Inheritance Planning: A Simple Guide for Owners*』（Merkle Bloom）を読むことをお勧めします。

ユーザーエクスペリエンスの改善

前述のように、広義にはウォレットとはイーサリアムへの主要なユーザーインターフェイスとして機能するソフトウェアアプリケーションです。そのため、イーサリアムのユーザーの大部分にとって、ウォレットがユーザーエクスペリエンスのほぼすべてです。しかし、長年にわたりウォレットのUXはほとんど改善されてきませんでした。例えば、執筆時点で最も人気のあるブラウザウォレットはMetaMaskですが、誕生以来それほど進化していません。僅差の2番手としてRabbyが最近登場しました。ここでは、ユーザーエクスペリエンスを劇的に改善しうるウォレットのアップデートについて検討します。

アカウントアブストラクション

イーサリアムにおけるアカウントアブストラクション（AA）は、ユーザーアカウントをより柔軟で使いやすくするための仕組みです。通常、第2章で説明したように、イーサリアムにはEOAとスマートコントラクトアカウントの2種類のアカウントがあります。AAはこの両方の利点を組み合わせたものです。通常のアカウント（EOA）がスマートコントラクトのように振る舞えるようになり、セキュリティ機能の追加や複数人によるトランザクション署名の許可など、アカウントの動作にカスタムルールを設定できます。

AAにより、イーサリアムはより使いやすく柔軟になります。例えば、秘密鍵だけに頼らずに紛失したアカウントをリカバリーしたり、イーサだけでなく任意のトークンでトランザクション手数料を支払ったりできるようになります。

AAを実装するための提案は複数あります。主な違いとして、一方の提案グループはイーサリアムプロトコルを変更せずにAAを実装しようとしたのに対し、もう一方の提案グループはイーサリアムプロトコル自体を変更するもので、Pectraアップグレードで実装される予定です。

AAに関する主なEIPとEthereum Requests for Comments（ERC）を見ていきましょう。これらの提案は急速に進化しており、本書が出版される頃にはAAの状況は大きく変わっている可能性があることに留意してください。それでも、何が実装され、近い将来何が計画されているかを把握しておくことは有益です。

> **ヒント**
>
> ERC、EIP、およびRollup Improvement Proposal（RIP）の違いを理解することが重要です。基本的に、EIPとRIPはイーサリアムネットワークの変更を中心とした改善であるのに対し、ERCはコアネットワークの機能に影響を与えないアプリケーション関連の変更です。

ERC-4337

ERC-4337は、ユーザーアカウントをより多機能で安全かつ使いやすくすることを目的としたイーサリアムの技術的進歩です。イーサリアムプロトコルを変更する一般的なアップグレードとは異なり、ERC-4337はスマートコントラクトとオフチェーンコンポーネントを通じて新しい機能を導入します。2024年9月時点で、AAの主要なオンチェーン実装はERC-4337です。

第2章で述べたように、従来のイーサリアムアカウントは2種類に分けられます。秘密鍵のみで制御されるEOAと、スマートコントラクトを通じて動作するコントラクトアカウントです。

EOAはシンプルで経済的であるため現在ユーザーウォレットの主流ですが、機能が限定的でセキュリティリスクにもさらされやすいという問題があります。秘密鍵を紛失すると資金を永久に失う可能性があり、マルチシグ設定やアカウントリカバリーオプションなどのセキュリティ機能をカスタマイズすることもできません。

一方、コントラクトアカウントはこれらのセキュリティ対策を組み込めますが、その複雑さとトランザクション手数料の高さから一般ユーザーへの普及が進んでいませんでした。ERC-4337は、EOAにスマートコントラクトの機能を持たせることで、EOAのシンプルさとコントラクトアカウントのセキュリティ・柔軟性を融合させます。

ERC-4337の実装では、図5-5に示すように、トランザクションを異なる方法で処理する*UserOperation*が導入されます。各トランザクションを直接ブロックチェーンにブロードキャストするのではなく、ユーザーはUserOperationを高レベルのメムプールに送信し、そこでトランザクションが一時的に保存されます。*バンドラー*と呼ばれる特別な参加者がこれらのUserOperationを収集・処理し、1つのイーサリアムトランザクションにまとめます。これによりネットワークの輻輳が軽減され、複数のオペレーションをより効率的に処理できるようになります。

また、ERC-4337では*ペイマスター*という新しい役割も導入されます。通常、イーサリアムのトランザクションではETHでガス代を支払う必要がありますが、ペイマスターにより代替トークンでのガス代支払いや、第三者によるガス代の代理支払いが可能になります。これにより、ETH

を保有していない新規ユーザーにとっての大きな障壁が取り除かれ、イーサリアムネットワークがより利用しやすくなります。

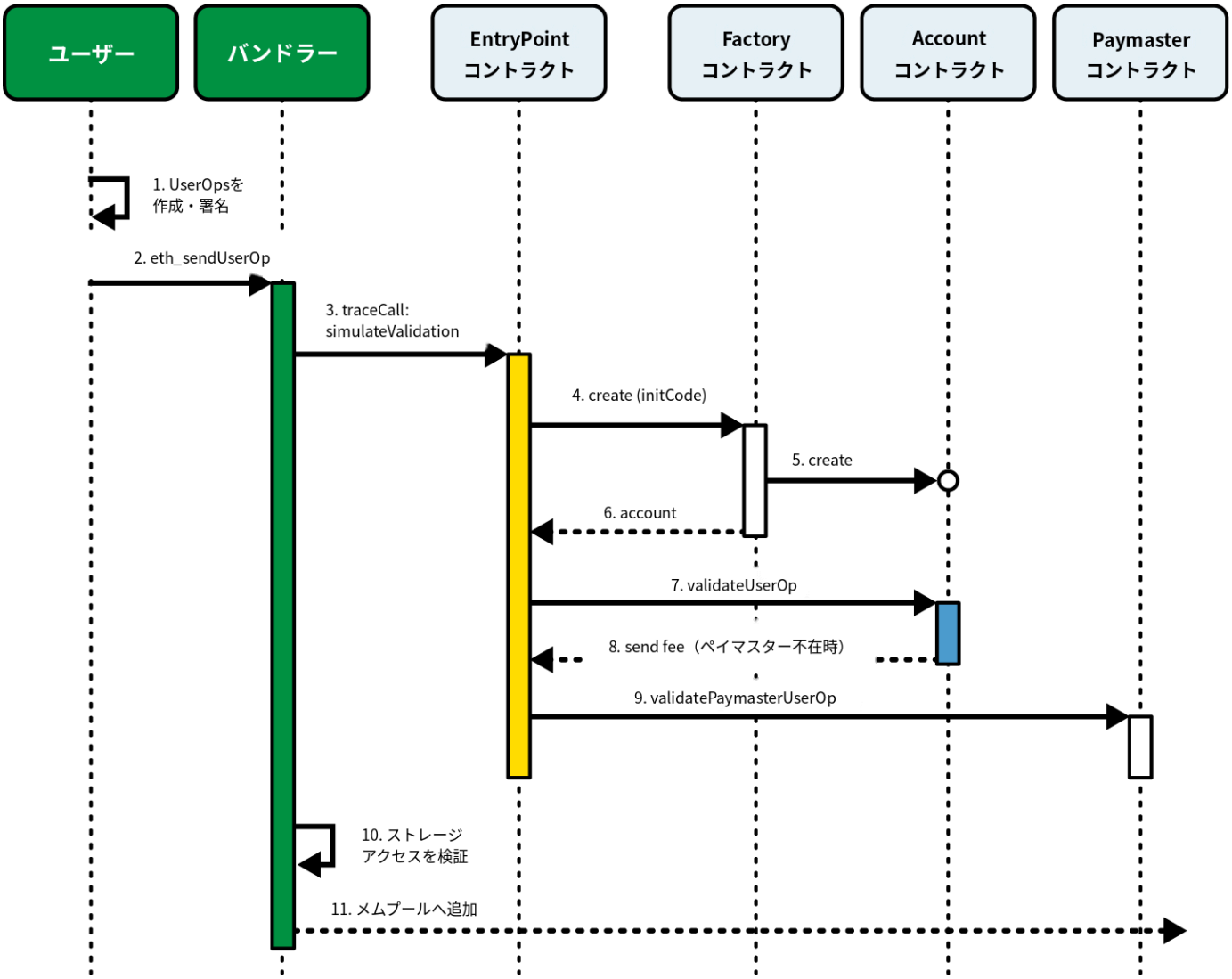


図5-5. ERC-4337の実装図

従来のパブリックメムプールではEOAの保留中のトランザクションを扱いますが、UserOperationはこれらのオペレーション専用に設計された*UserOperationメムプール*に送信されます。バンドラーはこのUserOperationメムプールを監視し、複数のUserOperationを収集して1つの「従来型」トランザクションにまとめます。まずEntryPointメソッドを通じて各UserOperationの有効性を検証し、検証後、バンドルしたトランザクションを通常のメムプールをバイパスして次の提案ブロックに直接送信します。バンドラーは自らブロックビルダーとして機能することも、ブロックビルダーと連携してトランザクションをブロックチェーンに追加することもできます。

EIP-2938

ERC-4337がイーサリアムのコアを変更せずにスマートコントラクトを通じてAAを実現するのに対し、EIP-2938はさらに一歩進んで、AAをプロトコル層自体に組み込むことを提案しています。このアプローチは、EVMがアカウントを扱う方法を根本的に変更することで、ユーザー、ス

マートコントラクト、アプリケーション間のより直接的で柔軟なやり取りを可能にします。EIP-2938により、ユーザーは従来のEOAなしでオペレーションを処理でき、秘密鍵ベースのアカウントへの依存を排除し、マルチシグアカウントやソーシャルリカバリーアカウントなど、より多様なアカウント管理システムをイーサリアムのプロトコルレベルで直接実現できます。

EIP-2938と既存のAA提案の主な違いは、アブストラクションをプロトコルの上にレイヤーとして載せるのではなく、プロトコルに直接統合する点にあります。EIP-2938では、イーサリアムは「トランザクション」ではなく「オペレーション」をサポートし、各オペレーションはより多機能でカスタマイズ可能なトランザクションタイプとなります。EIP-2938はイーサリアムのトランザクションをプログラム可能なオブジェクトとして再定義し、より広範なユースケースに対応します。アカウントとトランザクションを本質的にプログラム可能なものとして扱うことで、標準的な支払いやコントラクトとのやり取りを超えてイーサリアムの有用性を拡張し、ネットワーク上で構築される高度なアプリケーションにも対応できます。

RIP-7560

RIP-7560提案は、EIP-2938とERC-4337をネイティブAAのための統一的なアプローチに統合し、イーサリアムのトランザクションスコープを検証、実行、トランザクション後のステップに分割します。トランザクションの検証を認可とガス代の支払いに分離することで、あるコントラクトが別のアカウントのトランザクションのガス代をスポンサーすることが可能になります。このアプローチは、完全なネイティブAAの実現を目指しつつ、ERC-4337のエコシステムとの互換性を維持します。

ERC-4337はアプリケーションレベルでAAを導入していますが、プロトコル外の設計であるため制限があります。具体的には、追加のガスコスト、限定的な検閲耐性、非標準RPCメソッドへの依存、既存コントラクトとの互換性の制約などです。EIP-2938で提案されたネイティブAAは、これらの機能をプロトコルに直接組み込むことでより堅牢な代替手段を提示しますが、プロトコル変更なしで広くテスト・実装されてきたERC-4337の構造と完全には一致しません。

EIP-2938の組み込みプロトコルレベル機能とERC-4337から得られた実践的な知見を組み合わせることで、この提案は両方のアプローチの長所を統合します。このハイブリッドアプローチは、ガスコストの削減、検閲耐性の向上によるトランザクションの信頼性改善、現在および将来のイーサリアムコントラクトとの互換性確保を目指しています。EOAからスマートコントラクトアカウントへの段階的な移行により秘密鍵に関連するリスクも軽減され、イーサリアムがより安全で使いやすくなります。また、すでにERC-4337を使用しているユーザーがこれらの新機能をスムーズに採用できることも重要な点です。

図5-6は、RIP-7560で概説されたAAのトランザクションフローを表しています。このプロセスは、検証、デプロイメント、実行を処理する主要なコントラクトを組み込み、イーサリアムブロックチェーン上でユーザーアカウントとトランザクションを効率的かつモジュラーに管理します。

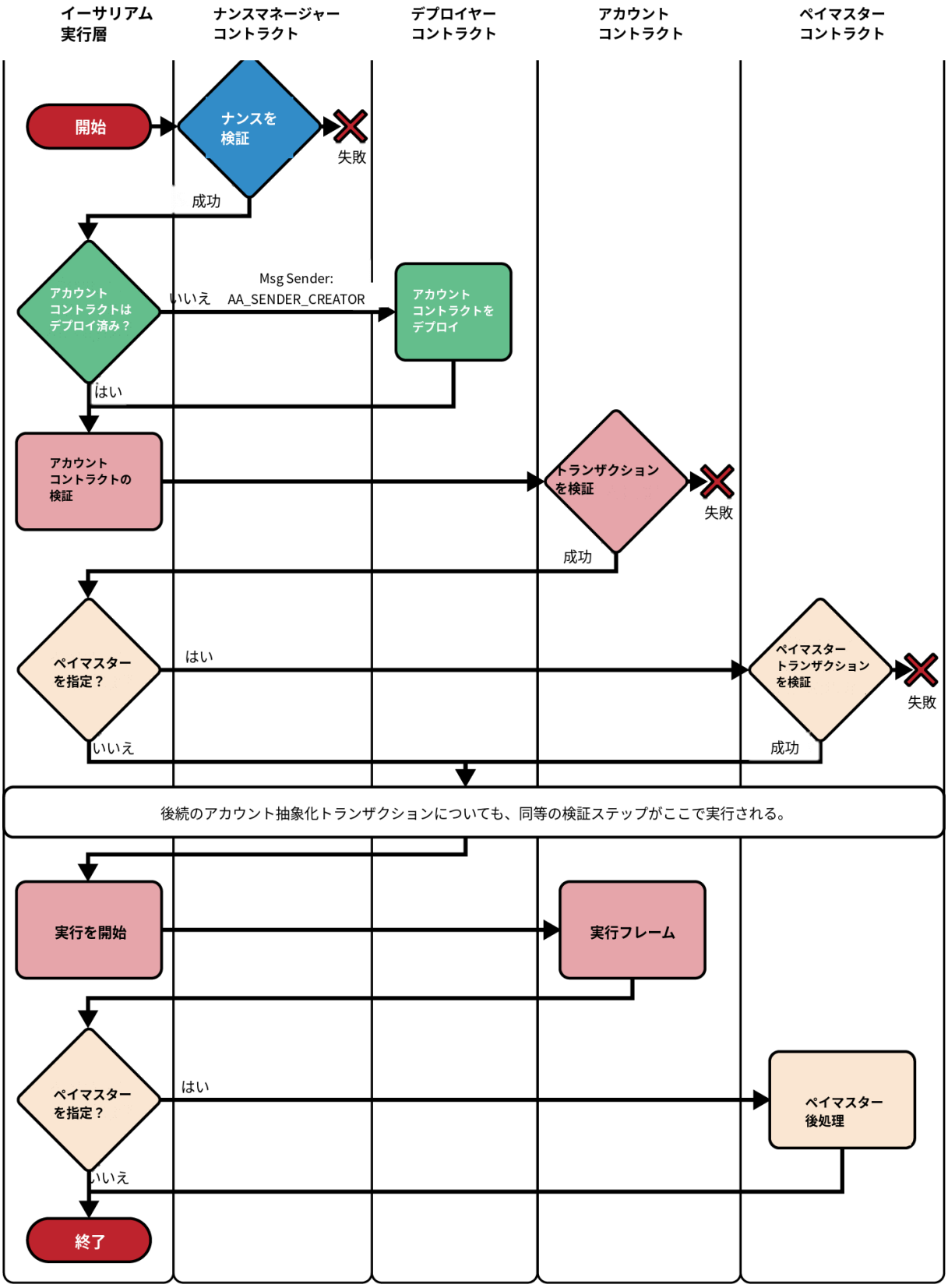


図5-6. RIP-7560のトランザクションフロー

フローは、Nonce Managerコントラクトによるナンスチェックから始まり、トランザクションが一意でありリプレイされていないことを確認します。これは二重支払いや偶発的な繰り返しを防ぐための重要なステップです。ナンスが有効であればシステムは先に進み、そうでなければトランザクションは停止されます。

次に、ユーザーのAccountコントラクト（スマートウォレット）がすでに存在するかどうかをチェックします。存在しない場合、Deployerコントラクトがウォレットを作成し、ユーザーがトランザクションに参加できるようにします。デプロイ済み、またはすでに存在する場合は、Accountコントラクト自体が検証チェックを実行し、トランザクションが正当でコントラクトのロジックに整合していることを確認します。これにより、承認されたアクションのみが実行され、各ユーザーのウォレットにセキュリティとカスタマイズの層が加わります。

このフローの特徴的な点は、Paymasterコントラクトをオプションで関与させられることです。指定された場合、このコントラクトはトランザクションのガス代を負担したり、代替の支払い方法を提供したりできます。ペイマスターが関与する場合は、トランザクションコストを正当に負担できることを確認するための独自の検証プロセスを経ます。この柔軟性によりスポンサーシップモデルが可能になり、ユーザーのトランザクション手数料の負担を軽減できる可能性があります。

すべての検証が完了すると、イーサリアム実行層がトランザクションの処理を開始します。実行中は、Execution Frameがトランザクションの詳細処理と進行の監視に使用されます。実行後、ペイマスターが関与していた場合は、トランザクションのスポンサーシップを完了するために必要な後処理を実行します。

EIP-5806

EIP-5806は新しいトランザクションタイプを導入し、「デリゲートコール」メカニズムを通じてEOAがカスタムコードを実行できるようにします。

現在、EOAはコントラクトのデプロイまたは「コール」トランザクションの送信しかできず、ブロックチェーンとのやり取りが制限されています。複数のアクションを1つのトランザクションにまとめたり、複雑なオペレーションを実行したりできないため、ユーザビリティに影響があります。AAがこれらの問題を解決できる可能性はありますが、採用への道筋はまだ定まっておらず、コストや譲渡不可能な資産のカストディの問題からすべてのユーザーが移行できるわけではありません。

この提案は、最小限のEVM変更と既存のセキュリティ標準を用いて、EOAがカスタムコードを実行するためのシンプルな方法を提供します。EOAが特定のコントラクトにコールをデリゲートできるようにすることで、マルチコールバッチングや安全なトークン転送などの複雑なオペレーションが可能になります。他のAA提案とは異なり、既存のアブストラクション手法を置き換えずにEOAのユーザーエクスペリエンスを改善することを目指しており、近い将来のEOA機能強化に向けた現実的な選択肢です。

EIP-3074

EIP-3074は、EOAが「インボーカー」（代理実行コントラクト）にトランザクションの制御をデリゲートできるようにする`AUTH`と`AUTHCALL`の2つのオペコードを導入します。ユーザーはインボーカーのアドレスとコミット（トランザクション値のハッシュ）を含むメッセージに署名することで、インボーカーが承認されたトランザクションのみを処理するよう保証します。リプレイ

保護はインボーカーが担当し、ユーザーは悪意のある動作を避けるためにこのコントラクトを信頼する必要があります。この提案はトランザクションのバッチ処理を改善し、柔軟なアカウントデリゲーションを可能にします。

`AUTH`は、ユーザーがスマートコントラクトに自分の代わりに行動する許可を与えるためのオペコードです。デジタル署名（ECDSA）で認可を検証し、将来のトランザクションのためにユーザーのアドレスを保存します。`AUTHCALL`は、認可されたスマートコントラクトがトークンの送信や他のコントラクトとのやり取りなどのトランザクションを実行できるようにします。例えば、10 DAIをETHにスワップしたい場合、1つの署名でインボーカーを認可（`AUTH`を使用）できます。インボーカーは`AUTHCALL`を使用してUniswapにDAIの使用を承認し、スワップを実行します。

当初、リプレイ保護やvalue、gas、その他の`AUTHCALL`引数などのフィールドも署名対象でした。設計が進化し、これらのタスクはインボーカーコントラクトにデリゲートされるようになったため、ユーザーがインボーカーを信頼することがより重要になりました。ユーザーは特定のコールプロパティをハッシュ化することで「コミット」できます。図5-7に示すように、インボーカーはリプレイ保護用のナンスなどコミットされた値がユーザーのコミットメントと一致する場合のみ検証を行います。これにより、インボーカーはユーザーが認可した内容だけを正確に処理します。

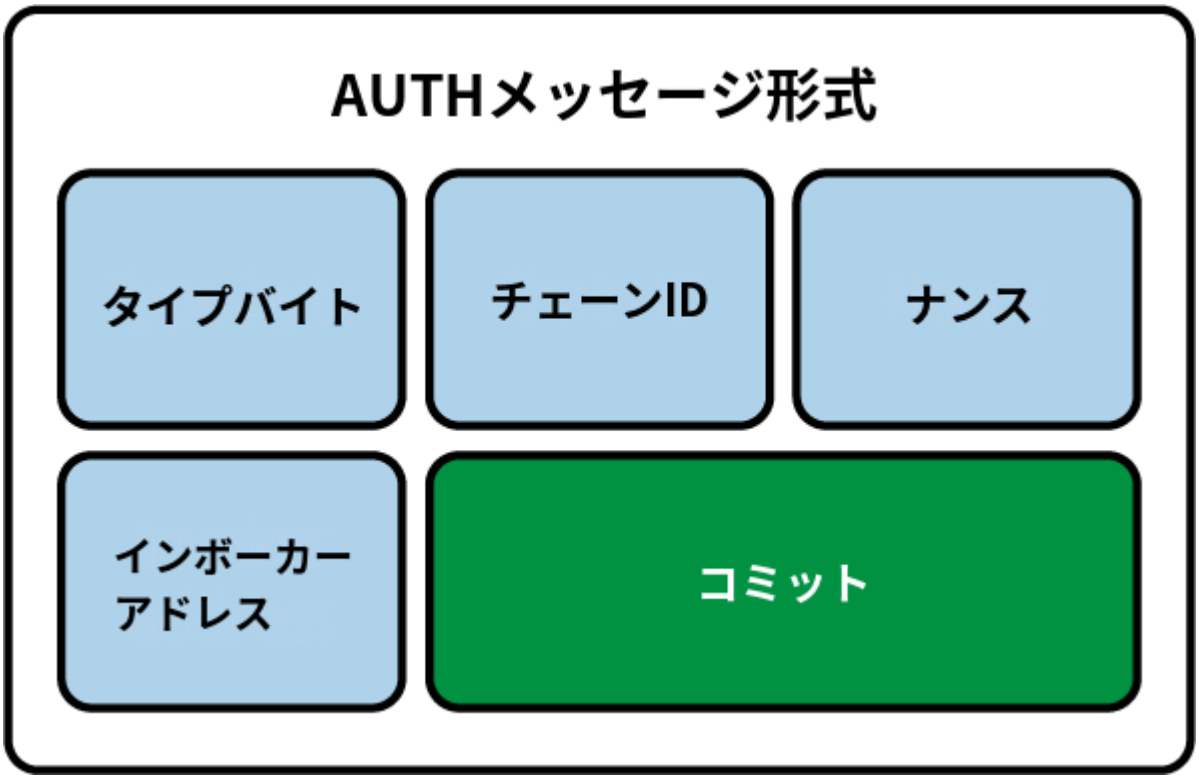


図5-7. EIP-3074のコミットメント検証

コミットハッシュにより、インボーカーは並列ナンスの許可や1つの署名での複数コールのバンドルなど、さまざまなルールを適用できます。これにより、図5-8に示すように、ERC-20のapprove-transferを1つのトランザクションに統合するなどのマルチコールフローが可能になります。

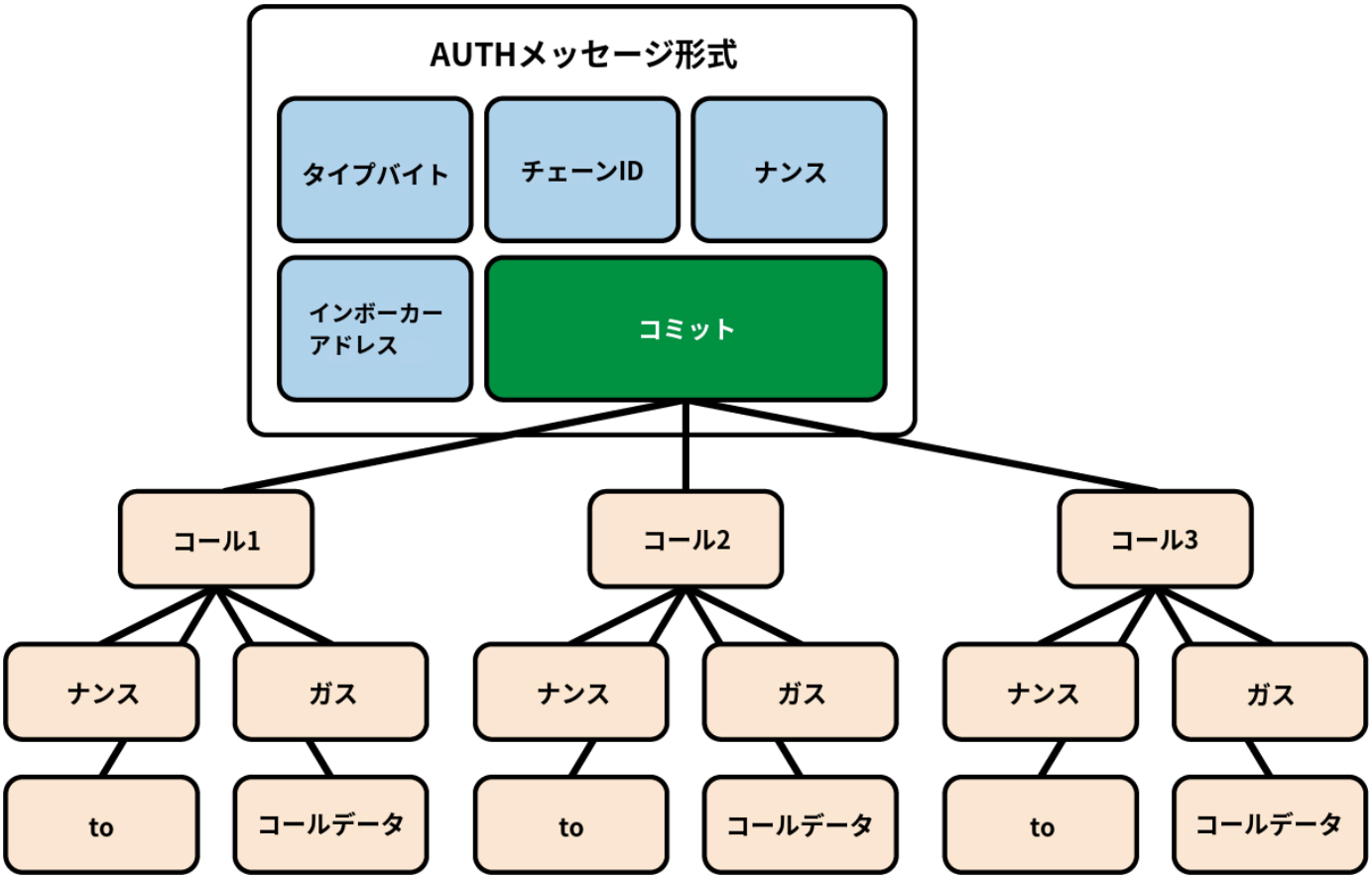


図5-8. EIP-3074のマルチコールフロー

さらに、デリゲートのアドレスとアクセスポリシーを含むコミットメッセージに署名することで、EOAの制御を他の鍵にデリゲートすることもサポートしています。図5-9に示すように、インボーカーはEOAの代わりにコールを中継する前にこれを検証します。

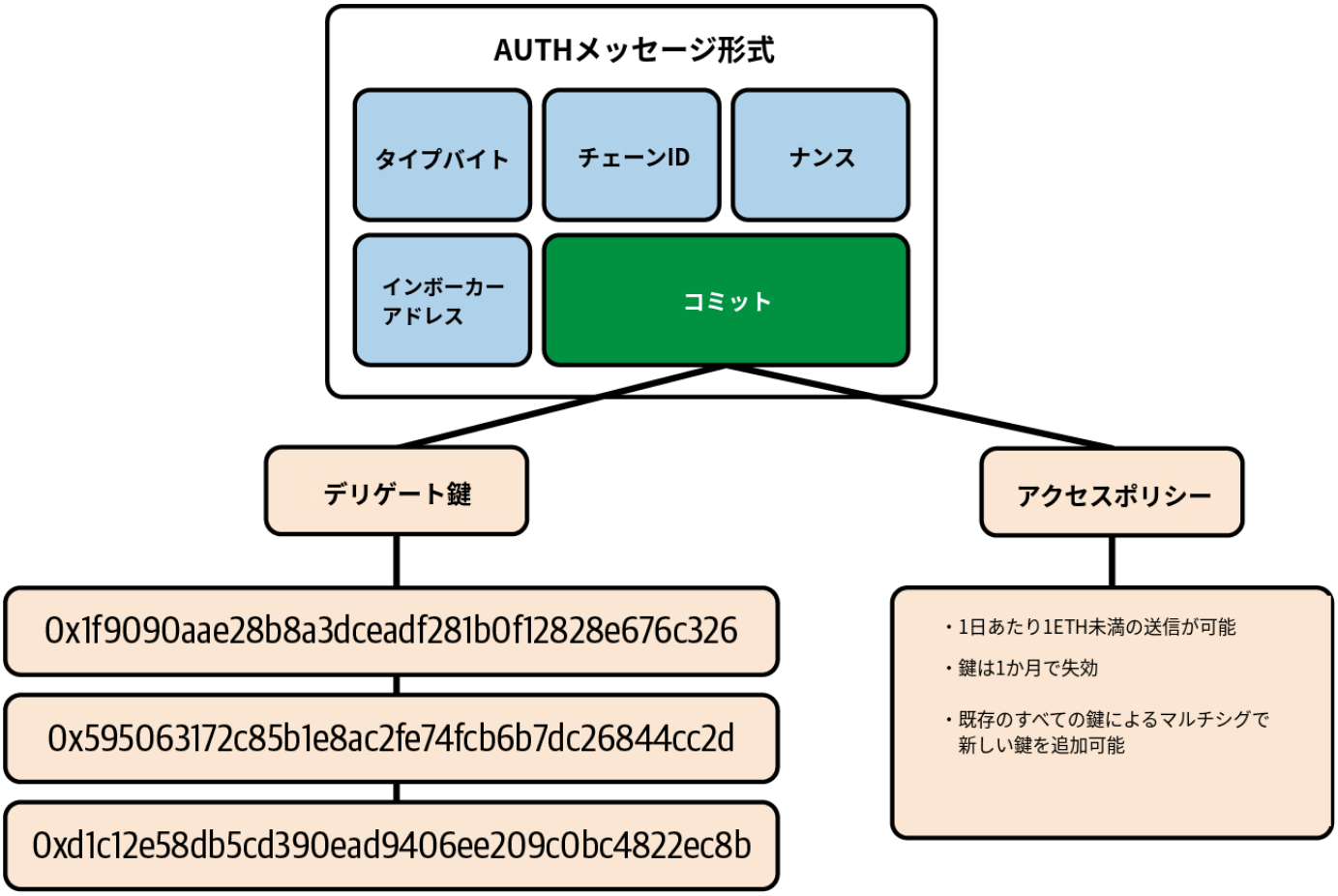


図5-9. EIP-3074のデリゲーションフロー

EIP-5003

EIP-5003は`AUTHUSURP`オペコードを導入します。これにより、EIP-3074を通じて認可されたアドレスにコードをデプロイし、（EIP-3607と併用した場合に）元のEOAの署名鍵を実質的に無効化できます。従来のEOAはイーサリアム上で多くの価値を管理していますが、鍵のローテーション、トランザクションのバッチ処理、ETHを保持せずにガス代の代理支払いを利用するといったことが容易にはできません。コントラクトアカウントとAAはこれらの問題に対処し、セキュリティ機能、認証方法、支出制限、鍵のローテーション、ソーシャルリカバリーなどのカスタマイズを可能にします。しかし、これらの利点はERC-4337のようなスマートコントラクトウォレットやアプリケーション層のソリューションを積極的に採用するユーザーに限られており、イーサリアムの誕生以来EOAに依存してきた多くの既存ユーザーにとっては、セキュリティとユーザビリティの改善が限定的なままです。

EIP-3074によりEOAからコントラクトアカウントへ署名権限をデリゲートできるようになりましたが、元の署名鍵を完全に取り消すことまではできません。このギャップがセキュリティリスクを生みます。デリゲーションされた権限があっても、元の秘密鍵は潜在的な脆弱性として残ります。漏洩した秘密鍵を取り消す手段が存在しないため、アカウント全体がリスクにさらされ続けます。秘密鍵が偶発的または悪意ある攻撃によって漏洩した場合、唯一の手段は新しいウォレットを作成して資産を個別に移行することですが、既存のスマートコントラクト内の一部の資産は移動できないため、コストがかかり不完全なプロセスになりがちです。

このEIPにより、`AUTHUSURP`はEOA保有者に安全な移行手段を提供します。EOAがスマートコントラクトに完全に制御を移譲し、元の鍵の権限を削除することで、セキュリティ上の脅威を排除できます。

EIP-7702

EIP-7702は、2025年初頭のPectraアップグレードで実装が予定されている、AAに関する最も重要なEIPです。（注：この章は2024年9月から10月に執筆されたため、未来形を使用しています。）EIP-7702はEIP-2718の下で「セットコードトランザクション」を導入し、EOAが認可されたアドレスに特定のコードの実行をデリゲートできるようにします。

> **注記**
>
> EIP-2718はイーサリアムに「型付きトランザクションエンベロープ（外枠）」フォーマットを導入し、プロトコル内で複数のトランザクションタイプをサポートします。この標準化されたフォーマットにより、既存の構造を維持しながら後方互換性のあるトランザクションのアップグレードが可能になります。詳細は第6章で説明します。このEIPはEIP-3607の制限も調整し、デリゲーション指定のフラグが付いたEOAからのトランザクションを許可します。これによりEOAの安全なデリゲーションが可能になり、イーサリアムのアカウント管理機能が強化されます。
>
> ユーザーがアカウントで実行するコードをどのように指定すべきかという疑問が生じます。主なオプションは、トランザクションにバイトコードを直接含めるか、コードへのポインターを参照するかです。最もシンプルなポインターは、オンチェーンにデプロイされたコードのアドレスです。一方、直接コード指定の場合は、EOAがトランザクションのcalldata内で任意のコードを実行できます。コードポインターに署名する際の考慮事項として、そのアドレスが別のチェーン上の意図したコードを指しているかどうかがあります。ウォレットのメンテナーは単一のEIP-7702認可メッセージをハードコードすることで、クロスチェーンでのコードの一貫性を確保できます。

重要なセキュリティ上の考慮事項があります。コードがEOAに組み込まれると、EIP-7702トランザクションでは`msg.sender == tx.origin`が成立する可能性があります。つまり、EOAコードがコールを発行するたびに、トランザクションの作成者がブロードキャストアドレスでもある状態になります。EIP-7702がなければ、このシナリオはトランザクションの最上位実行層でのみ発生します。しかしEIP-7702がトランザクションスポンサーリングを可能にするため、他の実行層でもこれが起こりえます。その結果、このEIPは従来の不変条件を崩します。`require(msg.sender == tx.origin)`チェックを含むスマートコントラクトに影響を与えます。このチェックには以下の3つの目的があります：

1. `msg.sender`がEOAであることの検証（`tx.origin`は常にEOAであるため）。この不変条件は実行層の深さに影響されず、コールの深さが1であれば`tx.origin`が`msg.sender`であることを確認できます。

2. フラッシュローンなどのアトミックサンドイッチ攻撃の防止。同じアトミックトランザクション内でターゲットコントラクトの実行前後にステートを変更する攻撃です。この保護はこのEIPにより損なわれます。ただし、この目的で`tx.origin`を使用することは一般的に悪いプラクティスとされており、マイナーがトランザクションを選択的に含めることですでに回避可能です。

3. リエントランシーに対する防御。

上記のうち、1番目と2番目についてはメインネットのコントラクトに実例があります。1番目がより一般的で、このEIPの影響も受けません。一方、3番目はこのEIPにより脆弱性が増しますが、提案の作成者による初期レビューでは、`tx.origin`に依存するリエントランシー保護の事例は確認されていません。

ヒント

このEIPの先行バージョンや類似の提案とは異なり、デリゲーション指定はいつでも取り消せます。取り消すには、アカウントの現在のナンスを使用して新しいアドレスを対象とするEIP-7702認可に署名し送信します。取り消さない限り、デリゲーションは有効なままです。

EOAの機能を改善することへの関心は高く、ユーザビリティや場合によってはセキュリティの向上が期待されています。主要なユースケースの1つはトランザクションスポンサーシップで、ガス代を賄うためのETHを保持せずにトランザクションを実行可能にします。従来のイーサリアムトランザクションではガス代はETHで支払われますが、他のトークンのみを保持しているユーザーやエコシステムの初心者にとっては不便です。EIP-7702は、トランザクション手数料を第三者が一時的または恒久的に負担できるモデルを導入し、ETHなしでトランザクションを完了できるようにします。これによりオンボーディングが簡素化され、ETH管理を必要とせずに特定のDAppやトークンを利用したい新規ユーザーにとってイーサリアムがより利用しやすくなります。

従来のイーサリアムアカウントは単一の秘密鍵にのみ依存しているため、紛失するとアカウントと資産へのアクセスを失います。EIP-7702の構造はさまざまなリカバリーメカニズムをサポートし、秘密鍵を紛失した場合でもアクセスを回復できるようにします。リカバリーメカニズムは、セキュリティを重視するユーザーや、鍵管理を困難に感じるブロックチェーン初心者にとって特に価値があります。リカバリーオプションとしてはマルチシグやソーシャルリカバリーがあり、選択した信頼できる相手にアカウントの復元を支援してもらうことができます。これはイーサリアムアカウントのセキュリティとユーザーフレンドリーさにおける大きな前進です。

EIP-7702はバッチ処理と権限の降格も導入します。バッチ処理により、単一のアトミックトランザクション内で複数のアクションを処理でき、ERC-20の承認に続くトークンの使用など、複雑なワークフローを2ステップではなく1ステップで完了できます。権限の降格は、特定の金額の使用や特定のアプリケーションとのやり取りのみを認可する制限付きサブキーを提供し、鍵の漏洩時の被害を最小限に抑えます。

このEIPは、従来アカウント残高とナンスの動作を支配してきた不変条件も変更します。EOAアカウントの残高はそのアカウントからのトランザクションによってのみ減少するという前提、およびトランザクションの実行中にEOAのナンスはインクリメントされないという前提を破ります。これらの変更はメムプールの設計や他のEIPに影響します。ただし、デリゲーション対象のアカウントはトランザクション内に明示的に列挙されているため、競合するトランザクションの転送を防ぐようにトランザクション伝播ルールを調整できます。ERC-4337およびRIP-7560との前方互換性も備えており、EOAが既存のEntryPointメカニズムを通じてERC-4337バンドルと互換性のあるコントラクトとして振る舞うことも可能です。

> **注記**
>
> 状態遷移の変更における一般的な設計目標は、EIP内の特殊ケースを最小限に抑えることです。このEIPの初期バージョンでは、アカウントのデリゲーション指定をクリアするための特殊ケースの追加を避けていました。0x0にデリゲートされたアカウントは基本的に通常のEOAと同様に動作しますが、そのアカウントとのやり取りには余計なコストがかかります。この余計なコストがEIP全体の採用に影響を与える可能性があるため、ユーザーがEOAを完全に元の状態に戻せるようにしました。

このEIPには複数のセキュリティ上の考慮事項があり、実装時に慎重に評価する必要があります。前述の`tx.origin`の問題に加えて、3つの主要な考慮事項があります：

- EOAがスマートコントラクトのように動作できるようになることで、トランザクション伝播に課題が生じます。従来、EOAはトランザクションを通じてのみ資金を送信できました。このEIPにより、EOAのデリゲートされたコードは誰でもいつでも呼び出せるようになり、ノードによる静的な残高検証を妨げます。つまり、ノードがアドレスの残高を検証することがより困難になります。

- スマートコントラクトウォレットの開発者は、実行なしでアカウントにコードを設定することの影響を考慮する必要があります。通常、コントラクトはinitcodeでデプロイされ、ストレージスロットが決定論的に初期化されます。このEIPはデリゲーションでのinitcode実行を省略するため、開発者は観察者によるセットアップのフロントランニングを防ぐために初期calldataを検証する必要があります。

- リプレイ保護（例：ナンスの処理）はデリゲートによって実装・署名される必要があります。これがなければ、悪意のある行為者が署名を再利用し、署名者のEOAに対するほぼ完全な制御を得る可能性があります。

アカウントアブストラクションの将来

イーサリアムにおけるAAは複雑ですが有望なアイデアであり、実現に向けた道筋も多数あります。各提案はユーザーのやり取りをよりシンプルで安全にするための異なるアプローチを提供しています。EIP-3074を基盤とするEIP-5003のように先行する提案を改善するものもあれば、EIP-5806のように単独で成立するものや後方互換性を欠くものもあります。この多様性は最適な

解決策の模索を反映しており、最終的な方向性はユーザーが最も使いやすく効果的だと感じる選択肢によって決まる可能性が高いです。将来の正確な姿はまだ見えませんが、AAはイーサリアムの利用体験を改善する大きな可能性を秘めています。

ソーシャルリカバリー

ソーシャルリカバリーに関連する主要なEIPはEIP-2429です。「Secret Multisig Recovery」と題されたEIP-2429は、信頼できる個人や組織を「ガーディアン」として指定することで、ユーザーがウォレットへのアクセスを回復できる仕組みを導入します。ユーザーが秘密鍵を紛失した場合、ガーディアンの協力によりウォレットの制御を取り戻せます。図5-10に示すように、ガーディアンはリカバリープロセス中のみ関与し、その権限は限定されています。

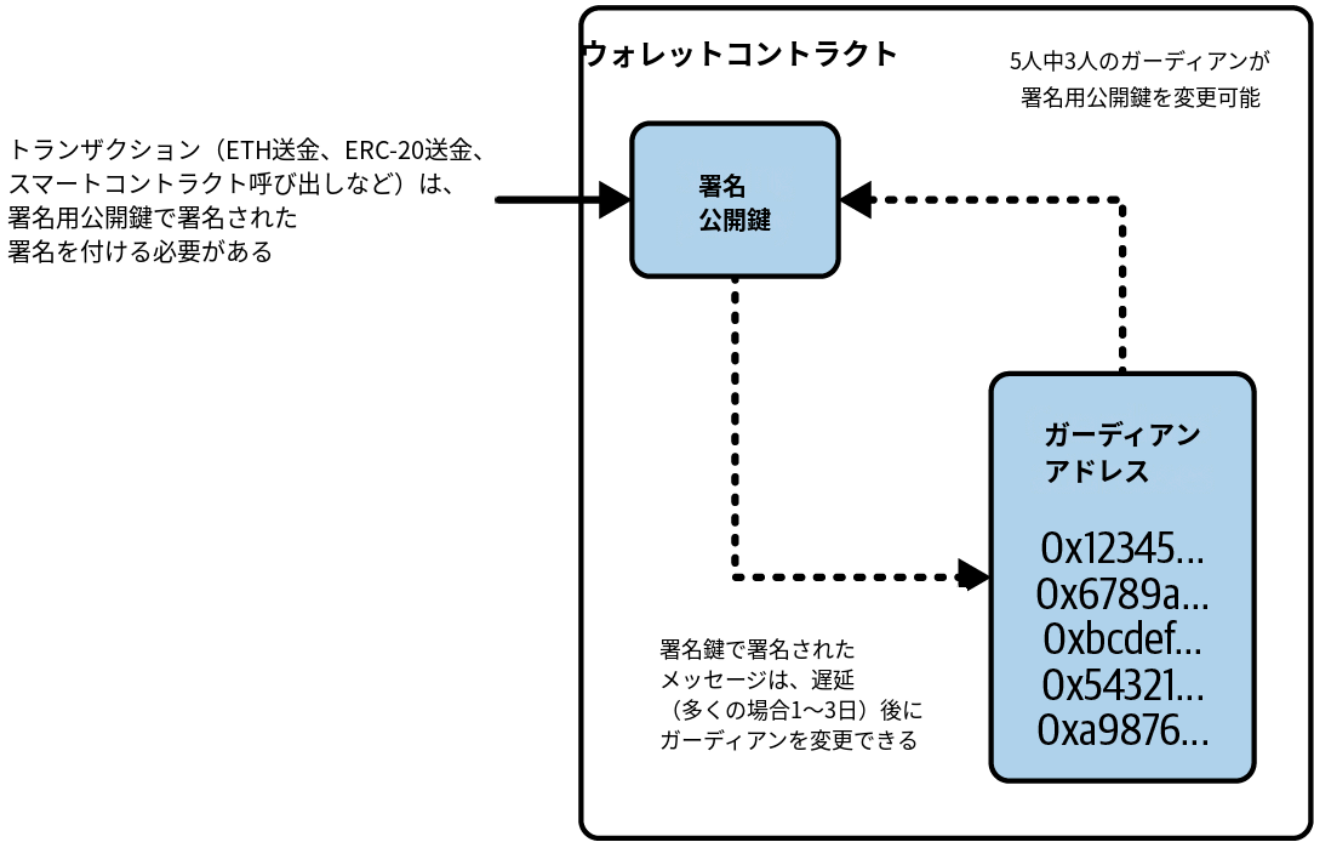


図5-10. ソーシャルリカバリーのフロー

注記

Vitalik Buterinは2021年に公開した記事で、「これが私のウォレットを保護するための好ましい方法、つまりソーシャルリカバリーにたどり着く」と述べています。現在も状況が同じかは不明ですが、ソーシャルリカバリーが依然としてButerinの推奨する方法である可能性があります。

通常の使用では、ソーシャルリカバリーウォレットは一般的なウォレットと同様に機能し、ユーザーは署名鍵でトランザクションに署名できます。鍵を紛失した場合はソーシャルリカバリー

が起動し、ユーザーがガーディアンに連絡すると、ガーディアンがウォレットの署名鍵を更新するトランザクションに署名します。ガーディアンには信頼できる個人、デバイス、または機関を指定できます。攻撃を防ぐためにガーディアンの身元はオンチェーンに保存されず、匿名性を保てます。ユーザーが死亡した場合は、ガーディアンが連携して資金をリカバリーすることも可能です。

ENS

Ethereum Name Service（ENS）は従来のドメイン名のWeb3バージョンのようなものですが、ウェブサイトにリンクする代わりに、人間が読める名前をイーサリアムアドレスに紐づけます。これにより、長く複雑なブロックチェーンアドレスの扱いがはるかに簡単になります。例えば、`0xd8dA6BF...`のようなアドレスに暗号通貨を送信する代わりに、`vitalik.eth`のような名前に送信するだけで済みます。これによりトランザクションが簡素化され、特に暗号通貨の初心者にとって取り組みやすくなります。

ENSをWeb2の世界と比較すると、DNSの仕組みに似ています。DNSは覚えにくいIPアドレスを`google.com`のようなURLに紐づけるため、ウェブサイトを訪れるのに数字の文字列を覚える必要がありません。しかし、ENSとDNSには重要な違いがあります。ENSは分散型であり、一度`.eth`名を所有すれば、中央の管理者がそれを制御することはありません。Web2ではドメイン名はレジストラによって管理され、検閲、停止、制限される可能性があります。ENSではイーサリアムウォレットを通じて名前を完全に制御できるため、`.eth`名はそれを保持するウォレットと同じレベルの安全性を持ちます。

暗号通貨を送受信するとき、長く複雑なイーサリアムアドレスの代わりに`andreas.eth`のような名前を使用すると、よりスムーズで安全になります。アドレスの入力ミスの可能性が減り、トランザクションにおけるリスクを軽減できます。

ENSのもう1つの特徴は、デジタルアイデンティティを提供できる点です。ウェブサイトのドメイン名を所有するのと同じように、ユニークな`.eth`名はブロックチェーン上の公開アイデンティティとして機能します。

まとめ

ウォレットは、ユーザー向けのあらゆるブロックチェーンアプリケーションの基盤です。ウォレットにより、ユーザーは鍵とアドレスのコレクションを管理できます。また、第6章で説明するように、ウォレットはデジタル署名を用いてイーサの所有権を証明し、トランザクションを承認する手段でもあります。

6章 トランザクション

*トランザクション*は、外部所有アカウントから発信された署名付きのメッセージです。これはイーサリアムネットワークに送信され、イーサリアムブロックチェーンに記録されます。この基本的な定義には、多くの驚くべき魅力的な詳細が隠されています。

別の観点から見ると、トランザクションは状態の変更を引き起こしたり、EVM（イーサリアム仮想マシン）でコントラクトを実行できる唯一の存在です。イーサリアムはグローバルなシングルトン状態マシンであり、トランザクションは状態マシンを作動させ、状態を変更します。コントラクトは単独では実行されません。そして、イーサリアムは自律的に実行されません。すべてはトランザクションから始まるのです。

この章では、トランザクションを分析し、どのように動作するかを示し、詳細について考察します。この章の内容のほとんどの部分は、すでにウォレットアプリケーションを開発しており、独自のトランザクションを低レベルで管理することに関心がある人を対象としています。既存のウォレットアプリケーションを使用しており問題がないのであれば、トランザクションの仕組みについて懸念を持つ必要はありません。とはいえ、この章で触れる詳細は興味深い内容のはずです。

トランザクションの構造

最初に、イーサリアムネットワーク上でシリアライズされて送信されるトランザクションの基本的な構造を見てみましょう。各クライアントはトランザクションを独自の内部データ構造でメモリに格納します。この構造には、ネットワーク上のシリアライズ済みトランザクションには含まれないメタデータが追加されます。ネットワークのシリアライゼーションはトランザクション唯一の標準形式です。

イーサリアム初期にはトランザクションの種類は1つだけでしたが、*EIP-2718*が、異なるトランザクションタイプを扱い、それぞれ異なる方法で処理する仕組みを導入しました。具体的には、すべてのトランザクションはトランザクションタイプを指定する1バイトで始まります。

```
transaction = tx_type || tx_payload
```

執筆時点（2025年6月）では、表6-1に示す5つのトランザクションタイプが存在します。

表6-1 EIP-2718トランザクションタイプ

タイプ識別子	名称
`0x00`	レガシートランザクション
`0x01`	EIP-2930トランザクション
`0x02`	EIP-1559トランザクション
`0x03`	EIP-4844トランザクション
`0x04`	EIP-7702トランザクション

それぞれについて詳しく見ていきましょう。

レガシートランザクション

*レガシートランザクション*は、以下のデータを含むシリアライズされたバイナリメッセージです。

チェーンID

トランザクションの送信先ネットワークのチェーンID。リプレイ攻撃防止の簡易的な仕組みとしてEIP-155で追加された

ナンス

メッセージの再利用を防ぐために使用されるシーケンス番号。発信元EOAから発行される

ガス価格

トランザクション発行者が支払ってもよいと考えるガスの価格（単位はwei）

ガスリミット

トランザクション発行者がこのトランザクションのために購入してもよいと考えるガスの最大量。支払うのはトランザクションで実際に使用されたガス量のみである。ガスリミットは支払ってもよいと考えるガスの最大量を表すにすぎない

受信者

宛先となるイーサリアムアドレス

バリュー

宛先に送信するイーサの量

データ

可変長バイナリデータのペイロード

v、r、s

発信元EOAのECDSAデジタル署名の3つの要素

トランザクションメッセージの構造は、RLP（Recursive Length Prefix）エンコーディングスキームを使用してシリアライズされます。このRLPエンコーディングスキームは、イーサリアムのシンプルかつバイト精度の高いデータシリアライゼーションのために特別に作成されました。イーサリアムのすべての数値は、8ビットの倍数の長さのビッグエンディアン整数としてエンコードされます。

なお、ここで示しているフィールドラベル（`to`、`gasLimit`など）は、RLPエンコードされたフィールド値を含む、シリアライズされたトランザクションデータの一部ではありません。一般的には、RLPにはフィールド区切り文字やラベルは含まれていません。RLPの長さプレフィックスが各フィールドの長さを識別するために使用されます。定義された長さを超えたバイト列は、次のフィールドの内容として扱われます。

これが実際に送信されるトランザクション構造ですが、ほとんどの内部表現やユーザーインターフェイスの視覚化では、トランザクションやブロックチェーンから得られる追加情報でこの構造が補われます。

たとえば、発信元のEOAを識別するアドレスには「from」データがありません。EOAの公開鍵はECDSA署名の`v`、`r`、`s`要素から導出でき、アドレスは公開鍵から導出できるためです。トランザクションに「from」フィールドが表示されている場合、それはトランザクションを視覚化するためのソフトウェアによって追加されたものです。

クライアントソフトウェアによってトランザクションに頻繁に追加されるその他のメタデータには、ブロック番号（ブロックチェーンに公開され含まれた際のもの）とトランザクションID（計算されたハッシュ値）が含まれます。このデータもトランザクションから得られるものであり、トランザクションメッセージ自体の一部を構成するものではありません。

EIP-2930トランザクション

EIP-2930トランザクションは、EIP-2718のタイプ付きトランザクションエンベロープを最初に使用するもので、トランザクションタイプは`0x01`です。基本的にはレガシートランザクションと同じですが、*アクセスリスト*という新しいフィールドが追加されています。アクセスリストは（アドレス、ストレージスロット）の配列です。これにより、トランザクションがアクセスする対象のコストを事前に支払えます。これにより、EVMでの実行時にユーザーが課されるガス代が低くなります。

> **注記**
>
> より正確に言うと、アクセスリストに含まれるアドレスとそのストレージスロットは、それぞれ`accessed_addresses`と`accessed_storage_keys`に含まれる。EVMはこれらを使用して、

> ウォーム（warm）アクセスとコールド（cold）アクセスを区別する。コールドアクセスはウォームアクセスよりもはるかに多くのガスを消費する。たとえば、`SLOAD` オペコードはストレージスロットがウォームの場合100ガス、それ以外の場合は2,100ガスを課す。

この新しいトランザクションタイプは、主にEIP-2929に起因する問題に対処するために導入されました。EIP-2929は状態アクセスオペコードのガスコストを増加させました。その結果、一部のスマートコントラクトでガス不足エラーが発生し、トランザクションを正しく処理できなくなりました。アクセスリストを導入することで、ユーザーはトランザクションがアクセスするアドレスとストレージスロットを事前に支払い、このような障害を防止できるようになりました。

EIP-1559トランザクション

EIP-1559トランザクションは、2021年8月5日のLondonハードフォークで導入され、トランザクションタイプは`0x02`です。このトランザクションは、新しいプロトコルパラメータであるベースフィー（基本手数料）を導入し、イーサリアムの手数料市場の構造を根本的に変えました。

ベースフィーは、イーサリアムネットワーク上でトランザクションを送信するために支払う必要のある最低手数料を表します。ブロックガスリミットは1,500万から3,000万ガスに倍増され、*ブロックガスターゲット*がブロックガスリミットの半分である1,500万ガスとして導入されました。平均負荷は従来と同じ水準を維持しつつ、必要に応じてブロックサイズを最大2倍まで拡大できるようにする、という考え方です。

ブロックの平均ガス使用量を1,500万ガスに維持するため、ベースフィーは固定値ではなく、ブロックの利用率に基づいて変動します。ブロックの使用ガスがブロックガスターゲットを上回ればベースフィーは増加し、下回れば減少します。

ブロックガスリミットは、図6-1に示すように、特定のブロックで切りのよい値に増加されてきました。1,000万、1,250万、1,500万、3,000万といった具合です。バリデータ（旧PoWではマイナー）は各ブロックでガスターゲットを微調整でき、その結果がガスリミットに直接反映されます。ただし、ブロックガスリミットは非常に重要な値なので、通常はコア開発者の推奨に従います。

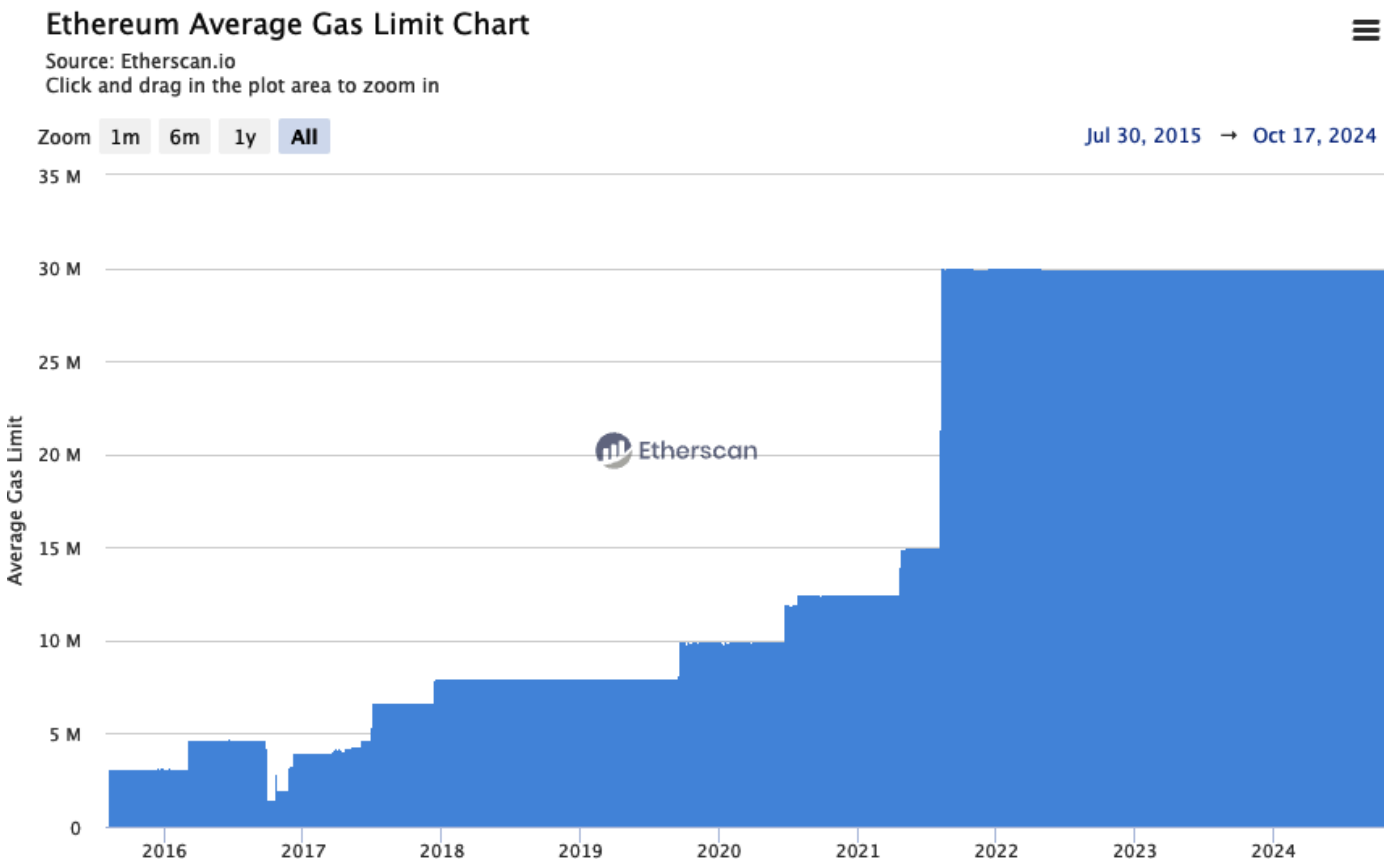


図6-1. ブロックガスリミットの推移

ベースフィーはブロックを作成するバリデータ（またはマイナー）には渡されず、直ちにバーン（焼却）され、ETHの総供給量が減少します。新たに*プライオリティフィー*（優先手数料）が導入されました。プライオリティフィーは、バリデータ（またはマイナー）がトランザクションを次のブロックに含めるよう動機付けるためのチップと考えることができます。

> **ヒント**
>
> 理論上は、必須であるベースフィーのみを支払い、プライオリティフィーをゼロにしたトランザクションを作成することもできる。プロトコルはバリデータへのチップの支払いを義務付けていない。しかし実際には、妥当な時間内にトランザクションが承認されるよう、常にプライオリティフィーを含めるべきである。なお、通常ウォレットはベースフィーとプライオリティフィーを自動的に処理し、適切な値に設定する。

EIP-1559トランザクションは、以下のデータを含むシリアライズされたバイナリメッセージです。

チェーンID

レガシートランザクションと同じ

ナンス

レガシートランザクションと同じ

ガス当たりの最大プライオリティフィー

トランザクションをブロックに含めるためのチップとして、発行者がバリデータに直接支払ってもよいと考えるガスの価格（単位はwei）

ガス当たりの最大フィー

ベースフィーとプライオリティフィーを含む、発行者が合計で支払ってもよいと考えるガスの価格（単位はwei）

ガスリミット

レガシートランザクションと同じ

受信者

レガシートランザクションと同じ

アクセスリスト

EIP-2930トランザクションと同じ

バリュー

レガシートランザクションと同じ

データ

レガシートランザクションと同じ

v、r、s

レガシートランザクションと同じ

すべてのトランザクションタイプと同様に、メッセージ構造はRLPエンコーディングスキームを使用してシリアライズされます。

EIP-4844トランザクション

EIP-4844トランザクションは、2024年3月13日のCancunハードフォークで導入され、トランザクションタイプは`0x03`です。4章の「KZGコミットメント」のセクションですでに触れており、16章でさらに詳しく説明します。このトランザクションは*blobキャリングトランザクション*とも呼ばれます。サイドカーとして*blob*を伴うためです。blobはEVMから直接アクセスできない大量のデータ（各約131,000バイト）ですが、コミットメントにはアクセスできます。

blobには*blobガス*という新しい種類のガスが使用されます。blobガスは通常のガスとは完全に分離され独立しています。EIP-1559に強く触発されたものですが、独自のターゲットルールに

従います。使用されたblobガスがターゲットblobガス使用量を上回ればblobガス価格は増加し、下回れば減少するという仕組みです。

シリアライズされたバイナリメッセージはEIP-1559と同じ形式を共有しますが、2つの新しいフィールドが追加されています。

blobガス当たりの最大フィー

blobに対して発行者が支払ってもよいと考えるblobガスの価格（単位はwei）

blobバージョン付きハッシュ

blobに関連する各KZGコミットメントのバージョン付きハッシュを表す32バイト値のリスト

> **注記**
>
> CancunハードフォークとEIP-4844トランザクションの導入に伴い、ブロックヘッダーに2つの新しい要素が追加された。
>
> **blobガス使用量**
>
> ブロック内のすべてのEIP-4844トランザクションが使用したblobガスの合計量
>
> **超過blobガス**
>
> 当該ブロック以前に、ターゲットを超えて消費されたblobガスの累計

EIP-7702トランザクション

EIP-7702トランザクションは、2025年5月7日のPectraハードフォークで導入され、トランザクションタイプは`0x04`です。EOAが自身のアカウントにコードを設定できるようにするものです。従来、EOAのコードは空であり、トランザクションを開始することはできても、スマートコントラクトとやり取りしない限り複雑な操作を実行することはできませんでした。EIP-7702はこの状況を変え、EOAが以下のような操作を行えるようにします。

バッチ処理

同一ユーザーからの複数の操作を1つのアトミックなトランザクションにまとめられる。たとえば、ERC-20の承認とその承認の使用を一度に行う操作は、多くの分散型取引所で非常に一般的なワークフローである

スポンサーシップ

アカウントXがアカウントYに代わってトランザクションの手数料を支払う

権限の縮小委譲

ユーザーがサブキーに署名し、アカウントへのグローバルアクセスよりもはるかに弱い特定の権限を付与できる。たとえば、1日の残高の最大1%までの支出権限や、特定のアプリケーションとのみやり取りする権限などである

内部実装の詳細はかなり複雑であり、興味がある場合はEIPの公式ウェブサイトを読むことを推奨します。ただし、概要はシンプルでありながら非常に強力です。EIP-7702により、EOAは自身にデリゲーションデジグネーター（委譲指定子）を割り当てることができます。デリゲーションデジグネーターは、イーサリアムメインネット上のスマートコントラクトを指します。EOAにトランザクションが送信されると、図6-2に示すように、指定されたアドレスのコードがEOA自身のコードであるかのように実行されます。

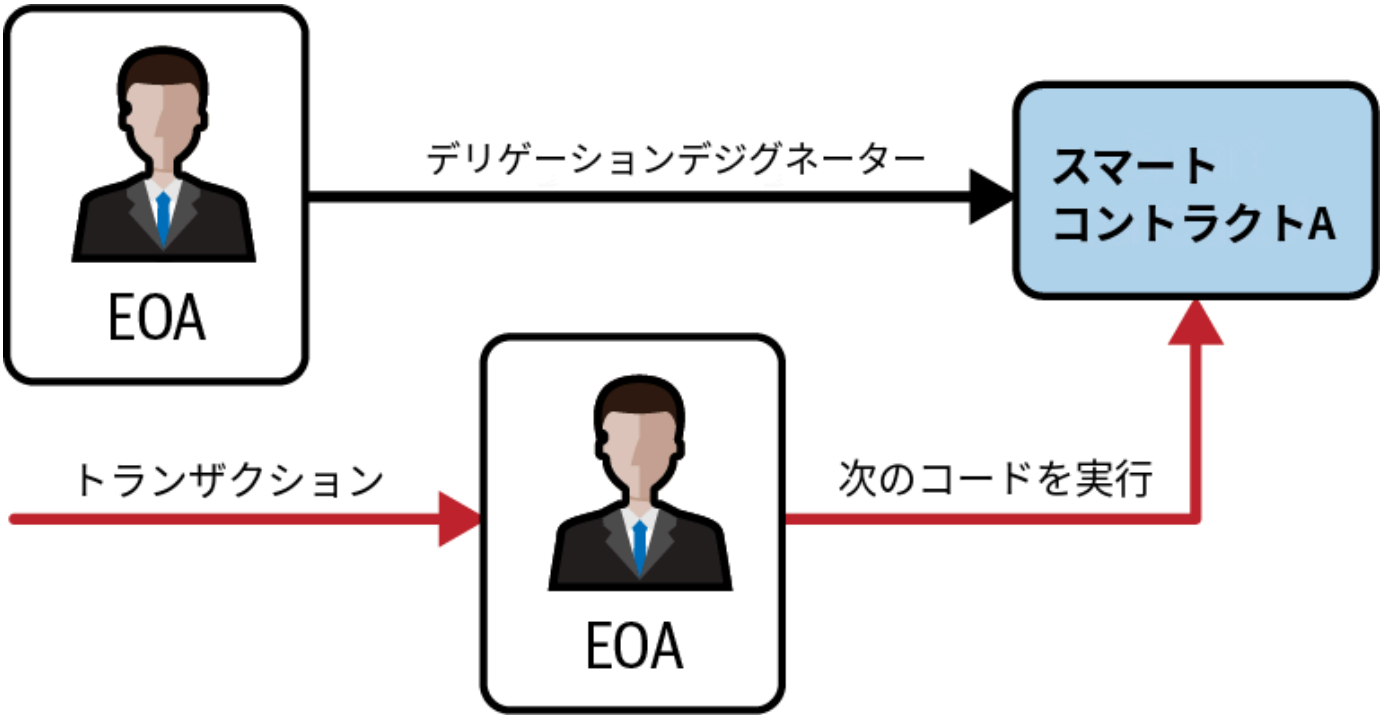


図6-2. EIP-7702のデリゲーションメカニズム

トランザクションナンス

ナンスは、トランザクションにおいて最も重要で、最も理解されていない要素の1つです。イーサリアムの「イエローペーパー」での定義は次のとおりです。

> ナンス：アドレスから送信されたトランザクションの数に等しいスカラー値。または、コードを持つアカウントの場合は、そのアカウントがコントラクトを作成した回数。

厳密に言えば、ナンスは送信元アドレスの属性です。つまり、送信元アドレスの文脈でのみ意味があります。しかし、ナンスはブロックチェーンのアカウントの状態の一部として明示的に保

存されていません。アドレスから発信された承認済みトランザクションの数をカウントすることによって、動的に計算されます。

> **翻訳者注釈**
>
> 原著では「明示的に保存されていない」「動的に計算される」と記述されているが、これは不正確である。実際にはナンスはアカウントステートの一部としてワールドステートトライに保存されている。go-ethereumの実装では `StateAccount` 構造体に `Nonce` フィールドが含まれており（core/types/state_account.go）、 `eth_getTransactionCount` もステートから直接読み取る。

トランザクションをカウントするナンスの存在が重要となるシナリオが2つあります。トランザクションが作成順に含まれるという利便性の機能と、トランザクションの重複保護という重要な機能です。それぞれのシナリオの例を見てみましょう。

シナリオ1

2つのトランザクションを作るとしましょう。6 etherの重要な支払いと、8 etherの支払いです。6 etherのトランザクションのほうが重要なので、最初に署名してブロードキャストし、次に8 etherのトランザクションに署名してブロードキャストします。残念なことに、アカウントには10 etherしかないので、ネットワークは両方のトランザクションを受け入れることができず、片方は失敗します。より重要な6 etherのトランザクションを最初に送ったので、そちらが通過し、8 etherのトランザクションが拒否されると思うかもしれません。しかし、イーサリアムのような分散システムでは、ノードがどちらの順序でトランザクションを受け取るかは決まっていません。特定のノードに一方のトランザクションがもう一方より先に届くという保証はないのです。そのため、一部のノードが先に6 etherのトランザクションを受信し、他のノードが先に8 etherのトランザクションを受信するケースがほぼ確実に起こります。ナンスがなければ、どちらが受け入れられ、どちらが拒否されるかはランダムです。

しかしナンスが含まれていれば、最初に送ったトランザクションはたとえば3のナンスを持ち、8 etherのトランザクションは次のナンス値（つまり4）を持つことになります。したがって、8 etherのトランザクションが最初に受け取られたとしても、ナンス0から3までのトランザクションが処理されるまでは無視されます。

シナリオ2

100 etherのアカウントを持っているとします。本当に購入したい商品があり、その支払いをイーサで受け付けてくれる人をオンライン上で見つけました。2 etherを送り、相手は商品を送ってくれます。その2 etherの支払いのために、自分のアカウントから相手のアカウントへ2 etherを送信するトランザクションに署名し、イーサリアムネットワークにブロードキャストして検証とブロックチェーンへの取り込みを待ちます。ここで、トランザクションにナンス値がなければ、同じアドレスに2 etherを再度送信した場合、そのトランザクションは最初のものとまったく同じに見えます。つまり、ネットワーク上のトランザクションは誰でも見ることができます。受信者や悪意ある第三者を含め、誰もが元のトランザクションをコピーして再送信すれば、

残高がなくなるまで何度でも「リプレイ」できてしまいます。しかし、トランザクションにナンス値があれば、同一の受信者アドレスに同じ量のイーサを複数回送信する場合であっても、毎回のトランザクションは一意になります。したがって、インクリメントされるナンスがトランザクションの一部として含まれることで、支払いの「複製」は不可能になります。

要約すると、ビットコインプロトコルの*UTXO（Unspent Transaction Output）*メカニズムとは対照的に、ナンスの使用はアカウントベースのプロトコルにとってまさに必要不可欠であることを強調しておきます。

ナンスの追跡

このセクション以降では、Foundryスイート、特に`cast`ツールを使用します。`cast`はブロックチェーンとの対話を非常に簡単にしてくれるツールです。以降の例を再現したい場合は、事前にインストールしておいてください。

まず、この章を通じて使用するウォレットをセットアップする必要があります。ターミナルウィンドウを開き、次のように入力します。

```
$ cast wallet new
Successfully created new keypair.
Address:     0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0
Private key: 0xd6d2672c6b4489e6bcd4e93b9af620fa0204b639b7d7f93765479c0846be0b58
```

> **警告**
>
> ここに記載されているアドレスに資金を送信すると、秘密鍵が公開されているため、誰でもその資金を自分自身に送金できてしまい、資金を失うことになる。

次に、秘密鍵をコンピュータのキーストアにインポートして、後から簡単に利用できるようにします。

```
$ cast wallet import example \
    --private-key 0xd6d2672c6b4489e6bcd4e93b9af620fa0204b639b7d7f93765479c0846be0b58
Enter password:
`example` keystore was saved successfully. Address: 0x7e41354afe84800680ceb104c5fc99ecb98a25f0
```

任意でパスワードを設定することもできます（推奨）。パスワードは、そのアカウントでトランザクションを作成する際に要求されます。これでセットアップは完了ですが、まだETHを保有していません。

残高はいつでも確認できます。まず、アカウントに関連付けられたアドレスを取得します。

```
$ cast wallet address --account example
Enter keystore password:
0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0
```

次に、ブロックチェーンに残高を問い合わせます。この章のすべての例では、テストネットブロックチェーンであるEthereum Sepoliaを使用します。

```
$ cast balance 0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0 --rpc-url https://ethereum-sepolia-rpc.publicnode.com
0
```

注記

最後の `cast` コマンドにある `--rpc-url` フラグに注目してほしい。対象のブロックチェーンのRPCエンドポイントを指定する必要がある。信頼性の高いRPCエンドポイントは有料の場合が多いが、実験するだけであれば（この章で行うように）、以下のような無料の選択肢が多数ある。

- Public Node
- LlamaNodes
- ChainList

無料のSepolia ETHトークンを入手するには、オンラインのファウセットを利用できます。ここでは、図6-3に示すGoogle Cloud Web3ファウセットを使用します。0.05 ETHが提供されます。Ethereum Sepolia Faucetにアクセスし、アドレスを貼り付けて「Receive 0.05 Sepolia ETH」ボタンをクリックしてください。すぐに0.05 ETHを受け取れるはずです。

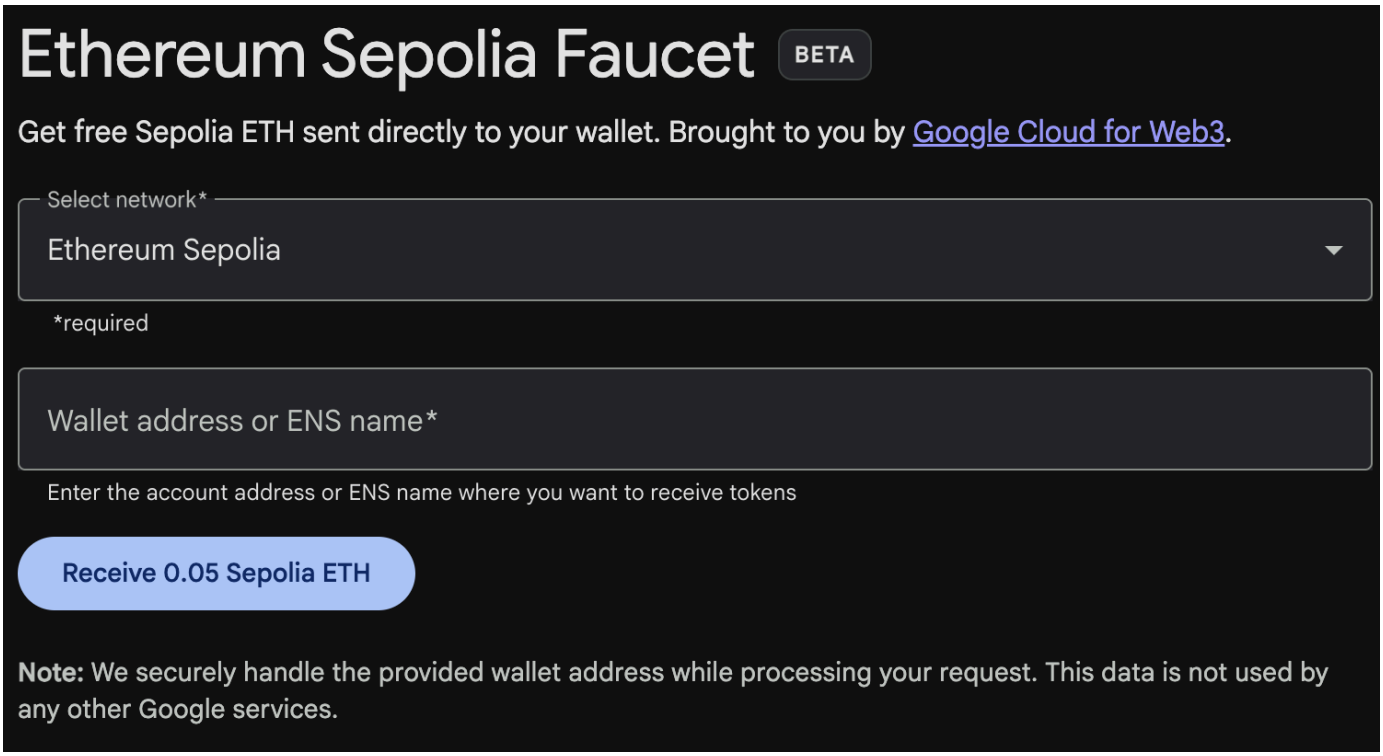


図6-3. Google Cloud Web3ファウセット

残高が変更され、0ではなくなっていることを確認できます。

```
$ cast balance 0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0 --rpc-url https://ethereum-sepolia-rpc.publicnode.com
50000000000000000
```

これでセットアップは完了です。トランザクションナンスの実験に戻りましょう。

実際には、ナンスはアカウントから発信された承認済み（つまりオンチェーン）トランザクションの最新のカウント数です。ナンスの値を確認するには、`cast`を使用してブロックチェーンに問い合わせます。新しいターミナルウィンドウを開き、次のように入力します。

```
$ cast nonce 0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0 --rpc-url https://ethereum-sepolia-rpc.publicnode.com
0
```

ヒント

ナンスはゼロベースのカウンタであり、最初のトランザクションはナンス0を持つ。実際、この例ではまだトランザクションを送信していない。また、RPCのレスポンスは常に次に利用可能なナンスを返す。たとえばあるアドレスがすでに10件のトランザクションを送信し、ナンス0〜9を使用しているとする。このときRPCレスポンスは10を返す。

ETHを送信してみましょう。イーサリアムの共同創設者であるヴィタリック・ブテリンのENSアドレスである`vitalik.eth`に0.001 etherを送信します。

```
$ cast send --account example vitalik.eth --value 0.001ether --rpc-url https://ethereum-sepolia-rpc.publicnode.com
blockHash               0xa1171309fd406e44e86be9695a597d2bf5c728738d140b9958cfb50276c32b1b
blockNumber             6989355
contractAddress
cumulativeGasUsed       18009816
effectiveGasPrice       11163498011
from                    0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0
gasUsed                 21000
logs                    []
logsBloom               0x00000000000
```

```
$ cast nonce 0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0 --rpc-url https://ethereum-sepolia-rpc.publicnode.com
11
```

このとおり、送信したトランザクションはナンスカウントを直ちに増加させませんでした。トランザクション送信後もナンスは10のままです。ネットワーク通信が落ち着き、トランザクションがブロックに含まれるまで数秒待つと、ナンスの呼び出しは期待どおりの数値11を返します。

> **注記**
>
> `cast send` コマンドで使用されている `--async` フラグに注目してほしい。このフラグを使用しない場合、`cast` はトランザクションがブロック内で承認されるまでターミナルをブロックする。このフラグを使用すると、トランザクションをネットワークに送信して直ちにトランザクションハッシュを返し、ブロックへの取り込みを待たない。

次に、別の例を見てみましょう。

```
$ cast nonce 0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0 --rpc-url https://ethereum-sepolia-rpc.publicnode.com
11
$ cast send --account example vitalik.eth --value 0.001ether --async --rpc-url https://ethereum-sepolia-rpc.publicnode.com
0x63188aa73247ffe06388a9adf399fa715e42fbc37ca53f77642a7860c80feb9d
$ cast nonce 0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0 --rpc-url https://ethereum-sepolia-rpc.publicnode.com
11
$ cast rpc eth_getTransactionCount 0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0 pending --rpc-url https://ethereum-sepolia-rpc.publicnode.com
"0xc"
$ cast nonce 0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0 --rpc-url https://ethereum-sepolia-rpc.publicnode.com
12
```

トランザクション送信前のナンスカウントは11です。トランザクションを送信し、直ちに新しいナンスを問い合わせます。前の例から予想されるとおり、トランザクションはまだメモリプール内で保留中でブロックに含まれていないため、ナンスはまだ更新されていません。しかし、トランザクションがまだ承認されていなくても実際のナンス値を取得できる新しいクエリを使用しています（`0xc` は16進数で12です）。数秒後にトランザクションがブロックに追加されると、`cast nonce` の呼び出しは新しい正しい値を返します。

`cast nonce` と `eth_getTransactionCount pending` の違いは、前者が承認済み（ブロックに含まれた）トランザクションのみを考慮するのに対し、後者はメモリプール内でまだ保留中のトランザクションも含めようとする点です。

警告

保留中のトランザクションをカウントするために `eth_getTransactionCount pending` を使用する場合は注意が必要である。このクエリはあるアドレスの実際のナンス値を返そうとするが、メモリプール内で承認待ちの他の保留中トランザクションが存在しないことを完全に保証する方法はない。

パブリックなメモリプールは普遍的なものではない。各ノードは独自のメモリプールを持っている。メモリプールとは保留中のトランザクションを一時的に保持する動的なレポジトリであり、ブロックチェーンで承認されるまでトランザクションを保管する。新しいトランザクションの受け入れや拒否に関するルールをカスタマイズすることも可能である。RPCプロバイダは大規模なノードネットワークを持ち、保留中トランザクションをほぼ網羅的に把握しているはずである。ただし、その値を100%正確なものとして扱うのは避けた方がよい。

ナンスの欠落・重複とトランザクション確認

トランザクションをプログラムで作成している場合、特に複数の独立したプロセスから同時に実行している場合、ナンスを把握することが重要です。

イーサリアムネットワークは、ナンスに基づいてトランザクションを順次処理します。ナンス0のトランザクションを送信してから、ナンス2のトランザクションを送信すると、2番目のトランザクションはどのブロックにも含まれません。2番目のトランザクションはメモリプールに保存され、イーサリアムネットワークは欠落したナンスを持つトランザクションが現れるのを待ちます。すべてのノードは、欠落したナンスが単に遅延しており、ナンス2のトランザクションが順序どおりに受信されなかっただけだと想定します。

欠落しているナンス1を含むトランザクションを送信すると、両方のトランザクション（ナンス1とナンス2）が処理されブロックに含まれます（もちろん両方が有効である場合）。欠落を埋めると、ネットワークはメモリプールで保持していた順序外のトランザクションをマイニングすることができます。

つまり、複数のトランザクションを順番に作成し、そのうちの1つが正式にどのブロックにも含まれなかった場合、以降のトランザクションはすべて「スタック」し、欠落しているナンスを持つトランザクションを待つことになります。トランザクションが無効であったりガスが不足していたりすると、ナンスシーケンスに意図しない「欠落」が発生する可能性があります。再び動かすには、欠落しているナンスを持つ有効なトランザクションを送信する必要があります。「欠落した」ナンスを持つトランザクションがネットワークによって検証されると、以降のナンスを含むすべてのブロードキャスト済みトランザクションが順次有効になることにも留意してください。トランザクションを「リコール（呼び戻し）」することはできません。

一方で、同じナンスで受信者や値が異なる2つのトランザクションを送信するなど、誤ってナンスを重複させた場合、1つが承認され、もう1つが拒否されます。どちらが承認されるかは、トランザクションを受け取った最初の検証ノードに到着する順番によって決まります。つまり、かなりランダムです。

ここまで見てきたように、ナンスの追跡は不可欠です。アプリケーションがナンスのプロセスを正しく管理しなければ、問題が発生します。残念ながら、並行処理を行おうとすると事態はさらに困難になります。次のセクションで詳しく見ていきましょう。

並行処理、トランザクションの生成とナンス

*並行処理*はコンピュータサイエンスにおける難問であり、特にイーサリアムのような分散型のリアルタイムシステムでは、時に予期せぬ問題が生じます。

簡単に言うと、並行処理とは複数の独立したシステムで同時に演算を行うことです。同じプログラム（たとえばマルチスレッド）、同じCPU（たとえばマルチプロセッシング）、または異なるコンピュータ（たとえば分散システム）上で実行できます。イーサリアムは定義上、操作（ノード、クライアント、DApp）の並行性を許容しつつ、コンセンサスを通じてシングルトン状態を強制するシステムです。

ここで、同じアドレスまたは複数のアドレスからトランザクションを生成している、複数の独立したウォレットアプリケーションがあるとします。このような状況の一例は、取引所が*ホットウォレット*（鍵がオンラインで保管されているウォレット。鍵がオンラインにならない*コールドウォレット*とは対照的）から引き出しを処理する場合です。理想的には、ボトルネックや単一障害点にならないよう複数のコンピュータで引き出しを処理すべきです。

しかし、複数のコンピュータから引き出しを行うと、厄介な並行処理の問題が急速に顕在化します。特にナンスの選択が重大な問題です。同じホットウォレットアカウントから、複数のコンピュータでトランザクションの生成、署名、ブロードキャストをどう調整すればよいのでしょうか。

1台のコンピュータを使用して、先着順にトランザクションに署名するコンピュータにナンスを割り当てるやり方が考えられます。しかし、このコンピュータ自体が単一障害点になります。さらに悪いことに、複数のナンスが割り当てられ、そのうち1つが使用されなかった場合（そのナンスでトランザクションを処理するコンピュータの障害のため）、以降のすべてのトランザクションがスタックします。

もう1つのアプローチは、トランザクションを生成するがナンスを割り当てないことです（したがってトランザクションは未署名のままとなります。ナンスはトランザクションデータの不可欠な部分であり、トランザクションを認証するデジタル署名に含める必要があることを思い出してください）。その後、署名を行いナンスを追跡する単一ノードにキューイングします。しかし、このアプローチもプロセス上のボトルネックになります。署名とナンスの追跡は、負荷がかかると混雑しやすい操作です。一方、未署名トランザクションの生成は並列化の必要性が低い部分です。いくらかの並行処理は実現できますが、プロセスの重要な部分で不十分なままです。

結局、これらの並行処理の問題と、独立したプロセスでアカウント残高やトランザクションの承認を追跡することの困難さが重なり、ほとんどの実装では並行処理を避けてボトルネックを作ることを余儀なくされます。たとえば、取引所のすべての引き出し処理を単一プロセスで実行するか、引き出しに対して完全に独立して機能し、断続的にリバランスするだけでよい複数のホットウォレットを設定するといった対応です。

トランザクションガス

ガスについては前の章で少し説明しましたが、14章でより詳細に取り上げます。ここでは、トランザクションの`gasPrice`と`gasLimit`の役割に関する基本事項を説明します。

ガスはイーサリアムの燃料です。ガスはイーサではありません。イーサに対する独自の為替レートを持つ、別の暗号通貨です。イーサリアムは、世界中の何千ものコンピュータで処理されるトランザクションが使用できるリソース量を制御するためにガスを使用します。オープンエンド（チューリング完全）な計算モデルでは、DoS攻撃や不用意にリソースを浪費するトランザクションを防ぐ必要があります。そのためには、なんらかの計測方式が欠かせません。

ガスがイーサから分離されているのは、イーサの価値の急激な変動からシステムを保護するためであり、また計算、メモリ、ストレージといった重要資源のコスト比率を管理するためです。

トランザクションの`gasPrice`フィールドでは、トランザクション発行者がガスと引き換えに支払う価格を設定できます。価格はガス単位あたりのweiで測定されます。

> **ヒント**
>
> 人気サイトのEtherscanでは、イーサリアムメインネットワークのガス価格や、その他の関連するガス指標に関する情報を提供している。

ウォレットは、トランザクションの承認を迅速化するために、発行したトランザクションの`gasPrice`を調整できます。`gasPrice`が高いほど、トランザクションはより早く承認される可能性が高くなります。逆に、優先順位の低いトランザクションは価格を下げることで、承認が遅くなります。`gasPrice`に設定できる最小値は、トランザクションが含まれるブロックのベースフィー（EIP-1559トランザクションで導入された概念）と等しくなります。

> **注記**
>
> Londonハードフォーク以前およびEIP-1559以前は、許容される`gasPrice`の最小値はゼロであった。つまり、ウォレットは完全に無料のトランザクションを生成できた。容量によっては承認されない場合もあったが、プロトコルに無料トランザクションを禁止する仕組

みはなかった。イーサリアムの初期数カ月間に、ブロックチェーンに正常に含まれた無料トランザクションの例がいくつか確認できる。

EIP-1559：ベースフィーとプライオリティフィー

すでに簡単に説明したとおり、EIP-1559はプロトコルの新しいパラメータであるベースフィーを導入することで、イーサリアムの手数料市場の構造を根本的に変更しました。ベースフィーは、トランザクションが有効と見なされブロックに含まれるために支払う必要のある最小ガス価格を表します。

ベースフィーとトランザクションが実際に支払うガス手数料との差額は、*プライオリティフィー*と呼ばれます。プライオリティフィーは、トランザクションを含むブロックを作成したバリデータに直接渡されます。

適切なガス価格の調べ方

Castは、複数のブロックにわたるガス価格の中央値を計算することで、ガス価格を提案します。

```
$ cast gas-price --rpc-url https://ethereum-sepolia-rpc.publicnode.com
4845187414
```

ガスに関連する2番目の重要なフィールドは`gasLimit`です。簡単に言えば、`gasLimit`はトランザクション発行者がトランザクションを完了するために購入する意思のあるガスの最大単位数を示します。単純な支払い、つまりあるEOA（外部所有アカウント）から別のEOAにイーサを送金するトランザクションでは、必要なガス量は21,000ガス単位に固定されています。イーサでの手数料を計算するには、21,000に支払う意思のある`gasPrice`（EIP-1559トランザクションの場合は`maxFeePerGas`）を掛けます。

トランザクションの宛先アドレスがコントラクトである場合、必要なガスの量を見積もることはできますが、正確に決定することはできません。コントラクトは異なる実行パスにつながる異なる条件を評価でき、実行パスによって総ガスコストが異なるためです。制御できず予測もできない条件に応じて、コントラクトは単純な計算だけを行うかもしれませんし、より複雑な計算を実行するかもしれません。

例を見てみましょう。呼び出されるたびにカウンタを増やし、呼び出し回数に等しい回数だけ特定のループを実行するスマートコントラクトを書いたとします。100回目の呼び出しで宝くじのような特別な賞を出すとすると、賞を計算するために追加の計算が必要です。コントラクトを99回呼び出すまでは単一の処理が行われますが、100回目の呼び出しではまったく異なる処理が発生します。そのために支払うガスの量は、トランザクションがブロックに含まれる前に他のいくつのトランザクションがその関数を呼び出したかに依存します。見積もりとしては99回目のト

ランザクションに基づいていたとしても、トランザクションが承認される直前に他の誰かが99回目にコントラクトを呼び出したとします。すると、あなたのトランザクションは100回目の呼び出しとなり、計算量（およびガスコスト）は想定よりもずっと高くなります。

イーサリアムでよく使われるたとえですが、`gasLimit`は車（トランザクション）の燃料タンクの容量と見立てることができます。旅に必要となる量のガソリン（トランザクションを検証するために必要な計算）をタンクに入れます。必要な量はある程度見積もることができますが、燃料消費を増加させる迂回（より複雑な実行パス）のような予期しない変更が発生する可能性があります。

しかし、この燃料タンクのたとえはやや誤解を招きます。実際には、旅行から帰ったあとに使用したガスの量に応じて支払う、ガソリンスタンド会社のクレジットアカウントのようなものです。トランザクションを送信する際、最初の検証ステップの1つとして、発行元アカウントに`maxFeePerGas × gasLimit`（レガシートランザクションの場合は`gasPrice × gasLimit`）を支払うのに十分なイーサがあることを確認します。しかし、トランザクションの実行が完了するまで、実際にはアカウントから金額が差し引かれることはありません。実際に請求されるのは、トランザクションで消費されたガス分だけです。ただし送信時点では、最大額を支払える残高が必要です。

トランザクション受信者

トランザクションの受信者は`to`フィールドで指定されます。このフィールドには20バイトのイーサリアムアドレスが格納されます。アドレスはEOA（外部所有アカウント）またはコントラクトのアドレスにすることができます。

イーサリアムは`to`フィールドに対してこれ以上の検証を行いません。20バイトの値はすべて有効と見なされます。対応する秘密鍵のないアドレスや、対応するコントラクトがないアドレスであっても、トランザクションは有効です。アドレスが存在する公開鍵から（したがって秘密鍵から）正しく導出されたかどうかを、イーサリアムは知る方法がありません。

> **警告**
>
> イーサリアムプロトコルは、トランザクション内の受信者アドレスを検証しない。対応する秘密鍵やコントラクトが存在しないアドレスに送信すると、イーサは「バーン（焼却）」され、永遠に使用できなくなる。検証はユーザーインターフェイスレベルで行う必要がある。

トランザクションを誤ったアドレスに送信した場合、送信されたイーサは焼却され、永遠にアクセスできなく（使用不可に）なります。ほとんどのアドレスは既知の秘密鍵を持っていないため、署名を生成して使用することができないためです。アドレスの検証はユーザーインターフェイスレベルで行われるものと想定されています（「大文字を使用するチェックサム付き16進数エ

ンコーディング（ERC-55）」を参照）。実際にイーサを焼却する正当な理由はいくつかあります。たとえば、ペイメントチャネルや他のスマートコントラクトでの不正行為に対する抑止策として使われます。イーサの量は有限であるため、イーサを焼却することで、焼却された分の価値がすべてのイーサ保有者に（保有するイーサの量に比例して）効果的に分配されます。

トランザクションのバリューとデータ

トランザクションの主な「ペイロード」は`value`と`data`という2つのフィールドに格納されています。トランザクションはバリューとデータの両方、バリューのみ、データのみ、またはバリューとデータの両方を持たないこともできます。この4つの組み合わせはすべて有効です。

バリューのみのトランザクションは支払いです。データのみのトランザクションは呼び出しです。バリューとデータの両方を持つトランザクションは、支払いと呼び出しの両方です。バリューもデータもないトランザクションは、おそらくガスの無駄でしかありませんが、それでも可能です。

これらの組み合わせをすべて試してみましょう。以前と同様に`cast`を使用して、Sepoliaテストネットでトランザクションを送信します。

最初のトランザクションにはバリュー（支払い）のみが含まれ、データペイロードはありません。

```
$ cast send --account example vitalik.eth --value 0.001ether --rpc-url https://ethereum-
sepolia-rpc.publicnode.com
```

図6-4に示すように、送信されたバリューは0.001 etherであり、データペイロード（Etherscan上のinput data）は空（`0x00`）です。

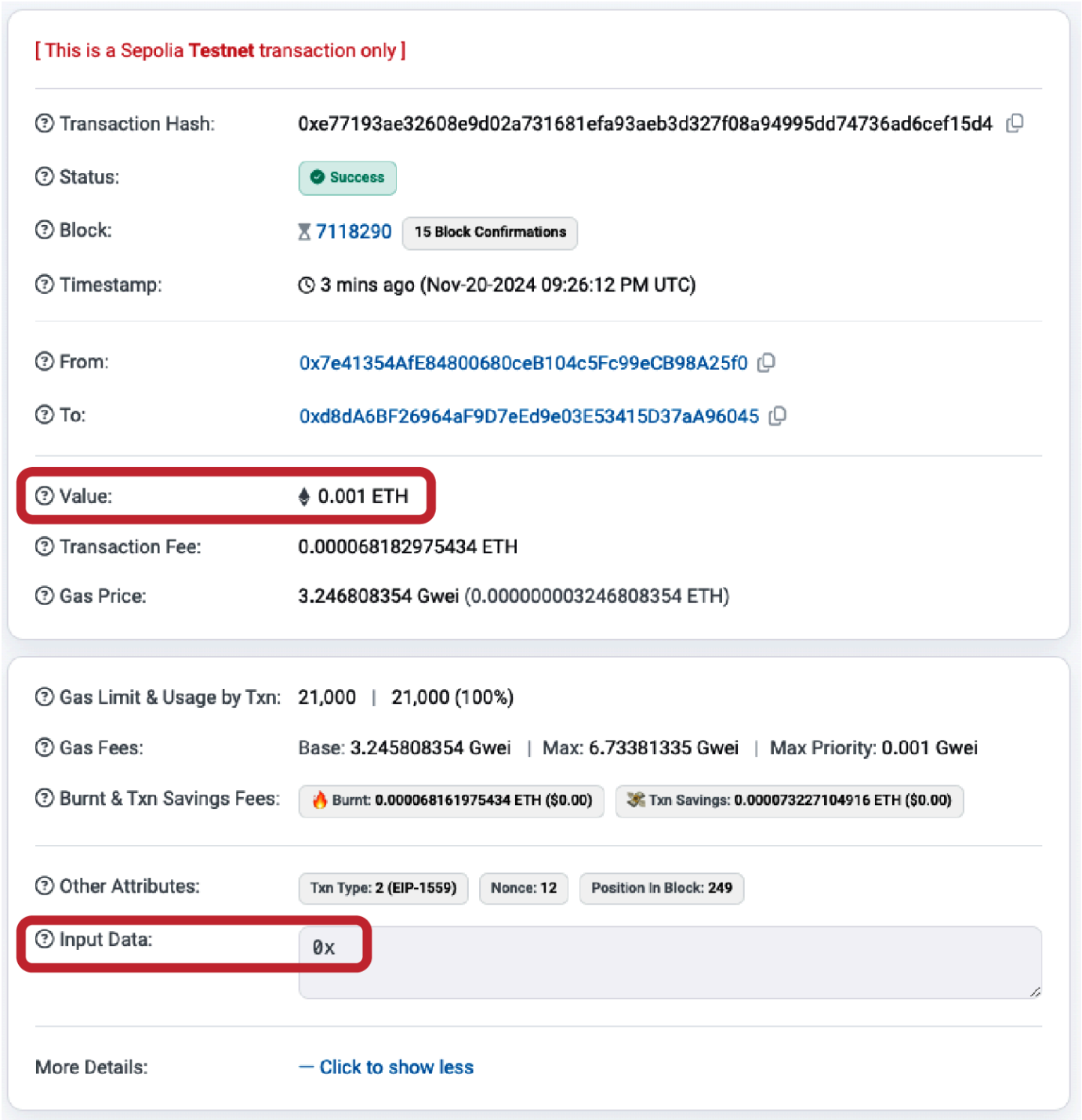


図6-4. バリューのみのトランザクション（支払い）

次の例では、バリューとデータペイロードの両方を指定しています（EOA宛に送信するため、このペイロードは無視されます）。

```
$ cast send --account example vitalik.eth 0x0001 --value 0.001ether --rpc-url https://ethereum-sepolia-rpc.publicnode.com
```

図6-5に示すように、input dataには値が含まれており、具体的には`0x0001`です。

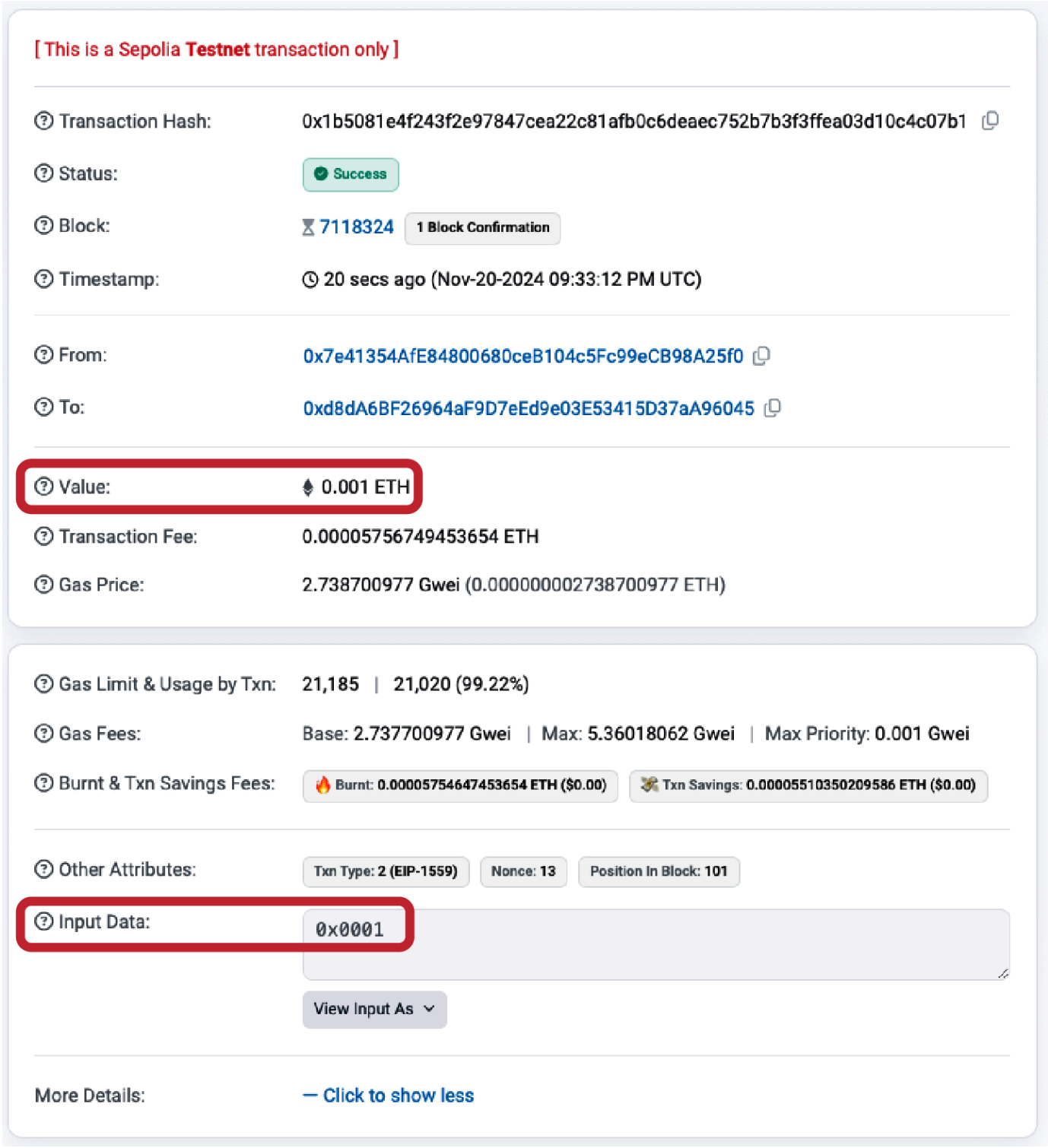


図6-5. バリューとデータの両方を含むトランザクション

次のトランザクションにはデータペイロードが含まれますが、バリューはゼロを指定しています。

```
$ cast send --account example vitalik.eth 0x0001 --rpc-url https://ethereum-sepolia-
rpc.publicnode.com
```

図6-6に示す確認画面では、トランザクションで送信されたバリューは0 etherで、データペイロードは`0x0001`です。

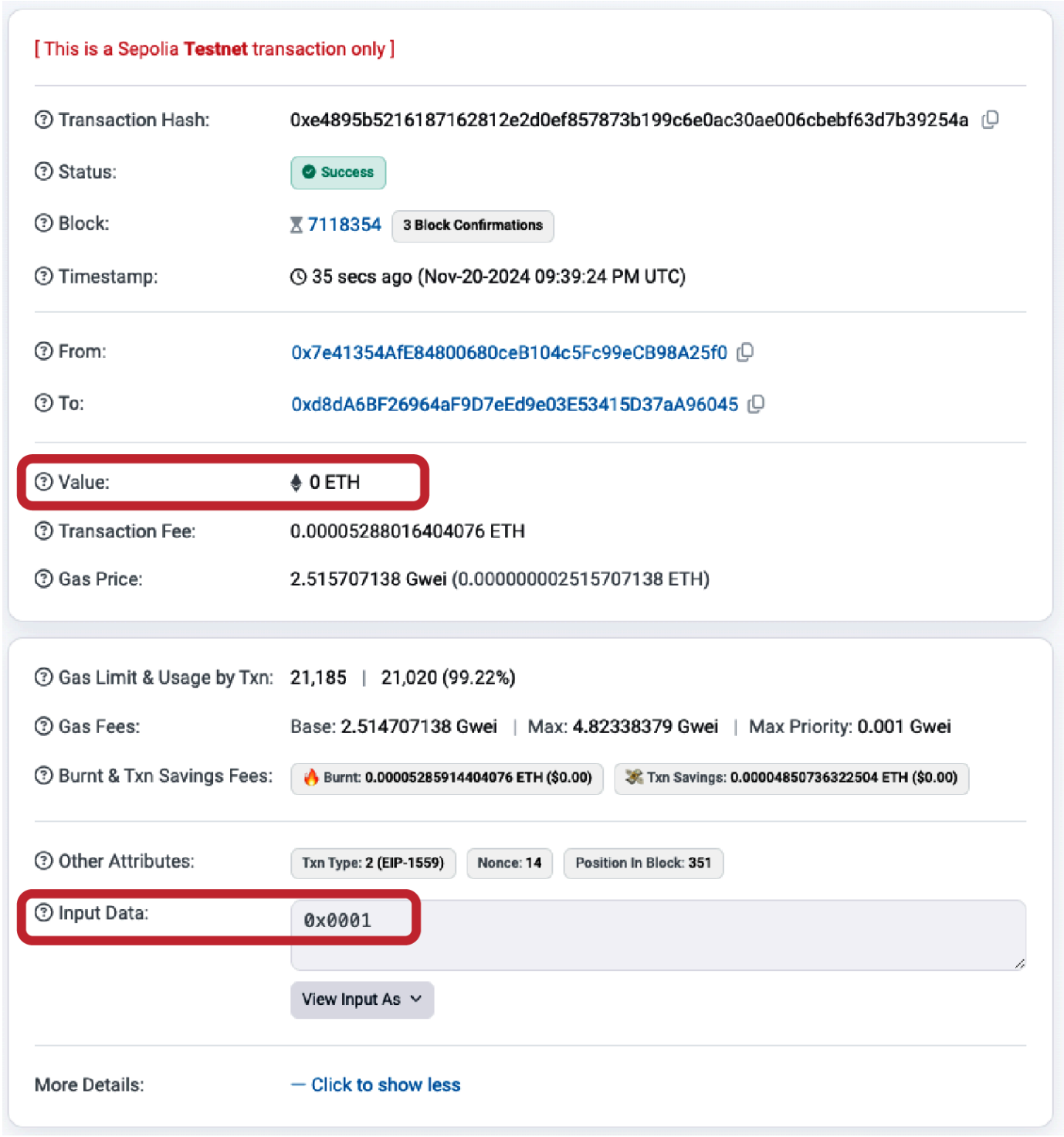


図6-6. データのみのトランザクション（呼び出し）

最後のトランザクションは、送信するバリューもデータペイロードも含んでいません。

```
$ cast send --account example vitalik.eth --rpc-url https://ethereum-sepolia-rpc.publicnode.com
```

図6-7に示すトランザクションは、0 etherを送信し、空のペイロードを含んでいます。

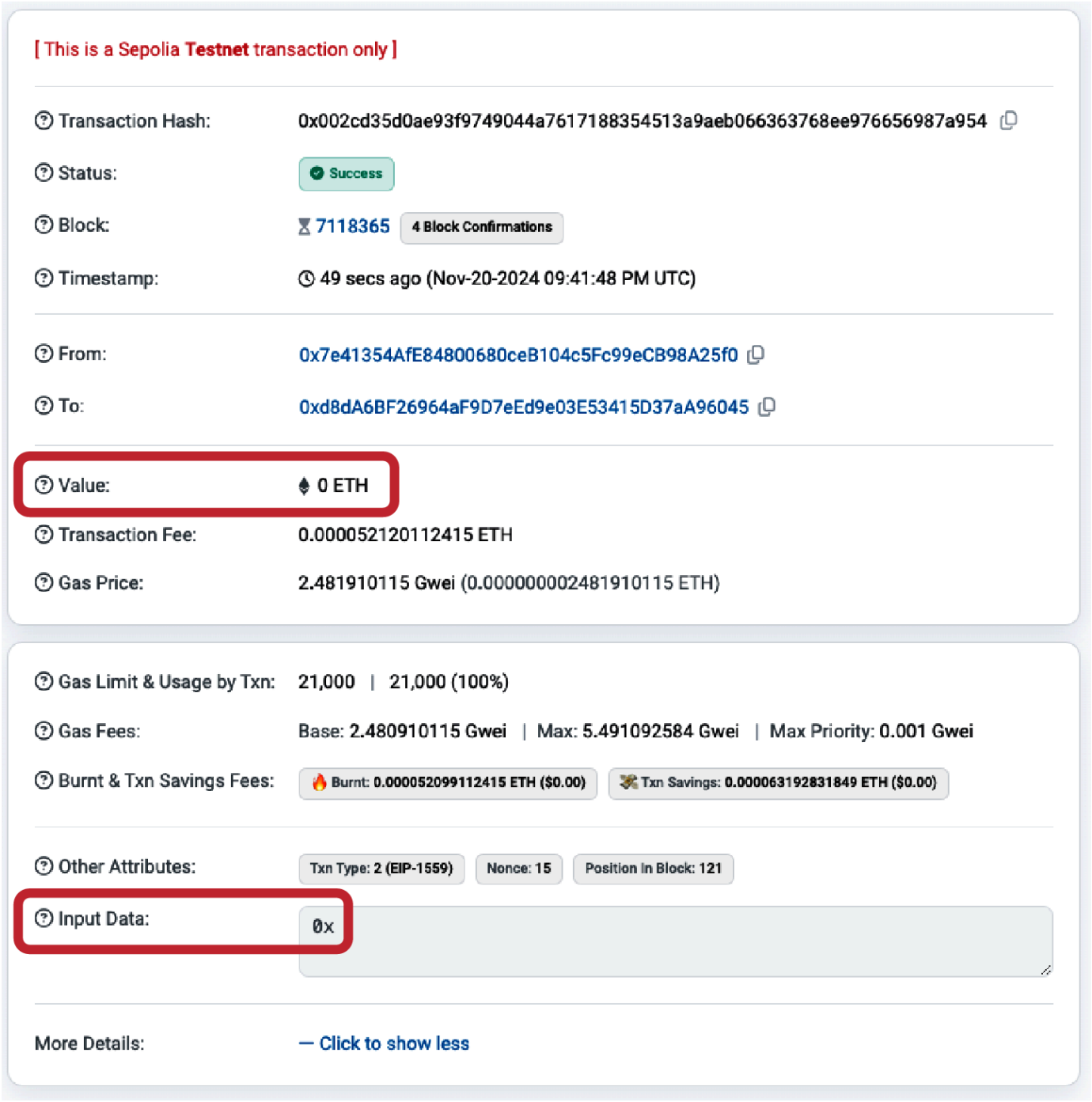


図6-7. バリューもデータもないトランザクション

EOAとコントラクトへのバリューの送信

バリューを含むイーサリアムトランザクションを構築した場合、それは支払いに相当します。宛先アドレスがコントラクトであるかどうかによって、トランザクションの動作は異なります。

EOA（外部所有アカウント）アドレス、つまりブロックチェーン上でコントラクトとしてフラグが立てられていないアドレスの場合、イーサリアムは状態の変更を記録し、送信されたバリューをアドレスの残高に追加します。以前に確認されていないアドレスの場合は、クライアントの状態の内部表現に追加され、残高は支払い額で初期化されます。

宛先アドレス（`to`）がコントラクト（またはEIP-7702トランザクションを通じてコントラクトを委任済みのEOA）の場合、EVMはコントラクトを実行し、トランザクションのデータペイロードに指定された関数の呼び出しを試みます。トランザクションにデータがない場合、EVMは*フォールバック関数*を呼び出し、その関数がpayable（支払い可能）であれば、フォールバック関数を実行して次の処理を決定します。フォールバック関数がない場合、トランザクションの効果はウォレットへの支払いとまったく同様に、コントラクトの残高を増加させることになります。

コントラクトは、関数が呼び出された時点で、または関数にコーディングされた条件に基づいて、即座に例外をスローすることで入金を拒否できます。関数が正常に（例外なく）終了すると、コントラクトの状態はコントラクトのイーサ残高の増加を反映するように更新されます。

EOAやコントラクトへのデータペイロードの送信

トランザクションにデータが含まれている場合、そのほとんどはコントラクトアドレス宛です。ただし、EOAにデータペイロードを送信することもイーサリアムプロトコルでは問題ありません。その場合、データの解釈はEOAにアクセスするために使用するウォレット次第です。イーサリアムプロトコルではデータは無視されます。また、ほとんどのウォレットは、自分が管理するEOAへのトランザクションで受信したデータを無視します。

将来的には、ウォレットがコントラクトのようにデータを解釈できるようにする標準規約が生まれるかもしれません。そうなれば、トランザクションがユーザーのウォレット内で実行されている機能を呼び出せるようになります。重要な違いは、EOAによるデータペイロードの解釈は、コントラクトの実行とは異なり、イーサリアムのコンセンサスルールの対象にならないことです。

ここでは、トランザクションがコントラクトアドレスにデータを送信していると仮定します。その場合、データはEVM（イーサリアム仮想マシン）によってコントラクトの呼び出しとして解釈されます。ほとんどのコントラクトは、このデータをより具体的に関数の呼び出しとして使用し、名前付き関数を呼び出してエンコードされた引数を渡します。

*アプリケーションバイナリインターフェイス（ABI）*に準拠したコントラクト（すべてのコントラクトがそうであると想定できます）に送信されるデータペイロードは、以下の16進シリアライズエンコーディングです。

関数セレクタ

関数のプロトタイプのKeccak-256ハッシュの最初の4バイト。これにより、コントラクトはどの関数を呼び出したいのかを明確に識別できる。

関数の引数

ABI仕様で定義されたさまざまな基本型の規則に従ってエンコードされた関数の引数。

例2-1では、引き出しのための関数を定義しました。

```
function withdraw(uint256 _withdrawAmount, address payable _to) public {
```

関数のプロトタイプは、関数名を含む文字列として定義され、関数名のあとに各引数のデータ型が括弧で囲まれ、コンマで区切られます。ここでの関数名は`withdraw`であり、2つの引数を取ります。

- `uint256`型の`_withdrawAmount`
- `address`型の`_to`

したがって`withdraw`のプロトタイプは次のようになります。

```
withdraw(uint256,address)
```

> **注記**
>
> `payable`キーワードはSolidityでアドレスがイーサを受け取れることを示すために使用されるが、関数セレクタの計算には含まれない。プロトタイプには基本型の`address`のみが含まれる。

この文字列のKeccak-256ハッシュを計算しましょう。

```
$ cast keccak256 "withdraw(uint256,address)"
0x00f714ce93c4a188ecc0c802ca78036f638c1c4b3ee9b98f3ed75364b45f50b1
```

ハッシュの最初の4バイトは`0x00f714ce`です。これが関数セレクタの値であり、どの関数を呼び出したいのかをコントラクトに伝えます。

次に、引数`withdraw_amount`と`_to`として渡す2つの値を計算しましょう。0.000001 etherを`vitalik.eth`（`0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045`）のアドレスに引き出します。前のステップで計算した関数セレクタと組み合わせてエンコードし、最終的なデータペイロード（*calldata*とも呼ばれます）を生成します。

```
$ cast calldata "withdraw(uint256,address)" 0.000001ether
0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045
0x00f714ce000000000000000000000000000000000000000000000000000e8d4a510000000000000000000000000000000000000000d8da6bf26964af9d7eed9e03e53415d37aa96045
```

これがトランザクションのデータペイロードです。`withdraw`関数を呼び出し、`withdraw_amount`として0.000001 ether、`_to`アドレスとして`vitalik.eth`を要求しています。

特別なトランザクション：コントラクト作成

ここで触れておくべき特殊なケースとして、ブロックチェーン上に新しいコントラクトを作成し、今後使用できるようにデプロイするトランザクションがあります。*コントラクト作成トランザクション*は、空の`to`フィールド（null）によって識別されます。`to`フィールドが空の場合、イーサリアムプロトコルは新しいコントラクトのデプロイ要求として解釈し、`data`フィールドに含まれるバイトコードを使用します。

ゼロアドレス（`0x0000000000000000000000000000000000000000`）はこれとは異なる概念です。ゼロアドレスはイーサを受け取ることができる有効な20バイトのアドレスです。ゼロアドレスにトランザクションを送信しても、コントラクトは作成されず、他のトランザクションと同様にイーサがそのアドレスに送金されるだけです。ゼロアドレスはさまざまなアドレスから支払いを受け取ることがあります。理由は2つ考えられます。偶然によるもの（イーサの損失）か、意図的な*イーサ焼却*（使用できないアドレスにイーサを送ることで意図的に破壊すること）です。ただし、意図的にイーサ焼却を行いたい場合は、ネットワークに対して意図を明確にし、代わりに特別に指定された焼却アドレスを使用すべきです。

```
0x000000000000000000000000000000000000dEaD
```

> **警告**
>
> 指定された焼却アドレスに送信されたすべてのイーサは使用不可能になり、永遠に失われる。

コントラクト作成トランザクションに含める必要があるのは、コントラクトを作成するコンパイル済みバイトコードを含むデータペイロードだけです。このトランザクションの唯一の効果は、コントラクトを作成することです。新規コントラクトに初期残高を持たせたい場合は、`value`フィールドにイーサ量を含めることもできますが、これは完全に任意です。データペイロードなし（コントラクトなし）でコントラクト作成アドレスにバリュー（イーサ）を送信すると、焼却アドレスに送信した場合と同じ効果になります。入金先のコントラクトが存在しないため、イーサは失われます。

例として、空の`to`フィールドとコントラクトバイトコードを含むデータペイロードを持つトランザクションを手動で作成することで、2章で使用した`Faucet.sol`コントラクトをデプロイできます。コントラクトはバイトコード表現にコンパイルする必要があります。これはSolidityコンパイラで行うことができます。

```
$ solc --bin Faucet.sol
Binary:
6080604052348015600e575f5ffd5…0033
```

Remixオンラインコンパイラからも同じ情報を得ることができます。

バイナリ出力を使用してトランザクションを作成できます。

```
$ cast send --account example --rpc-url https://ethereum-sepolia-rpc.publicnode.com --create
608060405234801560e575f5ffd5…0033
```

コントラクトが公開されると、図6-8に示すようにEtherscanブロックエクスプローラで確認できます。

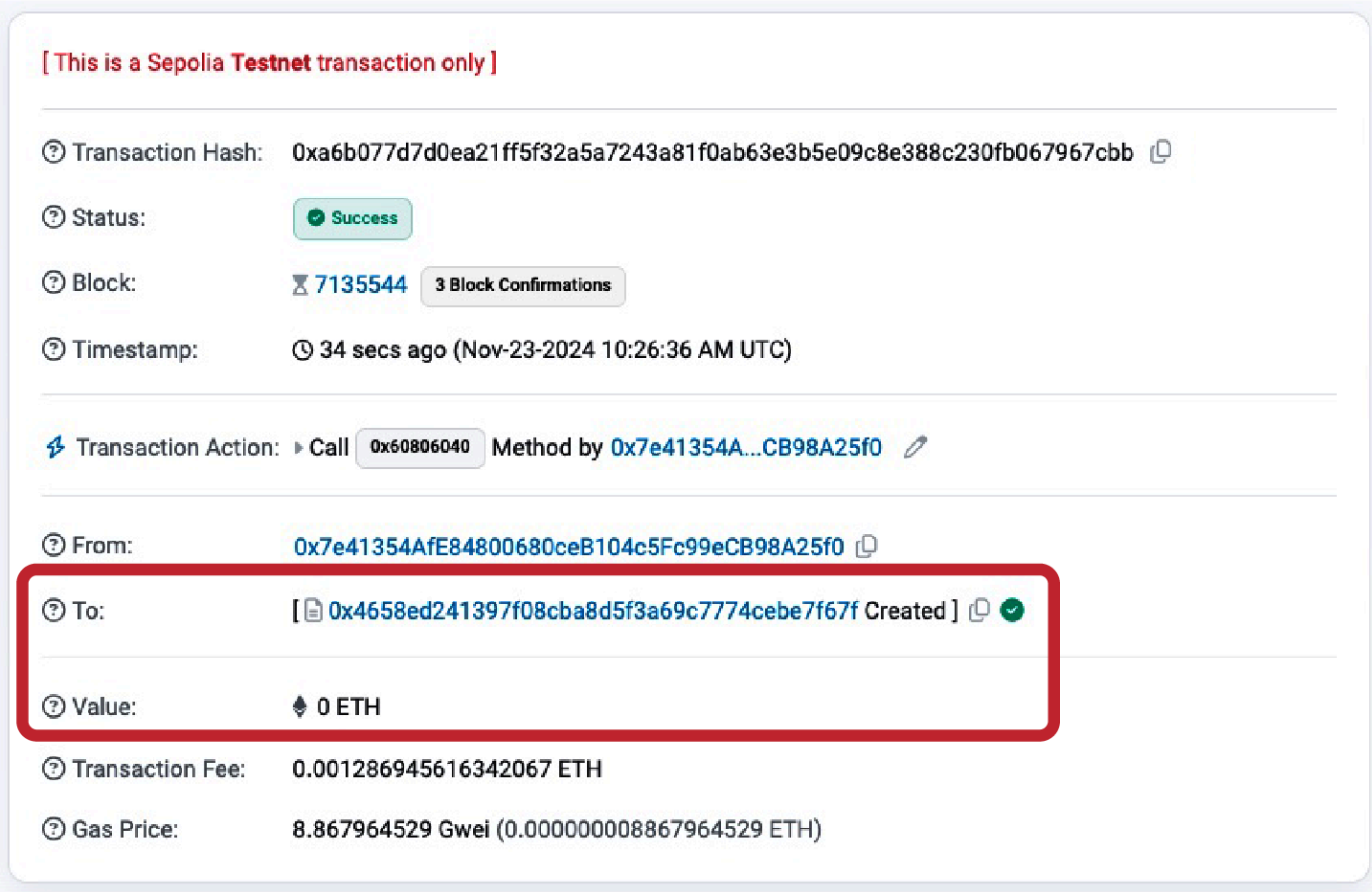


図6-8. Etherscan上のコントラクト作成トランザクション

トランザクションのレシートを確認して（トランザクションハッシュを使用して参照）、コントラクトに関する情報を取得できます。

```
$ cast receipt 0xa6b077d7d0ea21ff5f32a5a7243a81f0ab63e3b5e09c8e388c230fb067967cbb \
    --rpc-url https://ethereum-sepolia-rpc.publicnode.com
blockHash               0x6eb071eac79a84793321b086af96b32c1d861f04b0efc7354d0f6b8d5a8fa36a
blockNumber             7135544
contractAddress         0x4658eD241397F08cba8d5F3a69c7774cebE7f67F
cumulativeGasUsed       28390874
effectiveGasPrice       8867964529
from                    0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0
gasUsed                 145123
logs                    []
logsBloom
0x00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
```

```
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000
root
status                  1 (success)
transactionHash         0xa6b077d7d0ea21ff5f32a5a7243a81f0ab63e3b5e09c8e388c230fb067967cbb
transactionIndex        136
type                    2
blobGasPrice
blobGasUsed
authorizationList
```

レシートにはコントラクトのアドレス（`contractAddress`を参照）が含まれており、前のセクションで示したように、コントラクトへの送金やコントラクトからの受領に使用できます。

まず、新しく作成されたコントラクトアドレスを変数に保存します。

```
$ CONTRACT_ADDRESS=0x4658eD241397F08cba8d5F3a69c7774cebE7f67F
```

次に、イーサを送金してコントラクトに資金を入れます。

```
$ cast send --account example $CONTRACT_ADDRESS --value 0.02ether --rpc-url https://ethereum-
sepolia-rpc.publicnode.com
```

最後に、先ほど計算したデータペイロードを使用して`withdraw`関数を呼び出し、0.000001 etherを`vitalik.eth`のアドレスに引き出します。

```
$ cast send --account example $CONTRACT_ADDRESS \
0x00f714ce0000000000000000000000000000000000000000000000000000000000000000e8d4a510000000000000000000000000
000d8da6bf26964af9d7eed9e03e53415d37aa96045 \
  --rpc-url https://ethereum-sepolia-rpc.publicnode.com
```

しばらくすると、両方のトランザクションが図6-9に示すようにEtherscanで表示されます。

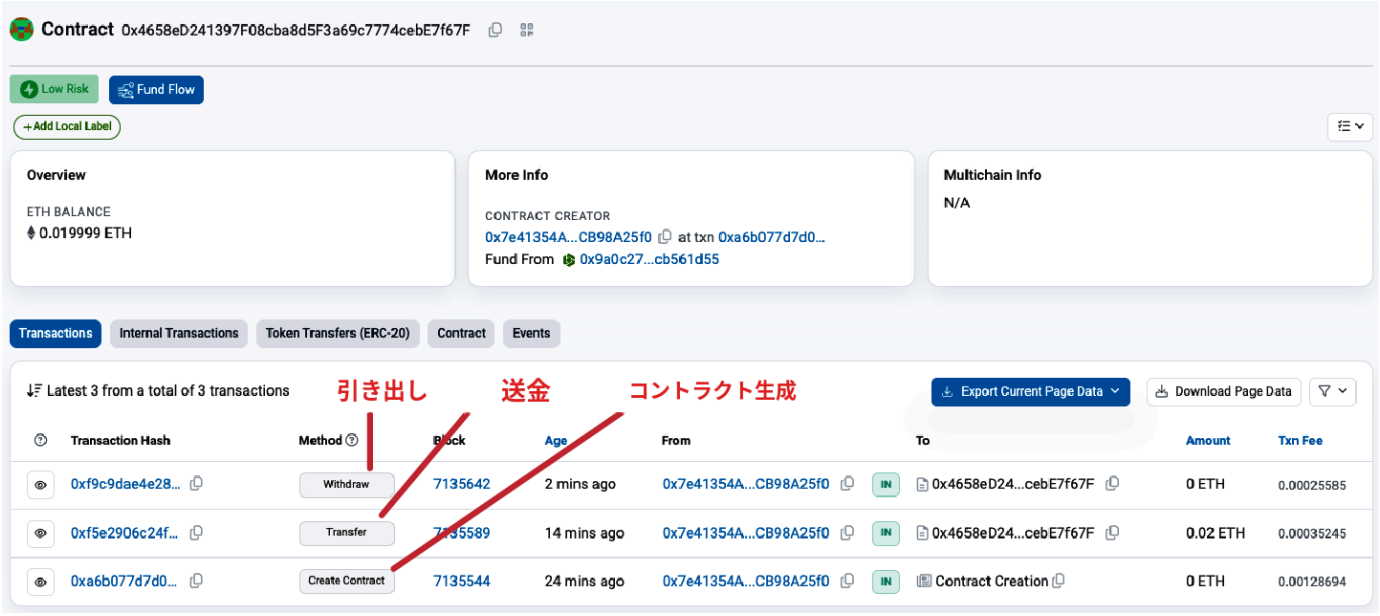


図6-9. コントラクトへの送金と引き出しのトランザクション

デジタル署名

ここまで、デジタル署名の詳細については触れてきませんでした。このセクションでは、デジタル署名がどのように機能するか、そして秘密鍵を明かすことなく秘密鍵の所有権証明を提示するためにどのように使用できるかを説明します。

ECDSA

イーサリアムで使用されるデジタル署名アルゴリズムは、*楕円曲線デジタル署名アルゴリズム（ECDSA）*です。「楕円曲線暗号の解説」で説明しているように、楕円曲線の秘密鍵と公開鍵のペアに基づいています。

デジタル署名はイーサリアムで3つの目的を果たします（以下のサイドバーを参照）。第1に、署名は秘密鍵の所有者（つまりイーサリアムアカウントの所有者）が、イーサの支払いまたはコントラクトの実行を承認したことを証明します。第2に、*否認防止*を保証します。つまり、承認の証拠を否定することはできません。第3に、署名はトランザクションデータが署名後に誰にも変更されていないこと、そして変更できないことを証明します。

デジタル署名の定義

Wikipediaによれば、デジタル署名は、デジタルメッセージやデジタルドキュメントの信頼性を提示するための数学的手法です。有効なデジタル署名は、既知の送信者によってメッセージが作成されたこと（認証）、送信者がメッセージの送信を否認できないこと（否認防止）、そしてメッセージが送信中に変更されなかったこと（完全性）を受信者が信じる根拠を示します。

デジタル署名の仕組み

デジタル署名は2つの部分からなる数学的手法です。1つ目は、秘密鍵（署名鍵）を使用してメッセージ（ここではトランザクション）から署名を作成するアルゴリズムです。2つ目は、メッセージと公開鍵だけを使って誰でも署名を検証できるようにするアルゴリズムです。

デジタル署名の作成

イーサリアムのECDSA実装では、署名される「メッセージ」はトランザクション、より正確にはトランザクションのRLPエンコードされたデータのKeccak-256ハッシュです。署名鍵はEOA（外部所有アカウント）の秘密鍵であり、結果は署名です。

```
Sig = Fsig (Fkeccak256 (m), k)
```

ここでの各変数の意味は以下のとおりです。

- `k` は署名する秘密鍵
- `m` はRLPエンコードされたトランザクション
- `Fkeccak256` はKeccak-256ハッシュ関数
- `Fsig` は署名アルゴリズム
- `Sig` は結果の署名

関数 `Fsig` は、一般に `r` および `s` と呼ばれる2つの値で構成される署名 `Sig` を生成します。

```
Sig = (r, s)
```

署名の検証

署名を検証するには、署名（`r` と `s`）、シリアライズされたトランザクション、および署名の作成に使用された秘密鍵に対応する公開鍵が必要です。本質的に、署名の検証とは、「この公開鍵を生成した秘密鍵の所有者だけが、このトランザクションでこの署名を生成できた」ということを意味します。

署名検証アルゴリズムは、メッセージ（ここではトランザクションのハッシュ）、署名者の公開鍵、および署名（`r` と `s` の値）を受け取り、署名がこのメッセージおよび公開鍵に対して有効である場合にtrueを返します。

ECDSAの数学

前述のように、署名は2つの値 `r` と `s` で構成され、数学的関数 `Fsig` によって作成されます。このセクションでは、関数 `Fsig` をより詳細に見ていきます。

署名アルゴリズムは、まず暗号学的に安全な方法で*一時的な*（エフェメラルな）秘密鍵を生成します。この一時的な鍵は`r`と`s`の値の計算に使用され、イーサリアムネットワーク上で署名されたトランザクションを監視している攻撃者が送信者の実際の秘密鍵を計算できないようにします。

4章で説明したように、一時的な秘密鍵は対応する（一時的な）公開鍵を導出するために使用されます。

- 暗号学的に安全な乱数`q`は、一時的な秘密鍵として使用される
- 対応する一時公開鍵`Q`は、`q`と楕円曲線生成元`G`から生成される

デジタル署名の`r`値は、一時公開鍵`Q`のx座標です。

そこから、アルゴリズムは署名の`s`値を以下のように計算します。

```
s ≡ q-1 (Keccak256(m) + r * k) (mod p)
```

ここでの各変数の意味は以下のとおりです。

- `q`は一時的な秘密鍵
- `r`は一時公開鍵のx座標
- `k`はEOA（外部所有アカウント）の所有者の署名用秘密鍵
- `m`はトランザクションデータ
- `p`は楕円曲線の素数位数

検証は署名生成関数の逆で、`r`と`s`の値および送信者の公開鍵を使用して、楕円曲線上の点である値`Q`（署名作成時に使用された一時公開鍵）を計算します。手順は以下のとおりです。

1. すべての入力が正しく構成されていることを確認する
2. `w = s-1 mod p`を計算する
3. `u1 = Keccak256(m) * w mod p`を計算する
4. `u2 = r * w mod p`を計算する
5. 楕円曲線上の点`Q ≡ u1 * G + u2 * K (mod p)`を計算する

ここでの各変数の意味は以下のとおりです。

- `r`と`s`は署名値
- `K`は署名者（EOAの所有者）の公開鍵
- `m`は署名されたトランザクションデータ
- `G`は楕円曲線生成元
- `p`は楕円曲線の素数位数

計算された点`Q`のx座標が`r`に等しい場合、検証者は署名が有効であると結論づけることができます。署名の検証では、秘密鍵は知られることも公開されることもないことに注意してください。

ヒント

ECDSAを説明するにはどうしても複雑な数学を扱わなければならず、完全に説明するのは本書の範囲を超えている。多数のガイドがオンラインで順を追って説明してくれる。「ECDSA explained」で検索するか、こちらにアクセスしてみてほしい。

トランザクション署名の実践

有効なトランザクションを生成するには、発行者がECDSAを使用してメッセージにデジタル署名する必要があります。「トランザクションに署名する」とは、実際には「RLPシリアライズされたトランザクションデータのKeccak-256ハッシュに署名する」ことを意味します。署名はトランザクションそのものではなく、トランザクションデータのハッシュに対して適用されます。

イーサリアムでトランザクションに署名するには、発行者が以下の手順を実行しなければなりません。

1. 特定のトランザクションタイプに必要なすべてのフィールドを含むトランザクションデータ構造を作成する
2. トランザクションデータ構造のRLPエンコードされたシリアライズメッセージを生成する
3. シリアライズされたメッセージのKeccak-256ハッシュを計算する
4. 発行元EOA（外部所有アカウント）の秘密鍵でハッシュに対するECDSA署名を計算する
5. ECDSA署名の計算結果である`v`、`r`、`s`の値をトランザクションに追加する

特別な署名変数`v`は、チェーンIDとリカバリ識別子の2つを示します。リカバリ識別子は、ECDSArecover関数が署名を検証する際に使用されます。非レガシートランザクションでは、チェーンIDがトランザクション自体の項目として直接含まれるため、`v`変数にはチェーンIDがエンコードされません。チェーンIDの詳細については、「EIP-155による生トランザクションの作成」を参照してください。リカバリ識別子は、公開鍵のy成分のパリティを示すために使用されます（詳細は「署名プレフィックス値（v）と公開鍵リカバリ」を参照してください）。

注記

ブロック番号2,675,000で、イーサリアムはSpurious Dragonハードフォークを実装した。この変更により、トランザクションリプレイ保護（あるネットワーク向けのトランザクションが別のネットワークでリプレイされることを防止する機能）を含む新しい署名方式が導入された。この新しい署名方式はEIP-155で規定されている。トランザクションの形式と署名に影響を与えるため、2つの形式のいずれかをとる3つの署名変数の最初のもの（すなわち`v`）に注意が必要である。`v`の値は、ハッシュ化されるトランザクションメッセージに含まれるデータフィールドを示す。

生トランザクションの作成と署名

この節では、`ethers.js`ライブラリを使用して生トランザクションを作成し、署名します。例6-1は、ウォレットの内部やユーザーに代わってトランザクションに署名するアプリケーションで通常使用される関数を示しています。

例6-1. 生のイーサリアムトランザクションの作成と署名

```
// 必要なパッケージを先にインストール:
//
// npm install ethers
//
// 実行方法: node eip1559_tx.js
import { ethers } from "ethers";

// RPCエンドポイントでプロバイダを作成
const provider = new ethers.JsonRpcProvider("https://ethereum-sepolia-rpc.publicnode.com");

// 秘密鍵
const privKey = "0xd6d2672c6b4489e6bcd4e93b9af620fa0204b639b7d7f93765479c0846be0b58";

// ウォレットインスタンスを作成
const wallet = new ethers.Wallet(privKey);

// ナンスを取得してトランザクションデータを作成
const txData = {
  nonce: await provider.getTransactionCount(wallet.address), // プロバイダからナンスを取得
  to: "0xb0920c523d582040f2bcb1bd7fb1c7c1ecebdb34", // 受信者アドレス
  value: ethers.parseEther("0.0001"), // 送信量（ここでは0.0001 ETH）
  gasLimit: ethers.toBeHex(0x30000), // ガスリミット
  maxFeePerGas: ethers.parseUnits("100", "gwei"), // ガスあたりの最大手数料
  maxPriorityFeePerGas: ethers.parseUnits("2", "gwei"), // 最大優先手数料
  data: "0x", // オプションのデータ
  chainId: 11155111, // SepoliaチェーンID
};

// RLPエンコードされたトランザクションハッシュを計算（署名前）
const unsignedTx = ethers.Transaction.from(txData).unsignedSerialized;
console.log("RLP-Encoded Tx (Unsigned): " + unsignedTx);
const txHash = ethers.keccak256(unsignedTx);
console.log("Tx Hash (Unsigned): " + txHash);

// トランザクションに署名
async function signAndSend() {
  // ウォレットでトランザクションに署名
  const signedTx = await wallet.signTransaction(txData);
  console.log("Signed Raw Transaction: " + signedTx);

  // 署名済みトランザクションをイーサリアムネットワークに送信
```

```
  const txResponse = await provider.broadcastTransaction(signedTx);
  console.log("Transaction Hash: " + txResponse.hash);

  // トランザクションがマイニングされるのを待つ
  const receipt = await txResponse.wait();
  console.log("Transaction Receipt: ", receipt);
}

signAndSend().catch(console.error);
```

サンプルコードを実行すると、以下の結果が得られます。

```
$ node eip1559_tx.js
RLP-Encoded Tx (Unsigned):
0x02f283aa36a7148477359400851748 76e8008303000094b0920c523d582040f2bcb1bd7fb1c7c1ecebdb34865af31
07a400080c0
Tx Hash (Unsigned): 0x31d43a580534a77c71324a8434df6f2df993b3d551b29d4b70d8a889768a53f7
Signed Raw Transaction:
0x02f87583aa36a7148477359400851748 76e8008303000094b0920c523d582040f2bcb1bd7fb1c7c1ecebdb34865af
3107a400080c001a03f8ed18cb03ee0fe3fbc3f0a7477a2f68db6ec84450e77e702b82a3f2c873aa4a0205c4f6a16ea
8ad13a148cc3105814cd4a6860cd26a771651199c85ccb7c7f0f
Transaction Hash: 0x07bfbeb337e19763a1f74d989dae2953807dcb06822354cfefb16405a11beb93
Transaction Receipt:  TransactionReceipt {
  provider: JsonRpcProvider {},
  to: '0xB0920c523d582040f2BCB1bD7FB1c7C1ECEbdB34',
  from: '0x7e41354AfE84800680ceB104c5Fc99eCB98A25f0',
  contractAddress: null,
  hash: '0x07bfbeb337e19763a1f74d989dae2953807dcb06822354cfefb16405a11beb93',
  index: 1,
  blockHash: '0x0ac051e8f615805c69eec6e193e39637adeb7cf314a0098d455e7d9ac395a7ee',
  blockNumber: 7135937,
  logsBloom:
'0x00000000000
```

EIP-1559生トランザクションの低レベル手動構築

以下の例は、EIP-1559（タイプ0x02）トランザクションを作成・署名するための完全な低レベルプロセスを示しています。フィールドリストを手動で構築し、RLPエンコードし、型付きペイロードに署名します。この手法により、`wallet.signTransaction`の内部で何が行われているかを正確に確認できます。

```
// npm install ethers@6
import { ethers } from "ethers";
const RLP = require('@ethereumjs/rlp');

// 自身のRPCと秘密鍵に置き換えてください
const rpcUrl = "https://ethereum-sepolia-rpc.publicnode.com"; // メインネット等も可
const provider = new ethers.JsonRpcProvider(rpcUrl);
const privateKey = "0xYOUR_PRIVATE_KEY_HERE";
const wallet = new ethers.Wallet(privateKey, provider);

const recipient = "0xRECIPENT_ADDRESS"; // 送信先アドレスの例

async function createAndSendRawEip1559Tx() {
  const chainId = await provider.getChainId(); // RPCコール
  const nonce = await provider.getTransactionCount(wallet.address); // RPCコール

  // パラメータの例 — 必要に応じて調整（またはprovider.estimateGas + getFeeDataを使用）
  const maxPriorityFeePerGas = ethers.parseUnits("2", "gwei");   // チップ
  const maxFeePerGas = ethers.parseUnits("30", "gwei");          // 手数料上限（ベース手数料＋チップ）
  const gasLimit = 21000n;
  const value = ethers.parseEther("0.01"); // 0.01 ETH
  const data = "0x"; // またはコントラクトのコールデータ
  const accessList = []; // 空またはEIP-2930形式のアクセスリスト

  // トランザクションタイプ2の未署名フィールド（厳密な順序）
  const unsignedFields = [
    ethers.toBeHex(chainId),              // chainId
    ethers.toBeHex(nonce),                // nonce
    ethers.toBeHex(gasLimit),             // gasLimit
    ethers.toBeHex(maxPriorityFeePerGas), // maxPriorityFeePerGas
    ethers.toBeHex(maxFeePerGas),         // maxFeePerGas
    recipient,                            // to
    ethers.toBeHex(value),                // value
    data,                                 // data
    accessList                            // accessList
  ];

  // 未署名フィールドをRLPエンコード
  const encodedUnsigned = RLP.encode(unsignedFields);

  // トランザクションタイプバイト（0x02）を先頭に付加
```

```
  const txType = Buffer.from('02', 'hex');
  const fullTx = Buffer.concat([txType, encodedUnsigned]);

  // 署名ハッシュ
  const signingHash = ethers.keccak256(fullTx);

  // ハッシュに署名
  const signingKey = new ethers.SigningKey(wallet.privateKey);
  const signature = signingKey.sign(ethers.getBytes(signingHash));

  // 型付きトランザクションではレガシーのvの代わりにyParity（0または1）を使用
  const signedFields = [
    ...unsignedFields,
    sig.v - 27, // yParity（0または1）
    sig.r,
    sig.s
  ];

  const encodedSigned = RLP.encode(signedFields);

  const rawTransaction = '0x02' + Buffer.from(encodedSigned).toString('hex');

  console.log("Raw transaction hex:\n", rawTransaction);

  // 生のトランザクションを送信
  const txHash = await provider.send("eth_sendRawTransaction", [rawTransaction]);
  console.log("Transaction sent! Hash:", txHash);

  const receipt = await provider.waitForTransaction(txHash);
  console.log("Mined in block", receipt.blockNumber);
}

createAndSendRawEip1559Tx().catch(console.error);
```

トランザクションの逆シリアライズ

ここまでで、イーサリアムネットワークへのトランザクションをゼロから作成して送信しました。次に、逆のプロセスをたどり、前の例で得られた署名済み生トランザクションから各フィールドを復元してみましょう。この作業により、トランザクションの各フィールドが実際にどのようにトランザクションに含まれているかを理解できます。正しい方法で抽出するだけでよいのです。

まず、署名済み生トランザクションの全体を見てみましょう。

```
0x02f87583aa36a7148477359400851748 76e8008303000094b0920c523d582040f2bcb1bd7fb1c7c1ecebdb34865af
3107a400080c001a03f8ed18cb03ee0fe3fbc3f0a7477a2f68db6ec84450e77e702b82a3f2c873aa4a0205c4f6a16ea
```

```
8ad13a148cc3105814cd4a6860cd26a771651199c85ccb7c7f0f
```

EIP-2718を思い出すと、すべてのイーサリアムトランザクションは以下の要素で構成されます。

- トランザクションタイプを指定する最初の1バイト
- トランザクションタイプのペイロード

この構造は、ほぼ常にその特定のトランザクションタイプの全フィールドのRLPエンコーディングに対応します。

このトランザクションは`0x02`で始まっています。これはトランザクションタイプを表し、`0x02`はEIP-1559トランザクションであることを意味します。次の部分は、EIP-1559トランザクションを構成するすべてのトランザクションフィールドのRLPエンコーディングです。

フィールドを素早くデコードするには、`cast`の`from-rlp`コマンドを使用し、トランザクションの残りの部分を入力として渡します。

```
$ cast from-rlp
f87583aa36a7148477359400851748 76e8008303000094b0920c523d582040f2bcb1bd7fb1c7c1ecebdb34865af3107a400080c001a03f8ed18cb03ee0fe3fbc3f0a7477a2f68db6ec84450e77e702b82a3f2c873aa4a0205c4f6a16ea8ad13a148cc3105814cd4a6860cd26a771651199c85ccb7c7f0f
["0xaa36a7","0x14","0x77359400","0x174876e800","0x030000","0xb0920c523d582040f2bcb1bd7fb1c7c1ecebdb34","0x5af3107a4000","0x",[],"0x01","0x3f8ed18cb03ee0fe3fbc3f0a7477a2f68db6ec84450e77e702b82a3f2c873aa4","0x205c4f6a16ea8ad13a148cc3105814cd4a6860cd26a771651199c85ccb7c7f0f"]
```

最後の`cast`コマンドの出力には、16進数の項目リストが含まれています。これらはEIP-1559トランザクションのフィールドです。1つずつ分析してトランザクションを再構成しましょう。

`0xaa36a7`

SepoliaのチェーンID。10進数では11155111です。

`0x14`

トランザクションで使用されたナンス。10進数では20です。ブロックエクスプローラで確認すると、実際に正しいことがわかります。

`0x77359400`

ガスあたりの最大優先手数料。10進数では2000000000で、ガスあたり2 gweiに相当します。

`0x174876e800`

ガスあたりの最大手数料。10進数では100000000000で、ガスあたり100 gweiに相当します。

`0x030000`

ガスリミット。10進数では196608です。

`0xb0920c523d582040f2bcb1bd7fb1c7c1ecebdb34`

受信者のアドレスです。

`0x5af3107a4000`

送信者から受信者に送られたweiでの値。10進数では10^14で、0.0001 etherに相当します。

`0x`

空のデータペイロードです。

`[]`

空のアクセスリストです。

`0x01`

署名の`v`値であり、`0x01`は楕円曲線のy座標が奇数（`0x00`は偶数）であることを意味します。

`0x3f8ed18cb03ee0fe3fbc3f0a7477a2f68db6ec84450e77e702b82a3f2c873aa4`

署名の`r`値です。

`0x205c4f6a16ea8ad13a148cc3105814cd4a6860cd26a771651199c85ccb7c7f0f`

署名の`s`値です。

EIP-155による生トランザクションの作成

EIP-155「シンプルリプレイアタックプロテクション」規格は、署名前にトランザクションデータ内にチェーンIDを含むリプレイ攻撃防御トランザクションエンコーディングを指定しています。この仕組みにより、あるブロックチェーン（例：イーサリアムメインネットワーク）向けに作成されたトランザクションが、別のブロックチェーン（例：イーサリアムクラシックやSepoliaテストネットワーク）上では無効となることが保証されます。したがって、あるネットワーク上でブロードキャストされたトランザクションを別のネットワーク上でリプレイすることはできません。規格名はこの特性に由来しています。

署名対象のデータにチェーンIDが含まれているため、チェーンIDが変更されると署名は無効になります。EIP-155によって、署名の有効性がチェーンIDに依存するため、別のチェーンでトランザクションをリプレイすることが不可能になります。

チェーンIDフィールドは、トランザクションが対象とするネットワークに応じた値を持ちます（表6-2参照）。

表6-2. チェーン識別子

チェーン	チェーンID
Ethereum mainnet	1
Ethereum Sepolia	11155111
Ethereum Holesky	17000

チェーン識別子の網羅的なリストについては、ChainListを参照してください。

結果としてトランザクション構造はRLPエンコードおよびハッシュ化され、署名されます。詳細についてはEIP-155仕様を参照してください。

署名プレフィックス値（v）と公開鍵リカバリ

「トランザクションの構造」で説明したように、トランザクションメッセージにはfromフィールドが含まれていません。発行者の公開鍵はECDSA署名から直接計算できるためです。公開鍵を取得できれば、アドレスを簡単に計算できます。署名者の公開鍵を復元するプロセスは*公開鍵リカバリ*と呼ばれます。

「ECDSAの数学」で計算した値`r`と`s`から、2つの公開鍵候補を計算できます。

まず、署名内にあるx座標の`r`値から2つの楕円曲線上の点`R`と`R'`を計算します。楕円曲線はx軸に対して対称であるため、任意の値xに対して、曲線上にはx軸の両側にそれぞれ1つずつ、合計2つの点が存在します。

`r`から、`r`の乗法逆数であるr^{-1}も計算します。

最後に、メッセージハッシュのn個の最下位ビットである`z`を計算します。ここでnは楕円曲線の位数です。

2つの公開鍵候補は以下のとおりです。

```
K1 = r–1 (sR – zG)
```

および

```
K2 = r–1 (sR′ – zG)
```

各変数の意味は以下のとおりです。

- `K1`と`K2`は署名者の公開鍵の2つの候補
- r^{-1}は署名の`r`値の乗法逆数

- `s`は署名の`s`値
- `R`および`R'`は一時的な公開鍵`Q`の2つの候補
- `z`はメッセージハッシュのn個の最下位ビット
- `G`は楕円曲線の生成元

より効率的にするために、トランザクション署名にはプレフィックス値`v`が含まれています。`v`は、2つのR値候補のうちどちらが一時的な公開鍵であるかを示します。`v`が偶数の場合は`R`が正しい値で、`v`が奇数の場合は`R'`が正しい値です。これにより、Rに対して1つの値、Kに対して1つの値だけを計算すれば済みます。

署名と送信の分離（オフライン署名）

トランザクションが署名されると、イーサリアムネットワークに送信する準備が整います。トランザクションの作成、署名、ブロードキャストの3つのステップは、通常は単一の操作として実行されます。たとえば`cast send`コマンドを使用する場合がその例です。しかし、「生トランザクションの作成と署名」で見たように、トランザクションの作成と署名を2つの別々のステップで行うことも可能です。署名済みトランザクションを取得したら、`ethers.JsonRpcProvider("...").broadcastTransaction`を使用して送信できます。この関数は16進数でエンコードされた署名済みトランザクションを受け取り、イーサリアムネットワーク上に送信します。

なぜ署名と送信を分離するのでしょうか。最も一般的な理由はセキュリティです。トランザクションに署名するコンピュータには、ロック解除された秘密鍵がメモリにロードされている必要があります。一方、送信を行うコンピュータはインターネットに接続されていなければなりません（かつイーサリアムクライアントを実行している必要があります）。この2つの機能が1台のコンピュータ上にあると、オンラインシステム上に秘密鍵が存在することになり、非常に危険です。

> **警告**
>
> 秘密鍵をオンラインで保管すると、マルウェアやリモートハッキングなど複数の攻撃にさらされ、フィッシング攻撃の被害も受けやすくなる。

署名と送信の機能を分離し、それぞれ別々のマシン（オフラインデバイスとオンラインデバイス）で実行することを*オフライン署名*と呼び、一般的なセキュリティ手法です。

図6-10にその処理の流れを示します。

1. アカウントの現在の状態（特に現在のナンスと利用可能な資金）を取得できるオンラインコンピュータ上で、未署名トランザクションを作成する

2. 未署名トランザクションを、QRコードやUSBフラッシュドライブなどを介して、「エアギャップ」のあるオフラインデバイスに転送し、トランザクションに署名する
3. 署名済みトランザクションを、QRコードやUSBフラッシュドライブなどを介して、オンラインデバイスに送り返し、イーサリアムブロックチェーン上にブロードキャストする

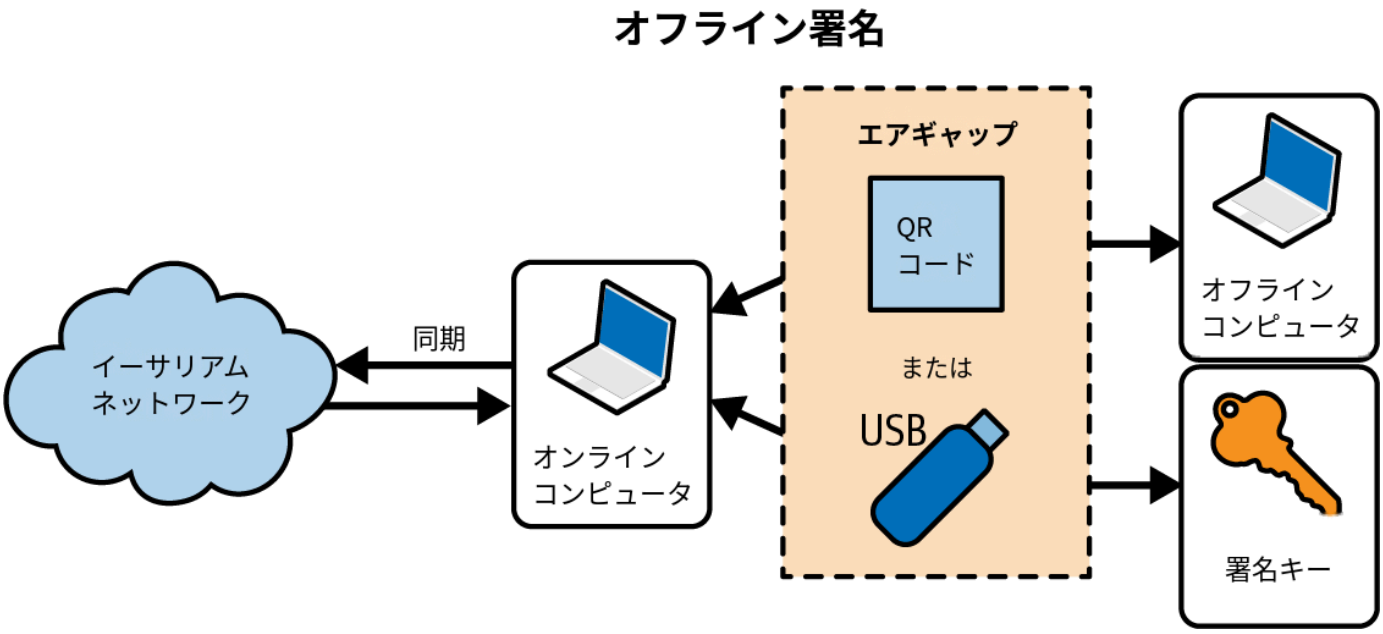


図6-10. オフライン署名のプロセス

必要なセキュリティレベルに応じて、「オフライン署名」コンピュータとオンラインコンピュータの隔離度は様々に設定できます。隔離されたファイアウォール付きサブネット（オンラインだが分離されている環境）から、*エアギャップシステム*と呼ばれる完全にオフラインのシステムまで幅広い選択肢があります。エアギャップシステムにはネットワーク接続が一切ありません。コンピュータはオンライン環境から「空気の壁」によって物理的に隔離されています。トランザクションに署名するには、データ記憶媒体またはウェブカメラとQRコードを使用して、エアギャップのあるコンピュータとの間でトランザクションを転送します。もちろん、署名するすべてのトランザクションを手動で転送しなければならないため、拡張性には限界があります。

完全なエアギャップシステムを利用できる環境は多くありませんが、わずかな隔離でも大きなセキュリティ上の利点があります。たとえば、メッセージキュープロトコルのみを通過させるファイアウォールを備えた隔離サブネットは、オンラインシステム上で署名する場合と比較して、攻撃対象を大幅に縮小し、はるかに高いセキュリティを提供できます。多くの企業がこの目的でZeroMQ（0MQ）などのプロトコルを使用しています。このような構成では、トランザクションはシリアライズされ、署名のためにキューに入れられます。キューイングプロトコルは、TCPソケットに類似した方法でシリアライズされたメッセージを署名コンピュータに送信します。署名コンピュータはキューからシリアライズされたトランザクションを慎重に読み込み、適切な鍵で署名し、発信キューに配置します。発信キューは署名済みトランザクションをイーサリアムクライアントに送信し、そこから送信が実行されます。

トランザクションのライフサイクル

この節では、トランザクションが署名された時点からブロックに取り込まれ、そのブロックがファイナライズされるまでのライフサイクル全体を説明します。

トランザクションの作成と署名

最初のステップは、トランザクションタイプを選択し、必要なすべてのフィールドを入力してトランザクションを作成することです。たとえば、「生トランザクションの作成と署名」の節ではEIP-1559トランザクションを作成しました。

トランザクションを作成したら、正しい秘密鍵で署名する必要があります（署名が無効だとトランザクションも無効になります）。署名によって最終的かつ確定的な署名済みトランザクションが得られます。このデータをネットワークに送信し、ブロックへの取り込みを待つことになります。

トランザクションのネットワークへの送信

トランザクションがイーサリアムプロトコルによって確認済みとみなされるには、ブロックに取り込まれなければなりません。ブロックに取り込まれなければ、自分だけが知っている署名済みトランザクションにすぎません。

ヒント

イーサリアムプロトコルでは、有効なチェーンに属するブロックに取り込まれたトランザクションのみが有効とみなされる。この点を理解しておくことが非常に重要である。状態を更新するには、署名済みトランザクションをネットワークに送信し、ブロックへの取り込みを待つ必要がある。署名済みトランザクション単体では、ブロックに取り込まれない限り何の効果も持たない。

署名済みトランザクションをネットワークに送信するには、イーサリアムノードにトランザクションを送る必要があります。自分自身のノードを使うこともできますし、Alchemy、Infura、Public Node（前の例で使用したもの）などのサードパーティノードを使うこともできます。

注記

デフォルトでは、すべてのウォレットがサードパーティノードを使用しており、ユーザーはイーサリアムネットワークを利用開始するために何もインストールする必要がない。ただし、プライバシーを最大化し、誰にも依存したくない場合は、自分自身のクライアント

を使用するべきである。イーサリアムノードの初回インストールに関する詳細なガイドは、3章を参照のこと。

署名済みトランザクションが最初のノードに到達すると、そのノードはスパムや無効なトランザクションを即座に排除するためにいくつかの検証を行います。検証に成功すると、ノードはトランザクションを自身の*メモリプール*（mempool）に追加し、接続しているピアの一部に伝播します。ピアの各々がトランザクションを検証し、自身のメモリプールに追加し、さらに別のピアに伝播します。

このプロセスはイーサリアムのP2Pゴシッププロトコルの一部であり、その結果、イーサリアムトランザクションはわずか数秒で世界中のすべてのイーサリアムノードに伝播します。各ノードの観点からは、トランザクションの発信元を識別することはできません。トランザクションを送ってきた隣接ノードがトランザクションの発行者である場合もあれば、別の隣接ノードから受信しただけの場合もあります。トランザクションの発信元を追跡したり、伝播を妨害したりするには、攻撃者が全ノードのかなりの割合を支配する必要があります。これがP2Pネットワーク、特にブロックチェーンネットワークに適用されるセキュリティとプライバシー設計の一部です。

注記

ノードがすべての隣接ノードにトランザクションをフラッディングせず、隣接ノードの一部にのみ送信するのはなぜか疑問に思うかもしれない。その理由は効率性と帯域幅の節約である。すべてのトランザクションをすべてのノードに送信すると、重複メッセージが大量に発生し、ネットワークトラフィックが膨大になる。さらにトランザクション数の増加に伴って指数関数的に増大するため、スケーラビリティが低下する。

ブロックの構築

この時点で、トランザクションはほぼすべてのイーサリアムノードに到達していますが、ブロックに取り込まれていないため、まだ確認済みではありません。次のブロックの提案を担当するバリデータが自身のメモリプールからトランザクションを取り出し、ブロックに追加してネットワークに公開するまで待つ必要があります。

トランザクションがブロックに取り込まれると、イーサリアムの状態が変更されます。単純な支払いの場合はアカウントの残高が変更され、コントラクトを呼び出す場合はコントラクトの内部状態が変更されます。これらの変更はトランザクションとともに*トランザクションレシート*の形式で記録され、レシートにはイベントも含まれます。「生トランザクションの作成と署名」で示した例の通り、トランザクションの最終レシートでこれらの内容を確認できます。

トランザクションのファイナライゼーション

トランザクションはブロックに取り込まれ、イーサリアムの状態を変更しました。しかし、トランザクションを含むブロックは、可能性としては極めて低いものの、トランザクションを含まない別のブロックに差し替えられる可能性が残っています。これを*ブロック再編成*（*ブロックリオーグ*）と呼びます。

トランザクションが取り消し不可能であることを完全に確認するには、トランザクションを含むブロックがイーサリアムのコンセンサスプロトコルによってファイナライズされるのを待つ必要があります（詳細は15章で説明します）。通常、ファイナライゼーションには約12分かかります。

> **注記**
>
> ファイナライゼーションを待つことでトランザクションの確定をブロックチェーン上で完全に保証できるのは事実であるが、通常は数ブロック待てば十分である。ウォレットの多くは、トランザクションがブロックに取り込まれた直後に確認済みとして表示する。

もう1つのライフサイクル

前節で説明したライフサイクルは、トランザクションが開始からファイナライズされたブロック内に到達するまでの従来の標準的な流れです。しかし、過去3〜4年の間に、そして特に現在では、トランザクションの新しいライフサイクルが確立されています。この新しいライフサイクルを説明するには、*提案者とビルダーの分離*（Proposer and Builder Separation）という概念を導入する必要があります。

MEVと提案者・ビルダーの分離

15章で詳しく説明しますが、バリデータは12秒ごとにイーサリアムチェーンを進めるための新しいブロックを提案する必要があります。バリデータはメモリプールからトランザクションを収集し、ブロックを埋めるよう取りまとめ、すべてのノードが検証・伝播できるようにネットワークに公開します。

従来、イーサリアムノードはバリデータへの報酬（具体的にはEIP-1559トランザクションの優先手数料）を最大化するために、トランザクション手数料に基づいてトランザクションの優先順位を決めていました。この手法は、マイナーやバリデータの収益を最適化する標準的な方法として長年使用されてきました。

しかし、2020年の「DeFi Summer」におけるイーサリアムの人気の高まり、そしてさらに2021年の強気相場において、新たな現象が出現しました。*最大抽出可能価値*（*MEV*、旧称マイ

ナー抽出可能価値）です。この概念は、イーサリアムのマイナーやバリデータがブロックを作成する方法を大きく変えました。

MEVとは、ブロック生産者が以下の戦略的判断によってブロックから抽出できる最大の価値を指します。

- ブロックにどのトランザクションを含めるか
- ブロック内のトランザクションの順序
- ブロックからどのトランザクションを除外するか

一見無害に思えますが、ブロック生産の動態に非常に大きな影響を及ぼしてきましたし、今もなお影響を与え続けています。たとえば、Uniswapのような分散型取引所で特定のトークンXを大量に購入するトランザクションを考えてみましょう。バリデータはこのトランザクションを事前に確認でき、価格が大幅に上昇することがわかります。そこで、この状況を利用して利益を得るために2つのトランザクションを追加できます。

1. 1つ目はトークンXを購入するトランザクションで、大量購入トランザクションの前に実行されるよう配置する
2. 2つ目は前のトランザクションで購入したトークンXを売却するトランザクションで、大量購入トランザクションの後に実行されるよう配置する

これを*サンドイッチ攻撃*と呼びます。バリデータは大量購入トランザクションが生み出すスリッページ（注文時と約定時の価格差）から利益を得ます。この概念を簡略化した例で説明しましょう。

あるユーザーがトークンxyzを1,000枚購入するトランザクション（tx1）を送信したとします（図6-11参照）。トークンxyzの価格が1,000ドルだとすると、100万ドル相当のトークンを購入することになります。この買い圧力によりトークン価格は1,010ドルに上昇します。

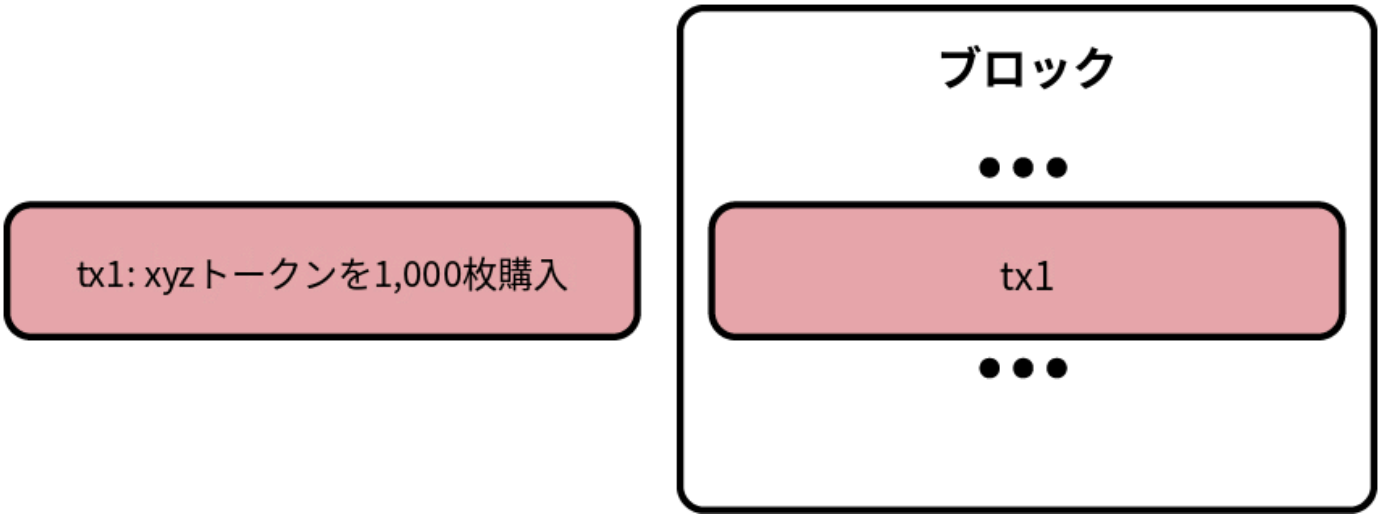


図6-11. MEV介入前のユーザートランザクション

しかし、バリデータはtx1をフロントランニングして利益を得る機会を見出します。バリデータは2つのトランザクションを作成します。tx0にはxyzトークン10枚の買い注文、tx2には同量の売り注文を含めます。図6-12のように、ユーザーのtx1の直前と直後にこれらのトランザクショ

ンを配置します。バリデータが実際にブロックを作成するアクターであるため、ブロック内のトランザクションの順序を自由に決定できるのです。

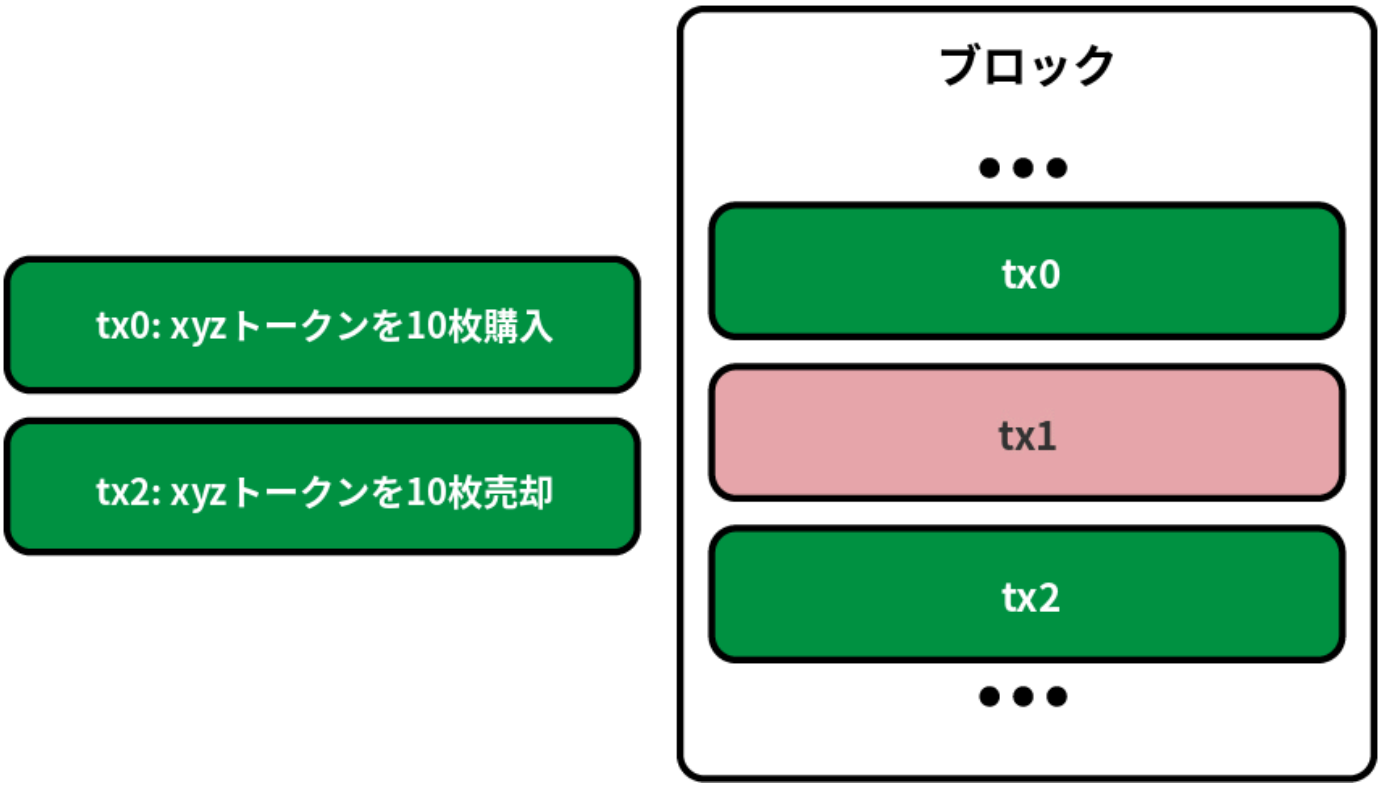


図6-12. バリデータによるサンドイッチ攻撃

その結果、バリデータはxyzトークン10枚を1,000ドルで購入し1,010ドルで売却することで、10ドル × 10 = 100ドルの利益を得ます。

これはバリデータが利益を最大化するために使用する非常にシンプルな戦略の1つです。他にもはるかに複雑な戦略が数多く存在します。競争が非常に激しいため、*ビルダー*という新しいアクターが出現しました。

実際のところ、これらの戦略を実行するには膨大な処理能力が必要であり、平均的なバリデータの能力をはるかに超えています（バリデータノードはわずか16 GBのRAMで運用できることを思い出してください）。つまり、MEVはバリデータの分散性をも脅かしていました。インフラに数百万ドルを費やし、より利益の高い「より良い」ブロックを構築するための新しい戦略の発見に尽力できる大規模な組織ほど有利になる構造だったのです。

この問題の解決策として、2021年1月にFlashbots v0.1がリリースされました。Flashbotsにより、ブロック提案者（かつてはマイナー、現在はバリデータ）は、最適なブロック構築を見つけるタスクをビルダーと呼ばれる新しいエンティティにトラストレスに委託できるようになりました。ビルダーはトランザクションでブロックを埋め、バリデータへの手数料が最大となるブロックを作成します（ビルダーも手数料の一部を受け取ります）。そのブロックをバリデータがネットワークに提案します。この役割分担のおかげで、バリデータは引き続き一般的なハードウェアで運用を続けることができます。

プライベートメモリプール

MEVのエコシステムは大きく成長し、現在ではビルダー同士が利益最大化のための最適な戦略を見つけるだけでなく、ブロックを埋めるために利用できるトランザクションの獲得でも競争しています。より多くのトランザクションを持つほど、戦略をより効果的に適用できるからです。

この状況が*プライベートメモリプール*の誕生につながりました。プライベートメモリプールでは、ユーザーやエンティティが一般のネットワークに公開することなく、ブロック生産者に直接トランザクションを送信できます。フロントランニングなどのリスクを軽減し、プライバシー重視のワークフローを実現する上で重要な役割を果たしています。

FlashbotsはFlashbots Protectという独自のソリューションを開発しました。最も人気のあるウォレットであるMetaMaskも、ユーザー向けにデフォルトでプライベートメモリプールの使用を開始しています（スマートトランザクションと呼ばれています）。

新しいトランザクションのライフサイクル

MEV、提案者とビルダーの分離、プライベートメモリプールは、ブロック生産の環境を劇的に変化させました。現在、多くのトランザクションは前節で説明した従来のライフサイクルをたどりません。トランザクションは作成・署名された後、プライベートメモリプールを通じてビルダーに直接送信されます。ビルダーがトランザクションを受け取り、最適なブロックに取り込もうと試み、最終的に次のブロックを提案するバリデータにブロック全体を送信します。トランザクションはパブリックメモリプールでの伝播をスキップし、構築済みブロックの中に直接出現するのです。

マルチシグネチャトランザクション

ビットコインのスクリプティング機能に精通しているならば、複数の当事者がトランザクションに署名した場合にのみ資金を使用できるビットコインの*マルチシグ*アカウントを作成できることをご存知でしょう（例：2-of-2や3-of-4の署名）。イーサリアムの基本的なEOAのバリュートランザクションには複数署名の規定がありません。しかし、スマートコントラクトを使えば、イーサやトークンの転送に必要な署名条件を自由に設定できます。

この機能を利用するには、マルチシグネチャ要件や支払い上限（またはその組み合わせ）といった支払いルールがプログラムされたウォレットコントラクトにイーサを転送する必要があります。支払い条件が満たされると、許可されたEOAからのリクエストに応じてウォレットコントラクトが資金を送信します。たとえば、マルチシグ条件でイーサを保護するには、マルチシグコントラクトにイーサを転送します。別のアカウントに資金を送りたい場合は、必要なすべてのユーザーが通常のウォレットアプリを使用してコントラクトにトランザクションを送信し、最終的なトランザクションの実行を承認する必要があります。

これらのコントラクトは、ローカルコードを実行する前に複数の署名を要求したり、他のコントラクトをトリガーしたりするようにも設計できます。セキュリティの強度は、最終的にはマルチシグコントラクトのコードによって決まります。

マルチシグネチャトランザクションをスマートコントラクトとして実装できることは、イーサリアムの柔軟性を示しています。現在、Gnosis Safeがマルチシグネチャアカウントを作成するためのデファクトスタンダードとなっています。この実戦で検証済みのスマートコントラクトスイートは、主要なプロトコルやDAOで広く使用されており、2024年11月時点で60億ドル以上のETHと740億ドル以上のERC-20トークンを保護しています（図6-13参照）。

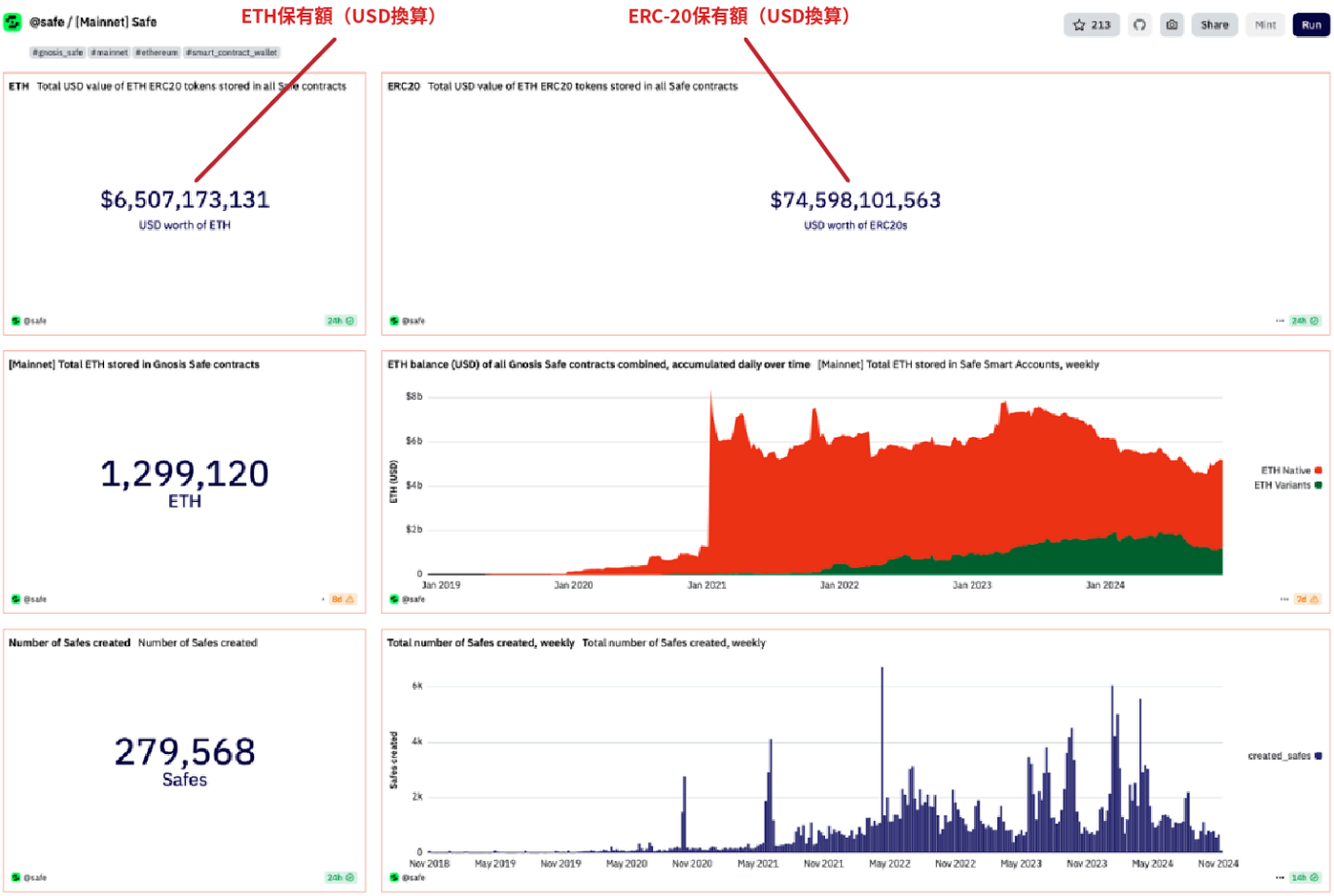


図6-13. 数十億ドル規模の価値を保護するGnosis Safe

注記

Gnosis Safeでトランザクションを実行する一般的なワークフローは以下のとおりである。

1. Safeの署名者の1人が、署名してイーサリアムネットワークに送信したいトランザクションを提案する
2. 他の署名者がトランザクションを確認し、その目的に同意すれば署名する
3. 定足数に達すると、トランザクションが最終的にネットワークに送信され、処理・実行される

まとめ

トランザクションは、イーサリアムシステムにおけるすべての活動の出発点です。トランザクションは、EVMがコントラクトを評価し、残高を更新し、イーサリアムブロックチェーンの状態を変更するための「インプット」です。次章では、スマートコントラクトの詳細を取り上げ、Solidityというコントラクト指向言語を用いたプログラミングを学びます。

7章 スマートコントラクトとSolidity

2章で述べたように、イーサリアムには外部所有アカウント（EOA）とコントラクトアカウントの2種類のアカウントがあります。EOAは多くの場合、イーサリアムプラットフォームの外部にあるウォレットアプリケーションなどのソフトウェアを介して、ユーザーが制御します。対照的に、コントラクトアカウントは、EVMによって実行されるプログラムコード（一般的に「スマートコントラクト」と呼ばれます）によって制御されます。

簡潔に言えば、EOAは関連するコードやデータストレージのないシンプルなアカウントですが、コントラクトアカウントは関連するコードとデータストレージの両方を持っています。EOAは、プロトコルから独立した「現実世界」における秘密鍵によって作成され暗号学的に署名されたトランザクションによって制御されます。これに対して、コントラクトアカウントは秘密鍵を持たないので、スマートコントラクトコードであらかじめ規定された方法で「自分自身を制御」します。両方のアカウントタイプは、イーサリアムアドレスによって識別されます。この章では、コントラクトアカウントと、それを制御するプログラムコードについて説明します。

スマートコントラクトとは何か？

「スマートコントラクト」という用語は、さまざまな異なるものを表現するために長年にわたって使われてきました。1990年代に、暗号学者であるニック・ザボ（Nick Szabo）がこの用語を作り、「当事者が他の約束を実行する手順まで含んだ、デジタル形式で規定された一連の約束」と定義しました。その後、スマートコントラクトの概念は発展していきましたが、それが顕著になったのは、2009年にビットコインが登場した分散型ブロックチェーンプラットフォームが導入されてからでした。

イーサリアムの文脈では、スマートコントラクトはスマートでも法的な契約でもないことを考えるとやや不適切な名称ですが、すでに定着しています。本書では、「スマートコントラクト」という用語は、イーサリアムネットワークプロトコルの一部としてEVMの中で——つまり、分散型イーサリアムワールドコンピュータにおいて——決定論的に実行される、イミュータブルなコンピュータプログラムを指します。

いま示した定義で使われている用語の意味を確認しておきましょう。

コンピュータプログラム

スマートコントラクトは単純なコンピュータプログラムです。この文脈では、「コントラクト」という言葉に（「契約」といった）法的な意味はありません。

イミュータブル

スマートコントラクトは一度デプロイされると、そのコードは変更できません。従来のソフトウェアとは異なり、スマートコントラクトを変更する唯一の方法は、新しいインスタンスをデプロイすることです。

決定論的

スマートコントラクトの実行結果は、実行を開始したトランザクションのコンテクストと実行時のイーサリアムブロックチェーンの状態が同じであれば、誰が実行しても同一になります。

EVMのコンテクスト

スマートコントラクトは非常に限られた実行コンテクストで機能します。自分の状態、そのコントラクトを呼び出したトランザクションのコンテクスト、最新ブロックに関する情報にアクセスできます。

分散型ワールドコンピュータ

EVMはすべてのイーサリアムノード上でローカルインスタンスとして動作しますが、すべてのインスタンスが同じ初期状態で動作し同じ最終状態を生成するため、システム全体が1つの「ワールドコンピュータ」として機能します。

スマートコントラクトのライフサイクル

通常、スマートコントラクトはSolidityのような高級言語で書かれています。ただし、実行するにはEVMで動作する低レベルのバイトコードにコンパイルする必要があります。コンパイルされると、特別なコントラクト作成トランザクションを使用してイーサリアムプラットフォームにデプロイされます。このトランザクションは、`to`フィールドが空であることによって識別されます（6章の「特別なトランザクション：コントラクト作成」を参照）。

各コントラクトは、発信アカウントとナンスの関数としてコントラクト作成トランザクションから導出された、イーサリアムアドレスによって識別されます。コントラクトのイーサリアムアドレスは、トランザクションの受信者としてコントラクトへの資金の送信やコントラクトの関数の呼び出しに使用できます。

EOAとは異なり、新しいスマートコントラクトのために作成されたアカウントに関連付けられた鍵はありません。コントラクトの作成者であっても、プロトコルレベルで特権を取得することはありません（スマートコントラクトに特権を明示的にコードとして書き込むことはできます）。コントラクトアカウントの秘密鍵を受け取ることはありません——それは存在しないのです。スマートコントラクトアカウントは自分自身を所有していると言えます。

重要なのは、コントラクトは「トランザクションによって呼び出された場合にのみ」実行されるという点です。イーサリアムのすべてのスマートコントラクトは、最終的にはEOAから開始されたトランザクションによって実行されます。コントラクトから別のコントラクトを呼び出し、さらにそこから別のコントラクトを呼び出すというように呼び出しを連鎖させることができま

す。しかし、このような一連の実行における最初のコントラクトは、常にEOAからのトランザクションによって呼び出されます。コントラクトが「自分自身で」または「バックグラウンドで」実行されることはありません。コントラクトは、トランザクションが実行をトリガーするまで事実上休止状態です。これは直接的な呼び出しでも、一連のコントラクトコールを経由した間接的な呼び出しでも同じです。スマートコントラクトは、いかなる意味でも「並行して」実行されないことにも留意してください。イーサリアムワールドコンピュータは、シングルスレッドマシンとみなすことができます。

トランザクションは、呼び出すコントラクトの数や呼び出されたときのコントラクトの動作に関係なく、アトミックです。すべての実行が正常に終了した場合にのみ、グローバルな状態（コントラクト、アカウントなど）の変更が記録され、トランザクションが全体として実行されたことになります。ここでいう正常な終了とは、プログラムがエラーなしで実行され、実行の終わりに達したことを意味します。エラーが原因で実行が失敗した場合、そのすべての影響（状態の変更）は、トランザクションが実行されなかったかのように「ロールバック」されます。実行に失敗したトランザクションは試行されたものとして記録され、実行のためにガスとして費やされたイーサは発信元アカウントから差し引かれますが、それ以外はコントラクトまたはアカウントの状態に影響を与えません。

前述のように、コントラクトのコードは一度デプロイされると変更できません。かつては、コントラクトのコードと内部状態（ストレージ）をアドレスから取り除き、空のアカウントとすることが可能でした。削除後、そのアドレスに送信されたトランザクションによってコードが実行されることはありません。なぜなら、実行するコードがなくなっているからです。

この削除は`SELFDESTRUCT`というEVMオペコードを使用して行われ、保存された状態を削除することでネットワークリソースの解放にインセンティブを与えるガスの払い戻しが提供されていました。

しかし、`SELFDESTRUCT`オペレーションは、アカウントの状態に対する大きな変更——特にすべてのコードとストレージの削除——を伴うため、2023年にEIP-6780によって非推奨となりました。イーサリアムロードマップにおける今後のアップグレードにより、このオペレーションは実行不可能になる予定です。

イーサリアム高級言語の紹介

EVMは、EVMバイトコードと呼ばれる特別な形式のコードを実行する仮想マシンです。これはx86_64などのマシンコードを実行するコンピュータのCPUに似ています。EVMの操作と言語については、14章で詳しく説明します。この節では、EVMで実行するためのスマートコントラクトの書き方について説明します。

スマートコントラクトをバイトコードで直接プログラムすることは可能ですが、EVMバイトコードはかなり扱いにくく、プログラマーが読んで理解するのは非常に困難です。その代わりに、ほとんどのイーサリアム開発者はプログラムを書くために高級言語を使い、それらをバイトコードに変換するためにコンパイラを使います。

どんな高級言語でもスマートコントラクトを書くことに使えるかもしれませんが、任意の言語をEVMバイトコードにコンパイルできるように適応させるのは非常に面倒な作業です。通常は難航するでしょう。スマートコントラクトは、非常に制約された最小限の実行環境（EVM）で機能します。加えて、EVM固有のシステム変数と関数の特別なセットが利用可能でなければなりません。そのため、汎用言語をスマートコントラクトの作成に適したものにするよりも、最初からスマートコントラクト言語を構築するほうが簡単です。その結果、スマートコントラクトを書くために、特定の目的のための言語が数多く登場しました。イーサリアムには、EVMで実行可能なバイトコードを生成するために必要なコンパイラといくつかの開発言語があります。

一般にプログラミング言語は、宣言型（declarative）と命令型（imperative）の2つのプログラミングパラダイムに分類できます。それぞれ関数型（functional）と手続き型（procedural）とも呼ばれています。

宣言型プログラミングでは、プログラムのロジック（logic）を表現する関数を書く一方、プログラムのフロー（flow）を表現する関数は書きません。宣言型プログラミングは副作用（side effect）がないプログラム、つまり関数の外側の状態変化のないプログラムを作成するために使用されます。宣言型プログラミング言語にはHaskellやSQLが含まれます。

一方、命令型プログラミングでは、プログラマーがプログラムのロジックとフローを組み合わせた一連のプロシージャを作成します。命令型プログラミング言語には、C++とJavaがあります。

一部の言語は「ハイブリッド」で、宣言型プログラミングを奨励するものの、命令型プログラミングのパラダイムを表現するためにも使用できます。このようなハイブリッドには、Lisp、JavaScript、Pythonなどが含まれます。一般に宣言型のパラダイムを書くために、どんな命令型言語でも使えますが、それはしばしばエレガントではないコードになります。逆に、純粋な宣言型言語を使用して、命令型パラダイムを書くことはできません。純粋な宣言型の言語には「変数」はないためです。

命令型プログラミングはプログラマーによってより広く使用されていますが、完全に期待どおりに実行するプログラムを書くことは非常に困難です。プログラム内のあらゆる部分がプログラム内の状態を変更できるため、プログラムが正しく実行されているかどうかの判断が難しくなり、バグが発生しやすくなります。これに対して宣言型プログラミングでは、副作用がなくプログラムのどの部分でも単独で理解できるため、プログラムの動作が理解しやすくなっています。

スマートコントラクトでは、バグは文字どおり資産に関わってきます。つまり、意図しない影響がないようにスマートコントラクトを作成することが極めて重要です。そのためにも、開発者はプログラムの動作について明確に推論できなければなりません。そのため宣言型言語は、汎用ソフトウェアにおいてよりもスマートコントラクトにおいてはるかに大きな役割を持ちます。とはいえ、最も広く使われているスマートコントラクト用言語（Solidity）は命令型です。プログラマーも人の常として、変化には抵抗するものです。

スマートコントラクト向けに現在サポートされている高級プログラミング言語は次のとおりです（人気順に挙げています）。

Solidity

JavaScript、C++、またはJavaに似た構文を持つ手続き型（命令型）プログラミング言語。イーサリアムのスマートコントラクト開発において、最もよく使用されている言語です。

Yul

スタンドアロンモードまたはSolidity内のインラインで使用される中間言語で、プラットフォーム間の高レベルな最適化に適しています。スマートコントラクトのセキュリティとEVMに関する高度な知識が必要なため、初心者はYulを学ぶ前にSolidityまたはVyperから始めるべきです。

Vyper

Pythonのような構文を持つコントラクト指向プログラミング言語で、ユーザーの安全性を優先し、言語設計と効率的な実行を通じて明確なコーディングプラクティスを奨励しています。

Huff

高効率で最小限のコントラクトコードを必要とする開発者が主に使用する低レベルプログラミング言語で、Solidityのような高級言語では実現できない高度な最適化が可能です。Yulと同様に、初心者には推奨されません。

Fe

PythonとRustにインスパイアされた、EVM向けの静的型付けスマートコントラクト言語です。イーサリアムが初めての開発者でも学びやすいことを目指しており、2021年1月のアルファリリース以来、まだ開発の初期段階にあります。

過去に開発されたものの、現在はメンテナンスされていない言語もあります。LLL、Serpent、Bambooなどがその例です。

このように、選択できる言語はたくさんあります。しかし、これらのなかでもSolidityが圧倒的に人気があり、事実上イーサリアム、さらには他のEVM互換ブロックチェーンの高級言語となっています。

Solidityでのスマートコントラクトの作成

Solidityはスマートコントラクトを明示的に記述するための言語で、イーサリアムワールドコンピュータの分散環境での実行を直接にサポートする機能を備えています。ギャヴィン・ウッド（本書初版の共著者）が作りました。その仕様が汎用性の高いものとなったため、他の複数のブロックチェーンプラットフォームでもスマートコントラクトの記述に使われるようになりました。クリスチャン・ライトゥイッセナー、その後アレックス・ベリックスサシィ、リアーナ・フシキヤン、平井洋一、および複数の元イーサリアムコアコントリビューターによって開発されました。Solidityは現在GitHub上の独立したプロジェクトとして開発、維持されています。

Solidityプロジェクトの主な「プロダクト」は、Solidity言語で書かれたプログラムをEVMバイトコードに変換するSolidityコンパイラであるsolcです。このプロジェクトは、イーサリアムスマートコントラクトの重要なABI規格も管理しています。ABIについては、この章で詳しく説明します。Solidityコンパイラの各バージョンは、Solidity言語の特定バージョンに対応し、それをコンパイルします。

はじめに、Solidityコンパイラのバイナリ実行ファイルをダウンロードします。その後、2章で始めた例から続けて、簡単なコントラクトを作成してコンパイルします。

Solidityのバージョンの選択

Solidityはセマンティックバージョニングと呼ばれるバージョニングモデルに従います。これはドット（.）で区切られた3つの数字（メジャー.マイナー.パッチ）として構造化されたバージョン番号を指定します。「メジャー」番号は、メジャーリリースと後方互換性のない変更があると増え、「マイナー」番号はメジャーリリース間で後方互換性のある機能が追加されると増え、「パッチ」番号は後方互換性のあるバグ修正に対して増えます。

本書執筆時点でのSolidityのバージョンは0.8.26です。プロジェクトの初期開発のためのメジャーバージョン0のルールは異なり、いつでも何でも変更される可能性があります。実際には、Solidityは「マイナー」番号をメジャーバージョンであるかのように扱い、「パッチ」番号をマイナーバージョンであるかのように扱います。つまり、0.8.26では8がメジャーバージョン、26がマイナーバージョンとみなされます。2章で見たように、Solidityプログラムは、互換性のあるSolidityの最小と最大バージョンを指定するプラグマ命令を含められます。この命令は、コントラクトのコンパイルに使用されます。Solidityは急速に進化しているので、最新のリリースをインストールすることを推奨します。

注記

Solidityの急速な進化がもたらすもう1つの結果は、ドキュメントが陳腐化するペースの速さです。現在、本書ではSolidityバージョン0.8.26を使用しており、本書の内容はすべてこのバージョンに基づいています。本書はSolidityを学ぶための確かな基礎を常に提供しますが、将来のバージョンでは構文や機能が変更される可能性があります。そのため、疑問があったり新しいことに遭遇した場合は、Solidityの公式ドキュメントを確認して最新の情報を得ることをお勧めします。

ダウンロードとインストール

Solidityのダウンロードおよびインストールを行い、使えるようにする方法はいくつかあります。バイナリリリースを利用する方法もありますし、ソースコードからコンパイルする方法もあります。詳細で最新の手順は、Solidityのドキュメントを参照してください。

以下が、aptパッケージマネージャを使って、Solidityの最新のバイナリリリースをUbuntu/Debianオペレーティングシステムにインストールする方法です。

```
$ sudo add-apt-repository ppa:ethereum/ethereum
$ sudo apt update
$ sudo apt install solc
```

solcをインストールしたら、次のコマンドを実行してバージョンを確認してください。

```
$ solc --version
solc, the solidity compiler commandline interface
Version: 0.8.26+commit.8a97fa7a.Linux.g++
```

開発環境

シンプルなテキストエディタを使ってSolidityスマートコントラクトを開発することは十分に可能ですが、HardhatやFoundryのような開発フレームワークを活用することで、開発者としての効率と生産性を大幅に向上させることができます。これらのフレームワークは、開発プロセスを簡素化し改善するための包括的なツール群を提供します。

たとえば、堅牢なテスト環境が用意されており、コントラクトの動作を検証するユニットテストを作成・実行できます。また、リアルなテストシナリオのためにメインネットのローカルインスタンスを作成するフォーク機能も備えています。

高度なデバッグおよびトレース機能により、コードをステップ実行して問題を迅速に特定・解決でき、時間の節約とエラーの削減につながります。加えて、スクリプティングとデプロイの自動化、機能を拡張するプラグインエコシステム、異なる環境間のシームレスなネットワーク管理もサポートしています。

これらの機能をワークフローに統合することで、シンプルなテキストエディタでは達成が難しい、より高いレベルのコード品質とセキュリティが確保されます。

フレームワーク以外にも、VS CodeのようなモダンなIDEを採用することで、生産性をさらに向上させることができます。VS CodeはSolidity向けの幅広い拡張機能を提供しています。具体的には、コードを読みやすくするシンタックスハイライト、複雑なプロジェクトの整理とナビゲーションを支援する高度なコメントやブックマークツール、コードの構造や潜在的な問題に関する洞察を提供するビジュアル分析ツールなどがあります。Remix IDEなどのWebベースの開発環境もあります。

これらのツールを組み合わせることで、コード品質が向上するだけでなく、開発プロセスが加速され、スマートコントラクトをより迅速かつ安全に構築してデプロイできるようになります。

シンプルなSolidityプログラムを書く

2章では、最初のSolidityプログラムを書きました。最初に`Faucet`コントラクトを作成したときは、Remix IDEを使用してそのコントラクトをコンパイルおよびデプロイしました。この節では、`Faucet`を見直し、改良し、装飾します。

最初のファウセットコントラクトは、例7-1のようなものでした。

例7-1 Faucet.sol：ファウセットを実装するSolidityコントラクト

```
// SPDX-License-Identifier: GPL-3.0
// Our first contract is a faucet!
contract Faucet {
    // Give out ether to anyone who asks
    function withdraw(uint _withdrawAmount, address payable _to) public {
        // Limit withdrawal amount
        require(_withdrawAmount <= 100000000000000000);
        // Send the amount to the address that requested it
        _to.transfer(_withdrawAmount);
    }
    // Accept any incoming amount
    receive() external payable {}
}
```

2章で見たように、コメント内のSPDXライセンス識別子は、スマートコントラクトがGPL-3.0の下でライセンスされていることを示しており、コードの使用と配布に関する法的権利と義務をユーザーと開発者に伝えています。

Solidityコンパイラ（solc）を使ってコンパイルする

それでは、コマンドラインでSolidityコンパイラを使用して、コントラクトを直接コンパイルします。Solidityコンパイラsolcはさまざまなオプションを提供しています。それらは`--help`引数を指定すれば確認できます。

solcの`--bin`引数と`--optimize`引数を使用して、例に挙げたコントラクトの最適化されたバイナリを生成します。

```
$ solc --optimize --bin Faucet.sol
======= Faucet.sol:Faucet =======
Binary:
608060405234801560e575f5ffd5b5060fa8061001b5f395ff3fe6080604052600436106043610601d575f3560e01c806
2f714ce146027575f5ffd5b36602357005b5f5ffd5b3480156031575f5ffd5b506041603d366004608d565b6043
565b005b67016345785d8a0000821156056575f5ffd5b604051600160160a01b0382169083156108fc0290849
05f818181858888f193505050501580156088573d5f5f3e3d5ffd5b505050565b5f5f60408385031215609d575f
5ffd5b8235915060208301356001600160a01b038116811460b9575f5ffd5b809150509250929050565fea264697
```

```
06673582212208935b6cf5d9070b7609ad59ac4b727e512522c674cacf09a2eff88dafa3242ee64736f6c634300
081b0033
```

solcが生成する結果は16進数にシリアライズされたバイナリで、イーサリアムブロックチェーンに送信できます。

イーサリアムコントラクトABI

コンピュータソフトウェアにおいてABI（Application Binary Interface）とは、2つのプログラムモジュール間の（しばしば、オペレーティングシステムとユーザープログラムの間の）インターフェイスです。ABIは、機械語（machine code）においてデータ構造と関数がどのようにアクセスされるかを定義します。これをAPIと混同しないでください。APIは高レベルで、しばしばソースコード（source code）として人間が読める形式で、データ構造と関数へのアクセスを定義します。つまり、ABIはデータを機械語へとエンコードあるいは機械語からデコードするための主要な方法です。

イーサリアムでは、ABIを使用してEVMのコントラクト呼び出しをエンコードし、トランザクションからデータを読み取ります。ABIの目的は、呼び出される関数をコントラクトに定義し、各関数が引数を受け入れてその結果を返す方法を記述することです。

コントラクトのABIは、関数定義（「関数」を参照）およびイベント（「イベント」を参照）のJSON配列で構成されます。関数定義は、`type`、`name`、`inputs`、`outputs`、`constant`、`payable`のフィールドを持つJSONオブジェクトです。イベント定義オブジェクトには、`type`、`name`、`inputs`、`anonymous`のフィールドがあります。

*Faucet.sol*サンプルコントラクトのABIを生成するには、solcコマンドラインSolidityコンパイラを使用します。

```
$ solc --abi Faucet.sol
======= Faucet.sol:Faucet =======
Contract JSON ABI
[{"inputs":[{"internalType":"uint256","name":"withdrawAmount","type":"uint256"}],
"name":"withdraw","outputs":[],"stateMutability":"nonpayable","type":"function"},
{"stateMutability":"payable","type":"receive"}]
```

コンパイラは、*Faucet.sol*によって定義されている2つの関数を記述するJSON配列を生成します。このJSONは、デプロイされた`Faucet`コントラクトにアクセスしたいどのアプリケーションでも使用できます。ABIを使用すると、ウォレットやDAppブラウザなどのアプリケーションは、正しい引数と引数タイプを使用して`Faucet`内の関数を呼び出すトランザクションを構築できます。たとえば、ウォレットが`withdraw`関数を呼び出すには、`withdrawAmount`という名前の`uint256`引数を指定する必要があることがわかります。ウォレットは、必要な値を入力するよう

にユーザーに促し、次にその値をエンコードして`withdraw`関数を実行するトランザクションを作成できます。

アプリケーションがコントラクトと対話するために必要なものは、ABIとそのコントラクトがデプロイされているアドレスだけです。

Solidityコンパイラと言語バージョンの選択

前述のコードで見たように、`Faucet`コントラクトはSolidityバージョン0.8.26で正常にコンパイルされています。しかし、Solidityコンパイラの異なるバージョンを使いたいとしたら、どうでしょうか。言語はまだ流動的であり、予期しない変更が加わるかもしれません。私たちのコントラクトはシンプルですが、仮にSolidityバージョン0.8.26で追加された機能を使っていたとして、それを0.8.25でコンパイルしようとするとどうなるでしょうか？

このような問題を解決するために、Solidityはバージョンプラグマ（version pragma）と呼ばれるコンパイラ命令を提供しています。これは、プログラムが特定のコンパイラ（および言語）バージョンを期待することをコンパイラに指示します。例を見てみましょう。

```
pragma solidity 0.8.26;
```

Solidityコンパイラはバージョンプラグマを読み込み、コンパイラのバージョンがバージョンプラグマと互換性がない場合はエラーを生成します。この場合、バージョンプラグマはこのプログラムがSolidityコンパイラのバージョン0.8.26でコンパイルできることを宣言しています。プラグマ命令はEVMバイトコードにコンパイルされません。互換性をチェックするために、コンパイラにのみ使用されるコンパイル時の命令です。

> **注記**
>
> プラグマ命令において、^記号は、指定されたバージョン以上のマイナーリビジョン（minor revision）でのコンパイルを許可することを示しています。たとえば、`pragma solidity ^0.7.1;`というプラグマ命令は、solcのバージョン0.7.1、0.7.2、0.7.3でコンパイルできますが、0.8.0ではコンパイルできないことを意味します（これはマイナーリビジョンではなくメジャーリビジョンです）。

ファウセットコントラクトにプラグマ命令を追加しましょう。新しいファイルに*Faucet2.sol*という名前を付けて、例7-2以降の例を見ながら変更を追跡します。

例7-2 Faucet2.sol：バージョンプラグマをファウセットに追加する

```
pragma solidity 0.8.26;
// SPDX-License-Identifier: GPL-3.0
```

```
// Our first contract is a faucet!
contract Faucet {
    // Give out ether to anyone who asks
    function withdraw(uint _withdrawAmount, address payable _to) public {
        // Limit withdrawal amount
        require(_withdrawAmount <= 100000000000000000);
        // Send the amount to the address that requested it
        _to.transfer(_withdrawAmount);
    }
    // Accept any incoming amount
    receive() external payable {}
}
```

バージョンプラグマを追加することは、コンパイラと言語のバージョンの不一致による問題を回避するため、ベストプラクティスです。この章では、他のベストプラクティスを検討し、`Faucet`コントラクトの改善を続けます。

Solidityを使ったプログラミング

この節では、Solidity言語の機能を見ていきます。2章で述べたように、最初のコントラクト例は非常に単純であり、さまざまな点で不備がありました。ここではSolidityの使い方を学びながら、徐々に改良していきます。しかしながら、Solidityは非常に複雑で急速に進化しているため、包括的なSolidityのチュートリアルにはなりません。ここでは基本をカバーし、自分で調べてゆけるようになるための十分な基礎を提供します。

データ型

まず、Solidityで提供されているいくつかの基本的なデータ型を見てみましょう。

ブーリアン型（bool）

論理演算子を使用したブール値、`true`または`false`です。論理演算子には、！（否定）、&&（論理積）、||（論理和）、==（等価演算子）、!=（非等価演算子）があります。

整数型（int、uint）

符号付き（`int`）および符号なし（`uint`）の整数。`int8`から`uint256`までの8ビット刻みで宣言します。サイズのサフィックスがない場合は、EVMのワードサイズに合わせて256ビットが使用されます。

固定小数点数型（fixed、ufixed）

固定小数点数。`(u)fixedMxN`で宣言されます。ここで*M*はビット単位のサイズ（8から256までの8刻み）、*N*はそのポイント以降の小数の数（最大18）です。たとえば、`ufixed32x2`です。

> **注記**
>
> 固定小数点数はSolidityではまだ完全にはサポートされていません。宣言はできますが、値の代入や読み出しはできません。

アドレス型（address）

20バイトのイーサリアムアドレス。`address`オブジェクトには多くの便利なメンバー関数があります。主なものとしては、`balance`（アカウント残高を返す）と`transfer`（アカウントへのイーサ送金）があります。

固定バイト配列型

`bytes1`から`bytes32`までで宣言された固定サイズのバイト配列です。

動的バイト配列型

`bytes`または`string`で宣言された可変サイズのバイト配列です。

列挙型（enum）

離散値を列挙するためのユーザー定義型です。`enum NAME {LABEL1, LABEL2, ...}`のように指定します。列挙型の基底型は`uint8`であるため、メンバーは256個以下であり、すべての整数型に明示的に変換できます。

配列型（array）

固定または動的のいずれかのタイプの配列です。`uint32[][5]`は、符号なし整数の5つの動的配列の固定サイズ配列です。

構造体型（struct）

変数をグループ化するためのユーザー定義データコンテナです。`struct NAME {TYPE1 VARIABLE1; TYPE2 VARIABLE2; ...}`のように指定します。

マッピング型（mapping）

キー ⇒ *値*ペアのハッシュルックアップテーブルです。`mapping(KEY_TYPE ⇒ VALUE_TYPE) NAME`のように指定します。

これらのデータ型に加えて、異なる単位を計算するために使用できる値リテラルもSolidityは提供しています。

時間単位

グローバル変数`block.timestamp`は、ブロックが公開されてブロックチェーンに追加された時刻を、Unixエポック（1970年1月1日）からの秒数で表します。`seconds`、`minutes`、`hours`、`days`の単位を、基本単位`seconds`の倍数に変換して、サフィックスとして使用できます。

> **注記**
>
> 1つのブロックには複数のトランザクションを含めることができるため、ブロック内のすべてのトランザクションは同じ`block.timestamp`を共有します。これは各トランザクションが開始された正確な瞬間ではなく、ブロックが公開された時刻を反映しています。

イーサ単位

`wei`と`ether`の単位を、基本単位`wei`の倍数に変換して、サフィックスとして使用できます。以前は`finney`と`szabo`の単位も利用可能でしたが、Solidityバージョン0.7.0で廃止されました。

ファウセットコントラクトの例では、`withdrawAmount`変数に`uint`（これは`uint256`のエイリアスです）を使用しました。`msg.sender`で設定した`address`変数も間接的に使用しました。この章の残りの部分では、これらのデータ型を使っていきます。

サンプルコントラクトの読みやすさを向上させるために、単位乗数を使ってみましょう。`withdraw`関数では、最大引き出し量を制限し、イーサの基本単位であるweiで制限を表します。

```
require(withdrawAmount <= 10000000000000000);
```

これでは数字列が長くてわかりにくいので、weiの代わりにイーサで値を表現するために、単位乗数`ether`を使用してコードを改善します。

```
require(withdrawAmount <= 0.1 ether);
```

変数：定義とスコープ

Solidityにおける変数や関数の定義構文は、他の静的型付け言語と似ています。各変数に型と名前を指定し、必要に応じて初期値を割り当てます。状態変数（state variables）については、可視性（visibility）を指定することもできます。デフォルトの可視性は`internal`であり、その変数はコントラクト自身とその派生コントラクトからのみアクセス可能です。他のスマートコントラクトからアクセスできるようにするには、`public`の可視性を使用する必要があります。

Solidityのスマートコントラクトには、3種類の変数スコープがあります。

状態変数（State variables）

スマートコントラクト内に永続的なデータを保存するもので、ブロックチェーン上に値を記録することにより*永続的な状態*（persistent state）として知られています。状態変数はスマートコントラクト内で定義されますが、関数の外側に置かれます。例：`uint public count;`

ローカル変数（Local variables）

計算処理中に使用される一時的なデータで、短期間だけ情報を保持します。ローカル変数はブロックチェーン上には保存されず、関数内に存在し、定義されたスコープの外からはアクセスできません。例：`uint count = 1;`

グローバル変数（Global variables）

Solidityが自動的に提供するもので、明示的な宣言やインポートなしに利用できます。ブロックチェーン環境に関する情報を提供し、プログラム内で使用するためのユーティリティ関数も含まれています。定義済みのグローバル変数は、次の項で網羅的に紹介します。

先ほど簡単に触れたとおり、状態変数は可視性を指定して宣言できます。Solidityは3つの異なる可視性レベルを提供しています。

- **public変数**: 自動的にgetter関数を生成し、外部のコントラクトがその値を読み取ることを可能にしますが、変更はできません。
- **internal変数**: コントラクト自身とその派生コントラクトからのみアクセス可能です。
- **private変数**: internal変数と似ていますが、派生コントラクトからもアクセスできません。

定義済みのグローバル変数とグローバル関数

EVMでコントラクトが実行されると、少数のグローバルオブジェクトへのアクセス権を持ちます。これらには`block`、`msg`、`tx`といったオブジェクトが含まれます。さらにSolidityは、いくつかのEVMオペコードを定義済み関数として公開しています。この項では、Solidityのスマートコントラクト内からアクセスできる変数と関数について調べます。

トランザクション／メッセージコールコンテクスト

`msg`オブジェクトは、このコントラクトの実行を開始したトランザクションコール（EOAによる）またはメッセージコール（コントラクトによる）です。これには有用な属性が多数含まれます。

msg.sender

msg.senderは本書ですでに使用しています。これは、このコントラクトコールを開始したアドレスを表します。これはトランザクションを送信した発信元のEOAとは限りません。コントラクトがEOAトランザクションによって直接呼び出された場合は、トランザクションに署名したアドレスになりますが、そうでなければコントラクトアドレスになります。

msg.value

このコールで送信されたイーサの値（wei単位）です。

msg.data

このコントラクト呼び出しのデータペイロードです。

msg.sig

データペイロードの最初の4バイトで、関数セレクタ（function selector）にあたります。

> **注記**
>
> あるコントラクトが別のコントラクトを呼び出すたびに、`msg` のすべての属性の値が新しい呼び出し元の情報を反映して変わります。唯一の例外はdelegatecall関数です。この関数は、元の `msg` コンテクスト内で別のコントラクトやライブラリのコードを実行します。

トランザクションコンテクスト

`tx` オブジェクトは、トランザクション関連の情報にアクセスする手段を提供します。

tx.gasprice

呼び出し元トランザクションのガス価格です。

tx.origin

このトランザクションの発信元EOAのアドレスです。

ブロックコンテクスト

`block` オブジェクトには、現在のブロックに関する以下の情報が含まれています。

block.basefee

現在のブロックのベースフィー（base fee）です。トランザクションがブロックに含まれるために必要な最低ガス料金を表す、動的に調整される値です。

block.blobbasefee

blobトランザクションのための動的に調整されるベースフィーです。EIP-4844によるイーサリアムのスケーラビリティ改善の一環として、大規模データを効率的に処理するために導入されました。

block.chainid

ブロックが現在構築されているブロックチェーンの一意の識別子です。

block.prevrandao

ビーコンチェーンによって提供される、前のブロックのランダムネスビーコンから導出された擬似乱数値です。乱数を必要とするスマートコントラクトに有用ですが、ある程度操作される可能性があるため、重要度の低い処理にのみ使用すべきです。

block.coinbase

現在のブロックの手数料とブロック報酬の受取人のアドレスです。

block.difficulty

Paris（The Merge）以前のEVMバージョンにおける現在のブロックのディフィカルティ（PoW）です。PoSコンセンサスモデルを採用するそれ以降のEVMバージョンでは、`block.prevrandao`の非推奨エイリアスとして機能します。

block.gaslimit

現在のブロックに含まれるすべてのトランザクションで消費できる最大ガス量です。

block.number

現在のブロック番号（ブロックチェーンの高さ）です。

block.timestamp

マイナーによって現在のブロックに配置されたタイムスタンプ（Unixエポック以降の秒数）です。

アドレスオブジェクト

入力として渡されるか、コントラクトオブジェクトからキャストされるかにかかわらず、すべてのアドレスには次のような多くの属性とメソッドがあります。

address.balance

アドレスの残高（wei単位）です。たとえば、現在のコントラクト残高は`address(this).balance`となります。

address.code

そのアドレスに保存されているコントラクトバイトコードです。EOAアドレスの場合は空のバイト配列を返します。

address.codehash

そのアドレスに保存されているコントラクトバイトコードのKeccak-256ハッシュです。

address.transfer(amount)

このアドレスに金額（wei単位）を送金します。エラーが発生した場合は例外をスローします。`Faucet`の例では、`msg.sender`アドレスのメソッドとして`msg.sender.transfer`としてこの関数を使用しました。

address.send(amount)

`transfer`と同様ですが、例外をスローする代わりにエラー時に`false`を返します。`send`の戻り値を必ず確認してください。

address.call(payload)

データペイロードを使用して任意のメッセージコールを構築できる低レベルの`CALL`関数です。エラー時には`false`を返します。注意：受信者は（偶然にまたは悪意を持って）あなたのガスをすべて使い果たし、コントラクトをOOG（out of gas）例外で停止させる可能性があります。`call`の戻り値を必ず確認してください。

address.delegatecall(payload)

`address(this).call(...)`のような低レベルの`DELEGATECALL`関数ですが、このコントラクトのコードが`address`のコードに置き換えられます。プロキシパターン（proxy pattern）の実装に特に有用です。エラー時には`false`を返します。警告：上級者向けです。

address.staticcall(payload)

`address(this).call(...)`のような低レベルの`STATICCALL`関数ですが、読み取り専用モードで実行されるため、呼び出された関数は状態を変更したりイーサを送信したりすることはできません。エラー時には`false`を返します。

> **注記**
>
> `address.send()`と`address.transfer()`はいずれも固定量の2,300ガスユニットを転送しますが、これはフォールバック（fallback）ロジックを実行するには不十分な場合があります。EIP-7702の導入に伴い、より柔軟な`address.call()`の使用が推奨されています。詳しくは9章で説明します。

組み込み関数

その他、知っておくべき関数は次のとおりです。

addmod, mulmod

モジュロ加算およびモジュロ乗算のための関数です。たとえば`addmod(x,y,k)`は(x + y) % kを計算します。

keccak256, sha256, ripemd160

さまざまな標準ハッシュアルゴリズムでハッシュを計算するための関数です。

ecrecover

署名からメッセージに署名したアドレスを復元します。

selfdestruct(recipient_address)

現在は非推奨です。かつてはコントラクトを削除し、アカウントに残っているイーサを受信者アドレスに送信するために使用されていました。EIP-6780以降、セルフデストラクト命令がコントラクト作成と同じトランザクション内で呼び出された場合にのみ削除が行われます。それ以外の場合は、資金は移動されますが、コントラクトとその状態はクリアされません。

this

現在のコントラクトです。`Address`型に明示的に変換して、現在実行中のコントラクトアカウントのアドレスを取得できます：`address(this)`。

super

継承（inheritance）階層において1つ上のレベルにあるコントラクトです。

gasleft

現在の実行コンテクストで残っているガスの量です。

blockhash(block_number)

ブロック番号で指定されたブロックのハッシュです。直近の256ブロック分のみ利用可能です。

blobhash(index)

現在のトランザクションに関連付けられた`index`番目のblobのハッシュです。

コントラクトの定義

Solidityの主要なデータ型は`contract`です。`Faucet`の例は、単に`contract`オブジェクトを定義しています。オブジェクト指向言語の他のオブジェクトと同様に、*コントラクト*はデータとメソッドを含むコンテナです。

Solidityは、コントラクトに似た他の2つのオブジェクト型を提供します。

インターフェイス（interface）

インターフェイスの定義は、関数が定義されず宣言されているだけであることを除いて、コントラクトとまったく同じ構造です。このタイプの宣言はしばしばスタブ（stub）と呼ばれ、実装なしに関数の引数と戻り値の型を伝えます。インターフェイスはコントラクトの「形」を規定します。継承された場合、インターフェイスによって宣言された関数はすべて、子によって定義されなければなりません。

ライブラリ（library）

ライブラリコントラクトは、一度だけデプロイされ、`delegatecall`メソッドを使用して他のコントラクトによって使用されることを意図したものです（「アドレスオブジェクト」を参照）。

関数

コントラクト内では、EOAトランザクションまたは別のコントラクトによって呼び出すことができる関数を定義します。`Faucet`の例では、`withdraw`と`receive`の2つの関数があります。

Solidityで関数を宣言するのに使う構文は以下のとおりです。

```
function FunctionName([parameters]) {public|private|internal|external} [virtual|override]
[pure|view|payable] [modifiers] [returns (return types)]
```

これらの各コンポーネントを見てみましょう。

FunctionName

関数の名前です。EOAからのトランザクション内で、もしくは別のコントラクトから、あるいは同じコントラクト内からでも、関数を呼び出すために使用されます。

parameters

名前のあとに、関数に渡す必要がある引数とその名前と型を指定します。`Faucet`の例では、`withdraw`関数の唯一の引数として`uint withdrawAmount`を定義しました。

次のキーワードのセット（`public`、`private`、`internal`、`external`）は、関数の可視性（visibility）を指定します。

public

publicはデフォルトです。このような関数は、他のコントラクト、EOAのトランザクション、またはコントラクト内から呼び出すことができます。`Faucet`の例では、両方の関数がpublicとして定義されています。

private

private関数はinternal関数と似ていますが、派生コントラクトから呼び出すことはできません。

internal

internal関数はコントラクト内からのみアクセスでき、他のコントラクトやEOAトランザクションから呼び出すことはできません。派生コントラクト（このコントラクトを継承したもの）から呼び出すことができます。

external

external関数はpublic関数と似ていますが、キーワード`this`を明示的に付けない限り、コントラクト内から呼び出すことはできません。

*internal*と*private*という用語は、やや誤解を招く表現であることに注意してください。なぜなら、コントラクト内の関数やデータは常にパブリックブロックチェーン上で*可視*であり、誰でもコードやデータを見ることができるからです。ここで説明しているキーワードは、関数を呼び出す方法とタイミングにのみ影響します。

> **注記**
>
> 関数の可視性と状態変数の可視性を混同しないでください。両者はキーワードとセマンティクスを共有していますが、異なるものです。状態変数の可視性はオプションですが、関数の可視性は明示的に定義しなければなりません。

2番目のキーワードのセット（`pure`、`view`、`payable`）は、関数の動作に影響を与えます。

pure

pure関数は、ストレージ内の変数を読み書きしません。保存されているデータやブロックチェーンの状態を参照せずに、引数に対してのみ操作し、データを返すことができます。pure関数は、副作用や状態なしに、宣言型プログラミングを促進することを目的としています。

view

viewとしてマークされた関数は、状態を変更しないことを約束します。コンパイラは`view`修飾子を強制せず、適用可能な場合に警告を生成するだけです。

payable

payable関数は入金を受け入れることができます。`payable`として宣言されていない関数は入金を拒否します。EVMの仕様により2つの例外があります。

- コインベース支払い
- `SELFDESTRUCT`の継承

これらは、フォールバック関数が`payable`として宣言されていなくても支払われます。支払い時にコード実行が行われないため、この挙動は理にかなっています。

次に、2つの特殊な関数、`receive`と`fallback`の動作を見てみましょう。

receive()

receive関数は、コントラクトがイーサを受け取れるようにします。コントラクトが空のcalldataでコールを受信したときにトリガーされ、通常は純粋なイーサ送金（`.send()`や`.transfer()`を使用したものなど）の際に発生します。`receive() external payable { ... }`として宣言され、引数を持つことも値を返すこともできませんが、virtualやオーバーライド（override）にでき、修飾子も持てます。

fallback()

フォールバック（fallback）関数は、コントラクトが他のどの関数シグネチャにも一致しないデータで呼び出されたときに実行されます。

`fallback() external [payable]`または`fallback(bytes calldata input) external [payable] returns (bytes memory output)`のいずれかで宣言できます。fallback関数にinputパラメータが含まれている場合、コントラクトに送信されたデータ全体が格納されます（`msg.data`と同等）。receive関数と同様に、fallback関数はpayableやvirtualにでき、オーバーライドや修飾子も使えます。

現在では、主に*プロキシパターン*（proxy pattern）の実装に使用されています。これはスマートコントラクトのアップグレード可能性を実現する設計パターンです。

> **注記**
>
> コントラクトにreceive関数がなく、payableなfallback関数がある場合、そのような送金時にfallback関数が実行されます。コントラクトにreceive関数もpayableなfallback関数もない場合、イーサを受け取ることができず、トランザクションは例外でリバートされます。

コントラクトのコンストラクタ

一度だけ使用される特別な関数があります。コントラクトが作成されると、定義済みであれば*コンストラクタ関数*が実行され、コントラクトの状態が初期化されます。コンストラクタはコントラクト作成と同じトランザクションで実行されます。コンストラクタ関数はオプションです。`Faucet`の例ではコンストラクタを持っていないことに気づくでしょう。

コンストラクタは`constructor`キーワードで指定できます。以下のようになります。

```
pragma 0.8.26
// SPDX-License-Identifier: GPL-3.0
contract MEContract {
 address owner;
 constructor () { // This is the constructor
  owner = msg.sender;
 }
}
```

コントラクトのライフサイクルは、EOAまたはコントラクトアカウントによる作成トランザクションから始まります。コンストラクタがある場合は、コントラクト作成の一部として実行され、作成中のコントラクトの状態を初期化します。コンストラクタは実行後、コントラクトのコードから取り除かれます。

> **注記**
>
> コンストラクタはpayableとしてマークすることもできます。これは、コントラクト作成トランザクションと一緒にETHを送信したい場合に必要です。コンストラクタがpayableでない場合、デプロイ時に送信されたETHはトランザクションをリバートさせます。

関数修飾子

Solidityは*関数修飾子*（function modifier）と呼ばれる特別な種類の関数を提供しています。関数宣言に修飾子の名前を追加することで、関数に修飾子を適用します。修飾子はコントラクト内の多くの関数に適用する条件を作成するために最もよく使われます。`destroy`関数にはすでにアクセス制御ステートメントがあります。この条件を表す関数修飾子を作成してみましょう。

```
modifier onlyOwner {
 require(msg.sender == owner);
 _;
}
```

`onlyOwner`という名前のこの関数修飾子は、修飾するすべての関数に条件を設定します。コントラクトの`owner`として格納されているアドレスが、トランザクションの`msg.sender`のアドレスと同じである必要があります。これはアクセス制御（access control）の基本的な設計パターンであり、コントラクトのオーナーだけが`onlyOwner`修飾子を持つ関数を実行できるようにします。

関数修飾子に独特な構文上の「プレースホルダ」があることに気づいたかもしれません。アンダースコアに続いてセミコロン（`_;`）です。このプレースホルダは、修飾される関数のコードに置き換えられます。本質的に、修飾子は修飾された関数を「囲み」、アンダースコア文字で識別される場所にコードを配置します。

関数修飾子を適用するには、その名前を関数宣言に追加します。関数には複数の修飾子を適用できます。それらは宣言された順序で、コンマ区切りのリストとして適用されます。

`onlyOwner`修飾子を使用する`changeOwner`関数を定義してみましょう。

```
function changeOwner(address newOwner) public onlyOwner {
 require(newOwner != address(0), "New owner address not set");
 owner = newOwner;
}
```

関数修飾子の名前（`onlyOwner`）はキーワード`public`の後ろにあり、`changeOwner`関数が`onlyOwner`修飾子によって修飾されていることを示しています。これは「オーナーだけが新しいオーナーアドレスを設定できる」と読むことができます。実際には、結果のコードは`onlyOwner`のコードを`changeOwner`の周りに「ラップ」することと同等です。

関数修飾子は非常に便利なツールです。関数の前提条件を記述し、それらを一貫して適用することを可能にし、コードを読みやすくし、結果としてセキュリティの監査も容易にします。アクセス制御に最もよく使用されますが、非常に用途が広く、他のさまざまな目的に使用できます。

コントラクトの継承

Solidityの`contract`オブジェクトは*継承*（inheritance）をサポートしています。これは基本コントラクトを追加機能で拡張するための仕組みです。継承を使用するには、キーワード`is`で親コントラクトを指定します。

```
contract Child is Parent {
 ...
}
```

この構文では、`Child`コントラクトは`Parent`のすべてのメソッド、機能、変数を継承します。Solidityは多重継承（multiple inheritance）もサポートしており、キーワード`is`のあとにコンマ区切りでコントラクト名を指定できます。

```
contract Child is Parent1, Parent2 {
 ...
}
```

継承チェーンの上位にある関数を呼び出すには、`Parent1.functionName()`のようにコントラクトを明示的に指定するか、フラット化された継承階層で1つ上のレベルの関数を呼び出したい場合は`super.functionName()`を使用します。

コントラクトの継承機能により、モジュール性と拡張性を備え、再利用を実現できるような方法でコントラクトを記述できます。シンプルで最も一般的な機能を実装するコントラクトから始

め、それらの機能をより専門的なコントラクトで継承して拡張していきます。

`Faucet`コントラクトでは、構築時に割り当てられたオーナーのアクセス制御を導入しました。この機能は非常に一般的で、多くのコントラクトが備えています。これを汎用コントラクトとして定義し、継承を用いて`Faucet`コントラクトに拡張できます。例をより充実させるために、アクセス制御とともに一時停止（pausable）機能を追加してみましょう。

まず、基本コントラクト`Owned`を定義します。`Owned`は`owner`変数を持ち、コンストラクタでその値を設定します。

```
contract Owned {
    address owner;
    // Contract constructor: set owner
    constructor() {
        owner = msg.sender;
    }
    // Access control modifier
    modifier onlyOwner {
        require(msg.sender == owner);
        _;
    }
}
```

次に、`Owned`を継承する基本コントラクト`Pausable`を定義します。

```
contract Pausable is Owned {
    bool paused;
    // Status check modifier
    modifier whenNotPaused {
        require(paused == false);
        _;
    }
    // Functions to pause/unpause user operations
    function pause() public onlyOwner {
        paused = true;
    }
    function unpause() public onlyOwner {
        paused = false;
    }
}
```

ご覧のとおり、`Pausable`コントラクトは`Owned`に定義された`onlyOwner`関数修飾子を使用できます。また、`Owned`に定義された`owner`アドレス変数とコンストラクタも間接的に使用されます。継承により各コントラクトはよりシンプルになり、その特定の機能に集中できるため、詳細をモジュール方式で管理できます。

これで、`Owned`コントラクトをさらに拡張し、`Faucet`でその機能を継承できます。

```
contract Faucet is Pausable {
    // Give out ether to anyone who asks
    function withdraw(uint _withdrawAmount, address payable _to) public whenNotPaused {
        // Limit withdrawal amount
        require(_withdrawAmount <= 0.1 ether);
        // Send the amount to the address that requested it
        _to.transfer(_withdrawAmount);
    }
    // Accept any incoming amount
    receive() external payable {}
}
```

`Pausable`を継承し、その`Pausable`が`Owned`を継承しているため、`Faucet`コントラクトは`whenNotPaused`修飾子を使用できるようになりました。その出力は`Owned`コントラクトのコンストラクタで定義されたオーナーによって制御できます。機能はこれらの関数が`Faucet`の中にあったときと同じですが、このモジュラーアーキテクチャのおかげで、関数や修飾子を再び書くことなく他のコントラクトで再利用できます。コードの再利用とモジュール化により、コードがよりクリーンになり、読みやすくなり、監査も容易になります。

継承したコントラクトの機能を一部変更したい場合もあるでしょう。幸い、Solidityにはそのための適切な機能があります。関数のオーバーライド（function overriding）です。virtualとして宣言された関数は、継承チェーンの上位にあるコントラクトによってオーバーライドでき、継承のアプローチを高い柔軟性で保ちます。

例を見てみましょう。一時停止機能を一方向にしたい、つまり一度停止したらコントラクトを再開できないようにしたいとします。これを実現するには、`Pausable`コントラクトの`unpause`関数を`virtual`としてマークし、`Faucet`コントラクトで`override`属性を付けて`unpause`関数を再宣言し、revertする新しい動作を定義する必要があります。

```
contract Pausable is Owned {
    bool paused;
    // Status check modifier
    modifier whenNotPaused {
        require(paused == false);
        _;
    }
    // Functions to pause/unpause user operations
    function pause() public virtual onlyOwner {
        paused = true;
    }
    function unpause() public virtual onlyOwner {
        paused = false;
    }
}
contract Faucet is Pausable {
    // Give out ether to anyone who asks
    function withdraw(uint _withdrawAmount, address payable _to) public whenNotPaused {
```

```
        // Limit withdrawal amount
        require(_withdrawAmount <= 0.1 ether);
        // Send the amount to the address that requested it
        _to.transfer(_withdrawAmount);
    }
    function unpause() public view override onlyOwner {
        revert("Disabled feature");
    }
    // Accept any incoming amount
    receive() external payable {}
}
```

ご覧のとおり、`Faucet`の`unpause()`関数は`override`キーワードで宣言する必要があります。`Pausable`コントラクトでは、一貫性のために`pause`と`unpause`の両関数をvirtualとしてマークしましたが、実際に変更する必要があったのは`unpause`だけです。

> **注記**
>
> Solidityで関数をオーバーライドする場合、可視性はよりアクセスしやすい方向にのみ変更できます。具体的には、externalからpublicに変更できますが、その逆はできません。ミュータビリティ（状態変更の可否）については、より厳しくでき、nonpayableからviewやpure（`unpause`で行ったように）へ、viewからpureへと変更できます。ただし1つ大きな例外があります。関数がpayableとしてマークされている場合、それを他のものに変更することはできません。

多重継承

Solidityで多重継承を使用する場合、コントラクトが継承される順序を決定するために、C3線形化（C3 linearization）アルゴリズムと呼ばれるものに依拠しています。このアルゴリズムは、継承順序が厳密で予測可能であることを保証し、循環継承のような問題を回避するのに役立ちます。簡単に言うと、このアルゴリズムは関数を探す際に基本コントラクトを確認する順序を決定します。この順序は右から左へ進みます。つまり右側のコントラクトが「最も派生した」ものとみなされます。たとえば、`contract C is A, B { }`というコントラクト宣言では、コントラクトBがコントラクトAよりも派生度が高くなります。

さて、C3線形化の使用に加えて、Solidityには追加の安全策があります。重要なルールの1つは、複数のコントラクトが同じ関数を持っている場合、どのコントラクトがオーバーライドされているかを明示的に示す必要があるということです。例を見てみましょう。

```
contract A {
    function foo() public virtual returns(string memory){
        return "A";
    }
```

```
}
contract B {
    function foo() public virtual returns(string memory){
        return "B";
    }
}
contract C is A, B {
}
```

一見すると、C3線形化がすべてを処理するのでうまくいきそうに見えます。しかし実際にはコンパイルされません。Solidityは「TypeError: Derived contract must override function `foo`. Two or more base classes define a function with the same name and parameter types.」というエラーをスローします。この問題を修正するには、AとBの両方から`foo()`関数を明示的にオーバーライドする必要があります。

```
contract C is A, B {
    function foo() public override(A, B) returns(string memory){
        return "C";
    }
}
```

そのため、SolidityがC3線形化を使用していても、コーディング中にそれについてあまり心配する必要はありません。Solidityは関数のオーバーライドを明示的に処理することを強制するからです。

ただし、C3線形化が重要になる場面の1つは、Solidityがコンストラクタの実行順序を決定するときです。コンストラクタはC3線形化の順序に従いますが、意外なことに逆順で実行されます。ただし、これは理にかなった挙動です。最も派生したコントラクトのコンストラクタは最後に実行されるべきだからです。先に実行されたコンストラクタが設定したものをオーバーライドする可能性があるためです。例を見てみましょう。

```
contract Base{
    uint x;
}
contract Derived1 is Base{
    constructor(){
        x = 1;
    }
}
contract Derived2 is Base{
    constructor(){
        x = 2;
    }
}
contract Derived3 is Derived1, Derived2 {
    uint public y;
    constructor() Derived1() Derived2() {
```

```
        y = x;
    }
}
```

この場合、`y`の値は期待どおり2になります。`Derived2`のコンストラクタが最後に実行され、`x`を2に設定するからです。

覚えておくべき重要な点は、コンストラクタ引数を提供する順序は実行順序に影響しないということです。たとえば、コンストラクタの呼び出しを次のように入れ替えることができます。

```
contract Derived3 is Derived1, Derived2 {
    uint public y;
    constructor() Derived2() Derived1() { // we switched the order here
        y = x;
    }
}
```

コンストラクタの順序を変更しても、結果は同じです。`y`の値は2になります。なぜなら、コンストラクタの実行順序はC3線形化によって決定され、コンストラクタを呼び出す順序によって決まるわけではないからです。

最後に注意点を1つ。Solidityが多重継承にC3線形化を使用していることにより、`super`キーワードが予想外の動作をすることがあります。`super`を呼び出すと、直接の親ではなく兄弟クラスの関数がトリガーされることがあります。これにより、継承チェーンにリストしていないクラスからメソッドが呼び出されるという意外な結果につながることがあります。

これはやや特殊なケースなので深くは掘り下げませんが、複雑な継承構成を持つコントラクトで`super`キーワードを使用する際には、必ずこの点を念頭に置いてください。

エラーハンドリング

コントラクトの呼び出しが終了し、エラーを返すことがあります。Solidityのエラーハンドリングは、`assert`、`require`、`revert`の3つの関数で処理します。

コントラクトがエラーで終了すると、複数のコントラクトが呼び出されていた場合、コントラクト呼び出しのチェーンの一番上まで、すべての状態変更（変数、残高などへの変更）が元に戻されます。これにより、トランザクションが*アトミック*であることが保証されます。つまり、トランザクションは正常に完了するか、状態に影響を与えずに完全に元に戻されるかのどちらかです。

`assert`関数と`require`関数は同じように動作し、条件を評価して、条件が偽の場合にはエラーで実行を停止します。慣例により、結果が正しいと予想されるときに`assert`を使用します。つまり、`assert`は内部条件をテストするために使用されます。一方、`require`は入力（関数の引数やトランザクションフィールドなど）をテストするときに使用され、条件に対する期待値を設定し

ます。`assert`が失敗した場合、`require`とは異なる動作をする点にも注目すべきです。`assert`は残りのガスをすべて消費します。そのため、発火した場合のコストが高くなります。これは、`assert`を「決して壊れてはならない不変条件」に対して使うべきとされる理由の1つです。

メッセージ送信者がコントラクトのオーナーであることをテストするために、関数修飾子`onlyOwner`で`require`を使用しました。

```
require(msg.sender == owner);
```

`require`関数は*ゲート条件*（gate condition）として機能し、条件が満たされていない場合は残りの関数の実行を防ぎ、エラーを生成します。エラーの原因を示すのに役立つテキストメッセージを含めることもできます。エラーメッセージはトランザクションログに記録され、ユーザーにエラーの内容と修正方法を知らせてユーザー体験を向上させるために、採用が推奨されています。そこで、`require`関数にエラーメッセージを追加してコードを改善できます。

```
require(msg.sender == owner, "Only the contract owner can call this function");
```

`revert`関数はコントラクトの実行を中止し、すべての状態変更を元に戻します。使い方は次の2つです。

- **ステートメント形式**: カスタムエラーを括弧なしで直接渡す
- **関数形式**: 文字列引数を取って括弧付きで呼び出す

カスタムエラーのほうがガス費用の点ではるかに安価ですが、エラー文字列もカスタムエラーもどちらもトランザクションログに記録されます。

```
revert();
revert("Error string");
revert CustomError(arg1, arg2);
```

コントラクト内の特定の条件は、明示的にチェックしなくても自動的にエラーを発生させます。たとえば、ファウセットコントラクトでは、引き出しリクエストを満たすのに十分なイーサがあるかどうかを確認しません。これは、送金を行うのに十分な残高がない場合、`transfer`関数がエラーで失敗し、トランザクションを元に戻すためです。

```
payable(msg.sender).transfer(withdrawAmount);
```

ただし、明示的にチェックし、失敗時に明確なエラーメッセージを提供するほうがよい場合もあります。送金の前に`require`ステートメントを追加することでそれを実現できます。

```
require(this.balance >= withdrawAmount,
 "Insufficient balance in faucet for withdrawal request");
payable(msg.sender).transfer(withdrawAmount);
```

このような追加のエラーチェックコードは、ガス消費量をわずかに増加させますが、省略した場合よりもわかりやすいエラーメッセージを提供できます。かつてはイーサリアムメインネットのコストが高かったため、ガス消費をできるだけ抑えることが必須でしたが、EIP-4844の導入によりそのコストは大幅に低下し、今日ではガス消費量はそれほど切迫した問題ではなくなりました。ただし、ガス効率と徹底的なエラーチェックの間で適切なバランスを取ることは依然として重要です。

Solidityはtry/catch機能により、エラーハンドリングにさらに多くの制御を提供します。これは、外部コントラクトを呼び出す際にエラーをより適切に処理できる非常に便利な機能です。何か問題が発生したときにトランザクション全体が失敗してリバートされるのではなく、エラーをキャッチして次に何をすべきかを決定できます。

try/catchを使用する場合、基本的に外部呼び出しを`try`ブロックでラップします。呼び出しが成功すれば、`try`ブロック内のコードが通常どおり実行されます。

しかし、呼び出し先のコントラクトでガス不足が発生したり、`require`ステートメントに引っかかったり、例外がスローされたりすると、コードは`catch`ブロックにジャンプし、そこでエラーを処理できます。

簡単な例を示します。

```
function sampleExternalCall(address target, uint amount) public {
    try ITargetContract(target).someFunction(amount) {
        // This runs if the call is successful
        emit Success("Call succeeded!");
    } catch {
        // This runs if the call fails
        emit Error("Call failed!");
    }
}
```

エラーの種類に応じて、さまざまな方法でエラーをキャッチできます。基本的な`catch`ブロックはすべてのエラーをキャッチしますが、特定のエラーをキャッチすることもできます。たとえば、`catch Error(string memory reason)`を使用してエラー文字列を返すエラーをキャッチしたり、`catch (bytes memory lowLevelData)`でデータを返さない低レベルのエラーを処理したりできます。また、`catch Panic(uint errorCode)`でオーバーフローやゼロ除算などのパニックエラーもキャッチできます。

try/catchは外部呼び出しでのみ機能します。同じコントラクト内の内部関数呼び出しには対応していません。同じコントラクト内の関数が失敗した場合、通常どおりリバートされ、try/catchでキャッチすることはできません。

イベント

トランザクションが（成功したかどうかによらず）完了すると、トランザクションレシートが生成されます。トランザクションレシートには、トランザクションの実行中に発生したアクションに関する情報を提供するログエントリが含まれています。*イベント*は、これらのログを構築するために使用されるSolidityの高レベルオブジェクトです。

イベントは、ライトクライアントやDAppサービスで特に有用です。ライトクライアントやDAppサービスは、特定のイベントを「監視」してユーザーインターフェイスに報告したり、基盤となるコントラクトのイベントを反映させるためにアプリケーションの状態を変更したりできます。

イベントオブジェクトは、シリアライズされた引数を取り、ブロックチェーン内のトランザクションログに記録されます。引数の前にindexedキーワードを置くことで、インデックス付きテーブル（ハッシュテーブル）の一部とし、アプリケーションで検索やフィルタリングが可能になります。

イベントの追加

ファウセットの例ではこれまでイベントを追加していないため、ここで追加しましょう。引き出しを記録するためのイベントと、入金を記録するためのイベントの2つを追加します。これらのイベントをそれぞれ`Withdrawal`と`Deposit`と呼びます。まず、`Faucet`コントラクト内でイベントを定義します。

```
contract Faucet is Pausable {
 event Withdrawal(address indexed to, uint amount);
 event Deposit(address indexed from, uint amount);
 [...]
}
```

ファウセットにアクセスするために構築されたユーザーインターフェイスで検索やフィルタリングができるように、アドレスをindexedにしました。

次に、emitキーワードを使用して、トランザクションログにイベントデータを埋め込みます。

```
// Give out ether to anyone who asks
function withdraw(uint withdrawAmount) public {
 [...]
 payable(msg.sender).transfer(withdrawAmount);
 emit Withdrawal(msg.sender, withdrawAmount);
}
// Accept any incoming amount
receive() external payable {
 emit Deposit(msg.sender, msg.value);
}
```

結果として、*Faucet.sol*コントラクトは例7-3のようになります。

例7-3. Faucet.sol：修正されたイベント付きのファウセットコントラクト

```
// Version of Solidity compiler this program was written for
pragma solidity 0.8.26;
// SPDX-License-Identifier: GPL-3.0
contract Owned {
    address owner;
    // Contract constructor: set owner
    constructor() {
        owner = msg.sender;
    }
    // Access control modifier
    modifier onlyOwner {
        require(msg.sender == owner);
        _;
    }
}
contract Pausable is Owned {
    event Paused();
    event Unpaused();
    bool paused;
    // Status check modifier
    modifier whenNotPaused {
        require(paused == false);
        _;
    }
    // Functions to pause/unpause user operations
    function pause() public onlyOwner {
        paused = true;
        emit Paused();
    }
    function unpause() public onlyOwner {
        paused = false;
        emit Unpaused();
    }
}
contract Faucet is Pausable {
    event Withdrawal(address indexed to, uint amount);
    event Deposit(address indexed from, uint amount);
    // Give out ether to anyone who asks
    function withdraw(uint withdrawAmount) public whenNotPaused {
        // Limit withdrawal amount
        require(withdrawAmount <= 0.1 ether);
        // Send the amount to the address that requested it
        payable(msg.sender).transfer(withdrawAmount);
        emit Withdrawal(msg.sender, withdrawAmount);
    }
    // Accept any incoming amount
```

```
    receive() external payable {
        emit Deposit(msg.sender, msg.value);
    }
}
```

イベントを捕捉する

コードを使ってオンチェーンイベントをどのように捕捉できるか見ていきましょう。具体的には、イーサリアムメインネット上のUSDTトークン転送を監視するスクリプトを書きます。これを行うにはWeb3ライブラリが必要です。web3.jsが最初に普及しましたが、近年ではethers.jsがそれを追い越しています。開発者にはethers.jsが好まれているため、ここでもそれを使用します。

まず、プロジェクトをセットアップしましょう。新しいプロジェクトフォルダを作成し、次のコマンドを実行してethersライブラリをインストールします。

```
npm i ethers
```

次に、USDTコントラクトのABIが必要です。Etherscanのコントラクトソースコードのすぐ下から取得し（図7-1を参照）、プロジェクトフォルダに保存します。

図7-1. EtherscanのUSDT ABIセクション

次に、イーサリアムネットワークへの接続方法について説明します。イベントをリッスンするには、継続的な接続が必要なため、WebSocketプロバイダーが必要です。有料のプロバイダー（より信頼性が高く高速）か、ChainListのようなウェブサイトで見つけられる無料のパブリックプロバイダーのどちらかを選ぶことができます。レート制限がかかる可能性はありますが、この例ではパブリックなもので十分です。

> **注記**
>
> イベントをリッスンするには、RPCエンドポイントではなくWebSocketが必要です。RPCエンドポイントは関数の呼び出しやデータの取得といった単発のリクエストには適していますが、イベントを捕捉するには、クライアントとサーバー間の通信回線を開いたまま維持できるWebSocket接続が必要です。

それでは、コードを見ていきましょう。

```
const ethers = require("ethers");
const ABI = require("./USDTabi.json"); // the ABI we fetched from etherscan
const usdtAddress = "0xdac17f958d2ee523a2206206994597c13d831ec7"; // USDT Contract
const wssProviderURL = "wss://ethereum-rpc.publicnode.com"; // a public websocket provider
const wssProvider = new ethers.providers.WebSocketProvider(wssProviderURL);
const usdtContract = new ethers.Contract(usdtAddress, ABI, wssProvider);
async function getTransfer(){
    usdtContract.on("Transfer", (from, to, value, event)=>{
        let transferEvent ={
            from: from,
            to: to,
            value: value,
            eventData: event,
        }
        console.log(JSON.stringify(transferEvent, null, 2))
    })
}
getTransfer()
```

USDTアドレスとWebSocketプロバイダーURLを宣言した後、スクリプトはethersライブラリを使用して新しいWebSocketProviderインスタンスを作成するところから始まります。このプロバイダーは、指定したWebSocket URLを通じてイーサリアムネットワークに接続します。次に、`ethers.Contract`クラスを使用してUSDTスマートコントラクトのインスタンスを作成します。USDTアドレス、ABI、およびWebSocketプロバイダーを渡します。

ここからがスクリプトの核心部分です。USDTコントラクトが発行する`Transfer`イベントを捕捉するために、コントラクトインスタンスにイベントリスナーを設定します。`Transfer`イベントが発生するたびに、このリスナーは転送（Transfer）に関連する4つのパラメータを受け取るコールバック関数をトリガーします。コールバック内では、転送の詳細を取得してオブジェクトにまとめ、そのオブジェクトをJSON文字列としてフォーマットします。最後に、`console.log`を使ってそのJSON文字列をコンソールに出力し、各転送で何が起こっているかをリアルタイムで正確に確認できるようにします。

スクリプトの出力例は次のとおりです。

```
{
  "from": "0xc169e35abb35f8e712eCF9F6d9465C96962CA383",
  "to": "0x7E73F680243A93a9D98C5Ce4b349451805fc37ca",
  "value": {
    "type": "BigNumber",
    "hex": "0x55b27b90"
  },
  "eventData": {
    "blockNumber": 20687220,
    "blockHash": "0xa5c3c518d7246e516e076ef8d43c387dcb54d06702e9e059c583ce28a7a271b8",
```

```
    "transactionIndex": 166,
    "removed": false,
    "address": "0xdAC17F958D2ee523a2206206994597C13D831ec7",
    "data": "0x0000000000000000000000000000000000000000000000000000000055b27b90",
    "topics": [
      "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef",
      "0x000000000000000000000000c169e35abb35f8e712ecf9f6d9465c96962ca383",
      "0x000000000000000000000000007e73f680243a93a9d98c5ce4b349451805fc37ca"
    ],
    "transactionHash":
      "0xb527a5a18f10ed9b65dda7a914715a0b0bbfd6db053d8f6b35805ad49a588cfd",
    "logIndex": 300,
    "event": "Transfer",
    "eventSignature": "Transfer(address,address,uint256)",
    "args": [
      "0xc169e35abb35f8e712eCF9F6d9465C96962CA383",
      "0x7E73F680243A93a9D98C5Ce4b349451805fc37ca",
      {
        "type": "BigNumber",
        "hex": "0x55b27b90"
      }
    ]
  }
}
```


このようなイベントは、オフチェーンの通信だけでなく、デバッグにも非常に役立ちます。開発中には、トランザクションレシートの「logs」エントリでこれらのイベントを確認でき、期待どおりに動作しないときに問題解決の手がかりとなります。

他のコントラクトの呼び出し

自身のコントラクトから他のコントラクトを呼び出すのは非常に便利ですが、潜在的に危険な操作です。この操作を実現するさまざまな方法を検討し、各方法のリスクを評価します。端的に言えば、リスクは自分が呼び出しているコントラクトや、自分のコントラクトを呼び出しているコントラクトについてよく知らないことから生じます。スマートコントラクトを書くときは、呼び出し相手が主にEOAだと想定していても、任意の複雑な、あるいは悪意あるコントラクトが自分のコードを呼び出したり、自分のコードから呼び出されたりする可能性を止められないことを心に留めておかなければなりません。

新しいインスタンスを作成する

別のコントラクトを呼び出す最も安全な方法は、そのコントラクトを自分で作成することです。そうすれば、そのインターフェイスと振る舞いを確実に把握できます。これを行うには、他のオブジェクト指向言語と同様に、キーワード `new` を使って簡単にインスタンス化できます。Solidityでは、 `new` キーワードがブロックチェーン上にコントラクトを作成し、それを参照する

ために使用できるオブジェクトを返します。`Token`という別のコントラクト内から`Faucet`コントラクトを作成して呼び出したい場合を考えてみましょう。

```
contract Token is Pausable {
    Faucet _faucet;
    constructor() {
        _faucet = new Faucet();
    }
}
```

このコントラクト構築メカニズムは、コントラクトの正確な型とそのインターフェイスを確実に知っていることを保証します。`Faucet`コントラクトは`Token`のスコープ内で定義する必要があり、定義が別のファイルにある場合は`import`ステートメントで取り込めます。

```
import "Faucet.sol";
contract Token is Pausable {
    Faucet _faucet;
    constructor() {
        _faucet = new Faucet();
    }
}
```

必要に応じて、コントラクト作成時にイーサの送金額を指定したり、コンストラクタに引数を渡すこともできます。

```
import "Faucet.sol";
contract Token is Pausable {
    Faucet _faucet;
    constructor() {
        _faucet = new Faucet{value: 0.5 ether}();
    }
}
```

この場合、`Faucet`のコンストラクタがpayableである必要があることに注意してください。

`Faucet`の関数を呼び出すこともできます。以下の例では、`Token`の`changeOwner`関数内から`Faucet`の`changeOwner`関数を呼び出しています。

```
import "Faucet.sol";
contract Token is Pausable {
    Faucet _faucet;
    constructor() {
         _faucet = new Faucet{value: 0.5 ether}();
    }
    function changeOwner(address newOwner) onlyOwner {
        _faucet.changeOwner(newOwner);
```

```
    }
}
```

重要な点として、あなたが`Token`コントラクトのオーナーであっても、新しい`Faucet`コントラクトを所有するのは`Token`コントラクト自体であり、あなたではありません。本章で前述したように、外部呼び出し中は`msg.sender`が変わります。この場合、`Faucet`の実行コンテクストでは`msg.sender`は`Token`のアドレスになります。

既存のインスタンスへのアドレスキャスト

コントラクトを呼び出すもう1つの方法は、コントラクトの既存インスタンスのアドレスをキャストすることです。この方法では、既存のインスタンスに既知のインターフェイスを適用します。そのため、アドレス指定しているインスタンスが実際に想定している型であることを確実に知っていることが極めて重要です。例を見てみましょう。

```
import "Faucet.sol";
contract Token is Pausable {
    Faucet _faucet;
    constructor(address _f) {
        _faucet = Faucet(_f);
        _faucet.withdraw(0.1 ether)
    }
}
```

ここでは、コンストラクタへの引数として提供されたアドレス`_f`を受け取り、`Faucet`オブジェクトにキャストしています。このアドレスが実際に`Faucet`オブジェクトであるかどうかは確実にはわからないため、前述の方法よりもはるかに危険です。`withdraw`を呼び出すとき、`Faucet`の宣言と同じ引数を受け入れて同じコードを実行すると想定していますが、確実ではありません。このアドレスの`withdraw`関数は、同じ名前であっても、期待とはまったく異なる動作をするかもしれません。このように、入力として渡されたアドレスを特定のオブジェクトにキャストするのは、コントラクトを自分で作成するよりもはるかに危険です。

低レベル呼び出し：call、delegatecall、staticcall

Solidityは、他のコントラクトを呼び出すためのさらに「低レベル」な関数を提供しています。これらは同じ名前のEVMオペコードに直接対応しており、コントラクト間の呼び出しを手動で構築できます。そのため、他のコントラクトを呼び出すための最も柔軟で、かつ最も危険なメカニズムです。これらは2つの値を返します。操作が成功したかどうかを示す`bool success`と、戻りデータを含む`bytes memory data`です。

`call`メソッドを使用した同じ例を示します。

```
contract Token is Pausable {
    constructor(address _faucet) {
        _faucet.call(abi.encodeWithSignature("withdraw(uint256)", 0.1 ether));
    }
}
```

ご覧のとおり、この種の`call`は関数へのブラインド呼び出しであり、生のトランザクションを構築するのと非常に似ていますが、コントラクトのコンテクスト内から行われます。`call`関数は問題がある場合に`false`を返すので、エラーハンドリングのために戻り値を評価できます。

```
contract Token2 is Pausable {
    constructor(address _faucet) {
        (bool res, ) = _faucet.call(
            abi.encodeWithSignature("withdraw(uint256)", 0.1 ether)
        );
        if (!res) {
            revert("Withdrawal from faucet failed");
        }
    }
}
```

`call`の変種として`staticcall`と`delegatecall`があります。「アドレスオブジェクト」のセクションで述べたように、`staticcall`は状態変更が発生しないことを保証する方法で別のコントラクトの関数を呼び出します。つまり、呼び出された関数は状態変数の変更、ブロックチェーンの状態との対話、またはイーサの送信ができません。

`delegatecall`は`msg`コンテクストが変わらないという点で`call`とは異なります。たとえば、`call`では`msg.sender`の値が呼び出し元のコントラクトに変更されるのに対し、`delegatecall`は呼び出し元のコントラクトと同じ`msg.sender`を保持します。基本的に`delegatecall`は、現在のコントラクトの実行コンテクスト内で別のコントラクトのコードを実行します。

`delegatecall`は、ライブラリからコードを呼び出すために最もよく使われます。また、別の場所に保存されているライブラリ関数を使用しながら、そのコードを自分のコントラクトのストレージデータと連携させるパターンも可能にします。これの明確な例がプロキシパターンです。

`delegatecall`は細心の注意を払って使用すべきです。特に呼び出し先のコントラクトがライブラリとして設計されていない場合、予期しない影響が生じる可能性があります。

`call`と`delegatecall`がライブラリやコントラクトを呼び出す際のさまざまな呼び出しセマンティクスを、例を使って説明しましょう。例7-4では、イベントを使用して各呼び出しの詳細をログに記録し、呼び出しの種類に応じて呼び出しコンテクストがどのように変化するかを確認します。

例7-4. CallExamples.sol：さまざまな呼び出しセマンティクスの例

```
pragma solidity 0.8.26;
contract CalledContract {
    event callEvent(address sender, address origin, address from);
    function calledFunction() public {
        emit callEvent(msg.sender, tx.origin, address(this));
    }
}
library CalledLibrary {
    event callEvent(address sender, address origin, address from);
    function calledFunction() public {
        emit callEvent(msg.sender, tx.origin, address(this));
    }
}
contract Caller {
    function makeCalls(CalledContract _calledContract) public {
        // Calling CalledContract and CalledLibrary directly
        _calledContract.calledFunction();
        CalledLibrary.calledFunction();
        // Low-level calls using the address object for CalledContract
        (bool res, ) = address(_calledContract).
            call(abi.encodeWithSignature("calledFunction()"));
        require(res);
        (res, ) = address(_calledContract).
            delegatecall(abi.encodeWithSignature("calledFunction()"));
        require(res);
    }
}
```

この例からわかるように、主なコントラクトは`Caller`であり、ライブラリ`CalledLibrary`とコントラクト`CalledContract`を呼び出します。呼び出されるライブラリとコントラクトの両方に、`calledEvent`というイベントを発行する同一の`calledFunction`関数があります。`calledEvent`イベントは、`msg.sender`、`tx.origin`、`this`の3つのデータをログに記録します。`calledFunction`が呼び出されるたびに、直接呼び出されるか`delegatecall`を通じて呼び出されるかに応じて、異なる実行コンテクスト（すべてのコンテクスト変数が異なる値を持つ可能性があります）が設定されます。

`Caller`では、最初にそれぞれの`calledFunction`を呼び出して、コントラクトとライブラリを直接呼び出します。次に、低レベル関数`call`と`delegatecall`を明示的に使用して`CalledContract.calledFunction`を呼び出します。これにより、さまざまな呼び出しメカニズムがどのように動作するかを確認できます。

コントラクトをデプロイし、`makeCalls`関数を実行して、イベントを捕捉しましょう。わかりやすくするために、出力されるアドレスをラベル（例：`CALLER_CONTRACT_ADDRESS`）に置き換えます。

`makeCalls`関数を呼び出して`CalledContract`のアドレスを渡した後、それぞれの異なる呼び出しによって発行された4つのイベントを捕捉しました。`makeCalls`関数の各ステップを見ていきま

しょう。

最初の呼び出しは次のとおりです。

```
_calledContract.calledFunction();
```

ここでは、`calledFunction`の高レベルABIを使用して、`CalledContract.calledFunction`を直接呼び出しています。発行されたイベントの引数は次のとおりです。

```
{
    sender: 'CALLER_CONTRACT_ADDRESS',
    origin: 'EOA_ADDRESS',
    from: 'CALLED_CONTRACT_ADDRESS'
}
```

ご覧のとおり、`msg.sender`は`Caller`コントラクトのアドレスです。`tx.origin`はトランザクションを`Caller`に送信した自分のアカウント`web3.eth.accounts[0]`のアドレスです。イベントの最後の引数からわかるように、イベントは`CalledContract`によって発行されました。

`makeCalls`の次の呼び出しはライブラリへのものです。

```
CalledLibrary.calledFunction();
```

私たちがコントラクトを呼び出した方法と同じに見えますが、非常に異なった振る舞いをします。発行された2番目のイベントを見てみましょう。

```
{
    sender: 'EOA_ADDRESS',
    origin: 'EOA_ADDRESS',
    from: 'CALLER_CONTRACT_ADDRESS'
}
```

今回、`msg.sender`は`Caller`のアドレスではありません。代わりに自分のアカウントのアドレスであり、トランザクションのoriginと同じです。その理由は、ライブラリを呼び出すとき、呼び出しは常に`delegatecall`であり、呼び出し元のコンテクスト内で実行されるからです。つまり、`CalledLibrary`のコードが実行されているとき、そのコードが`Caller`の内部で実行されているかのように、`Caller`の実行コンテクストを継承します。変数`this`（発行されたイベントで`from`として表示）は、`CalledLibrary`内からアクセスされた場合でも`Caller`のアドレスです。

次の2つの呼び出しでは、低レベル関数の`call`と`delegatecall`を用いて、先ほど見たものと同じイベントが発行されることで、私たちの想定が正しいことを確認します。

ガスに関する検討事項

第14章でより詳細に説明しますが、ガスはスマートコントラクトプログラミングにおいて慎重に考慮すべき重要な要素です。ガスは、イーサリアムにおいてトランザクションが消費する計算量の上限を定めるリソースです。計算中にガスリミットを超えると、以下の一連の事象が発生します。

- 「ガス不足」（out of gas）例外がスローされる。
- 実行前のコントラクトの状態が復元（リバート）される。
- ガスの支払いに使用されたすべてのイーサがトランザクション手数料として徴収され、*返金されない*。

ベストプラクティス

ガスはトランザクションを開始したユーザーによって支払われるため、ユーザーはガス費用の高い関数の呼び出しを避けるべきです。このように、コントラクトの関数にかかるガス費用を最小限に抑えることは、プログラマーにとって最も望ましいことです。この目的達成のため、スマートコントラクトを作成する際に推奨されるプラクティスがあります。

動的サイズの配列を避ける

関数が各要素に対して操作を実行したり、特定の要素を検索したりする動的なサイズの配列のループは、ガスを使用し過ぎる危険性があります。実際、コントラクトは期待される結果を得る前に、あるいはすべての要素を遂行する前に、ガスを使い果たしてしまうかもしれません。このような場合、何も結果を得ることなく、時間とイーサを浪費することになります。

他のコントラクトへの呼び出しを避ける

他のコントラクトを呼び出すことは、特にその関数のガス費用がわからない場合、ガス不足になるリスクを伴います。十分にテストされていて、広く使用されているライブラリを使用するようにしてください。他のプログラマーからのテスト量が少なければ少ないほど、そのライブラリを使用するリスクは高くなります。

冗長なストレージアクセスを避ける

ストレージ変数へのアクセスは、読み取りであれ書き込みであれ、メモリ変数の操作よりもはるかに多くのガスを消費します。そのため、可能な限りストレージへの直接アクセスを避けるのがよいでしょう。たとえば、計算中にストレージ変数を何度も読み取る必要がある場合、まずその値をメモリ変数にコピーしておくのがよい方法です。こうすることで、毎回ストレージにアクセスする代わりに、より安価なメモリ変数に繰り返しアクセスでき、ガス費用を節約できます。

注記

これを理解するために補足すると、Solidityはデータを保持するための場所をいくつか使い分けています。storage（永続的で高価）、memory（実行中は一時的でより安価）、calldata（主に外部関数の入力に使用される読み取り専用領域）、stack（非常に短命な値に使用され、アクセスコストが最も安い）です。適切な場所の選択は、データの使用方法、永続化の必要性、変更可能かどうか、外部から渡されるかどうかによって決まります。これらの違いについては第14章で詳しく掘り下げますが、パフォーマンスとコストへの影響について早い段階から考えておくと役立ちます。

ガス費用の見積もり

イーサリアムが最初にローンチされたとき、ガス費用の見積もりはオークションの落札価格を推測するようなものでした。ビットコインがトランザクション手数料を処理する方法と似ていて、自分でガス価格を設定し、マイナーが最も高い入札のトランザクションを優先していました。これは、混雑時にはトランザクションを迅速に処理してもらうために、しばしばより多くの費用を提示しなければならないことを意味していました。この仕組み自体は機能していましたが、ネットワークが混雑しているときにはガス価格が非常に高騰するなど、価格が不安定になりがちでした。

そして2021年にEIP-1559が導入され、状況は一変しました。適切なガス価格を推測する代わりに、イーサリアムはネットワークの活動状況に基づいて自動的に調整されるベースフィーを導入しました。これにより、ガス手数料はより予測可能になりました。さらに、急いでいる場合にはチップ（*優先手数料*と呼ばれる）を追加して処理を速めることもできます。現在ではガス費用の見積もりがより簡単になり、トランザクションを処理してもらうためだけに過剰に支払うことも少なくなりました。

トランザクションのガス費用をどのように見積もれるか、詳しく見ていきましょう。まず、すべてのトランザクションには、ガス費用に関する2つの主要な要素があります。ベースフィーと優先手数料（priority fee）です。

ベースフィー

これは、トランザクションをブロックに含めてもらうために支払う必要のある最低限のガス量です。ベースフィーはネットワークが自動的に決定し、混雑度に基づいて動的に調整されます。ブロックが満杯のときはベースフィーが上がり、ブロックの利用が少ないときは下がります。

優先手数料（チップ）

これは、マイナー（またはPoSにおけるバリデータ）にトランザクションの優先処理を促すために追加する手数料です。トランザクションをより速く処理してもらうためのチップのようなものです。この手数料は自分で設定できますが、ウォレットアプリケーションがトランザクションの取り込み速度の希望に応じて適切な値を提案してくれます。

トランザクションの合計ガス費用は、使用されたガス量（トランザクションの複雑さに依存）に実効ガス価格を掛けて算出されます。実効ガス価格は、ベースフィーと優先手数料の合計です。

ガス費用を見積もるには、以下の手順に従います。

1. 現在のベースフィーを確認します。これは、Etherscan Gas Trackerなどのガス追跡ツールや、Web3ライブラリ（例：ethersの `maxFeePerGas()`）を使って確認できます。

2. トランザクションの処理速度に応じて、優先手数料（チップ）を選択します。急いでいる場合はチップを増やすことができます。ガス追跡ツールは、現在のネットワーク状況と処理の緊急度に基づいて適切なチップ額を判断するのに役立ちます。

3. 合計ガス価格（ベースフィー + チップ）に推定ガス使用量を掛けます。開発者であれば、Web3ライブラリ（例：ethersの `estimateGas()`）を使って推定ガス使用量を計算できます。一般ユーザーの場合は心配いりません。トランザクションを送信する際にウォレットアプリが自動的に処理してくれます。

たとえば、ベースフィーが20 gwei、チップを2 gweiに設定し、トランザクションが50,000ガスを使用する場合、推定ガス費用は次のようになります。

(20 gwei + 2 gwei) × 50,000 = 1,100,000 gwei

つまり、110万gwei、すなわち0.0011 ETHとなります。

コントラクトをメインネットにデプロイするときに驚かないように、開発ワークフローに関数のガス費用の評価を組み込んでおきましょう。

まとめ

本章では、スマートコントラクトについて詳しく学び、コントラクトプログラミング言語であるSolidityについて解説しました。簡単なコントラクトの例である*Faucet.sol*を取り上げ、それを徐々に改良してより複雑にしながら、Solidity言語のさまざまな側面を学んできました。第8章では、もう1つのコントラクト指向プログラミング言語であるVyperを扱います。VyperとSolidityを比較し、2つの言語の設計上の違いを示しながら、スマートコントラクトプログラミングの理解を深めていきます。

第8章　スマートコントラクトとVyper

Vyperは、EVM向けに設計された確立されたコントラクト指向プログラミング言語であり、開発者がわかりやすいコードを書けるようにすることで、優れた監査性の提供を目指しています。実際、Vyperの原則の1つは、誤解を招くようなコードを開発者が事実上書けないようにすることです。

本章では、スマートコントラクトに関するよくある問題を検討し、Vyperコントラクトプログラミング言語を紹介したうえで、Solidityと比較してその違いを説明します。

脆弱性とVyper

2023年だけで、イーサリアムエコシステムにおけるスマートコントラクトの脆弱性により、約20億ドルが盗まれました。スマートコントラクトの脆弱性は、コードに起因します。こうした脆弱性が意図的とは考えにくいものの、結果として望ましくないコードがユーザの予期しない資金喪失を招いており、理想的な状況とは言えません。Vyperは安全なコードをより簡単に書けるよう、また誤解を招くコードや脆弱なコードを誤って書いてしまうことがより難しくなるよう設計されています。

Solidityとの比較

Vyperが安全でないコードを書きにくくする方法の1つは、Solidityのいくつかの機能を意図的に除外することです。こうした機能除外の判断は、セキュリティ最優先の原則と、Pythonの明確さやシンプルさから着想を得たVyperのルーツを反映しています。Vyperでのスマートコントラクト開発を検討する場合、Vyperにどの機能がなく、なぜないのかを理解することが重要です。この節では除外された機能を詳しく調べ、その理由を見ていきます。

機能セットを削減して曖昧さを減らす一方で、Vyperは開発者や監査者の実際のニーズに応えるよう進化してきました。たとえば、コントラクトを単一ファイルに収めるという当初の方針は監査性の最大化に役立ちましたが、プロトコルが大規模かつ複雑になるにつれてボトルネックとなりました。

この問題に対処するため、現代のVyperでは洗練されたモジュールシステムが導入されています。状態へのアクセスやコード再利用に対する厳密な制御を維持しつつ、コントラクトを複数のファイルに分割できるようになりました。このモジュールシステムは従来の継承ではなくコンポジションの原則に従い、構造と可読性のバランスをうまく取っています。DeFiプロトコルやステーキングシステムなど高い信頼性が求められる分野では、コードの明確さと監査のしやすさからVyperの採用が広がっています。

修飾子

7章で見たように、Solidityでは修飾子を用いた関数を書くことができます。たとえば、次に示す`changeOwner`関数は、`onlyBy`と呼ばれる修飾子のコードを処理の一部として実行します。

```
function changeOwner(address _newOwner)
 public
 onlyBy(owner)
{
 owner = _newOwner;
}
```

この修飾子は所有権に関するルールを強制します。以下に示すとおり、`changeOwner`関数に代わって事前チェックを実行する仕組みです。

```
modifier onlyBy(address _account)
{
 require(msg.sender == _account);
 _;
}
```

しかし修飾子は、ここに示したようなチェックを実行するためだけに存在するわけではありません。実際、修飾子は呼び出し元の関数の実行中に、スマートコントラクトの環境を大きく変化させることができます。端的にいえば、修飾子の影響範囲は*広範*です。

別のSolidityの例を見てみましょう。

```
enum Stages {
 SafeStage,
 DangerStage,
 FinalStage
}
uint public creationTime = block.timestamp;
Stages public stage = Stages.SafeStage;
function nextStage() internal {
 stage = Stages(uint256(stage) + 1);
}
modifier stageTimeConfirmation() {
 if (stage == Stages.SafeStage &&
 block.timestamp >= creationTime + 10 days)
 nextStage();
 _;
}
function a()
 public
 stageTimeConfirmation
 // More code goes here
```

```
{
}
```

開発者は自身のコードが呼び出す他のコードを常にチェックすべきです。ただし、時間的制約がある場合や疲労で集中力が低下している場合など、特定の状況下ではコードのある1行を見落とすこともありえます。特に関数呼び出しの階層を追いながら変数の状態を把握し、巨大なファイル内を探索する場面ではなおさらです。

先に挙げた例をもう少し詳しく見てみましょう。開発者が`a`という公開関数を書いているとします。この開発者はコントラクトに触れるのが初めてで、他の誰かが書いた修飾子を利用しようとしています。一見すると`stageTimeConfirmation`修飾子は、呼び出し元の関数に関連してコントラクトの経過時間をチェックしているだけのように見えます。

しかし実際には、この修飾子は`nextStage`というもう1つの関数を呼び出しており、開発者はそれに気づかないかもしれません。この単純化されたシナリオでは、公開関数`a`を呼び出すだけでスマートコントラクトの`stage`変数が`SafeStage`から`DangerStage`に変わります。

Vyperでは修飾子が完全に廃止されています。Vyperの推奨は次のとおりです。修飾子でアサーションのみを行いたい場合は、インラインのチェックとアサートを関数の一部として使用します。スマートコントラクトの状態を変更する場合も同様に、変更を関数内に明示します。こうすれば、読み手は関数の動作を理解するために修飾子のコードを頭の中で展開して関数に当てはめる必要がなくなり、可監査性と可読性が向上します。

クラスの継承

プログラマーは継承を使って、既存のソフトウェアライブラリからあらかじめ用意された機能や性質、振る舞いを獲得し、すでに書かれたコードを活用できます。継承は強力で、コードの再利用を促進します。Solidityはオブジェクト指向プログラミングの重要な機能であるポリモーフィズム（多態性）と多重継承をサポートしていますが、Vyperはどちらもサポートしていません。

Vyperの考え方では、継承を実装するにはコードの作成者や監査者が複数のファイル間を行き来して、プログラムの動作を理解する必要があります。また、多重継承はコードを理解不可能なほど複雑にする可能性があるとVyperは考えています。これはSolidityのドキュメントでも暗に認められており、多重継承がどのように問題になりうるかの例が示されています。

インラインアセンブリ

インラインアセンブリを用いると、開発者はEVMへの低レベルなアクセスが可能になり、SolidityのプログラムからEVMの命令に直接アクセスして操作を実行できます。たとえば、次のアセンブリコードはメモリ番地`0x80`に3を加算します。

```
3 0x80 mload add 0x80 mstore
```

インラインアセンブリを使用すると、変数名を検索してその変数が読み書きされるすべての箇所を特定できなくなります。Vyperは、追加機能と引き換えに可読性を犠牲にする代償が大きすぎると判断しました。そのためインラインアセンブリはサポートしていません。

関数のオーバーロード

関数のオーバーロードを用いると、開発者は同じ名前の関数を複数書くことができます。どの関数が使われるかは、その時々で渡される引数の型に依存します。例として、次の2つの関数を見てみましょう。

```
function f(uint256 _in) public pure returns (uint256 out) {
 out = 1;
}
function f(uint256 _in, bytes32 _key) public pure returns (uint256 out) {
 out = 2;
}
```

最初の関数（名称は`f`）はuint256型の入力引数を1つ受け取ります。2つ目の関数（こちらの名称も`f`）は2つの引数を受け取り、1つはuint256型、もう1つはbytes32型です。異なる引数を取る同名の関数が複数定義されている状況は混乱を招くため、Vyperは関数のオーバーロードをサポートしていません。

変数の型キャスト

Vyperは型変換に対してSolidityとは大きく異なるアプローチを取っており、利便性よりも明示的で安全な型の扱いを優先しています。すべての型変換は組み込みの`convert()`関数で明示的に行う必要があり、データ型がいつどのように変更されるかを開発者が常に把握できます。

型変換は、情報が失われる可能性のあるものと、失われないものの2種類に分けられます。Vyperの`convert()`関数はどちらも扱えますが、予期しない動作を防ぐために常に明示的な境界チェックを行います。たとえば、大きな整数型から小さな整数型に変換する場合、値が変換先の型の範囲に収まらなければトランザクションをリバートします。

Vyperにおける型変換の構文はシンプルです。

```
# 整数型間の変換
small_value: uint8 = 42
large_value: uint256 = convert(small_value, uint256)  # 安全なアップキャスト
back_to_small: uint8 = convert(large_value, uint8)    # 境界チェック付きダウンキャスト
```

この明示的なアプローチにより、型変換の場面ではVyperのコードはSolidityよりも冗長になります。しかしすべての変換が意図的かつ明示的になるため、値の切り捨てや予期しないオーバーフローが誤って発生する余地はありません。`convert()`関数は、変換でデータが失われる場合や入力値が変換先の型の有効範囲外である場合にトランザクションをリバートします。

デコレータ

関数の開始部に、次に挙げるデコレータを使うことができます。

@internal

`@internal`デコレータは、関数をコントラクトの外部からアクセス不可にする。これはデフォルトの関数可視性であるため、省略可能である。

@external

`@external`デコレータは、関数を外部から可視かつ実行可能にする。たとえば、イーサリアムウォレットもコントラクトを参照するときにこれらの関数を表示する。

@view

`@view`デコレータを付けた関数は状態変数を変更できない。実際、このデコレータが付いた関数が状態変数を更新しようとすると、コンパイラはプログラム全体を（適切なエラーで）拒否する。

@pure

`@pure`デコレータを付けた関数は、ブロックチェーンの状態を読み取ったり、pureでないメソッドや他のコントラクトを呼び出したりできない。

@payable

`@payable`デコレータを持つ関数のみがvalue（送金額）を転送できる。

@deploy

`@deploy`デコレータは、コントラクトのコンストラクタ関数を示す。この関数はコントラクトがブロックチェーンにデプロイされるときに1度だけ実行され、通常は初期状態変数や設定のセットアップに用いられる。Vyperでは`__init__()`関数のみに`@deploy`を付けることができ、コントラクトにコンストラクタロジックを含める場合はこのデコレータが必須である。

@raw_return

`@raw_return`デコレータを付けた関数は、ABIエンコーディングを適用せずに生のバイト列を返す。他コントラクトの呼び出し結果を、追加のエンコードなしでそのまま転送したいプロキシコ

ントラクトやヘルパーコントラクトで特に有用である。ただし重要な制限があり、`@external`関数にのみ使用でき、インターフェース定義では使用できない。他のコントラクトからこの関数を呼び出す場合は、戻り値がABIエンコードされていない可能性があるため、インターフェース呼び出しではなく`raw_call`を使用する。

@nonreentrant

`@nonreentrant`デコレータは関数にロックをかけ、対象の関数が実行完了するまで再度呼び出せないようにする。リエントランシー攻撃とは、外部コントラクトが保護された関数にコールバックし、予期しない動作や悪用を引き起こす攻撃である。このデコレータはそれを防ぐために使用される。Vyperはデフォルトでリエントランシー防止のプラグマもサポートしており、明示的にオーバーライドしない限りすべての外部関数がリエントラント不可となる。たとえば、コントラクトAがこのデコレータを使用してコントラクトBに外部呼び出しを行った場合、コントラクトBがコントラクトAの保護された関数にコールバックしようとするとトランザクションはリバートされる。

> **注記**
>
> Vyperバージョン0.2.15、0.2.16、0.3.0のnonreentrant機能には、2023年半ばに発見されCurveプロトコルへの攻撃に悪用された重大なバグが含まれていた。リエントランシーの脆弱性については9章で詳しく取り上げる。

Vyperはデコレータのロジックを明示的に実装しています。たとえば`@payable`と`@view`の両方を持つ関数があると、Vyperのコンパイルは失敗します。送金額（value）を転送する関数は定義上、状態を更新するため`@view`にはなりえません。Vyperの関数には`@external`か`@internal`のどちらかを付ける必要があります（両方は不可です）。

関数と変数の順序

Vyperのスコープと宣言はC99のスコープ規則に従っており、想像以上の柔軟性があります。モジュールシステムを使用しない場合、Vyperのコントラクトは依然として単一ファイルに収める必要があります。ただし、以前のバージョンにあった厳密な順序制約は、モジュールスコープの宣言では緩和されています。

モジュールスコープ（関数本体の外側）で宣言された変数や関数は、正式な宣言よりも前であってもコントラクト全体から参照できます。多くの現代的なプログラミング言語における前方宣言と同様に、状態変数の参照や関数の呼び出しを、ファイル内での記述位置より前に行えます。

現代のVyperにおけるスコープの動作を示す例を見てみましょう。

```
# この関数は下で宣言される状態変数を参照できる
@external
def get_stored_value() -> uint256:
    return self.stored_data  # 後で宣言される変数を参照
# この関数は上の関数を呼び出すことができる
@external
def check_if_positive() -> bool:
    return self.get_stored_value() > 0
# 状態変数の宣言 - 上の関数からアクセス可能
stored_data: public(uint256)
```

ただし関数スコープ内では、Vyperは依然として厳密な順序規則を維持しています。ローカル変数は使用前に宣言する必要があり、定数やイミュータブル変数などモジュールレベルの宣言名をローカル変数でシャドーイングすることはできません。このアプローチは、Pythonの柔軟性と、スマートコントラクト開発のニーズとのバランスを取ったものです。スマートコントラクト開発では、状態変数や関数の関係が明確に見えることがセキュリティ監査上重要となります。

コンパイル

Vyperを試す最も簡単な方法は、Remixオンラインコンパイラを使うことです。Webブラウザだけを用いてスマートコントラクトを作成し、コンパイルできます（プラグインマネージャでvyper-remixプラグインを有効化する必要があります）。

注記

Vyperには組み込みの共通インターフェース（ERC-20やERC-721など）が付属しており、これらのコントラクトとのやりとりをすぐに利用できる。Vyperにおけるコントラクトはグローバル変数として宣言する必要がある。ERC-20変数の宣言例は次のとおりである。
`from vyper.interfaces import ERC20  token: ERC20`

コマンドラインからコントラクトをコンパイルすることもできます。Vyperのコントラクトはそれぞれ *.vy* 拡張子のファイルに保存されます。Vyperをインストールすれば、次のコマンドでコントラクトをコンパイルできます。

```
vyper ~/hello_world.vy
```

コンパイラは豊富な出力オプションを備えています。人間が読みやすいJSON形式のABI記述を取得するには、次のコマンドを使用します。

```
vyper -f abi ~/hello_world.vy
```

開発やテストでは、バイトコード、オペコード、インターフェースファイルなどの追加出力が必要になることもあるでしょう。コンパイラは多数の出力フォーマットをサポートしています。

```
vyper -f abi,bytecode,interface,source_map ~/hello_world.vy
```

現代のVyperには高度な最適化モードも含まれています。`--optimize gas`（デフォルト）でガス効率の最適化、`--optimize codesize`でコントラクトサイズの最小化が可能です。新しい実験的なVenom IRパイプラインは`--experimental-codegen`で有効化でき、さらに優れた最適化を実現します。

Remixやコマンドラインコンパイラは学習や実験に最適ですが、大規模なプロジェクトでは包括的な開発フレームワークが必要になります。

- **ApeWorx（旧Ape）**: 自動テストやデプロイスクリプティング、さまざまなネットワークとの統合といった機能を備え、Vyperを手厚くサポートしている。
- **Foundry**: 主にSolidity向けだがVyperの開発もサポートしており、強力なテストやシミュレーション機能を提供している。

これらのフレームワークは、複雑なアプリケーションを構築するプロフェッショナルなスマートコントラクト開発者に必要な成熟した開発環境を提供します。

コンパイラレベルでのオーバーフローエラーに対する保護

ソフトウェアにおけるオーバーフローエラーは、実際の価値を扱う場合に破滅的な結果をもたらすことがあります。たとえば、2018年4月中旬のあるトランザクションは、57,896,044,618,658,100,000,000,000,000,000,000,000,000,000,000,000,000,000,000,000,000を超えるBECトークンの不正な移転を示しています。このトランザクションは、Beauty ChainのERC-20トークンコントラクト（*BecToken.sol*）における整数オーバーフローが原因でした。

Vyperのコア機能の1つは、スマートコントラクト開発を歴史的に悩ませてきたオーバーフローエラーのリスクを軽減する組み込みのオーバーフロー保護です。Vyperのオーバーフロー保護は包括的で、整数演算の例外ケースを処理するSafeMath相当の保護を含んでいます。加算や減算、乗算、除算などの操作はデフォルトで安全であり、オーバーフローやアンダーフローが発生すると例外がスローされます。加えて、以下のタイミングごとにクランプ（値の範囲チェック）を使って値の制限を強制します。

- 定数リテラルの読み込み
- 関数への値の受け渡し
- 変数への代入

Solidityの最近のバージョン（0.8.0以降）でも、Vyperが当初から提供してきたものと同様のネイティブなオーバーフローチェックがコンパイラレベルで統合されています。これにより現代

のSolidityでは算術演算にオーバーフローチェックが自動的に含まれ、SafeMathのような追加ライブラリなしでオーバーフローエラーのリスクが大幅に低減されました。

この変更でSolidityはVyperのセキュリティ最優先のアプローチに近づきましたが、Vyperの実装はクランプ操作や言語全体にわたるより一貫した境界チェックを含め、依然としてより包括的です。

根本的な違いは思想にあります。Vyperはオーバーフロー保護をコア原則として最初から設計されたのに対し、Solidityは歴史的な脆弱性への対処として後から追加しました。このアプローチの違いは、安全でないコードをデフォルトで書きにくくするというVyperの設計思想を反映しています。

データの読み書き

ストレージへのデータの保存、読み込み、修正にはコストがかかりますが、ほとんどのスマートコントラクトに必要な操作です。スマートコントラクトは、2つの場所にデータを書き込むことができます。

グローバル状態

特定のスマートコントラクトの状態変数は、イーサリアムのグローバルな状態トライ木に記録されている。スマートコントラクトは、その特定のコントラクトアドレスに関連するデータのみを保存、読み取り、修正できる（つまり、スマートコントラクトは他のスマートコントラクトに対して直接読み書きできない）。

ログ

スマートコントラクトは、ログイベントを通じてイーサリアムのチェーンデータに書き込むことができる。Vyperでは、イベントの宣言と使用の構文がクリーンかつわかりやすく、コードの明確さを重視するVyperの方針に沿っている。

Vyperのイベント宣言は構造体の宣言に似ています。たとえば、`MyLog`というイベントの宣言は次のように記述します。

```
event MyLog:
    arg1: indexed(address)
    arg2: uint256
    message: indexed(bytes[100])
```

インデックス付き引数は最大4つまで持てます（検索可能なトピックとなります）。インデックスなしの引数はイベントデータの一部となり、個数に制限はありません。インデックス付き引数はイベントのフィルタリングや検索に有用で、インデックスなしの引数はより大量のデータを含められます。

ログイベントの発行には`log`文を使用します。

```
log MyLog(msg.sender, 42, b"Hello, Vyper!")
```

`pass`文を使って引数なしのイベントを作成することもできます。

```
event SimpleEvent: pass
# コード内で使用する場合：
log SimpleEvent()
```

スマートコントラクトはログイベントを通じてイーサリアムのチェーンデータに書き込めます。ただし、自身が作成したオンチェーンのログイベントを読み込むことはできません。とはいえ、ログイベントでチェーンデータに書き込む利点の1つは、軽量クライアントがパブリックチェーン上でログを検出し読み取れることです。たとえば、公開されたブロックの`logsBloom`の値からログイベントの有無を判定できます。ログイベントの存在が確認されれば、該当するトランザクションレシートからログデータを取得できます。

まとめ

Vyperは、強力で興味深いコントラクト指向のプログラミング言語です。その設計はセキュリティとシンプルさを優先し、「正確さ」に偏っています。このアプローチにより、プログラマーはより良いスマートコントラクトを作成し、深刻な脆弱性を引き起こす落とし穴を避けることができるでしょう。

ただし、すべてのものにはトレードオフがあります。Vyperの厳格な設計原則はセキュリティとコードの明確さを高めますが、他の言語にある柔軟性を犠牲にしています。また、VyperはSolidityほど広く使われておらず、開発も進んでいません。そのため、利用できるリソースやライブラリ、ツールが少ないのが現状です。コミュニティのサポートや既製のソリューション、充実したドキュメントを求める開発者にとっては課題となりえます。

次章では、スマートコントラクトのセキュリティをより詳細に検討します。スマートコントラクトで発生しうるあらゆるセキュリティ上の問題について読めば、Vyperの設計上のいくつかの重要なポイントがより明確になるでしょう。

第9章 スマートコントラクトのセキュリティ

セキュリティは、スマートコントラクトを書く際に最も重要な考慮事項の一つです。スマートコントラクトプログラミングの分野では、ミスのコストが高く、容易に悪用されます。この章では、セキュリティのベストプラクティスとデザインパターン、および*セキュリティアンチパターン*（スマートコントラクトに脆弱性をもたらしうる慣行やパターン）について見ていきます。

他のプログラムと同様に、スマートコントラクトは書かれた通りに正確に実行されます。しかし、実行結果が必ずしもプログラマーの意図通りとは限りません。さらに、すべてのスマートコントラクトはパブリックであり、どのユーザーもトランザクションを作成するだけでやり取りできます。あらゆる脆弱性が悪用される可能性があり、損失はほぼ常に回復不能です。したがって、ベストプラクティスに従い、十分にテストされたデザインパターンを使用することが極めて重要です。

堅牢な開発を、セキュリティの「スイスチーズモデル」における最初のレイヤーと考えてください。各保護レイヤーはスイスチーズのスライスのように機能します。単独では完璧ではありませんが、組み合わせることでより強力な防御を生み出します。最初のレイヤーは堅実な開発慣行に従うことです。具体的には、信頼性の高いデザインパターンの使用、明確で意図的なコードの記述、既知の落とし穴の積極的な回避が含まれます。この基盤となるレイヤーが、脆弱性からコントラクトを保護するための最良のスタートを提供します。その先には、テスト、コードレビュー、バグバウンティなどの追加レイヤーが保護を加えますが、すべては開発慣行から始まります。

セキュリティのベストプラクティス

*防御的プログラミング*は、スマートコントラクトに特に適したプログラミングスタイルです。以下の点を重視しており、すべてがベストプラクティスです。

ミニマリズム / シンプルさ

コードを書く前であっても、すべてのコンポーネントが本当に必要かどうか一歩引いて問いかける価値があります。設計をもっとシンプルにできないか、特定のデータ構造が不必要な攻撃対象面を導入していないかを検討すべきです。アーキテクチャが決まった後も、行数を減らし、エッジケースを排除し、本質的でない機能を削除する機会を探して、批判的な目でコードを見直すべきです。シンプルなコントラクトは推論、テスト、監査がより容易です。一部のDeFiプロトコルは正当に数千行に成長しますが、コードベースの大きさを自慢する人がいる場合は懐疑的であるべきです。より多くのコードは多くの場合、より多くの価値ではなく、より多くのバグを意味します。

コードの再利用

車輪の再発明をしないようにしましょう。必要な機能のほとんどを提供するライブラリやコントラクトがすでに存在する場合は、それを再利用してください。例えば、OpenZeppelinは広く採用され、徹底的にテストされ、コミュニティによって継続的にレビューされているコントラクトスイートを提供しています。自身のコード内では、DRY原則（Don't Repeat Yourself）に従ってください。コードスニペットが複数回繰り返されている場合、それを関数やライブラリとして書き直して再利用できないか検討してください。多くのデプロイメントで実戦テストされたコードは、どれほど自信があっても、書いたばかりのコードよりもほぼ常に安全です。「ここでは発明されていない」症候群に注意してください。機能やコンポーネントをゼロから構築して「改善」しようとする誘惑です。セキュリティリスクは改善の価値よりも大きいことが多いです。再利用は怠惰ではなく、賢明で防御的なエンジニアリングです。

コード品質

スマートコントラクトのコードは容赦がありません。すべてのバグが金銭的損失につながる可能性があります。スマートコントラクトのプログラミングを汎用プログラミングと同じように扱うべきではありません。SolidityでDAppを書くことは、JavaScriptでWebウィジェットを作成することとは異なります。むしろ、航空宇宙工学やそれに類する容赦のない分野と同様に、厳密なエンジニアリングとソフトウェア開発方法論を適用すべきです。コードを「ローンチ」した後は、問題を修正するためにできることはほとんどありません。コードがアップグレード可能であっても、何か問題が発生した場合に対応する時間は非常に限られることが多いです。プロジェクトのバグを自分より先に誰かが発見した場合、攻撃は1つまたは数個のトランザクションで展開され、介入できるはるか前の数秒以内に被害が完了していることを意味します。

可読性 / 監査可能性

コードは明確で理解しやすくあるべきです。読みやすいほど、監査も容易になります。スマートコントラクトはパブリックです。誰でもバイトコードを読むことができ、十分なスキルを持つ人はリバースエンジニアリングできます。したがって、開発者コミュニティの集合知を活用し、オープンソース開発の最高水準の恩恵を受けるために、協調的でオープンソースの方法論を使用して公開で開発することが有益です。イーサリアムコミュニティの一部であるスタイルと命名規約に従って、十分に文書化され読みやすいコードを書くべきです。

テストカバレッジ

可能な限りすべてをテストしてください。スマートコントラクトはパブリックな実行環境で動作し、誰でも好きな入力で実行できます。関数の引数などの入力が適切な形式であるか、適切な範囲内であるか、善意の目的であるかを決して仮定すべきではありません。コードの実行を続行させる前に、すべての引数が期待される範囲内にあり適切にフォーマットされていることを確認するためにテストしてください。

セキュリティリスクとアンチパターン

スマートコントラクトのプログラマーとして、最も一般的なセキュリティリスクに精通しておくべきです。そうすることで、コントラクトをこれらのリスクにさらすプログラミングパターンを検出し回避できます。以降のセクションでは、さまざまなセキュリティリスク、脆弱性がどのように発生しうるかの例、およびそれらに対処するために使用できる対策や予防的ソリューションを見ていきます。

以下のアンチパターンはWeb2セキュリティと同様に、攻撃を実行するためにしばしば組み合わされます。実世界の攻撃は通常、この章の例よりも複雑です。

リエントランシー

イーサリアムのスマートコントラクトの特徴の一つは、外部コントラクトのコードを呼び出して利用できることです。また、コントラクトは通常Etherを扱うため、さまざまな外部ユーザーアドレスにEtherを送信することも多くあります。これらの操作ではコントラクトが外部呼び出しを行う必要があります。攻撃者はこの外部呼び出しを乗っ取り、コールバック（フォールバック関数やフックなど、通常は`transfer`）を通じてコントラクトにさらなるコードを実行させることができます。コントラクト自身への再帰的な呼び出しも可能です。この種の攻撃は、2016年に発生した悪名高いDAOハッキングで使用されました。発見が比較的容易であり修正コストも低いにもかかわらず、あれから何年も経った今でもこの脆弱性を悪用した攻撃が多数確認されています。

脆弱性

この種の攻撃は、コントラクトが状態の更新を完了する前に、攻撃者が実行中のコントラクトの制御を奪うことで発生します。コントラクトは処理の途中にあるため、まだ状態（例えば重要な変数）を更新していません。攻撃者はこの脆弱なタイミングでコントラクトに「再入」し、不整合な状態を利用して意図されていない動作を引き起こすことができます。コントラクトがまだ安全で整合性のある状態に落ち着いていないため、攻撃者はセーフガードを回避し、データを操作し、資金を流出させることが可能になります。

リエントランシーは、実際の例がなければ理解が難しい概念です。例9-1のシンプルな脆弱性のあるコントラクトを見てみましょう。このコントラクトはイーサリアムの金庫として機能し、預金者が1週間に1 Etherのみ引き出せるようになっています。

例9-1. EtherStore: リエントランシーに対して脆弱なコントラクト

```
1 contract EtherStore {
2      uint256 public withdrawalLimit = 1 ether;
3      mapping(address => uint256) public lastWithdrawTime;
4    mapping(address => uint256) balances;
5
```

```
6    function depositFunds() public payable{
7      balances[msg.sender] += msg.value;
8    }
9
10    function withdrawFunds() public {
11        require(block.timestamp >= lastWithdrawTime[msg.sender] + 1 weeks);
12        uint256 _amt = balances[msg.sender];
13        if(_amt > withdrawalLimit){
14            _amt = withdrawalLimit;
15        }
16      (bool res, ) = address(msg.sender).call{value: _amt}("");
17        require(res, "Transfer failed");
18      balances[msg.sender] = 0;
19        lastWithdrawTime[msg.sender] = block.timestamp;
20    }
21 }
```

このコントラクトには`depositFunds`と`withdrawFunds`の2つのパブリック関数があります。`depositFunds`関数は単純に送信者の残高を増加させます。`withdrawFunds`関数は送信者が残高を引き出すことを可能にします。この関数は、直近1週間に引き出しが行われていない場合にのみ成功するように設計されています。

脆弱性は17行目にあります。コントラクトがユーザーに要求された量のEtherを送信する箇所です。例9-2のコントラクトを作成した攻撃者を考えてみましょう。

例9-2. Attack.sol: EtherStoreコントラクトのリエントランシー脆弱性を悪用するコントラクト

```
1 contract Attack {
2  EtherStore public etherStore;
3
4  // etherStore変数をコントラクトアドレスで初期化
5  constructor(address _etherStoreAddress) {
6      etherStore = EtherStore(_etherStoreAddress);
7  }
8
9  function attackEtherStore() public payable {
10      // 最も近いether単位で攻撃
11      require(msg.value >= 1 ether, "no bal");
12      // depositFunds()関数にethを送信
13      etherStore.depositFunds{value: 1 ether}();
14      // 攻撃開始
15      etherStore.withdrawFunds();
16  }
17
18  function collectEther() public {
19      payable(msg.sender).transfer(address(this).balance);
20  }
21
22  // receive関数 - fallback()関数でも同様に動作する
```

```
23  receive() external payable {
24      if (address(etherStore).balance >= 1 ether) {
25          // 被害者コントラクトへのリエントラント呼び出し
26          etherStore.withdrawFunds();
27      }
28  }
29 }
```

この攻撃はどのように行われるのでしょうか。まず、攻撃者は`EtherStore`のコントラクトアドレスを唯一のコンストラクターパラメーターとして、悪意あるコントラクト（アドレスを`0x0...123`とします）を作成します。これにより、パブリック変数`etherStore`が攻撃対象のコントラクトを指すように初期化されます。

次に、攻撃者は1以上のEtherを指定して`attackEtherStore`関数を呼び出します。ここでは1 Etherと仮定します。この例では、他の複数のユーザーがこのコントラクトにEtherを預け入れており、現在の残高が10 Etherであると仮定します。以下の手順で攻撃が進行します。

- *Attack.sol*、13行目: `EtherStore`コントラクトの`depositFunds`関数が`msg.value`として`1 ether`で呼び出される（大量のガスとともに）。送信者（`msg.sender`）は悪意あるコントラクト（`0x0...123`）である。したがって、`balances[0x0..123] = 1 ether`となる。
- *Attack.sol*、15行目: 悪意あるコントラクトが`EtherStore`コントラクトの`withdrawFunds`関数を呼び出す。過去に引き出しが行われていないため、要件チェック（`EtherStore`コントラクトの11行目）を通過する。
- *EtherStore.sol*、16行目: コントラクトが悪意あるコントラクトに`1 ether`を送り返す。
- *Attack.sol*、23行目: 悪意あるコントラクトへの送金により`receive`関数が実行される。
- *Attack.sol*、24行目: `EtherStore`コントラクトの残高は10 Etherから9 Etherになっているため、この`if`文は条件を満たす。
- *Attack.sol*、26行目: フォールバック関数が`EtherStore`の`withdrawFunds`関数を再度呼び出し、`EtherStore`コントラクトに*再入*する。
- *EtherStore.sol*、10行目: `withdrawFunds`への2回目の呼び出しにおいて、18行目がまだ実行されていないため、攻撃コントラクトの残高は依然として1 Etherである。つまり、`balances[0x0..123] = 1 ether`のままである。`lastWithdrawTime`変数も同様である。再び要件チェックを通過する。
- *EtherStore.sol*、16行目: 攻撃コントラクトがさらに`1 ether`を引き出す。
- *Attack.sol*の24行目で指定されているように、`EtherStore.balance >= 1`でなくなるまで`EtherStore`コントラクトへの再入が繰り返される。
- *Attack.sol*、24行目: `EtherStore`コントラクトに1 Ether未満しか残っていない場合、この`if`文は条件を満たさなくなる。これにより、`EtherStore`コントラクトの17-19行目が（`withdrawFunds`関数の各呼び出しに対して）実行される。
- *EtherStore.sol*、18行目と19行目: `balances`マッピングと`lastWithdrawTime`マッピングが設定され、実行が終了する。

最終的に、攻撃者は1回のトランザクションで`EtherStore`コントラクトからすべてのEtherを引き出すことに成功します。

フォールバック関数で傍受されるネイティブEther送金はリエントランシー攻撃の一般的なベクトルですが、リスクの原因となる仕組みはこれだけではありません。ERC-721やERC-777などの複数のトークン標準には、リエントランシー攻撃を可能にするコールバックメカニズムが含まれています。例えば、ERC-721の`safeTransfer`関数は、コントラクトへのトークン転送時に受信者の`onERC721Received`関数を呼び出すことを保証します。同様に、ERC-777トークンでは転送時に`tokensReceived`関数を通じてフックを起動できます。

古典的なリエントランシーパターンを超えて

リエントランシー攻撃は、単一の関数やコントラクトに限定されません。古典的なリエントランシーは、同じ関数が完了する前に再入するものですが、検出しにくいバリエーションも存在します。クロスファンクションリエントランシー、クロスコントラクトリエントランシー、そして中でも最も発見が困難なリードオンリーリエントランシーなどです。

リードオンリーリエントランシーは、他のコントラクトのview関数に依存するコントラクトを悪用します。view関数は状態を変更しませんが、他のコントラクトが依存するデータを返します。多くの場合、リエントランシー保護は適用されていません。リエントラント呼び出しによって攻撃者がターゲットコントラクトを一時的に不整合な状態にし、別のコントラクト（被害者）がview関数を通じてこの不安定な状態を問い合わせることで問題が発生します。

具体的な流れを見てみましょう。

1. 攻撃者のコントラクトが、再入可能な脆弱なコントラクト（コントラクトAと呼びます）と対話する。このコントラクトは、他のプロトコルが依存するデータを保持している。
2. コントラクトAが攻撃者のコントラクトへのコールバックをトリガーし、攻撃者のコントラクトロジックが実行される。
3. フォールバック内にいる間に、攻撃者のコントラクトが別のプロトコル、コントラクトBを呼び出す。コントラクトBはコントラクトAに接続されており、コントラクトAが提供するデータに依存している。
4. コントラクトBは問題を認識せずにコントラクトAからデータを読み取る。しかし、コントラクトAの状態はまだ更新が完了していないため古いままである。

このサイクルが終了する時点で、攻撃者はすでにコントラクトAの古いデータを利用してコントラクトBを悪用しています。その後、コントラクトAのコールバックと元の呼び出しは通常通り完了します。このプロセスを図9-1に示します。

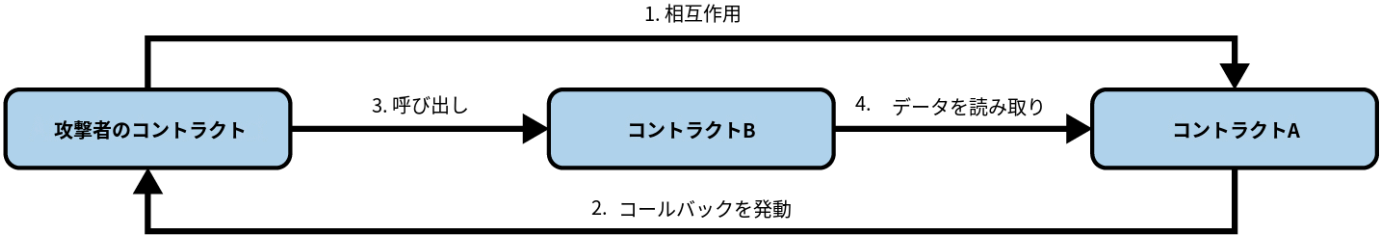


図9-1 リードオンリーリエントランシー

ここでの要点は、コントラクトBがコントラクトAのデータを信頼しているにもかかわらず、コントラクトAの状態がまだ追いついていないということです。攻撃者はこのタイムラグを悪用します。この種の攻撃に対する防御は困難です。開発者はview関数が状態を変更しないため安全だと考え、リエントランシーロックで保護しないことが多いためです。

リードオンリーリエントランシーが教えてくれるのは、リードオンリー関数であっても、外部コントラクトから依存されている場合には危険になりうるということです。

予防テクニック

リエントランシーの問題を防ぐための最初のベストプラクティスは、スマートコントラクトの記述時にcheck-effect-interactionパターンに従うことです。このパターンは、外部コントラクトとの対話の前にすべての状態変数の変更を行うことを保証します。例えば、*EtherStore.sol*コントラクトでは、状態変数を変更する行は外部呼び出しの前に配置されるべきです。外部アドレスと対話するコードが関数内で最後に実行されるようにすることが目標です。必要な更新がすでに完了しているため、再入時に外部コントラクトが内部状態に干渉することを防止できます。

もう一つの有用なテクニックは、リエントランシーロックの適用です。*リエントランシーロック*は、関数の実行中にコントラクトを「ロック」するシンプルな状態変数であり、他の外部呼び出しによる割り込みを防ぎます。以下のようなmodifierで実装できます。

```
contract EtherStore {
    bool lock;
      uint256 public withdrawalLimit = 1 ether;
      mapping(address => uint256) public lastWithdrawTime;
    mapping(address => uint256) balances;

      modifier nonReentrant {
      require(!lock, "Can't reenter");
      lock = true;
      _;
      lock = false;
    }

    function withdrawFunds() public nonReentrant{
        [...]
    }
}
```

この例では、`nonReentrant` modifierがlock変数を使用して、`withdrawFunds`関数の実行中に再入されることを防いでいます。`nonReentrant` modifierは開始時にコントラクトをロックし、関数の完了後にアンロックします。しかし、車輪の再発明をすべきではありません。独自のリエントランシーロックを作成する代わりに、OpenZeppelinのReentrancyGuardのようなよくテストされたライブラリを利用する方が望ましいです。これらのライブラリは安全でガス効率の良いソリューションを提供しています。

注記

Solidity 0.8.24でイーサリアムにトランジエントストレージが導入されたことに伴い、OpenZeppelinはReentrancyGuardTransientを導入しました。これはReentrancyGuardの新しいバリアントであり、トランジエントストレージを活用して大幅に低いガスコストを実現しています。EIP-1153によって有効化されたトランジエントストレージは、単一のトランザクション期間中にのみ必要なデータをより安価に保存する方法を提供し、リエントランシーガードや類似の一時的なロジックに最適です。ただし、ReentrancyGuardTransientはEIP-1153が利用可能なチェーンでのみ使用できるため、実装前にターゲットチェーンがこの機能をサポートしていることを確認してください。

もう一つのアプローチは、Solidityの組み込み関数である`transfer`および`send`を使用してEtherを送信することです。これらの関数は限られた量のガス（2,300ユニット）のみを転送するため、通常は受信コントラクトがリエントラント呼び出しを実行するには不十分です。そのため、リエントランシーに対するシンプルな防御策となります。しかし、注目すべき欠点があります。受信者が、フォールバック関数やreceive関数に悪意のないロジックを持つスマートコントラクトの場合、送金が失敗し資金がロックされる可能性があります。EIP-7702の導入により、このリスクはさらに高まっています。EIP-7702はEOAにフォールバックロジックを含むコードを付与することを可能にします。より多くのEOAがこの機能を採用するにつれ、`transfer`や`send`を使用するトランザクションは、ガス不足によりトランザクションの通常の実行中にリバートする可能性が高くなります。セキュリティの観点からも、このアプローチは将来にわたって安全とは言えません。将来のハードフォークで特定の操作のガスコストが削減された場合、2,300ユニットでも再入に十分となり、それまで安全だった前提が崩れる可能性があります。したがって、`transfer`や`send`は限定的なケースでは依然として有用ですが、慎重に使用し、主要な防御策として依存すべきではありません。

リードオンリーおよびクロスコントラクトリエントランシーには特別な注意が必要です。これらの攻撃は2つの異なるプロトコルが関与する可能性があるため、協調的な予防が課題となります。プロジェクトがデータのために外部プロトコルに依存している場合、統合されたロジックがどのように機能するかを深く調査する必要があります。各プロジェクトが単独では安全であっても、統合時に脆弱性が現れる可能性があります。

実例: DAO攻撃

リエントランシーは、2016年に発生したDAO攻撃において主要な役割を果たしました。この攻撃はイーサリアムの初期開発段階における最大のハッキングの一つでした。当時、コントラクトには1億5,000万ドル以上が保有されており、これはEtherの流通供給量の15%に相当しました。ハッキングの影響を元に戻すために、イーサリアムコミュニティは最終的にイーサリアムブロックチェーンを分割するハードフォークを選択しました。その結果、Ethereum Classic（ETC）がオリジナルのチェーンとして存続し、ハッキングの影響を元に戻すためのルール更新が適用されたフォーク版が、現在知られているイーサリアムとなりました。

実例: Libertify

リエントランシーが唯一の攻撃ベクトルであった最近の事例として、2023年7月に発生したLibertifyの事件があります。DeFiプロトコルであるLibertifyは40万ドルの被害を受けました。悪用された関数のコードを確認し、どのように攻撃が行われたかを見てみましょう。

```
function _deposit(
    uint256 assets,
    address receiver,
    bytes calldata data,
    uint256 nav
) private returns (uint256 shares) {
        /*
        バリデーション
    */
    uint256 returnAmount = 0;
    uint256 swapAmount = 0;
    if (BASIS_POINT_MAX > invariant) {
        swapAmount = assetsToToken1(assets);
        returnAmount = userSwap( // 外部呼び出し
            data,
            address(this),
            swapAmount,
            address(asset),
            address(other)
        );
    }
    uint256 supply = totalSupply(); // 状態の更新
    if (0 < supply) {
        uint256 valueToken0 = getValueInNumeraire(
            asset,
            assets - swapAmount,
            MathUpgradeable.Rounding.Down
        );
        uint256 valueToken1 = getValueInNumeraire(
            other,
            returnAmount,
            MathUpgradeable.Rounding.Down
        );
        shares = supply.mulDiv(
            valueToken0 + valueToken1,
            nav,
            MathUpgradeable.Rounding.Down
        );
    } else {
        shares = INITIAL_SHARE;
    }
    uint256 feeAmount = shares.mulDiv(
        entryFee, BASIS_POINT_MAX, MathUpgradeable.Rounding.Down
```

```
    );
    _mint(receiver, shares - feeAmount);
    _mint(owner(), feeAmount);
}
```

これはリエントランシーの問題の典型的な例です。これほど分かりやすいものは最近ではなかなか見られません。根本的な問題はリエントランシー保護の欠如でした。`userSwap()`関数により、攻撃者は元の呼び出しが`totalSupply`を更新する前に`deposit()`関数に再入することができました。これにより、攻撃者は実際に受け取るべき量よりも多くのシェアをミントし、コントラクトから利益を搾取しました。

DELEGATECALL

`CALL`および`DELEGATECALL`オペコードは、イーサリアムの開発者がコードをモジュール化するために有用です。コントラクトへの標準的な外部メッセージ呼び出しは`CALL`オペコードによって処理され、*呼び出された*コントラクトのコンテクストでコードを実行します。一方、`DELEGATECALL`は別のコントラクトのコードを*呼び出し元*のコントラクトのコンテクストで実行します。つまり、ストレージ、`msg.sender`、`msg.value`はすべて変更されません。`DELEGATECALL`を理解するための分かりやすい考え方は、呼び出し元のコントラクトが呼び出し先のコントラクトのバイトコードを一時的に借用し、自身のコードであるかのように実行するというものです。これにより、再利用可能なロジックを一度デプロイして多くのコントラクトで利用するプロキシコントラクトやライブラリのような強力なパターンが可能になります。これら2つのオペコードの違いはシンプルで直感的ですが、`DELEGATECALL`の使用は、特にストレージレイアウトに関して、微妙で予期しない動作を引き起こす可能性があります。詳細については、ロイ・ルーのこのトピックに関するEthereum Stack Exchangeの質問とSolidityのドキュメントを参照してください。

脆弱性

`DELEGATECALL`のコンテクスト保持特性により、脆弱性のないカスタムライブラリを構築することは想像以上に容易ではありません。ライブラリ自体のコードは安全で脆弱性がなくても、別のアプリケーションのコンテクストで実行された場合に新たな脆弱性が発生する可能性があります。フィボナッチ数を使ったやや複雑な例を見てみましょう。例9-3のライブラリは、フィボナッチ数列および類似の形式の数列を生成できます。（注: このコードは*https://oreil.ly/EHjOb*を元に変更されています。）

例9-3. FibonacciLib: カスタムライブラリの欠陥のある実装

```
1 // ライブラリ コントラクト - フィボナッチ風の数値を計算
2 contract FibonacciLib {
3     // 標準フィボナッチ数列の初期化
4     uint256 public start;
5     uint256 public calculatedFibNumber;
6
```

```
7     // 数列の0番目の数を変更
8     function setStart(uint256 _start) public {
9         start = _start;
10    }
11
12    function setFibonacci(uint256 n) public {
13        calculatedFibNumber = fibonacci(n);
14    }
15
16    function fibonacci(uint256 n) internal view returns (uint) {
17        if (n == 0) return start;
18        else if (n == 1) return start + 1;
19        else return fibonacci(n - 1) + fibonacci(n - 2);
20    }
21 }
```

このライブラリは、数列の*n*番目のフィボナッチ数を生成する関数を提供します。ユーザーが数列の開始数（`start`）を変更し、新しい数列の*n*番目のフィボナッチ風の数を計算できるようにしています。

次に、このライブラリを利用するコントラクトを見てみましょう。

```
contract FibonacciBalance {
    address public fibonacciLibrary;
    // 引き出す現在のフィボナッチ数
    uint256 public calculatedFibNumber;
    // フィボナッチ数列の開始番号
    uint256 public start = 3;
    uint256 public withdrawalCounter;
    // フィボナッチ関数セレクター
    bytes4 constant fibSig = bytes4(keccak256("setFibonacci(uint256)"));
    // コンストラクター - コントラクトにEtherをロード
    constructor(address _fibonacciLibrary) payable {
        fibonacciLibrary = _fibonacciLibrary;
    }
    function withdraw() public {
        withdrawalCounter += 1;
        // 現在の引き出しユーザーのフィボナッチ数を計算
        // calculatedFibNumberを設定する
        (bool success, ) = fibonacciLibrary.delegatecall(
            abi.encodeWithSelector(fibSig, withdrawalCounter)
        );
        require(success, "Delegatecall failed");
        payable(msg.sender).transfer(calculatedFibNumber * 1 ether);
    }
    // ユーザーがフィボナッチ ライブラリ関数を呼び出せるようにする
    fallback() external {
        (bool success, ) = fibonacciLibrary.delegatecall(msg.data);
        require(success, "Delegatecall failed");
```

```
    }
}
```

このコントラクトは、参加者がコントラクトからEtherを引き出すことを可能にします。引き出し量は参加者の引き出し順に対応するフィボナッチ数と等しくなります。つまり、最初の参加者は1 Ether、2番目も1 Ether、3番目は2 Ether、4番目は3 Ether、5番目は5 Etherという具合です（コントラクトの残高が引き出そうとしているフィボナッチ数より少なくなるまで続きます）。

このコントラクトにはいくつか説明が必要な要素があります。まず、興味深い変数`fibSig`があります。これは文字列`"setFibonacci(uint256)"`のKeccak-256ハッシュの最初の4バイトを保持しています。これは*関数セレクター*として知られ、スマートコントラクトのどの関数を呼び出すかを指定するためにcalldataに配置されます。21行目の`delegatecall`関数で、`fibonacci(uint256)`関数を実行したいことを指定するために使用されます。`delegatecall`の2番目の引数は関数に渡すパラメーターです。次に、FibonacciLibライブラリのアドレスがコンストラクターで正しく参照されていると仮定します。

このコントラクトにエラーを見つけられますか？このコントラクトをデプロイし、Etherを入金して`withdraw`を呼び出すと、おそらくリバートするでしょう。

状態変数`start`がライブラリとメインの呼び出し元コントラクトの両方で使用されていることに気づいたかもしれません。ライブラリコントラクトでは、`start`はフィボナッチ数列の開始を指定するために使用され`0`に設定されていますが、呼び出し元コントラクトでは`3`に設定されています。また、`FibonacciBalance`コントラクトのフォールバック関数がすべての呼び出しをライブラリコントラクトに渡すことにも気づいたかもしれません。これにより、ライブラリコントラクトの`setStart`関数を呼び出すことが可能になります。コントラクトの状態が保持されることを思い出すと、この関数がローカルの`FibonacciBalance`コントラクトの`start`変数の状態を変更できるように思えるかもしれません。もしそうであれば、`calculatedFibNumber`が`start`変数に依存しているため（ライブラリコントラクトで確認できる通り）、より多くのEtherを引き出すことが可能になります。しかし実際には、`setStart`関数は`FibonacciBalance`コントラクトの`start`変数を変更しません（変更できません）。このコントラクトの根本的な脆弱性は、`start`変数の変更よりもはるかに深刻です。

実際の問題を議論する前に、状態変数がコントラクト内でどのように保存されるかを理解するために少し寄り道しましょう。*状態変数*または*ストレージ変数*（個々のトランザクションを超えて永続する変数）は、コントラクト内で導入された順に*スロット*に配置されます。（ここにはいくつかの複雑な点があります。より詳しい理解についてはSolidityのドキュメントを参照してください。）

例として、ライブラリコントラクトを見てみましょう。このコントラクトには`start`と`calculatedFibNumber`の2つの状態変数があります。最初の変数`start`は、コントラクトのストレージの`slot[0]`（最初のスロット）に保存されます。2番目の変数`calculatedFibNumber`は、次に利用可能なストレージスロットである`slot[1]`に配置されます。`setStart`関数は入力を受け取り、`start`をその入力値に設定します。したがって、この関数は`setStart`関数に提供した入力値で

`slot[0]`を設定します。同様に、`setFibonacci`関数は`calculatedFibNumber`を`fibonacci(n)`の結果に設定します。これも単にストレージの`slot[1]`を`fibonacci(n)`の値に設定しているだけです。

次に、`FibonacciBalance`コントラクトを見てみましょう。ストレージの`slot[0]`は`fibonacciLibrary`アドレスに対応し、`slot[1]`は`calculatedFibNumber`に対応します。この不正確なマッピングに脆弱性が存在します。`delegatecall`は*コントラクトのコンテクストを保持*します。つまり、`delegatecall`を介して実行されるコードは、呼び出し元コントラクトの状態（すなわちストレージ）に対して作用します。

`withdraw`の21行目で`fibonacciLibrary.delegatecall(fibSig,withdrawalCounter)`を実行していることに注目してください。これは`setFibonacci`関数を呼び出し、先ほど説明した通りストレージの`slot[1]`を変更します。現在のコンテクストでは`calculatedFibNumber`に相当します。これは期待通りの動作です（実行後、`calculatedFibNumber`が変更されます）。しかし、`FibonacciLib`コントラクトの`start`変数がストレージの`slot[0]`に位置していることを思い出してください。現在のコントラクトでは`fibonacciLibrary`アドレスに相当します。つまり、`fibonacci`関数は予期しない結果を返します。`start`（`slot[0]`）を参照しますが、現在の呼び出しコンテクストではそれは`fibonacciLibrary`アドレスであり、`uint`として解釈するとかなり大きな値になります。したがって、`withdraw`関数はリバートする可能性が高いです。`calculatedFibNumber`が返す`uint(fibonacciLibrary)`の量のEtherをコントラクトが保持していないためです。

さらに悪いことに、`FibonacciBalance`コントラクトでは、ユーザーが27行目のフォールバック関数を通じてすべての`fibonacciLibrary`関数を呼び出すことができます。先ほど説明した通り、これには`setStart`関数も含まれます。この関数が誰でもストレージの`slot[0]`を変更または設定できることを説明しました。この場合、ストレージの`slot[0]`は`fibonacciLibrary`アドレスです。したがって、攻撃者は悪意あるコントラクトを作成し、そのアドレスを`uint256`に変換して（Pythonで`int('<address>',16)`を使えば簡単にできます）、`setStart(<attack_contract_address_as_uint>)`を呼び出すことができます。これにより`fibonacciLibrary`が攻撃コントラクトのアドレスに変更されます。その後、ユーザーが`withdraw`やフォールバック関数を呼び出すたびに、悪意あるコントラクトが実行されます（コントラクトの全残高を盗むことができます）。`fibonacciLibrary`の実際のアドレスが変更されたためです。このような攻撃コントラクトの例は以下の通りです。

```
contract Attack {
    uint256 private storageSlot0; // fibonacciLibraryに対応
    uint256 private storageSlot1; // calculatedFibNumberに対応
    // フォールバック - 指定された関数が見つからない場合に実行される
    fallback() external {
        storageSlot1 = 0; // calculatedFibNumberを0に設定し、withdrawが
        // 呼ばれてもEtherを送信しないようにする
        payable(<attacker_address>).transfer(this.balance); // すべてのEtherを奪取
    }
}
```

この攻撃コントラクトはストレージの`slot[1]`を変更することで`calculatedFibNumber`を変更しています。原理的には、攻撃者は任意のストレージスロットを変更して、このコントラクトに対してあらゆる種類の攻撃を実行できます。これらのコントラクトをRemixに入力し、さまざまな攻撃コントラクトや`delegatecall`関数を通じた状態変更を試してみることをお勧めします。

`delegatecall`が状態を保持するといった場合、コントラクトの変数名ではなく、変数名が指し示す実際のストレージスロットについて述べていることに注意することも重要です。この例からわかるように、単純なミスが攻撃者によるコントラクト全体とそのEtherの乗っ取りにつながる可能性があります。

予防テクニック

Solidityはライブラリコントラクトを実装するための`library`キーワードを提供しています（詳細はドキュメントを参照）。これにより、ライブラリコントラクトがステートレスで自己破壊不可能であることが保証されます。ライブラリをステートレスに強制することで、このセクションで示したストレージコンテクストの複雑さが軽減されます。ステートレスなライブラリは、攻撃者がライブラリの状態を直接変更してそのコードに依存するコントラクトに影響を与える攻撃も防ぎます。一般的な経験則として、`DELEGATECALL`を使用する場合は、ライブラリコントラクトと呼び出し元コントラクトの両方の呼び出しコンテクストに細心の注意を払い、可能な限りステートレスなライブラリを構築してください。

実例: Parityマルチシグウォレット（2回目のハッキング）

2回目のParityマルチシグウォレットのハッキングは、よく書かれたライブラリコードであっても、意図されたコンテクスト外で実行された場合にどのように悪用されうるかを示す例です。このハッキングについては、「Parity Multisig Hacked. Again」など、多くの優れた解説があります。これらの参考資料に加えて、悪用されたコントラクトを見てみましょう。

この攻撃は7年前のものであるため、以下のコードスニペットは最新のSolidityバージョンの構文に更新されており、より読みやすく理解しやすくなっています。

ライブラリコントラクトは以下の通りです。

```
1 contract WalletLibrary is WalletEvents {
2
3   ...
4
5   // コントラクトがまだ初期化されていない場合にのみ通過
6   modifier only_uninitialized { if (m_numOwners > 0) revert(); _; }
7
8   // コンストラクター - オーナー配列をmultiownedに、
9   // 制限をdaylimitに渡す
10   function initWallet(address[] memory _owners, uint256 _required, uint256
11         _daylimit) public only_uninitialized {
12     initDaylimit(_daylimit);
```

```
13     initMultiowned(_owners, _required);
14   }
15
16   // コントラクトを破壊し、すべてを`_to`に送信
17   function kill(address _to) onlymanyowners(keccak256(msg.data)) external {
18     selfdestruct(_to);
19   }
20
21   ...
22
23 }
```

ウォレットコントラクトは以下の通りです。

```
1 contract Wallet is WalletEvents {
2
3   ...
4
5   // メソッド
6
7   // 他の関数がマッチしない場合に呼び出される
8   fallback() external payable {
9     // 単にキャッシュが送られただけか？
10     if (msg.value > 0)
11       Deposit(msg.sender, msg.value);
12     else if (msg.data.length > 0)
13       _walletLibrary.delegatecall(msg.data);
14   }
15
16   ...
17
18   // フィールド
19   address constant _walletLibrary =
20     0xcafecafecafecafecafecafecafecafecafecafe;
21 }
```

`Wallet`コントラクトは基本的にすべての呼び出しをデリゲートコールを介して`WalletLibrary`コントラクトに渡していることに注目してください。このコードスニペットの定数`_walletLibrary`アドレスは、実際にデプロイされた`WalletLibrary`コントラクト（`0x863DF6BFa4469f3ead0bE8f9F2AAE51c91A907b4`にあったもの）のプレースホルダーとして機能しています。

これらのコントラクトの意図された動作は、コードベースと主要機能が`WalletLibrary`コントラクトにある、シンプルで低コストのデプロイ可能な`Wallet`コントラクトを持つことでした。残念ながら、`WalletLibrary`コントラクト自体がコントラクトであり、独自の状態を保持しています。なぜこれが問題になりうるか分かりますか？

`WalletLibrary`コントラクト自体に呼び出しを送信することが可能でした。具体的には、`WalletLibrary`コントラクトを初期化し、オーナーになることができました。実際に、あるユーザーが`WalletLibrary`コントラクトの`initWallet`関数を呼び出し、ライブラリコントラクトのオーナーになりました。同じユーザーがその後`kill`関数を呼び出しました。ユーザーがライブラリコントラクトのオーナーであったためmodifierを通過し、ライブラリコントラクトは自己破壊しました。現存するすべての`Wallet`コントラクトがこのライブラリコントラクトを参照しており、この参照を変更する方法を持たないため、Etherの引き出し機能を含むすべての機能が`WalletLibrary`コントラクトとともに失われました。その結果、このタイプのすべてのParityマルチシグウォレットのすべてのEtherが即座に失われるか、永久に回復不能となりました。

> **注記**
>
> 攻撃者はその後GitHubに現れ、「うっかり壊してしまいました」という印象的なコメントを残しました。彼はイーサリアムの初心者で、スマートコントラクトを試していただけだと主張しました。

エントロピーの幻想

イーサリアムブロックチェーン上のすべてのトランザクションは*決定論的状態遷移*操作です。つまり、すべてのトランザクションは不確実性なしに計算可能な方法でイーサリアムエコシステムのグローバル状態を変更します。この性質は、イーサリアムにはエントロピーやランダム性の源泉が存在しないという根本的な含意を持ちます。初期の頃、分散化された方法でランダム性を生成することは大きな課題でした。しかし年月を経て、この問題を解決するためのいくつかの確実なソリューションが開発されてきました。

脆弱性

開発者がイーサリアム上でスマートコントラクトを構築する際、ゲーム、宝くじ、その他の予測不可能性を必要とする機能のために、ランダム性の源泉がしばしば必要になります。課題は、イーサリアムがブロックチェーンとして本質的に決定論的であり、コンセンサスを維持するためにすべてのノードが同じ結果に到達しなければならないことです。そのため、真のランダム性を導入するにはある程度の創意工夫が必要になります。

多くの開発者が頼ってきたアプローチの一つは、ブロック変数（ブロックハッシュ、タイムスタンプ、ブロック番号など）をシードとして乱数を生成することです。これらの値はランダムに見えるかもしれませんが、実際には現在のブロックを提案するバリデータによって制御されています。例えば、ゲームの結果が次のブロックハッシュの末尾が偶数かどうかに基づくDAppを想像してください。バリデータはこのプロセスを操作できます。ブロックを提案しようとしている際にハッシュが望む結果に合わない場合、例えばトランザクションの順序を変更して、有利な方向にブロックハッシュを変えることが可能です。

ブロック変数からランダム性を導出する場合、バリデータの操作だけがリスクではありません。他のスマートコントラクトもこれらのブロック変数の値を知ることができるため、結果が有利な場合にのみ脆弱なコントラクトと対話することが可能です。

予防テクニック

過去と比較して、イーサリアムの開発者はランダム性を生成するための信頼性の高い方法を持つようになりました。`PREVRANDAO`と*検証可能なランダム関数*（VRF）です。

VRFはランダム性が公正で偏りがないことを保証する暗号学的証明です。VRFはChainlinkなど複数のプロバイダーによってサポートされています。VRFは乱数とともにその公正性を検証する証明を生成します。この証明は誰でも検証可能であり、ランダム性が安全であることを保証します。VRFはスマートコントラクトでランダム性を安全に取得するための標準的な分散型ソリューションとなっています。

もう一つの確実な選択肢は、PoSへの移行とともにイーサリアムに導入された`PREVRANDAO`オペコードです。このオペコードは`PREVRANDAO`値を取得するために使用され、PoSブロック生成の不可欠な要素であるRandaoプロセスに由来します。本質的に、Randaoはバリデータがそれぞれデータの一部を提供することでランダム性を生成する共同作業です。`PREVRANDAO`は前のブロックからのこのプロセスの結果であり、信頼性の高いランダム性の源泉として機能します。`PREVRANDAO`値を操作するには相当数のバリデータを侵害する必要があるため信頼性が高く、そのような悪用は非現実的で経済的に不可能です。開発者はコントラクトでこの値を使用できますが、`PREVRANDAO`は前のブロックの値を表し、すでに既知であることに留意する必要があります。コミットメント時にこの値が予測可能にならないようにするため、スマートコントラクトは将来のブロックの`PREVRANDAO`値にコミットすべきです。こうすることで、コミットメント時には値が不明となります。

警告

攻撃者がエポックの最終スロットに割り当てられたプロポーザーの制御権を得た場合、Randaoは操作される可能性があります。スマートコントラクトでランダム性を生成するために`PREVRANDAO`が信頼できる選択かどうかを判断するには、操作のコストと利益を慎重に比較検討する必要があります。Randaoの改ざんは高コストになりえますが、コントラクトが高価値の資産を扱う場合は、代わりに分散型オラクルソリューションを使用する方が安全です。

`PREVRANDAO`やVRFのようなソリューションが広く文書化されアクセス可能になったことで、開発者が安全でないブロック変数をランダム性の源泉として使用することは最近では珍しくなっています。しかし、近道が選ばれたり、開発者がこれらのツールを知らない場合には、依然としてミスが発生します。

実例: Fomo3D

Fomo3Dはイーサリアムの宝くじゲームで、プレイヤーが「キー」を購入してタイマーを延長し、タイマーがゼロになった時点で最後の購入者となって賞金プールを獲得することを競いました。以下のコードに示すように、ランダム性の低いエアドロップ機能が含まれていました。

```
function airdrop()
        private
        view
        returns(bool)
    {
        uint256 seed = uint256(keccak256(abi.encodePacked(

            (block.timestamp).add
            (block.difficulty).add
            ((uint256(keccak256(abi.encodePacked
            (block.coinbase)))) / (block.timestamp)).add
            (block.gaslimit).add
            ((uint256(keccak256(abi.encodePacked
            (msg.sender)))) / (block.timestamp)).add
            (block.number)
        )));
        if((seed - ((seed / 1000) * 1000)) < airDropTracker_) {
            return(true);
        } else {
            return(false);
        }
    }
```

悪意あるコントラクトは、シードの計算に使用される値を事前に知ることができるため、勝利につながる場合にのみ`airdrop`関数をトリガーすることが可能でした。このコントラクトが悪用されたのは当然のことです。

未チェックのCALL戻り値

Solidityで外部呼び出しを行う方法はいくつかあります。外部アカウントへのEther送信は一般的に`transfer`メソッドを介して実行されます。しかし、`send`関数も使用でき、より汎用的な外部呼び出しには`CALL`オペコードをSolidityで直接使用することもできます。`call`と`send`関数は、呼び出しが成功したか失敗したかを示すBooleanを返します。つまり、これらの関数には単純ですが重要な注意点があります。外部呼び出し（`call`または`send`で初期化されたもの）が失敗しても、トランザクション自体はリバートしません。関数は単に`false`を返すだけです。開発者が外部呼び出しの失敗時にリバートが発生すると思い込み、戻り値をチェックしないことがよくあるミスです。

脆弱性

例9-4のコントラクトを考えてみましょう。

例9-4. 脆弱なLottoコントラクト

```
1 contract Lotto {
2
3     bool public payedOut;
4     address public winner;
5     uint256 public winAmount;
6
7     // ... ここに追加機能
8
9     function sendToWinner() public {
10         require(!payedOut);
11         payable(winner).send(winAmount);
12         payedOut = true;
13     }
14
15     function withdrawLeftOver() public {
16         require(payedOut);
17         payable(msg.sender).send(address(this).balance);
18     }
19 }
```

これはLotto風のコントラクトで、`winner`が`winAmount`のEtherを受け取り、通常は誰でも引き出せるわずかな残高が残ります。脆弱性は11行目に存在し、レスポンスをチェックせずに`send`が使用されています。この簡単な例では、トランザクションが失敗する`winner`（ガス不足やフォールバック関数で意図的にスローするコントラクトなど）の場合、Etherが送信されたかどうかに関係なく`payedOut`が`true`に設定されます。この場合、誰でも`withdrawLeftOver`関数を通じて`winner`の賞金を引き出すことができます。

予防テクニック

最初の防御線は、例外なく`send`関数と低レベル呼び出しの戻り値を常にチェックすることです。現在では、どの静的解析ツールもこの問題をフラグ付けするため、見落としにくくなっています。

Etherを送信する際、どの方法を使用するか慎重に検討する必要があります。失敗時にトランザクションを自動的にリバートさせたい場合、`transfer`はデフォルトで失敗を処理するため魅力的に見えるかもしれません。しかし、`send`と`transfer`はどちらも2,300ガスユニットしか転送しないため、受信者（コントラクト、またはEIP-7702によりEOAでも）にフォールバックロジックがある場合、容易に失敗する可能性があります。この進化するコンテクストを踏まえると、より安全で柔軟なアプローチは代わりに`call`を使用し、戻り値を明示的にチェックし、エラーを

適切に管理することです。これにより、ガス転送を完全に制御でき、より幅広い受信者とコントラクトの互換性を保つことができます。

実例: EtherpotとKing of the Ether

Etherpotは例9-4のコントラクトとさほど変わらないスマートコントラクトの宝くじでした。このコントラクトの失敗の主な原因は、ブロックハッシュの不正な使用でした（直近256ブロックのハッシュのみ使用可能。「定義済みグローバル変数と関数」を参照）。しかし、このコントラクトは未チェックの`call`値の問題も抱えていました。

例9-5の`cash`関数を見てみましょう。ここでも以下のコードスニペットは最新のSolidityバージョンの構文に更新されています。

例9-5. Lotto.sol: コードスニペット

```
1 function cash(uint256 roundIndex, uint256 subpotIndex) public {
2    uint256 subpotsCount = getSubpotsCount(roundIndex);
3    if(subpotIndex>=subpotsCount)
4        return;
5    uint256 decisionBlockNumber = getDecisionBlockNumber(roundIndex,subpotIndex);
6    if(decisionBlockNumber>block.number)
7        return;
8    if(rounds[roundIndex].isCashed[subpotIndex])
9        return;
10    //サブポットは一度しか換金できない。二重支払いを防ぐため
11    address winner = calculateWinner(roundIndex,subpotIndex);
12    uint256 subpot = getSubpot(roundIndex);
13    payable(winner).send(subpot);
14    rounds[roundIndex].isCashed[subpotIndex] = true;
15    //ラウンドを換金済みとしてマーク
16 }
```

13行目で`send`関数の戻り値がチェックされておらず、次の行で当選者に資金が送信されたことを示すBooleanが設定されていることに注目してください。このバグにより、当選者がEtherを受け取っていないにもかかわらず、コントラクトの状態では当選者がすでに支払い済みであることを示す状況が発生しえます。

このバグのより深刻なバージョンがKing of the Etherコントラクトで発生しました。未チェックの失敗した`send`がどのようにコントラクトを攻撃するために使用されうるかを詳述した優れた事後分析が書かれています。

ERC-20のケース

SolidityでERC-20トークンを扱う場合、トークン転送の戻り値を単にチェックするだけでは安全なやり取りを保証するには不十分です。すべてのERC-20トークンがERC-20標準に厳密に従っ

ているわけではなく、特に古いトークンではその傾向が強いためです。一部のトークンは、操作が失敗した際に直接リバートや例外をスローするのではなく、転送の完了時にBoolean値を返します。また、値を一切返さないトークンもあり、標準的なメソッドでやり取りする際に曖昧な動作が生じます。Tether（USDT）は、ERC-20標準に完全には準拠していない広く普及したトークンの代表例です。

これを軽減するために、OpenZeppelinのSafeERC20のようなライブラリを使用します。このライブラリは標準的なERC-20操作（`transfer`、`transferFrom`、`approve`など）をラップし、これらのバリエーションを適切に処理します。トークンが`false`を返した場合、ライブラリはトランザクションを確実にリバートします。トークンが値を返さない場合、リバートが発生しなければ操作が成功したとみなします。

レースコンディションとフロントランニング

この脆弱性を本当に理解するために、イーサリアムでトランザクションがどのように機能するかを簡単に振り返りましょう。トランザクションを送信すると、それはノードのネットワークにブロードキャストされ、保留中のトランザクションの待機場所であるmempoolに配置されます。バリデータはmempoolからこれらのトランザクションを取り出してブロックを構築します。ブロック内のトランザクションは特定の順序で順次実行され、各トランザクションがブロックチェーンのグローバル状態を変更するため、トランザクションの結果はブロック内の位置によって変わりえます。このトランザクションの順序付けは、トランザクション実行の結果に大きく影響しうるため重要です。

> **注記**
>
> 実際には、ブロック内のトランザクションの位置を制御することは、主に支払いに帰着します。当初は、より高いガス価格を提示するだけで順序に影響を与えることができました。現在では、ビルダー-プロポーザー分離を実装するFlashbotsインフラストラクチャのおかげで（まだイーサリアムにネイティブには組み込まれていませんが）、ユーザーは特定の順序でトランザクションのバンドルを送信し、オフチェーンリレーシステムを通じてその組み込みに入札できます。これらのプロセス（レガシーのmempoolベースのシステムと新しいビルダーベースのシステムの両方）については、第6章でより詳しく説明しています。

脆弱性

*フロントランニング*とは、この順次実行を悪用して、フロントランナーに有利な形でブロックに他のトランザクションを挿入する行為です。具体的には、市場や特定のコントラクトに影響を与える可能性のある保留中のトランザクションを監視し、元のトランザクションの前に処理されるように自分自身のトランザクションを送信します。保留中のトランザクションの情報を利用して利益を得ることができ、多くの場合、元の送信者に不利益をもたらします。コードがこのダイ

ナミクスを考慮し、ブロック内のトランザクション順序の変更に対して耐性を持つように設計されていることが重要です。

シンプルな例でこれがどのように機能するかを見てみましょう。例9-6のコントラクトを考えてみましょう。

例9-6. FindThisHash: フロントランニングに対して脆弱なコントラクト

```
contract FindThisHash {
    bytes32 constant public hash =
      0xb5b5b97fafd9855eec9b41f74dfb6c38f5951141f9a3ecd7f44d5479b630ee0a;
    constructor() payable {} // Etherをロード
    function solve(string memory solution) public {
        // ハッシュの原像を見つけることができれば、1000 Etherを受け取る
        require(hash == keccak256(abi.encodePacked(solution)));
        payable(msg.sender).transfer(1000 ether);
    }
}
```

このコントラクトが1,000 Etherを持っているとしましょう。SHA-3ハッシュ`0xb5b5b97fafd9855eec9b41f74dfb6c38f5951141f9a3ecd7f44d5479b630ee0a`の原像を見つけたユーザーは、解答を送信して1,000 Etherを取得できます。あるユーザーが解答が`Ethereum!`であることを突き止めたとしましょう。パラメーターとして`Ethereum!`を指定して`solve`を呼び出します。残念ながら、攻撃者がmempool内のトランザクションを発見し、その有効性を確認した上で、ブロック内でより高い優先度を持つ同等のトランザクションを送信しました。攻撃者のトランザクションが先に処理されるため、元のトランザクションはリバートします。

予防テクニック

フロントランニングの脆弱性はさまざまな形で現れ、スマートコントラクトやプロトコルの特定のロジックに依存することが多いです。トランザクションの順序付けによって操作が悪用される可能性がある場合、フロントランニングの脆弱性が存在します。したがって、ソリューションは通常、特定の問題に合わせて調整されます。例えば、自動マーケットメーカー（AMM）プロトコルは、スワップ中に受け取るトークンの最小数をユーザーが設定できるようにすることでこの問題に対処しています。これはフロントランニングを完全に防ぐものではありませんが、攻撃者が抽出できる潜在的な利益を大幅に制限し、被害を軽減してユーザーを極端なスリッページから保護します。

もう一つの一般的なテクニックは、コミット-リビールスキームの使用です。このアプローチでは、ユーザーはまず隠された情報を含むトランザクションを送信します。通常はハッシュとして表現されます（コミットフェーズ）。このトランザクションがブロックに含まれた後、ユーザーは実際のデータを公開する2番目のトランザクションを送信します（リビールフェーズ）。攻撃者は最初のトランザクションの詳細を手遅れになるまで見ることができないため、この方法はフロントランニングを効果的に防ぎます。ただし、トレードオフとして2つの別々のトランザクションが必要となり、より高いコストと追加の遅延が発生します。ユーザーエクスペリエンスの

低下に加えて、トランザクション間の必要な遅延は、時間に敏感なアプリケーションにおいて実用上の制約となりえます。

実例: AMMとminAmountOut

一般的な実世界のフロントランニング脆弱性を見てみましょう。AMMプロトコルを統合したスマートコントラクトが、受け取るトークンの最小数を設定せずにスワップを実行する場合に発生し、そのスワップをフロントランニング攻撃に対して脆弱にします。受け取るトークンの最小数が適切に設定されていない場合（または低すぎる場合）、スワップトランザクションはサンドイッチ攻撃（フロントランニングの特定のタイプ）に対して脆弱になります。

サンドイッチ攻撃の展開は以下の通りです。フロントランナーがmempool内の保留中のトランザクションを監視し、最小出力量を強制していないスワップトランザクションを発見します。攻撃者はスワップの直前に買いトランザクションを送信し、トークン価格を人為的に引き上げます。スワップトランザクションはこの膨らんだ価格で処理され、期待よりも少ないトークンしか受け取れません。さらに、スワップトランザクション自体が価格をさらに引き上げます。その直後に、攻撃者はこの高い価格でトークンを売却し、価格を元に戻して、スワップトランザクションが生み出した価格差から利益を得ます。この戦略はスワップトランザクションを2つの取引で「挟む」ため、*サンドイッチ攻撃*と呼ばれています。この問題を修正するには、AMMを統合するスマートコントラクトがオラクルのような信頼できるソースから実際の資産価格を取得し（時間加重平均価格ベースでも可）、スワップ実行時に正確な最小出力量を計算して強制する必要があります。

サービス拒否

このカテゴリーは非常に広範ですが、根本的には、コントラクトまたはコントラクトの一部を一定期間、場合によっては永続的に使用不能にする攻撃で構成されます。「実例: Parityマルチシグウォレット（2回目のハッキング）」で説明したケースのように、コントラクト内の資金が永久にロックされる可能性もあります。

脆弱性

コントラクトが使用不能になる方法はさまざまです。ここでは、DoS脆弱性につながるあまり明白ではないSolidityのコーディングパターンをいくつか紹介します。

外部から操作可能なマッピングまたは配列のループ処理

このパターンは通常、オーナーが例9-7のコントラクトのように`distribute`のような関数で投資家にトークンを分配したい場合に現れます。

例9-7. DistributeTokensコントラクト

```
1 contract DistributeTokens {
2     address public owner; // どこかで設定される
3     address[] investors; // 投資家の配列
4     uint[] investorTokens; // 各投資家が受け取るトークン量
5 
6     // ... transfertoken()を含む追加機能
7 
8     function invest() public payable {
9         investors.push(msg.sender);
10         investorTokens.push(msg.value * 5); // 送信されたweiの5倍
11         }
12 
13     function distribute() public {
14         require(msg.sender == owner); // オーナーのみ
15         for(uint256 i = 0; i < investors.length; i++) {
16             // ここでtransferToken(to,amount)は"amount"分の
17             // トークンをアドレス"to"に転送する
18             transferToken(investors[i],investorTokens[i]);
19         }
20     }
21 }
```

このコントラクトのループは人為的に膨張させることができる配列を反復処理していることに注目してください。攻撃者は多数のユーザーアカウントを作成し、`investors`配列を非常に大きくすることができます。リスクはループ自体だけでなく、`transferToken`やその他のロジックなど、ループ内の操作の累積ガスコストにもあります。追加のイテレーションごとに使用される総ガスが増加し、配列が十分に大きくなると、ループを完了するために必要なガスがブロックガスリミットを超える可能性があります。その時点で、`distribute`関数は事実上使用不能になります。

注記

この種のDoSは状態を変更する関数に限定されません。読み取り専用のview関数でさえ、大きな配列をループ処理する場合はアクセス不能になる可能性があります。オンチェーンではガスを消費しませんが、RPCエンドポイントは`eth_call`の実行に対して独自の任意のガス上限を適用します。そのため、view関数がこれらの制限を超えるほどのロジックを実行する場合、RPC呼び出しは失敗します。

外部呼び出しに基づく状態の進行

コントラクトは、新しい状態に進むためにアドレスへのEther送信や外部ソースからの入力待ちが必要なように書かれることがあります。これらのパターンは、外部呼び出しが失敗したり外部の理由で阻止されたりした場合にDoSにつながる可能性があります。Ether送信の例では、ユーザーがEtherを受け付けないコントラクトを作成することができます。コントラクトが新しい

状態に進むためにEtherの送信を必要とする場合、Etherを受け付けないユーザーのコントラクトにはEtherを送信できないため、コントラクトは新しい状態を達成できません。

予期しない問題

DoSの問題は予期しない形で発生する可能性があり、常に悪意ある攻撃が関与するとは限りません。時として、予期せぬ出来事だけでコントラクトの機能が中断されることがあります。例えば、スマートコントラクトがオーナーの秘密鍵に依存して特定の特権関数を呼び出す場合、その鍵が紛失または侵害されると問題が生じます。その鍵がなければ、重要な関数は永久にアクセス不能となり、コントラクト全体の運用が停止する可能性があります。オーナーがセールを最終化する関数を呼び出す必要があるICO（イニシャルコインオファリング）コントラクトを想像してください。鍵が紛失した場合、誰もそれを呼び出すことができず、トークンは永久にロックされたままになります。

予期しない中断のもう一つの例は、コントラクトが認識も意図もしないままEtherが送信される場合です。`selfdestruct`（現在は非推奨）と呼ばれるメソッドを使用するか、コントラクトがデプロイされる前にあらかじめ決まったアドレスにEtherを送信することで、Etherをコントラクトに「強制的に」送り込むことができます。コントラクトが自身の関数を通じて受け取るすべてのEtherの会計を制御していると想定している場合、これらの招かれざる資金にどう対処すべきかわからず、意図しない動作が生じる可能性があります。銀行口座に予期しないお金が入っているようなものです。嬉しい場合もありますが、口座残高がずれることも意味し、その正確な数値に依存するシステムが誤作動し始める可能性があります。

予防テクニック

DoSの問題はさまざまな形で現れるため、ソリューションも通常は状況に応じたものとなります。

ブロックガスリミットに達するリスクのある長いリストはかなり一般的な状況であるため、対処法をいくつか提案できます。最初の例では、コントラクトは外部ユーザーによって人為的に操作可能なデータ構造をループ処理すべきではありません。各投資家が`withdraw`関数を呼び出して独立してトークンを請求する引き出しパターン（プルオーバープッシュパターン）が推奨されます。長いリストを反復処理する関数には、ページネーション機能の実装が良いソリューションです。

DoSに対する一般的なソリューションは、何が問題になりうるかをできる限り調査し、セーフガードを実装することです。

実例: ZKsync Era Gemholicの資金ロック

先ほど述べた通り、最も予期しないエラーがスマートコントラクトのDoS問題につながる可能性があります。最近の例として、イーサリアムのL2ソリューションであるZKsync Era上にスマートコントラクトをデプロイしたプロジェクトGemholicがあります。Gemholicはトークンセール

で調達した921 ETH（約170万ドル）にアクセスできないという重大な問題に直面しました。根本原因は、スマートコントラクトがZKsync Eraではサポートされていない`transfer()`関数に依存していたことです。ZKsync EraはEVM機能の多くと互換性がありますが、完全なEVM等価ではないため、`transfer()`のような一部の機能はイーサリアムメインネットと同じようには動作しません。この非互換性により、スマートコントラクトが意図通りにEtherを引き出すことができず、Gemholicの資金がロックされました。幸いにも、ZKsyncのチームが介入し、資金のロックを解除する「エレガントなソリューション」を開発し、Gemholicが再びアクセスできるようにしました。残念ながら、この「エレガントなソリューション」の詳細は公開されていません。

浮動小数点と精度

本書執筆時点で、Solidity v0.8.29は固定小数点数と浮動小数点数を完全にはサポートしていません。この設計上の選択は、ブロックチェーンの決定論性に対する根本的な要件に由来しています。ネットワーク内のすべてのノードが同じ入力から同一の結果に到達し、コンセンサスを維持する必要があるためです。残念ながら、浮動小数点演算は異なるハードウェアアーキテクチャ間で本質的に非決定論的であり、同一の計算からわずかに異なる結果を生じる可能性があります。

ブロックチェーンアプリケーションはネットワークのフォークを防ぎセキュリティを維持するために絶対的な決定論性を必要とするため、Solidityは開発者に整数型を使用した浮動小数点表現の実装を求めています。正しく実装しないとより煩雑でエラーが発生しやすくなりますが、金融計算とスマートコントラクトロジックがネットワーク内のすべてのノードで同一の結果を生成することを保証できます。

脆弱性

固定小数点数はまだSolidityによって完全にはサポートされていません。宣言はできますが、代入の対象や代入元にはできません。つまり、開発者は標準的な整数データ型を使用して独自に実装する必要があります。このプロセスで開発者が陥りうる落とし穴がいくつかあります。このセクションではそのいくつかを紹介します。コード例（例9-8）から始めましょう。

例9-8. FunWithNumbers

```
1 contract FunWithNumbers {
2    uint256 constant public tokensPerEth = 10;
3    uint256 constant public weiPerEth = 1e18;
4    mapping(address => uint) public balances;
5
6    function buyTokens() public payable {
7        // weiをethに変換し、トークン レートを掛ける
8        uint256 tokens = msg.value/weiPerEth*tokensPerEth;
9        balances[msg.sender] += tokens;
10    }
11
```

```
12    function sellTokens(uint256 tokens) public {
13        require(balances[msg.sender] >= tokens);
14        uint256 eth = tokens/tokensPerEth;
15        balances[msg.sender] -= tokens;
16        payable(msg.sender).transfer(eth*weiPerEth);
17    }
18 }
```

このシンプルなトークンの売買コントラクトにはいくつかの明白な問題があります。トークンの売買に関する数学的計算は正しいものの、浮動小数点数の欠如により誤った結果が生じます。例えば、8行目でトークンを購入する際、値が1 Ether未満の場合、最初の除算の結果は0となり、最終的な乗算の結果も`0`になります（例: 200 weiを`1e18` `weiPerEth`で割ると0になります）。同様に、トークンを売却する際、10未満のトークン数は0 Etherとなります。実際、ここでの丸めは常に切り捨てであるため、29トークンの売却は2 Etherとなります（29トークン / 10 `tokensPerEth` = 2.9、切り捨てると2になります）。

このコントラクトの問題は、精度が最も近いEther（すなわち1e18 wei）単位でしかないことです。より高い精度が必要な場合、ERC-20トークンのデシマルを扱う際にこれが厄介になります。実際のケースでは、精度の損失は小さく見えるかもしれませんが、容易に増幅され悪用される可能性があります。フラッシュローンは、例えば、攻撃者が前払いコストなしで大量の資本を借り入れることを可能にし、わずかな不整合さえも悪用可能にします。

予防テクニック

スマートコントラクトで適切な精度を保つことは非常に重要であり、特に経済的な意思決定を反映する比率やレートを扱う場合はなおさらです。使用する比率やレートが分数の大きな分子を許容するようにすべきです。例えば、この例では`tokensPerEth`というレートを使用しました。大きな数値となる`weiPerTokens`を使用する方が良かったでしょう。対応するトークン数を計算するには、`msg.value/weiPerTokens`とすればよく、より精度の高い結果が得られます。

もう一つの戦術は、演算の順序に注意することです。この例では、トークン購入の計算は`msg.value/weiPerEth*tokenPerEth`でした。除算が乗算の前に行われていることに注目してください。Solidityは一部の言語とは異なり、記述された順序で演算を実行することを保証しています。乗算を先に、除算を後に行うことで、この例はより高い精度を達成できたでしょう。`msg.value*tokenPerEth/weiPerEth`とすればよいのです。

最後に、数値の任意精度を定義する場合、値をより高い精度に変換し、すべての数学的演算を実行してから、出力に必要な精度に戻すことが良いアイデアです。通常、`uint256`はガス使用量に最適であるため使用されます。これらは範囲内で約60桁の精度を提供し、その一部を数学的演算の精度に充てることができます。Solidityではすべての変数を高精度で保持し、外部アプリで低精度に変換する方が良いです。これは本質的にERC-20トークンコントラクトで`decimals`変数が機能する仕組みです。MetaMaskで1,000 USDTを送信する場合、実際には1,000,000,000単位のUSDTを送信しており、これはUSDTのデシマル（1e6）を掛けた1,000です。

精度を高めた数学演算の処理例を見るために、WadとRayの数学を紹介しましょう。*Wad*は18桁の精度を持つ小数を表し、EtherのようなERC-20トークンに一般的な18デシマルと完全に一致します。これによりトークン残高の表現に最適で、計算中に十分な精度を確保できます。一方、*Ray*は27桁の精度を持ち、ゼロに非常に近い比率の計算に有用です。DS-Mathとして知られる最初のSolidity固定小数点数学ライブラリが、これらの高精度な数値を扱うための構造を提供しました。

MakerDAOの開発者がWadとRayを自身のプロジェクトのニーズに合わせて当初作成しました。Etherの18デシマル標準を考えると（ほとんどのERC-20トークンもこの規約に従っていますが、例外も多数あります）、Wadは主要な金融単位に最適であり、Rayは精密な端数調整が必要なケースに予約されました。DS-Mathがこのアプローチを開拓しましたが、現在では精密なSolidity数学演算のためのライブラリがさらに多数利用可能です。AaveのWadRayMath、SolmateのFixedPointMathLib、OpenZeppelinのMathライブラリは、今日利用可能な選択肢のほんの一部です。

実例: ERC-4626インフレーション攻撃

ここでは、OpenZeppelinのERC-4626実装の簡略版を使用して、実際に一般的に悪用される精度損失の脆弱性を見てみましょう。ERC-4626はトークン化されたボールト標準であり、ユーザーが資産（USDTなど）をボールトに預け入れ、ボールトの資産に対する持ち分を表すシェアを受け取ることができます。例9-9は、使用するコントラクトの簡略版です。

例9-9. オリジナルのERC4626 OpenZeppelin実装の簡略版

```
1 abstract contract ERC4626 is ERC20, IERC4626 {
2    using Math for uint256;
3    IERC20 private immutable _asset;
4
5    constructor(IERC20 asset_) {
6        _asset = asset_;
7    }
8
9    function totalAssets() public view returns (uint256) {
10        return _asset.balanceOf(address(this));
11    }
12    function deposit(address receiver, uint256 assets) public {
13        SafeERC20.safeTransferFrom(_asset, msg.sender, address(this), assets);
14        uint256 shares = _convertToShares(assets, Math.Rounding.Down);
15        _mint(receiver, shares);
16        emit Deposit(msg.sender, receiver, assets, shares);
17    }
18    function _withdraw(address receiver, uint256 assets) public {
19        uint256 shares = _convertToShares(assets, Math.Rounding.Up);
20        _burn(msg.sender, shares);
21        SafeERC20.safeTransfer(_asset, receiver, assets);
22        emit Withdraw(msg.sender, receiver, msg.sender, assets, shares);
```

```
23    }
24    function _convertToShares(uint256 assets, Math.Rounding rounding) internal view
      returns (uint256) {
25        uint256 supply = totalSupply();
26        return
27            (assets == 0 || supply == 0)
28                ? assets
29                : assets.mulDiv(supply, totalAssets(), rounding); // (assets * supply) /
                  totalAssets()
30    }
31    function _convertToAssets(uint256 shares, Math.Rounding rounding) public view returns
      (uint256) {
32        uint256 supply = totalSupply();
33        return
34            (supply == 0)
35                ? shares
36                : shares.mulDiv(totalAssets(), supply, rounding); // (shares * totalAssets())
                  / supply
37    }
38 }
```

では、攻撃がどのように展開するかを見てみましょう。

攻撃者は新しく作成されたERC-4626ボールトを注視しており、一つが現れるのを見つけます。すぐにボールトの資産のわずか1単位だけを預け入れ、自分用のシェアをミントします。この時点で、ボールト内の総資産は1（攻撃者が預け入れた分のみ）であり、シェアの総供給量も1（攻撃者が1シェアをミントしたため）です。

ここからが巧妙になります。攻撃者は別のユーザーが大きな金額を預け入れるのを待ちます。1,000 USDTとしましょう。しかし正規のトランザクションが処理される前に、攻撃者が先手を打ち、1,000 USDTをボールトコントラクトに直接転送してフロントランニングします。重要なのは、攻撃者はボールトの`deposit`関数を使用せず、`USDT.transfer()`を呼び出すだけだということです。この1,000 USDTの「寄付」によりボールトの`totalAssets()`は1000e6 + 1に膨張しますが、シェアの`totalSupply()`は1のままです。1,000 USDTは実際には1,000e6として計上されることに留意してください。これはUSDTのデシマル（1e6）を掛けた1,000です。

被害者の預け入れがようやく処理されると、スマートコントラクトはユーザーに何シェアをミントすべきかを計算しようとします。シェアを計算する式を思い出してください。

（assets × supply）/ `totalAssets()`

今回のケースでは、被害者は1,000e6 USDTを預け入れており、式は以下のようになります。

1,000e6 × 1 /（1,000e6 + 1）= 0.999

切り捨ての丸めメカニズムにより、結果はゼロシェアとなります。被害者は1,000 USDTの預け入れに対して何も受け取れません。

一方、攻撃者は依然として1シェアを保持しており、そのシェアをバーンしてボールトの総残高である2,000 USDTを引き出すことができます。攻撃者はすべての資金を持ち去り、被害者は何も手にできません。

> **注記**
>
> OpenZeppelinはその後、仮想オフセットとデシマルオフセットの両方を導入することで、この攻撃を防ぐようERC-4626の実装を更新しました。デシマルオフセットはボールトシェアに使用される小数点以下の桁数を増加させ、丸め誤差を最小化し精度損失攻撃の収益性を低下させます。仮想オフセットは為替レート計算に仮想資産とシェアを追加し、攻撃者が初期変換レートを操作する能力を制限し、デッドシェアの作成からボールトを保護します。

価格操作

DeFiプロトコルが円滑に運用されるためには、正確な資産価格設定が不可欠です。DeFiプロトコルは、現在の資産価値を提供するプライスオラクルに依存しています。*オラクル*とは、スマートコントラクトに現実世界の情報を供給するデータフィードのことです。価格操作攻撃はこれらのオラクルを標的にします。つまり、スマートコントラクトのコード自体ではなく、コントラクトが依存するデータを狙います。この操作はDeFiプロトコルの動作を大きく変え、通常は存在しないアービトラージの機会を生み出す可能性があります。結果は、攻撃者がシステムを悪用して多額の利益を得ることです。

脆弱性

このシンプルなシナリオを想像してください。攻撃者が価格設定のために安全でないオラクルに依存するレンディングプロトコルを見つけます。資産の価格を操作して実際よりも低く見せかけることで、攻撃者は本来借りられるよりも多くの資産を借り入れることができます。その後、借り入れた資産を実際の市場価格で売却して利益を得ます。この脆弱性の根本は、資産価格の決定に操作可能なオンチェーン価格指標に依存していることにあります。操作は*フラッシュローン*（同一トランザクションブロック内で返済しなければならない即時かつ無担保のローン）を使用して増幅されることが多いです。

予防テクニック

価格を決定する必要がある場合、最善の選択はChainlink、RedStone、Pythなどの分散型オラクルを使用することです。これらのオラクルは分散化されているため、攻撃者がネットワーク内のノードの50%以上を制御する必要があり、侵害がはるかに困難になります。ただし、制限もあります。例えば、すべての資産に対して利用可能とは限りません。そのような場合は、時間加重平均価格（TWAP）オラクルを利用できます。

TWAPオラクルは、追加のセキュリティを備えたオンチェーンデータから資産価格を導出します。過去5分間などの定義された時間枠にわたる資産の平均価格を計算することで機能します。計算から現在のブロックを除外することで、TWAPオラクルはフラッシュローン攻撃に対して効果的に保護します。しかし、TWAPオラクルは十分な資金を持つ攻撃者による操作に完全に免疫があるわけではありません。ここでの鍵は期間の長さを調整することです。期間が長いほど、攻撃者が価格を操作するために必要な資本が増えます。しかし、期間が長いほどTWAP価格が実際の市場価格からより乖離する可能性もあります。したがって、プロジェクトの具体的なニーズとリスクプロファイルに基づいてTWAPを微調整することが重要です。

使用するオラクルに関係なく、提供されるデータを盲目的に信頼すべきではありません。オラクルデータを他のソースに対して定期的に検証することが良い慣行です。例えば、オラクル価格を他のソースの価格と比較し、重大な差異がある場合にフラグを立てるスクリプトを書くことができます。そのような差異が発見された場合、さらなる問題を防ぐためにプロトコルを一時停止できます。

プロトタイプ例: AMMオンチェーンデータへの依存

この例で見るように、脆弱なオラクルモジュールはプロトコル自体の一部であることが多いです。一般的な攻撃シナリオは、スマートコントラクトがUniswapのようなオンチェーンAMMプロトコルから直接資産価格を導出する場合に発生します。4,000 USDCと1 ETHのリザーブを持つUniswap V2プールを想像してください。スマートコントラクトは1 ETHが4,000 USDCの価値があると想定するかもしれません。しかし、推定価格がさらなる状態変更操作に使用される場合、この想定は非常にリスクが高くなります。そのような場合、攻撃者はフラッシュローンを使用して大きなスワップを実行し、プールの残高を変更することでETHの推定価格を変えることができます。この操作された価格に依存する脆弱なプロトコルは、攻撃者によって悪用されます。

幸い、この特定の攻撃ベクトルはよく知られています。かつてほど頻繁には悪用されなくなりましたが、注目度の高いインシデントでは依然として登場します。例えば、2025年5月にMobius Tokenが210万ドルの被害を受けました。直接的なトリガーは`mint`関数内での1018の誤った乗算でしたが、コントラクトにはそれとは別の同様に重大な脆弱性も含まれていました。BNB/USDT価格の計算にオンチェーン指標に依存していたため、操作にさらされていたのです。数学的なバグがなくても、コントラクトは短期間で悪用されていたでしょう。このようなコードがどのようにして本番環境に到達し、これほど多くのTVL（ロックされた総価値）を保護することになったのか疑問に思うかもしれません。チームはコントラクトのソースコードを公開しないことを選択し、コードを隠すことが安全性を提供すると考えていました。隠蔽によるセキュリティは機能しないことを改めて示す事例です。特にこれほど大きなリスクがある場合はなおさらです。

注記

オンチェーンデータから推定された価格の使用は、それらの価格が状態変更操作に適用される場合にのみリスクがあります。価格がフロントエンドがデータを取得するために使用するview関数のように情報提供目的のみである場合、攻撃は実行不可能です。しかし、外

部コントラクトがそのようなview関数から価格を取得し、状態変更操作に使用する場合は、操作に対して脆弱になります。

実例: Mango Markets

Mango Marketsの攻撃では、トレーダーがプラットフォームの価格操作の脆弱性を利用して1億1,600万ドル以上を搾取しました。攻撃者は2つのウォレットにまたがる1,000万ドルを使用して、1個あたり3.8セントの価格で4億8,300万枚のMango無期限先物（MNGO-PERP）をオープンしました。その後、3つの異なる取引所で400万ドル相当のMNGOを購入し、オラクルが報告する価格を2,300%引き上げました。この膨張したパーペチュアルポジションを担保として使用し、攻撃者はMango Marketsから1億1,600万ドルを借り入れ、大きな不良債権を残して資金を持ち逃げしました。価格操作攻撃でよくあるように、これはハッキングではなくシステムの仕組みの操作であり、基盤となるコードを破ることなくMangoの流動性を悪用したものでした。

攻撃者との交渉

攻撃者とプロトコルは、盗まれた資金のうちどれだけを返却するかをオンチェーンで直接交渉することが多く、プロトコル側はすべての告訴を取り下げることに同意します。これらの取引は一般的ですが、法的にはほとんど効力を持たない可能性が高いです。通常、プロトコルは攻撃者に約10%のバウンティを提供します。つまり、攻撃者が盗んだ資金の90%を返却すれば、プロトコルは追及を停止することに同意します。このような交渉は一般的ですが、この事例で起こったことは特に注目に値するものでした。

攻撃後、攻撃者はMango MarketsのDAOに取引を提案しました。コミュニティが別のSolanaプロジェクトであるSolendを救済するために以前引き受けた不良債権をカバーすることに同意すれば、盗まれた資金の大部分を返却するというものです。これに対して、Mangoチームは2番目の提案を提示しました。攻撃者が最大6,700万ドルを返却し、4,700万ドルをバグバウンティとして保持するという内容です。合意には不良債権に関するすべての請求の放棄、およびトークンが返却された後は刑事告訴や攻撃者の資金凍結を行わないという約束が含まれていました。最初の提案は否決され、2番目の提案が可決されました。これにより、Mango Marketsは10月15日に6,700万ドルの資産が実際に返却されたとツイートしました。

攻撃者の一人がTwitterで自身の正体を明かし、この攻撃を「非常に収益性の高い取引戦略」と呼び、すべてがプロトコルの意図された設計の範囲内で行われたと主張したことで、事態は法的な展開を見せました。しかし、米国当局は異なる見方をし、市場操作の容疑で彼を逮捕しました。Mango Marketsはその後民事訴訟を起こし、合意は脅迫下で行われたため無効であるべきだと主張し、4,700万ドルの損害賠償を求めました。DAOは法的に比較的新しい概念であるため、この訴訟は多くの注目を集めており、分散型組織が法的紛争をどのように処理するかの前例を作る可能性があります。

暗号資産の奇妙な法的情勢を象徴するかのように、攻撃者は実際に2025年5月の詐欺事件で勝訴しました。裁判官は、利用規約のないパーミッションレスなプロトコルに対して詐欺を行うことはできないと判断しました。しかし、ここからが重要です。当局がMango Marketsの当初の捜

査中に彼のデバイスを調べたところ、1,200枚以上の児童性的虐待の画像と動画が発見され、現在それにより4年以上の刑に服しています。優れたDeFi攻撃であっても、基本的な人間の品位への違反からは逃れられないことを証明しています。

不適切な入力検証

見落とされがちな主要な脆弱性の一つが、不適切な入力検証です。ユーザーや外部ソースからの入力が適切に検証されない場合、スマートコントラクトへの影響は軽微な問題から大規模な資金損失まで多岐にわたります。適切な入力検証は、コントラクトの動作を操作しようとする悪意あるアクターと、資金損失につながりうるユーザーや管理者による純粋なミスの両方から保護するのに役立ちます。適切な予防策を講じなければ、一見無害な見落としがコントラクトの実行に重大な問題をもたらす可能性があります。

脆弱性

本質的に、不適切な入力検証は、スマートコントラクトが受け取るデータやパラメーターを処理前に十分にチェックしない場合に発生します。特定の値が特定の条件を満たすことを確認しなければ、純粋なユーザーの誤りと潜在的な攻撃の両方に道を開くことになります。ユーザーは誤って不正なデータを入力するかもしれませんし、攻撃者は意図的にコントラクトに予期しないデータを送り込む可能性があります。これにより意図されたロジックがバイパスされ、予期しない状態変更が発生し、コントラクトが予測不能な動作をする原因となります。

この脆弱性のシンプルな例は、セッター関数がアドレスをゼロアドレスでないことを検証せずに資金の受取先として設定する場合に発生します。誤ってゼロアドレスを受取先に設定すると、ゼロアドレスに送信された資金は永久にロックされ、取り戻すことができなくなります。

スマートコントラクト開発における一般的で危険な誤解は、ソースコードをプライベートに保つことで何らかの形で悪用から保護できるという考えです。この章で以前、隠蔽によるセキュリティがなぜ機能しないかを議論しましたが、この原則は入力検証にも当てはまります。開発者はコードが公開されなければ発見されないと想定して、関数を保護しないままにすることがあります。しかし、攻撃者はコントラクトのバイトコードをリバースエンジニアリングして、機密性の高い保護されていない関数を特定できますし、実際にそうしています。例えば、いくつかのクローズドソースMEVボットが保護されていないフラッシュローンコールバックを通じて悪用され、数百万ドルの損失が発生しています。コードを隠してもリスクは隠せません。

予防テクニック

では、不適切な入力検証に対してどのように防御すればよいでしょうか。最初のステップはシンプルです。受け取る入力が有効であると決して仮定しないことです。入力がEOAから、別のコントラクトから、あるいは時には同じコントラクトからであっても、厳密にチェックする必要があります。入力の長さだけでなく、エッジケースや最小値・最大値のような境界条件も検証する必要があります。見落としてはならない典型的なエッジケースはゼロ値です。

再利用可能な検証ロジックは、安全で保守しやすいスマートコントラクトを書くための重要な部分です。これらの検証ブロックは、最も適した方法に応じてmodifierまたは内部関数のいずれかを使用して実装できます。modifierは、一貫した宣言的な方法で複数の関数に事前条件や事後条件を付与するのに特に有用です。例えば、関数の入力がゼロアドレスでないことを確認したり、機密性の高い操作を実行する前に呼び出し元が適切な権限を持っているかチェックしたりするためにmodifierを使用できます。内部関数も同じ目的を達成でき、特に検証が複雑なロジックに依存する場合や値を返す必要がある場合には、より柔軟性を提供することがあります。

アクセス制御

権限について言えば、堅牢なアクセス制御を実装することが重要です。`msg.sender`はパラメーターであり、そのように扱うべきです。カスタムロジックも選択肢ですが、OpenZeppelinのような信頼できるライブラリを使用することで、複雑さを最小限に抑えながら安全にアクセスを管理できます。

1つのエンティティが完全な制御を必要とするシンプルなプロジェクトでは、開発者はOpenZeppelinのOwnableコントラクトを使用できます。これは、重要な関数に対する権限を持つ単一の「オーナー」を指定します。セキュリティを強化するために、Ownable2Stepの使用を推奨します。このバージョンには2段階のオーナーシップ移転プロセスが含まれており、オーナーシップの偶発的な喪失を防ぐのに役立ちます。

より複雑なニーズには、OpenZeppelinのAccessControlを使用して、それぞれ異なる権限を持つ複数のロールを作成できます。ロールベースのアクセス制御により、異なるユーザーに特定のタスクを割り当てることができ、大規模なプロジェクトに最適です。

適切なアクセス制御を実装する前に、externalおよびpublic関数を誰が呼び出す可能性があるかについてのすべての想定を検証する必要があります。スマートコントラクトはパブリックでトラストレスな環境で動作するため、意図したエンティティのみがコントラクトとやり取りすると仮定することはできません。実際、攻撃者がこれらの関数を呼び出して意図しない動作をトリガーしようとすると常に想定すべきです。

プロトタイプ例: 任意の呼び出し

最も一般的な攻撃は任意の呼び出しを伴います。ここでは、脆弱なスマートコントラクトが攻撃者に呼び出し先のアドレスを提供することを許可します。このような状況では、コントラクトは事実上、攻撃者が望む任意の呼び出しを実行します。これを悪用する方法の一つは、操作された値を返すことで、コントラクトを騙して本来転送すべきでないトークンを転送させることです。

以下は脆弱なイールドアグリゲータープロトコルのサンプルコードです。

```
contract Aggregator {
    function stake( ... ) external {
        ...
```

```
    }
    function claimMultipleStakingRewards(address[] calldata _claimContracts) external {
        uint256 totalRewards;
        for (uint256 i = 0; i < _claimContracts.length; i++) {
            totalRewards += IClaimContract(_claimContracts[i]).claimStakingRewards(
            msg.sender
            );
        }
        IERC20(stakingToken).transfer(msg.sender, totalRewards);
    }
}
```

目的はシンプルです。`claimMultipleStakingRewards`関数はユーザーが提供したステーキングコントラクトアドレスの配列をループし、各アドレスで`claimStakingRewards`関数を呼び出し、総報酬を集計します。最後に、蓄積された報酬をユーザーのアドレスに送信します。問題は、コントラクトが`_claimContracts`内のアドレスが実際に信頼されたステーキングコントラクトを指しているかどうかを確認していないことです。これにより任意の外部呼び出しへの道が開かれます。

例えば、攻撃者は以下のようなコントラクトをデプロイできます。

```
contract Attack {
    function claimStakingRewards(address ) external pure returns (uint256) {
        return 1_000_000 ether; // 捏造された報酬
    }
}
```

この悪意あるコントラクトはステーキングコントラクトのふりをして、単に膨張した報酬値を返します。`Aggregator`がこれに対して`claimStakingRewards`を呼び出すと、呼び出し元が大量のトークンを受け取る権利があると騙されます。追加のチェックがなければ、`Aggregator`はそれを盲目的に合計に加算し、実際のトークンを攻撃者に転送します。`claimMultipleStakingRewards`で信頼されたコントラクトのみを許可する基本的な許可リストがあれば、これは回避できたでしょう。

署名リプレイ攻撃

イーサリアム上の署名は非常に有用であり、オフチェーンでアクションを承認し、コストのかかるオンチェーントランザクションの必要性を減らすことができます。例えば、トークンの転送やスマートコントラクトの特定の機能へのアクセスなど、特定のアクションを誰かに代わって実行することを承認する場合、許可を与えるオフチェーンメッセージに署名できます。コントラクトはその署名を検証し、オンチェーンで直接やり取りすることなくアクションを実行します。これにより、ガスレストランザクションも可能になります。オフチェーンで署名し、リレイヤーがオンチェーンで送信してガス代を支払います。スマートコントラクトはこれらの署名を検証して、常にオンチェーンでのやり取りを必要とせずにアクションが安全に承認されていることを保証できます。

しかし、データが署名されたら、論理的には一度だけ使用されるべきです。署名されたトランザクションが再利用できる場合、リプレイ攻撃への道が開かれます。リプレイ攻撃とは、攻撃者が署名を再利用して、資金の転送やコントラクト状態の変更など、同じアクションを許可なく複数回実行するものです。スマートコントラクトは、各署名済みメッセージが一意でありリプレイできないことを保証することで、リプレイ攻撃を防ぐように設計されなければなりません。

脆弱性

リプレイ攻撃に対して脆弱な例のコントラクト（例9-10）を見てみましょう。

例9-10. Token: 署名リプレイ攻撃に対して脆弱なコントラクト

```
1 contract Token {
2    mapping(address => uint256) public balances;
3    struct Signature {
4        bytes32 r;
5        bytes32 s;
6        uint8 v;
7    }
8    event Transfer(address indexed from, address indexed to, uint256 amount);
9    function transfer(uint256[] memory _amount, address[] memory _from, address[]
     memory _to,  Signature memory _signature) public {
10        bytes32 messageHash = keccak256(abi.encodePacked(_from, _to, _amount));
11        address signer = ecrecover(messageHash, _signature.v, _signature.r, _signature.s);
12        for(uint256 i = 0; i < _from.length; i++){
13            address __from = _from[i];
14            address __to = _to[i];
15            uint256 __amount = _amount[i];
16            require(balances[__from] >= _amount[i], "Insufficient balance");
17            require(signer == _from[i], "Invalid signature");
18            balances[__from] -= __amount;
19            balances[__to] += __amount;
20            emit Transfer(__from, __to, __amount);
21        }
22    }
23 }
```

一見すると、これは便利なコントラクト関数に見えます。有効な署名を持つ誰もが、署名者がガスを支払う必要なく複数の転送を実行できます。管理者がオフチェーンでデータに署名し、他の誰か（おそらくサービス）がオンチェーンでトランザクションを送信できます。しかし、署名の処理は簡単ではなく、この非常に短いコードには相当数の問題が含まれています。

最も明白な問題は、同じ署名を何度も再利用することを防ぐメカニズムがないことです。署名が使用済みかどうかを追跡する方法がなければ、攻撃者は被害者の残高が枯渇するまでトランザクションを単純に繰り返すことができます。修正はかなりシンプルです。署名されるデータにノンス（一度だけ使用される値、通常は各トランザクションでインクリメントされるカウンター）

を追加する必要があります。署名を検証するコントラクトは、提供されたノンスが以前に使用されていないことを確認する責任があります。これにより各署名も一意になり、リプレイを防止します。イーサリアムのトランザクションはこの理由からすでにノンスを使用しています。

もう一つの重大な問題は署名の可鍛性です。これは暗号学的署名が改変されて、同じ元のメッセージに対して異なるが依然として有効な署名を生成できる場合に発生します。コントラクトで使用されている組み込みの`ecrecover`関数はこの問題に対して脆弱です。攻撃者は有効な署名を微調整して、元の署名済みメッセージは同じでありながら機能する別の署名を作成できます。これを回避するために、開発者はOpenZeppellin ECDSAライブラリが提供するような、より安全な署名検証方法を使用すべきです。可鍛な署名は、署名をリプレイ攻撃防止のための一意な識別子として使用すべきでない理由です。ノンスを使用してください。

潜在的な署名の操作に対処しましたが、署名されるデータ自体も操作可能な場合はどうでしょうか。このコントラクトでは可能です。問題は`abi.encodePacked`の使用にあります。コンパクトなエンコーディングでメモリ使用量が少ないため選ばれることが多いですが、その効率性にはトレードオフがあります。具体的には、`abi.encodePacked`は長さ情報や境界を追加せずに生のバイトを連結するため、異なる入力セットが同じ出力を生成する可能性があります。これがどのように作用するか見てみましょう。

簡略化のため、金額は8ビット（16進2桁）、アドレスは12ビット（16進3桁）と仮定しましょう。パラメーターが以下の通りだとします。

```
_amount = [0x64, 0x64]
_from = [0x001, 0x002]
_to = [0x003, 0x003]
```

`abi.encodePacked`を使用すると、これらの値は`0x6464001002003003`に結合されます。しかしここからが厄介になります。`0x002`を`_from`から`_to`に移動しても、`abi.encodePacked`からは以前とまったく同じ出力が得られます。

```
_amount = [0x64, 0x64]
_from = [0x001]
_to = [0x002, 0x003, 0x003]
```

この新しい値のセットに対しても、`abi.encodePacked`は同じ出力`0x6464001002003003`を返します。これは、ユーザー`0x002`が有効な署名を使用しつつ入力パラメーター`_from`と`_to`を変更でき、実行される転送が`0x001`から`0x002`へのものだけであるとコントラクトを騙せることを意味します。この例で使用されているコードは入力の検証が非常に不十分であり、この問題のある状況を許しています。いずれにしても、配列のような動的データ型に対して署名を生成する際に`encodePacked`を避けるべきであることを示しています。これらの場合は`abi.encode`を使用すべきであり、動的データを連結する場合でも曖昧さのない出力を生成し、この種の攻撃を効果的に防ぎます。

しかし、もう一つの問題があります。このコントラクトが複数のチェーンにデプロイされた場合はどうなるでしょうか。同じ署名がすべてのチェーンで有効となり、クロスチェーンリプレイ攻撃の機会が生まれます。攻撃者は1つのチェーンでユーザーのアクティビティを監視し、他のチェーンで署名を再利用できます。これを防ぐために、署名されたメッセージにコンテクストデータを含める必要があります。最低限`chainId`を含めるべきです。ユースケースによっては、コントラクトアドレスやバージョンも含める必要があるかもしれません。幸い、ゼロから新しいソリューションを考え出す必要はありません。EIP-712はコンテクスト対応の署名を可能にすることでこれを解決する標準です。また、混乱するバイト文字列の代わりに、署名する内容についての読みやすい情報をユーザーに表示することで、ユーザーエクスペリエンスも向上させます。

予防テクニック

リプレイ攻撃やその他の脆弱性を防ぐためには、各署名が一意で安全であり、一度しか使用できないことを保証する必要があります。ノンス、安全な署名処理、コンテクスト対応の署名を通じて、これを簡単に達成できます。

ノンスは一意性を保証するために使用されます。各署名済みメッセージにノンスを追加することで、攻撃者が署名を再利用することを防ぎます。署名を検証するコントラクトは、使用されたノンスが一意であることを確認します。署名が使用されると、そのノンスは無効になり、リプレイの試みを阻止します。

署名を検証する際には、プレーンな組み込み`ecrecover`関数の使用を避け、署名の可鍛性に対して免疫のあるOpenZeppelinのECDSAライブラリを利用すべきです。また、署名は操作可能なため、一意な識別子として使用してはなりません。

動的データに署名する場合、`abi.encodePacked`の代わりに`abi.encode`を使用することで、入力を適切に分離して操作を防ぎ、改ざんや誤解釈ができないようにします。

最後に、署名を扱う場合は常にEIP-712を実装すべきです。クロスチェーンリプレイを防ぐための`chainId`などのコンテクスト対応性の追加に加えて、EIP-712は不透明なバイト文字列の代わりに署名する内容の明確で意味のある可視化をユーザーに提供することでユーザーエクスペリエンスを向上させます。これによりトランザクションの理解が容易になるだけでなく、ユーザーが不審なリクエストをより認識しやすくなるため、フィッシング攻撃がより困難になり、ユーザーの安全性も向上します。

実例: TCHトークン

2024年5月、TCHトークンは一般的な署名可鍛性の脆弱性により悪用されました。問題はコントラクトの`burnToken`関数にあり、トークンのバーンを承認するために署名を検証していました。署名リプレイ攻撃を防ぐために、コントラクトは使用済みの署名をマッピングに保存していました。しかし、署名が改ざんされた場合、この防御を回避することができました。

攻撃者は以前に送信された署名を収集し、署名の一部である`v`と`s`の値を変更することでこれを悪用しました。署名は改変されましたが、`ecrecover`を使用した検証は依然として通過しまし

た。変更された署名は元の署名とは異なるため、すでに使用済みとは認識されず、新しいバージョンがマッピングに保存されました。このトリックにより、攻撃者はPancakeSwap流動性ペアが所有する大量のTCHトークンを繰り返しバーンすることができました。これにより攻撃者はプール内のトークン価格を操作し、自らが引き起こした価格変動から利益を得ることができました。

スマートコントラクトの設定ミス

設定ミスは、技術的には脆弱性ではないものの、スマートコントラクトに深刻な影響を与えうる厄介な問題の一つです。スマートコントラクトは書いて監査を受けたら終わりではなく、デプロイ、保守、そして時にはアップグレードも必要です。設定ミスはこれらの段階でしばしば発生します。例えば、DeFiプロトコルには多数のパラメーターが付随しており、これらのいずれかが誤って設定された場合、大きな損失につながる可能性があります。残念ながら、この種の問題は監査でも発見が困難です。監査人がデプロイメントやアップグレードスクリプトを見落とすことが多いためです。設定ミス自体は脆弱性ではありませんが、脆弱性への入り口を作る可能性があるため、デプロイメントと管理の段階で特に慎重であることが重要です。

設定ミスの問題は多岐にわたるためカテゴリー分けが難しく、すべてをリストアップしようとする代わりに、何が問題になりうるかを理解するためにいくつかの実例に直接取り組みましょう。

実例: yUSDT

最もシンプルな設定ミスのケースを見てみましょう。Yearn FinanceのyUSDTトークンにおけるストレージ変数の設定ミスで、2023年4月の攻撃につながりました。yUSDTトークンはUSDTベースのデリバティブに投資することで利回りを生成するはずでしたが、設定ミスにより、実際には異なるトークン（IUSDC）を原資産として使用していました。驚くべきことに、これは1,000日以上気づかれずにいました。この設定ミスにより、攻撃者がシステムを操作し、プールから価値を引き出し、実質的に無料でyUSDTをミントすることが可能になりました。その結果、yUSDTの価値はゼロに下落し、攻撃者は1,160万ドルの利益を持ち逃げしました。

実例: Ronin Bridge

2024年8月、Ronin Bridgeはコントラクトのアップグレードからわずか1時間後にハッキングされました。根本原因はアップグレードプロセスでのミスでした。重要な変数`_totalOperatorWeight`が初期化されていなかったのです。この変数は`initializeV3`関数で設定されるはずでしたが、アップグレード中に`initializeV4`のみが呼び出され、前のバージョンからの必要なセットアップがスキップされました。この見落としによりコントラクトが露出されました。この事例では、ホワイトハットMEVボットが攻撃をフロントランニングし、盗まれた4,000 ETHを返却することができましたが、アップグレード手順の徹底的なレビューとテストの重要性を強調しています。

注記

コントラクトのアップグレード直後にハッキングが発生するのは単なる偶然だと思うなら、それは間違いです。ブラックハットもホワイトハットもコントラクトのアップグレードを注意深く監視しています。ブラックハットは悪用するための弱点を探し、ホワイトハットは攻撃を防ごうとします。チームは小さなコード変更のセキュリティリスクを過小評価し、監査プロセスを省略することがよくあります。残念ながら、コントラクトを破壊するのにそれほど多くは必要ありません。この事例が示すように、脆弱性は常にスマートコントラクト自体にあるとは限らず、アップグレードの実行方法にある場合もあります。

実例: Sonne Finance

2024年5月のSonne Financeハッキングの根本原因は、典型的なプロトコルのバグではなく、マーケットアクティベーションプロセスの欠陥でした。多くのプロトコルと同様に、SonneはCompound v2で見つかった「空のマーケット」バグを認識していました。オープンだが資金が入っていないマーケットが悪用されて、プロトコル全体が枯渇する可能性があるというものです。このバグの標準的な修正は、マーケットがアクティベートされる際にアトミックに資金を預け入れることで、マーケットがいかなる時点でも空にならないようにすることです。

Sonneにはこれを処理するための計画がありました。マーケットの追加、資金の預け入れ、そしてマーケットの使用開始の3つのアクションすべてをタイムロックを通じて行う予定でした。正しい順序で実行されていれば、プロセスは機能していたでしょう。問題は、Sonneがこれらの各ステップをガバナンスタイムロックコントローラーで別々のトランザクションとしてスケジュールしたために、実行順序が強制されなかったことです。Sonneチームはガバナンスの`EXECUTOR_ROLE`をすべての人にアクセス可能にし、タイムロックが期限切れになった後は誰でもガバナンストランザクションを実行できるようにしていました。この設定は異例ですが、本質的に問題があるわけではありませんでした。しかし、この特定の状況では壊滅的であることが判明しました。タイムロックが期限切れになった後、誰でもアクションを順不同で実行できる状態になっていたのです。

攻撃者は資金の預け入れを待たずにキューに入っているすべてのタイムロックアクションを単純に実行し、空で脆弱なマーケットを開いたままにしました。この資金のないマーケットを悪用することで、プロトコルから2,000万ドルを引き出しました。

ここでの重要な教訓は、ガバナンスアクションがセキュリティを確保するために特定の順序で実行される必要がある場合、アトミックにすべきだということです。例えば、OpenZeppelin Timelockを使用している場合は、`schedule()`ではなく`scheduleBatch()`でスケジュールすべきです。Sonneの間違いはこれらのアクションを個別にキューに入れることを許可したことで、それが露出につながりました。

予防テクニック

設定ミスの問題やこれまで議論してきたようなコストのかかるミスを避けるために、スマートコントラクトのライフサイクル全体を通じて、特にデプロイメント、アップグレード、および重要なガバナンスアクションに関しては、事前対応的なアプローチを取る必要があります。デプロイメントとアップグレードのスクリプトが徹底的にテストされ監査されていることを常に確認すべきです。コード自体の監査を超えて、メインネットに関わるスクリプトに細心の注意を払い、プロセスのあらゆる側面が本番に近い環境でテストされていることを確認する必要があります。

コントラクトライブラリ

再利用可能な既存のコードは多数存在し、オンチェーンにデプロイされた呼び出し可能なライブラリとしても、オフチェーンのコードテンプレートライブラリとしても利用可能です。イーサリアムで最も広く使用されているリソースはOpenZeppelinスイートで、さまざまなトークンの実装から異なるプロキシアーキテクチャ、`Ownable`、`Pausable`、`ReentrancyGuard`のようなコントラクトでよく見られるシンプルな動作まで、豊富なコントラクトライブラリです。このリポジトリのコントラクトは広範にテストされており、場合によっては事実上の標準実装として機能しています。これらは無料で使用でき、OpenZeppelinと増え続ける外部コントリビューターのリストによって構築・保守されています。

その他の注目すべきコントラクトライブラリには、ParadigmのSolmateとVectorizedのSoladyがあります。Solmateは設計においてより独自の方針を持ち、Soladyは主にガスの最適化に焦点を当てています。

追加リソース

スマートコントラクトのセキュリティは非常に奥深くニュアンスに富むため、この高度な主題についてさらに学びたい読者のためにリソースのリストを以下に示します。

Cyfrin Updraft Smart Contract Security and Auditing

270以上のレッスンを含む包括的な24時間コース

Secureum Bootcamp

イーサリアムスマートコントラクトセキュリティ監査に焦点を当てた3か月の集中ブートキャンプ

Ethernaut

OpenZeppelinによるSolidityベースのウォーゲームで、プレイヤーがスマートコントラクトのレベルをハッキングして一般的な脆弱性を学ぶ

Damn Vulnerable DeFi

フラッシュローン、プライスオラクル、ガバナンス、NFTなどをカバーする18のチャレンジを特徴とするCTF（Capture the Flag）プラットフォーム

Capture the Ether

クラシックなCTFスタイルのイーサリアムセキュリティゲーム

QuillCTF

QuillAuditsによるイーサリアムセキュリティパズルのコレクション

Paradigm CTF

Paradigmが主催する、経験豊富なスマートコントラクトハッカー向けの年次オンラインCTFコンペティション。チャレンジは非常に高度で最先端の攻撃を反映しており、公式ソリューションとライトアップが公開されることも多く、学習リソースとしても機能する

結論

Solidityコンパイラはアップデートにより、整数オーバーフローやデフォルトの可視性の問題などのリスクを軽減するようになりました。そのため、本書の初版で言及されていた古い落とし穴のいくつかを削除し、より現在的で関連性の高い脆弱性に焦点を当てることができました。いずれにしても、スマートコントラクトの領域で活動する開発者が知り理解すべきことはまだ多くあります。スマートコントラクトの設計とコード記述においてベストプラクティスに従うことで、多くの深刻な落とし穴と罠を回避できるでしょう。

おそらく最も基本的なソフトウェアセキュリティの原則は、信頼できるコードの再利用を最大化することです。暗号学では、この原則は非常に重要であるため、「独自の暗号を作るな」という格言に凝縮されています。スマートコントラクトの場合、コミュニティによって徹底的に精査された自由に利用可能なライブラリからできる限り多くを活用することに相当します。

第10章 トークン

token（トークン）という言葉は、古英語の*tācen*に由来し、記号やシンボルを意味します。一般的には、交通トークン、コインランドリーのトークン、アーケードゲームのトークンなど、本質的な価値がほとんどない私的発行の特殊用途コイン状のものを指す言葉として使われてきました。今日では、ブロックチェーン上で管理されるトークンがこの言葉の意味を再定義しつつあります。所有可能で、資産・通貨・アクセス権を表現するブロックチェーンベースの抽象概念を意味するようになっています。

*token*という言葉と取るに足らない価値との結びつきは、物理的なトークンの用途が限定的だったことに大きく関係しています。物理的なトークンは特定の事業者・組織・場所に限定されることが多く、容易に交換できず、通常は単一の機能しか持ちません。ブロックチェーントークンでは、こうした制約が取り除かれるか、より正確に言えば、制約そのものを再定義できるようになります。多くのブロックチェーントークンはグローバルに複数の目的を果たし、流動性のある市場で互いにまたは他の通貨と交換できます。使用や所有に関する制約がなくなることで、「取るに足らない価値」という認識も過去のものとなっています。

本章では、トークンのさまざまな用途と、トークンの作成方法について説明します。また、代替可能性や内在性といったトークンの属性についても議論します。最後に、トークンの基盤となる標準規格や技術を検証し、独自のトークンを構築する実験を行います。

トークンの使われ方

トークンの最も明白な用途は、デジタルの私的通貨です。しかし、これは可能な用途のひとつに過ぎません。トークンはさまざまな機能を果たすようにプログラムでき、それらはしばしば重複します。たとえば、あるトークンが投票権、アクセス権、リソースの所有権を同時に持つことがあります。以下のリストが示すように、通貨は最初の「アプリ」に過ぎません。

通貨

トークンは通貨の一形態として機能し、その価値は私的な取引を通じて決定される。

リソース

トークンは、シェアリングエコノミーやリソース共有環境で獲得または生成されたリソースを表現できる。たとえば、ネットワーク上で共有可能なリソースを表すストレージトークンやCPUトークンなどである。

資産

トークンは、内在的または外在的、有形または無形の資産の所有権を表現できる。たとえば、金、不動産、自動車、石油、エネルギー、MMOGのアイテムなどである。

アクセス

トークンはアクセス権を表現し、ディスカッションフォーラム、限定的なウェブサイト、ホテルの部屋、レンタカーなど、デジタルまたは物理的な資産へのアクセスを付与できる。

エクイティ

トークンは、デジタル組織（例: DAO）または法人（例: 株式会社）における株主資本を表現できる。

投票

トークンはデジタルまたは法的システムにおける投票権を表現できる。

コレクティブル

トークンはデジタルコレクティブル（例: CryptoPunks）または物理的なコレクティブル（例: 絵画）を表現できる。

アイデンティティ

トークンはデジタルアイデンティティ（例: アバター）または法的なアイデンティティ（例: 国民ID）を表現できる。

アテステーション

トークンは、何らかの権威者または分散型評判システムによる事実の証明やアテステーションを表現できる（例: 婚姻届、出生証明書、大学の学位）。

ユーティリティ

トークンはサービスへのアクセスや支払いに利用できる。

多くの場合、ひとつのトークンがこれらの機能を複数兼ね備えています。物理的な同等物では常に不可分に結びついてきたため、区別が難しいこともあります。たとえば物理的な世界では、運転免許証（アテステーション）はアイデンティティ文書（アイデンティティ）でもあり、この2つを分離できません。デジタルの領域では、以前は混在していた機能を分離し、独立して発展させることが可能です（例: 匿名のアテステーション）。

トークンと代替可能性

Wikipediaによると、「経済学において、代替可能性とは財や商品の個々の単位が本質的に交換可能である性質のこと」です。トークンが*代替可能*であるとは、任意の単位を別の単位に置き換えても価値や機能に違いが生じないことを意味します。

非代替性トークン（NFT）は、各トークンが固有の有形または無形のアイテムを表現するため、相互に交換できないトークンです。たとえば*特定の*ファン・ゴッホの絵画の所有権を表すトークンは、ピカソの絵画を表す別のトークンと同等ではありません。同じ「アート所有権トークン」システムの一部であっても同様です。同じく、*特定の*デジタルコレクティブル（特定のCryptoKittyなど）を表すトークンは、他のどのCryptoKittyとも交換できません。各NFTにはシリアルナンバーなどの固有の識別子が関連付けられています。

本章の後半で、代替可能トークンと非代替性トークンの両方の例を見ていきます。

> **注記**
>
> *代替可能*（fungible）は「お金と直接交換可能」という意味で使われることがよくある（たとえば、カジノのトークンは「換金」できるが、コインランドリーのトークンは通常できない）。本書ではこの意味では使用していない。

カウンターパーティリスク

*カウンターパーティリスク*とは、取引の*相手方*が義務を果たさないリスクのことです。取引に2つ以上の当事者が関与すると、カウンターパーティリスクはさらに増大します。たとえば貴金属の預かり証を持っていて、それを誰かに売る場合、そのトランザクションには少なくとも3つの当事者が存在します。売り手、買い手、そして貴金属の保管者です。物理的な資産を保有する保管者は、必然的にトランザクションの履行に関わる当事者となり、その資産に関連するあらゆるトランザクションにカウンターパーティリスクを追加します。所有権トークンの交換を通じて資産が間接的に取引される場合、保管者の存在による追加のカウンターパーティリスクが一般的に発生します。保管者は資産を保有しているのでしょうか。トークン（証書、権利書、権原、デジタルトークンなど）の移転に基づく所有権の移転を認識（または許可）するのでしょうか。資産を表すデジタルトークンの世界でも非デジタルの世界と同様に、トークンが表す資産を誰が保有し、その原資産にどのようなルールが適用されるかを理解することが重要です。

トークンと内在性

intrinsic（内在的）という言葉は、ラテン語の*intra*（内側から）に由来します。一部のトークンはブロックチェーンに内在するデジタルアイテムを表現します。こうしたデジタル資産はトークン自体と同様にコンセンサスルールによって管理されます。この点には重要な意味があります。内在的な資産を表すトークンには追加のカウンターパーティリスクが伴いません。CryptoKittyの鍵を保有していれば、他の誰かがそのCryptoKittyをあなたのために保管しているわけではなく、あなたが直接所有しています。ブロックチェーンのコンセンサスルールが適用され、秘密鍵の所有（すなわち管理）が仲介者なしに資産の所有と同等になります。

逆に、多くのトークンは不動産、企業の議決権株式、商標、金塊、債券など*外在的*なものを表現するために使用されます。ブロックチェーンの「内側」にないこれらのアイテムの所有権は、トークンを管理するコンセンサスルールとは別に法律・慣習・方針によって管理されます。つまり、トークンの発行者と所有者は現実世界の従来型の契約に依然として依存する可能性があります。こうした外在的資産は保管者によって保有され、外部の登記簿に記録され、またはブロックチェーン環境外の法律や方針によって管理されるため、追加のカウンターパーティリスクを伴います。

ブロックチェーンベースのトークンの最も重要な意義のひとつは、外在的資産を内在的資産に変換してカウンターパーティリスクを除去できることです。良い例として、企業のエクイティ（外在的）からDAOまたは類似の（内在的）組織におけるエクイティトークンや投票トークンへの移行が挙げられます。ステーブルコインも、法定通貨にペッグされ国債や現金準備などの外在的資産によって裏付けられたブロックチェーンベースのトークンとして、もうひとつの例です。

ユーティリティか、エクイティか、それとも金集めか

ほぼすべてのイーサリアムプロジェクトが何らかのトークンを伴ってローンチされているようです。しかし、これらすべてのプロジェクトに本当にトークンが必要なのでしょうか。「すべてをトークン化しよう」というスローガンはキャッチーに聞こえますが、現実ははるかに複雑です。トークンはコミュニティを組織しインセンティブを付与する強力なツールになり得ますが、投機やハイプの代名詞にもなっています。

理論上、トークンには2つの主要な目的があります。まず*ユーティリティトークン*です。ユーティリティトークンは、特定のエコシステム内でサービスやリソースへのアクセスを提供するために設計されています。たとえば分散型ネットワーク上のストレージ容量やDAppのプレミアム機能へのアクセスを表すトークンがあります。この場合、トークンの価値はプラットフォーム内での機能に紐づきます。次に*エクイティトークン*です。エクイティトークンは企業の株式のように機能することを意図しています。DAOにおける議決権やプロジェクトの利益配分など、所有権や支配権を表現できます。

しかし実際には、これらのカテゴリの区別はしばしば曖昧です。多くのユーティリティトークンは依然として大部分が投機的であり、ユーザーはアクセス資格としてよりも資産として保有しています。エクイティ的なトークンはガバナンス権を付与する場合がありますが、意味のある参加を保証するメカニズムを欠いていることが多いです。トークンを経済モデルに深く統合しているプロジェクトもありますが、そうしたケースは例外的です。

ここで疑問が生じます。トークンは本質的に悪いものなのでしょうか。まったくそうではありません。トークンはコミュニティの創造とインセンティブ付与や、DAOにおける分散型ガバナンスの推進に非常に効果的です。しかし現実には、ほとんどのトークンはユーティリティではなく利益を主な動機としてローンチされています。トークンのローンチや投資を考えているなら、厳しい問いを投げかける価値があります。そのトークンはプロトコルに本当に必要な目的を果たしているのか、それとも単なる資金調達ツールなのか。トークンなしでもプロジェクトは同じよう

にうまくいくのではないか。これらの問いに正直に答えることで、本物のイノベーションとマーケティング主導のハイプを区別できます。

トークンの状況はまだ発展途上であり、トークンは本質的に善でも悪でもありません。その価値は設計と実装のあり方に依存します。課題は、意味のあるものと無意味なものを分別し、次のミームコインの誘惑に抗うことにあります。

注記

本章の執筆時点（2025年1月）、新しく選出された大統領ドナルド・トランプが独自のミームコインをローンチし、わずか1日で時価総額150億ドルに達した。多くの人が暗号資産に好意的な政策を期待していたが、代わりに得られたのはミームコインであった。この出来事は、ハイプが実質に勝ることが多い、暗号資産を支配する投機的な熱狂を浮き彫りにしている。

あひるだ

トークンは長い間、スタートアップのお気に入りの資金調達ツールでした。イノベーション、分散化、そして時には完全な経済的自由を約束します。しかし問題があります。一般への証券の提供はほとんどの法域で規制された活動であり、トークンは容易に証券の領域に踏み込み得るのです。何年もの間、プロジェクトはトークンを「ユーティリティトークン」と称し、将来のサービスへのアクセスの先行販売に過ぎないと主張して規制を回避しようとしてきました。そのロジックは、トークンがエクイティの持分でなければ証券ではないというものです。しかし古い格言にあるように、「あひるのように歩き、あひるのように鳴くなら、それはあひるだ」のです。そして規制当局、特に米国証券取引委員会（SEC）はこのあひるたちに注目しています。

過去数年間、SECはトークンオファリングに対してますます積極的な姿勢をとり、ユーティリティとエクイティの境界線をまたごうとするプロジェクトを摘発してきました。たとえば2020年、SECはXRPトークンをめぐってRipple Labsを提訴し、未登録証券であると主張しました。Ripple側はXRPは通貨であり証券ではないと反論しましたが、裁判所の部分的な判決はこれらのケースがいかに微妙であるかを示しました。

イーサリアム自体も監視を免れていませんでした。2018年、元SEC当局者はイーサが「十分に分散化されている」ため証券ではないと宣言しました。しかし2024年になっても、SECはイーサリアムのPoSへの移行により再び精査の対象になる可能性を示唆しています。なぜでしょうか。ステーキング報酬が配当に類似しており、配当は証券の特徴だからです。こうした展開は、規制の状況がいかに流動的で予測不可能かを示しています。

興味深くもあり苛立たしくもあるのは、これらのケースがいかに頻繁に意味論の問題に帰着するかということです。プロジェクトはトークンをサービスへのチケットのようなユーティリティツールだと主張します。しかし購入者の主な動機が投機であれば、SECは端的に証券と見なしま

す。問題は、こうした判断に使用される法的枠組みがブロックチェーン技術の登場よりはるか以前に作られたものである点です。1940年代に投資契約を定義するために開発されたハウィーテストは、分散型ネットワークやプログラマブルな資産を想定していません。そのため暗号資産プロジェクトへの適用は必ずしも簡単ではありません。イノベーターは資金を調達してコミュニティを構築したいと考え、規制当局は投資家を誤解から保護したいと考えています。その結果、何十億ドルもの資金とエコシステム全体の命運をかけた法廷劇が繰り返し展開されることになります。

イーサリアム上のトークン

ブロックチェーントークンはイーサリアム以前にも存在していました。ある意味では、最初のブロックチェーン通貨であるビットコインもトークンの一種です。イーサリアム以前にもビットコインや他の暗号通貨上で多くのトークンプラットフォームが開発されました。しかしイーサリアム上で最初のトークン標準が導入されたことが、トークンの爆発的な増加につながりました。

ヴィタリック・ブテリンは、イーサリアムのような汎用プログラマブルブロックチェーンの最も明白で有用なアプリケーションのひとつとしてトークンを提案しました。実際、イーサリアムの最初の1年間はブテリンや他の人々がイーサリアムのロゴとスマートコントラクトのサンプルが背面にプリントされたTシャツを着ているのをよく見かけました。このTシャツにはいくつかのバリエーションがありましたが、最も一般的なものはトークンの実装を示していました。

イーサリアム上でのトークン作成の詳細に入る前に、トークンの動作の概要を把握しておくことが重要です。トークンはイーサとは異なります。イーサリアムプロトコルはトークンについて何も知らないからです。イーサの送信はイーサリアムプラットフォームの固有のアクションですが、トークンの送信や所有はそうではありません。イーサリアムアカウントのイーサ残高はプロトコルレベルで処理されますが、トークン残高はスマートコントラクトレベルで処理されます。イーサリアム上で新しいトークンを作成するには、新しいスマートコントラクトを作成する必要があります。デプロイされたスマートコントラクトが所有権、送金、アクセス権などすべてを処理します。スマートコントラクトは必要なアクションを自由に実装できますが、既存の標準に従うのがおそらく最も賢明です。次にそうした標準について見ていきます。

ERC-20トークン標準

最初の標準は2015年11月にファビアン・フォゲルシュテラーによってERCとして提案されました。GitHubのイシュー番号20が自動的に割り当てられ、「ERC-20トークン」という名前が生まれました。現在、トークンの大多数がERC-20標準に基づいています。ERC-20のリクエストフォーコメントは最終的にEIP-20となりましたが、今でもほとんどの場合は元の名前であるERC-20で参照されています。

ERC-20は代替可能トークンの標準であり、ERC-20トークンの異なる単位は交換可能で固有の性質を持ちません。ERC-20標準はトークンを実装するコントラクトの共通インターフェースを定義しており、互換性のあるトークンは同じ方法でアクセスおよび使用できます。このインターフェースは、標準のすべての実装に存在しなければならない必須関数と、開発者が追加できるオプションの関数および属性で構成されています。

ERC-20の必須関数とイベント

ERC-20に準拠するトークンコントラクトは、少なくとも以下の関数とイベントを提供する必要があります。

`totalSupply`

現在存在するトークンの総単位数を返す。ERC-20トークンは固定または可変の供給量を持つことができる。

`balanceOf`

アドレスが与えられると、そのアドレスのトークン残高を返す。

`transfer`

アドレスと数量が与えられると、transferを実行したアドレスの残高からそのアドレスへ、指定された数のトークンを送金する。

`transferFrom`

送信者、受信者、数量が与えられると、あるアカウントから別のアカウントへトークンを送金する。 `approve` と組み合わせて使用する。

`approve`

受信者アドレスと数量が与えられると、承認を発行したアカウントから指定された数量まで複数回の送金を実行する権限を、そのアドレスに付与する。

`allowance`

所有者アドレスと支出者アドレスが与えられると、支出者が所有者から引き出すことを承認されている残りの数量を返す。

`transfer`

成功した送金（ `transfer` または `transferFrom` の呼び出し）時に発行されるイベント。ゼロ値の送金でも発行される。

`approval`

`approve`の呼び出しが成功した時に記録されるイベント。

ERC-20のオプション関数

前のセクションで列挙した必須関数に加えて、以下のオプション関数も標準で定義されています。

Name

トークンの人間が読める名前（例:「US dollars」）を返す。

Symbol

トークンの人間が読めるシンボル（例:「USD」）を返す。

decimals

トークンの数量を分割するために使用される小数点以下の桁数を返す。たとえば、小数点以下の桁数が2の場合、トークン数量1,000は実際には残高10を意味する。

Solidityで定義されたERC-20インターフェース

ERC-20のインターフェース仕様をSolidityで表すと、以下のようになります。

```
contract ERC20 {
   function totalSupply() public view returns (uint256 theTotalSupply);
   function balanceOf(address _owner) public view returns (uint256 balance);
   function transfer(address _to, uint256 _value) public returns (bool success);
   function transferFrom(address _from, address _to, uint256 _value) public returns
      (bool success);
   function approve(address _spender, uint256 _value) public returns (bool success);
   function allowance(address _owner, address _spender) public view returns
      (uint256 remaining);
   event Transfer(address indexed _from, address indexed _to, uint256 _value);
   event Approval(address indexed _owner, address indexed _spender, uint256 _value);
}
```

ERC-20のデータ構造

ERC-20の実装を調べると、2つのデータ構造が含まれていることがわかります。ひとつは残高を追跡するもの、もうひとつはアロワンスを追跡するものです。Solidityでは*データマッピング*を使用して実装されています。

最初のデータマッピングは、所有者ごとのトークン残高の内部テーブルを実装します。トークンコントラクトはこれにより、誰がトークンを所有しているかを追跡できます。各送金は、ある

残高からの差し引きと別の残高への加算です。

```
mapping(address account => uint256) _balances;
```

2番目のデータ構造はアロワンスのデータマッピングです。次のセクションで説明するように、ERC-20トークンではトークンの所有者が支出者に権限を委任し、所有者の残高から特定の金額（アロワンス）を使用することを許可できます。ERC-20コントラクトは2次元のマッピングでアロワンスを追跡します。主キーはトークン所有者のアドレスで、支出者アドレスとアロワンスの金額にマッピングされます。

```
mapping(address account => mapping(address spender => uint256)) public _allowances;
```

ERC-20のワークフロー:「transfer」と「approveとtransferFrom」

ERC-20トークン標準には2つの送金関数があります。なぜ2つあるのか疑問に思うかもしれません。

ERC-20では2つの異なるワークフローが可能です。最初のワークフローは、`transfer`関数を使用する単純な単一トランザクションのワークフローです。このワークフローは、ウォレットが他のウォレットにトークンを送信する際に使用されます。トークントランザクションの大多数は`transfer`ワークフローで行われます。

transferコントラクトの実行は非常に単純です。アリスがボブに10トークンを送りたい場合、アリスのウォレットはトークンコントラクトのアドレスにトランザクションを送信し、ボブのアドレスと`10`を引数として`transfer`関数を呼び出します。トークンコントラクトはアリスの残高（−10）とボブの残高（+10）を調整し、`Transfer`イベントを発行します。

2番目のワークフローは、`approve`に続いて`transferFrom`を使用する2トランザクションのワークフローです。このワークフローにより、トークンの所有者は自身の管理権を別のアドレスに委任できます。最もよく使われるのはトークンの配布のためにコントラクトに管理権を委任するケースですが、取引所でも使用できます。たとえば企業がICOのためにトークンを販売する場合、クラウドセールコントラクトのアドレスに対して一定数のトークンの`approve`を行えます。クラウドセールコントラクトはトークンコントラクトの所有者の残高から各購入者へ`transferFrom`で送金できます（図10-1参照）。

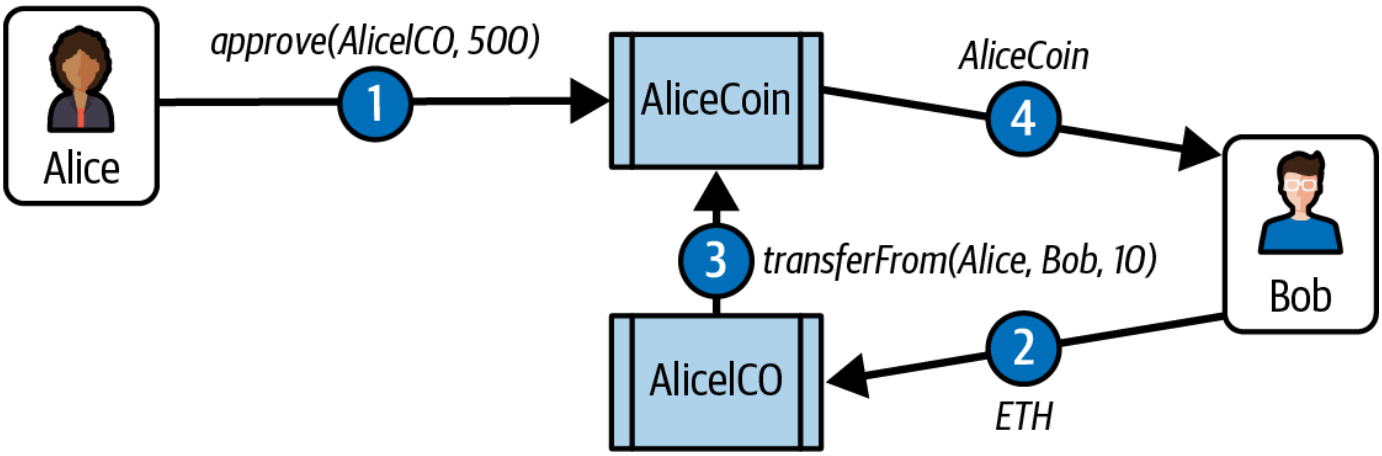


図10-1 ERC-20トークンの2段階の`approve`と`transferFrom`ワークフロー

> **注記**
>
> *イニシャルコインオファリング*（ICO）は、企業や組織がトークンを販売して資金を調達するクラウドファンディングの仕組みである。この用語は、公開企業が証券取引所で投資家に株式を売り出すプロセスである*新規株式公開*（IPO）に由来する。高度に規制されたIPO市場とは異なり、ICOはオープンでグローバルであり、混沌としている。

`approve`と`transferFrom`のワークフローでは、2つのトランザクションが必要です。アリスが`AliceICO`コントラクトにAliceCoinトークンの全体の50%を購入者（ボブやチャーリーなど）に販売することを許可したいとしましょう。まず、アリスは`AliceCoin` ERC-20コントラクトをローンチし、すべてのAliceCoinを自分のアドレスに発行します。次に、アリスはトークンをイーサと交換して販売できる`AliceICO`コントラクトをローンチします。その後、アリスは`approve`と`transferFrom`のワークフローを開始します。アリスは`AliceCoin`コントラクトにトランザクションを送信し、`AliceICO`コントラクトのアドレスと`totalSupply`の50%を引数として`approve`を呼び出します。これにより`Approval`イベントがトリガーされます。これで`AliceICO`コントラクトはAliceCoinを販売できるようになります。

`AliceICO`コントラクトがボブからイーサを受け取ると、見返りにAliceCoinをボブに送る必要があります。`AliceICO`コントラクト内にはAliceCoinとイーサの交換レートがあります。アリスが`AliceICO`コントラクト作成時に設定した交換レートにより、`AliceICO`コントラクトに送信されたイーサの量に対してボブが受け取るトークン数が決まります。`AliceICO`コントラクトがAliceCoinの`transferFrom`関数を呼び出す際、アリスのアドレスを送信者、ボブのアドレスを受信者として設定し、交換レートを使用して`value`フィールドでボブに送金されるAliceCoinトークンの数を決定します。`AliceCoin`コントラクトはアリスのアドレスからボブのアドレスへ残高を送金し、`Transfer`イベントをトリガーします。`AliceICO`コントラクトは、アリスが設定した承認上限を超えない限り、`transferFrom`を無制限に呼び出すことができます。`AliceICO`コントラクトは`allowance`関数を呼び出すことで、販売可能なAliceCoinトークンの数を追跡できます。

ERC-2612: 「permit」によるガスレス送金

第9章では、ERC-20トークンの従来の`transfer`と`transferFrom`フローの詳細を検証しました。これらの方式はトークン送金の基盤として機能してきましたが、限界がないわけではありません。どちらも送信者がブロックチェーンと直接やり取りする必要があり、ガス代を賄うためにネイティブの暗号通貨を手元に持っていなければなりません。トークンがネイティブ資金を持たないまったく新しいアドレスに送信される場合、これは大きな障壁となります。ストレスの溜まる体験であり、理想的とは程遠いものです。

ここでERC-2612が登場します。ERC-2612はERC-20トークン標準への巧みな追加機能であり、ユーザーがブロックチェーンに直接触れることなくトークンの送金を承認できるようにします。仕組みはこうです。送金を承認するためにオンチェーントランザクションを送信する代わりに、受信者のアドレス、トークン数、有効期限、nonceなどの必要なデータにウォレットで署名するだけです。これにより署名が作成され、送金を実行する必要がある人（受信者であれ他の当事者であれ）はその署名をトークンコントラクトの`permit`メソッドに提出できます。コントラクトは署名を読み取り、検証し、承認を処理します。最初のステップでガスを支払う必要はありません。効率的で安全であり、プロセスから多くの手間を省きます。

ERC-2612が機能するにはトークン開発者がERC-20コントラクトを拡張してこの機能を含める必要があります。導入されればユーザーに2つの主要なメリットを提供します。第一にプロセス全体が簡素化されます。すべての送金を個別に承認する代わりに、ひとつの署名で許可を与えられます。第二に必要なトランザクション数が削減されるため、ガスコストが節約されます。

ERC-20の実装

ERC-20と互換性のあるトークンは約30行のSolidityコードで実装可能ですが、ほとんどの実装はより複雑です。潜在的なセキュリティ脆弱性に対処するためです。EIP-20標準はConsensysとOpenZeppelinによって開発された2つの実装に言及しています。ConsensysのEIP-20トークンは2018年以降メンテナンスされていませんが、OpenZeppelinのERC-20トークンが開発者にとってのデファクトスタンダードとなり、積極的にメンテナンスされています。この実装は資金調達上限、トークン化されたボールト、ベスティングスケジュールなど、より複雑なERC-20互換トークンを実装するOpenZeppelinライブラリの基盤となっています。

独自のERC-20トークンのローンチ

独自のトークンを作成してローンチしましょう。この例ではFoundryフレームワークを使用します。この例は、Foundryをすでにインストールして設定済みであり、基本的な操作に精通していることを前提としています。

トークンの名前は「Mastering Ethereum Token」、シンボルはMETとします。まず、以下のコマンドでFoundryプロジェクトディレクトリを作成して初期化しましょう。

```
$ mkdir METoken
$ cd METoken
$ forge init
```

以下のディレクトリ構造が作成されるはずです。

```
METoken/
├─── foundry.toml
├─── lib
│    └─── forge-std
│         └─── ...
├─── README.md
├─── script
│    └─── Counter.s.sol
├─── src
│    └─── Counter.sol
└─── test
     └─── Counter.t.sol
```

`Counter`はFoundryのデフォルトのサンプルコントラクトで、独自のテストとデプロイスクリプトが付属しています。トークンコントラクトのスペースを確保するため、関連ファイルをすべて削除します。

この例では、Solidityベースのトークンの業界標準であるOpenZeppelinライブラリをインポートします。

```
$ forge install OpenZeppelin/openzeppelin-contracts
[...]
Installed openzeppelin-contracts v5.2.0
```

*METoken/lib/openzeppelin-contracts/contracts*内に、すべてのOpenZeppelinコントラクトが表示されます。OpenZeppelinライブラリにはERC-20トークンよりもはるかに多くのものが含まれていますが、ここではそのごく一部のみを使用します。

次に、トークンコントラクトを書きましょう。新しいファイル*METoken.sol*を作成し、例10-1のコードをコピーします。OpenZeppelinライブラリからすべての機能を継承するため、コントラクトは非常にシンプルです。

例10-1 METoken.sol: ERC-20トークンを実装するSolidityコントラクト

```
pragma solidity 0.8.28;
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
contract METoken is ERC20 {
    constructor(uint256 initialSupply) ERC20("METoken", "MET") {
        _mint(msg.sender, initialSupply);
```

```
    }
}
```

ここでは、ERC-20コントラクトのコンストラクタに名前として`"METoken"`、シンボルとして`"MET"`を渡しています。トークンの初期供給量はデプロイ時にコンストラクタのパラメータとして提供され、このトークンコントラクトのデプロイヤー（`msg.sender`）に送られます。decimalsにはデフォルト値の18を使用しています。これはERC-20トークンで広く採用されている標準です。

Foundryを使用して`METoken`コントラクトをコンパイルできます。

```
$ forge build
[⠒] Compiling...
[⠢] Compiling 6 files with Solc 0.8.28
[⠆] Solc 0.8.28 finished in 36.18ms
Compiler run successful!
```

`METoken`コントラクトをブロックチェーンにデプロイするためのデプロイスクリプトを設定しましょう。*METoken/script*フォルダに新しいファイル*METokenDeploy.s.sol*を作成し、以下のコードをコピーします。

```
pragma solidity 0.8.28;
import {Script, console} from "forge-std/Script.sol";
import {METoken} from "../src/METoken.sol";
contract METokenDeployer is Script {
    METoken public _METoken;
    function run() public {
        vm.startBroadcast();
        _METoken = new METoken(50_000_000e18);
        vm.stopBroadcast();
    }
}
```

この例では、初期供給量として5000万を渡しています。1e18を掛けていることに気づきましたか。それがトークンのdecimalsです。*X*トークンの残高を持つためには、*X* × 10 ^ decimals のトークン数量が必要であることを思い出してください。

> **注記**
>
> *スクリプトの接尾辞*.s.sol*は、ファイルの目的を素早く識別するためのFoundryの命名規則である。要件ではなく、*scriptフォルダ内にスクリプトファイルを配置するだけで十分であるが、開発中に役立つ非常に良い慣行である。テストの*.t.sol*接尾辞にも同様のことが当てはまる。

イーサリアムのテストネットワークにデプロイする前に、ローカルブロックチェーンを起動してすべてをテストしましょう。Foundryツールボックスのもうひとつのツールであるローカルのイーサリアム開発ノードAnvilを使用します。使い方は簡単で、新しいターミナルを開いて`anvil`と入力するだけです。コンソールに利用可能なアカウント、秘密鍵、チェーンID、RPC URLなどの情報のリストが表示されます。RPC URLは、Foundry（または任意のイーサリアムクライアント）がローカルブロックチェーンノードと通信し、トランザクション、コントラクトのデプロイ、データの取得を可能にするエンドポイントです。AnvilのデフォルトRPCは*http://127.0.0.1:8545*です。デプロイスクリプトにローカルブロックチェーンへのデプロイを指示するには、AnvilのRPC URLをコンソールパラメータとしてフラグ`--rpc-url "http://127.0.0.1:8545"`で提供する必要があります。

最後に必要なのはデプロイヤーアカウントの秘密鍵です。ローカルブロックチェーンなので、イーサリアムのメインネットやテストネットで使用するアドレスにはここでは資金がなく、実際の秘密鍵を不必要にさらしたくありません。代わりにAnvil起動時に生成されるテストアカウントを使用します。これらのアカウントにはローカルチェーン上に10,000 ETHがプリロードされており、開発とテストに最適です。

以下を実行してトークンをデプロイする準備が整いました。

```
$ forge script script/METokenDeploy.s.sol --broadcast --rpc-url "http://127.0.0.1:8545"
--private-key <DEPLOYER_PRIVATE_KEY>
[⠒] Compiling...
No files changed, compilation skipped
Script ran successfully.
```

注記

`--broadcast`フラグが何のためにあるのか疑問に思うかもしれない。Foundryに実際にブロックチェーンにトランザクションをブロードキャストするよう指示するためである。このフラグがなければ、Foundryはトランザクションをシミュレートするだけである。

コンソール出力は、デプロイスクリプトが正常に実行されたことを通知しています。Anvilを実行しているターミナルを見ると、コントラクト作成を含む多くのアクティビティが確認できます。

```
  Transaction: 0xd01e3a90e1f2ee60112658e92f4ebf04c24df67d2ec1315cfb79d145729d15ec
   Contract created: 0x5FbDB2315678afecb367f032d93F642f64180aa3
   Gas used: 941861
   Block Number: 1
   Block Hash: 0x748b6058dea932317cacf45bb63be82f253554f359b97ace224e35979a92b00a
   Block Time: "Fri, 31 Jan 2025 19:10:42 +0000"
```

METokenは以下のアドレスに正常にデプロイされました。

```
0x5FbDB2315678afecb367f032d93F642f64180aa3
```

また、forgeの`create`コンソールコマンドを使用してトークンをデプロイすることもできます。

```
$ forge create METoken --broadcast --rpc-url http://127.0.0.1:8545 --private-key
<DEPLOYER_PRIVATE_KEY> --constructor-args 5000000000000000000000000
```

ここでは、decimalsを考慮して総供給量がコンストラクタ引数として渡されています。

METokenとのやり取り

コントラクトとのやり取りにはいくつかの方法があります。Remix（第2章で使用したもの）、FoundryのChiselのようなSolidity REPL、またはethers.jsのようなJavaScriptライブラリを使用できます。Foundryスクリプトを使用してトランザクションを実行することもでき、この例ではそれを使用します。

イーサリアムアドレスは40文字の16進数文字列であり、読みやすいとは言えません。例をわかりやすくするため、使用する2つのアドレスにニックネームを割り当てます。METコントラクトをデプロイしたアドレスをDeployer、2番目のアドレスをアリスとします。アリスにもAnvilのプリファンドアドレスのひとつを使用します。

Foundryスクリプトを作成して、DeployerのMEToken残高を確認し、アリスにMETokenを送信しましょう。以下のコードスニペットの内容をコピーして、新しいファイル*METoken/script/METokenInteraction.s.sol*に貼り付けます。

```
pragma solidity 0.8.28;
import {Script, console} from "forge-std/Script.sol";
import {METoken} from "../src/METoken.sol";
contract METokenInteraction is Script {
    METoken public _METoken = METoken(0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512);
    address alice = 0x70997970C51812dc3A010C7d01b50e0d17dc79C8;
    function run() public {
        vm.startBroadcast();
        uint256 ourBalance = _METoken.balanceOf(msg.sender);
        console.log("Deployer initial balance:", ourBalance);
        uint256 aliceBalance = _METoken.balanceOf(alice);
        console.log("Alice initial balance:", aliceBalance);
        uint256 amountToTransfer = 50e18;
        bool success = _METoken.transfer(alice, amountToTransfer);
        if (success) {
            console.log("Transfer successful");
        } else {
            console.log("Transfer failed");
            revert();
        }
```

```
        ourBalance = _METoken.balanceOf(msg.sender);
        console.log("Deployer final balance:", ourBalance);
        aliceBalance = _METoken.balanceOf(alice);
        console.log("Alice final balance:", aliceBalance);
        vm.stopBroadcast();
    }
}
```

以下のコンソールコマンドでスクリプトを実行できます。

```
$ forge script script/METokenInteraction.s.sol --private-key <DEPLOYER_PRIVATE_KEY>
--rpc-url "http://127.0.0.1:8545" -vv
```

デプロイ時に使用したものと同じ秘密鍵を使用することが重要です。これにより、`msg.sender`が初期トークン供給量を保有するデプロイヤーアドレスに対応することが保証されます。

> **注記**
>
> Foundryの`-v`フラグは、`forge script`や`forge build`などのコマンド実行時の出力の詳細度を制御する。`v`の数を増やすと、より多くの情報を含むように出力の詳細度が上がる。コンソールログを表示するには少なくとも`-vv`が必要であり、最大の詳細度は`-vvvv`で提供される。

スクリプトを実行すると、コンソールに以下が出力されます。

```
[⠊] Compiling...
[⠒] Compiling 1 files with Solc 0.8.28
[⠢] Solc 0.8.28 finished in 312.39ms
Compiler run successful!
Script ran successfully.
== Logs ==
  Deployer initial balance: 500000000000000000000000000
  Alice initial balance: 0
  Transfer successful
  Deployer final balance: 499999500000000000000000000
  Alice final balance: 50000000000000000000
```

このスクリプトでは、まずデプロイヤーとアリスの現在のトークン残高をログに記録します。次に、デプロイヤーからアリスに50トークンを送金し、再度残高をログに記録します。トークンのdecimalsが18であるため、50トークンは50e18として表現されることに注意してください。そのためゼロが大量に並びます。

ERC-20トークンのコントラクトアドレスへの送信

ここまで、ERC-20トークンのセットアップと、あるアカウントから別のアカウントへのトークン送金を行いました。これらのデモンストレーションで使用したアカウントはすべてEOA、つまり秘密鍵によって制御されるアカウントであり、コントラクトではありません。METトークンをコントラクトアドレスに送信するとどうなるでしょうか。確認してみましょう。

まず、テスト環境に別のコントラクトをデプロイしましょう。この例では、以下の*NaiveFaucet.sol*コントラクトを使用します。

```
pragma solidity 0.8.28;
contract NaiveFaucet {
    receive() external payable {}
    // コントラクトからイーサを引き出す関数
    function withdraw(uint256 amount) public {
        require(amount <= address(this).balance, "Insufficient balance in faucet");
        payable(msg.sender).transfer(amount);
    }
}
```

ディレクトリは以下のようになります。

```
METoken/
+---- src
|   +---- NaiveFaucet.sol
|   +---- METoken.sol
```

`NaiveFaucet`コントラクトをコンパイルしてデプロイしましょう。

```
$ forge create NaiveFaucet --broadcast --rpc-url http://localhost:8545
--private-key <DEPLOYER_PRIVATE_KEY>
[⠒] Compiling...
[⠒] Compiling 1 files with Solc 0.8.28
[⠒] Solc 0.8.28 finished in 8.69ms
Compiler run successful!
Deployer: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
Deployed to: 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
Transaction hash: 0x4d1947547e3cfec8db670f3c1b7ff309b41de8aacee42165578a3ddf8619f63f
```

`NaiveFaucet`コントラクトがアドレス`0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0`にデプロイされました。では、以下のスクリプトを*METoken/script/METokenSend.s.sol*にコピーして、`NaiveFaucet`コントラクトにMETを送信しましょう。

```
pragma solidity 0.8.28;
import {Script, console} from "forge-std/Script.sol";
import {METoken} from "../src/METoken.sol";
```

```
contract METokenSend is Script {
    METoken public _METoken = METoken(0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512);
    address naiveFaucet = 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0;
    function run() public {
        vm.startBroadcast();
        uint256 amountToSend = 100e18;
        bool success = _METoken.transfer(naiveFaucet, amountToSend);
        if (success) {
            console.log("Transfer successful");
        } else {
            console.log("Transfer failed");
            revert();
        }
        uint256 faucetBalance = _METoken.balanceOf(naiveFaucet);
        console.log("Faucet balance:", faucetBalance);
        vm.stopBroadcast();
    }
}
```

以下で実行できます。

```
$ forge script script/METokenSend.s.sol --private-key <DEPLOYER_PRIVATE_KEY>
--rpc-url "http://127.0.0.1:8545" -vv
[⠊] Compiling...
[⠒] Compiling 1 files with Solc 0.8.28
[⠊] Solc 0.8.28 finished in 413.41ms
Compiler run successful!
Script ran successfully.
== Logs ==
  Transfer successful
  Faucet balance: 100000000000000000000
```

ここでも送金を開始するアドレスであるため、デプロイヤーの秘密鍵を使用する必要があります。

`NaiveFaucet`コントラクトに100 METを送金しました。では、これらのトークンをどのように引き出すのでしょうか。

*NaiveFaucet.sol*は非常にシンプルなコントラクトであることを思い出してください。`withdraw`という*イーサ*を引き出すための関数がひとつあるだけです。METやその他のERC-20トークンを引き出す関数はありません。`withdraw`を使用するとイーサの送信を試みますが、`NaiveFaucet`にはまだイーサの残高がないため失敗します。

`METoken`コントラクトは`NaiveFaucet`が残高を持っていることを知っていますが、その残高を送金できるのはコントラクトのアドレスから`transfer`呼び出しを受け取った場合のみです。何らかの方法で`NaiveFaucet`コントラクトに`METoken`の`transfer`関数を呼び出させる必要があります。

次に何をすべきか考えているなら、やめてください。この問題に解決策はありません。`NaiveFaucet`に送信されたMETは永久にスタックしたままです。`NaiveFaucet`コントラクトだけがMETを送金できますが、`NaiveFaucet`コントラクトにはERC-20トークンコントラクトの`transfer`関数を呼び出すコードがありません。

おそらくこの問題を予見していたかもしれません。しかし予見していなかった可能性の方が高いでしょう。実際、何百人ものイーサリアムユーザーがERC-20機能を持たないコントラクトに誤ってさまざまなトークンを送金してきました。長年にわたり驚くべき数百万ドル相当の資金がこのようにして「スタック」し、永久に失われてきました。

「approveとtransferFrom」ワークフローのデモンストレーション

`NaiveFaucet`コントラクトはERC-20トークンを処理できませんでした。`transfer`関数を使用してトークンを送信したことでトークンが失われてしまいました。ここでコントラクトを書き直してERC-20トークンを処理できるようにしましょう。リクエストした誰にでもMETを配布するフォーセットに改造します。

新しいフォーセットコントラクト*METFaucet.sol*は、例10-2のようになります。

例10-2 METFaucet.sol: METoken用のフォーセット

```
pragma solidity 0.8.28;
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
contract METFaucet {
    IERC20 public _METoken;
    address public _METOwner;
    constructor(address _metokenAddress, address metOwner) {
        _METoken = IERC20(_metokenAddress);
        _METOwner = metOwner;
    }
    // コントラクトからMETokenを引き出す関数
    function withdraw(uint256 amount) public {
        require(amount <= 10e18, "At most 10 MET");
        require(_METoken.transferFrom(_METOwner, msg.sender, amount), "Transfer failed");
    }
}
```

基本的な`Faucet`の例からかなり多くの変更を加えました。`METFaucet`は`METoken`の`transferFrom`関数を使用するため、2つの追加変数が必要です。ひとつは`METoken`コントラクトのアドレスを保持し、もうひとつはフォーセットからの引き出しを承認するMETの所有者のアドレスを保持します。この場合の所有者は初期供給量を受け取ったデプロイヤーです。`METFaucet`コントラクトは`METoken.transferFrom`を呼び出し、所有者からフォーセットの引き出しリクエスト元のアドレスへMETを移動するよう指示します。

これら2つの変数を以下のように宣言します。

```
IERC20 public _METoken;
address public _METOwner;
```

フォーセットは`METoken`と`METOwner`の正しいアドレスで初期化する必要があるため、カスタムコンストラクタを宣言する必要があります。

```
// METFaucetコンストラクタ - METokenコントラクトのアドレスと
// transferFromの承認を受ける所有者アドレスを提供する
constructor(address _metokenAddress, address metOwner) {
    _METoken = IERC20(_metokenAddress);
    _METOwner = metOwner;
}
```

次の変更は`withdraw`関数です。`transfer`を呼び出す代わりに、`METFaucet`は`METoken`の`transferFrom`関数を使用し、`METoken`にフォーセットの受信者へMETを送金するよう依頼します。

```
// METokenのtransferFrom関数を使用する
_METoken.transferFrom(metOwner, msg.sender, withdraw_amount);
```

最後に、フォーセットはもうイーサを送信しないので、誰かが`METFaucet`にイーサを送信してスタックするのを防ぐべきでしょう。着信するイーサを拒否するにはコントラクトから`receive`関数を削除するだけで十分です。

*METFaucet.sol*のコードの準備ができたので、METトークンのアドレスとそのデプロイヤーをアドレスパラメータとして提供してデプロイできます。

```
$ forge create METFaucet --broadcast --rpc-url http://localhost:8545 --private-key
<DEPLOYER_PRIVATE_KEY> --constructor-args
"0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512""0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266"
[⠒] Compiling...
No files changed, compilation skipped
Deployer: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
Deployed to: 0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9
Transaction hash: 0xa8bfbde9489ee40d41328a80538d0d3e7778b7f3b896c1d51897bf85bb25cec2
```

`METFaucet`コントラクトが`0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9`にデプロイされ、テストの準備がほぼ整いました。まず、`METFaucet`コントラクトが所有者のMETトークンを使用できるようにする*METApprove.s.sol*スクリプトを作成しましょう。

```
pragma solidity 0.8.28;
import {Script, console} from "forge-std/Script.sol";
import {METoken} from "../src/METoken.sol";
contract METApprove is Script {
    METoken public _METoken = METoken(0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512);
```

```
    address _METFaucet = 0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9;
    function run() public {
        vm.startBroadcast();
        bool success = _METoken.approve(_METFaucet, type(uint256).max);
        if (success) {
            console.log("Approve successful");
        } else {
            console.log("Approve failed");
            revert();
        }
        vm.stopBroadcast();
    }
}
```

以下で実行できます。

```
$ forge script script/METApprove.s.sol --broadcast --private-key <DEPLOYER_PRIVATE_KEY>
--rpc-url "http://127.0.0.1:8545" -vv
[⠒] Compiling...
[⠃] Compiling 2 files with Solc 0.8.28
[⠊] Solc 0.8.28 finished in 327.90ms
Compiler run successful!
Script ran successfully.
== Logs ==
  Approve successful
```

次に、セカンダリアドレスが`METFaucet`コントラクトとやり取りして10 METトークンを引き出し、操作の前後の残高をログに記録するスクリプトを作成しましょう。例10-3のように*METFaucetWithdraw.s.sol*スクリプトを作成します。

例10-3 METFaucetWithdraw: フォーセット引き出しスクリプト

```
pragma solidity 0.8.28;
import {Script, console} from "forge-std/Script.sol";
import {METoken} from "../src/METoken.sol";
import {METFaucet} from "../src/METFaucet.sol";
contract METFaucetWithdraw is Script {
    METoken public _METoken = METoken(0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512);
    METFaucet public _METFaucet = METFaucet(0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9);
    function run() public {
        vm.startBroadcast();
       uint256 balanceBefore = _METoken.balanceOf(msg.sender);
       console.log("Alice balance before:", balanceBefore);
       _METFaucet.withdraw(10e18);
       uint256 balanceAfter = _METoken.balanceOf(msg.sender);
       console.log("Alice balance after:", balanceAfter);
        vm.stopBroadcast();
```

```
    }
}
```

アリスのアドレスからこのスクリプトを実行するには、スクリプト実行時にアリスの秘密鍵を提供します。

```
$ forge script script/METFaucetWithdraw.s.sol --broadcast --private-key <ALICE_PRIVATE_KEY>
--rpc-url "http://127.0.0.1:8545" -vv
[⠒] Compiling...
[⠃] Compiling 1 files with Solc 0.8.28
[⠊] Solc 0.8.28 finished in 330.97ms
Compiler run successful!
Script ran successfully.
== Logs ==
  Alice balance before: 0
  Alice balance after: 10000000000000000000
```

結果からわかるように、`approve`と`transferFrom`のワークフローを使用して、あるコントラクトに別のトークンコントラクトで定義されたトークンの送金を承認できます。適切に使用すればERC-20トークンはEOAと他のコントラクトの両方で使用できます。ただしERC-20トークンを正しく管理する責任はユーザーインターフェースに委ねられています。ユーザーがERC-20トークンをコントラクトアドレスに誤って送金しようとし、そのコントラクトがERC-20トークンを受け取る機能を備えていない場合、トークンは失われます。

ERC-20トークンの問題点

ERC-20トークン標準の採用はまさに爆発的でした。何千ものトークンがローンチされ、新しい機能の実験やさまざまな「クラウドファンディング」オークションおよびICOでの資金調達に使用されました。しかしコントラクトアドレスへのトークン送金の問題で見たように、いくつかの潜在的な落とし穴があります。

ERC-20トークンの目立ちにくい問題のひとつは、トークンとイーサ自体の間の微妙な違いが露呈することです。イーサは受信者アドレスを宛先とするトランザクションで送金されますが、トークンの送金は*特定のトークンコントラクトの状態*内で行われ、受信者のアドレスではなくトークンコントラクトが宛先となります。トークンコントラクトは残高を追跡しイベントを発行します。トークンの送金では受信者に実際にトランザクションが送られるわけではありません。受信者のアドレスがトークンコントラクト内のマッピングに追加されるだけです。イーサをアドレスに送信するトランザクションはアドレスの状態を変更しますが、トークンをアドレスに送金するトランザクションはトークンコントラクトの状態のみを変更し、受信者アドレスの状態は変更しません。ERC-20トークンをサポートするウォレットであっても、ユーザーが特定のトークンコントラクトを明示的に「ウォッチ」対象に追加しない限りトークン残高を認識しません。一部のウォレットは管理するアドレスの残高を検出するために人気のあるトークンコントラクトをウォッチしますが、既存のERC-20コントラクトのごく一部に限られます。

実際、ユーザーがすべてのERC-20トークンコントラクトのすべての残高を追跡*したい*とは考えにくいでしょう。多くのERC-20トークンは使用可能なトークンというよりもメールスパムに近いものです。イーサのアクティビティがあるアカウントに対して、ユーザーを引き付けるために残高を自動的に作成します。特にプレセール時に作成された長いアクティビティ履歴を持つイーサリアムアドレスがあれば、どこからともなく現れた「ジャンク」トークンでいっぱいになっているでしょう。もちろんアドレスは実際にはトークンでいっぱいではなく、あなたのアドレスを含んでいるのはトークンコントラクトの方です。これらの残高が見えるのは、アドレスの閲覧に使用するブロックエクスプローラーやウォレットがこれらのトークンコントラクトをウォッチしている場合のみです。

トークンはイーサと同じようには振る舞いません。イーサは`send`関数で送信され、コントラクト内の任意のpayable関数または任意の外部所有アカウントに受け入れられます。トークンはERC-20コントラクトにのみ存在する`transfer`、または`approve`と`transferFrom`の関数で送信され、（少なくともERC-20では）受信コントラクト内のpayable関数をトリガーしません。トークンはイーサのような暗号通貨と同じように機能することを意図していますが、その幻想を壊すいくつかの違いがあります。

`approve`と`transferFrom`のパターンに注目しましょう。新しいユーザーにとって`approve`-`transferFrom`システムは特に誤解を招きやすいものです。多くのユーザーはトークンの送金が単一の操作であると想定しているため、2段階のプロセスに遭遇すると混乱し直感に反すると感じます。さらに悪いことに`approve`アクションは無害に見えますが、罠になり得ます。ユーザーがコントラクトを承認する際、知らないうちに無制限の権限を付与してしまい、悪意のあるアクターが後からトークンを流出させる恐れがあります。この理解のギャップがフィッシング攻撃を容易にし、システムの設計が新規参入者の期待と一致していないことを浮き彫りにしています。

別の問題も考えてみましょう。イーサを送信したりイーサリアムのコントラクトを使用したりするにはガス代を支払うためにイーサが必要です。トークンを送信するにも*イーサが必要です*。トークンでトランザクションのガス代を支払うことはできず、トークンコントラクトがガス代を代わりに支払うこともできません。たとえば取引所でビットコインをあるトークンに交換するとしましょう。そのトークンコントラクトを追跡し残高を表示するウォレットでトークンを「受け取り」ます。ウォレット内の他の暗号通貨と同じように見えます。しかしそのトークンを送信しようとするとウォレットはイーサが必要であることを通知します。混乱するかもしれません。トークンを受け取るときにはイーサは必要なかったからです。イーサを持っていないかもしれませんし、そのトークンがイーサリアム上のERC-20トークンであることすら知らなかったかもしれません。独自のブロックチェーンを持つ暗号通貨だと思っていた可能性もあります。幻想が崩れた瞬間です。

ERC-2612の`permit`関数やEIP-4337・EIP-7702のようなスマートウォレットが提供するガス代スポンサーシップ機能により、この問題を部分的に解決できます。ERC-2612ではオフチェーンメッセージに署名してトークンのアロワンスを承認でき、オンチェーンの承認トランザクションを省略できます。EIP-4337やその他のスマートウォレットアーキテクチャでは、第三者がERC-20トークンでの払い戻しと引き換えにガス代を肩代わりできます。課題はトークン間でのERC-2612の採用が限定的であることと、スマートウォレットがまだ広く普及していないことです。

これらの問題のいくつかはERC-20トークンに固有のものです。それ以外はイーサリアム内の抽象化やインターフェースの境界に関連するより一般的な問題です。トークンインターフェースの変更で解決できるものもあれば、イーサリアムの基本構造（EOAとコントラクトの区別、トランザクションとメッセージの区別など）の変更が必要なものもあります。正確には「解決」できないものもあり、基礎的な違いに関わらずユーザーエクスペリエンスを一貫させるユーザーインターフェース設計が必要になるかもしれません。

以下のセクションでは、これらの問題のいくつかに対処しようとするさまざまな提案を見ていきます。

ERC-223: 提案されたトークンコントラクトインターフェース標準

ERC-223の提案は、送信先アドレスがコントラクトかどうかを検出し、コントラクトへのトークンの不注意な送金の問題を解決しようとするものです。ERC-223ではトークンを受け取るように設計されたコントラクトが`tokenFallback`という名前の関数を実装することを要求します。送金先がコントラクトで、そのコントラクトがトークンをサポートしていない場合（つまり`tokenFallback`を実装していない場合）、送金は失敗します。

送信先アドレスがコントラクトかどうかを検出するために、ERC-223のリファレンス実装は、巧妙な方法でインラインバイトコードの小さなセグメントを使用します。

```
function isContract(address _addr) private view returns (bool is_contract) {
  uint256 length;
    assembly {
       // ターゲット アドレスに格納されたコードのサイズを取得する。アセンブリが必要
       length := extcodesize(_addr)
    }
    return (length>0);
}
```

注記

`extcodesize`は、指定されたアドレスに格納されているバイトコードのサイズを返す。歴史的に、EOAとスマートコントラクトの主な違いはこの点にあった。EOAにはコードがなく、コントラクトにはあった。しかし、その前提はもはや成立しない。EIP-7702により、EOAにもコードをアタッチできるようになり、境界は完全に曖昧になった。 重要なエッジケースもある。コントラクトのコンストラクタフェーズでは、コードはまだオンチェーンに格納されていない。EVMはコンストラクタの実行完了後にのみコントラクトのバイトコードをアドレスに書き込む。そのため、コンストラクタ内からコントラクト自身のアドレス（またはデプロイがまだ完了していない別のコントラクト）に対して`extcodesize`を呼び出すと、そのアドレスに最終的にコードが含まれるにもかかわらず`0`が返される。結果として、アドレスにコードがない場合、EOAかまだ構築中のコントラクトの可能性がある。コードがある場合は、デプロイ済みのコントラクトかカスタムコードペイロードを使用す

るEOAのいずれかの可能性がある。要するに、このチェックではアドレスがコントラクトかどうかを信頼性をもって判定することはもはやできない。

ERC-223のコントラクトインターフェース仕様は以下のとおりです。

```
interface ERC223Token {
  uint256 public totalSupply;
  function balanceOf(address who) public view returns (uint256);
  function name() public view returns (string _name);
  function symbol() public view returns (string _symbol);
  function decimals() public view returns (uint8 _decimals);
  function totalSupply() public view returns (uint256 _supply);
  function transfer(address to, uint256 value) public returns (bool success);
  function transfer(address to, uint256 value, bytes data) public returns (bool success);
  function transfer(address to, uint256 value, bytes data, string custom_fallback)
      public returns (bool success);
  event Transfer(address indexed from, address indexed to, uint256 value,
                 bytes indexed data);
}
```

ERC-223は広く実装されておらず、ERCのディスカッションスレッドでは、後方互換性やコントラクトインターフェースレベルでの変更の実装とユーザーインターフェースでの変更の実装のトレードオフについて議論が行われています。議論は続いています。

ERC-777: 実現し得た未来

ERC-777は、トークンのやり取りにフックという新しいアプローチをもたらします。フックはトークンの送金中にトリガーされる関数です。これらのフックはERC-20と完全に互換性があり、既存のシステムがERC-777トークンとシームレスにやり取りできることを保証します。

送信者のフック`tokensToSend`はトークンがアカウントから送出される前に実行され、ロギングや条件チェックなどのロジックを送信者が追加できるようにします。一方、受信者のフック`tokensReceived`はトークンがアカウントに着金した時に動作します。ERC-777のフックアーキテクチャの中核にあるのがERC-1820レジストリで、送信者の`tokensToSend`と受信者の`tokensReceived`という必要なフックをどのアドレスが実装しているかを追跡します。これにより両当事者が送金を処理する準備ができている場合にのみ送金が成功し、トークンの喪失などの一般的な問題を防止します。

フックはトランザクションも簡素化します。ERC-20では、コントラクトへのトークン送金にはしばしば面倒な2段階プロセス（`approve`してから`transferFrom`）が必要です。ERC-777はフックを通じてアトミックなトランザクションを可能にし、この手順を不要にします。効率的で直感的であり、率直に言って待ち望まれていた改善です。

ERC-777のもうひとつの興味深い機能はオペレーターメカニズムです。承認されたアドレス（多くの場合は取引所や決済処理業者などのスマートコントラクト）が保有者に代わってトークンを送信およびバーンできるようにします。保有者はいつでもオペレーターへの承認を付与または取り消すことができ、どの第三者が自分のトークンを管理できるかを完全にコントロールできます。すべての承認または取り消しはイベントを発行し、承認変更の可視性を提供します。

ERC-777のコントラクトインターフェース仕様は以下のとおりです。

```
interface ERC777Token {
    function name() public view returns (string);
    function symbol() public view returns (string);
    function totalSupply() public view returns (uint256);
    function granularity() public view returns (uint256);
    function balanceOf(address owner) public view returns (uint256);
    function send(address to, uint256 amount, bytes userData) public;
    function authorizeOperator(address operator) public;
    function revokeOperator(address operator) public;
    function isOperatorFor(address operator, address tokenHolder)
        public constant returns (bool);
    function operatorSend(address from, address to, uint256 amount,
                          bytes userData,bytes operatorData) public;
    event Sent(address indexed operator, address indexed from,
               address indexed to, uint256 amount, bytes userData,
               bytes operatorData);
    event Minted(address indexed operator, address indexed to,
                 uint256 amount, bytes operatorData);
    event Burned(address indexed operator, address indexed from,
                 uint256 amount, bytes userData, bytes operatorData);
    event AuthorizedOperator(address indexed operator,
                             address indexed tokenHolder);
    event RevokedOperator(address indexed operator, address indexed tokenHolder);
}
```

ERC-777トークンの問題点

ERC-777はトークンのやり取りをより直感的にする一方で、フックにはいくつかの重大な課題があります。中でも顕著なのはリエントランシー攻撃のリスクです。リエントランシー攻撃は、コントラクトが状態を完全に更新する前にロジックへ再入できる点を悪用するもので、深刻な結果をもたらすことがあります。ERC-777のフックはトークン送金中に送信者と受信者の両方に実行制御を渡すという性質上、こうした攻撃にとって理想的なシナリオを生み出します。そのため開発者は脆弱性を回避するためにこれらのフックを慎重に扱う必要があります。

ERC-777トークンの統合によるリエントランシーの最も悪名高い事例は2020年4月のUniswap v1のインシデントです。分散型取引所プロトコルであるUniswapがERC-777のフックにより意図せず準備金を攻撃にさらしてしまいました。攻撃者は操作の途中でUniswapコントラクトにコールバックし、トークンとイーサの準備金の不一致を悪用して資金を流出させました。この脆弱性

はUniswapとERC-777の連携方法に固有のものでしたが、フックがもたらすリスクを如実に示す出来事でした。

もうひとつ注目すべき問題はDoS攻撃の可能性です。たとえば、あるコントラクトが複数のアカウントにERC-777トークンを配布する場合を考えましょう。受信者のひとつが `tokensReceived` フック中にトランザクションをリバートするようプログラムされた悪意のあるコントラクトであれば、配布プロセス全体が阻止されてしまいます。

これらのリスクはERC-777の統合がERC-20の単純な置き換えではないことを浮き彫りにしています。開発者は脆弱性を軽減するためにリエントランシーロックなどのベストプラクティスを採用する必要があります。

実現し得た未来

ERC-777は、イーサリアムのトークン標準の次なる進化となり得たものでした。ユーザーエクスペリエンスとトークンの取り扱いに関するERC-20の多くの課題に対処しています。トークンの喪失を防止し、アトミックなトランザクションを可能にし、より豊かなコントラクトインタラクションを構築できることは、すべて説得力のある進歩です。

しかしイーサリアムコミュニティは躊躇しました。DoSやリエントランシーのシナリオの可能性が暗い影を落としたのです。開発者はリスクが適切な予防措置で管理可能であるにもかかわらず、ERC-777の統合の複雑さとリスクを恐れました。課題に立ち向かう代わりに、馴染み深いが限定的なERC-20にとどまったのです。トークンをイーサリアムの哲学により近づけながら優れた機能を提供する標準を受け入れる機会を逃したと言えます。

結局のところERC-777のフックが悪者なのではなく、不適切な実装が問題です。少しの努力と安全なコーディング慣行の遵守があれば、ERC-777はよりシームレスなイーサリアムエコシステムへの道を切り拓けるでしょう。この問題をさらに研究するにはOpenZeppelinライブラリによるERC-777の非推奨化に関する議論が参考になります。

ERC-721: NFT標準

ここまでERC-20のような代替可能トークンの標準を探求してきました。ERC-20では各単位が交換可能であり、システムが気にするのはアカウント残高のみです。ここからはまったく異なるものに踏み込みましょう。ERC-721、非代替性トークン（NFT）の標準です。2021年の熱狂の中でNFTのことを耳にしたことがあるかもしれません。CryptoPunks、Bored Ape Yacht Club、NBA Top Shotなどのデジタルコレクティブルが見出しを飾り、驚くべき金額で取引されました。OpenSeaやRaribleなどの取引プラットフォームは暗号資産の世界で誰もが知る名前になりました。

しかしNFTの何が特別なのでしょうか。ERC-20トークンとは異なり、NFTの本質はユニークさにあります。各トークンは固有のアイテムの所有権を表し、そのアイテムはデジタルコレクティ

ブル、ゲーム内アセット、アート作品、さらには不動産や自動車のような現実世界の資産など何でもあり得ます。ERC-721の強みはその柔軟性にあります。トークンが表すものがユニークであり番号で識別できる限り、何を表現しても構いません。内部的には各NFTが`uint256`型の識別子を持つことで実現されています。あるトークンを別のトークンと区別するシリアルナンバーのようなものと考えてください。ERC-20ではアカウントと保有量のマッピングで残高を追跡しますが、ERC-721では残高に加えて各固有のトークンIDとその所有者のマッピングも行います。

```
mapping (uint256 => address) private _owners;
```

構造のこの微妙な変化がすべてを変えます。各NFTは特定の所有者に紐づけられ、その履歴、来歴、固有の属性を直接追跡できます。だからこそ、ERC-721は個別性が重要なアプリケーションに最適なのです。唯一無二のアート作品の所有権証明、ゲーム内のレアな剣、メタバースの土地区画などがその例です。

ERC-721を真に強力で多用途にしているのはオプションのERC-721メタデータ拡張です。各トークンIDをURI（Uniform Resource Identifier）に紐づけることができます。このURIはNFTの名前、説明、画像などを記述するメタデータを指します。URIは中央集権型サーバーを指す標準的なHTTPリンクの場合もあれば、分散型ストレージのためのIPFS（Inter-Planetary File System）リンクの場合もあります。中央集権型サーバーはオフラインになったり完全に消滅したりする可能性があり、NFTをサードパーティのインフラに依存させます。一方IPFSはメタデータがアクセス可能かつ改ざん耐性を維持することを保証するため、NFTメタデータの保存に推奨される選択肢です。各トークンをメタデータURIに関連付ける機能によりNFTは豊かな説明データやマルチメディアコンテンツを持つことができ、そのユニークさと有用性がさらに高まります。

NFTの投機的な使用、特にデジタルコレクティブルとしての使用は紛れもなく世間の注目を集めてきました。これらのトークンはハイプの象徴となり、希少性、セレブリティの推薦、コミュニティの感情によって価値が牽引されました。しかし派手な見出しの向こう側には、産業を再定義する可能性を持つ実用的なアプリケーションの世界があります。たとえばNFTはサプライチェーン管理において商品の来歴を追跡し、消費者に真正性と透明性を保証するために使用できます。不動産の権利書を表現し所有権の移転を自動化して不正を減少させることで、不動産取引を効率化できます。教育分野では改ざん不可能で世界中からアクセスできる検証可能な資格情報（学位や証明書など）の発行に活用できます。

コレクティブルがNFTに注目を集めましたが、NFTの真の可能性はこうした実用的で変革的な用途にあります。ユニークさ、追跡可能性、プログラマビリティを組み合わせることでERC-721トークンはデジタル経済の基盤的な構成要素となる可能性を秘めています。風変わりなアート作品をミントする場合でも生命を救う医療記録をトークン化する場合でも、ERC-721は投機を超えたソリューションの構築を開発者に可能にします。

ERC-721のコントラクトインターフェース仕様は以下のとおりです。

```
interface ERC721 /* is ERC165 */ {
    event Transfer(address indexed _from, address indexed _to, uint256 _deedId);
    event Approval(address indexed _owner, address indexed _approved,
                   uint256 _deedId);
```

```
    event ApprovalForAll(address indexed _owner, address indexed _operator,
                         bool _approved);
    function balanceOf(address _owner) external view returns (uint256 _balance);
    function ownerOf(uint256 _deedId) external view returns (address _owner);
    function transfer(address _to, uint256 _deedId) external payable;
    function transferFrom(address _from, address _to, uint256 _deedId)
        external payable;
    function approve(address _approved, uint256 _deedId) external payable;
    function setApprovalForAll(address _operator, boolean _approved) payable;
    function supportsInterface(bytes4 interfaceID) external view returns (bool);
}
```

ERC-1155: マルチトークン標準

イーサリアムは代替可能トークンと非代替性トークンの基盤を築いたERC-20とERC-721から大きく進歩しました。ERC-1155（マルチトークン標準）は両方の強みを組み合わせて、効率的で多用途なトークン管理を可能にします。

あなたがゲーム開発者だと想像してください。プレイヤーがゴールドコイン（代替可能）、ユニークな剣（非代替性）、スタックで消費できるポーション（半代替性）を収集できるシステムを作りたいとします。ERC-20やERC-721ではトークンの種類ごとに別々のスマートコントラクトが必要です。新しいコントラクトごとにガスコストが増加し、コントラクト間のパーミッションやインタラクションも管理する必要があるため複雑さが増します。ERC-1155はこの問題を解決します。単一のコントラクトをデプロイするだけで複数のトークンタイプを管理できます。各トークンタイプは固有のIDで識別され、コントラクトはあらゆるトークンタイプの組み合わせに対して送金、残高、メタデータの取得を含むすべての操作を処理できます。この効率的なアプローチにより冗長性とトランザクションコストが削減され、ERC-1155は広く採用された標準となっています。

ERC-1155のバッチ操作は際立った機能です。単一のトランザクションで複数のトークンを送金またはクエリでき、ガスを節約しスケーラビリティを向上させます。さらにERC-1155にはトークンのロックなどの問題を防ぐ安全メカニズムが含まれています。スマートコントラクトにトークンを送金する際、受信コントラクトは`IERC1155Receiver`インターフェースを実装している必要があり、そうでなければトランザクションはリバートされます。これらのフックによりトークン送金中の高度なインタラクションが可能になります。たとえばコントラクトはトークンを受信した際にゲーム内リーダーボードの更新やイベントのトリガーなどカスタムロジックを実行できます。しかしERC-777で見たようにこれらのフックが慎重に扱われないとリエントランシー攻撃などの脆弱性を招く可能性があります。開発者はERC-1155トークンを実装する際にchecks-effects-interactionsパターンやリエントランシーロックなどの適切な保護措置を講じてコントラクトを安全に保つ必要があります。

ERC-1155のコントラクトインターフェース仕様は以下のとおりです。

```
interface IERC1155 /* is IERC165 */ {
    event TransferSingle(address indexed operator, address indexed from, address indexed to,
uint256 id, uint256 value);
    event TransferBatch(
        address indexed operator,
        address indexed from,
        address indexed to,
        uint256[] ids,
        uint256[] values
    );
    event ApprovalForAll(address indexed account, address indexed operator, bool approved);
    event URI(string value, uint256 indexed id);
    function balanceOf(address account, uint256 id) external view returns (uint256);
    function balanceOfBatch(
        address[] calldata accounts,
        uint256[] calldata ids
    ) external view returns (uint256[] memory);
    function setApprovalForAll(address operator, bool approved) external;
    function isApprovedForAll(
        address account,
        address operator
    ) external view returns (bool);
    function safeTransferFrom(
        address from,
        address to,
        uint256 id,
        uint256 value,
        bytes calldata data
    ) external;
    function safeBatchTransferFrom(
        address from,
        address to,
        uint256[] calldata ids,
        uint256[] calldata values,
        bytes calldata data
    ) external;
}
```

トークン標準の使い方

　前のセクションでは、トークンコントラクトのいくつかの提案された標準と、広く普及している2つの標準を確認しました。これらの標準は具体的に何をするのでしょうか。これらの標準を使うべきでしょうか。どのように使うべきでしょうか。これらの標準を超える機能を追加すべきでしょうか。どの標準を使うべきでしょうか。次にこれらの疑問のいくつかを検討します。

トークン標準とは何か、その目的は何か

トークン標準とは、実装のための*最低限の*仕様です。つまり、たとえばERC-20に準拠するためには、ERC-20標準で規定された関数と振る舞いを最低限実装する必要があります。標準に含まれない機能を追加で実装することも自由です。

これらの標準の主な目的はコントラクト間の*相互運用性*を促進することです。これによりすべてのウォレット、取引所、ユーザーインターフェース、その他のインフラコンポーネントが仕様に従ったあらゆるコントラクトと予測可能な方法で*やり取り*できます。つまりERC-20標準に従ったコントラクトをデプロイすれば、ウォレットのアップグレードやあなた側の労力なしに、すべての既存のウォレットユーザーがシームレスにトークンの取引を開始できます。

標準は*記述的*であることを意図しており*規範的*ではありません。関数をどのように実装するかはあなた次第であり、コントラクトの内部動作は標準に関係ありません。標準には特定の状況下での振る舞いを規定する機能的な要件がありますが、実装を規定するものではありません。その一例として値がゼロに設定された場合のtransfer関数の振る舞いがあります。ERC-20標準はこの場合にトランザクションをリバートすべきかどうかを規定していません。

これらの標準を使うべきか

これらのすべての標準を前にして、各開発者はジレンマに直面します。既存の標準を使うか、それともその制約を超えて革新するかという問題です。

このジレンマは簡単には解決できません。標準は従わなければならない狭い「轍」を作ることで、必然的にイノベーションの能力を制限します。一方で基本的な標準は何百ものアプリケーションの経験から生まれたものであり、大多数のユースケースに適合することが多いです。

この考慮事項の一部としてさらに大きな問題があります。相互運用性と広範な採用の価値です。既存の標準を使用すればその標準に対応して設計されたすべてのシステムの価値を得られます。標準から逸脱する場合はすべてのサポートインフラを自分で構築するか、他者にあなたの実装を新しい標準として支持してもらうコストを考慮する必要があります。独自の道を切り開き既存の標準を無視する傾向は「Not Invented Here（NIH）症候群」と呼ばれ、オープンソース文化とは対極にあるものです。一方で進歩とイノベーションは時に伝統からの逸脱に依存します。難しい選択ですので慎重に検討してください。

注記

Wikipediaによると、「Not Invented Here」とは、社会的、企業的、または制度的文化が採用する姿勢であり、外部起源であることやロイヤルティなどのコストを理由に、既存の製品、研究、標準、または知識の使用や購入を避けることである。

標準の検出: EIP-165

これまで見てきたようにERC-20のような標準はトークンとウォレット間のやり取りを簡素化します。しかしスマートコントラクトがどのインターフェースをサポートしているかをどのように識別するのでしょうか。ここでEIP-165が登場します。コントラクトがインターフェースを宣言し検出するための標準化された方法を提供します。

EIP-165はコントラクトが実装しているインターフェースを宣言するための方法を定義します。コントラクトは`supportsInterface`関数を使用して、指定されたインターフェースID（インターフェース内のすべての関数セレクタのXORとして計算される一意の識別子）に対して`true`または`false`の値を返します。たとえばインターフェースに`foo()`と`bar(int256)`が含まれる場合、そのIDは以下のように導出されます。

```
foo.selector ^ bar.selector
```

このアプローチにより他のコントラクトやツールがやり取りの前に互換性を検証できます。たとえばマーケットプレイスはNFTコントラクトがERC-721インターフェースをサポートしていることを確認してからトークンをリストできます。

EIP-165を実装するにはコントラクトがOpenZeppelinのERC-165のような基底クラスを継承し、`supportsInterface`メソッドをオーバーライドしてサポートするインターフェースを含めます。

```
contract MyContract is IMyContract, ERC165 {
    function supportsInterface(bytes4 interfaceId) public view override returns (bool) {
        return interfaceId == type(IMyContract).interfaceId ||
super.supportsInterface(interfaceId);
    }
}
```

EIP-165が実際にどのように機能するかをより深く理解するために、多用途で広く使用されているマルチトークンコントラクトの標準であるERC-1155を見てみましょう。

```
abstract contract ERC1155 is Context, ERC165, IERC1155, IERC1155MetadataURI,
    IERC1155Errors {
[...]
    /**
     * @dev {IERC165-supportsInterface}を参照。
     */
    function supportsInterface(bytes4 interfaceId) public view virtual override(ERC165,
IERC165) returns (bool) {
        return
            interfaceId == type(IERC1155).interfaceId ||
            interfaceId == type(IERC1155MetadataURI).interfaceId ||
            super.supportsInterface(interfaceId);
    }
```

```
[...]
}
```

警告

EIP-165は、コントラクトが自身の機能を正直に報告することに依存している。悪意のある、または不適切に実装されたコントラクトは、インターフェースをサポートしていると偽って主張する可能性があり、潜在的な問題を引き起こし得る。EIP-165は開発者エクスペリエンスを向上させ摩擦を減少させるが、セキュリティの保証として扱うべきではない。

コントラクトが他者に代わってインターフェースを実装するようなより高度なシナリオでは、開発者はERC-1820を検討できます。ERC-1820はインターフェースのサポートを追跡するためにグローバルレジストリを使用します。EIP-165よりも複雑ですが、分散型システムにとってより大きな柔軟性を提供します。

成熟によるセキュリティ

標準の選択に加えて*実装*の選択という並行する課題があります。ERC-20のような標準を使うと決めたら、互換性のある設計をどのように実装するかを決定する必要があります。イーサリアムエコシステムで広く使用されている既存の「リファレンス」実装がいくつかありますし、ゼロから独自に書くこともできます。この選択もまたセキュリティに重大な影響を及ぼし得るジレンマを表しています。

既存の実装は「実戦テスト済み」です。安全であることを証明するのは不可能ですが、その多くが数百万ドル、場合によっては数十億ドル相当のトークンを支えています。繰り返し精力的に攻撃されてきましたが、これまで重大な脆弱性は発見されていません。独自に書くのは容易ではありません。コントラクトが侵害される微妙な方法は数多くあります。十分にテストされ広く使用されている実装を使う方がはるかに安全です。この例ではOpenZeppelinのERC-20標準の実装を使用しました。この実装は最初からセキュリティに重点を置いているからです。

既存の実装を使用する場合、それを拡張することもできます。ただしこの衝動には注意が必要です。複雑さはセキュリティの敵です。追加するコードの一行一行がコントラクトの攻撃対象面を拡大し、潜在的な脆弱性を含んでいる可能性があります。コントラクトの上に多くの価値を載せ、誰かがそれを破るまで問題に気づかないかもしれません。

ヒント

標準と実装の選択は、安全なスマートコントラクト設計全体の重要な部分であるが、唯一の考慮事項ではない（第9章を参照）。

トークンインターフェース標準の拡張

これまで議論してきたトークン標準はトークンの作成と管理のための基本的な機能を提供します。しかしこれらは意図的に最小限に設計されており、プロジェクトが特定のニーズに合わせて拡張・適応する余地を残しています。時間の経過とともに多くのプロジェクトがこれらの標準を基にして構築し、ユーザビリティ、セキュリティ、柔軟性を向上させる機能を導入してきました。イーサリアムスマートコントラクトの主要なライブラリであるOpenZeppelinがこうした拡張のための定番のソースとなっています。ERC-20、ERC-721、ERC-1155トークンの最も注目すべき拡張のいくつかを見てみましょう。

たとえばERC-20ではバーンメカニズムの追加があります。バーンによりトークンを流通から永久に除去でき、供給量を削減できます。これはデフレモデルや意図的な供給量制御を必要とするトークンエコノミクスに有用です。一方で一部のプロジェクトはキャップを組み込み、総供給量にハードリミットを設定して事前に定義された閾値を超えるトークンがミントされないようにしています。

ERC-20のもうひとつの興味深い追加機能は、投票機能です。トークン保有者がトークンを通じて直接ガバナンスの意思決定に参加できるようにします。この機能を実装するプロジェクトは、分散型の意思決定プロセスを構築し、ステークホルダーがプロトコルの進化に発言権を持てるようにします。

ERC-721にも同様に創造的な拡張が見られます。ロイヤルティ支払い機能によりクリエイターはNFTが取引されるたびに販売額の一定割合を受け取れます。URIストレージも一般的な追加機能で、メタデータを動的に保存・取得でき、進化する属性を持つNFTに特に有用です。列挙可能な拡張により開発者はアドレスが保有するすべてのトークンを効率的にリストでき、マーケットプレイスやウォレットの構築が容易になります。

ERC-1155（マルチトークン標準）も取り残されていません。ERC-1155の拡張には、バーン可能なトークンやポーザブルなコントラクトがあり、ゲームやトークン化されたサプライチェーンなどのユースケースに柔軟性を追加します。一部の実装ではメタデータの処理も強化し、トークンの詳細がアクセスしやすく更新しやすい状態を維持します。

これら以外にもクラウドファンディング、ブロックリスト、アローリスト、送金手数料の実装など、さまざまなニーズに対応する無数の拡張が存在します。開発者はこれらの機能をOpenZeppelinのOwnableやAccess Controlなどの他の標準ライブラリと組み合わせ、さらに多くの実戦テスト済みのリソースを活用することが多いです。

柔軟性には責任が伴います。トークン標準の拡張にはイノベーションと相互運用性のバランスが求められます。カスタム機能の作成は魅力的に見えるかもしれませんが、不必要な複雑さとリスクを招くことが多いです。OpenZeppelinのような確立されたライブラリと拡張を活用することでセキュリティとコード品質を確保し、開発コストを大幅に削減できます。堅牢でテスト済みのソリューションがすでに存在するのに車輪を再発明する必要はありません。

まとめ

トークンは単なるデジタル通貨ではありません。ガバナンス権、アクセス資格、アイデンティティ、現実世界の資産を表現できます。その多用途性はERC-20、ERC-721、ERC-1155のような標準のおかげで初めて可能になるものであり、これらの標準がウォレット、取引所、DApp間のシームレスな相互運用性を保証し、より効率的で相互に接続されたブロックチェーンエコシステムを構築しています。本章ではさまざまな種類のトークンとトークン標準を見てきました。そして最初のトークンとそれに関連するアプリケーションを構築しました。

第11章 オラクル

この章では、イーサリアムのスマートコントラクトに外部データソースを提供するシステムである*オラクル*について説明します。*オラクル*という用語はギリシャ神話に由来し、神々と通信して未来のビジョンを見ることができる人物を指していました。ブロックチェーンの文脈では、オラクルはイーサリアム外部の質問に答えることができるシステムです。理想的には、オラクルは分散化の原則に基づいて動作するトラストレスなシステムであり、信頼を必要としません。

オラクルが必要な理由

イーサリアムプラットフォームの重要な構成要素はEVMです。EVMは分散ネットワーク内のあらゆるノードで、コンセンサスルールに制約されながらプログラムを実行し、イーサリアムの状態を更新します。コンセンサスを維持するために、EVMの実行は完全に決定論的（同じ入力に対して必ず同じ結果を返す）でなければならず、イーサリアムの状態と署名済みトランザクションという共有の文脈のみに基づく必要があります。これには特に重要な2つの結果があります。1つ目は、EVMとスマートコントラクトが利用できる固有のランダム性のソース（乱数源）が存在しないこと、2つ目は、外部データはトランザクションのデータ部分としてのみ導入できることです。

この2つの結果についてさらに掘り下げてみましょう。EVMにランダム関数が存在しない理由を理解するために、仮にランダム関数が存在した場合に、コンセンサスを達成しようとするとどうなるかを考えてみてください。ノードAはコマンドを実行してスマートコントラクトのストレージに3を格納する一方、同じスマートコントラクトを実行するノードBは代わりに7を格納します。つまりノードAとノードBは、まったく同じコードをまったく同じ条件で実行したにもかかわらず、結果の状態について異なる結論に達します。実際には、スマートコントラクトが評価されるたびに異なる結果の状態が得られる可能性があります。そのため、世界中で独立して動作する多数のノードからなるネットワークが、結果の状態について分散型のコンセンサスに到達する方法はありません。実際にはetherの送金を含む連鎖的な影響が次々と波及するため、この例よりもはるかに深刻な事態になります。

EVMには真のランダム性のソースが存在しないため、暗号学的に安全なハッシュ関数（決定論的であり、実際にEVMの一部として組み込まれている）を使ってランダム性を代替しても、多くのアプリケーションにとって十分ではない点に注意してください。賭けの支払いを解決するためにコイン投げをシミュレートするギャンブルゲームを考えてみましょう。このゲームは表か裏をランダム化する必要があります。ブロック提案者はゲームをプレイし、自分が勝つブロックにのみトランザクションを含めることで有利に立てます。

ここで、2つ目の結果である外部データの導入方法について考えてみましょう。すべてのノードは署名済みトランザクションの内容について合意できるため、ランダム性のソース、価格情報、天気予報などの外部情報は、ネットワークに送信されるトランザクションのデータ部分とし

て導入できます。しかし、そのようなデータは出所が検証できないため、そのまま信頼するわけにはいきません。データの信頼性をどう保証するか、という問題が依然として残っています。この章の残りの部分で詳しく説明するオラクルを使って、これらの問題の解決を試みます。

オラクルのユースケースと例

オラクルは理想的には、フットボールの試合結果、金の価格、真の乱数など、外部の（つまり「現実世界の」またはオフチェーンの）情報を、スマートコントラクトが使用できるようにイーサリアムプラットフォームに取り込むためのトラストレス（または少なくともほぼトラストレス）な方法を提供します。またDAppフロントエンドに直接データを安全に中継するためにも使用できます。そのためオラクルは、オフチェーンの世界とスマートコントラクトの間のギャップを埋めるメカニズムと考えることができます。主にトークン価格などの情報を異なるブロックチェーン間で伝達するために使用されます。

スマートコントラクトが現実世界のイベントやデータに基づいて契約関係を強制できるようになれば、その適用範囲は劇的に広がります。しかし同時に、イーサリアムのセキュリティモデルに外部リスクをもたらす可能性もあります。人が亡くなった際に資産を分配する「スマート遺言」コントラクトを考えてみましょう。スマートコントラクトの分野で頻繁に議論されるこの事例は、信頼されたオラクルのリスクを浮き彫りにしています。そのようなコントラクトが管理する相続額が十分に高ければ、オラクルをハッキング[1]して所有者が亡くなる前に資産の分配を開始するインセンティブは非常に高くなります。

[1] ここでのハッキングには、中央集権的なオラクルの人間のオペレーターを買収することも含まれ得る。

また、学業証明書や政府発行の身分証明書のように、特定の機関が管理する非公開データを提供するオラクルも存在します。大学や政府機関のようなデータソースは完全に信頼されており、データの真偽はそのソースの権威に依存しており、独立した検証が困難です。そのためこのようなデータはトラストレスに提供できません。スマートコントラクトへのデータブリッジも提供するという理由から、これらのデータソースも「オラクル」の定義に含めています。オラクルが提供するデータは一般的に、パスポートや業績記録のようなアテステーション（権威ある機関がデータの正当性を保証した署名付き証明書）の形を取ります。アテステーションは、特にアイデンティティやレピュテーションの検証において、ブロックチェーンプラットフォームの将来の成功に大きく関わるため、ブロックチェーンプラットフォームがどのようにアテステーションを提供できるかを探ることは重要です。

オラクルが提供し得るデータのその他の例を以下に示します:

- 量子的な現象や熱雑音（サーマルノイズ）といった物理的ソースから得られる乱数やエントロピー（例: 宝くじスマートコントラクトの当選者を公正に選出するため）

- 自然災害に連動するパラメトリックトリガー（例: 地震債券のリヒタースケール[2]測定値のような大災害債券スマートコントラクトのトリガー）
- 為替レートデータ（例: 暗号通貨を法定通貨に正確にペグするため）

[2] リヒタースケール（ローカルマグニチュード、ML）は小〜中規模の地震の測定に用いられてきたが、現在は大規模地震の測定精度に優れるモーメントマグニチュード（Mw）が主流となっている。

- 資本市場データ（例: トークン化された資産や証券のバスケットの価格決定）
- ベンチマーク参照データ（例: スマート金融デリバティブへの金利の組み込み）
- 静的または準静的データ（例: 証券識別子、国コード、通貨コードなど）
- 正確な時間測定に基づくイベントトリガーの時刻およびインターバルデータ
- 気象データ（例: 天気予報に基づく保険料計算）
- 予測市場の解決のための政治イベント
- 予測市場の解決およびファンタジースポーツコントラクトのためのスポーツイベント
- ジオロケーションデータ（例: サプライチェーン追跡で使用）
- 保険コントラクトのための損害検証
- 相互運用性機能のための他のブロックチェーンで発生したイベント
- etherの市場価格（例: 法定通貨ガス価格オラクルのため）
- フライト統計データ（例: 航空券のプーリングのためにグループやクラブが使用）

以下のセクションでは、基本的なオラクルパターン、計算オラクル、分散型オラクルを含む、オラクルの実装方法のいくつかについて検討します。

オラクルの設計パターン

すべてのオラクルは、定義上いくつかの主要な機能を提供します。これらには以下の能力が含まれます:

- オフチェーンソースからデータを収集する
- 署名されたメッセージでデータをオンチェーンに転送する
- データを利用可能にする

データがスマートコントラクトで利用可能になると、オラクルのスマートコントラクトの`retrieve`関数を呼び出すメッセージコールを通じて、他のスマートコントラクトからアクセスできます。イーサリアムノードやネットワーク対応クライアントから直接アクセスすることも可能です。

オラクルを構築する3つの主要な方法は、即時読み取り、パブリッシュ-サブスクライブ、およびリクエスト-レスポンスに分類できます。

即時読み取り

最もシンプルなタイプのオラクルから始めましょう。即時読み取りオラクルは、「ethereumbook.infoのアドレスは何ですか」や「この人は18歳以上ですか」のような、即座の判断にのみ必要なデータを提供します。図11-1に示します。

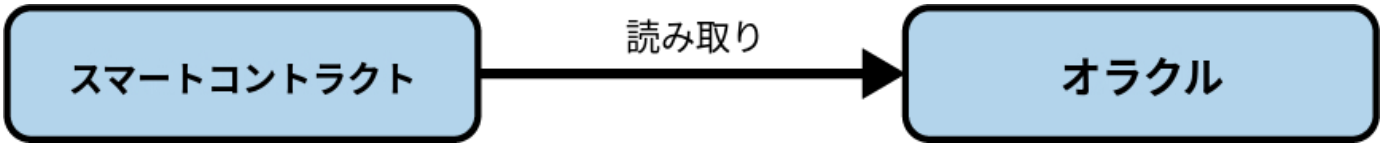


図11-1 即時読み取りオラクル

この種のデータを照会したい人は、「ジャストインタイム」ベースで照会する傾向があります。情報が必要な時にだけ照会し、二度と行わないかもしれません。このようなオラクルの例には、学業証明書、ダイヤルコード、機関の会員資格、空港識別子、自己主権ID（self-sovereign ID）などの組織に関するデータや組織が発行したデータを保持するものがあります。

このタイプのオラクルは、他のスマートコントラクトがオラクルコントラクトへのリクエストコールで検索できるよう、データをコントラクトストレージに一度保存します。後から更新することも可能です。オラクルのストレージ内のデータは、トランザクションを発行する手間やガスコストを負担せずに、ブロックチェーン対応の（つまりイーサリアムクライアントに接続された）アプリケーションから直接検索することもできます。例えば、アルコールを購入する顧客の年齢を確認する必要がある店舗は、この方法でオラクルを利用できます。このタイプのオラクルは、データリクエストに応答するためにサーバーを運用・保守しなければならない組織や企業にとって魅力的です。

オラクルが保存するデータは、効率性やプライバシーの理由から、生データそのものではない可能性が高い点に注意してください。大学は過去の学生の学業成績証明書のためにオラクルを設定するかもしれません。しかし証明書の完全な詳細（履修したコースや取得した成績のページにも及ぶ可能性がある）を保存するのは過剰です。代わりに証明書のハッシュ値で十分です。同様に、政府は市民IDをイーサリアムプラットフォーム上に置きたいと考えるかもしれませんが、含まれる詳細情報は明らかにプライベートに保つ必要があります。この場合もデータをハッシュ化し（ソルト〔ハッシュ値の推測を困難にするランダムなデータ〕付きのマークルツリーでより慎重に）、ルートハッシュのみをスマートコントラクトのストレージに保存するのが効率的な方法です。

パブリッシュ-サブスクライブ

次の構成はパブリッシュ-サブスクライブです。定期的かつ頻繁に変更が予想されるデータのブロードキャストサービスをオラクルが提供し、それをオンチェーンのスマートコントラクトがポーリングするか、オフチェーンのデーモンが更新を監視します。図11-2に示します。

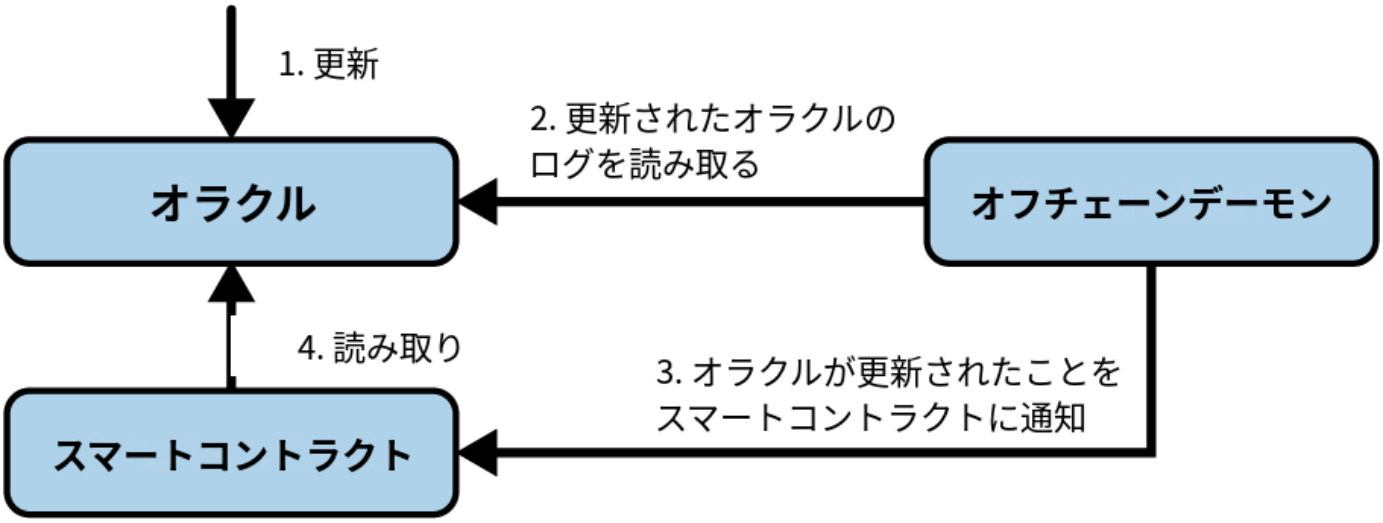


図11-2 パブリッシュ-サブスクライブオラクル

> **注記**
>
> オフチェーンのデーモンを省略することも可能である。オラクルは更新のタイムスタンプを保存し、データが読み取られる際にスマートコントラクトに渡すことができる。この方法で、スマートコントラクトはデータの最終更新日時を知り、使用するかどうかを選択できる。

このカテゴリはRSSフィードやWebSubなどと類似のパターンを持っています。オラクルは新しい情報で更新され、新しいデータが利用可能であることを示すフラグが立ち、サブスクライバーに更新が通知されます。関心のある当事者は、最新の情報が変更されたかどうかを確認するためにオラクルをポーリングするか、オラクルコントラクトへの更新をリッスンして発生時に対応する必要があります。例としては、価格フィード、気象情報、経済・社会統計、交通データなどがあります。

ポーリングはWebサーバーの世界では非常に非効率的ですが、ブロックチェーンプラットフォームのピアツーピアの文脈では、必ずしも非効率とは言えません。イーサリアムクライアントはコントラクトストレージの変更を含むすべての状態変化に追従する必要があるため、データ変更のポーリングは同期済みクライアントへのローカルコールにすぎません。イーサリアムのイベントログによりアプリケーションがオラクルの更新を容易に監視できるため、このパターンはある意味で「プッシュ」サービスとさえ見なせます。

リクエスト-レスポンス

リクエスト-レスポンスのカテゴリは最も複雑です。データ空間が大きすぎてスマートコントラクトに保存できず、ユーザーは一度にデータセット全体のごく一部しか必要としない場合に使用します。図11-3に示します。データプロバイダビジネスにも適用可能なモデルです。

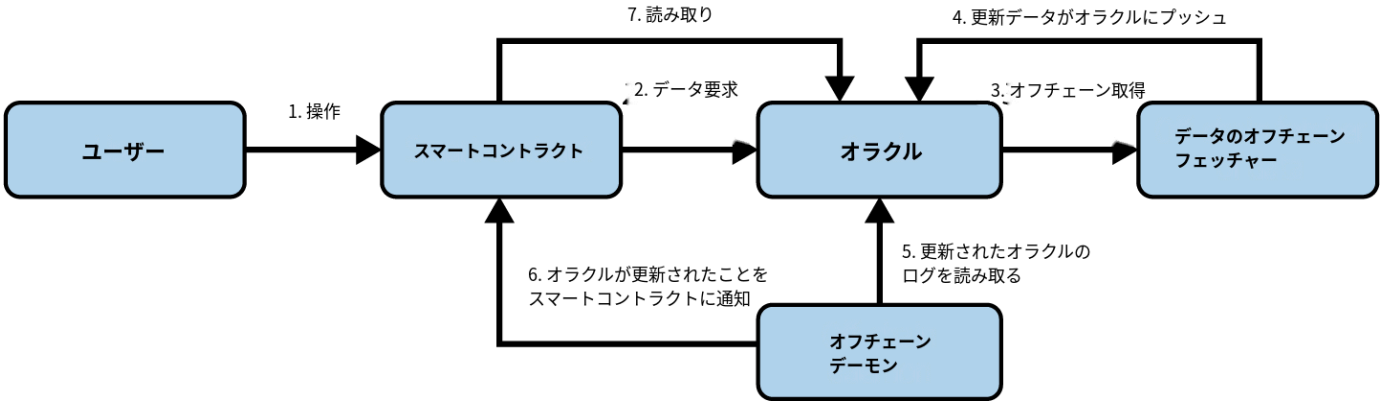


図11-3 リクエスト-レスポンスオラクル

実際にこのようなオラクルは、オンチェーンのスマートコントラクトと、リクエストを監視してデータを取得・返却するオフチェーンインフラストラクチャのシステムとして実装される可能性があります。DAppからのデータリクエストは通常、複数のステップを含む非同期プロセスです。

このパターンでは、まずEOAがDAppへトランザクションを送り、オラクルスマートコントラクトに定義された関数を呼び出します。この関数はオラクルへのリクエストを開始し、リクエストデータの詳細を記述する関連引数に加えて、コールバック関数やスケジューリングパラメーターなどの補足情報を伴います。このトランザクションが検証されると、オラクルリクエストはオラクルコントラクトが発行したEVMイベントまたは状態変更として観察できます。そこから引数を取得し、オフチェーンデータソースへの実際のクエリに使用できます。

オラクルはリクエストの処理に対する支払い、コールバックのためのガス支払い、リクエストデータへのアクセス許可も要求する場合があります。最終的に結果のデータはオラクルの所有者によって署名され、特定の時点でのデータの有効性が証明されます。そしてリクエストを行ったDAppに、直接またはオラクルコントラクトを介してトランザクションで配信されます（図11-3参照）。スケジューリングパラメーターに応じて、オラクルは定期的にデータを更新するさらなるトランザクションをブロードキャストする場合があります（例: 1日の終わりの価格情報）。

リクエスト-レスポンスオラクルの手順は以下のように要約できます:

1. DAppからクエリを受信する。

2. クエリを解析する。

3. 支払いとデータアクセス権限が提供されていることを確認する。

4. オフチェーンソースから関連データを取得する（必要に応じて暗号化する）。

5. データを含むトランザクションに署名する。

6. トランザクションをネットワークにブロードキャストする。

7. 通知などのさらなる必要なトランザクションをスケジューリングする。

他にもさまざまなスキームが可能です。例えばデータはEOAから直接リクエストして返却でき、オラクルスマートコントラクトが不要になります。同様にリクエストとレスポンスはIoT対応のハードウェアセンサとの間で行うこともできます。つまりオラクルは人間、ソフトウェア、またはハードウェアのいずれでもあり得ます。

ここで説明したリクエスト-レスポンスパターンは、クライアント-サーバーアーキテクチャでよく見られます。アプリケーションが双方向の対話を行える有用なメッセージングパターンですが、特定の条件下では不適切な場合もあります。例えばオラクルから金利を要求するスマート債券は、レートが常に正確であることを保証するために、リクエスト-レスポンスパターン[3]の下で毎日データをリクエストしなければならない場合があります。金利はめったに変わらないため、ここではパブリッシュ-サブスクライブパターンの方が適切かもしれません。特にイーサリアムの限られた帯域幅を考慮すると、なおさらです。

[3] この非同期性のため、リクエストを行うDApp/コントラクトは遅延レスポンスを処理できるよう設計する必要がある。同じトランザクション内で即座にデータを期待することはできない。

パブリッシュ-サブスクライブは、パブリッシャー（この文脈ではオラクル）がレシーバーに直接メッセージを送信するのではなく、公開されたメッセージを個別のクラスに分類するパターンです。サブスクライバーは1つ以上のクラスへの関心を表明し、関心のあるメッセージのみを取得できます。このパターンでは、オラクルは金利が変更されるたびに自身の内部ストレージに書き込みます。複数のサブスクライブ済みDAppはオラクルコントラクトから読み取るだけでよく、ネットワーク帯域幅への影響を軽減しつつストレージコストも最小化できます。

ブロードキャストまたはマルチキャストパターンでは、オラクルはすべてのメッセージをチャネルに投稿し、サブスクライブするコントラクトはさまざまなサブスクリプションモードでチャネルをリッスンします。例えばオラクルは暗号通貨の為替レートチャネルにメッセージを公開するかもしれません。あるスマートコントラクトは移動平均計算のために時系列データとしてチャネルの全コンテンツをリクエストし、別のスマートコントラクトはスポット価格計算のために最新レートのみを必要とするかもしれません。ブロードキャストパターンは、オラクルがサブスクライブするコントラクトのアイデンティティを知る必要がない場合に適しています。

データ認証

DAppが照会するデータのソースが権威的かつ信頼できると仮定しても（これは重要な前提条件です）、未解決の疑問が残ります。オラクルとリクエスト-レスポンスのメカニズムが異なるエンティティによって運用される可能性がある場合、このメカニズムをどう信頼できるのでしょう

か。データが転送中に改ざんされる可能性は十分にあるため、オフチェーンの方法で返却データの完全性を証明できることが重要です。データ認証への2つの一般的なアプローチは、*真正性証明*（authenticity proofs）と*信頼できる実行環境*（TEE: Trusted Execution Environment）です。

真正性証明は、データがソースからブロックチェーンに至る途中で改変されていないことを暗号学的に保証します。これらの証明は、転送メカニズムへの依存を、データプロバイダ、セキュアなサードパーティ、分散型ノードネットワークなどの検証可能な証明者へと移行させます。暗号署名やゼロ知識アテステーションをオンチェーンで検証することで、スマートコントラクトは受信した情報が確かに適切な権限者から来ており改ざんされていないことを確認できます。多くの場合、元のデータソースが特別な署名ロジックを実装する必要はありません。

注記

実用的な例としてChainlink VRFがある。Chainlinkのドキュメントには、「各リクエストに対して、Chainlink VRFは1つ以上のランダム値と、それらの値がどのように決定されたかの暗号学的証明を生成する。証明はオンチェーンで公開・検証された後に、消費アプリケーションが使用できるようになる」と記載されている。つまりChainlink VRFはランダム値の生成方法を証明でき、これがまさに真正性証明である。

Chainlink VRFは、デプロイされたスマートコントラクトがChainlinkオラクルネットワークにランダム数をリクエストし、オラクルが予測できないヒントを提供することで動作する。各オラクルは秘密鍵を使用してオフチェーンでランダム数を生成し、結果と対応する暗号学的証明をオンチェーンに公開する。スマートコントラクトはオラクルの公開鍵と元のヒントを使用して、このランダム出力が操作されていないことを検証できる。証明は完全にオンチェーンで検証されるため、攻撃者は暗号学的チェックを無効にすることなく結果を改ざんすることはできない。ノードが侵害されたり応答しなくなったりした場合、その障害はオンチェーンに記録され、最終的にそのノードはさらなるランダム性の提供から除外される。

TEEは、コードとデータを保護・証明する特殊なハードウェアエンクレーブを活用することで、これらの保証を強化します。計算がそのようなエンクレーブ内で実行されると、CPUは外部プロセスが計算に干渉したり基礎データを見たりできないようにし、完全性と機密性の両方を保持します。エンクレーブは、暗号学的ハッシュ値で識別される特定のコードがセキュアな環境内で実行されていることを示す、デジタル署名された「アテステーション」（ハードウェアが発行する実行環境の真正性証明）を提供できます。これによりスマートコントラクトは、外部データや計算がオンチェーンに到着する前に悪意を持って改変されていないことについて、より強力な保証を得られます。またセキュアエンクレーブはプライバシー機能も実現します。機密性の高い入力を暗号化してエンクレーブ内で処理すれば、外部に公開せずに済むためです。

多くの最新のオラクルネットワークでは、これらの方法を組み合わせてデータ認証をさらに強化しています。

1つ目のアプローチは、異なるソースからデータを取得・検証する独立したノードの分散型セットに依存し、正しい結果についてコンセンサスに達する方法です。このマルチオペレータシステムにより、単一の不正行為者がデータフィードを侵害するリスクは大幅に軽減されます。ノードにはインセンティブが与えられ、多くの場合は正直であり続けるために経済的担保によって拘束されているためです。

2つ目は、高度な暗号プロトコルを組み込み、基礎的な詳細を公開せずにデータが特定のエンドポイントから取得されたことを証明する方法です。これにより中央集権的な検証者への依存を軽減しています。

3つ目は、データ取得ロジックを実行するためにハードウェアエンクレーブを展開する方法です。これによりホスト環境が侵害されても、ブロックチェーンに送信される最終結果が改変されず独立して検証できることを保証します。

具体的な実装にかかわらず、最大の課題はDAppに取得・渡されるすべてのデータが正確で信頼できることの保証です。真正性証明はデータがどこでどのように取得されたかを追跡・検証する堅牢な方法を提供し、TEEはオフチェーン情報の収集と中継のプロセス全体を保護するハードウェアに裏付けられた手段を提供します。

> **警告**
>
> TEEは比較的新しいため、大部分はまだテストが不十分であり、多くの未発見の脆弱性が含まれている可能性がある。将来、新たなエクスプロイトが発見されTEEが侵害される可能性が高い。

計算オラクル

ここまではデータのリクエストと配信の文脈でのみオラクルについて説明してきました。しかしオラクルは任意の計算を実行するためにも使用できます。この機能はイーサリアムの固有のブロックガスリミットと比較的高い計算コストを考えると特に有用です。単にクエリの結果を中継するだけでなく、計算オラクルを使って一連の入力に対して計算を実行し、オンチェーンでは計算不可能だった可能性のある結果を返すことができます。例えば計算オラクルを使って計算負荷の高い回帰分析（統計的な予測計算）を実行し、債券コントラクトの利回りを推定できます。

ZKコプロセッサ（ゼロ知識証明を用いてオフチェーンで計算し、その正確さをオンチェーンで検証するシステム）として知られるLagrangeとBrevisは、スマートコントラクトがオンチェーンでの検証を保証しつつ、集中的な計算をオフチェーンで実行できるようにします。例えばBrevisでは、DAppが前もってゼロ知識証明を生成せずに、複雑なタスクや履歴データをリクエストできます。ネットワークは結果をオプティミスティックに[4]投稿し、正しいと仮定します。結果がオンチェーンに公開されると、誰でも異議を申し立てられるチャレンジ期間が設けられます。異議が申し立てられた場合、提案者は結果を検証するための完全なゼロ知識証明を作成しな

ければなりません。証明が提供されない場合や、証明により初期結果が不正確であると判明した場合、チャレンジャーは報酬を受け取り、偽の結果を提出した者はペナルティを受けます。異議が発生しなければ結果は有効として承認されるため、必要なゼロ知識証明の数を劇的に削減しコストを低減できます。

[4] オプティミスティック（楽観的）とは、結果が正しいと仮定して先に公開し、異議申し立てがあった場合にのみ証明を要求する設計思想を指す。オプティミスティックロールアップと同様のアプローチである。

分散型オラクル

中央集権的なデータまたは計算オラクルは多くのアプリケーションには十分ですが、イーサリアムネットワークにおける単一障害点になります。データの可用性を確保し、個々のデータプロバイダのネットワークとオンチェーンのデータ集約システムを構築する手段として、分散型オラクルを中心にいくつかのスキームが提案されています。

Chainlinkは、レピュテーションコントラクト、注文マッチングコントラクト、集約コントラクトの3つの主要なスマートコントラクトと、データプロバイダのオフチェーンレジストリで構成される分散型オラクルネットワークを提案しています。

レピュテーションコントラクトはデータプロバイダのパフォーマンスを追跡し、そのスコアはオフチェーンレジストリの内容を構成するために使われます。注文マッチングコントラクトはレピュテーションコントラクトを使ってオラクルからの入札を選択し、クエリパラメーターと必要なオラクルの数を含むサービスレベルアグリーメントを確定します。これにより購入者は個々のオラクルと直接トランザクションを行う必要がありません。集約コントラクトは複数のオラクルからのレスポンス（コミット-リビールスキーム〔まず値のハッシュをコミットし、後から元の値を公開することで操作を防ぐ手法〕で送信される）を収集してクエリの最終的な集合結果を計算し、その結果をレピュテーションコントラクトにフィードバックします。

このような分散型アプローチの主要な課題の1つは、集約関数の定式化です。Chainlinkは加重レスポンスの計算を提案しており、各オラクルのレスポンスに対して有効性スコアを報告できるようにしています。しかし、不正なスコアを検出するのは容易ではありません。このシステムは、他のピアの回答から大きく外れた値（外れ値）を不正とみなすという前提に依存していますが、そのような値が必ずしも不正とは限らないためです。レスポンスの分布内でのオラクルレスポンスの位置に基づいて有効性スコアを計算すると、平均的な回答よりも正しい回答にペナルティを科すリスクがあります。そのためChainlinkは標準的な集約コントラクトのセットを提供しつつ、カスタマイズされた集約コントラクトの指定も可能にしています。

関連するアイデアとして*シェリングコインプロトコル*があります。ここでは複数の参加者が値を報告し、中央値が「正しい」回答として採用されます。報告者はデポジットを提供する必要があり、そのデポジットは中央値に近い値に有利になるよう再分配されるため、他の参加者と類似した値を報告するインセンティブが生まれます。回答者が調整のための自然で明白な目標と見な

す共通の値、すなわち*シェリングポイント*[5]と呼ばれるものは、実際の値に近いと予想されます。

[5] シェリングポイントとはゲーム理論家トーマス・シェリング（Thomas Schelling）が提唱した概念で、事前の調整なしに参加者が自然に選ぶ焦点となる解のこと。ノーベル経済学賞受賞者シェリングの著書『紛争の戦略』（1960年）に由来する。

クロスチェーンメッセージングプロトコル

多くのアプリケーションは、それぞれ独自のコミュニティガバナンス、コンセンサスルール、トークン標準を持つ複数のチェーン間でのデータ転送やインタラクションを頻繁に必要とします。クロスチェーンプロトコルはブロックチェーン間の通信を促進する重要なツールとして登場し、スマートコントラクトや分散型アプリケーションがより広範なサービス、流動性、データにアクセスできるようにしています。オラクルが外部情報を単一のブロックチェーンにブリッジするのに対し、クロスチェーンプロトコルはエコシステム全体を接続することでその概念を拡張しています。

クロスチェーンプロトコルを理解する1つの方法は、ブロックチェーンを接続する特殊な「通信レイヤー」として捉えることです。外部データの取り込みだけでなく、チェーン間の情報転送を促進するプロトコルです。

人気のあるクロスチェーンイニシアチブの中で、LayerZeroはブロックチェーン間の軽量メッセージングフレームワークを提供しています。メッセージの「トランスポート」と「バリデーション」に焦点を当て、より効率的で柔軟な相互運用性レイヤーの提供を目指しています。LayerZeroの設計は、クロスチェーントランザクションを検証するために協力する2つの主要なオフチェーンエンティティ、オラクルとリレイヤーを中心に構成されています。図11-4に示します。

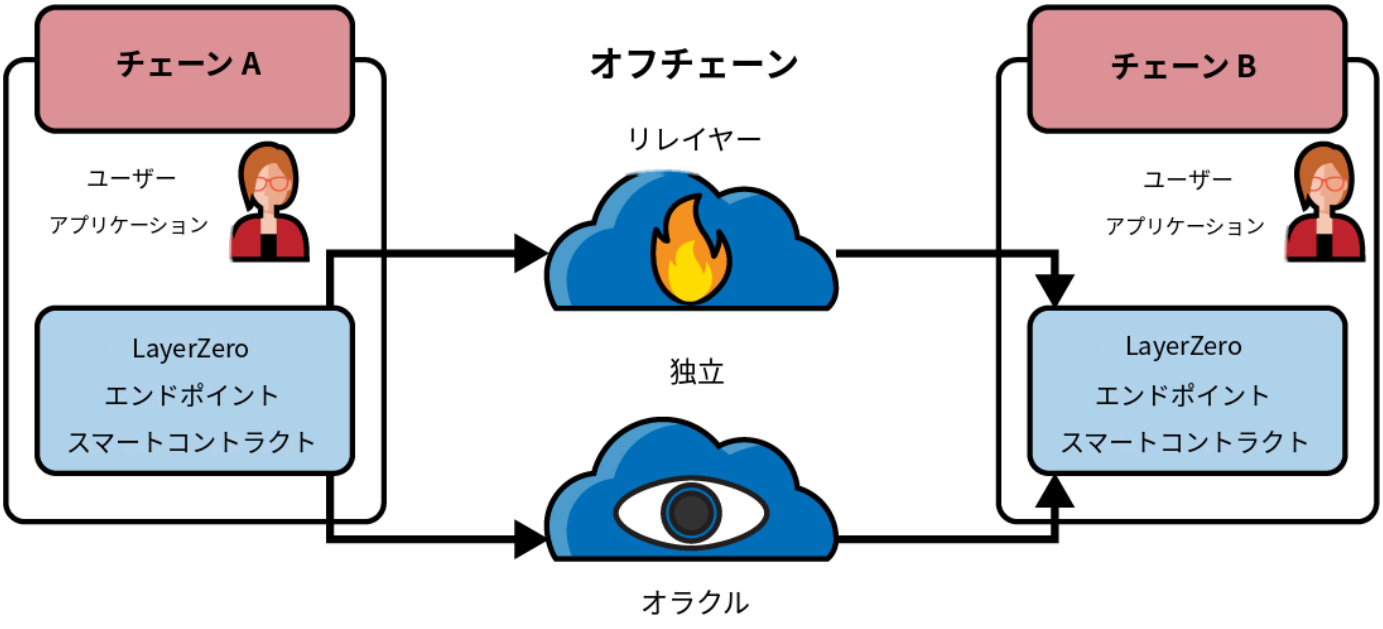


図11-4 LayerZeroクロスチェーンアーキテクチャ

オラクルはトランザクションの証明またはブロックヘッダに対する独立したクエリを実行し、リレイヤーは証明自体を渡します。信頼を分散化するために、ユーザーが設定可能なオラクルとリレイヤーのセットを使用できます。オラクルとリレイヤーが同じデータを提供した場合、宛先チェーン上のLayerZeroのスマートコントラクトはメッセージを有効として受け入れます。これにより開発者は、単一のブリッジングプロバイダや中央集権的なエンティティに依存せずに複雑な相互運用性ソリューションを作成できます。

もう1つの有名なプロジェクトであるWormholeは、主にSolanaとイーサリアム間の転送を可能にするために誕生しました。その後Binance Smart Chain、Hyperliquid、Avalancheなどの他のネットワークにも拡大しています。Wormholeのアプローチは、単一のチェーン上のイベントを監視し、イベントを証明するメッセージに署名するガーディアンのネットワークに基づいています。十分な数のガーディアンが署名するとアテステーションは有効と見なされ、対応するイベント（トークン転送など）が宛先チェーンで認識されます。図11-5に示します。このスキームはトークンブリッジングだけでなく、クロスチェーンガバナンス提案やNFT転送などのより複雑なタスクにも役立ちます。Wormholeは複数のガーディアンの複合的なセキュリティを活用して単一障害点のリスクを軽減しようとしていますが、ガーディアンセットの慎重な選定と維持が必要です。

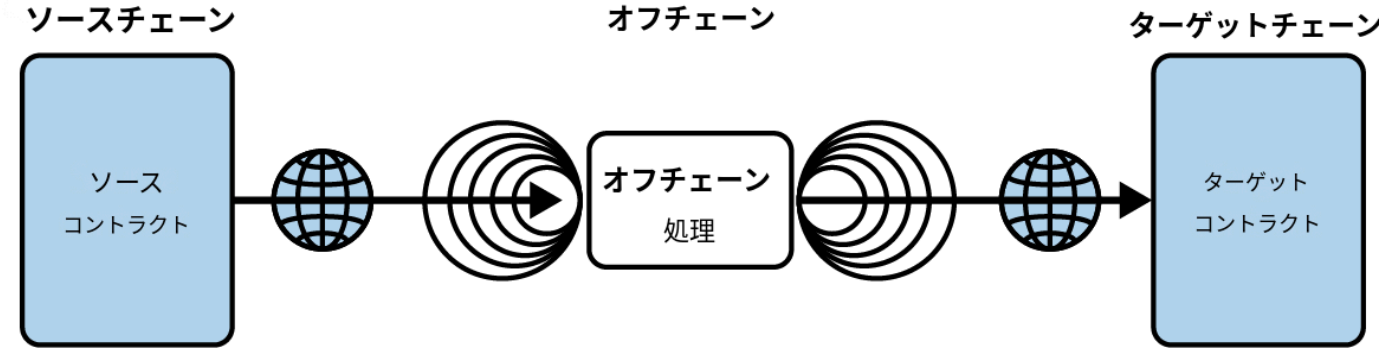


図11-5 Wormholeクロスチェーンアーキテクチャ

ChainlinkのCross-Chain Interoperability Protocol（CCIP）は、組織の既存のオラクルネットワークを基盤として、ブロックチェーン間のセキュアなメッセージングとトークン転送のための汎用フレームワークを提供しています。異なるネットワーク間のイベントを検証するために分散型オラクルに依存し、高レベルの信頼最小化に焦点を当てています。CCIPはソースチェーン上でトークンをロックまたはバーンし、宛先チェーン上でミントまたはアンロックできるため、DAppが複数のエコシステムにまたがって機能を拡張できます。Chainlinkが分散型データフィードや検証可能なランダム性のために開発した堅牢なインフラストラクチャを再利用しており、これらのサービスにすでに依存しているプロジェクトがクロスチェーン操作に拡大するための自然なパスを提供します。

注記

CircleのCross-Chain Transfer Protocol（CCTP）も言及に値する。CCIPと同様に動作するが、主にUSDCを異なるチェーン間でブリッジするために使用される。CCTPはChainlinkによってCCIPに統合されている。

これらのプロトコルに加えて、それぞれわずかに異なるニッチを埋める相互運用性レイヤーやブリッジングソリューションが増え続けています。例えばPolkadotやCosmosのようなプロジェクトは、パラチェーンやハブのような設計を活用してシームレスな資産とデータの交換を促進するために、最初からクロスチェーン機能を備えて構築されました。CosmosのInter-Blockchain Communication（IBC）プロトコルはクライアント検証を使用し、接続された各チェーンが他のチェーンの「ライトクライアント」を保存します。Polkadotはリレーチェーン内のバリデータの共有セットを通じてパラチェーンを保護し、各パラチェーンからのトランザクションを統一されたコンセンサスにバンドルします。これらのアーキテクチャはスケーラビリティとセキュリティを優先していますが、特にイーサリアム系の環境に慣れている開発者にとっては独自の習得コストを伴います。

まとめ

ここまで見てきたように、クロスチェーンプロトコルとオラクルは外部情報をコントラクトの実行に取り込むことで、スマートコントラクトに不可欠な機能を提供しています。もちろんオラクルは重大なリスクも伴います。信頼されたソースが侵害される可能性がある場合、オラクルが供給するスマートコントラクトの実行も侵害されかねません。オラクルの使用を検討する際は、信頼モデルについて十分に注意する必要があります。オラクルを無条件に信頼すると、スマートコントラクトは誤った入力に対して脆弱になる可能性があります。しかしセキュリティの前提条件を慎重に検討すれば、オラクルは非常に役立ちます。

分散型オラクルはこれらの懸念の一部を解決し、イーサリアムのスマートコントラクトにトラストレスな外部データを提供できます。慎重に選択すれば、オラクルが提供するイーサリアムと「現実世界」の間のブリッジを活用し始めることができます。

またクロスチェーンプロトコルがイーサリアムと他のエコシステムの間のブリッジとしてどのように機能するかも見てきました。オラクルに関連する潜在性とリスクの多くを引き継ぎつつ、ユースケースと機能の範囲をさらに拡大しています。

第12章 分散型アプリケーション（DApp）

この章では、DAppとは何か、どのように機能するか、コアアーキテクチャがどのように構成されているかを説明します。基本を説明した後、実践的なハンズオン形式の例を通じて最初のDAppをゼロから構築します。スマートコントラクトのデプロイ、フロントエンドの統合、スタック全体を本番環境へのデプロイに向けて整えるところまでを扱います。最後には、動作するDAppと、これらの革新的なアプリケーションを支える中核概念の確かな理解が得られます。

DAppとは

*DApp*は*分散型アプリケーション*（decentralized application）の略です。これは従来のアプリケーションとはまったく異なるパラダイムです。従来のアプリケーションは通常、図12-1に示すようなアーキテクチャを持っています:

- アプリのロジック部分のクローズドソースコード
- アプリケーションデータを保存するための中央集権的なデータベース
- ユーザーがアプリにアクセスするための単一のフロントエンド

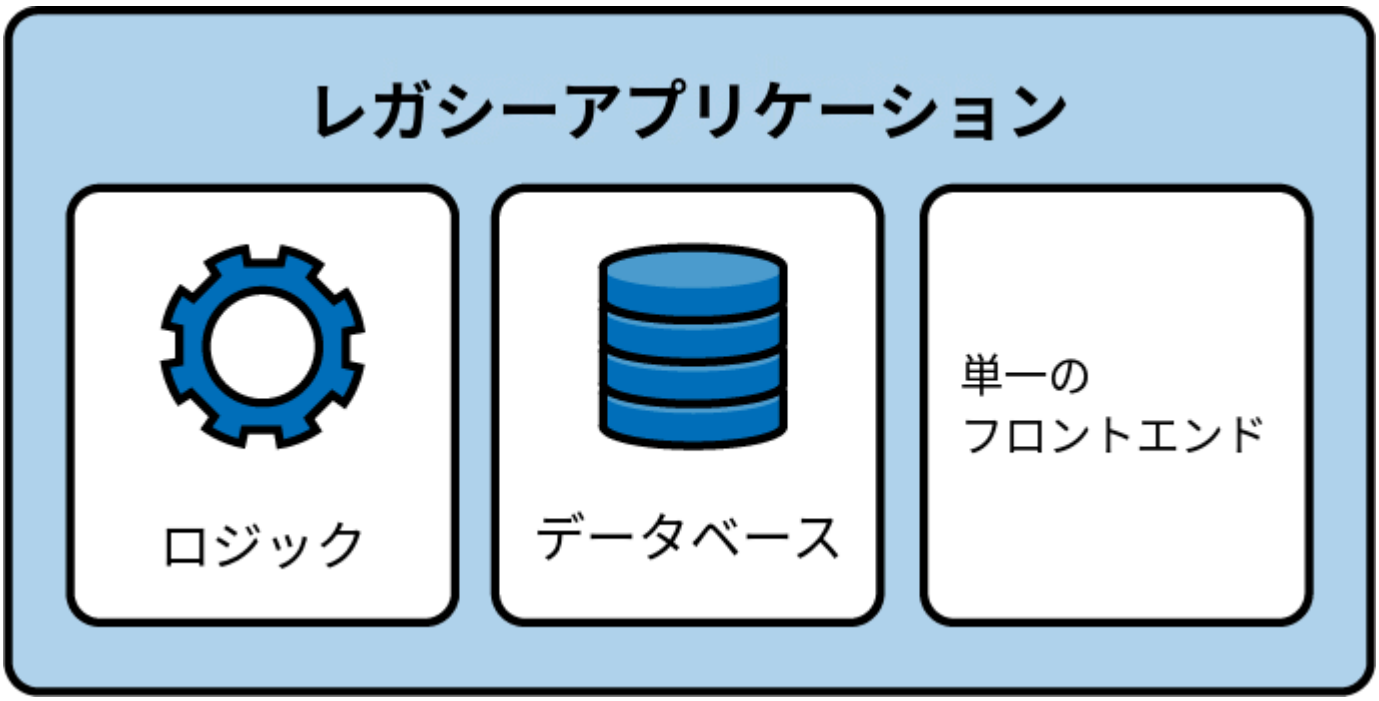


図12-1 従来のアプリケーションの一般的なアーキテクチャ

Instagram、TikTok、銀行アプリなど、今スマートフォンに入っているアプリケーションを思い浮かべてください。おそらく非常に似たアーキテクチャに依存しています。アプリケーションの背後にいるチームがアクセスを許可した場合にのみ利用できます。公式サイトがサービス停止中の場合にInstagramアカウントにログインできる代替ウェブサイトは存在しません。

DAppには2つの明確な目標があります。単一障害点を持たないことと、チーム全体がいなくなっても利用者が使い続けられるプロダクトであることです。DAppのアーキテクチャは、図12-2に示すように次のように整理できます:

- 複数のイーサリアムスマートコントラクトがDAppのロジック部分の基盤を形成する。多くの場合、Solidity（またはVyper）のコードもオープンソースである
- スマートコントラクトはデータも保持でき、データベースとして機能し、必要なユーザー情報をすべて収集する
- 公式フロントエンドを通じてアプリにアクセスできる。メインのフロントエンドが何らかの理由で動作しない場合、代替フロントエンド（コミュニティが作成したものを含む）で簡単に置き換えられる

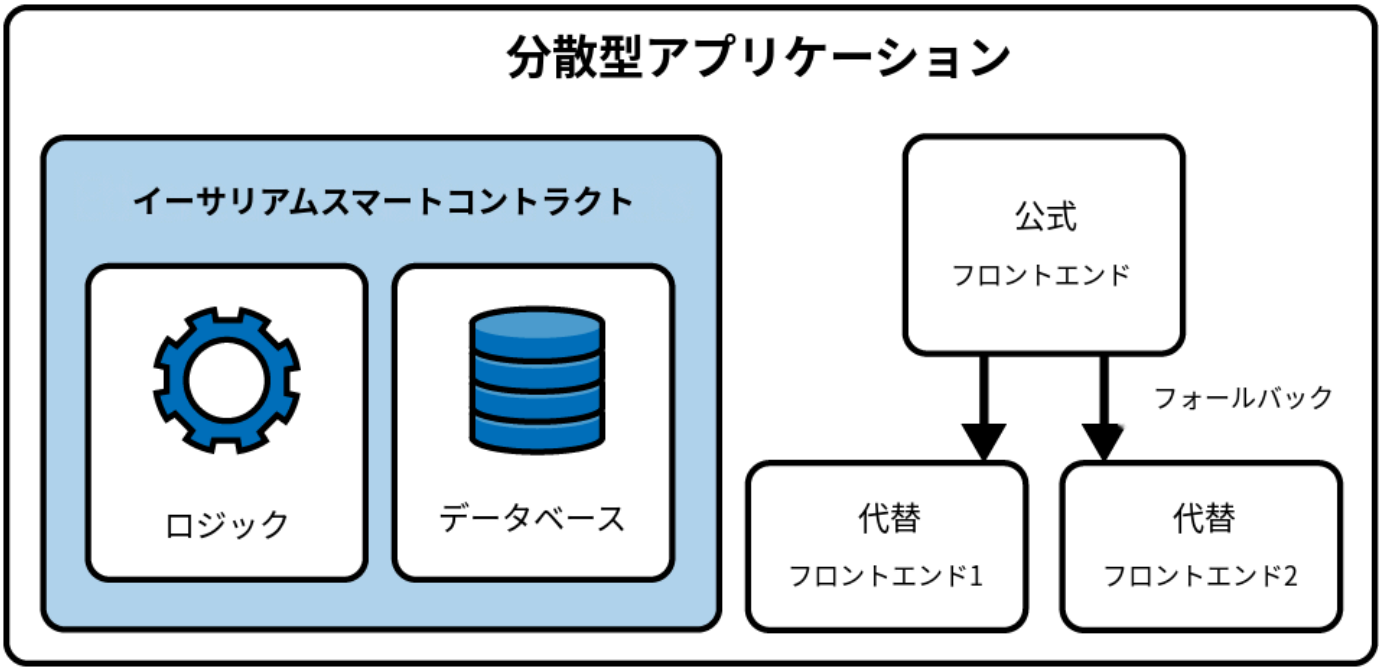


図12-2 DAppの一般的なアーキテクチャ

注記

DAppはある程度の中央集権化を伴うオフチェーンコンポーネントを持つこともできるが、通常これらはアプリケーションのコアロジックにとって不可欠なものではない。通常時にアプリケーションの高速化に役立つことはあるが、最悪の場合はオンチェーンデータだけで動作できるべきである。ただし、これは常に当てはまるわけではなく、中央集権的なコンポーネントにコアロジックが依存するDAppも存在する。それらは真の分散型アプリではなく、従来の中央集権型アプリケーションと完全な分散型アプリケーションの間のハイブリッド形態に近い。

次のセクションでは、DAppスタックの各コンポーネントをさらに掘り下げ、それぞれがどのように機能し、互いにどう関係しているか、そしてイーサリアムプロトコルとどう関わるかを理解します。

バックエンド（スマートコントラクト）

DAppでは、コアビジネスロジックとデータストレージをスマートコントラクトに記述し、中央集権的なサーバーではなくイーサリアムブロックチェーン上で実行します。ブロックチェーンが分散型バックエンドとして機能し、トランザクションの実行・状態の変更・記録の保持を、単一のエンティティではなくネットワーク全体がトラストレスに行います。

ユーザーはDAppにアクセスするために中央集権的なチームを信頼する必要がありません。イーサリアムネットワークは常に適切に動作し、曜日や時間に関係なくすべてのトランザクションを処理してスマートコントラクトの状態を正しく更新するからです。

さらに、このアーキテクチャは非常に強力な*検閲耐性*を持ちます。従来のアプリケーションでは、政府の法律などの理由でアプリへのアクセスが禁止されている国が多くあります。例えば、2025年時点でFacebookはブラジル、中国、イラン、北朝鮮、ミャンマー、ロシア、トルクメニスタン、ウガンダで禁止されています。これらの国の人々はプロフィールを作成することもプラットフォームにログインすることもできません。

イーサリアム上に構築されたDAppでは、この種の検閲は実現できません。特定のDAppの公式ウェブサイトを禁止することは可能ですが、あるアドレスがチェーン上のスマートコントラクトとインタラクションすることは誰にも防げません。誰でもDAppの代替フロントエンドを作成でき、それを使ってDAppと再びインタラクションできます。

Tornado Cashの物語

ここでTornado Cashの話に触れておきます。Tornado Cashは分散型ミキシングサービス[1]です。誰でもどこからでも、追跡可能な、あるいは「汚染された」暗号通貨を他の暗号通貨と混合できます。資金の実際の送信者と受信者の間のリンクをすべて断ち切ることで、元のソースへの追跡を困難にします。

[1] ミキシングサービス（Mixer）とは、複数のユーザーの資金をいったんプールして混合し、送金元と送金先のアドレスの対応関係を不明瞭にする技術・サービスのこと。プライバシー保護を目的とする一方、マネーロンダリングへの悪用が問題視されることも多い。

2022年8月8日、米国財務省の外国資産管理局（OFAC）はTornado Cashをブロックリストに登録し、米国市民と企業による使用を事実上禁止しました。このプラットフォームは70億ドル以上の暗号通貨のマネーロンダリングで告発されました。2日後の8月10日、Tornado Cashの開発者の1人であるAlexey Pertsevが、プラットフォームを作成したという理由だけでアムステルダムで逮捕されました。公式GitHubリポジトリは削除され、開発者アカウントも停止されました。2024年12月時点で公式ウェブサイトはアクセスできないままです。

これらを考えると、Tornado CashのDAppは運用を停止したと思うかもしれません。しかし現実はまったく異なります。これらの出来事以降、サービスの注目度（と流動性）は低下しました

が、プロトコルは完全に機能しており、さまざまなIPFSホスト型ゲートウェイを通じてアクセスできます。つまり、元の公式ウェブサイトがなくても、世界中の誰もが以前と同様にTornado Cashを使用できるのです。

データストレージ

データストレージとは、ユーザーのデータを保存するためのソリューションを指します。スマートコントラクトへのデータの保存と読み取りは可能ですが、コストが高くスケーラビリティにも課題があります。また、すべてをオンチェーンに保存する必要もありません。

通常、スマートコントラクトは重要な状態情報を保存し、DAppのロジックを強制します。アカウント残高、所有権記録、計算結果などの重要な情報はスマートコントラクトに直接保存します。これにより、機密性が高く価値のあるデータは完全に透明で改ざん耐性があり、イーサリアムノードを持つ誰もがアクセスできる状態を維持できます。

それ以外の情報は、従来のデータベースのような分散化の程度が低いデータストレージソリューションに保存できます。例えば、DApp開発者は高速なデータクエリのためにインデクサを使用したり、ユーザーデータの保存に中央集権的なデータベースを使用したりすることが多く、フロントエンドがイーサリアムチェーンに直接アクセスする必要がなくなります。

注記

*ブロックチェーンインデクサ*は、ブロックチェーンに保存されたデータを高速かつ効率的にクエリ・分析できるツールである。トランザクションデータを取得し、人と機械の双方が扱いやすい形式に変換してデータベースにロードする。

実際には、ブロックチェーン内のデータを直接「検索」することはできない。例えば、特定のアカウントがブロック15364050でどれだけのUSDCトークンを保有していたかを知りたいとする。USDCスマートコントラクトにアクセスして調べることはできない。履歴データを保存していないからである。そのブロックから現在までに発生したすべてのトランザクションを取得・フィルタリングし、そのアカウントに関連するすべてのUSDC転送情報を抽出して、ようやく答えが得られる。これは現実的なアプローチではない。ここでインデクサが活躍する。インデクサはデータベースのような構造を維持し、履歴情報を含む必要な情報へのクエリを即座に実行して素早く回答を返す。

重要なのは、誰もがDAppの正しい動作を検証できるように、本質的なデータとアプリケーションロジックのみをオンチェーンに保存するということです。それ以外はオフチェーンに置くことができますし、そうすべきです。

IPFS

*IPFS*は、P2Pネットワーク内のピア間でオブジェクトを分散して保存するコンテンツアドレッサブルストレージシステムです。*コンテンツアドレッサブル*とは、各コンテンツ（ファイル）をハッシュ化し、そのハッシュでファイルを識別する仕組みです。ハッシュを指定してリクエストすれば、任意のIPFSノードからファイルを取得できます。

IPFSはWebアプリケーションの配信プロトコルとしてHTTPを置き換えることを目指しています。単一のサーバーにWebアプリケーションを保存する代わりに、ファイルをIPFS上に保存し、任意のIPFSノードから取得できます。詳細についてはIPFSのドキュメントをお読みください。

マークルツリー

よく使われるソリューションの1つは、マークルツリー構造でデータをオフチェーンに保存し、マークルルートのみをオンチェーンに保存する方法です。すべてのデータをスマートコントラクトに保存する必要がなく（ガス代（トランザクション手数料）が非常に高くなるため）、それでもオンチェーンで検証を実行できます。

> **注記**
>
> 編集者注: 以下のコード例は特定の技術的コンテクストで「whitelist」に言及している。この用語には差別的・排他的な含みがあるが、業界とそのドキュメント全体で広く使用されている。包括性を尊重しつつも、著者はここでは技術的概念の明確さのために用語をそのまま維持した。

最も一般的なユースケースは、NFTコレクションを作成し、特定のアドレスをホワイトリストに登録して、パブリックセール開始前に低い価格でNFTをミントできるようにする場合です。方法は2つあります。

1つ目は、スマートコントラクト内にストレージ変数を作成し、各アドレスをブーリアン値にマッピングする方法です。ホワイトリストに登録されたアドレスにはtrueを設定します。このマップで特定のアドレスがホワイトリストに登録されているかを確認できます。ユーザーはミントトランザクション送信時に何も提供する必要がなく、コントラクトは`msg.sender`がホワイトリストマップに含まれているかを確認するだけです:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

// import OpenZeppelin contracts.
import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

// A simplified NFT contract with a whitelist mint function that uses a mapping to
```

```
// store whitelisted addresses.
contract MyWhitelistNFT is ERC721, Ownable {
    // ... rest of the contract

    // mapping for whitelisted addresses
    mapping(address => bool) public isWhitelisted;

    /**
     * @notice Whitelist mint function.
     */
    function whitelistMint() payable {
        // ... rest of the function

        // check if the user is whitelisted.
        require(isWhitelisted[msg.sender], "You are not whitelisted");

        // mint the NFT.
        _safeMint(msg.sender, nextTokenId);
        nextTokenId++;
    }
}
```

2つ目は、マークルツリーをオフチェーンで作成し、マークルルートのみをコントラクト内にオンチェーンで保存する方法です。ホワイトリストに登録された各ユーザーにマークル証明を渡し、ユーザーがそのマークル証明をコントラクトに送信すると、コントラクトがオンチェーンで証明の有効性を検証します（通常はライブラリを使用）：

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

// import OpenZeppelin contracts.
import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/cryptography/MerkleProof.sol";

// A simplified NFT contract with a whitelist mint function that uses a merkle tree to
// store whitelisted addresses.
contract MyWhitelistNFT is ERC721, Ownable {
    // ... rest of the contract

    // the merkle root of the off-chain generated Merkle Tree.
    bytes32 public merkleRoot;

    /**
     * @notice Whitelist mint function.
     * @dev User must provide a merkle proof to prove they are whitelisted.
     * @param _merkleProof The proof that msg.sender is whitelisted.
     */
    function whitelistMint(bytes32[] calldata _merkleProof) external payable {
```

```
        // ... rest of the function

        // verify that (msg.sender) is in the merkle tree using the provided proof.
        bytes32 leaf = keccak256(abi.encodePacked(msg.sender));
        bool isValidLeaf = MerkleProof.verify(_merkleProof, merkleRoot, leaf);
        require(isValidLeaf, "Invalid Merkle Proof: Not whitelisted");

        // mint the NFT.
        _safeMint(msg.sender, nextTokenId);
    }
}
```

2つ目のオプションは、特に大規模なホワイトリストの場合、1つ目よりもはるかに安価で効率的です。

> **注記**
>
> この方法はガスコストを大幅に削減するが、ツリーと証明が監査可能でない場合、悪意あるホワイトリスト作成者による不正のリスクが生じる。理想的には、ツリーの生成に使用した元のリストをオープンソースで公開し、マークルルートの有効性を誰でも検証できるようにすべきである。

フロントエンド（ウェブユーザーインターフェース）

DAppのフロントエンドは、React、Angular、Vueなどの代表的なWeb2フレームワークで作成します。必要に応じて、イーサリアムチェーンとのインタラクションは`viem`や`ethers.js`のようなライブラリで抽象化します。

DAppフロントエンドを構築する素朴なアプローチは、すべてのコンポーネントの更新にチェーンから直接データを読み取ることです。例えば、あるアカウントのトークン残高を表示する場合、ブロックチェーンにクエリして回答を取得し、新しいブロックごとにこの処理を繰り返します。

この素朴なアプローチの主な問題は、ウェブサイトが非常に遅くなり、ユーザーが操作するうえで強いストレスを感じるフロントエンドになることです。前述のとおり、開発者がDAppアーキテクチャに中央集権的なデータストレージコンポーネントを使用し、チェーンとのインタラクションを最小限にするのはこのためです。そのため通常のフロントエンドは一部の中央集権的なコンポーネントに依存しますが、ブロックチェーン上で確認すればすべての情報をいつでも検証できます。公式フロントエンドを信頼しない場合や気に入らない場合は、特定のDApp用に独自の代替フロントエンドを作成することもできます。

図12-3は完全な（簡略化された）DAppアーキテクチャを示しています。

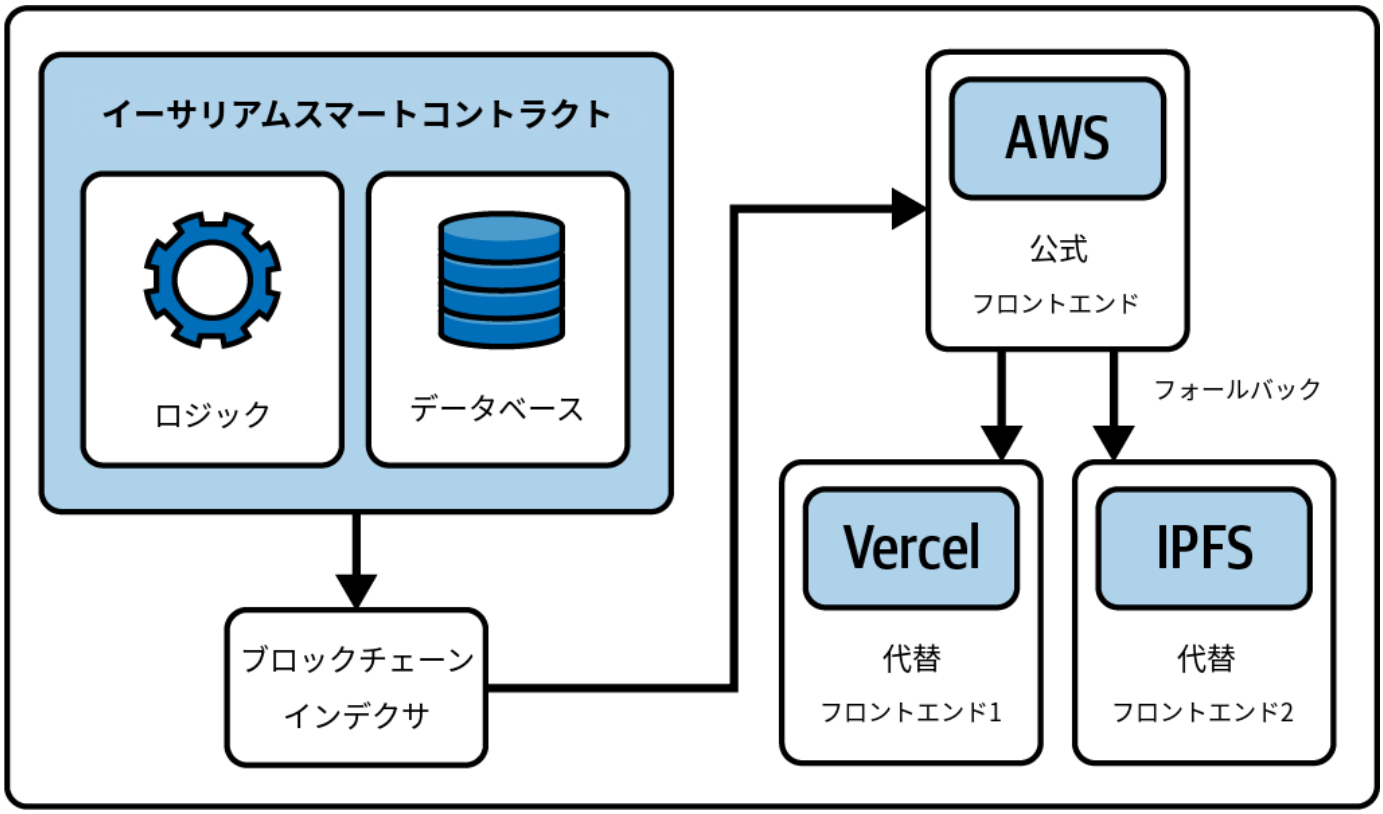


図12-3 完全なDAppアーキテクチャ

基本的なDAppの例

ここまでDAppの基本的な概念を見てきました。次は実際にDAppを自分で構築してみましょう。

イーサリアム上でDAppをゼロから構築するチュートリアルはオンラインに数多くありますが、Speedrun Ethereumを強くお勧めします。素早く学んですぐに構築を始めるには最も効果的な方法です。イーサリアム上でのDApp構築の知識を深めるために、Speedrun Ethereumのすべてのチャレンジを完了し、BuidlGuidlコミュニティに参加することをお勧めします。

このセクションでは、非常に基本的な分散型アプリケーション、いわば「Hello World」DAppを構築します。事前の経験は不要で、必要なのはコンピュータとインターネット接続だけです。

インストール要件

このチュートリアルに従うには、コンピュータにNode.jsとyarnのインストールが必要です。公式ウェブサイトからダウンロードしてインストールしてください。開発環境を素早く作成できる便利なツールScaffold-ETH 2を使用します。

DAppの作成

ターミナルを開いて以下のコマンドを実行しましょう:

```
$ npx create-eth@latest
```

プロジェクト名を尋ねられます。このデモでは「mastering-ethereum」を選びました:

```
 +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
 | Create Scaffold-ETH 2 app |
 +-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+

? Your project name: mastering-ethereum
```

次に、使用するSolidityフレームワークを尋ねられます。ここではHardhatを使用しますが、慣れているものを自由に選んでください:

```
? What solidity framework do you want to use?
❯ hardhat
   foundry
  none
```

数秒後に、「Congratulations」と以下のような「Next steps」が表示されるはずです:

```
  Congratulations! Your project has been scaffolded! 🎉

  Next steps:

  cd mastering-ethereum

        Start the local development node

        yarn chain

        In a new terminal window, deploy your contracts

        yarn deploy

    In a new terminal window, start the frontend

    yarn start

 Thanks for using Scaffold-ETH 2 🙏 , Happy Building!
```

チェーンの起動

DAppの構築を開始する準備が整いました。プロジェクトフォルダに移動します:

```
$ cd mastering-ethereum
```

`packages/hardhat/contracts` に移動すると、`YourContract.sol` というサンプルスマートコントラクトがあります。`contracts` フォルダはDAppプロジェクトに必要なすべてのスマートコントラクトを配置する場所です。

`packages/nextjs` 内にはDAppフロントエンドのためのNext.jsフレームワーク構造が既に配置されています。

今回は非常に基本的なチュートリアルなので、スマートコントラクトをゼロから書いたりフロントエンドを変更したりはしません。通常のワークフローを素早く示すためにデフォルトのままにしておきます。

まず、ローカル開発用のチェーンを起動する必要があります。最終的なプロダクトはイーサリアムメインネットにデプロイしたスマートコントラクトを使用しますが、DAppの構築とテストに実際のチェーンを使用すべきではありません。非常に遅く、不要なコストが発生するからです。Scaffold-ETHにはローカル開発用の新しいチェーンを即座に起動する便利なコマンドが付属しています。以下を実行するだけです:

```
$ yarn chain
```

コントラクトのデプロイ

次に、前のステップで設定したローカルチェーンにコントラクトをデプロイします。ここでもScaffold-ETHには簡単なコマンドがあります。新しいターミナルを開いて以下を入力してください:

```
$ yarn deploy
```

以下のような出力が表示されるはずです:

```
Generating typings for: 2 artifacts in dir: typechain-types for target: ethers-v6
Successfully generated 6 typings!
Compiled 2 Solidity files successfully (evm target: paris).
deploying "YourContract" (tx:
0x8ec9ba16869588c2826118a0043f63bc679a4e947f739e8032e911475e77dcb4)...: deployed at
0x5FbDB2315678afecb367f032d93F642f64180aa3 with 532743 gas
👋  Initial greeting: Building Unstoppable Apps!!!
📝  Updated TypeScript contract definition file on ../nextjs/contracts/deployedContracts.ts
```

このコマンドは`YourContract`というサンプルスマートコントラクトをローカルチェーンにデプロイします。新しいDAppを構築する際には`packages/hardhat/deploy`の`00_deploy_your_contract.ts`ファイルを変更し、必要なすべてのコントラクトをデプロイできるようにします。

先ほど`yarn chain`コマンドを実行したターミナルに戻ると新しいログが表示されているはずです。特に以下のようなログです:

```
eth_sendTransaction
  Contract deployment: <UnrecognizedContract>
  Contract address:    0x5fbdb2315678afecb367f032d93f642f64180aa3
  Transaction:         0x8ec9ba16869588c2826118a0043f63bc679a4e947f739e8032e911475e77dcb4
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
  Value:               0 ETH
  Gas used:            532743 of 532743
  Block #1:            0xa7b8e3b6f82eccb3542279573dbf8efa2b876ff00807a8619feef191007e06d9
```

`Contract deployment`と`Contract address`の行に注目してください。ローカルチェーン上の特定のコントラクトアドレスにコントラクトが正常にデプロイされたことを示しています。

フロントエンドの起動

Scaffold-ETHのサンプルには基本的な組み込みフロントエンドが付属しており、グラフィカルインターフェースですぐにコントラクトとインタラクションを開始できます。新しいターミナルウィンドウを開いて以下を入力してください:

```
$ yarn start
```

以下のような出力が返されるはずです:

```
yarn start
  ▲ Next.js 14.2.21
  - Local:        http://localhost:3000

 ✓ Starting...
 ✓ Ready in 1767ms
```

localhostのURLをコピーしてブラウザに貼り付けてください。図12-4に示すようなフロントエンドが表示されるはずです。

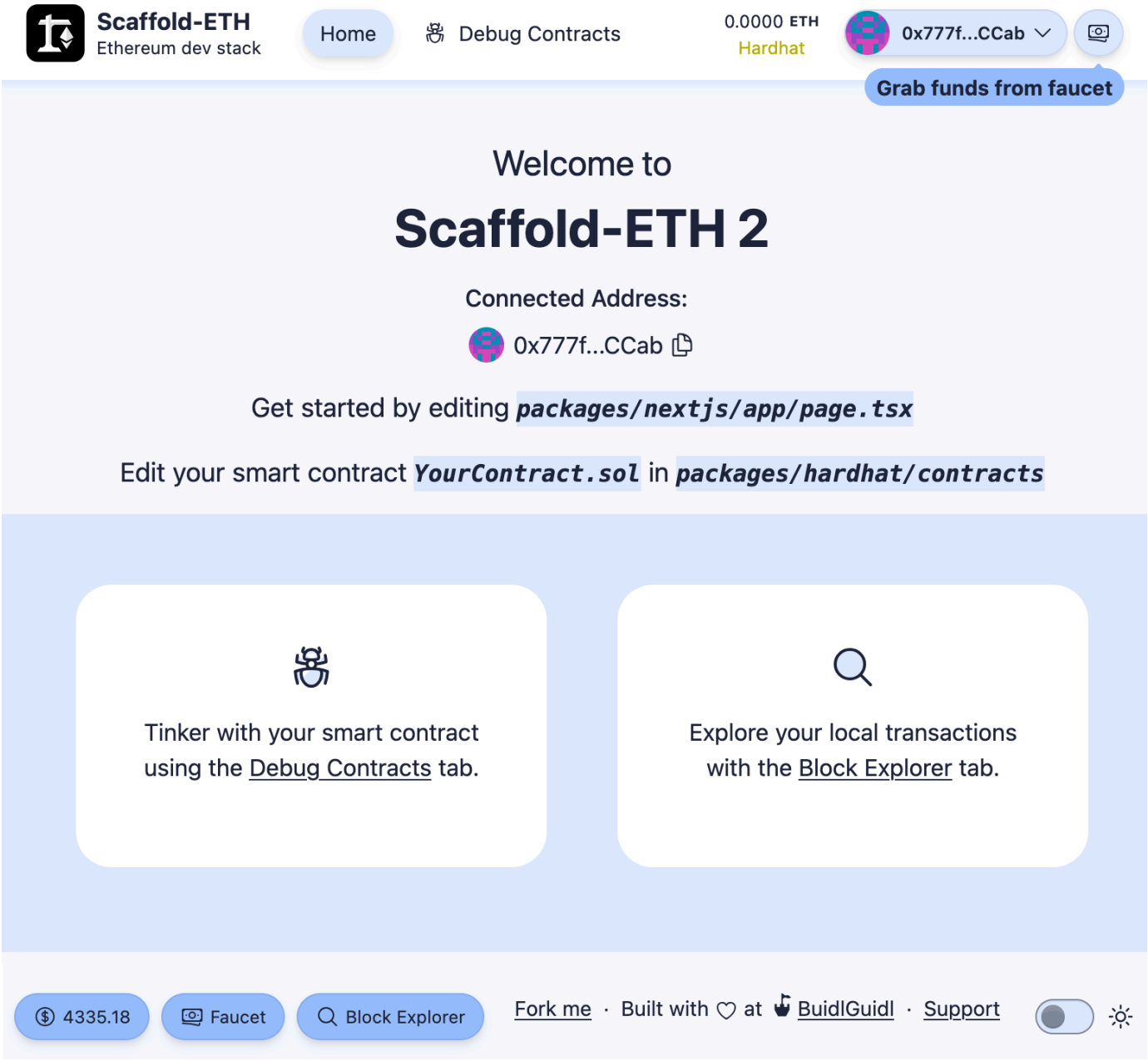


図12-4 Scaffold-ETHフロントエンド

コントラクトとのインタラクション

おめでとうございます。すべてのセットアップが完了しました。DAppでの実験や操作を始められます。開発ワークフローに不可欠な便利な機能がいくつかあります:

- バーナーウォレット
- Debug Contractsセクション

図12-4のとおり、右上隅には、ランダムに生成された見慣れないウォレットがすでにウェブサイトに接続されています。これがバーナーウォレットです。Webブラウザ内に秘密鍵が一時的に保存される自動生成アドレスです。ページを更新してもバーナーアドレスは変わりません。

バーナーウォレットは開発ワークフローを大幅に便利にする機能です。毎回Web3ウォレットを開いてウェブサイトに接続する必要がありません。もちろん準備ができたらウォレットを接続することもできます。ドロップダウンメニューからDisconnectを選択し、Connect Walletをクリックしてリストから好みのウォレットを選択してください。図12-5、12-6、12-7に示します。

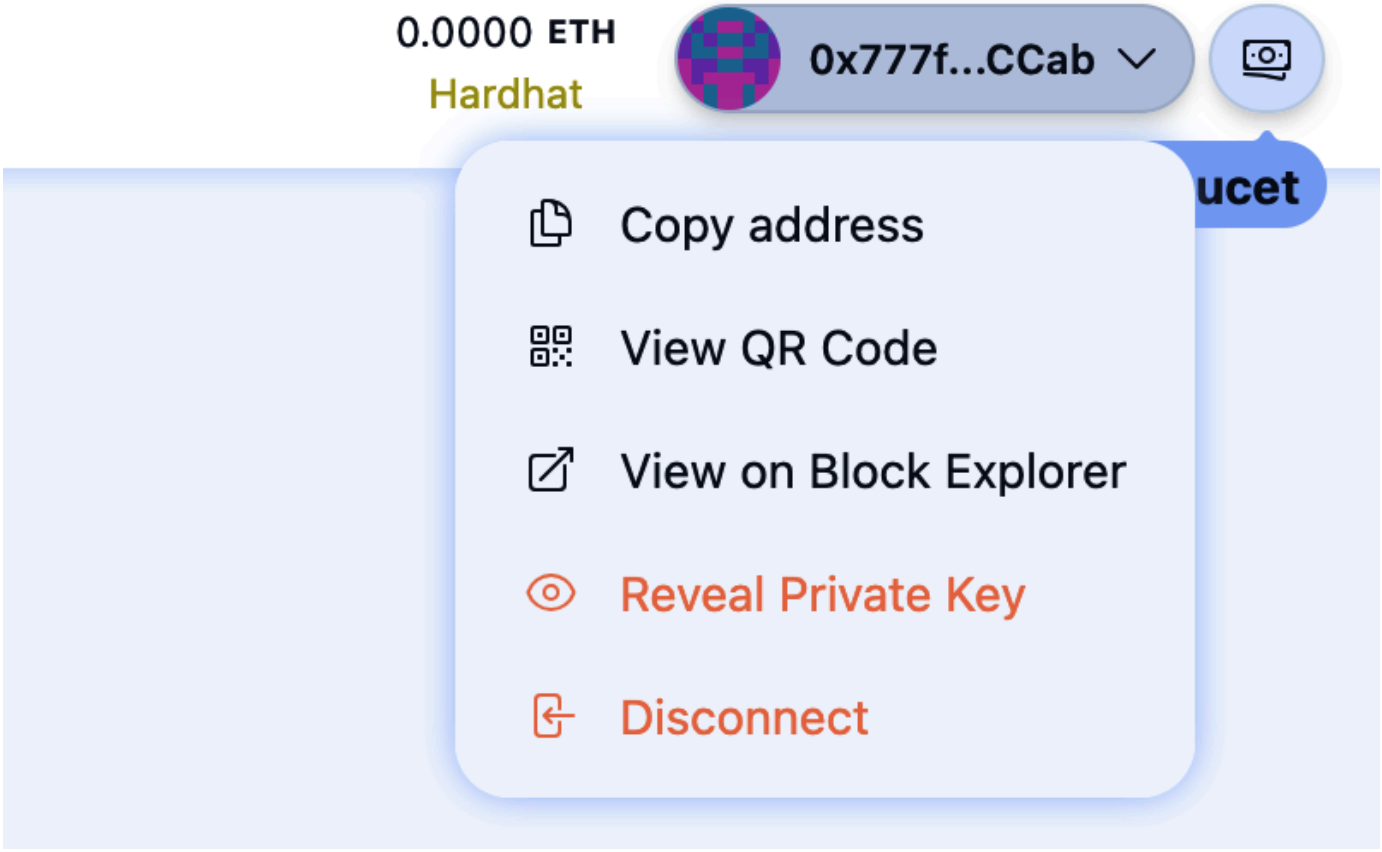


図12-5 バーナーウォレットの切断

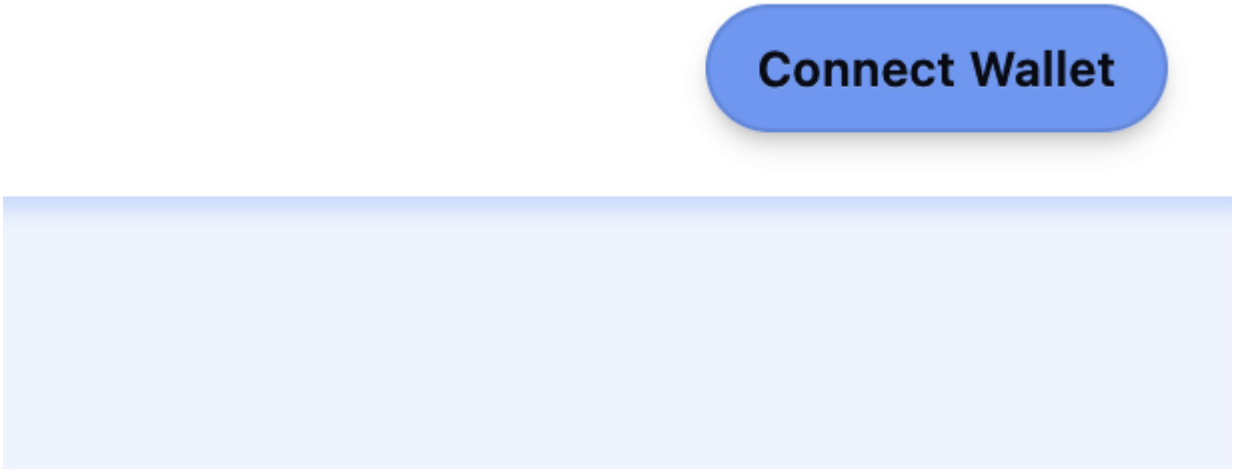


図12-6 Connect Walletボタン

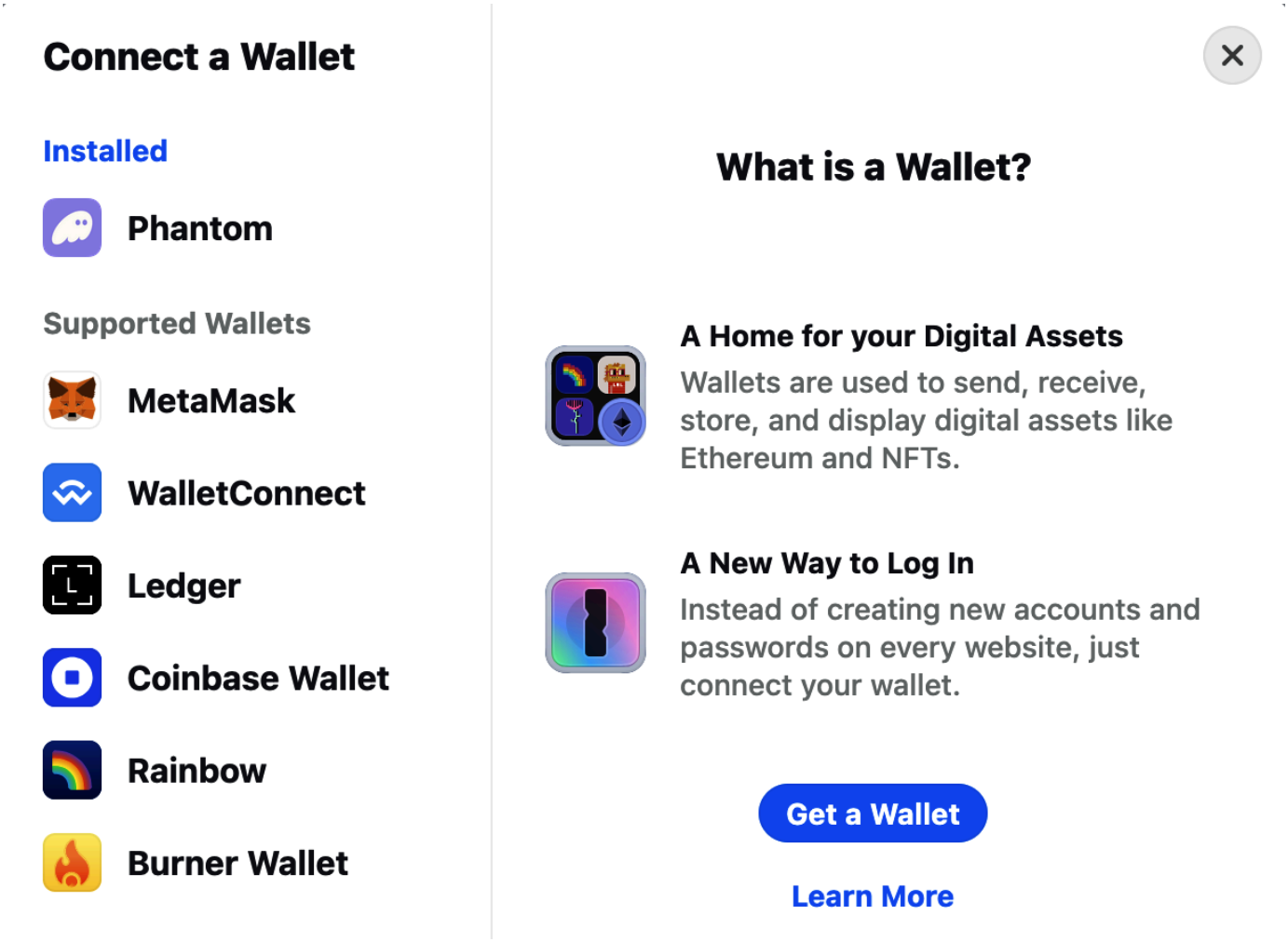


図12-7 ウォレットの選択

2つ目の強力な機能はDebug Contractsセクションです。ページ中央のDebug Contractsリンクをクリックして開きます。デフォルトのサンプルでは図12-8のような表示になるはずです。

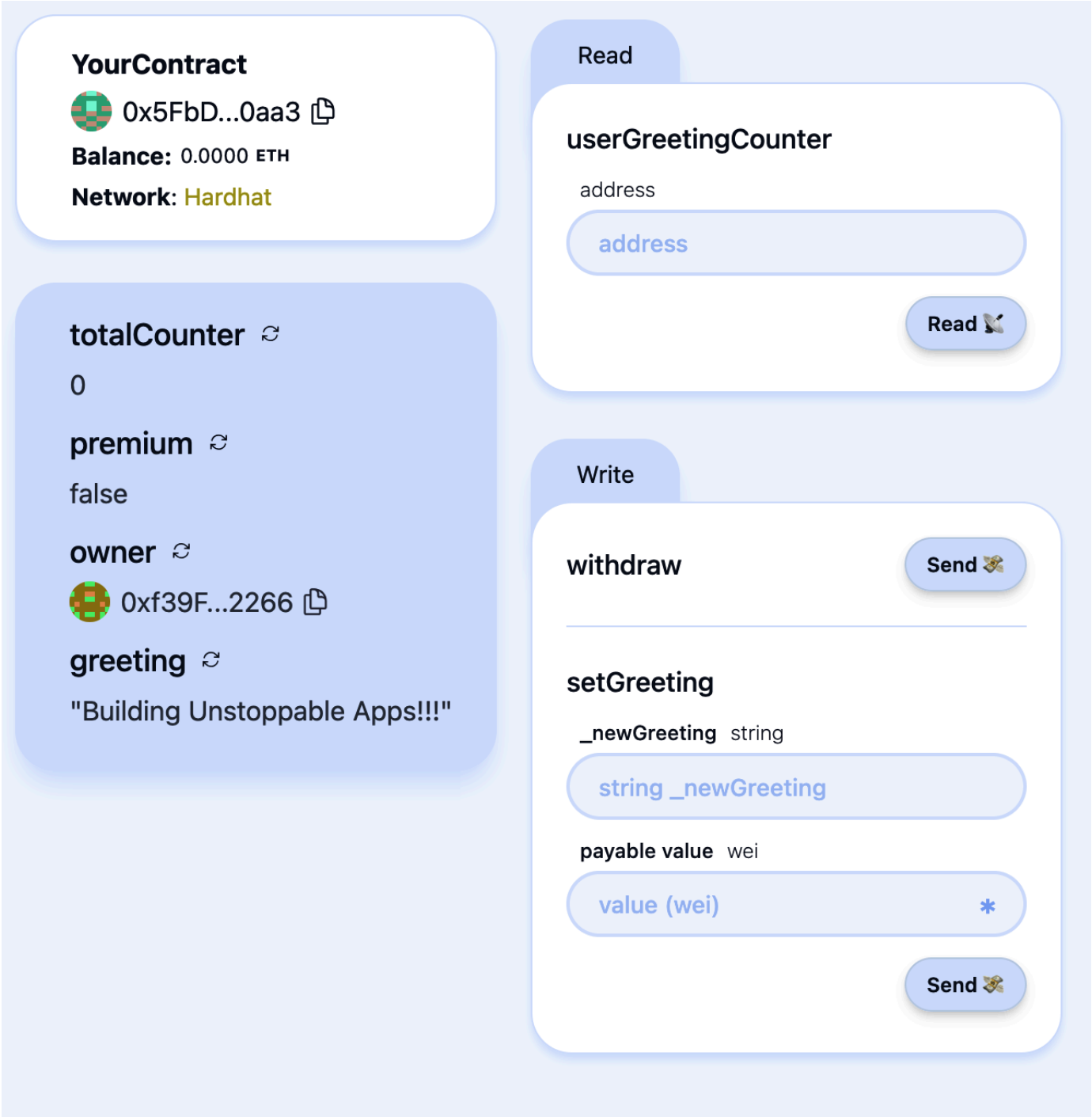


図12-8 Debug Contractsセクション

ここではフロントエンドを構築しなくても、すべてのコントラクトと簡単にインタラクションできます。開発中にコントラクトが期待どおりに動作しているか確認するのに非常に便利です。

簡単なデモンストレーションをしましょう。まず、デプロイ済みコントラクトへトランザクションを送信できるようにバーナーウォレットに資金を供給します。図12-9に示す右端のボタンをクリックするだけです。ほぼ即座にETHを受け取り、残高が増えるのが確認できます。

図12-9 Grab fundsボタン

次に`setGreeting`セクションに移動し、「_newGreeting」フィールドに`hello world`、「payable value」フィールドに`0.1`と入力します。「payable value」フィールドの右にあるアスタリスクをクリックすると、ETHの値がwei（イーサリアムの最小単位、1 ETH = 10^{18} wei）に変換されます。最後にSendをクリックしてトランザクションを送信し、コントラクトの状態がどう変化するか確認しましょう。

図12-10はSendボタンを押す前のコントラクトの状態です。コントラクトは0 ETHを保持し、greetingは「Building Unstoppable Apps!!!」、premiumはfalse、totalCounterは0です。

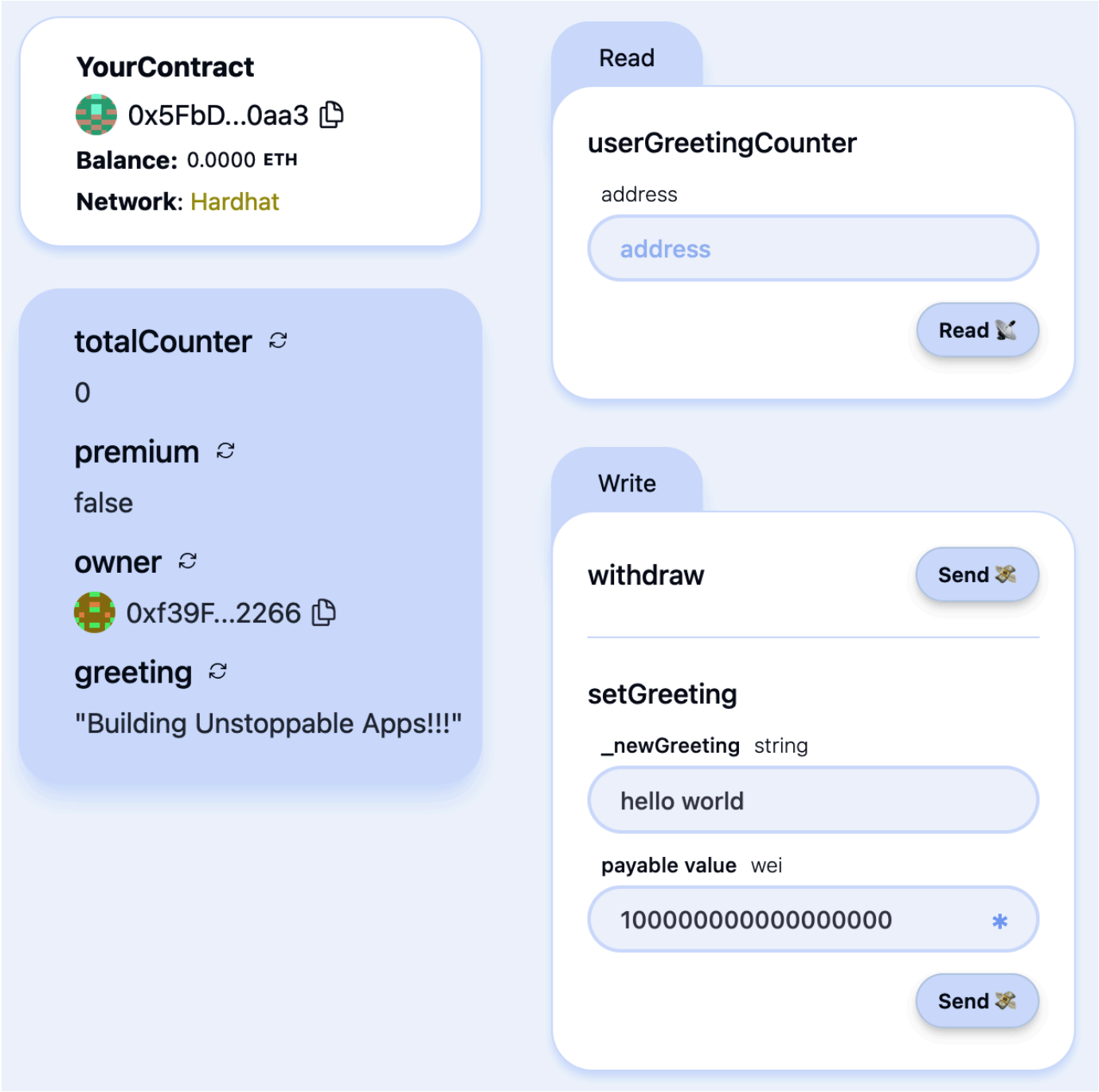


図12-10 トランザクション前のコントラクト状態

図12-11はトランザクション送信後のコントラクトの状態です。コントラクトは0.1 ETHを保持し、greetingは「hello world」、premiumはtrue、totalCounterは1に変わっています。

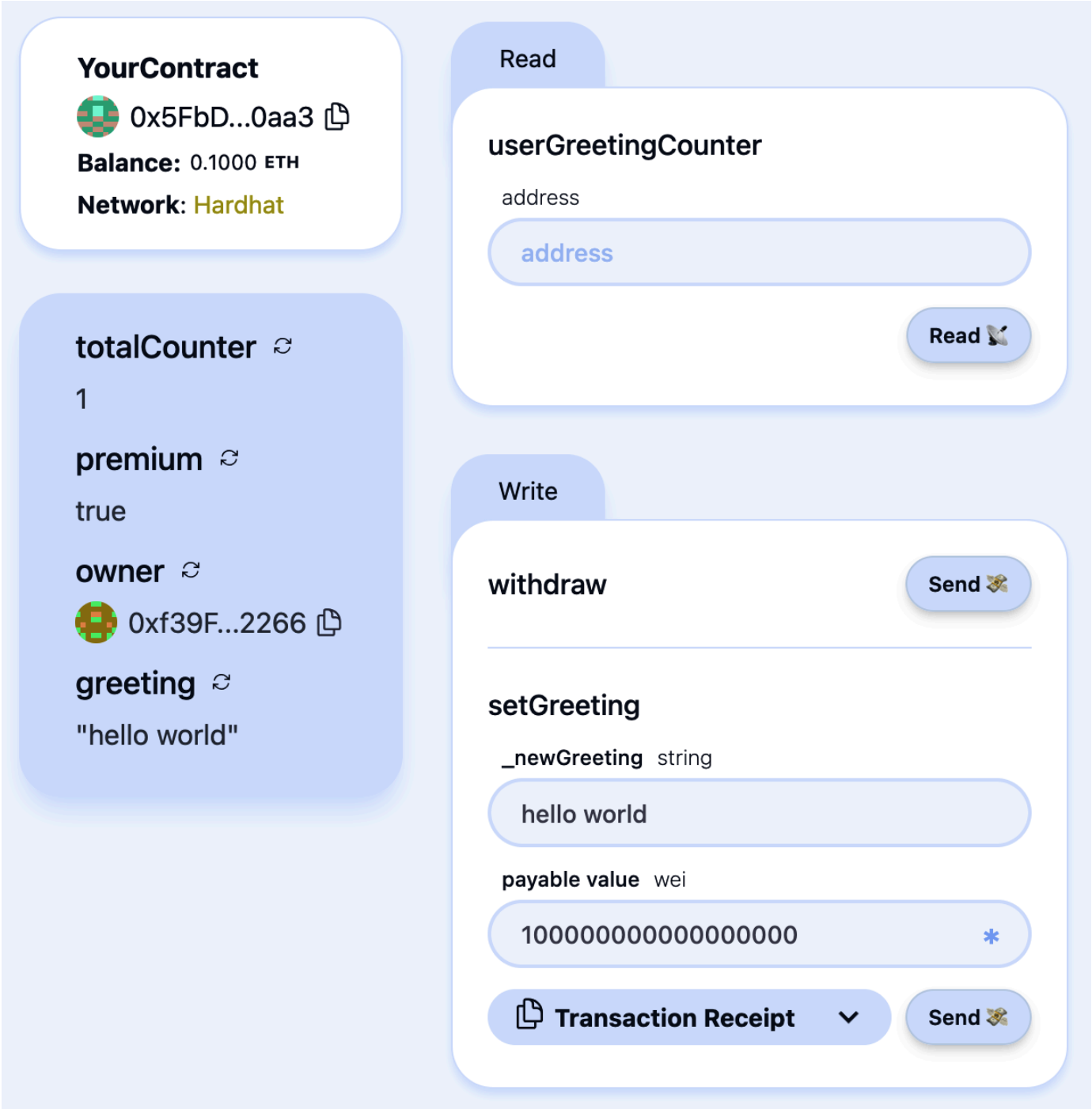


図12-11 トランザクション後のコントラクト状態

いろいろ試して、入力やアクションに応じてコントラクトがどう動作するか確認してみてください。

Vercelへのデプロイ

分散型アプリケーションが完成したら、Vercelのような本番環境に公開できます。Vercelはアプリケーションをインターネットに簡単にデプロイできるフロントエンドのホスティングサービスです。購入したカスタムドメインを設定すれば、ドメイン名を入力するだけでDAppにアクセスできるようになります。

Scaffold-ETHはここでもDAppをVercelに即座にデプロイするコマンドを提供しています。ターミナルを開いて以下を入力します:

```
$ yarn vercel:yolo
```

Vercelアカウントをリンクし（持っていない場合は新規作成し）、プロジェクト名を選択するだけです。数分後にはDApp全体がVercelにデプロイされ、世界中の誰もが試せるようになります。

Vercelプロファイルに移動すると新しく作成されたプロジェクトが表示されます。図12-12のとおり、Domainsフィールドに Vercelが自動生成したウェブサイトドメインがあります。

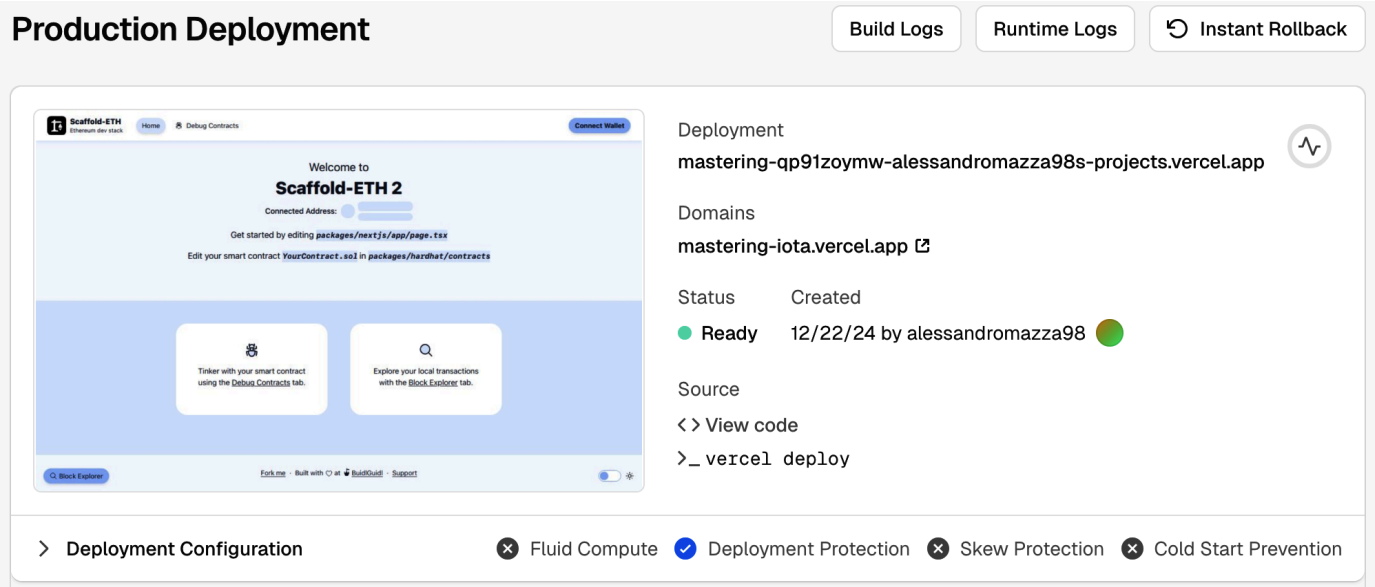


図12-12 Vercelプロジェクトページ

DAppのさらなる分散化

前のセクションではDAppをVercelにデプロイしました。VercelはDAppのホスティングで最も人気のあるソリューションの1つですが、分散型サービスではありません。すべてがVercel独自のサーバー上にあり、Vercelのポリシーに基づいてユーザーを検閲したり、DAppとインタラクションするすべての人のIPアドレスを追跡したりする可能性があります。

DAppをさらに分散化するには、IPFSのようなグローバルP2Pノードネットワーク上にフロントエンドをホストできます。注目すべき新興サービスとして`eth.limo`があります。中央集権的なプラットフォームが提供する通常のウェブサイト体験と、IPFSや類似技術による堅牢性と分散化の両立を目指しています。

分散型ウェブサイト

Eth.limoは、より優れた分散型ウェブサイト（*DWebサイト*とも呼ばれる）を実現するための鍵となるサービスで、通常のアプリと同じ方法でアクセスできます。ENS技術に基づき、`.eth`ドメイン名でイーサリアムアドレスをユーザーフレンドリーにします。イーサリアムの共同創設者の1人であるヴィタリック・ブテリンは、ENSで自身のウォレットアドレス`0xd8dA6BF26964aF9D7eEd9e03E53415D37aA96045`を覚えやすい名前`vitalik.eth`とリンクしています。

ENS（Ethereum Name Service）は単にドメインとイーサリアムアドレスをリンクするだけでなく、IPFSウェブサイトのコンテンツハッシュ（`bafybeif3dy…ga4zu`のような文字列）にも解決できます。IPFSウェブサイトの主な問題は、ほとんどの主要Webブラウザがこれらを適切に解決してコンテンツを表示できないことです。ここで`eth.limo`が登場します。このサービスはENS名とIPFSコンテンツのリバースプロキシとして動作します。`*.eth.limo`（`.eth.limo`で終わるすべてのウェブサイト）へのリクエストをキャプチャし、該当するENSレコードからIPFSコンテンツハッシュを解決して、HTTPS経由で静的コンテンツを返します。例えばヴィタリック・ブテリンの公式ブログにアクセスすると、裏側では以下のことが起きています:

1. Eth.limoが`vitalik.eth` ENSドメインへのリクエストを受信する。
2. ヴィタリックが事前に設定した、ウェブサイトのホームページを含むIPFSコンテンツハッシュに解決する。
3. HTTPS経由で対応する静的コンテンツを返す。

これにより、Webブラウザさえあれば世界中の誰もが設定やセットアップなしでIPFS上のコンテンツに簡単にアクセスできます。

制限事項

DWebサイト全般、特に`eth.limo`は急速に進化している技術ですが、この章の執筆時点（2025年6月）ではまだいくつかの制限があります。まず、IPFSは静的ファイルのみを処理できるため、サーバーサイドの計算は実行できません。また、`eth.limo`を使用するにはENSを購入してフロントエンドのIPFSコンテンツハッシュにリンクする必要があります。さらに、常に`*.eth.limo`カスタムドメインを使用しなければなりません。末尾の`.limo`部分を削除するとWebブラウザはENS名をDApp用のIPFSフロントエンドに解決できません。これがおそらく多くのDAppが`eth.limo`を使用しない理由です。

> **ヒント**
>
> ほとんどのWebブラウザはまだENSとIPFSに対応していないが、一部のブラウザはサポートを追加し始めている。一例としてBraveがある。

`eth.limo`自体は潜在的に中央集権的なサードパーティであり、予告なく動作を停止する可能性がある点にも注意が必要です。停止した場合、DAppはIPFSを通じて引き続きアクセスできます

が、`.eth.limo`のURLはユーザーをDAppにリダイレクトしなくなります。

完全に分散型のウェブサイトを構築するこのソリューションに興味がある場合は、公式ウェブサイトで詳細を確認できます。

IPFSへのデプロイ

Scaffold-ETH 2にはDAppをIPFSに素早くプッシュできるコマンドもあります。新しいターミナルを開いて以下を入力するだけです:

```
$ yarn ipfs
```

これで完了です。以下のような出力が表示されるはずです:

```
  Creating an optimized production build ...

 ✓ Compiled successfully
 ✓ Linting and checking validity of types

 ✓ Collecting page data

 ✓ Generating static pages (8/8)
 ✓ Collecting build traces

 ✓ Finalizing page optimization

...

🚀  Upload complete! Your site is now available at:
https://community.bgipfs.com/ipfs/bafybei…
```

表示されたウェブサイトにアクセスすると、IPFSでホストされたフロントエンドでDAppが正常に動作しているはずです。「bafy…」という文字列がIPFSコンテンツハッシュです。個人のENSを持っていてこのIPFSホスト型サイトにリダイレクトしたい場合は`eth.limo`の設定が必要です。

`yarn ipfs`コマンドの裏側で実際に起きていることは以下の通りです:

1. フロントエンドがビルドされ、IPFSにアップロードする準備が整った静的ファイルが作成される。
2. 静的ファイルがBuidlGuidl（Scaffold-ETH 2のメンテナー）のIPFSコミュニティノードを通じてIPFSにアップロードされる。
3. IPFSコンテンツのリバースプロキシとして機能するURL（`community.bgipfs.com/<ipfs-content-hash>`）が返される。

独自のIPFSノードを実行してクラスターをピン留めする方法については、BuidlGuidl IPFSを参照してください。

アプリからDAppへ

ここまでのセクションで、分散型アプリケーションを段階的に構築してきました。開発ワークフローを効率化するためにScaffold-ETHを使用し、ローカルチェーンを起動してコントラクトをデプロイし、ローカルフロントエンドですぐにDAppのインタラクションとテストを開始しました。次にDAppのフロントエンドをVercelに公開し、本番環境へのデプロイがいかに簡単かを示しました。最後にフロントエンドをIPFSにアップロードし、ENSや`eth.limo`のようなソリューションでDAppをさらに分散化する方法を見ました。これにより特別なアプリをインストールしなくても誰でもアクセスできるようになります。

図12-13は、完全に分散化されたアプリケーションを作成するために必要なエンジニアリングスタックの概要です。

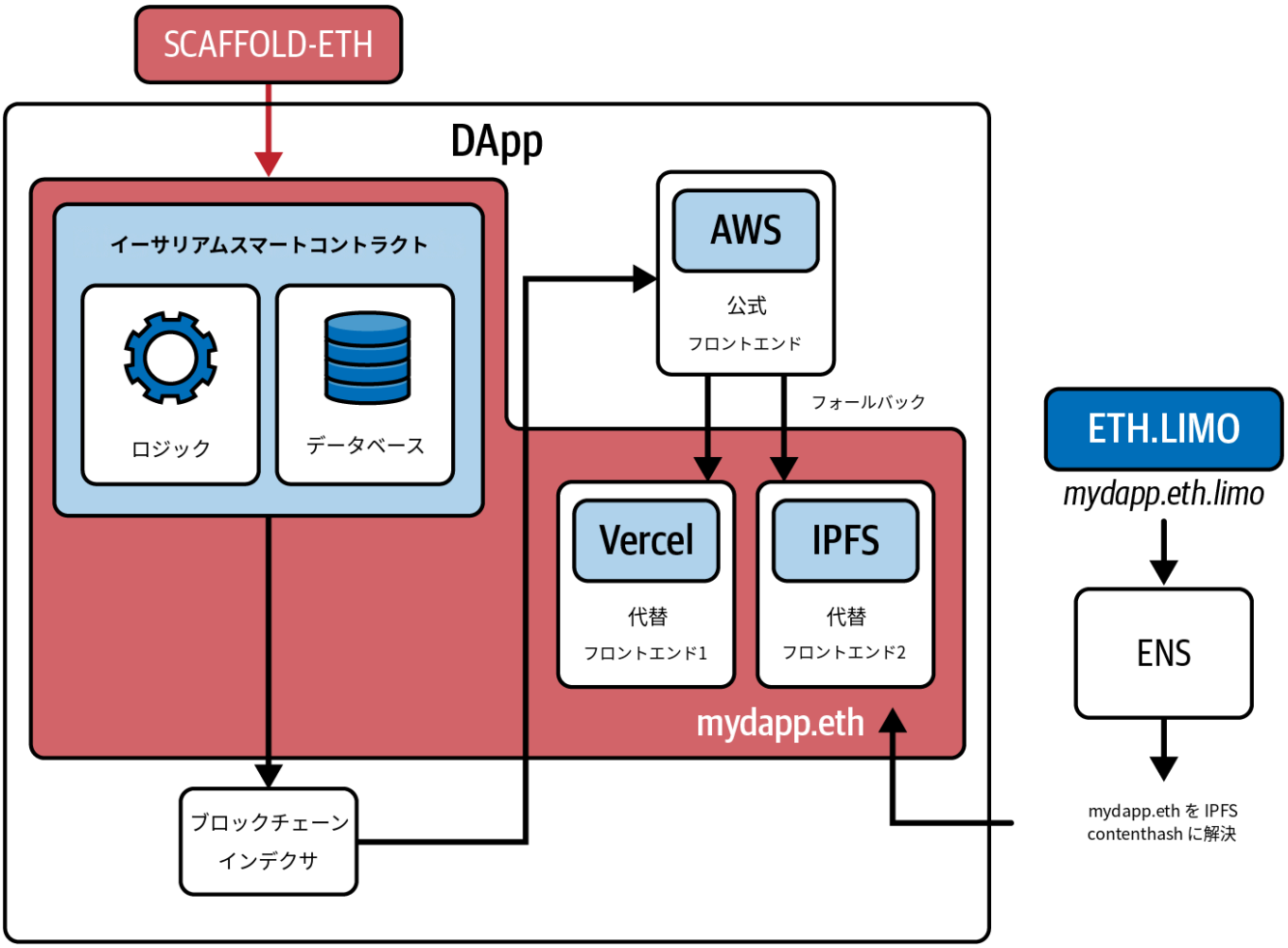


図12-13 DAppのフルエンジニアリングスタックの簡潔な概要

まとめ

この章では、最新のツールで開発ワークフローを効率化しながら、基本的なDAppをゼロから構築する方法を見てきました。次の章では、イーサリアム上で最も重要なDAppとDAppのカテゴリを詳しく見ていきます。これらは総称してDeFiと呼ばれています。

第13章 分散型金融（DeFi）

イーサリアムのスマートコントラクトは、単純な暗号資産の取引を超えた可能性の世界を切り開きました。*分散型金融*（DeFi）は、ブロックチェーン上で完全に動作する包括的な金融エコシステムを構築することで、この可能性をさらに一段上へと引き上げています。貸し付け、借り入れ、取引、投資といった従来の金融サービスを、銀行やブローカーなどの中央集権的な機関なしで提供できる世界を想像してみてください。イーサリアムブロックチェーン上のスマートコントラクトがすべてを処理し、金融における自律性とイノベーションの新時代をもたらしています。

この分散型のアプローチは金融サービスへのアクセスを民主化し、従来の金融では欠けていることが多い透明性とセキュリティをもたらします。ブロックチェーン上のすべてのトランザクションは公開されており、変更不可能なため、誰でもデータの真正性と完全性を検証できます。この透明性により不正や腐敗のリスクが低減し、より信頼性の高い金融環境が構築されます。

DeFiは世界規模で金融サービスへのアクセスを広げる機会ももたらします。従来の銀行インフラが未発達またはアクセス不可能な地域では、DeFiが実行可能な代替手段を提供します。スマートフォンとインターネット接続さえあれば、誰でもグローバル経済に参加できます。以前は手が届かなかった融資、貯蓄口座、投資機会へのアクセスを提供することで、何百万もの人々の生活を向上させる可能性があります。イーサリアムのスマートコントラクトのプログラマビリティにより、従来の金融システムでは実装が困難または不可能な複雑な金融商品やサービスを作成できます。

現在、DeFiの主要なユーザーは、おそらく発展途上国の十分な銀行サービスを受けられない人々や銀行口座を持たない人々ではなく、暗号資産の高い投機性を活用しようとする先進国の個人です。投機の余地は常にありますが、包括的な金融商品を世界中の誰もがアクセスできるようにすることが重要です。

DeFiと従来の金融の比較

DeFiは、従来の金融（Traditional Finance、以下TradFi）システムに対するサイファーパンクの回答であり、まだ進化中ではあるものの、すでに熱心なユーザーと革新的なビルダーの独自のニッチを見つけている分野です。DeFiとTradFiの区別は複雑ですが、最も重要な違いは交換媒体とブロックチェーンの固有の特性（オープン性や透明性など）にあります。DeFiは主に暗号資産を使用し、さまざまな程度で分散化されていることが多いのに対し、TradFiは法定通貨に依存しており、これは常に最大限に中央集権化されています。

TradFiシステムは多くの場合、参入障壁が高いです。銀行口座の開設、ローンの取得、金融市場への投資には、通常多くの書類と各種規制要件への準拠が必要です。このプロセスは、特に銀

行サービスが行き届いていない地域の人々を中心に、世界の人口の大部分を排除する可能性があります。図13-1にその対比を示します。

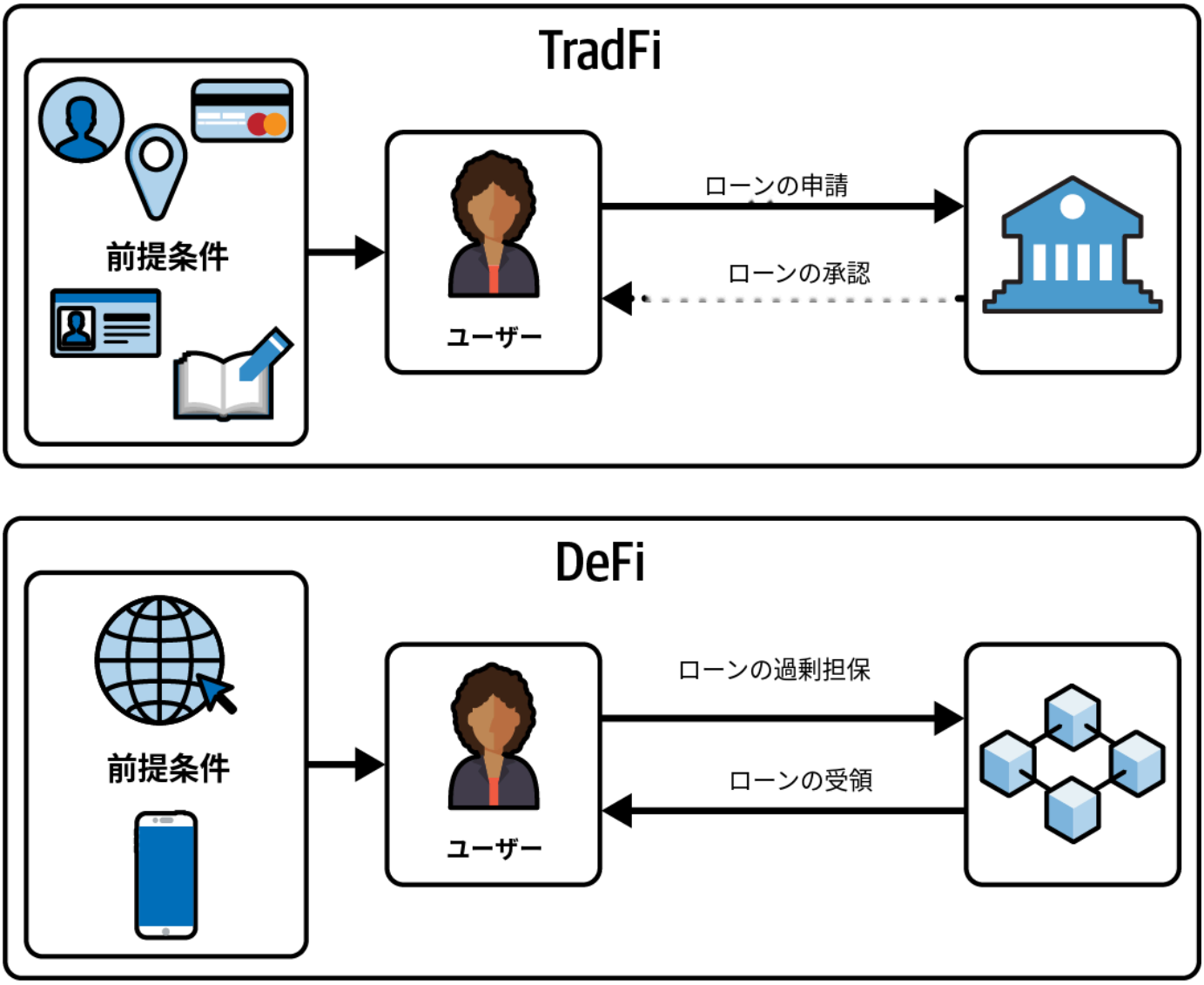


図13-1 従来の金融とDeFiのローン要件

一般的なユーザーがローンを申し込むには、身分証明書、証明可能な住所、社会保障番号、銀行口座、署名済み書類、良好なクレジットスコアなど多くの前提条件が必要です。これらすべてを揃えても、リクエストが拒否される可能性があり、承認されたとしてもおそらく長い時間がかかります。

DeFiでは、必要な前提条件は3つだけです。スマートフォン、インターネット接続、そしてローンを過剰担保するのに十分な資産です。これらが揃えば、ローンは即時、分散型、かつパーミッションレス（許可不要）です。

DeFiは設計上、誰でもアクセスしやすく作られています。インターネット接続さえあれば誰でもDeFiプロトコルを利用でき、従来のシステムから排除されている何十億もの人々に金融サービスを開放します。DeFiの高いアクセス性は、TradFiにおける信用力評価や不正への対処の仕組みを適用することを困難または不可能にしており、この点を欠点と見る人もいれば改善と見る人もいます。

DeFiのもうひとつの魅力的な側面は、従来の金融システムでは不可能な新しい金融商品を生み出せることです。たとえばフラッシュローンは、借り入れと返済が同一トランザクション内で完結する仕組みで、担保なしで大量の資金を瞬時に借りられます。DeFi固有のこの機能は、裁定取引、担保スワップ、その他の複雑な金融操作の可能性を広げ、TradFiでは再現できません。

DeFiの基本要素

ビットコインのような暗号資産が貨幣の概念を改善し分散化することを目指す一方で、DeFiプロジェクトはこの基盤の上に構築し、金融サービスを分散化・改善します。DeFiが提供する金融サービスを十分に理解するには、いくつかの重要な概念を知っておく必要があります。

DeFiにおけるトークンの受容性

イーサリアムのDeFiエコシステムでは、各トークンは個別のコントラクトとして動作します。名前や機能が似ていても、実際にはまったく異なる形態の通貨であるトークンが存在するため、初心者には混乱を招くことがあります。

たとえばイーサリアム上のArbitrumロールアップでは、USDC.eとUSDCという2つのトークンがほぼ同一に見えますが、リスクと受容性の点で大きく異なります。どちらも1ドルにペグされた価値を維持することを目指しています。しかしUSDCはArbitrumチェーン上でネイティブに発行されているのに対し、USDC.eはUSDCのブリッジ版（別のブロックチェーンからArbitrumに橋渡しされたもの）であり、別のチェーンからArbitrumに転送されたトークンを表しています。

これら2つのトークンのリスクプロファイルは大きく異なります。USDC.eはUSDC固有のすべてのリスクを抱えつつ、スマートコントラクトの脆弱性の可能性など、ブリッジプロセスに関連する追加リスクも負っています。各種DeFi金融サービスでコインやトークンがどの程度広く受け入れられているかを示す受容性も両者で異なります。USDC.eのみをサポートするプロトコル、USDCのみをサポートするプロトコル、両方を受け入れるプロトコルがあります。

分散型取引所

分散型取引所（DEX）は、中央機関や仲介者なしに他のユーザーと直接暗号資産を取引できるプラットフォームです。企業に取引の仲介を依頼する代わりに、DEXはスマートコントラクトでトランザクションを自動的に管理します。これによりユーザーは自分の資金の管理権を維持できます。

DEXの進化

中央集権的な取引所で取引したことがあれば、オーダーブック〔訳注：株式市場の注文板と同様の仕組み〕モデルに馴染みがあるでしょう。このモデルでは、買い注文と売り注文がユーザーの希望価格とともにリストされます。買い注文と売り注文が一致すると取引が執行されます。オーダーブックはすべての未決注文を表示し、トレーダーは市場の深さと流動性を確認できます。

オンチェーンでは、ブロックチェーンが従来のウェブサイトよりもはるかに遅く使用コストも高いため、このモデルは本格的に普及しませんでした。オーダーブックでの取引のレイテンシと、すべての注文に対する手数料の支払いにより、ユーザー体験は劣悪でした。2017年から2018年にかけてEtherDeltaがオンチェーンオーダーブックの実装を試み、その後いくつかの純粋なオーダーブックモデルも試されましたが、広く普及することはありませんでした。

Bancorは*自動マーケットメーカー*（AMM）モデルを最初に開拓しました。オーダーブックモデルとは異なり、AMMは買い手と売り手による注文の発注に依存しません。代わりにユーザーがトークンのペアを提供する流動性プールを使用します[1]。価格はプール内のトークンの比率に基づく数式で決定されます。このモデルにより継続的な流動性と取引が可能になります。

Uniswapは、シンプルかつ効果的な$x \times y = k$の数式でAMMモデルを大幅に改善し普及させました。この数式では2つのトークンの数量を掛け合わせた値が一定に保たれます。xとyはプール内の2つのトークンの数量を表し、kは定数です。取引が行われるとトークンの数量は変化しますが、2つの数量の積は同じままであり、プールが常に流動性を提供することを保証します。このイノベーションにより分散型取引所での取引がよりアクセスしやすく効率的になりました。

[1] トークンは常にペアであるとは限らない。3つ以上のトークンを持つ流動性プールも存在するが、最も一般的でシンプルな構成はペアである。

DEXでは、流動性を提供し、その貢献に対する手数料を得ることで、誰でもマーケットメーカーになれます。十分な流動性がある限り、図13-2に示すように取引は迅速に、中央機関の許可なしに行われます。[2]

[2] 図13-2からは、BTCがDEXでネイティブに交換可能であるように見えるかもしれない。しかし、実際にはそうではない。DEX上のBTCは、Wrapped Bitcoin（WBTC）のようなデリバティブコントラクトで表されることが多い。WBTCはBTCの価格にペグされた資産であるが、BTCブロックチェーン上のネイティブBTCよりもはるかに多くのリスクを伴う。ネイティブ資産は通常、それぞれのネイティブチェーンでのみ利用可能である。この例では、プールで使用されるBTCはネイティブBTCではなく、ステーブルコインが法定通貨を表すのと同様のデリバティブである。

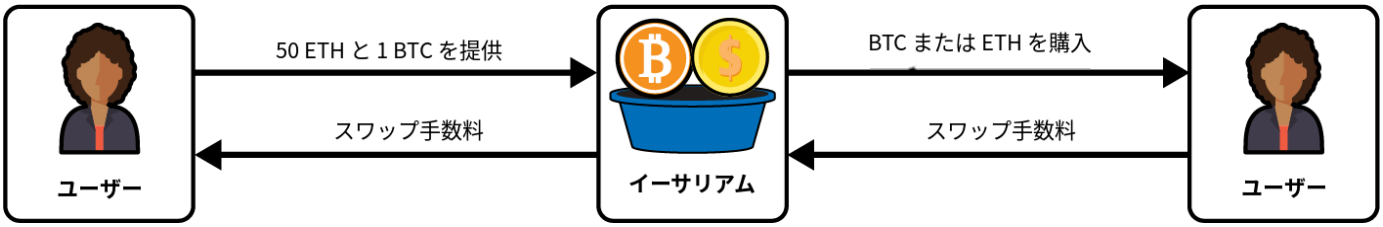


図13-2 分散型取引所の流動性プール

インパーマネントロス

DEXで流動性を提供することはリスクフリーではありません。スマートコントラクトのハッキングに加えて、*インパーマネントロス*と呼ばれる、より見落としやすいリスクがあります。インパーマネントロスを理解するには、まずUniswap V2プールの基本モデルを把握する必要があります。

Uniswap V2のような多くの人気DEXは、定積数式$x \times y = k$を使用します。ここでxとyは流動性プール内の2つのトークンの数量、kは定数です。トークンの価格は、プール内のトークンの比率によって決定されます。たとえば、プールに100 USDCと10 ETHが含まれている場合、1 ETHの価格は100 ÷ 10 = 10 USDCとなります。取引によりプールが200 USDCと10 ETHに変化した場合、1 ETHの価格は200 ÷ 10 = 20 USDCとなります。

この数式はプールの仕組みを理解するための簡略化された方法です。流動性を提供することでユーザーはマーケットメーカーとして行動します。ユーザーがプールからETHを購入すると、流動性提供者はETHを売却してUSDCを受け取ります。ユーザーがETHを売却すると、流動性提供者はETHを購入してUSDCを渡します。その見返りとして取引手数料を得ます。ただし、このプロセスにはインパーマネントロスのリスクがあります。インパーマネントロスは、プール内のトークンの価格が預け入れ時と比較して変化した場合に発生します。ETHの価格が大幅に上昇または下落した場合、手数料を徴収していても、プールの保有資産の価値は元のトークンを単純に保有していた場合よりも低くなる可能性があります。

一方で、このオープン性はDEXにとって重要な課題も生んでいます。誰でもブロックチェーンを作成してDEXを立ち上げられるため、現在さまざまなブロックチェーンにわたって100以上（おそらくそれ以上）のDEXが存在します。この豊富さは流動性を断片化し、暗号資産のスワップを流動性が統合されたプラットフォームよりも非効率にしています。この断片化は*スリッページ*の増大につながる可能性があります。スリッページとは、取引の期待価格と実際の執行価格の差です。流動性が不十分な場合に高いスリッページが発生し、取引が予想よりも不利な価格で執行されます。

定積数式が機能する理由

$x \times y = k$の数式のエレガンスは、その数学的形状である双曲線にあります。これは単なる偶然ではなく、流動性プールの完全な枯渇を防ぐための意図的な設計上の選択です。

線形価格設定の問題

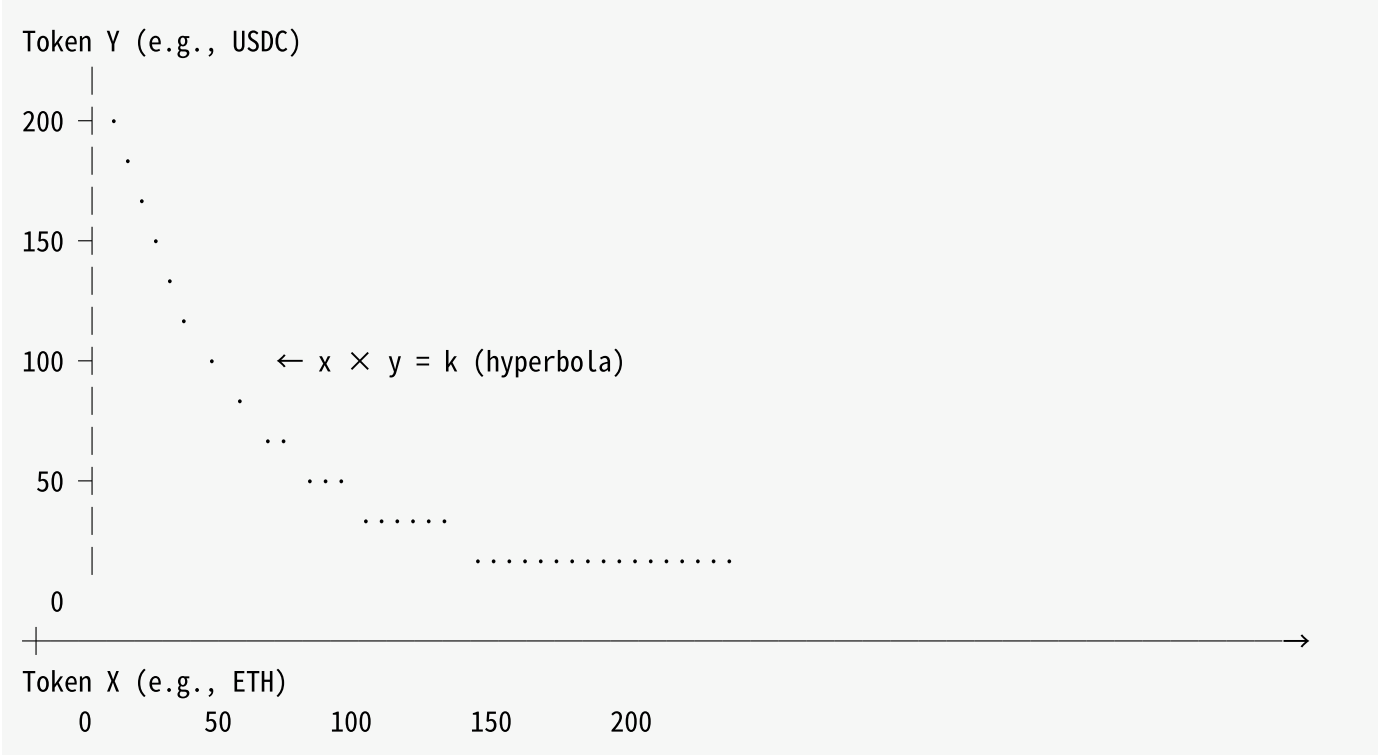
DEXが代わりにシンプルな線形価格設定の数式を使用し、残りの流動性に関係なく1 ETHのスワップで常に正確に2 USDCが得られる場合を想像してみてください。これは壊滅的な脆弱性を生みます。

1. 攻撃者はすべてのUSDCをETHにスワップし、プールからETHを完全に枯渇させることができる
2. プールは0 ETHとすべてのUSDCを保持することになる
3. 残りの流動性に基づいて価格が調整されないため、攻撃者は攻撃を逆方向にも実行できる
4. 一連の取引により、プール全体が両サイドでゼロまで枯渇する可能性がある

固定価格または線形曲線がAMMに根本的に適さない理由は、プールの完全な枯渇を許してしまう点にあります。

双曲線による解決

定積数式$x \times y = k$は重要な特性を持つ双曲線を生成します。この曲線はどちらの軸にも接しません。一方のトークンを大量に購入しようとすると、価格は指数関数的に上昇し、どちらのトークンもゼロまで枯渇させることは数学的に不可能です。



100 ETHと200 USDC（k = 20,000）のプールを考えてみましょう。

- **1 ETHをUSDCにスワップ**: 約1.98 USDC（1:2の比率に近い）を受け取る
- **50 ETHをUSDCにスワップ**: 66.67 USDCしか受け取れない（線形モデルが示す100ではない）
- **200 USDC全額を取得しようとした場合**: 無限のETHを提供する必要がある

出力を計算する数式は以下のとおりです。

```
Δy = (y × Δx) / (x + Δx)
```

ここでΔxは入力量、Δyは出力量です。Δxが無限大に近づくと、Δyはy（準備金の総額）に近づきますが、到達することはありません。

この漸近的な振る舞いがAMMを安全にしています。プールサイズに対する取引が大きくなるほど交換レートは悪化します。これによりスリッページ（期待価格と実際の執行価格の差）が生じ、プールを大幅に枯渇させる取引を自然に抑止します。

双曲線は流動性プールが自己保護的であることを保証します。トークンが希少になるほど価格が高くなり、完全な枯渇を防ぐ経済的障壁を生み出します。このエレガントな数学的特性こそが、Uniswapのシンプルな数式をトラストレスかつパーミッションレスな取引において革命的なものにした要因です。

注記

DeFiの現在の状況において最も重要なプロジェクトのひとつであるUniswapは、ヴィタリック・ブテリンの2016年のReddit投稿に触発されました。プログラミング経験がなかったとされるヘイデン・アダムスが、Vyperプログラミング言語を使用してUniswap V1の開発に1年を費やしました。

レンディングマーケット

レンディングマーケット（*マネーマーケット*とも呼ばれる）は、スマートコントラクトでプロセス全体を自動化・保護し、暗号資産の貸し借りを促進する分散型プラットフォームです。従来の金融システムとは異なり、銀行のような仲介者なしで運営され、ローンをより透明かつ効率的に処理できます。

レンディングマーケットでは、暗号資産で利子を得たいユーザーは資金をレンディングプールに預け入れることができます。これらの預金はプラットフォームの全体的な流動性に貢献します。貸し手はプール内の需要と供給に基づいてアルゴリズムで決定されることが多い利率で利子を得ます。借り入れの需要が高いほど利率も高くなり、より多くの貸し手がプールに資産を提供するインセンティブが生まれます。

一方、借り手は担保を提供することでこれらの資金にアクセスできます。担保は通常、借り入れたい金額よりも高い価値が必要です。この過剰担保はデフォルトのリスクを軽減するためにレンディングにおいて重要です。ほとんどの暗号資産は非常にボラティリティ（価格変動の激しさ）が高く、ローンが過剰担保されていなければ不良債権[3]をレンディングマーケットに残す可能性があるためです。担保はスマートコントラクトにロックされ、借り手がローンを返済できない場合に清算されて未返済額を補填します。

[3] 不良債権とは、借り手がローンの返済を怠り、残りの担保が未払い額を補填するのに不十分な場合に発生する。突然の市場変動や不適切な担保評価によって起こり得る。担保を売却して債務を補填する清算とは異なり、不良債権は回収不能のまま残り、レンディングプロトコルとそのユーザーに損失をもたらす。

このシステムは貸し手を保護しレンディングプールの支払い能力を維持します。図13-3にこの簡略版を示します。貸し手が流動性を提供して借り手が支払う年間利子を徴収し、借り手は提供された流動性を引き出します。

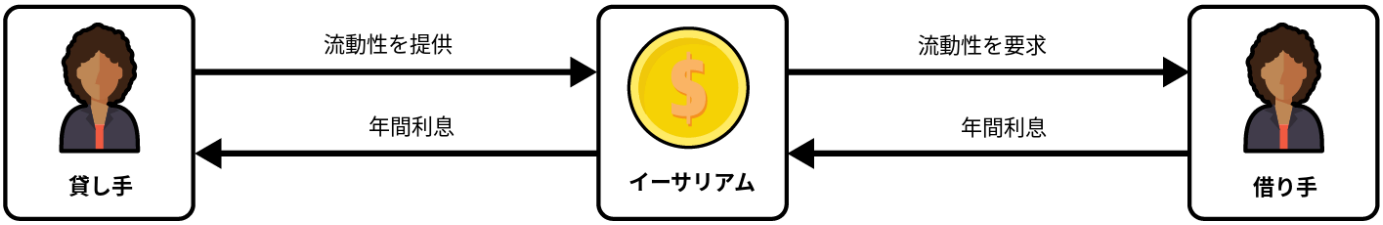


図13-3 レンディングマーケットの基本フロー

レンディングマーケットの利率は動的で、市場状況に応じて変動します。プラットフォームのアルゴリズムは利用可能な資金の供給とローンの需要のバランスを取るために利率を継続的に調整します。これにより貸し手と借り手の両方が公正な市場主導型の利率の恩恵を受けられる、効率的で反応性の高い金融エコシステムが構築されます。

担保率はレンディングマーケットのもうひとつの重要な側面です。担保率はローンを確保するために必要な担保の量を決定します。たとえば一般的な担保率は150%で、100ドル相当の暗号資産を借りるには図13-4に示すように少なくとも150ドル相当の担保を預け入れる必要があります。これにより価格変動による潜在的な損失を吸収するバッファが確保されます。

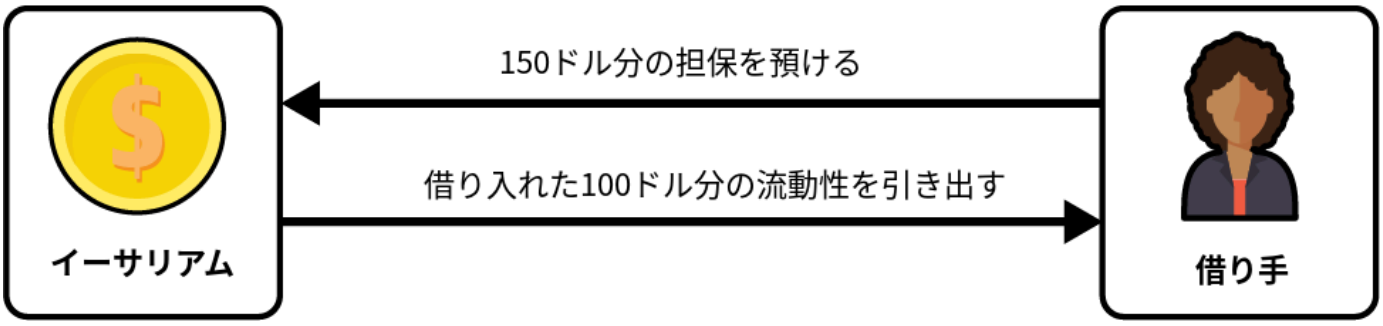


図13-4 レンディングマーケットの担保

担保の価値が一定の閾値を下回るとプラットフォームのスマートコントラクトが清算プロセスを開始します。担保を売却してローンを返済し、貸し手を潜在的な損失から保護します。清算メカニズムはレンディングプールの安定性と支払い能力を維持するために不可欠です。

DeFiにおけるインセンティブ

DeFiはオープンな設計であり、効果的に機能するには適切なインセンティブが不可欠です。たとえばほとんどのレンディングマーケットの清算プロセスは、清算可能なローンを継続的に監視するユーザーのインセンティブに依存しています。そのようなローンを特定すると、ユーザーはそれを清算します。ユーザーがこのタスクを実行する動機づけとして、清算額の一部が報酬として支払われます。

このインセンティブの概念はブロックチェーンとDeFiの基本的な部分です。これらのシステム内の多くのメカニズムは、参加者がネットワークの機能性とセキュリティを維持・改善する方向で行動するよう、ゲーム理論の原則に基づいて設計されています。インセンティブはユーザーの行動をプロトコルの全体的な目標に合わせ、自己持続的なエコシステムを構築します。

DeFiの多くのプロトコルに共通する強みは、互いに組み合わせて使える「コンポーザビリティ」（composability）にあります。レンディングマーケット単体では、過剰担保が必要なため特に目新しく感じないかもしれません。しかし、ローン機能を他のDeFiプロトコルと組み合わせることで、多様な戦略を実現できる強力なツールになります。

たとえばレンディングマーケットとDEXを組み合わせることで、*ショート*と呼ばれる金融商品を再現できます。ショートは資産の価格が下落すると予想する場合に使用する戦略です。基本的には資産を借りて現在の価格で売却し、後でより低い価格で買い戻して、借りた資産を返却し差額を得ることを期待します。

DeFiでショートを実現する方法は以下のとおりです。

1. **資産Aを担保化する**: レンディングマーケットに資産Aを担保として預け入れる。
2. **資産Bのローンを取得する**: ショートしたい資産Bを借りる。
3. **DEXで資産Bを売却する**: 借りた資産Bを分散型取引所で売却する。

これにより1倍のレバレッジで効果的に資産Bをショートできます。[4]資産Bの価格が下落した場合、より低い価格で買い戻してローンを返済し差額を保持できます。図13-5にこれを示します。DeFiプロトコルのコンポーザビリティにより、従来の金融戦略を分散型環境で再現できます。

[4] ショートポジションのサイズは借り入れ額に直接関連する。

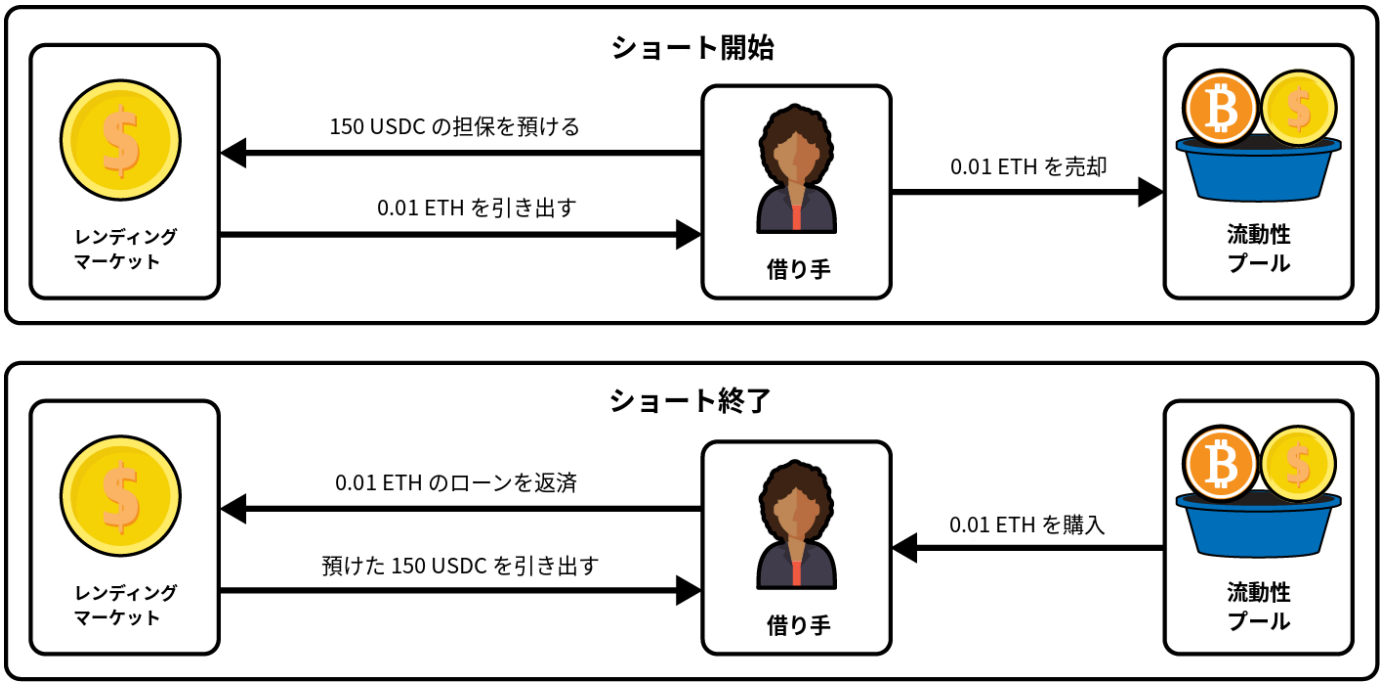


図13-5 DeFiを使用したショート戦略

レンディングマーケットの用途は広範で、すぐには明らかでないかもしれません。お金があるのに、なぜ過剰担保のローンを組む必要があるのかと疑問に思うかもしれません。自分の資金を使えばよいのではないか、と。一部のシナリオではそのとおりです。しかしレンディングマーケットとDEXを組み合わせることで金融商品を再現でき、ショートの代わりに資産の「ロング」も可能になります。レンディングマーケットのもうひとつの利点は課税対象イベントの回避です。多くの国や地域では資産の借り入れは課税対象となる取引とみなされませんが、資産の売却は課税対象です。

レンディングマーケットの潜在的な用途は広大で本書の範囲を超えています。しかしレンディングマーケットは資本効率を向上させ、他のプロトコルと組み合わせることでユーザーに大きな柔軟性を提供します。過剰担保を必要としないレンディングマーケットも存在しますが、現在のところ広く普及していません。

オラクル

イーサリアムのオラクルは第11章で詳しく説明したとおり、現実世界のデータをブロックチェーンに取り込み、スマートコントラクトが外部情報を利用できるようにするサービスです。たとえば暗号資産の価格データ、気象条件、スポーツのスコアを提供し、スマートコントラクトがこの外部データに基づいて実行できるようにします。

オラクルはレンディングマーケットのセキュリティを含むDeFiの多くの側面に不可欠です。レンディングマーケットではトークンの価格をDEXから取得できます。ただし、DEXのみに依存すると、レンディングマーケットがフラッシュローン攻撃に対して脆弱になる可能性があります。このような攻撃では攻撃者が大量の資金を借り入れてDEX上のコインやトークンの価格を操作し、この価格変動を利益のために悪用します。

訳注

フラッシュローン攻撃の典型的な手順は以下のとおりである。①フラッシュローンで大量の資金を無担保で借りる → ②その資金でDEX上の特定トークンを大量に売買し、価格を一時的に操作する → ③操作された価格を利用して別のDeFiプロトコル（レンディング等）から不当な利益を得る → ④利益を確保した上でフラッシュローンを返済する。これらがすべて同一のトランザクション内で実行される。

オラクルのもうひとつの重要な機能は擬似乱数をオンチェーンに取り込む能力です。イーサリアムは決定論的であるため、すべての計算は毎回同じ結果を生成する必要があり、ノードがすべてのブロックとトランザクションを一貫して検証できるようにしています。しかしこの決定論的な性質はイーサリアムのコンポーネント内に真の乱数が存在できないことを意味します。

たとえばカジノプラットフォームを構築する場合、乱数の生成は不可欠です。ブロックタイムスタンプ、ハッシュ、トランザクションカウントから擬似乱数を作成する方法もありますが、ブロックプロポーザーがこれらのプロパティを事前に予測または操作してカジノを悪用できるため、このアプローチは攻撃に対して脆弱です。

オラクルによりイーサリアムブロックチェーンは外部データを使用でき、さまざまなアプリケーションに対してより多くの可能性が開かれます。

> **ヒント**
>
> この説明からはオラクルが中央集権的な存在であり、分散型であるべきDeFiがオラクルに依存しているように見えるかもしれない。しかしDeFiの一部の側面はオラクルに依存しているものの、オラクル自体は多くの場合高度に分散化されており、単一障害点を持たない。

ステーブルコイン

*ステーブルコイン*は、通常米ドル、ユーロ、または商品のバスケットなどの準備資産にペグされた安定した価値を維持するよう設計された暗号資産の一種です。セキュリティや分散化などの暗号資産の利点と従来の法定通貨の安定性を組み合わせることを目指しています。

ステーブルコインはさまざまなメカニズムで安定性を実現します。*法定通貨担保型ステーブルコイン*は、銀行やその他の信頼できるカストディアンが保有する法定通貨の準備金で裏付けられています。たとえば各USDCまたはUSDTトークンは通常、同等額の米ドル（USD）で裏付けられています。

*暗号資産担保型ステーブルコイン*は他の暗号資産で裏付けるという異なるアプローチを取ります。暗号資産のボラティリティの高さを考慮して、ペグを維持できるよう多くの場合過剰担保されています。よく知られた例はMakerDAOのDAI[5]で、ユーザーはイーサリアムやその他の暗号資産を担保としてロックアップしてDAIを発行します。過剰担保は基礎となる資産のボラティリティに対するバッファを提供します。

[5] MakerDAOは現在Skyに、DAIは現在USDSに名称変更されている。

さらに*アルゴリズム型ステーブルコイン*があります。担保に依存せず、アルゴリズムとスマートコントラクトでステーブルコインの供給を管理し価値を安定に保ちます。市場の需要に基づいて供給を拡大または縮小し目標価格を維持します。TerraUSD（UST）はアルゴリズム型ステーブルコインの著名な例でしたが、重大な問題に直面し崩壊しました〔訳注：2022年5月にUSTのペグ崩壊が発生し、関連するLUNAとあわせて400億ドル以上の損失を引き起こしたとされる〕。

成功し十分な資本を持つアルゴリズム型ステーブルコインはまだ存在しません。これまでの試みはすべて壊滅的に失敗するか、プロダクトマーケットフィットを見つけられませんでした。

ステーブルコインはDEXの流動性にも大きく貢献しています。安定した予測可能な資産を提供することでよりスムーズな取引と優れた市場効率を促進します。ステーブルコインにより、トレーダーは価格変動を心配することなくポジションの出入りが容易になり、効率的な市場運営が促進されます。

法定通貨担保型ステーブルコインの限界

興味深い概念として、CoinbaseやBitfinexでそれぞれドルと交換できるUSDCやUSDTのようなステーブルコインは時価総額の成長に限界があります。*時価総額*とは流通しているすべてのコインの総価値であり、コインの現在の価格に総供給量を掛けて計算されます。

この限界の理由は、特定のブロックチェーン上のステーブルコインの総価値がそのブロックチェーンの攻撃コストを超えると、悪意のある行為者が攻撃を試みる重大なインセンティブが生まれるためです。ブロックチェーンの侵害による潜在的な報酬がコストを上回る可能性があり、現実的かつ定量化可能なリスクとなります。

幸いなことにイーサリアムのような堅牢で安全なブロックチェーンの攻撃は容易でも安価でもありません。攻撃の高いコストと複雑さが強力な抑止力となります。しかしあらゆるブロックチェーン上のステーブルコインのスケーラビリティとセキュリティを考慮する際には、この概念を常に念頭に置くことが重要です。

リキッドステーキング

*リキッドステーキング*は、ユーザーがproof-of-stake（PoS）ネットワークに暗号資産をステーキングしつつ、それらの資産の流動性を維持できるメカニズムです。通常トークンがステーキングされるとステーキング期間が終了するまでロックされ、アクセスや取引ができません。リキッドステーキングはステーキングされた資産を表すデリバティブトークンを発行することでこの問題を解決します。このデリバティブトークンは取引、転送、または他のDeFiプロトコルで使用でき、ユーザーはステーキング報酬を得ながら流動性を維持できます。

たとえばリキッドステーキングプラットフォームでETHをステーキングすると、stETHのようなデリバティブトークンを受け取れます。ETHはステーキングされたまま報酬を得続けますが、stETHトークンは自由に取引したり他のDeFi活動で使用したりでき、資金へのアクセスを失わずにステーキングの恩恵を得られます。

注記

リキッドステーキングに類似した新しいタイプのデリバティブが登場しつつあります。*リキッドリステーキング*として知られるものです。EigenLayerのようなプロトコルでは、トークンはイーサリアムチェーンのためにステーキングされるだけでなく、リステーキング

と呼ばれるプロセスで他のサービスの保護にも使用されます。この新しい展開は、イーサリアムのコンセンサスメカニズムに過負荷をかけるのではないかという懸念を生んでいます。

Lidoが発行するstETHのように、ステーキング時に受け取るトークン（リキッドステーキングトークン）は、ETHの価格に連動するステーブルコインのように機能します。そのため、ドルにペグされたステーブルコインに見られるような典型的なペグ関連リスクだけでなく、スラッシングに関連する追加リスクも伴います。リキッドステーキングプロトコルはユーザーが預け入れたETHでバリデータノードを作成し、ステーキング報酬を得てデリバティブトークンの保有者に再分配します。このプロセスにはバリデータノードが正しく運用されなかった場合にステーキングされたETHの一部が失われるスラッシングのリスクがあります。

これらのリスクに加えてリキッドステーキングはブロックチェーン自体にシステミックリスクを生み出す可能性があります。たとえば執筆時点でLidoはステーキングされた全ETHの29%を保有しています。この割合が33%以上に増加した場合、重大な問題を引き起こす可能性があります。[6]2022年6月、Lido DAOにおいてLidoのステーキングパワーを制限し33%のラインを超えることを防いでイーサリアムの潜在的なシステミックリスクを回避するための投票が行われました。残念ながら、この投票は可決されませんでした。

[6] いかなるステーカーもステーキングされたETHの33%以上を保有している場合、理論上、チェーンを攻撃してファイナリティプロセスを停止させることができる。

注記

反対票の大部分がわずか数個のウォレットから来たものであったため、投票をめぐって論争もあった。

RWA（現実世界の資産）

DeFiの文脈において*現実世界の資産*（RWA）とは、トークン化されてブロックチェーン上に取り込まれた有形資産や従来の金融資産を指します。不動産やコモディティから株式、債券、ファインアートまであらゆるものが含まれる可能性があります。これらの資産のトークン化とは、その価値をブロックチェーンプラットフォーム上で取引、貸し付け、借り入れが可能なデジタルトークンに変換することです。

RWAはやや議論の余地があります。多くの場合、カストディアンを必要とし、ブロックチェーンのトラストレスの原則と矛盾するためです。このセクターはDeFiにおいて本格的に台頭した最後のセクターのひとつであり、これまでのところRWAプロトコルで重大な問題は発生していません。暗号資産の基本理念に反するにもかかわらずRWAは多くの新しい可能性を開きます。

RWAの重要な応用のひとつは債券をオンチェーンに持ち込み、ユーザーが相対的にリスクフリーの利率にアクセスできるようにしたことです。これにより、米国外のユーザーでも、2023〜2024年に米国で提供されていた約5%の金利を得られるようになりました。RWAのもうひとつの価値ある用途は不動産の一部を所有するなど、トークン化された資産の分割所有です。

このセクターはまだ初期段階にあり、その潜在能力をすべて明らかにしたわけではありません。しかし、資産の管理・保管（カストディ）に関するリスクは無視できません。

ブリッジとオムニチェーンプロトコル

ブロックチェーンエコシステムが進化するにつれて、固有の特徴と利点を持つ多種多様なネットワークが発展してきました。しかしこれらのネットワークはしばしば孤立して運用されており、ネットワーク間の資産やデータのシームレスな転送が制限されています。ブリッジとオムニチェーンプロトコルはチェーン間の連携を促進することでこの問題を解決し、より相互接続されたブロックチェーンエコシステムの構築を目指しています。

> **注記**
>
> ブリッジは、ほとんどのブロックチェーンが他のチェーンの状態を認識しないため、非常に中央集権的であることが多い。チェーンAからチェーンBに資金を転送する場合、ブリッジ操作を承認してチェーンB上の流動性をアンロックする中央機関が通常存在する。この中央集権化がブリッジがDeFiにおいて最も頻繁にハッキングされるプロトコルのひとつである理由である。

*ブリッジ*は図13-6に示すように、異なるブロックチェーン間での資産やデータの転送を促進する特化型プロトコルです。コネクターとして機能し、トークンやその他のデジタル資産をあるチェーンから別のチェーンに移動できるようにします。たとえばトークンをイーサリアムからBinance Smart Chain（BSC）に転送したい場合にブリッジを使用します。

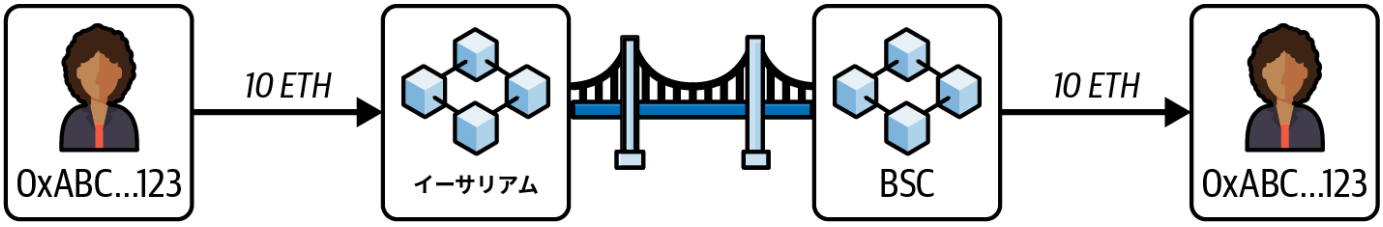


図13-6 2つのブロックチェーンを接続するブリッジ

あるチェーンから別のチェーンにトークンをブリッジするモデルは数多くあります。すべてのブリッジは特定のモデルを使用します。最も一般的なものは以下のとおりです。

ラップドトークンブリッジ（ロックとミント）

このモデルではブリッジがチェーンA上でトークンを受け取りスマートコントラクトにロックし、チェーンB上でラップド（または「プロプライエタリ」な）トークンをミントします。ラップドトークンはロックされたトークンを表す受領証として機能します。元のトークンを取得するにはチェーンB上でラップドトークンをバーンし、チェーンA上のトークンをアンロックします。

たとえば「Mastering Bridge」を使用してETHをイーサリアムからBSCに転送するとします。イーサリアム上のブリッジのスマートコントラクトにETHを送るとブリッジはBSC上で「MasteringETH」のようなラップドトークンをミントします。イーサリアムに戻るにはBSC上のMasteringETHをバーンし、ブリッジが元のETHをリリースします。これらのラップドトークンはチェーンB上でネイティブトークンを取得するためにDEXでスワップする必要があることが多いです。

ミントとバーン

このモデルはトークンのミントおよびバーン機能を管理するプロジェクトで一般的に使用されます。トークンをロックする代わりにブリッジはチェーンA上でトークンをバーン（供給量を減少）し、チェーンB上で同等量をミントします。プロジェクトがトークンのスマートコントラクトに対する権限を持っている必要があります。たとえばプロジェクトはイーサリアム上のETHベースのトークンをバーンし、BSC上で同じトークンをミントしてチェーン間で総供給量を維持できます。

流動性ブリッジ

最も一般的なタイプである流動性ブリッジは複数のチェーン上のトークンプールに依存します。トークンをブリッジすると、ブリッジはその流動性を使用して通常は手数料と引き換えに宛先チェーン上で同等のトークンを送ります。ラップドトークンブリッジとは異なり新しいトークンはミントされません。ブリッジは両方のチェーン上にすでにトークンを保有しています。

先ほどの例に従いイーサリアムからBSCにETHをブリッジする場合、イーサリアム上のブリッジにETHを送ります。ブリッジはBSC上の流動性プールからETHをリリースします。ブリッジがミントを管理しない場合、転送を促進するためにBSC上に十分な流動性を維持する必要があります。このモデルはシンプルさから人気がありますが、ブリッジが安全に流動性を管理できるかどうかに依存します。

第11章でクロスチェーンメッセージングプロトコルとして議論した*オムニチェーンプロトコル*は、複数のブロックチェーンにわたるシームレスな通信と相互運用性を同時に可能にすることでクロスチェーン連携の概念を拡張します。これらのプロトコルは、異なるブロックチェーンが摩擦なく連携できる統一レイヤーを構築します。これによりチェーン間での資産、データ、さらにはスマートコントラクト機能の転送を可能にすることを目指しています。図13-7はユーザーがイーサリアム上でスワップを開始しBSC上でスワップを完了して資金を受け取る、シンプルなオムニチェーンメッセージングプロトコルを示しています。

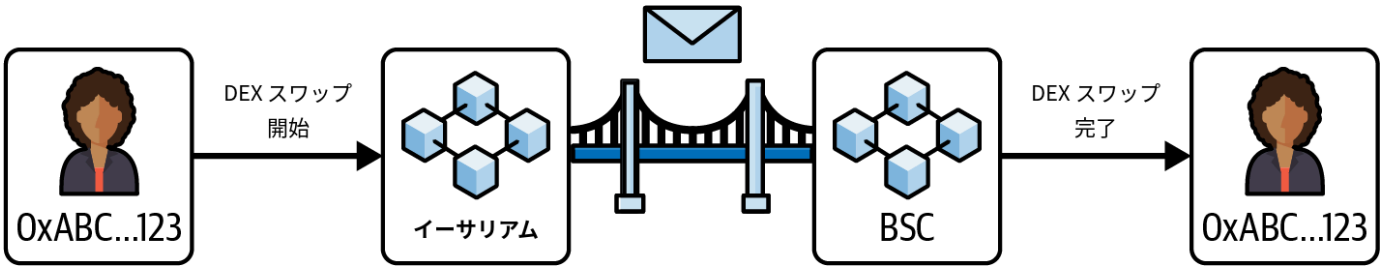


図13-7 オムニチェーンプロトコルの例

中央集権型・非中央集権型の金融（CeFi・DeFi）

分散化を定義することは常に困難であり、多くの場合現実を記述するというよりも望ましい結果を表現する手段です。ほとんどのDeFiプロトコルは真に分散化されていません。重要な権限を持つアドレスやコア開発チームによる意思決定に依存していることが多く、分散型であるはずのシステム内に権力の集中を生み出しています。

DeFiはまだ初期段階にあり、その未来を形作り、正しい方向に導くのはアーリーアダプターの役割です。分散化は目標ですが、それが常に現在の現実であるとは限りません。DeFiのような新興市場は、迅速な意思決定や変更の実施のために、ある程度の中央集権化から恩恵を受けることもあります。こうした中央集権的な意思決定は、急速に進化する環境で適応・成長するために必要な俊敏性をもたらします。

DeFiが成熟するにつれて、中央集権的な要素を段階的に減らし、より分散型のモデルへと移行することが目標であるべきです。アーリーアダプターは、現時点での効率性・迅速な意思決定へのニーズと、分散化という長期的なビジョンのバランスを取るうえで、非常に重要な役割を担っています。固有のトレードオフを理解し対処することで、DeFiコミュニティは、オープン性、透明性、包括性という基本原則に沿ったシステムの進化を導けます。

DeFiにおけるリスクと課題

DeFiには専門家を含むユーザーにもしばしば誤解される独自のリスクと課題があります。すべてのDeFiプロトコルは特定の経済的リスクと一般的なスマートコントラクトリスクを伴います。さらに分散化の程度に応じてカストディアンリスクや中央集権化の問題が存在する場合があります。

スマートコントラクトリスクは、プロトコルを問わず共通して当てはまる汎用的なリスクです。スマートコントラクトはハッキングされる可能性があり、カストディアンは悪意を持って行動したりミスを犯したりする可能性があります。一方、経済的リスクはより複雑で、各DeFiの基本要素やプロトコルに固有のものです。たとえばレンディングマーケットにおけるリスクは清算であり、適切なオラクルを使用しないマーケットでは不当に発生する場合があります。DEXで流

動性を提供する際の主なリスクのひとつはインパーマネントロスです。ステーブルコインやリキッドステーキングの場合、主なリスクはペグの喪失です。

DeFiプロトコルを使用する前に関連するすべてのリスクを理解することが極めて重要です。リスクに加えてDeFiは重大な課題にも直面しています。多くのプロトコルは既存プロトコルのフォークであり、真のイノベーションを提供せずに既存プロジェクトに対して「ヴァンパイア攻撃」（高い報酬で競合プロトコルのユーザーや流動性を奪う戦術）を試みています。これは流動性とユーザーを断片化するだけでなくエコシステム全体の有効性を希釈します。

流動性はほとんどのプロトコルにとって不可欠です。好況市場ではうまく機能するかもしれませんが、低迷期には大幅に劣化することが多いです。ほとんどのDeFiプロトコルは自律的に持続できず、トークン報酬などのインセンティブがある間だけ利用される傾向があります。これは、ユーザーがプロダクト自体に価値を感じていないか、インセンティブがなければコストが利益を上回ってしまうためです。

DeFiはある程度のプロダクトマーケットフィット（製品と市場ニーズの一致度）を示していますが、将来は不確実です。グローバルインフラの魅力的な一部へと進化する可能性があり、これが最も可能性の高い帰結ですが、一部のユーザーのためのニッチ市場にとどまる可能性もあります。

DeFiにとっての重要な課題のひとつは規制です。規制環境は国や規制機関によって大きく異なり、ほとんどの規制当局は完全に分散化されたシステムに反対しています。スマートコントラクトが不変でオンチェーンにデプロイされている場合、規制当局は直接介入できませんが、開発者やユーザーをターゲットにしてスマートコントラクトの使用を困難にすることは可能です。

その好例がTornado Cashです。開発者のアレクセイ・ペルツェフはオランダで64か月の禁固刑を宣告されました。北朝鮮のハッキンググループLazarusによる不正資金の洗浄にTornado Cashが使用されたとされることを受けて米国財務省がTornado Cashをブロックリストに追加し、ペルツェフは2022年8月にマネーロンダリングの容疑で逮捕されました。Tornado Cashプロトコルは引き続き利用可能ですが流動性は大幅に減少しユーザー体験の悪化につながっています。さらにTornado Cashを使用したアドレスは中央集権的な取引所でフラグが立てられ、使用がさらに困難になっています。

同様の状況が他のエコシステムでも発生しています。たとえばSamourai Walletの開発者も法的措置に直面しています。このパターンは規制当局がブロックチェーン自体を攻撃できないものの、そのユーザーや開発者をターゲットにできることを示しています。

注記

本書の著者はSamourai WalletやTornado Cash、その他の類似事例に関する法的専門知識を十分に持ち合わせていないが、分散型コードを書いたことを理由に個人に対して取られる法的措置を支持しない。コードは自由であるべきであり、ツールの制作者が他者による悪用のために罰せられるべきではない。今後このような不当な結果に直面する開発者が出ないことを願う。

まとめ

DeFiはユーザーが自分のお金をより柔軟に活用できるようにし、フラッシュローンのような新しい機会と革新的な金融の基本要素を生み出しています。暗号資産分野における新興市場としてDeFiは急速に発展していますが、トークン交換、ステーブルコイン、デリバティブの作成を超えた適切なマーケットフィットをまだ見つけていません。

第14章 イーサリアム仮想マシン(EVM)

イーサリアムプロトコルとその運用の中心には、*イーサリアム仮想マシン*(Ethereum Virtual Machine)、つまり*EVM*があります。名前から推測できるように、それは計算処理エンジンであり、Microsoftの.NET Frameworkの仮想マシンやJavaなどの他のバイトコードでコンパイルされたプログラミング言語のインタプリタと大きく異なるものではありません。この章ではEVMの詳細を見ていきます。特に、イーサリアムのステート（状態）更新に関わる命令セット、構造、操作などを見ていきます。

EVMとは何か？

EVMはイーサリアムの構成要素の１つで、スマートコントラクトをデプロイあるいは実行します。あるEOA（外部所有アカウント）から別のEOAへの単純な送金トランザクションは、実際のところEVMを関与させる必要はありませんが、それ以外のすべてのトランザクションに関してはEVMによって計算がなされた状態への更新を伴います。おおまかに言うと、イーサリアムブロックチェーン上で実行されるEVMは、それぞれ永続的なデータストアを伴い、多くの実行可能オブジェクトを格納した、グローバル分散コンピュータとみなせます。

EVMは*準チューリング完全*状態マシンです。どのようなスマートコントラクトに対しても、すべての実行過程は、実行のために利用可能なガスの量によって有限の計算ステップ数に制限されるため、「準」と呼んでいます。このようにして、*停止性問題*は「解決」し（すべてのプログラムは、実行した場合にいつかは停止する）、実行が（意図せず、あるいは悪意をもって）永遠に実行され、イーサリアムプラットフォームが完全に停止する状況が回避されます。停止性問題については、後のセクションで詳しく説明します。

EVMは*スタックベースアーキテクチャ*を持っており、一時的な値はすべてスタックに格納されます。EVMは主に組み込みのハッシングや楕円曲線演算を容易にするために、256ビットのワードサイズで動作し、以下に示す、アドレスを指定してアクセスが可能ないくつかのデータコンポーネントも持っています。

- 実行されるスマートコントラクトのバイトコードが読み込まれた、不変のプログラムコードROM
- すべての領域が明示的にゼロに初期化された揮発性のメモリ
- 単一のトランザクションの処理中にのみ保持する（イーサリアムのステートの一部ではない）一時ストレージ
- メモリと同様にゼロに初期化されたイーサリアムの状態の一部である永続ストレージ

また、実行中に利用可能な環境変数とデータのセットも存在します。それらの詳細に関しては、章の後半で詳しく説明します。

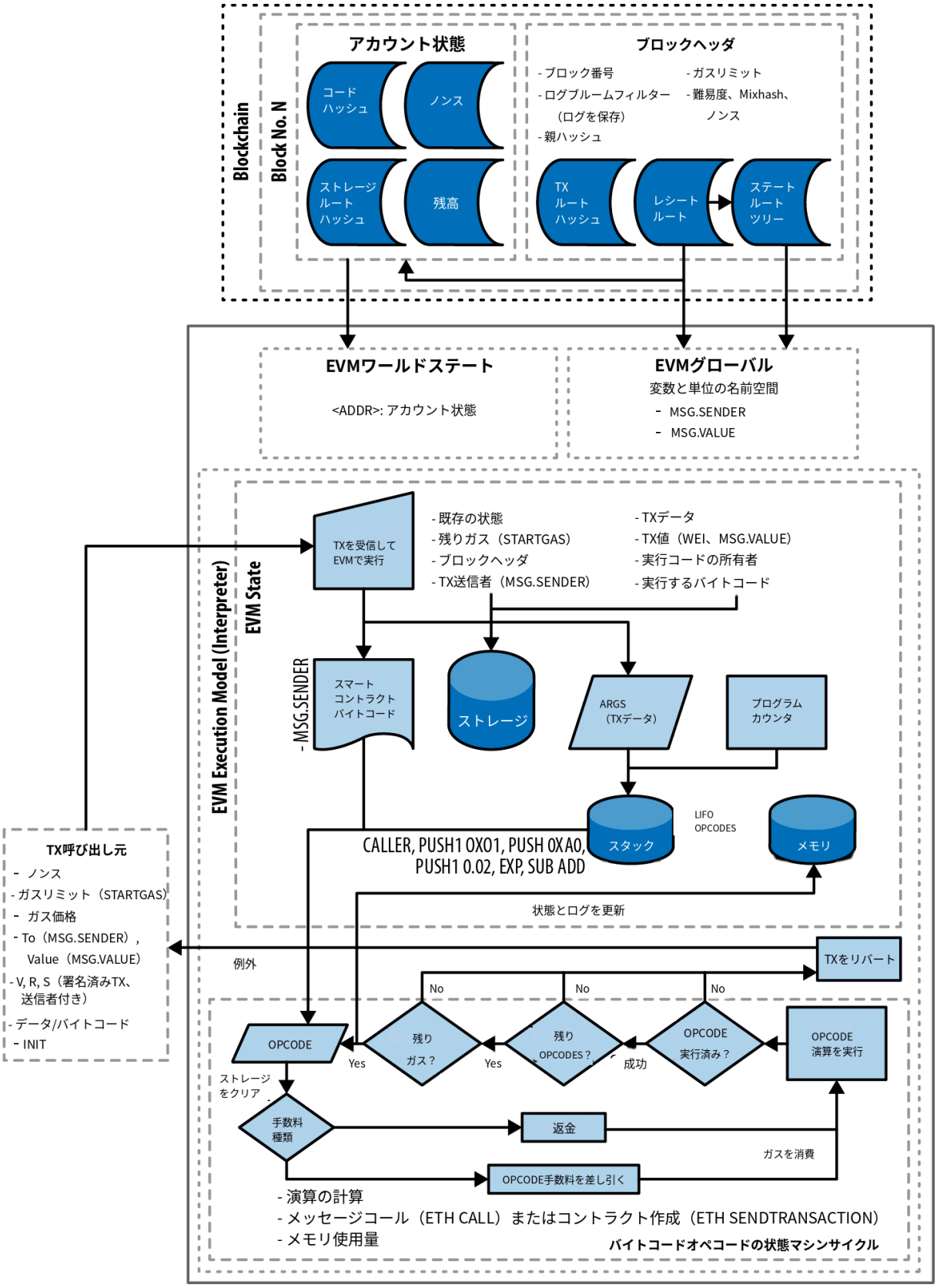


図14-1. EVMアーキテクチャと実行コンテクスト

既存の技術との比較

*仮想*マシンという用語は、通常、VirtualBoxやQEMUなどのハイパーバイザーや、LinuxのKVMなどのオペレーティングシステムのインスタンス全体の仮想化など、現実のコンピュータの仮想化に用いられます。実際のハードウェア、システムコール、そしてその他のカーネル機能それぞれに対して、ソフトウェアとしての抽象化を定める必要があります。

現実のコンピュータの仮想化に比べると、EVMははるかに限られた領域で動作します。EVMは単なる計算処理エンジンなので、たとえばJVM（Java Virtual Machine：Java仮想マシン）の仕様と同じように、計算処理とストレージの抽象化を規定します。全体的な観点から見ると、JVMは、基盤となるホストOSやハードウェアに依存しない実行環境を規定するように設計されており、異なるシステム間で互換性を保つことができます。JavaやScala（JVMを使用する）やC#（.NETを使用する）などの高級プログラミング言語は、それぞれの仮想マシンのバイトコード命令セットにコンパイルされます。同様に、EVMは独自のバイトコードで記述された命令セット（次の項で説明します）を実行します。バイトコードで記述された命令セットは、Solidity、Vyper、Yulなどの高級スマートコントラクトプログラミング言語をコンパイルすることで生成されます。

EVMはスケジューリング機能を持ちません。なぜなら、実行順序の決定はEVMの外部で取りまとめられているからです。イーサリアムクライアントは、検証されたブロックトランザクションを一通り見て、どのスマートコントラクトをどの順序で実行する必要があるかを決定します。この意味で、イーサリアムワールドコンピュータはJavaScriptのようにシングルスレッドです。また、EVMには、「システムインターフェイス」の処理や「ハードウェアのサポート」はどちらもありません。お互いにやり取りする物理的なマシンは存在しないのです。イーサリアムワールドコンピュータは完全に仮想的なものです。

他のブロックチェーンは何をしているか？

EVMは間違いなく暗号通貨分野で最も広く使用されている仮想マシンです。ほとんどの代替L1およびL2ブロックチェーンは、既存のすべてのツールやフレームワークとの互換性を維持し、イーサリアムコミュニティから直接プロジェクトや開発者を引き付けるためにEVMを使用しています。

それにもかかわらず、近年、多くの異なる仮想マシンが登場しています。Solana VM、Wasm VM、Cairo VM、Move VMはおそらく最も有名で興味深いものであり、それぞれに独自の利点と欠点があります。これらはスマートコントラクト開発に対して異なるアプローチを取っています：

カスタム言語

CairoやMoveのような一部のプラットフォームは、スマートコントラクトを書くために特別に設計されたプログラミング言語を作成しています。これは、イーサリアムがその仮想マシンである

EVMのためにSolidityやVyperを使用しているのと同様です。

標準言語

そのほかのSolanaやWebAssembly（Wasm）を使用するプラットフォームなどは、広く使用されているプログラミング言語でスマートコントラクトを書くことを可能にしています。例えば、これらのプラットフォームはスマートコントラクト開発にRustをサポートしていることが多いです。

これらの代替仮想マシンがEVMと異なるもうひとつの領域は、トランザクションの*並列化*です。EVMは並列化なしにトランザクションを順次処理することはすでに述べました。いくつかのプロジェクトはこの欠点を取り上げ、改善しようとしました。例えば、Solana VMとMove VMの両方がトランザクションを並列に実行できますが、常に可能というわけではありません。たとえば、2つのトランザクションが同じコントラクトとやり取りして同じストレージを変更する場合、それらは並列に実行できません。

仮想マシンのパフォーマンスを向上させるこれらの取り組みは、イーサリアムの外部だけで行われているわけではないことを言わなければなりません。実際、多くのチームが並列化やEVMバイトコードからネイティブマシンコードへの*事前コンパイル(ahead-of-time)*（AOT）や*実行時コンパイル(just-in-time)*（JIT）などのクールな機能を追加することで、EVMの現在の制限を打破しようと取り組んでいます。

EVM命令セット（バイトコード演算）

EVM命令セットでは、次のような多くの操作が用意されています。

- 算術演算とビット単位の論理演算
- 実行コンテクストに関する問い合わせ
- スタック、メモリ、ストレージへのアクセス
- 制御フローに関する操作
- ロギング、関数コール、その他の演算子

一般的なバイトコード操作に加えて、EVMはアカウント情報（アドレスや残高）やブロック情報（ブロック番号や現在のガス価格）にアクセスする操作も用意されています。　利用可能なオペコードとその動作に着目してEVMを詳細に調べてみましょう。想定どおりかもしれませんが、オペランドはすべてスタックから取得され、その結果（適用可能な場合は）がしばしば、スタックの先頭に書き戻されます。

使用可能なオペコードは、次のカテゴリーに分類できます。

算術演算

算術演算命令には以下が含まれます：

```
ADD        // スタックの上から2つの要素を足し合わせる
MUL        // スタックの上から2つの要素を掛け合わせる
SUB        // スタックの上から2つの要素を引き算する
DIV        // 整数除算
SDIV       // 符号付き整数除算
MOD        // モジュロ（剰余）演算
SMOD       // 符号付きモジュロ演算
ADDMOD     // 任意の数のモジュロ加算
MULMOD     // 任意の数を法とする乗法
EXP        // 指数演算
SIGNEXTEND // 2の補数符号付き整数の長さを拡張
SHA3       // メモリブロックの Keccak-256ハッシュを計算
```

　すべての算術演算は、特に明記されていない限り、2^256を法として実行され、0^0（ゼロの0乗）は1であるとみなされることに注意してください。

スタック操作

スタック、メモリ、ストレージ管理命令には以下が含まれます：

```
POP      // スタックの一番上の項目を削除する
MLOAD    // メモリからワードをロードする
MSTORE   // ワードをメモリに保存する
MSTORE8 // メモリに 1バイトを保存する
SLOAD    // ストレージからワードをロードする
SSTORE   // ワードをストレージに保存する
TLOAD    //一時ストレージからワードをロードする
TSTORE   //一時ストレージにワードを保存する
MSIZE    // アクティブなメモリのサイズをバイト数で取得する
PUSH0    //スタックに値0を配置する
PUSHx    // xバイトの要素をスタックに置く
         // xは1〜32（フルワード）を含む任意の整数
DUPx     // スタックの x番目の要素を複製する
         // xは1〜16を含む任意の整数
SWAPx    // スタックの最初の要素と（x+1）番目の要素を交換する
         // xは1から16までの任意の整数
```

処理フロー操作

制御フロー命令には以下が含まれます：

```
STOP        // 実行を停止する
JUMP        // プログラムカウンタを任意の値に設定する
JUMPI       // プログラムカウンタを条件付きで変更する
PC          // プログラムカウンタの値を取得する（インクリメント前の値）
            // この命令に対応）
JUMPDEST    // 有効なジャンプ先としてマークする
```

システムに関連する操作

プログラムを実行するシステムのオペコードには以下が含まれます：

```
LOGx            // x個のトピックを持つログレコードを追加（xは0から4までの任意の整数）
CREATE          // 関連付けられたコードで新しいアカウントを作成する
CALL            // 別のアカウントへのメッセージコール、つまり別のアカウントのコードの実行
CALLCODE        // 別のアカウントのコードでこのアカウントにメッセージコールを行う
RETURN          // 実行を停止して出力データを返す
DELEGATECALL    // 代替アカウントのコードを使用してこのアカウントにメッセージコールを行うが、
                // 送信者とバリューの現在の値を保持する
STATICCALL      // 静的なメッセージコールをこのアカウントへ行うが、どのアカウントの状態も変更できない。
REVERT          // 実行を停止し、状態変更を元に戻し、データと残りのガスを返す
INVALID         // 指定された無効な命令
SELFDESTRUCT    // 実行を停止し、コントラクト作成時のトランザクション中に実行された場合は、
                // 削除するアカウントとして登録する。使用は強く非推奨であり、
                // オペコードは非推奨と見なされている
```

論理演算

比較とビット論理のオペコードには以下が含まれます：

```
LT          // より小さいかどうかの比較
GT          // より大きいかどうかの比較
SLT         // 符号付きの小なり比較
SGT         // 符号付きの大なり比較
EQ          // 等価比較
ISZERO      // 単純NOT演算子
AND         // ビット単位の AND演算
OR          // ビット単位の OR演算
XOR         // ビット単位の XOR演算
NOT         // ビット単位の NOT演算
BYTE        // 256ビットワードから 1バイトを取得
```

環境に対する操作

実行環境情報を扱うオペコードには以下が含まれます：

```
GAS             // 利用可能なガスの量を取得する
                //（この命令に関する支払いが終わったあとのガス）
ADDRESS         // 現在実行中のアカウントのアドレスを取得する
BALANCE         // 指定されたアカウントの残高を取得する
ORIGIN          // この EVMの実行の 起点となった EOAのアドレスを取得する
CALLER          // この実行の直接の呼び出し元のアドレスを取得する
CALLVALUE       // この実行の呼び出し側がデポジットしたイーサ値を取得する
CALLDATALOAD    // この実行の呼び出し元が送信した入力データを取得する
CALLDATASIZE    // 入力データのサイズを取得する
CALLDATACOPY    // 入力データをメモリにコピーする
CODESIZE        // 現在の環境で実行されているコードのサイズを取得する
CODECOPY        // 現在の環境で実行されているコードをメモリにコピーする
GASPRICE        // トランザクションの作成元が指定したガス価格を取得する
EXTCODESIZE     // アカウントコードのサイズを取得する
EXTCODECOPY     // アカウントのコードをメモリにコピーする
RETURNDATASIZE  // 現在の環境で、前の呼び出し元の出力データのサイズを取得する
RETURNDATACOPY  // 直前の呼び出しからのデータ出力をメモリにコピーする
```

ブロックに関連する操作

現在のブロックの情報にアクセスするオペコードには以下が含まれます：

```
BLOCKHASH       // 最近生成が完了した256のブロックの内の 1つのハッシュを取得する
COINBASE        // ブロック報酬を受け取るアドレスを取得する
TIMESTAMP       // ブロックのタイムスタンプを取得する
NUMBER          // ブロックの番号を取得する
PREVRANDAO      // 前のブロックのRANDAOミックスを取得。このオペコードは
                // The Mergeハードフォーク以降、DIFFICULTYに代わるもの
GASLIMIT        // ブロックのガス上限を取得する
```

イーサリアムのステート（状態）

EVMの役割は、イーサリアムのステート（状態）を更新することです。ステートの更新は、スマートコントラクトのコードの実行の結果として、プロトコルによる定義どおりに有効な状態遷移を計算することで行われます。これは、イーサリアムはトランザクションベースのステートマシン（transaction-based state machine）であるという説明にもつながります。つまり、外部アクター（アカウント所有者とバリデータ）がトランザクションを作成し、受け付け、順序づけるこ

とで状態遷移を開始しているということです。以下では、イーサリアムの状態を構成するものについて見てきます。

最上位レベルでは、イーサリアムの*ワールドステート*があります。ワールドステートは、イーサリアムアドレス（160ビット値）からアカウントへのマッピングです。下位レベルでは、各イーサリアムアドレスは、イーサ残高（アカウントが所有するwei数として保存）、ノンス（EOAの場合はこのアカウントから正常に送信されたトランザクションの数、コントラクトアカウントの場合はコントラクトアカウントによって作成されたコントラクトの数を表す）、アカウントのストレージ（スマートコントラクトのみが使用する永続的なデータストア）、およびアカウントのプログラムコード（これもスマートコントラクトアカウントの場合のみ）で構成されるアカウントを表します。従来、EOAは常にコードを持たず、空のストレージを持っていました。しかし、EIP-7702（2025年5月のPectraアップグレードで有効化）は、EOAがスマートコントラクトにコード実行を委任できるようにすることで、この前提を変更します。EOAが委任にオプトインすると、そのコードフィールドは`0xef0100 || address`の形式の23バイトの*委任指定*に設定されます。ここで`address`はターゲットコントラクトの20バイトアドレスです。`0xef`はEIP-3541により禁止されたオペコードであり、この指定がデプロイ可能なコントラクトコードと混同されないことを保証します。EVMが委任されたEOAへの呼び出しに遭遇すると、ターゲットアドレスからコードをロードしてEOAのコンテクストで実行します。このことは委任されたEOAに対する`EXTCODESIZE`は23（委任指定のサイズ）を返し、委任されたコード内で実行される`CODESIZE`はターゲットコントラクトのコードのサイズを返すことを意味します。EOAは委任されたコードを実行する際にストレージも使用できますが、委任コントラクト間を移行する際の衝突を防ぐために、プロトコルはERC-7201の名前空間付きストレージレイアウトの使用を推奨しています。重要なのは、委任されたEOAは引き続きトランザクションを送信できることです。トランザクションに署名して送信できるEOAの機能はそのままですが、さらにスマートコントラクト機能を持つようになります。委任はゼロアドレス（`0x0000000000000000000000000000000000000000`）を委任先に指定することでクリアできます。

トランザクションによってスマートコントラクトのコードが実行されると、現在作成されているブロックおよび処理中の特定のトランザクションに関して必要なすべての情報を含むEVMがインスタンス化されます。特に、EVMのプログラムコードROMには、コントラクトアカウントのコードが呼び出されて読み出され、プログラムカウンタはゼロに設定され、ストレージ情報がコントラクトアカウントのストレージから読み出され、メモリはすべてゼロに設定され、すべてのブロックおよび環境変数が設定されます。ここで重要な変数は、この実行のために設定されたガス供給量です。ガス供給量は、トランザクションの開始時にトランザクション送信者が支払ったガスの量に設定されています（詳細は「実行中のガス会計」を参照）。コード実行が進むにつれて、ガス供給量は実行された操作のガス費用に応じて減少します。いずれかの時点でガス供給量がゼロ未満になると、*ガス切れ*（Out of Gas: OOG）例外が発生します。コントラクトコードの実行は直ちに停止し、トランザクションは放棄されます。イーサリアムの状態への変更は適用されません。ただし、送信者のノンスがインクリメントされ、停止点までのコード実行に使用されたリソースに対する対価としてブロックの受益者に支払うためにイーサ残高が減少します。この時点で、EVMはイーサリアムワールドステートのサンドボックス化されたコピー上で実行されていると考えることができ、何らかの理由で実行が完了できない場合、このサンドボックスは完全に破棄されます。ただし、実行が正常に完了した場合、呼び出されたコントラクトのストレージ

データへの変更、作成された新しいコントラクト、開始されたイーサ残高の転送を含め、実世界の状態はサンドボックス版に一致するように更新されます。

コード実行は再帰的なプロセスです。コントラクトは他のコントラクトを呼び出すことができ、各呼び出しは新しい呼び出し対象の周りにもうひとつのEVMがインスタンス化されます。各インスタンスは、上のレベルのEVMのサンドボックスからサンドボックスワールドステートが初期化されます。インスタンス化（コンテクスト）の際には、指定された量のガスがガス供給として与えられます（もちろん、上のレベルで残っているガス量を超えない）。このため、実行を完了するのに十分なガスが与えられなかった場合は例外で停止する可能性があります。この場合も、サンドボックス状態は破棄され、実行は上のレベルのEVMに戻ります。

イーサリアムステートレス

執筆時点（2025年6月）では、すべてのイーサリアムノードは、含まれるすべてのトランザクションを再実行してすべての新しいブロックの正確性を確認できるようにするために、最新の状態（前述のワールドステート）を計算して維持する必要がありますが、部分的にそれを取り除く計画があります。

そのアイデアは、サーチャーやビルダーなどの限定されたアクターのセットが、新しいブロックを作成して公開するために引き続き状態にアクセスする必要がある一方で、他のすべてのノードは状態なしでそれらのブロックを暗号的に検証できるようにすることです。これは*ステートレスネス*と呼ばれます。

ステートレスネスは、コアプロトコルにいくつかの変更が必要なため、イーサリアムロードマップ上ではまだまだはるか先の話です。

ブロック提案者とビルダーの分離（ePBS）

トランザクションを収集してブロックを作成する作業と、P2Pネットワークに提案する作業を分離します。前者は、サーチャーやビルダーと呼ばれる高度に専門化されたエンティティによって行われ、超最適化されたブロックを作成できます。後者はイーサリアムバリデータノードによって行われます。ePBSはプロトコルにまだ組み込まれていませんが、メインネットではすでに実現されています。実際、イーサリアムブロックの大半は、非常に小さな大手ビルダーのセットによって構築されています。

Verkleツリー

イーサリアムが状態を保存するために現在使用しているデータ構造であるマークルパトリシアツリーの置き換えです。これにより、状態の正当性を検証するために必要な暗号証明のサイズが大幅に削減され、変更前のマークルパトリシアツリーよりも検証が高速になります。

注記

> 他のハッシュベースのバイナリツリーが、バークルツリーの代替としてテストされている。核心的なアイデアは、迅速かつ容易に検証できる小さな証明を作成できる状態用のデータ構造を持つことである。

これら2つのアップグレードを組み合わせることで、ブロックを作成する強力なハードウェアを備えた大規模組織のみが完全な状態を保存・アクセスする必要があるというシナリオが実現します。新しいブロックと共に、*暗号証人*が作成されます。これは、ブロックに含まれるトランザクションに基づいて新しい状態が正しく計算されたことを証明する最小限のデータセットです。

他のすべてのノード（バリデータノードを含む）は、全体の状態のハッシュである*ステートルートのみ*を保存します。新しいブロックを受信すると、そのブロックに関連付けられた証人を使用してその正確性を検証します。

これにより、完全な状態を保存する必要も、すべてのトランザクション（EVM内で）を再実行する必要もなくなり、イーサリアムノードの実行が非常に軽量になります。さらに、引き続き第三者を信頼する必要がなくすべてが正しいことを検証できます。スマートフォンでもノードを実行できるようになるでしょう…

研究は急速に進んでいますが、メインネットでステートレスネスを実現するまでにはおそらくまだ数年かかるでしょう。

マークルパトリシアツリー

現在、Ethereumの状態は*修正マークルパトリシアツリー*と呼ばれる非常に特殊なデータ構造を使用して保存されています。前のセクションでマークルパトリシアツリー（以下、MPT）について簡単に触れましたが、その仕組みと理由、そしてEthereumが状態を保存する方法としてどのように使用されているか（そしてそれだけではありません）を理解することは非常に重要です。なぜなら、同じ考え方がバークルツリーにも当てはまるからです。MPTについて詳しく説明する前に、マークルツリーについて知っておく必要があります。なぜなら、マークルツリーはMPTの基盤となるからです

マークルツリー

マークルツリーは、1988年にラルフ・マークルがより良いデジタル署名を構築しようとして発明した非常に古いデータ構造です。データベース全体を送信せずに、データがデータベースに存在し、改ざんされていないことを検証する必要がある場合に非常に効率的です。

データのコレクションからMerkleツリーを作成するのは簡単です。データをいくつかのチャンクに分割し、それらのチャンクを一緒にハッシュし、最終的に1つのチャンクだけになるまでこの最後のステップを再帰的に繰り返します。そのチャンクは*Merkleルート*です。これは、ツリーを作成するために使用されたすべてのデータの一種のデジタル指紋です。

バイナリMerkleツリー（Merkleツリーの最も単純な形式）をゼロから作成して、もう少し慣れ親しみましょう。8つのデータチャンクから始めます。これらは英語の異なる単語と考えてみましょう。特定のハッシュ関数を使用して各チャンクをハッシュします。第4章ですでに述べたように、イーサリアムはKeccak-256ハッシュ関数を使用します。図14-2でhash_1、hash_2などとして表されるMerkleツリーの葉を取得します。次に、各リーフのペアを連結して再びハッシュし、hash_12、hash_34などを作成します。連結とハッシュのこのプロセスをさらに2回繰り返し、単一の最終結果であるMerkleルート：hash_12345678を取得します。

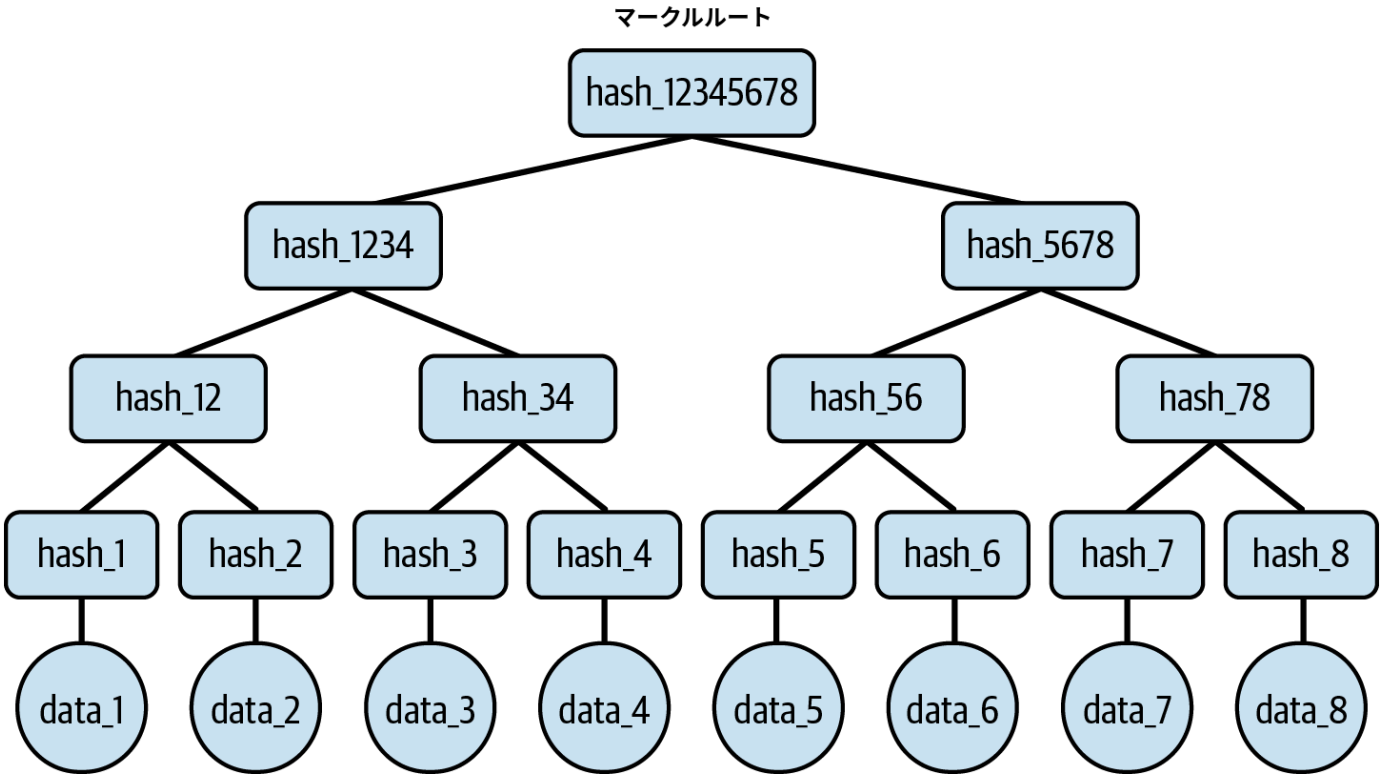


図14-2. バイナリMerkleツリー

データを保存するためになぜMerkleツリーが必要なのか疑問に思うかもしれません。各チャンクを従来のデータベースに保存するよりも複雑ではないでしょうか？

答えはイエスです。キーバリューやSQLデータベースにすべてのチャンクを保存するよりもはるかに複雑です。この種のデータ構造を使用する唯一の理由は、チャンクのいずれかがデータのコレクションに存在し、操作されていないことの暗号証明を安価に提供するのに非常に効率的だからです。実際、特定のチャンクを持っていることの証明を提供するように求められた時に、通常のデータベースを使用してデータを保存している場合、私たちが嘘をついていないことを確認できるようにデータセット全体を公開する必要があります。

前の例を使用して、具体的に見てみましょう。data_1がデータセットに含まれていることを証明したいとします。単純なアプローチは、data_1からdata_8までの全データセット（合計8項目）を提供することです。Merkleツリーを使用すると、hash_2、hash_34、hash_5678のみを提供すればよいです。その後、誰でも自分でMerkleルートを計算し、最初に計算した（公開されている）Merkleルートと比較できます。一致すれば、図14-3に示すように、data_1が初期データセットの一部であることを完全に確信できます。

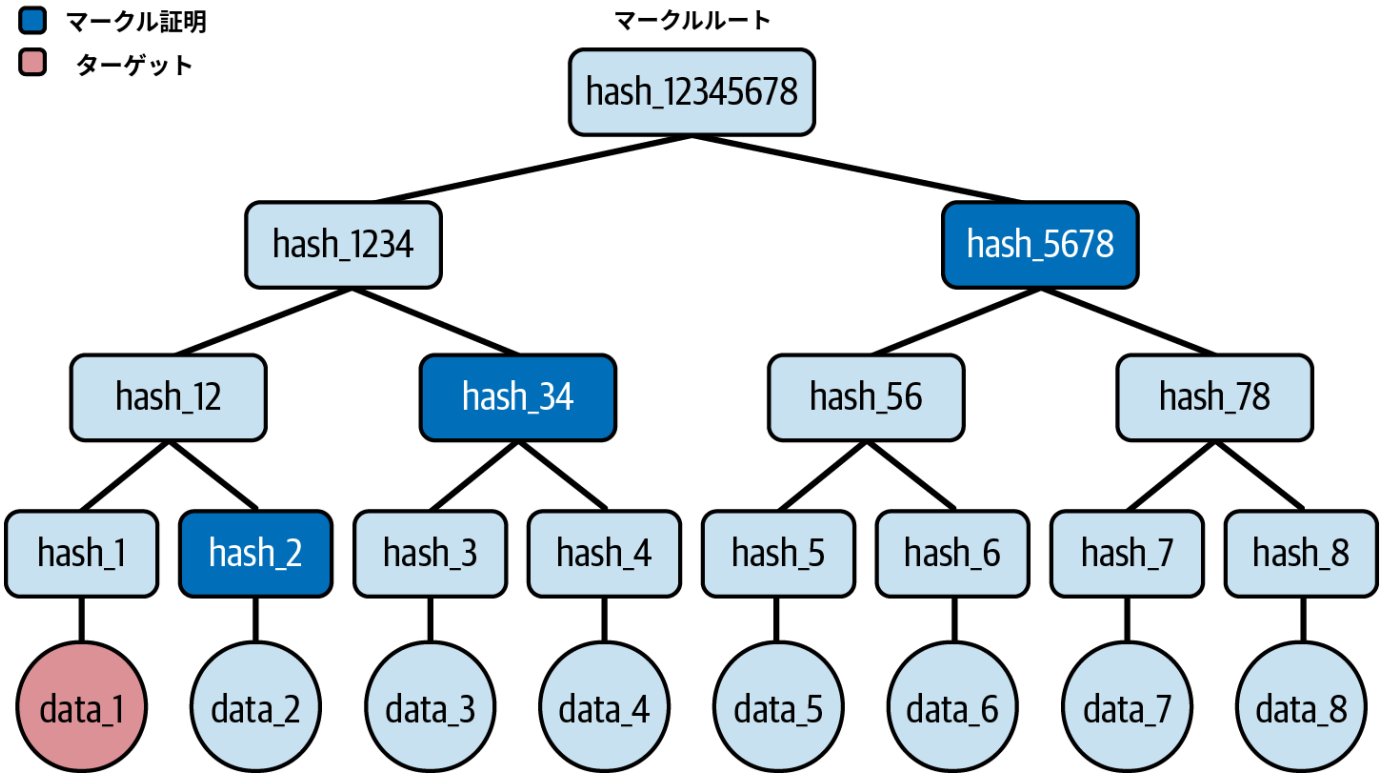


図14-3. data_1がツリーに含まれていることを検証するMerkle証明

ヒント

Merkleツリーを再構築するには、次の手順に従う：

1. data_1をハッシュしてhash_1を取得する
2. hash_1を提供されたhash_2と連結してハッシュし、hash_12を取得する
3. hash_12を提供されたhash_34と連結してハッシュし、hash_1234を取得する
4. hash_1234を提供されたhash_5678と連結してハッシュし、最終的なMerkleルートを取得する

Merkleツリーなしの単純なアプローチでは8つのデータが必要なのに対し、3つのデータのみを使用していることに注意してください。これは単なる単純な例です。大量のデータの場合は節約効果はより大きくなります。

数学的に言えば、Merkleツリーは図14-4に示すように単純なアプローチが線形なO(n)に対して、O(log(n))の計算量を提供します。

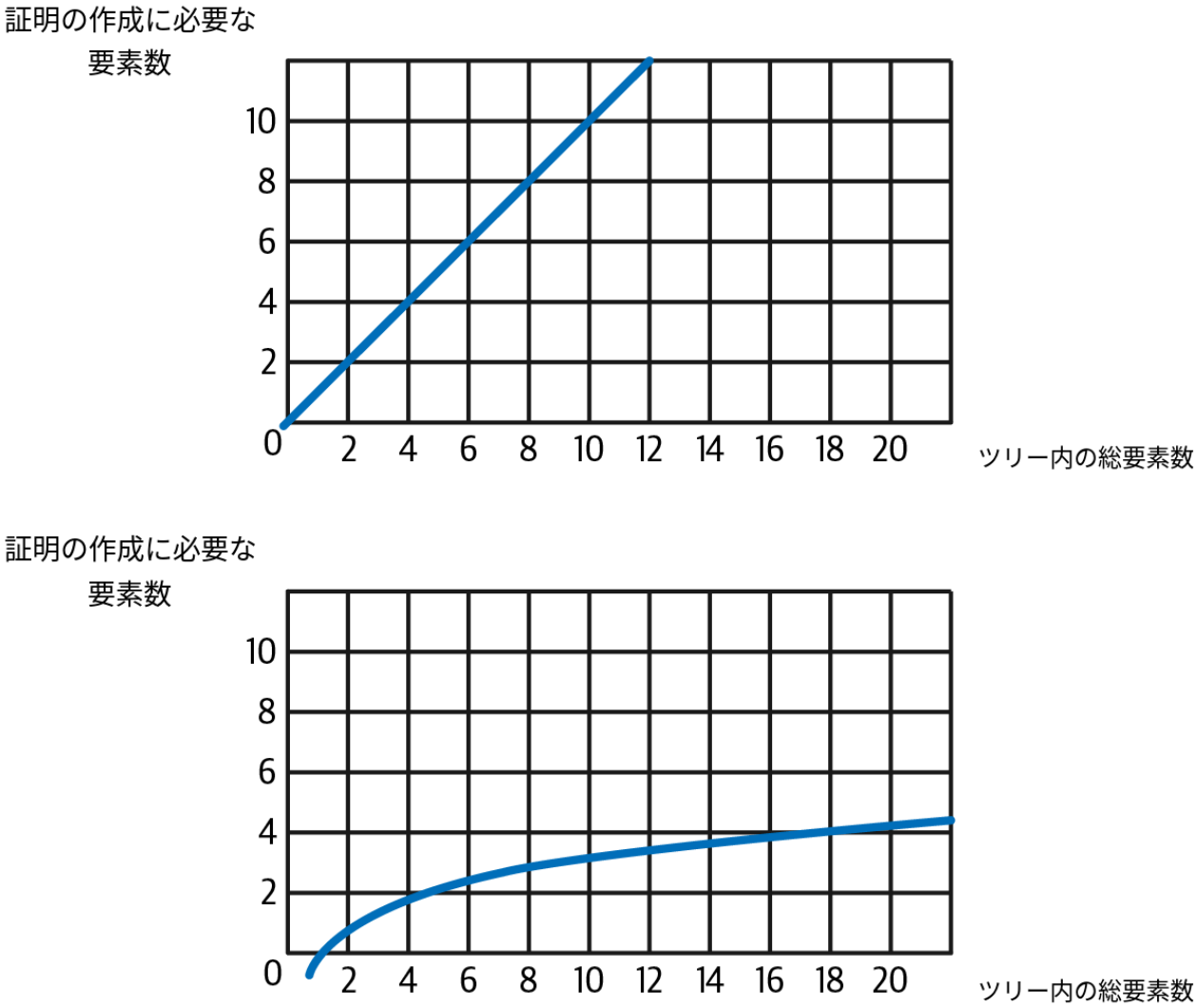


図14-4. O(n)（上）対O(log(n))（下）

イーサリアムの世界では、これはアドレスの残高、トランザクションの結果、または特定のスマートコントラクトのバイトコードに関する証明を、より安価で簡単に提供できることを意味します。

BitcoinにおけるMerkleツリー

Bitcoinはブロックチェーン技術におけるMerkleツリー活用の先駆者です。実際、すべてのBitcoinブロックにはそのブロックに含まれるすべてのトランザクションのMerkleルートが含まれているため、ブロックヘッダーを変更せずに（PoWも損なわずに）トランザクションを変更することは不可能です。

イーサリアムにおけるマークルパトリシアツリー（Merkle-Patricia Trie）

イーサリアムは同じ概念を取り入れ、独自のニーズに合わせていくつかの変更を加えました。Merkleツリーは、Bitcoinトランザクションのような不変の永続的なデータには適しています。

しかし、イーサリアムの状態は常に変化するためデータを頻繁に変更できるようにしながら、有用な特性を維持するためにMerkleツリーを調整する必要があります。

ここでマークルパトリシアツリーが登場します。名前は、マークルツリー、Patricia（Practical Algorithm to Retrieve Information Coded in Alphanumeric）、および*retrieval*（検索）に由来する*trie*という単語の組み合わせから来ており、最適化の目的を思い出せる名前になっています。

基本的に、マークルパトリシアツリーは各ノードに16の子を持つ修正されたマークルツリーです。キー自体がツリー内の正しい位置に到達するパスにエンコードされているため、多くのキー・バリュー項目（キーはアドレスであり、バリューは残高、ノンス、コード（存在する場合）などの各アドレスのアカウント情報）を持つイーサリアムの状態データに適しています。

次のキーバリュー項目を保存したいとします。

```
car → Schumacher
```

Carは（0x）6 3 6 1 7 2として16進エンコードされます。そのため、まず初めにMerkleルートから6番目の子を選択します。次に、３番目の子供を選択します。このプロセスをそのキーに関連付けられた値（この例ではSchumacher）を読み取ることができる最終値に到達するまで繰り返します。この例を図14-5に示します。

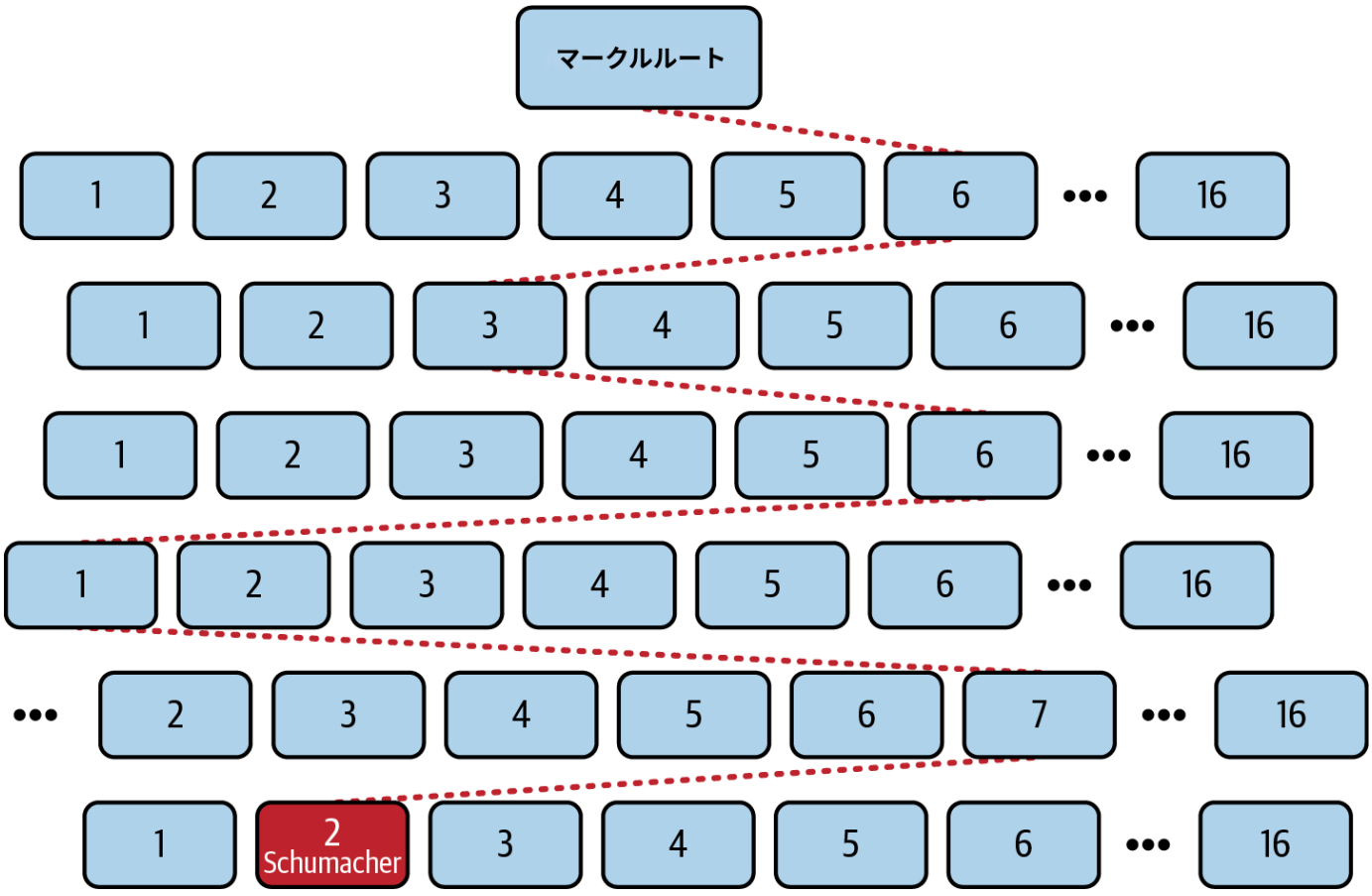


図14-5. キーバリュー項目car --> Schumacherのマークルパトリシアツリーへのエンコード

特に、イーサリアムは4つのマークルパトリシアツリーを使用します：

ステートツリー

全体の状態を保存する

トランザクションツリー

ブロックに含まれるすべてのトランザクションを保存する

レシートツリー

ブロックに含まれるすべてのトランザクションの結果を保存する

ストレージツリー

スマートコントラクトのデータを保存する

　すべてのイーサリアムブロックヘッダーには、ステート、トランザクション、レシートツリーのマークルルートが含まれ、すべてのアカウント（ステートツリーに含まれる）は独自のストレージツリーのマークルルートを保存します。

EVMコンポーネントの詳細

　このセクションでは、EVMの各コンポーネントがどのように機能するかを詳しく見ていきます。最後に実際の例を使って動作を確認します。

スタック

　スタックは、操作を実行するために*後入れ先出し*（LIFO）の順序に従う非常に単純なデータ構造であり、すべてのアイテムは32バイトのオブジェクトです。同時に最大1,024個のアイテムを扱えます。

EVMは、さまざまな種類のオペコードを通じてスタックにアイテムをプッシュおよびポップでき、図14-6に示すようにアイテムの順序を操作できます。

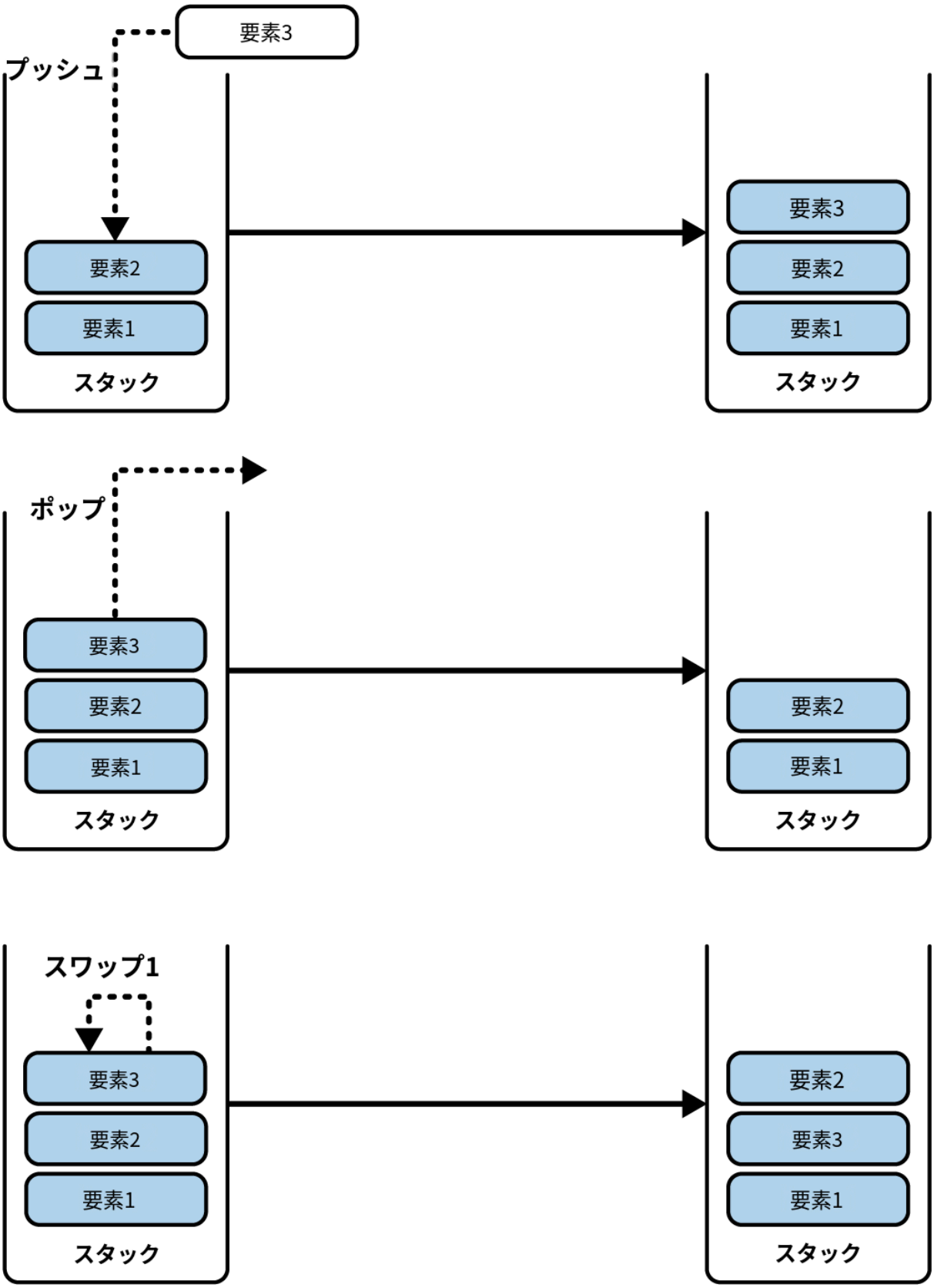


図14-6. EVMスタックはLIFO操作順序に従う

メモリ

EVMメモリはバイト単位でアドレス指定可能なデータ構造で、基本的に非常に長いバイト配列です。実際、メモリ内のすべてのバイトは32バイト（256ビット）のキーを使用してアクセスでき、最大2^256バイトを含むことができます。揮発性であり、実行終了後に削除され、常に0に初期化されます。

メモリから単一バイトを読み書きすることは可能ですが、ほとんどの操作では、図14-7に示すように、より大きなデータチャンク（通常は32バイトチャンク）の読み書きが必要です。

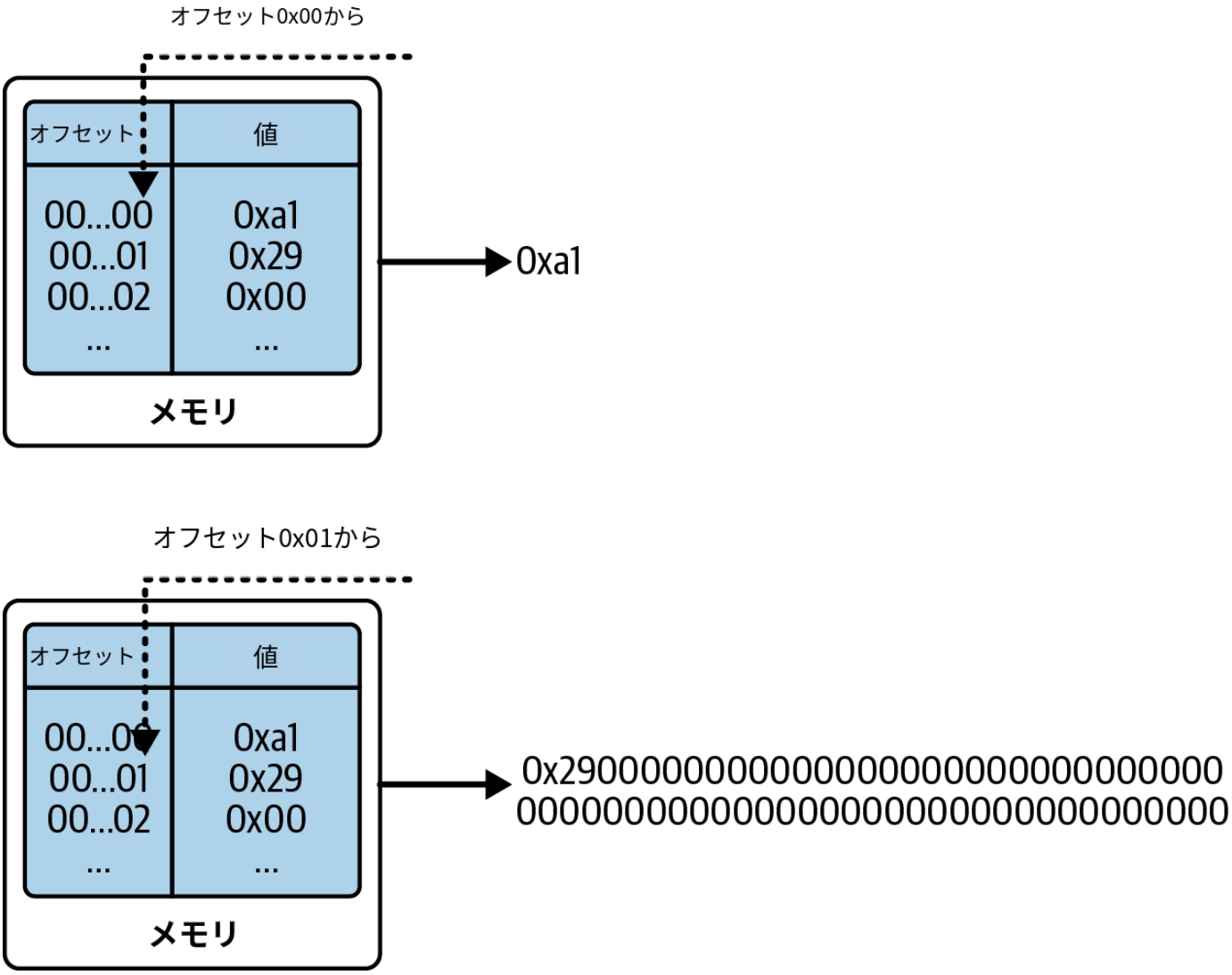


図14-7. EVMメモリは揮発性の、バイト単位でアドレス指定可能なデータ構造

注記

技術的には、EVMから単一のバイトを読み取ることはできない。読み取れるのは32バイトのワード全体のみである。単一バイトの読み取りと同じ結果を得るには、EVMはそのバイトを含むワード全体をロードし、選択したバイトのみを返すために「カット」する必要がある。

ストレージ

EVMストレージは、キー（多くの場合*スロット*と呼ばれる）と値がそれぞれ32バイトの長さのキーバリューデータ構造です。これはすべてのスマートコントラクトの永続的な記憶領域であり、保存されたすべての値は異なるトランザクション間やブロックにわたって無期限に保持されます。各スマートコントラクトは自身のストレージにのみアクセスと変更が可能で、値を含まないスロットにアクセスしようとすると、エラーをスローせずに常に値0を返します。図14-8は、2つのコントラクトのストレージの基本的な表現を示しています。

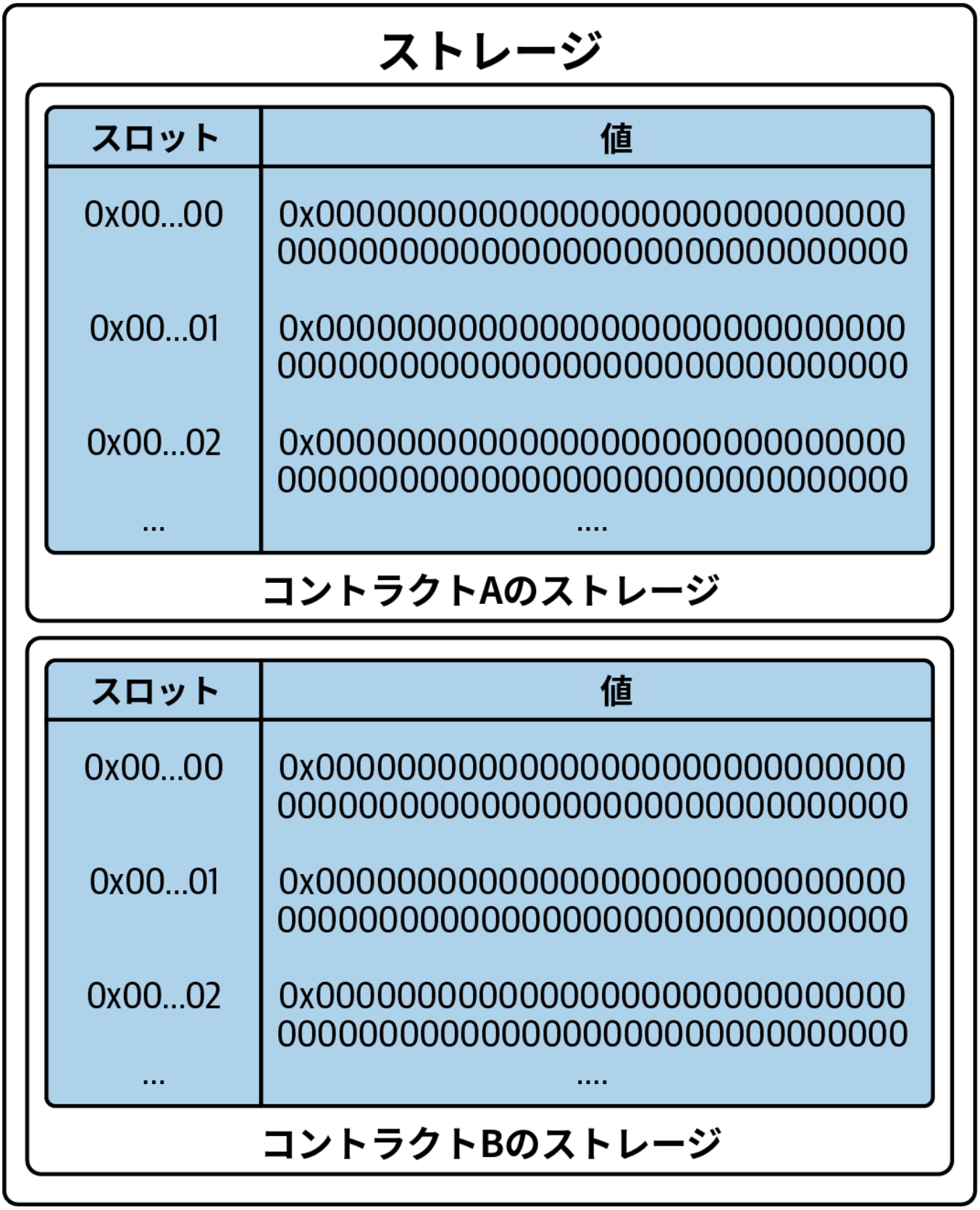


図14-8. EVMストレージはキーバリューデータ構造を持つ永続的な記憶領域

EIP-1153で追加された*一時ストレージ*もあり、通常のストレージと同じように動作しますが、唯一の違いはトランザクションの実行後に完全に破棄されることです。このため、通常のストレージよりもはるかに安価に使用できます。

コールデータ

コールデータは不変のデータ構造であり、次の*コールフレーム*（サンドボックスEVM環境）への入力として送信されるバイト列を格納します。たとえば、コントラクト作成トランザクションでは、コールデータにはデプロイされるコントラクトのバイトコードが含まれます。単純なETH転送の場合などでは空になります。

具体的な例ですべてをまとめる

コントラクトAに対するトランザクションを実行しているとします。EVMバイトコードは次のとおりです：`60425F525F3560AB145F515500`

初期コールデータは次のとおりです：

```
00000000000000000000000000000000000000000000000000000000000000ab
```

EVMバイトコードを人間が読める形式で表現しましょう：

```
[00]            PUSH1           42
[02]            PUSH0
[03]            MSTORE
[04]            PUSH0
[05]            CALLDATALOAD
[06]            PUSH1           AB
[08]            EQ
[09]            PUSH0
[0a]            MLOAD
[0b]            SSTORE
[0c]            STOP
```

> **注記**
>
> 各EVMオペコードは一意の1バイト値（0x00から0xFFの範囲）で識別される。たとえば、0x60はPUSH1オペコード、0x5FはPUSH0などである。すべてのオペコードとその16進表現の一覧については、EVM codesを参照のこと。

EVMがこれらのオペコードをどのように実行し、スタック、メモリ、ストレージがどのように操作されるかを見てみましょう。図14-9はEVMの初期状態を示しています。

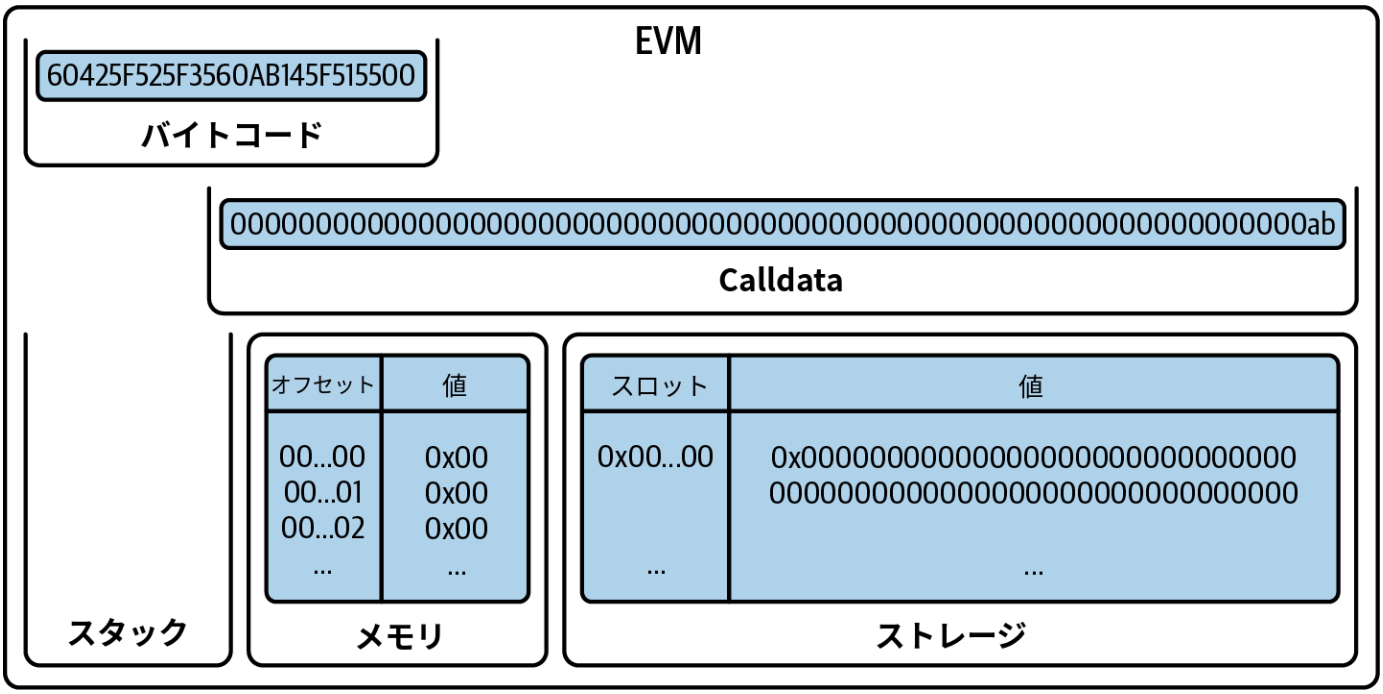


図14-9. EVMの初期状態

最初のオペコードは0x42をスタックにプッシュします（図14-10）。なお、プッシュオペコードは自身の次のバイトをプッシュするデータとして取得します。

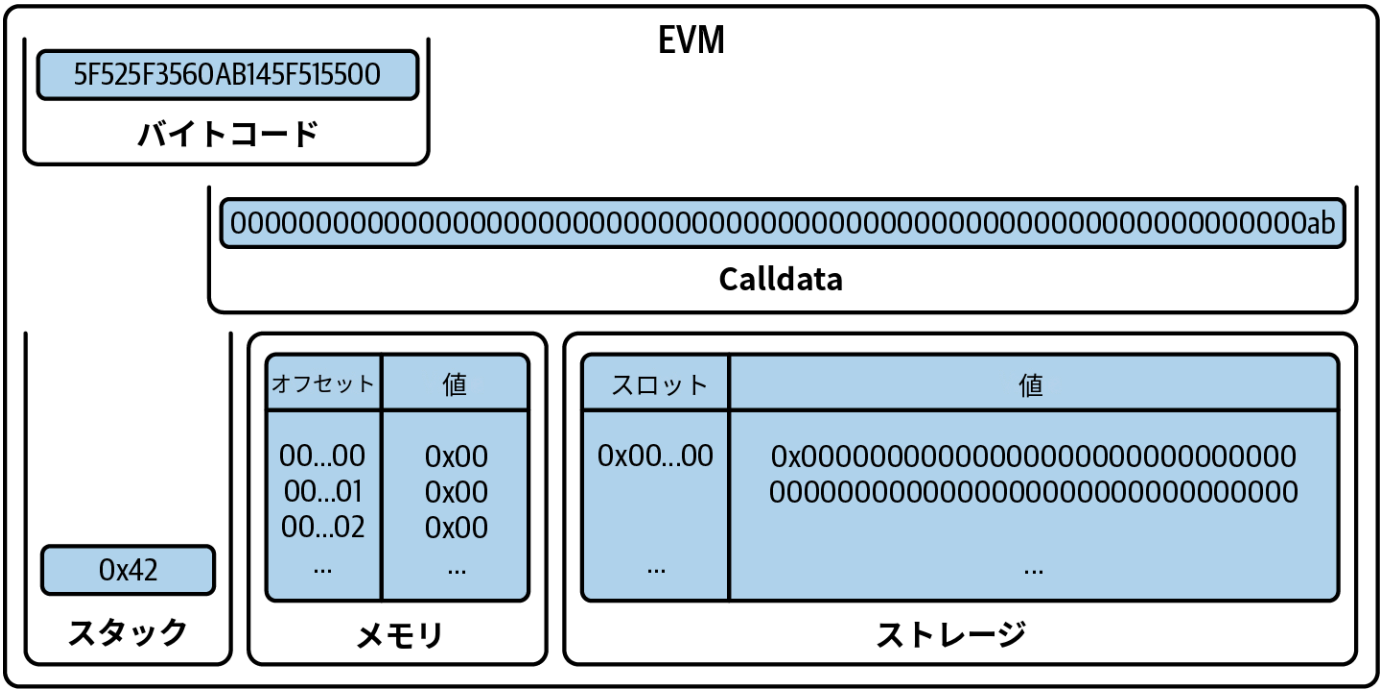


図14-10. PUSH1 0x42後のEVM

次に、PUSH0は0x00をスタックにプッシュします（図14-11）。

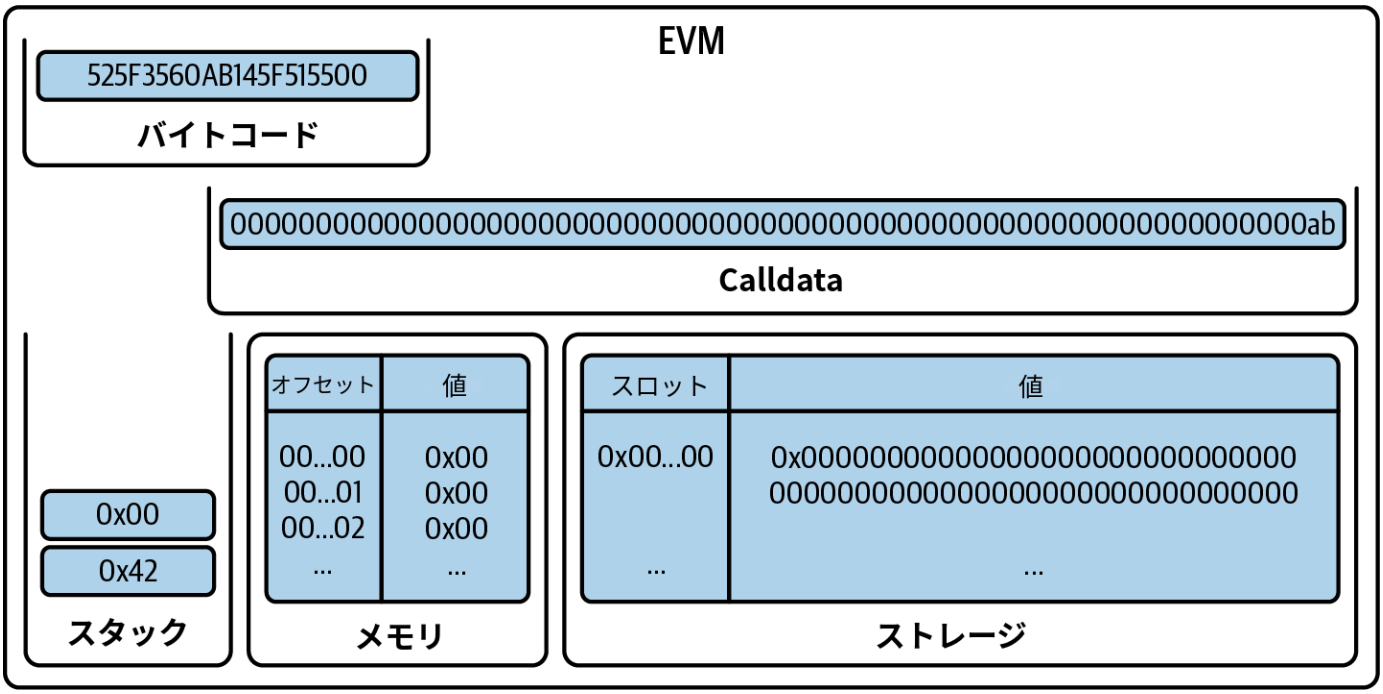


図14-11. PUSH0後のEVM

MSTOREはスタックから2つのアイテムをポップし、1つ目をオフセット（バイト単位）、2つ目をそのオフセットからメモリに書き込む値として解釈します。バイト0x42の前にメモリ上に多くの先頭のゼロ埋め（31バイト分）があることに注意してください。これはスタック内のすべての値が32バイトであるためです。ほとんどの場合、書き込み時に先頭のゼロ埋めは無視できますが（図14-12ではアイテム0x42が確認できます）、実際には32バイトの値であることを忘れないでください。

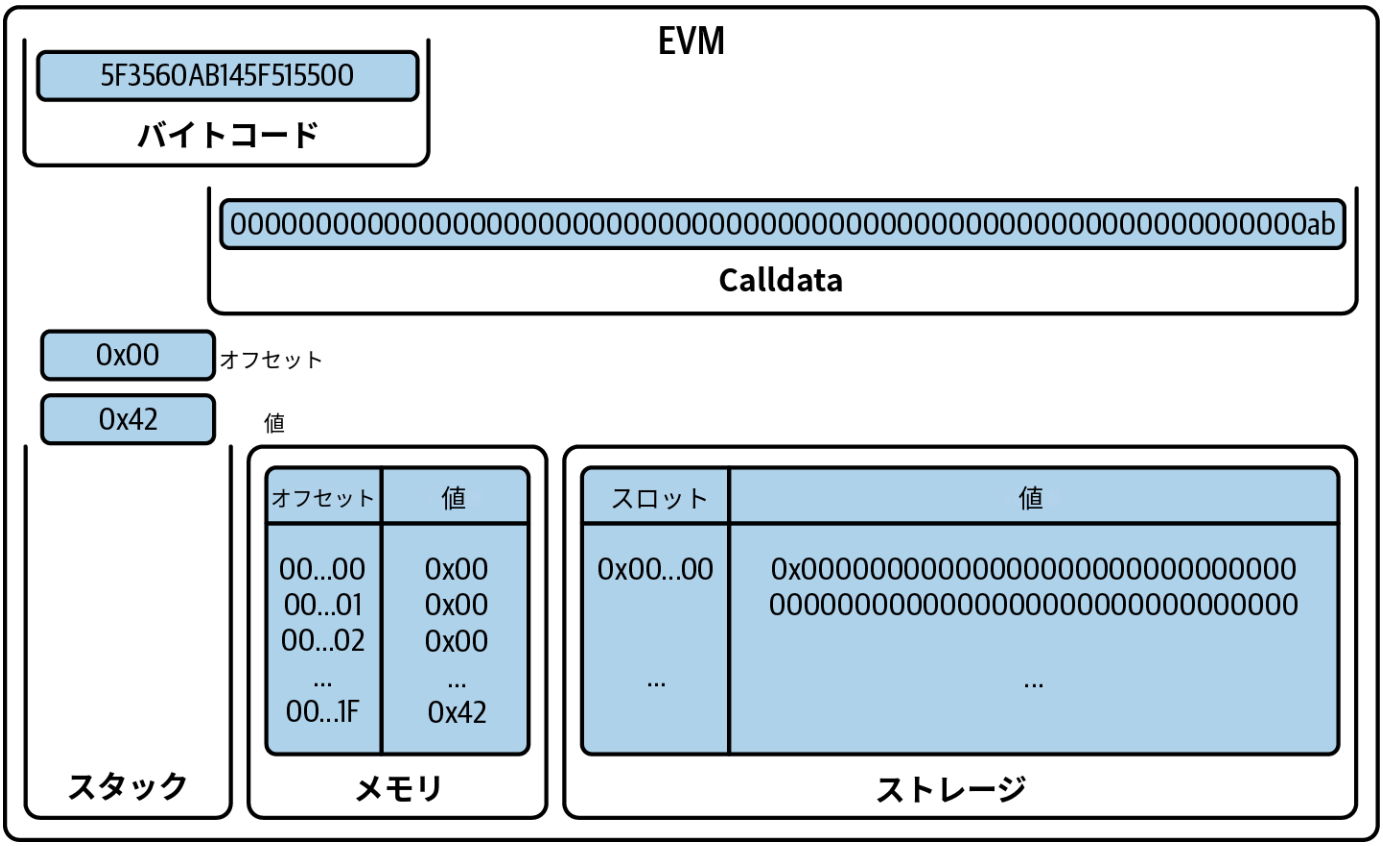


図14-12. MSTORE後のEVM

次に、再びPUSH0が0x00をスタックにプッシュします（図14-13）。

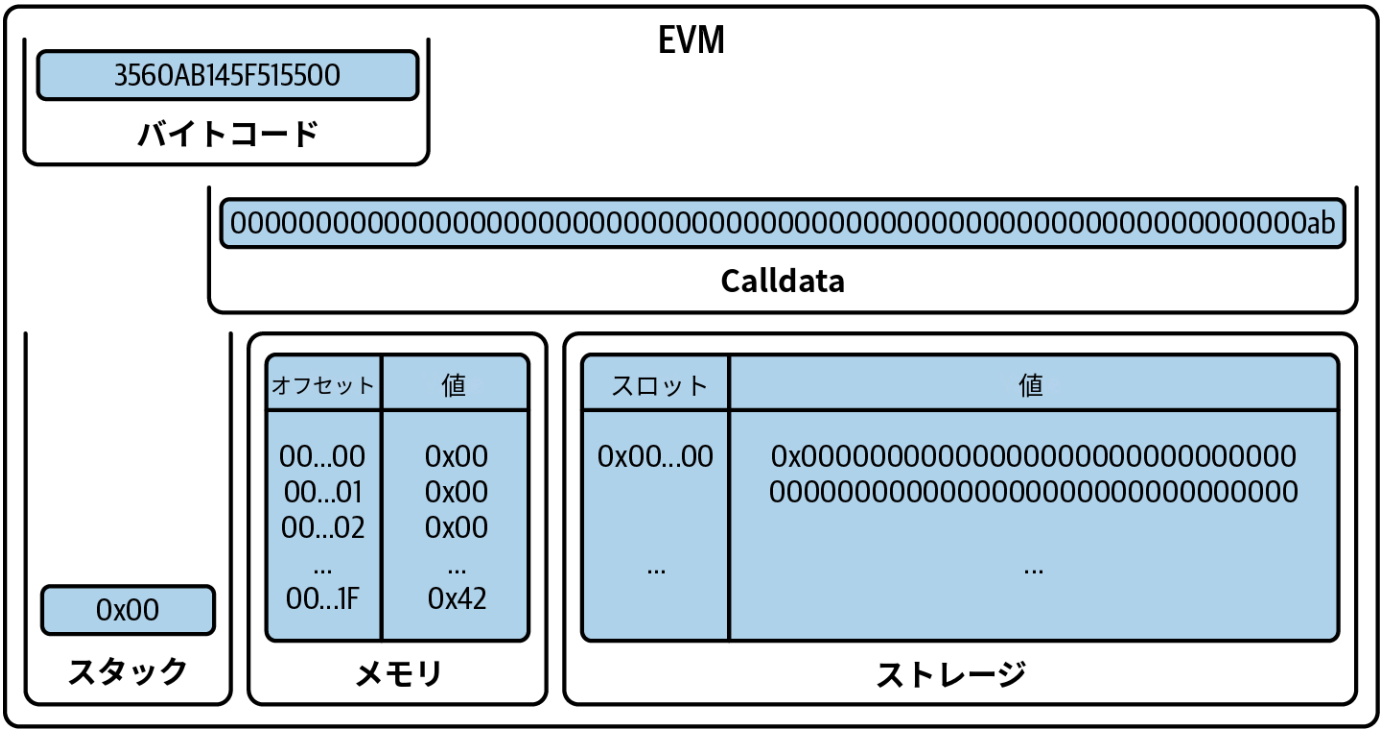


図14-13. PUSH0後のEVM

CALLDATALOADはスタックから1つの要素を取り出し、オフセットとして解釈し、そのオフセットから始まるコールデータの32バイト値を返してスタックにプッシュします。ここでは、図14-14に示すように0xabを返しています（先頭のゼロ埋めは重要でないため省略します）。

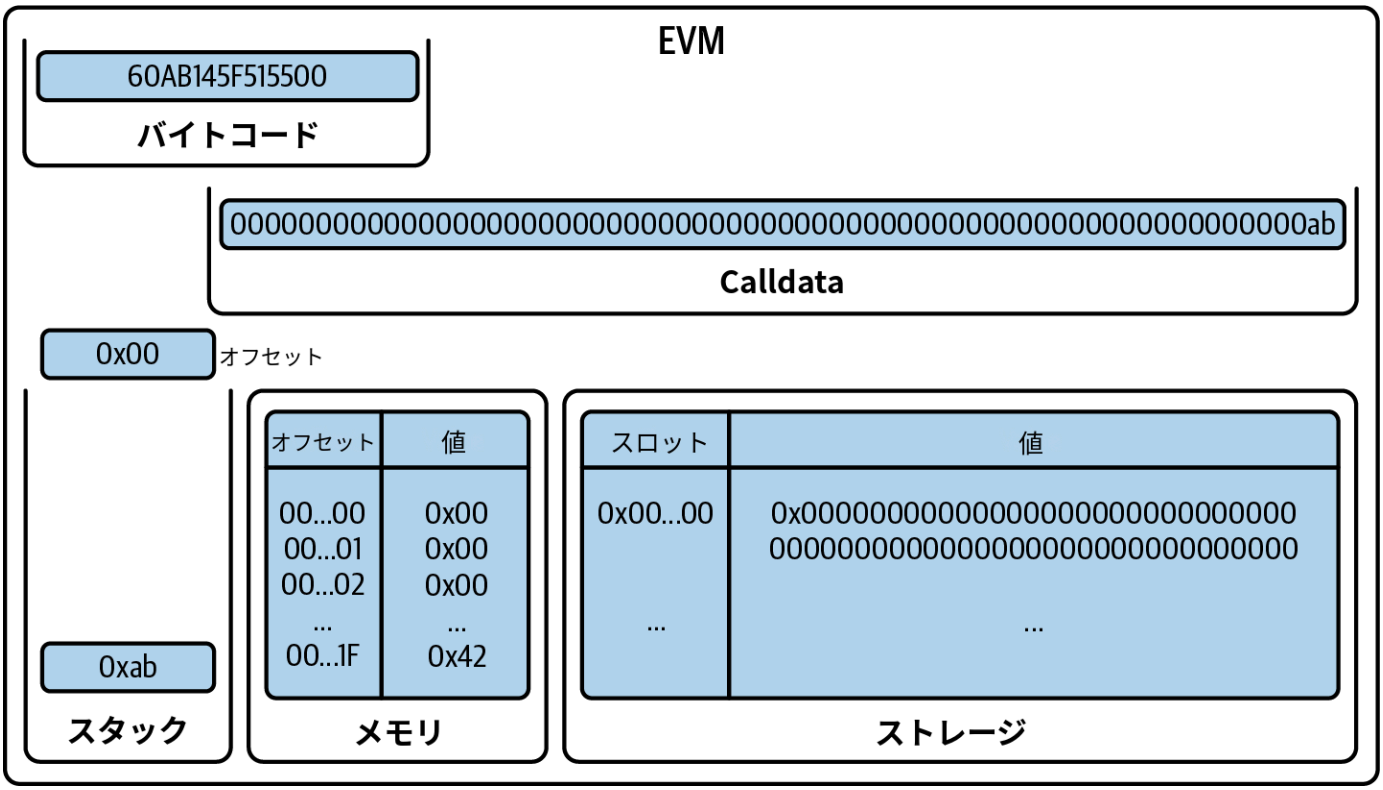


図14-14. CALLDATALOAD後のEVM

PUSH1は0xabをスタックにプッシュします（図14-15）。

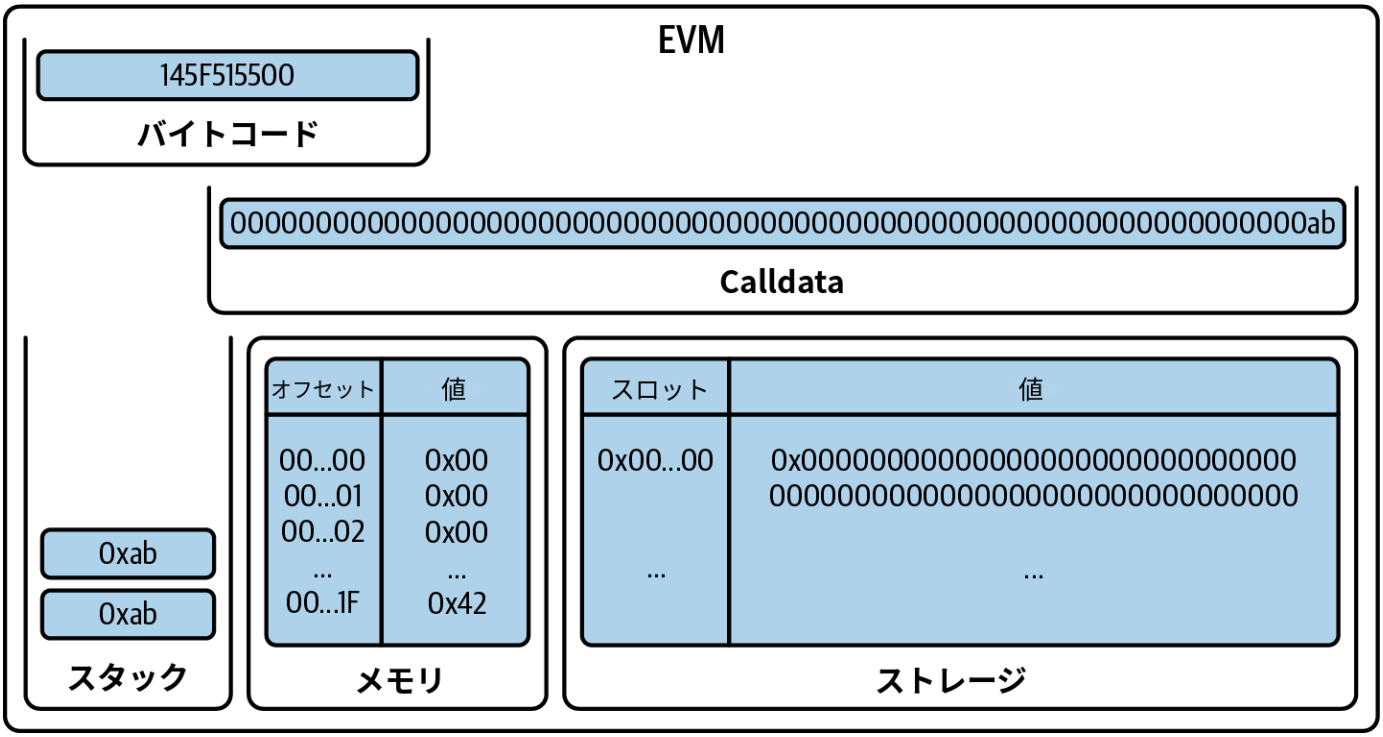


図14-15. PUSH1 0xab後のEVM

EQはスタックから2つのアイテムをポップして比較し、等しい場合は1を、そうでない場合は0を返します。この例では、図14-16に示すように、2つの値が等しいため0x01を返します。

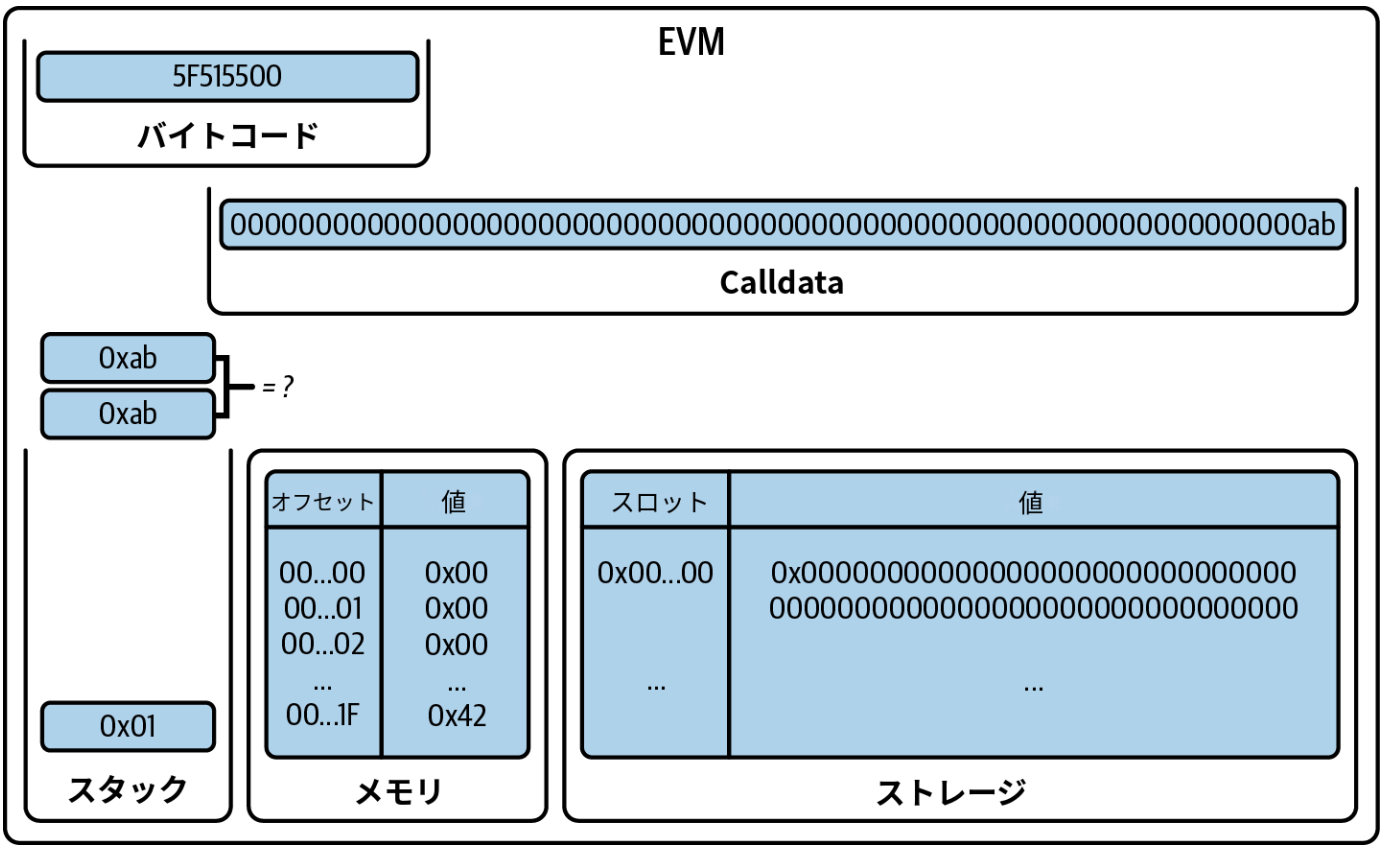


図14-16. EQ後のEVM

再び、PUSH0が0x00をスタックにプッシュします（図14-17）。

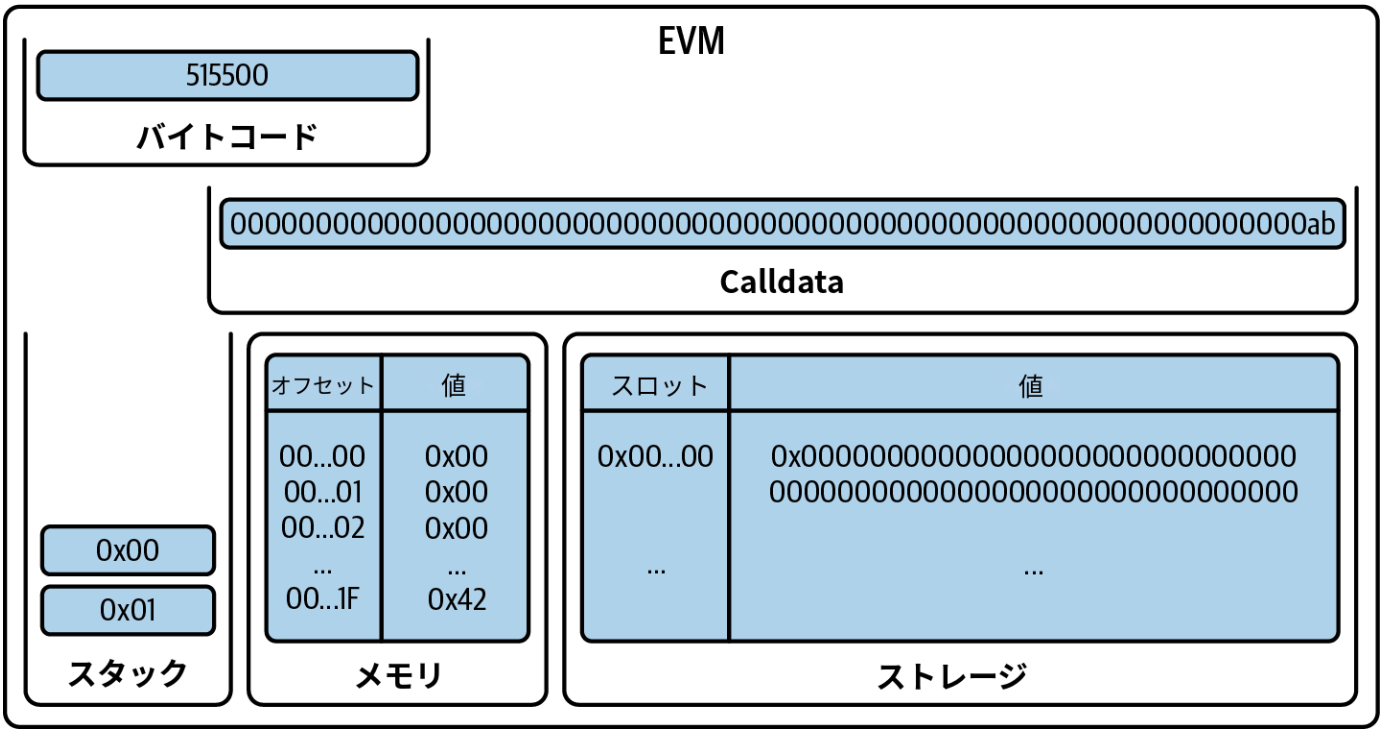


図14-17. PUSH0後のEVM

MLOADはスタックから1つの要素を取り出し、オフセットとして解釈し、そのオフセットから始まるメモリの32バイトを読み取って結果をスタックにプッシュします（図14-18）。

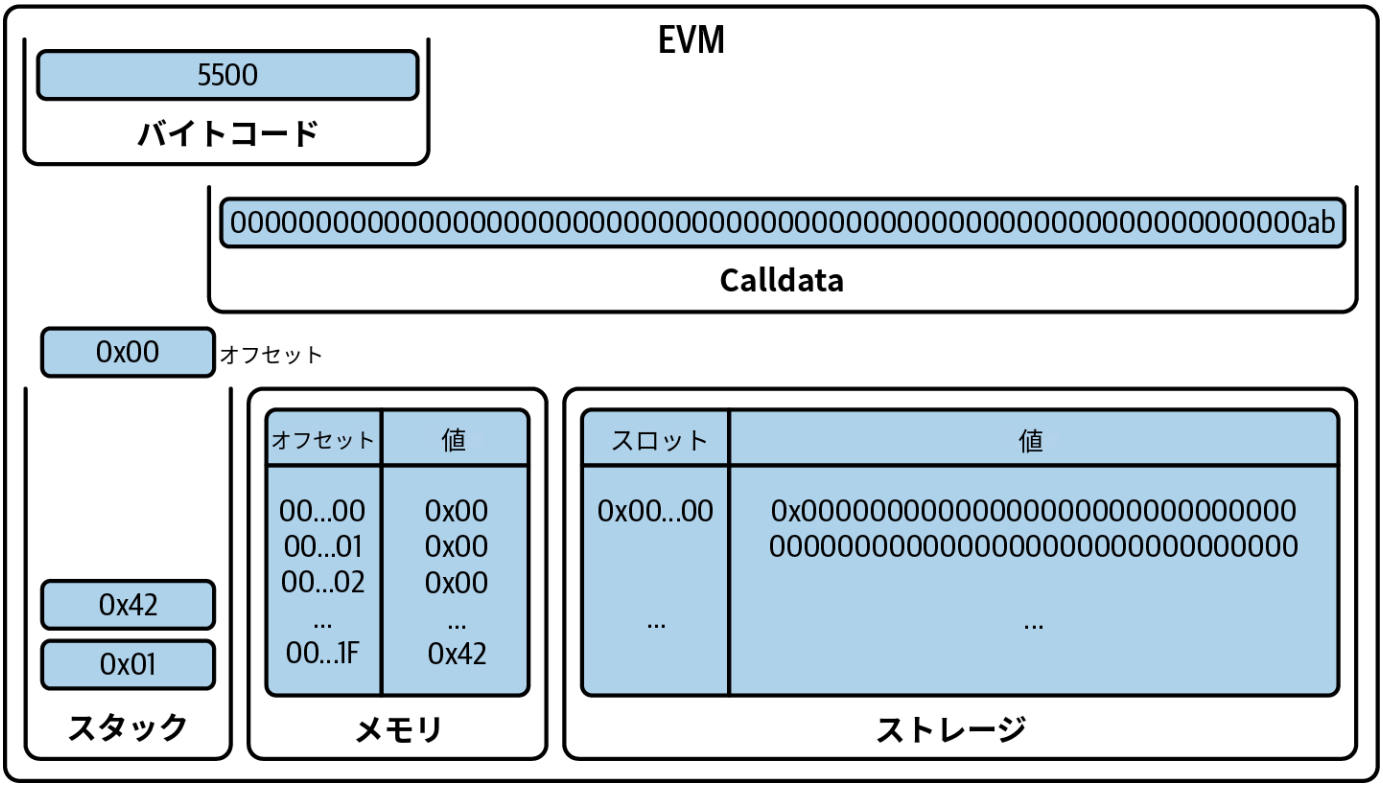


図14-18. MLOAD後のEVM

次に、SSTOREがスタックから2つのアイテムを取り出し、1つ目をスロット番号、2つ目をコントラクトストレージの該当スロットに保存する値として解釈します（図14-19）。

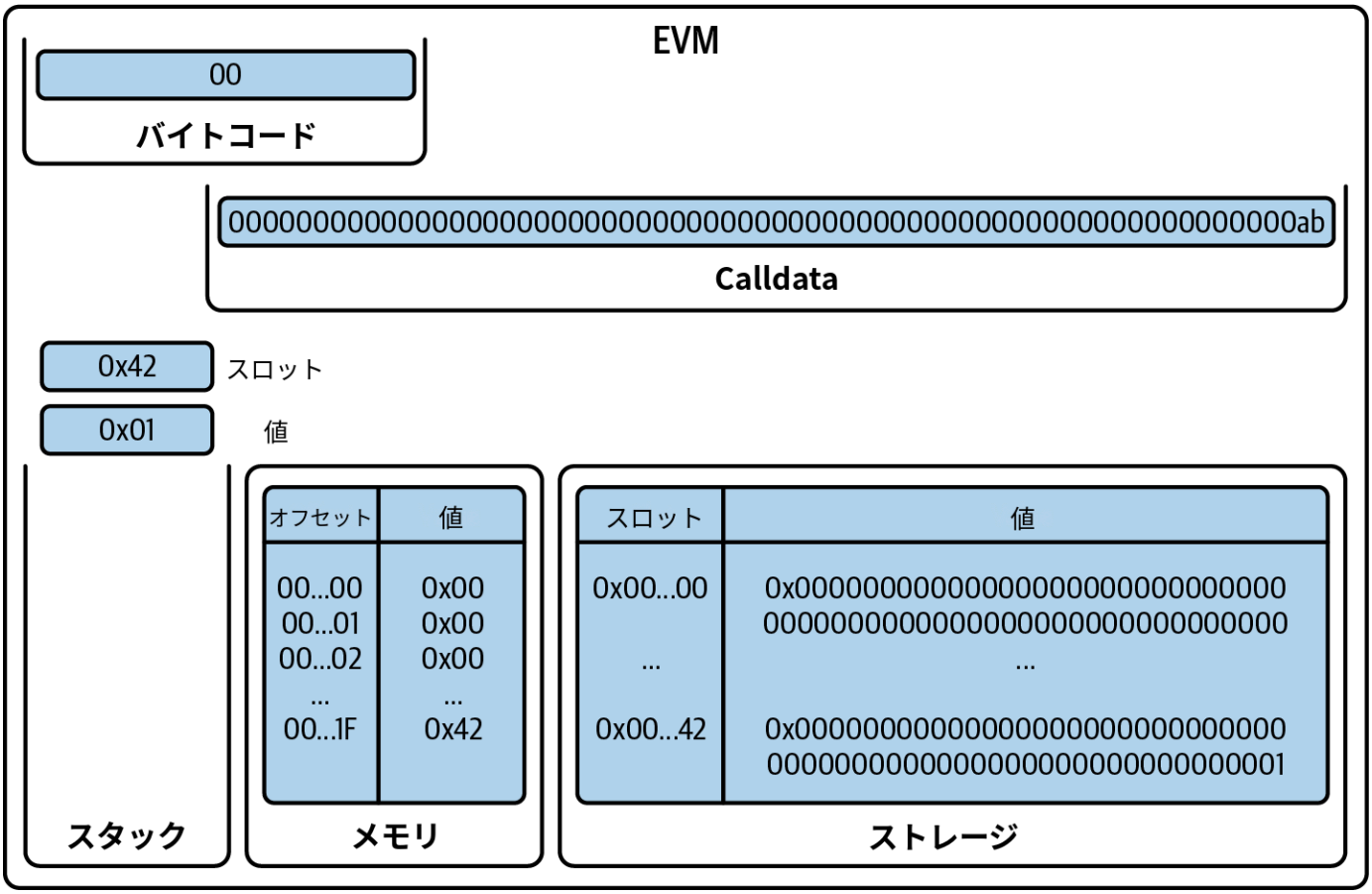


図14-19. SSTORE後のEVM

最後に、STOPオペコードが実行を停止し、図14-20に示すようにEVMは正常に終了します。

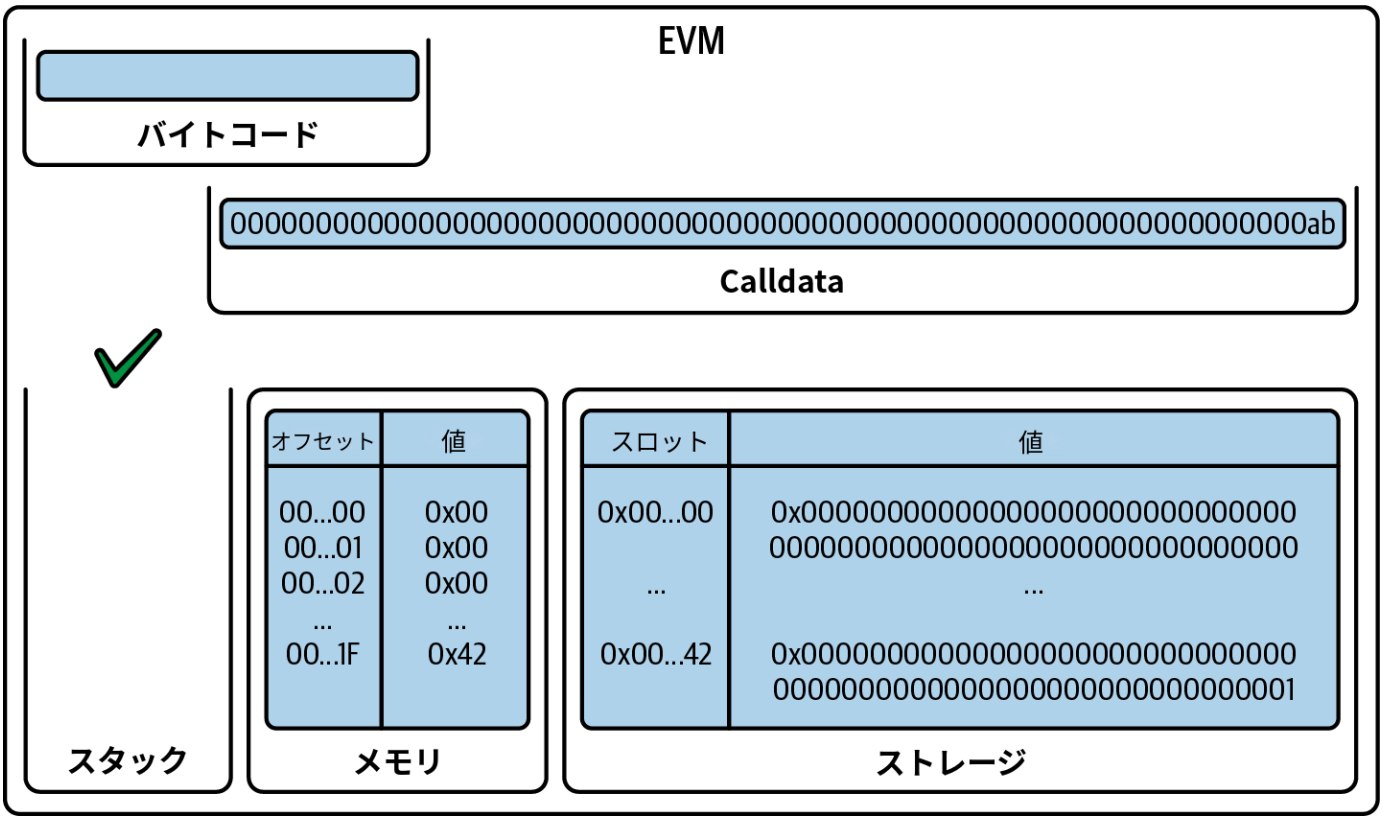


図14-20. STOP後のEVM

ヒント

前の例ではオペコードPUSH0を使用した。すべてのEVM互換ブロックチェーンがこのオペコードをサポートしているわけではないため、クロスチェーンコントラクトをデプロイする際には注意が必要である。EVM Diffは、EVM互換チェーン間のこうした微妙な違いを確認できる便利なウェブサイトである。

SolidityのEVMバイトコードへのコンパイル

第7章ではSolidityについてすでに解説しました。ここでは、SolidityがEVMで実行可能なEVMバイトコードにどのようにコンパイルされるかを見ていきます。

SolidityソースファイルをEVMバイトコードにコンパイルするには、いくつかの方法があります。第2章ではオンラインのRemixコンパイラを使用しました。本章では、コマンドラインで`solc`実行ファイルを使用します。コンピュータにSolidityをインストールするには、こちらの手順に従ってください。

オプションの一覧を表示するには、次のコマンドを実行します：

```
$ solc --help
```

`--opcodes` コマンドラインオプションを使用すると、簡単にSolidityソースファイルから生のオペコードストリームを生成することができます。このオペコードストリームではいくつかの情報が取り除かれます（`--asm` オプションは完全な情報を生成します）が、ここでの説明に関しては十分です。たとえば、Solidityファイル `Example.sol` をコンパイルし、出力されるオペコードを `BytecodeDir` という名前のディレクトリに送信するには、次のコマンドを実行します：

```
$ solc -o BytecodeDir --opcodes Example.sol
```

`--asm` を使用して、より人間が読みやすい出力を生成することもできます：

```
$ solc -o BytecodeDir --asm Example.sol
```

次のコマンドは、サンプルプログラムのバイトコードバイナリを生成します。

```
$ solc -o BytecodeDir --bin Example.sol
```

Solidityソースファイルに含まれる具体的なコントラクトごとに、出力されるオペコードは異なります。私たちの簡単なSolidityファイル `Example.sol` には、Exampleという名前のコントラクトが1つだけあります。

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.27;

contract Example {
  address contractOwner;

  function test() public {
    contractOwner = msg.sender;
  }
}
```

ご覧のとおり、このコントラクトは、コントラクトを実行した最後のアカウントのアドレスを永続的な状態変数として保持することだけを行います。

`BytecodeDir` ディレクトリを見るとオペコードのファイル `Example.opcode` があります。このファイルはExampleコントラクトのEVMオペコードの命令セットを含んでいます。テキストエディタで `Example.opcode` ファイルを開くと、次のように表示されます：

```
PUSH1 0x80 PUSH1 0x40 MSTORE CALLVALUE DUP1 ISZERO PUSH1 0xE JUMPI PUSH0 PUSH0 REVERT JUMPDEST
POP PUSH1 0xA9 DUP1 PUSH1 0x1A PUSH0 CODECOPY PUSH0 RETURN INVALID PUSH1 0x80 PUSH1 0x40 MSTORE
CALLVALUE DUP1 ISZERO PUSH1 0xE JUMPI PUSH0 PUSH0 REVERT JUMPDEST POP PUSH1 0x4 CALLDATASIZE LT
PUSH1 0x26 JUMPI PUSH0 CALLDATALOAD PUSH1 0xE0 SHR DUP1 PUSH4 0xF8A8FD6D EQ PUSH1 0x2A JUMPI
JUMPDEST PUSH0 PUSH0 REVERT JUMPDEST PUSH1 0x30 PUSH1 0x32 JUMP JUMPDEST STOP JUMPDEST CALLER
PUSH0 PUSH0 PUSH2 0x100 EXP DUP2 SLOAD DUP2 PUSH20 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
MUL NOT AND SWAP1 DUP4 PUSH200xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF AND MUL OR SWAP1 SSTORE
```

```
POP JUMP INVALID LOG2 PUSH5 0x6970667358 0x22 SLT KECCAK256 JUMPI 0xBB RETURNDATACOPY SWAP14
CALLVALUE 0xB3 0xB1 SMOD BLOBHASH STATICCALL MCOPY PUSH10 0x856E7132D8FEED4D83B6 0xB2 0xBE
PUSH30 0x8B43532C818BFD64736F6C634300081B00330000000000000000000000000000
```

`--asm`オプションを使用してこのファイルをコンパイルすると、`BytecodeDir`ディレクトリに`Example.evm`という名前のファイルが生成されます。このファイルには、EVMバイトコード命令のやや高レベルな記述と、わかりやすい注釈が含まれています：

```
/* "Example.sol":61:171  contract Example {... */
  mstore(0x40, 0x80)
  callvalue
  dup1
  iszero
  tag_1
  jumpi
  revert(0x00, 0x00)
tag_1:
  pop
  dataSize(sub_0)
  dup1
  dataOffset(sub_0)
  0x00
  codecopy
  0x00
  return
stop

sub_0: assembly {
        /* "Example.sol":61:171  contract Example {... */
      mstore(0x40, 0x80)
      callvalue
      dup1
      iszero
      tag_1
      jumpi
      revert(0x00, 0x00)
    tag_1:
      pop
      jumpi(tag_2, lt(calldatasize, 0x04))
      shr(0xe0, calldataload(0x00))
      dup1
      0xf8a8fd6d
      eq
      tag_3
      jumpi
    tag_2:
      revert(0x00, 0x00)
        /* "Example.sol":109:169  function test() public {... */
    tag_3:
```

```
      tag_4
      tag_5
      jump    // in
    tag_4:
      stop
    tag_5:
        /* "Example.sol":154:164  msg.sender */
      caller
        /* "Example.sol":138:151  contractOwner */
      0x00
      0x00
        /* "Example.sol":138:164  contractOwner = msg.sender */
      0x0100
      exp
      dup2
      sload
      dup2
      0xffffffffffffffffffffffffffffffffffffffff
      mul
      not
      and
      swap1
      dup4
      0xffffffffffffffffffffffffffffffffffffffff
      and
      mul
      or
      swap1
      sstore
      pop
        /* "Example.sol":109:169  function test() public {... */
      jump    // out
    auxdata:
0xa2646970667358221220 57bb3e9e34b3b10749fa5e69856e7132d8feed4d83b6b2be7d8b43532c818bfd64736f6c6
34300081b0033
}
```

`--bin`オプションは、機械可読な16進バイトコードを生成します：

```
608060405234801560 0e575f5ffd5b5060a980601a5f395ff3fe608060405234801560 0e575f5ffd5b5060043610602
6575f3560e01c8063f8a8fd6d14602a575b5f5ffd5b60306032565b005b335f5f6101000a81548173ffffffffffffff
ffffffffffffffffffffffff021916908373ffffffffffffffffffffffffffffffffffffffff16021790555056fea
2646970667358221220 57bb3e9e34b3b10749fa5e69856e7132d8feed4d83b6b2be7d8b43532c818bfd64736f6c6343
00081b0033
```

「EVM命令セット（バイトコード演算）」に記載されているオペコードの一覧を使用して詳細を調べることができます。しかし、作業量がかなり多くなってしまうので、最初の4つの命令を調べるところから始めてみましょう：

```
PUSH1 0x80 PUSH1 0x40 MSTORE CALLVALUE
```

ここでは`PUSH1`の後に生のバイトの`0x80`が続いています。このEVM命令は、プログラムコード内のオペコードに続く1バイトを（リテラル値として）取り出し、スタックにプッシュします。次のように、32バイトまでのサイズの値をスタックにプッシュすることができます：

```
PUSH32 0x436f6e6772617475617461696f6e732120536f6f6e20746f206d617374657221
```

`Example.opcode`の2つ目の`PUSH1`は、`0x40`をスタックの先頭にプッシュします（すでに存在する`0x80`を1つ下のスロットに押し込みます）。

次は`MSTORE`で、これはEVMのメモリに値を保存するメモリへの記録演算です。`MSTORE`は2つの引数を取ります。ほとんどのEVM上の演算と同様に、引数をスタックから取得します。各引数について、スタックは「ポップ」されます。すなわち、スタックの一番上の値が取り出され、スタック内の他のすべての値の位置が1つ上にシフトされます。`MSTORE`の最初の引数は、保存する値を格納するメモリのアドレスです。このプログラムではスタックの先頭に`0x40`があるため、`0x40`はスタックから削除されてメモリアドレスとして使用されます。2番目の引数は保存される値で、ここでは`0x80`です。`MSTORE`演算が実行された後、スタックは再び空になりますが、メモリ上の`0x40`に`0x80`（10進数で128）の値があります。

次のオペコードは`CALLVALUE`です。`CALLVALUE`は、実行を開始したメッセージコールと一緒に送信されたイーサの量（weiで表される）をスタックの一番上にプッシュする環境オペコードです。

コードが影響を与える低いレベルの状態変化を完全に理解できるまで、このようにコードを一歩一歩解析し続けることもできますが、おそらく、現段階ではあまり理解の助けにはならないでしょう。この章の後半で、もう一度こういったコードの観察に戻ることにします。

コントラクトのデプロイを行うコード

イーサリアムプラットフォームでは、コントラクトを作成しデプロイする際に使用されるコードとコントラクト自体のコードの間には重要で小さな違いが存在します。新しいコントラクトを作成するには、特別なトランザクションが必要です。このトランザクションは`to`フィールドを空（null）に設定し、その`data`フィールドをコントラクトの生成コード（initiation code）に設定します。そのようなコントラクト作成トランザクションが処理されるときは、新しいコントラクトアカウントのコードとしてトランザクションの`data`フィールドのコードがそのまま登録されるわけではありません。その代わりに、プログラムコードROMにロードされたトランザクションの`data`フィールドのコードを使用してEVMがインスタンス化され、そのデプロイコードの実行結果が新しいコントラクトアカウントのコードとして登録されます。これは、デプロイ時にイーサリアムのワールドステートを利用して新しいコントラクトをプログラム的に初期化した

り、コントラクトのストレージに値を設定したり、イーサを送信したり、新しいコントラクトをさらに作成したりできるようにするためです。

コマンドラインで`solc`を使用するなど、コントラクトをオフラインでコンパイルする場合には、*デプロイメントバイトコード*（deployment bytecode）または*ランタイムバイトコード*（runtime bytecode）を取得できます。

デプロイメントバイトコードは、新しいコントラクトの初期化のあらゆる場面で利用されます。デプロイメントバイトコードは、トランザクションが新しいコントラクトを呼び出す時に最終的に実行される実際のバイトコード（つまり、ランタイムバイトコード）を含む上、コントラクトのコンストラクタに基づいてすべてを初期化するコードも含みます。

その一方で、ランタイムバイトコードは、新しいコントラクトが呼び出される時に最終的に実行されるバイトコードそのものです。それ以上のものではありません。ランタイムバイトコードは、デプロイ中にコントラクトを初期化するのに必要なバイトコードを含まないのです。

例として、前の章で作成した簡単な`Faucet.sol`コントラクトを見ていきましょう。

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity 0.8.27;

contract Faucet {
  // 要求があれば誰にでもイーサを提供
  function withdraw(uint256 _withdrawAmount, address payable _to) public {
    // 引き出し額を制限
    require(_withdrawAmount <= 1000000000000);
    // 要求したアドレスにその金額を送信
    _to.transfer(_withdrawAmount);
  }

  // イーサを受け取る関数。msg.dataは空でなければならない
  receive() external payable {}

  // msg.dataが空でない場合に呼び出されるフォールバック関数
  fallback() external payable {}
}
```

デプロイメントバイトコードを取得するには、`solc --bin Faucet.sol`を実行します。ランタイムバイトコードのみが必要な場合は、`solc --bin-runtime Faucet.sol`を実行します。これらのコマンドの出力を比較すると、ランタイムバイトコードがデプロイメントバイトコードのサブセットであることがわかります。つまり、ランタイムバイトコードはデプロイメントバイトコード内に完全に含まれています。

オンチェーンでコントラクトをデプロイするCREATE対CREATE2

オンチェーンで新しいコントラクトをデプロイできるオペコードは`CREATE`と`CREATE2`の2つだけです。両者の主な違いは、新しく作成されるコントラクトのアドレスの決定方法にあります。`CREATE`では、アドレスは次のように計算されます：

```
address = keccak256[rlp(sender_address ++ sender_nonce)][12:]
```

送信者アドレスとそのノンスをRLPエンコードしたものをKeccak-256でハッシュ化し、その右端20バイトとなります。

`CREATE2`は、生成されるアドレスが送信者の状態（つまりノンス）に依存しないコントラクトを開発者が作成できるように、2019年のConstantinopleハードフォークで追加されました。`CREATE`オペコードとまったく同じように動作しますが、アドレスは次のように計算されます：

```
address = keccak256(0xff ++ sender_address ++ salt ++ keccak256(init_code))[12:]
```

ここで：

- `init_code`は新しいコントラクトのデプロイメントバイトコードです。
- `salt`は32バイトの値（スタックから取得）です。

バイトコードの逆アセンブル

高級言語としてのSolidityがEVM上でどのように動作するかを理解する上で、EVMのバイトコードを逆アセンブルすることは良い方法です。使用できる逆アセンブラはいくつかあります。

- **Ethersplay**は、逆アセンブラであるBinary Ninja用のEVMプラグインです。なお、プラグインを使用するにはBinary Ninjaの完全版を購入する必要があります。
- **Heimdall**は、バイトコード分析と未検証コントラクトからの情報抽出を専門とする高度なEVMスマートコントラクトツールキットです。

このセクションでは、Heimdallを使用して図14-21を作成します。`Faucet.sol`のランタイムバイトコードを取得した後、Heimdallに入力してEVM命令の中身を確認します。

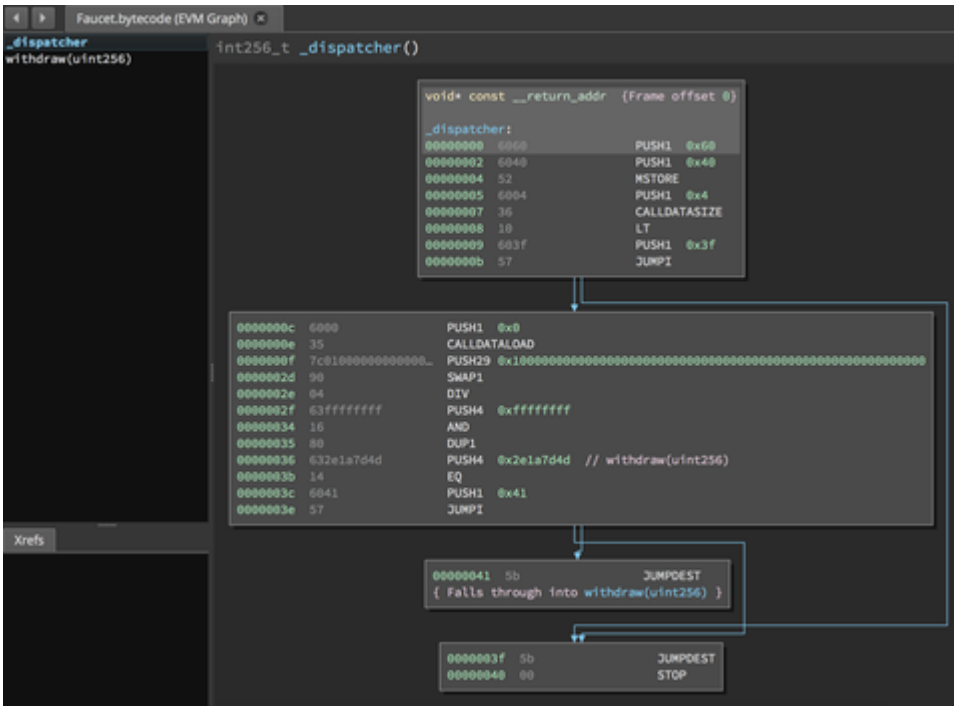


図14-21. Faucetランタイムバイトコードの逆アセンブル

Heimdallのインストール

まず、コンピュータにRustがインストールされていることを確認してください。インストールされていない場合は、次のコマンドを実行します：

```
$ curl https://sh.rustup.rs -sSf | sh
```

次に、以下の2つのコマンドを順番に実行します：

```
$ curl -L http://get.heimdall.rs | bash
$ bifrost
```

これでHeimdallが正しくインストールされているはずです。次のコマンドで確認できます：

```
$ heimdall --version
```

次のような表示が出れば成功です：

```
$ heimdall --version
heimdall 0.8.4
```

注記

Heimdallのインストール方法に関する最新情報については、GitHubリポジトリにある公式ドキュメントを参照のこと。

Heimdallによるバイトコードの逆アセンブル

Heimdallを正しくインストールしたので、図14-21で見たのと同じグラフを生成してみましょう。`Faucet.sol`コントラクトのランタイムバイトコードを使って、次のコマンドを実行します：

```
$ heimdall cfg <ここにランタイムバイトコードを挿入>
```

実際のコマンドは次のようになります：

```
$ heimdall cfg 608060405260043610610…
```

実行すると`output`という新しいフォルダが作成されます。その中に移動し、さらに`local`フォルダに移動すると、`cfg.dot`ファイルがあるはずです：

```
$ cd output
$ cd local
$ ls # ファイルが表示されるはずです
```

`.dot`ファイルなので、正しく表示するには専用のプログラムが必要です。ここでは、`.dot`ファイルの内容を貼り付けるとグラフを生成してくれるウェブサイトを使用します。

まず、`.dot`ファイルの内容をコピーします：

```
$ cat cfg.dot
```

このコマンドでファイルの全内容が画面に出力されます。それをコピーして、*制御フローグラフ（CFG）オンラインジェネレータ*を開き、図14-22に示すようにウェブページの左側に貼り付けます。

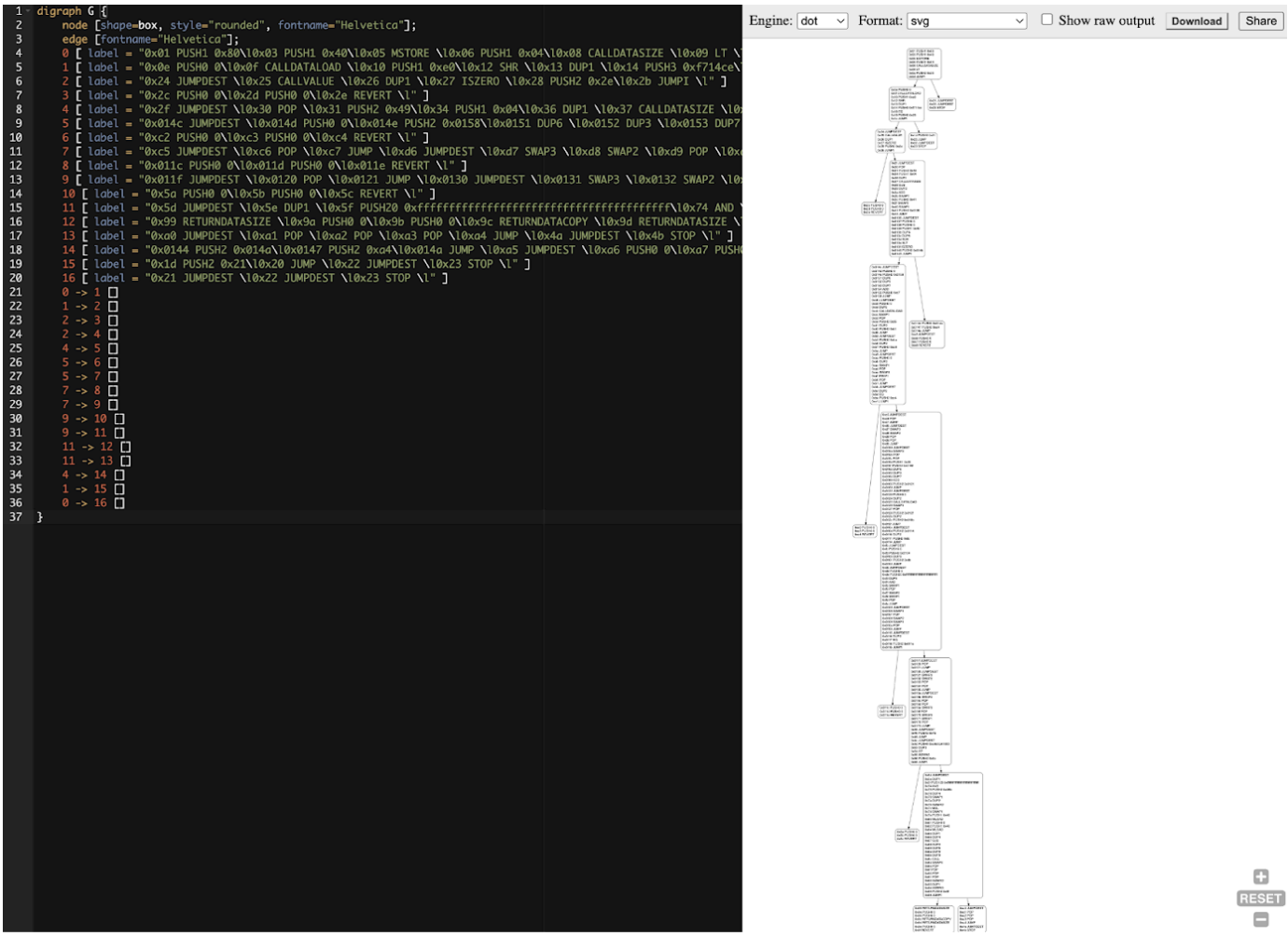


図14-22. Faucet.solコントラクトの制御フローグラフ（CFG）

図14-23は`Faucet.sol`コントラクトの初期バイトコードを示しています。ご覧のとおり、前の`Example.sol`コントラクトと同じパターン`PUSH1 0x80 PUSH1 0x40 MSTORE`で始まります。

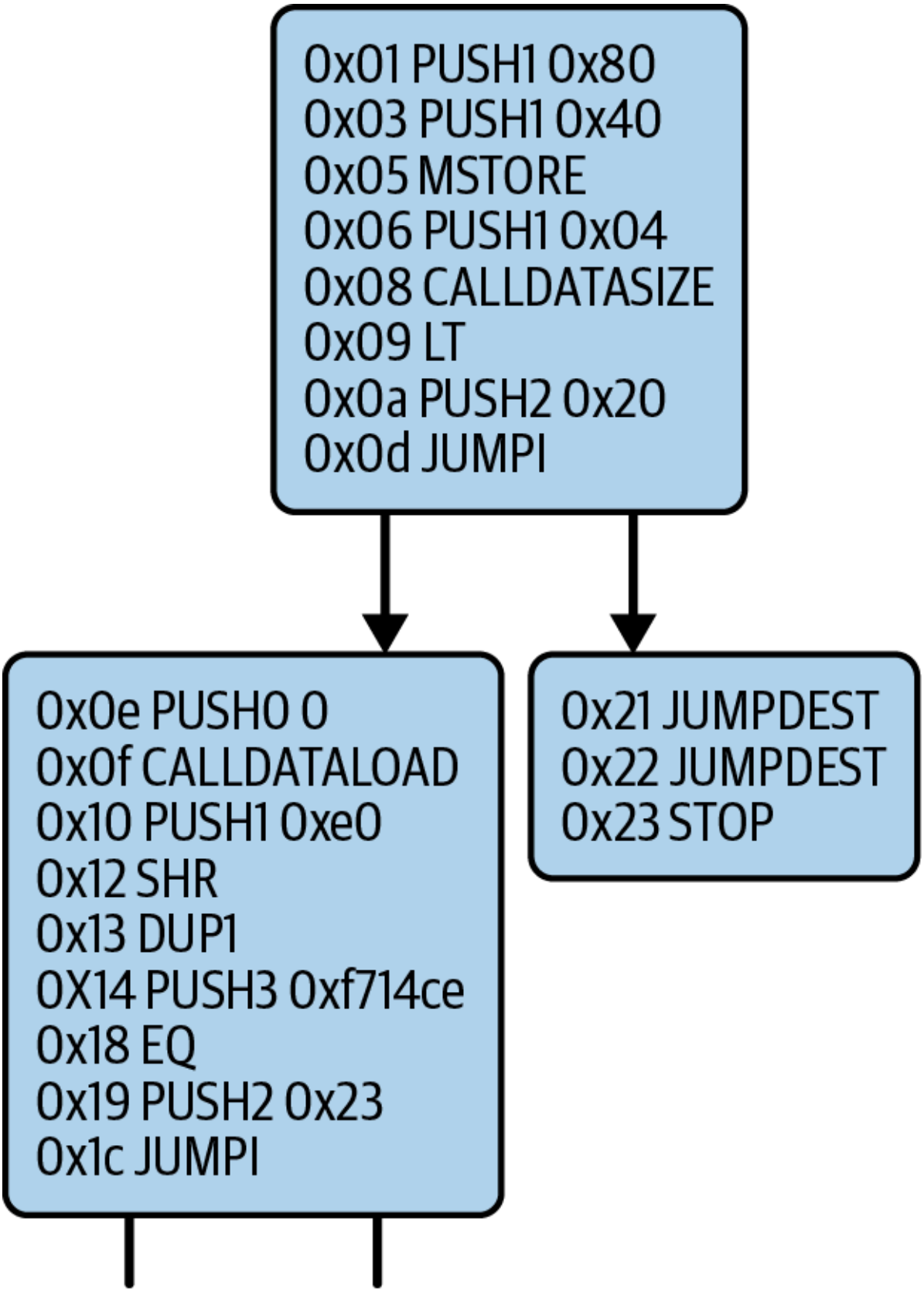


図14-23. CFGグラフの最初の部分の拡大

ABI準拠のスマートコントラクト（すべてのコントラクトが準拠していると想定してよいです）にトランザクションを送信すると、トランザクションは最初にそのスマートコントラクトのディスパッチャ（dispatcher）と対話します。ディスパッチャは、トランザクションの`data`フィールドを読み込み、関係する部分を適切な関数に送信します。逆アセンブルした`Faucet.sol`ランタイ

ムバイトコードの冒頭部分に、ディスパッチャの例があります。おなじみの`MSTORE`命令のあとに、次の命令セットが表示されます：

```
PUSH1 0x04
CALLDATASIZE
LT
PUSH2 0x0020
JUMPI
```

これまで見てきたように、`PUSH1 0x04`は`0x04`をスタックの先頭に置きます。スタックの先頭以外は空です。`CALLDATASIZE`は、トランザクションとともに送信されたデータのバイト数を取得し（送信されたデータをコールデータ〔calldata〕と呼びます）、その数値をスタックにプッシュします。これらの操作が実行されると、スタックは次のようになります：

```
スタック
<トランザクションからのコールデータの長さ>
0x4
```

次の命令は`LT`で「Less Than」の省略形です。`LT`命令は、スタックの一番上のアイテムがスタックの次のアイテムよりも小さいかどうかを調べます。このケースでは、`CALLDATASIZE`の結果が4バイトより小さいかどうかを調べています。

トランザクションのコールデータが少なくとも4バイトであることをEVMが確認するのは、*関数識別子*の動作が関係しています。各関数は、そのKeccak-256ハッシュの最初の4バイトで識別されます。関数の名前と、関数が引数として取るものをKeccak-256ハッシュにかけることで関数の識別子を推測できます。今回の場合：

```
keccak256("withdraw(uint256,address)") = 0x00f714ce...
```

したがって、関数`withdraw(uint256,address)`の関数識別子は、ハッシュ関数を通した結果の最初の4バイトの`0x00f714ce`です。関数識別子の長さは常に4バイトで、コントラクトに送信されるトランザクションのデータフィールド全体が4バイト未満の場合、フォールバック関数（fallback function）が定義されていない限り、トランザクションがやり取り可能な関数は存在しないことになります。今回は`Faucet.sol`にフォールバック関数を実装したので、コールデータの長さが4バイト未満の場合、EVMはこの関数にジャンプします。

`LT`はスタックの上から2つの値を取り出し、トランザクションの`data`フィールドが4バイト未満の場合は1をプッシュします。それ以外の場合は0をプッシュします。この例では、コントラクトに送信されたトランザクションの`data`フィールドが4バイト未満であると仮定します。

`PUSH2 0x0020`命令は、スタックにバイト値`0x0020`をプッシュします。この命令の後、スタックは次のようになります：

```
スタック
0x0020
0x1
```

次の命令は`JUMPI`で、これは「jump if」を略したものです。次のように動作します：

```
jumpi(label, cond) // "cond"が真ならば"label"の位置にジャンプする
```

　このケースでは`label`は`0x0020`で、スマートコントラクトのフォールバック関数が存在する場所です。`cond`の引数は1で、これは先ほどの`LT`命令の結果です。この一連の流れを説明すると、トランザクションデータが4バイト未満の場合、コントラクトはフォールバック関数にジャンプするということです。

`0x20`では、2つの`JUMPDEST`命令の後、`STOP`命令だけが続きます。フォールバック関数を宣言しましたが、中身は空のままだからです。図14-24に示すように、フォールバック関数を実装していなかった場合、コントラクトは代わりに例外を投げます。

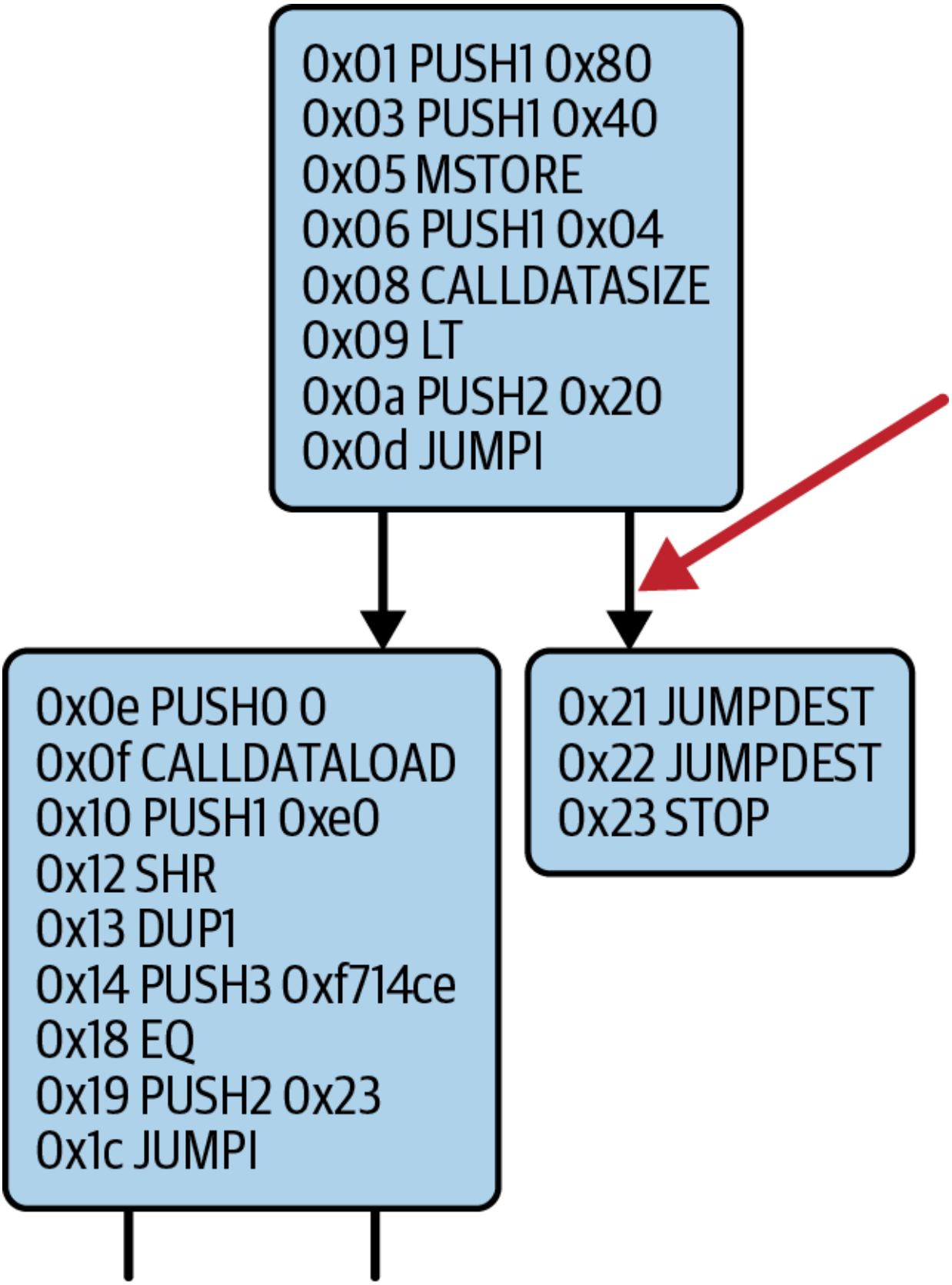


図14-24. フォールバック関数につながるJUMPI命令

注記

Heimdallはバイトコードをオフセット0x01から始まるものとして表現するが、EVMは実際にはオフセット0x00から解釈する。前の例では、JUMPI命令は条件が真の場合にオフセッ

> トOx20にジャンプするようEVMに指示しているが、グラフ上ではオフセットOx20がOx21として表示されている。目安として、EVMの各オフセットに1を加えればグラフ上の位置を見つけることができる。

ディスパッチャの中央ブロックを調べてみましょう。4バイトより長いコールデータを受け取った場合、`JUMPI`命令はフォールバック関数へのジャンプを実行しません。その代わりに、コードの実行は次の命令に進みます：

```
PUSH0 0x0
CALLDATALOAD
PUSH1 0xe0
SHR
DUP1
PUSH3 0xf714ce
EQ
PUSH2 0X23
JUMPI
```

`PUSH0`は0をスタックにプッシュし、スタックは再び空の状態になります。`CALLDATALOAD`は、スマートコントラクトに送信されたコールデータ内のインデックスを引数として受け取り、そのインデックスから32バイトを読み込みます：

```
calldataload(p) // バイト位置pから始まる32バイトのコールデータを読み出す
```

`PUSH0`コマンドから渡されたインデックスが0だったため、`CALLDATALOAD`はバイト0から始まるコールデータの32バイトを読み取り、（元の`0x0`をポップした後）スタックのトップにプッシュします。`PUSH1 0xe0`命令の後、スタックは次のようになります：

```
スタック
0xe0
<バイト0から始まるコールデータの32バイト>
```

`SHR`はスタック上の32バイト値の要素に対して`0xe0`ビット（224ビット、28バイト）の論理右シフトを実行します。コールデータを28バイト右にシフトすることで、コールデータの最初の4バイトを分離します。実際、右にシフトすると、右端からはみ出したビットは破棄され、左端には0が補填されます。コールデータの最初の4バイトは、トリガーしたい関数の関数識別子を表すことを忘れないでください。

論理ビットシフトの例

例を使ってよりよく理解できます。スタックが次のようになっているとしましょう：

```
スタック
0x1234567890 // 5バイトの要素
```

最初の2バイト（つまり`0x1234`）のみを取得したいとします。EVMオペコードのみを使用してこれを達成するには、次のようにできます：

```
PUSH1 0x18 // これは16進数で24を表し、24ビット = 3バイト
SHR
```

実際、スタックのアイテムを3バイト右にシフトすると（各バイトは2桁の16進数で表されることを忘れないでください）、次のアイテムが得られます：

```
0x0000001234 | 4567890
```

`4567890`の部分は破棄され、残るのは次のとおりです：

```
スタック
0x1234
```

先頭のゼロ埋めは重要でないため省略します。

新しいスタックは次のとおりです：

```
スタック
<データで送信された関数識別子>
```

次の命令は`DUP1`で、スタックの最初のアイテムである関数識別子を複製します。スタックは次のようになります：

```
スタック
<データで送信された関数識別子>
<データで送信された関数識別子>
```

次に`PUSH3`命令があり、データ`0xf714ce`が続きます。このオペコードは単にデータをスタックにプッシュします。このオペコードの後、スタックは次のようになります：

```
スタック
0xf714ce
<データで送信された関数識別子>
<データで送信された関数識別子>
```

さて、`0xf714ce`は見覚えがありますか？`withdraw(uint256,address)`関数の関数識別子が何であったか覚えていますか？`0x00f714ce`です…先頭のゼロ埋めを省略すれば同じ数値であることがわ

かります。

次の命令`EQ`はスタックの上位2つのアイテムを取り出し、それらを比較します。これがディスパッチャの主な働きです。トランザクションの`msg.data`フィールドで送信された関数識別子が`withdraw(uint256,address)`のものと一致するかどうかを比較します。両者が等しい場合、`EQ`は1をスタックにプッシュし、最終的にwithdraw関数にジャンプするために使用されます。そうでなければ、`EQ`は0をスタックにプッシュします。

コントラクトに実際に送られたトランザクションが`withdraw(uint256,address)`の関数識別子で始まっている場合、スタックは次のようになります：

```
スタック
1
<データで送信された関数識別子>（0x00f714ceであることが判明）
```

次に、`PUSH2 0x23`があります。これは、コントラクト内で`withdraw(uint256,address)`関数が存在しているアドレスです。この命令の実行後、スタックは次のようになります：

```
スタック
0x23
1
<msg.dataで送信された関数識別子>
```

次に`JUMPI`命令が実行され、スタック上の上位2つの要素が引数として再び受け入れられます。この場合、`JUMPI(0x23, 1)`によりEVMに`withdraw(uint256,address)`関数の位置へジャンプするように命令し、関数のコードの実行へと進めることができます。

チューリング完全性とガス

すでに触れたように、簡単に言えば、任意のプログラムを実行できるならば、システムやプログラミング言語は*チューリング完全*です。しかし、この能力には非常に重要な注意が伴います。つまり、プログラムの中には実行に永遠の時間がかかるものがあるということです。重要なことは、プログラムを見るだけでは永遠に実行されるかどうかはわからないということです。実際にプログラムを実行して、終了を待って確認しなければなりません。当然、永遠に実行されるならば、確認するために永遠に待たなければいけません。これは*停止性問題*と呼ばれ、対処されなければイーサリアムにとって大きな問題になります。

停止性問題のため、イーサリアムワールドコンピュータは、決して停止しないプログラムを実行するよう要求される可能性があります。偶然そうなるかもしれませんし、悪意によるものかもしれません。イーサリアムがスケジューラなしのシングルスレッドマシンのように動作することはすでに説明しました。したがって、無限ループに陥った場合、イーサリアムは利用できなくなってしまいます。

しかし、ガスを用いることで解決策があります。あらかじめ指定された最大量の計算が実行されたあとも実行が終了していなければ、プログラムの実行はEVMによって停止されます。これにより、EVMは準チューリング完全マシンになります。つまり、入力されたプログラムを実行できますが、プログラムが特定の計算量内で終了する場合のみです。この制限はイーサリアムでは固定されていません。ガスを支払うことで最大値（*ブロックガス上限*と呼ばれる）まで増やすことができ、時間の経過とともにその最大値を増やすことに全員が同意できます。それでも、常に制限が設けられており、実行中に大量にガスを消費するトランザクションは停止されます。

以下のセクションでは、ガスに焦点を当て、その仕組みを詳しく見ていきます。

ガスとは何か？

ガスはイーサリアムにおける基本単位で、イーサリアムのブロックチェーンでアクションを実行するために必要な計算リソースとストレージリソースを測定するためのものです。ビットコインではトランザクション手数料はキロバイト単位のトランザクションサイズのみを考慮しますが、イーサリアムではトランザクションとスマートコントラクトコードの実行によって処理されるすべての計算ステップを考慮する必要があります。

トランザクションまたはコントラクトによって実行される各操作は、一定量のガスを必要とします。イーサリアムのイエローペーパーからいくつかの例を挙げておきます。

- 2つの数値を加算すると3ガスがかかる
- Keccak-256ハッシュの計算には、30ガス+ハッシュされる256ビットのデータごとに6ガスが必要
- トランザクションの送信には21,000ガスがかかる

ガスはイーサリアムの極めて重要な構成要素であり、イーサの（変動する）価格とバリデータが行う作業への報酬との間の緩衝として、そしてDoS攻撃に対する防御策として二重の役割を担っています。偶発的または悪意のある無限ループやネットワーク内のその他の計算上の浪費を防ぐために、各トランザクションの開始者は、支払う予定の計算量に上限を設定する必要があります。こうすることでガスシステムは攻撃者が「スパム」トランザクションを送信することを阻止しています。これが有効なのは、攻撃者は消費する計算資源、帯域幅、およびストレージ資源に比例してガスを支払う必要があるためです。

実行中のガス会計

トランザクションの完了にEVMが必要な場合、最初にトランザクションのガス上限で指定された量のガス供給が行われます。実行されるすべてのオペコードにはガスの費用が定義されているため、EVMのガス供給はEVMがプログラムを実行していくにつれて減少していきます。各操作の前に、EVMは操作の実行に十分なガスがあることを確認します。十分なガスがない場合、実行は停止され、トランザクションは元に戻されます。

ガスを使い切ることなくEVMが実行の終了に正常に到達した場合、使用されたガス費用はトランザクション手数料としてバリデータに支払われ、トランザクションで指定されたガス価格に基づいてイーサに変換されます：

```
バリデータ報酬 = ガス費用 × ガス価格
```

ガス供給に残っているガスは送信者に払い戻され、トランザクションで指定されたガス価格に基づいて再びイーサに変換されます：

```
残りガス = ガス上限 − ガス費用
返金イーサ = 残りガス × ガス価格
```

実行中にトランザクションが「ガス切れ」になった場合、操作は直ちに終了し、OOG例外が発生します。トランザクションは元に戻され、状態へのすべての変更がロールバックされます。トランザクションは失敗しましたが、バリデータはその時点までの計算作業をすでに実行しており、それに対する補償を受ける必要があるため、送信者にはトランザクション手数料が請求されます。

ガス会計の考慮事項

イーサリアムブロックチェーンを攻撃から守るために、EVMで実行できるさまざまな操作の相対的なガス費用は慎重に選ばれています。より計算量のかかる操作の実行にはより多くのガス費用がかかります。たとえば、`SHA3`関数の実行（30ガス）は、`ADD`命令の実行（3ガス）の10倍の費用がかかります。さらに重要なことに、`EXP`などの一部の操作では、オペランドのサイズに基づいて追加の支払いが必要です。また、EVMメモリを使用したり、コントラクトのオンチェーンストレージにデータを格納したりするためのガス費用もあります。

現実世界のコストとガス費用を一致させることの重要性は、攻撃者がコストの不一致を発見して悪用した2016年に実証されました。その攻撃では、非常に計算コストの高いトランザクションが生成され、イーサリアムメインネットをほぼ停止させました。この不一致は、相対的なガス費用を調整したハードフォーク（コードネーム「Tangerine Whistle」）によって解決されました。

イーサリアムの将来におけるガス会計

ガスの計量は、イーサリアムがネットワーク内のトランザクション負荷全体を処理するうえで極めて重要であり、今後もそうあり続けます。ガス費用が特定の種類の動作に対する重要なインセンティブであることを理解することが非常に重要です。将来、各オペコードが消費するガス量が変更される可能性があります。

たとえば、EIP-4844 blobトランザクションを導入したCancunアップグレードの前は、すべてのL2がトランザクションのコールデータの一部としてイーサリアム上にデータを記録していました。そのデータはイーサリアムのすべてのノードに永久に保存されます。L2がblobトランザクシ

ョンを通じてイーサリアム上にデータを記録するより良い方法を持つようになった今、将来的にはロールアップがblobトランザクションを使用することを奨励し、すべてのデータを永久に保存するノードの負担を軽減するために、コールデータが現在よりもさらに高価になる可能性があります。

ガス費用対ガス価格

*ガス費用*はEVM上の計算と利用されたストレージの計測単位であり、ガス自体はイーサを尺度とする*価格*を持っています。トランザクションを実行する際、送信者はガスの各単位について支払うガス価格を（イーサで）指定し、市場がイーサの価格と計算操作のコスト（ガスで測定）の関係を決定できるようにします：

```
トランザクション手数料 = 使用された総ガス × 支払われたガス価格（イーサで）
```

新しいブロックを構築するとき、イーサリアムネットワーク上のバリデータは他より高いガス価格を提示しているものを選択することで、未処理トランザクションの中から処理すべきものを選ぶことができます。したがって、より高いガス価格を提供することは、バリデータにトランザクションを含めるよう促すことになり、より早く承認を得ることができます。

実際には、トランザクションの送信者は、使用予定のガス量以上のガス上限を設定します。もしガス上限が消費されたガスよりも高く設定されている場合は、送信者は消費総量を超えた分を返金として受け取ることができます。これは、バリデータには実際に実行した分のみが補償されるからです。

ガス費用とガス価格を明確に区別することは重要です。要点をまとめておきます。

- *ガス費用*は、特定の操作を実行するために必要なガスの単位数です。
- *ガス価格*は、トランザクションをイーサリアムネットワークに送信する際に、ガスの単位ごとに支払う意思のあるイーサの量です。

ヒント

ガスには価格があるが、「所有」することも「消費」することもできない。ガスは、どれだけの計算作業が行われているかの総数として、EVM内にのみ存在する。送信者にはイーサでトランザクション手数料が請求され、EVM会計のためにガスに変換され、その後バリデータに支払われるトランザクション手数料としてイーサに戻される。

負のガス費用

イーサリアムは、コントラクト実行中に使用されたガスの一部を払い戻すことで、使用済みストレージ変数の削除を奨励しています。EVMで負のガス費用を持つ操作は1つだけです：ストレージアドレスを非ゼロ値からゼロに変更すること（`SSTORE[x] = 0`）は返金に値します。返金さ

れるガスの量は固定されておらず、この操作の前後のストレージスロットの値に依存します。返金メカニズムの悪用を避けるため、トランザクションの最大返金額は総ガス費用の5分の1（切り捨て）に設定されています。

過去には、負のガス費用を持つ別の操作がありました：`SELFDESTRUCT`。コントラクトの削除（`SELFDESTRUCT`を通じて）は24,000ガスを返金しました。現在、`SELFDESTRUCT`オペコードは非推奨であり、もう使用しないことが推奨されています。

> **ヒント**
>
> ガス返金はトランザクションの最後に適用される。したがって、トランザクションを最後まで実行するのに十分なガスがない場合、失敗し、返金は行われない。

ブロックガス上限

ブロックガス上限は、ブロック内のすべてのトランザクションによって消費されるガスの最大量であり、ブロックに収まるトランザクションの数を制限します。

たとえば、ガス上限が30,000、30,000、40,000、50,000、50,000に設定された5つのトランザクションがあるとします。ブロックガス上限が180,000の場合、それらのトランザクションのうちの4つはブロックに収まることができ、5番目のトランザクションは将来のブロックを待たなければなりません。前述のように、バリデータはどのトランザクションをブロックに含めるかを決定します。異なるバリデータは、異なる組み合わせを選択する可能性が高くなります。というのも、バリデータはトランザクションをネットワークから異なる順番で受け取るのが一般的だからです。

バリデータが、現在のブロックガス上限よりも多くのガスを必要とするトランザクションを含めようとすると、ブロックはネットワークによって拒否されます。ほとんどのイーサリアムクライアントは、「トランザクションがブロックガス上限を超えています」という警告を出すことで、そのようなトランザクションの発行を停止します。Etherscanによると、執筆時点（2025年6月）のイーサリアムメインネットのブロックガス上限は3,600万ガスであり、約1,428の基本的な（つまり、ETH転送）トランザクション（それぞれ21,000ガスを消費）がブロックに収まることを意味します。

誰がブロックガス上限を選択するか？

2021年8月5日のEIP-1559の導入前は、マイナー（当時イーサリアムはPoWベースのコンセンサスアルゴリズムを使用していました）には、ブロックガス上限に投票できる組み込みメカニズムがあり、後続のブロックで容量を増減できました。ブロックのマイナーは、どちらの方向にも1/1024（0.0976%）の係数でブロックガス上限を調整するために投票できました。この結果、マイナーのハッシュパワーに従って、ネットワークのニーズに基づいて調整可能なブロックサイズになりました。

現在、バリデータは*ガスターゲット*（つまり、ブロックが平均的に消費すべきガス量）に投票し、ガス上限はターゲットの2倍として定義されます。各バリデータが従う必要がある1/1024の調整係数は以前と同じに維持されます。

なぜブロックガス上限は上昇しないのか？

バリデータがガスターゲットに投票でき、それが直接ガス上限に変換されるなら、なぜブロックガス上限は3,000万にほぼ固定されたままで10億には上昇しないのかと疑問に思うかもしれません。より大きなブロックガス上限は、より多くのトランザクションがブロックに収まることを意味するため、エンドユーザーにとってトランザクションはより安くなります。

答えは、ガス上限を上げることにはネットワークの分散化に関連するいくつかの欠点があるということです。実際、より大きなブロックはより多くのトランザクションを含むことができますが、それはブロックを時間内に検証することがより困難になることも意味し、一般的なハードウェアで構成されたイーサリアムノードの淘汰につながり、完全なブロックを検証できるのは強力なサーバだけになる可能性があります。それだけでなく、より大きなブロックは状態をより大きく成長させることも意味します。最終的な結果は以前と同じです：大きなサーバだけがノードを完全に実行できるようになります。

歴史的に、図14-25に示すように、ブロックガス上限はプロトコルのアップグレード中に一度に引き上げられてきました。その値は通常、すべてのクライアントがトランザクションの負荷を処理し、ブロックを時間内に処理できることが確実なコア開発者が提案するレベルに設定されています。

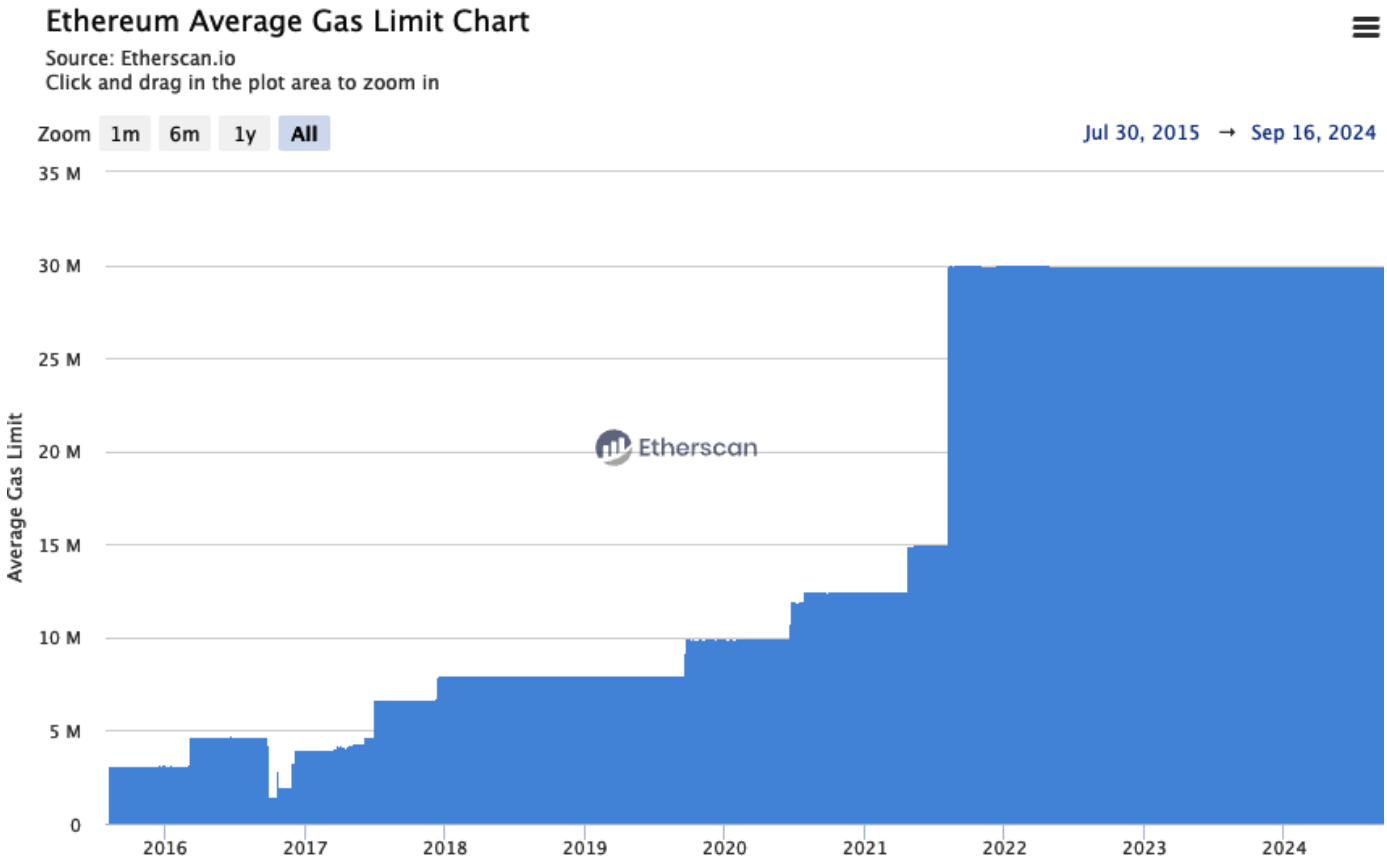


図14-25. イーサリアム平均ガス上限チャート

なぜブロックガス上限を引き上げないのか？

ブロックガス上限は3年以上にわたって3,000万に「固定」されているように見えますが、PBS（「イーサリアムステートレス」を参照）では、洗練されたハードウェアを持つアクターがブロックを構築し（ビルダー）、それをプロポーザー（バリデータ）に送信してP2Pネットワークで公開するため、実際に使用されるガスをほぼ一定に保ちながらブロックガス上限を無制限に引き上げるインセンティブがあります。

事実として、EIP-1559に従って、ブロックガスターゲットが実際に使用されるガスよりもはるかに高い場合、ベースフィーは減少し続け、ガス手数料のほぼすべてがバーンされずにバリデータに渡るようになります（ベースフィーはバーンされ、それらのETHが供給から削除されます）。

したがって、バリデータにとっての利点は2つあります：

- ETH供給を減らしてバーンする代わりに、ガス手数料を自分たちのために保持できる。
- 必要に応じて、多くのMEV活動を取り込む大きなブロックを作成でき、さらに多くの手数料を得られる。

これについて興味がある場合は、James Prestwichの記事で詳しく読んでください。

具体的な実装

すべてのイーサリアムノードには、本章で説明したEVMの具体的な実装があります。最も有名で広く使用されているものを以下に示します：

Go-ethereum EVM

Gethは最も広く採用されている最古の実行クライアントです。Goで書かれたEVMの完全な実装を含んでいます。

Execution-spec EVM

Ethereum FoundationはGitHub上でイーサリアム実行クライアントに関連する仕様を含むPythonリポジトリを管理しています。Pythonで書かれた完全なEVM実装を持っています。

Revm

Revmは、その優れたカスタマイズ性と適応性により、通常のイーサリアムクライアント以外で最も使用されているEVM実装の1つです。

Evmone

Evmoneはおそらく最速のEVM実装です。C++で書かれ、Ethereum FoundationのIpsilonチームによって管理されています。

Besu EVM

BesuはConsensysによって管理されている実行クライアントです。Javaで書かれたEVMの完全な実装を持っています。

Nethermind EVM

Nethermind EVMはC#で書かれた実行クライアントで、EVMの完全な実装を維持しています。

最大のEVMアップグレード：EVMオブジェクトフォーマット

EVMオブジェクトフォーマット（EOF）は、2015年の誕生以来、EVMへの最大のアップグレードです。過去にもEVMへの変更はありましたが、それらは主にガスの計量や新しいオペコードの導入に焦点を当てたものであり、EVMはGavin Woodが最初に設計したものとほぼ同一のままです。

EVMは依然として優れています。今日イーサリアム（およびその他すべてのEVM互換ブロックチェーン）で行われているすべての活動は、EVMのおかげで実現しています。だからと言って完璧というわけではありません。近年、スマートコントラクト開発者はEVMのさまざまな側面に対処し、その制約を克服するためのテクニックを学ぶ必要がありました。

EOFは、デプロイ時に一度だけ検証を行う、EVM用の拡張可能でバージョン管理されたコンテナフォーマットです。このセクションでは、EVMの主要な制限と、EOFがそれらをどのように克服しようとしているかを見ていきます。

> **注記**
>
> 本書の最終改訂時点（2025年6月）では、EOFアップグレードはイーサリアムコミュニティ内での合意が不十分なため、無期限に延期されている。実装に関する明確なタイムラインは現在なく、最終的に採用されない可能性もある。それでもなお、EOFがEVMとイーサリアムエコシステム全体にどのような影響を与えうるかを理解するうえで、このセクションには価値があると考える。

JUMPDEST解析

　レガシーEVMは、作成時にオンチェーンに公開されるバイトコードを検証しません。一見すると、任意のコントラクトをデプロイできるため良いことに思えるかもしれませんが、実際には多くの非効率性を生んでいます。存在しないオペコードを含むバイトコードをデプロイしたり、ま

ったく実行されないコードを追加したり、`PUSH`オペコードが必要とする直後のデータバイトが欠落したままのコードを追加したりできてしまいます。EVMは実行時にこれらすべてをチェックしなければならず、複雑さが増し全体的なパフォーマンスが低下します。以下はEVMバイトコードに存在しないオペコードが含まれている例です：

```
600C600052602060000C
```

バイトコードを人間が読める形式のオペコードに変換すると以下のようになります：

```
[00]            PUSH1           0C
[02]            PUSH1           00
[04]            MSTORE
[05]            PUSH1           20
[07]            PUSH1           00
[09]            NOT-EXISTING
```

オペコード`0C`は存在せず、EVMがこの地点に到達するとパニック（異常終了）を引き起こし処理を中断します。2番目のバイトも`0C`ですが、EVMはここでは失敗しません。最後の`0C`バイトはオペコードとして解釈され無効なため失敗しますが、2番目の`0C`は最初の`PUSH1`オペコードのプッシュデータとして解釈されるためです。

この実行時チェックの重要な部分が*JUMPDEST解析*です。すべてのクライアントはコントラクトが実行されるたびに実行時にJUMPDEST解析を行う必要があります。このJUMPDEST解析とは何か、なぜレガシーEVMで必要なのかを理解しましょう。

> **ヒント**
>
> コントラクトが呼び出されるたびにJUMPDEST解析を行わなくて済むように、一部のイーサリアムノード実装ではすべてのコントラクトについてJUMPDESTマップを保存している。このマップはコントラクトのデプロイ時に作成され、ノードのデータベースに保存される。

レガシーEVMはバイトコードの制御フローを管理するために動的ジャンプ（`JUMP`と`JUMPI`オペコード）のみを使用します。たった2つのオペコードで実行フローを制御できるため非常に便利です。しかし動的ジャンプは非常にコストが高く、実行時に深い検証が必要になります。しかし実際のコードでは、ジャンプ先は`JUMP`オペコードの直前に`PUSH`で固定値としてスタックに積まれることが多く、動的にジャンプ先を決める必要がないケースがほとんどです。以下は簡単な例です：

```
…600956600601610101 5b6001…
```

人間が読める形式のオペコードに変換すると以下のようになります：

```
[00]             PUSH1           09
[02]             JUMP
[03]             PUSH1           06
[05]             ADD
[06]             PUSH2           0101
[09]             JUMPDEST
[0a]             PUSH1           01
```

ジャンプ先のオフセット`0x09`は、`JUMP`オペコードの直前に`PUSH1`オペコードによってスタックにプッシュされています。EVMは`0x09`をスタックにプッシュし、次に`JUMP`オペコードを実行して先ほどプッシュされた`0x09`を入力として取り、そのオフセットの命令に実行を移動します。`0x09`には`JUMPDEST`オペコードがあるため有効なジャンプ先と見なされ、実行が正しく続行されます。

無効な宛先にジャンプしないようにするために、実行時の検証が必要です。有効なジャンプ先は、プッシュデータには含まれない`JUMPDEST`命令のみです。これは非常に重要な点です。レガシーEVMではコードとデータが明確に区別されていません。そのためバイトコード内でバイト`0x5b`（`JUMPDEST`オペコード）に出会った場合、コントラクト全体を解析しなければ、それが本物の`JUMPDEST`オペコードなのかプッシュデータの一部なのかを確実に判断できません。

この微妙な違いを示す次の例を見てみましょう：

```
…6009566006016101015b6001…
…6009566006016201015b6001…
```

この2つのバイトコードはほぼ同一に見えます。実際、異なるのは1ビットだけです。しかしそれだけで結果に大きな違いが生じます。最初のバイトコードは前の例と同じです。2番目のバイトコードは人間が読める形式では以下のようになります：

```
[00]             PUSH1           09
[02]             JUMP
[03]             PUSH1           06
[05]             ADD
[06]             PUSH3           01015b
[0a]             PUSH1           01
```

この2番目のコードを実行しようとすると、無効なジャンプ先のために失敗します。オフセット`0x09`にはまだ`0x5b`バイト（`JUMPDEST`オペコード）があるため少し奇妙に感じるかもしれません。問題はこの場合、`0x5b`バイトがプッシュデータの一部であるため、有効なジャンプ先とは見なされないことにあります。そのため実行はこの位置で失敗します。

JUMPDEST解析とは、どのジャンプ先が有効でどれが無効かを把握するためにコントラクトを解析するプロセスです。EVMがコントラクトを実行する際に無効なジャンプ先を検出してパニックできるようにするためのものです。

コントラクトAとやり取りするトランザクションを送信するとします。EVMはコントラクトAをロードすると、まずJUMPDEST解析を実行して有効なジャンプ先のマップを保存し、その後トランザクションの実行を開始します：

```
…600956600601610101 5b6001601556600601620101 5b6001…
```

人間が読める形式では以下のようになります：

```
[00]            PUSH1           09
[02]            JUMP
[03]            PUSH1           06
[05]            ADD
[06]            PUSH2           0101
[09]            JUMPDEST
[0a]            PUSH1           01
[0c]            PUSH1           15
[0e]            JUMP
[0f]            PUSH1           06
[11]            ADD
[12]            PUSH3           01015b
[16]            PUSH1           01
```

EVMはすべてのバイトコードを走査し、各`0x5b`バイトが有効なジャンプ先か無効なジャンプ先かをマークしたマップを作成します。図14-26に示します。

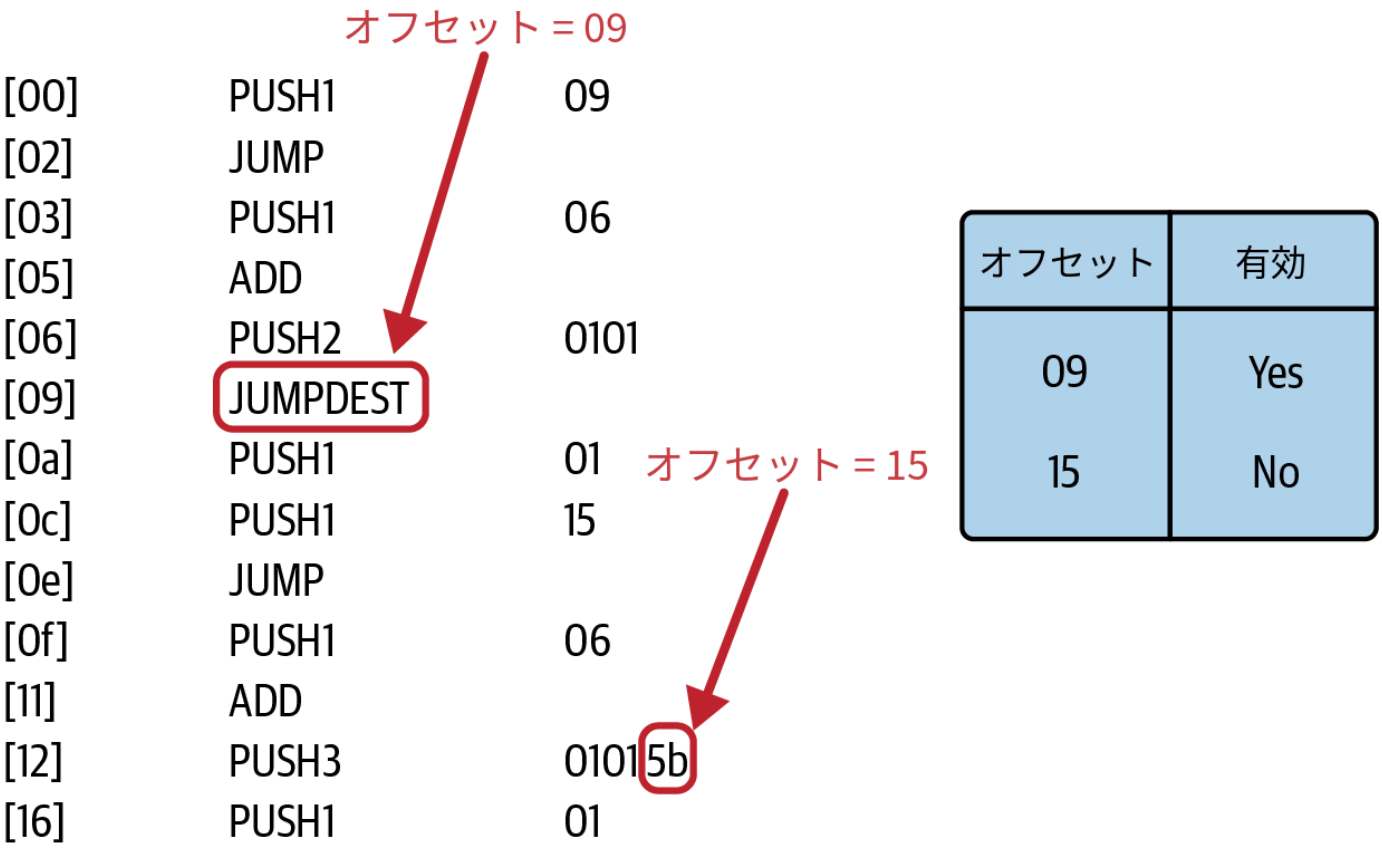


図14-26. クライアントが有効なジャンプ先と無効なジャンプ先を区別するためにJUMPDESTマップを作成する

機能の追加と廃止

オペコードや特定の機能を追加・廃止することは、見た目ほど簡単ではありません。`BLOBHASH`、`BLOBBASEFEE`、`BASEFEE`など様々なオペコードが追加されてきたのは事実ですが、EVMのバイトコードが構造化されておらず検証もされていないため、常に複雑な作業になります。

以下の例を見てみましょう。無効なオペコード`0x0C`を含んでいます。これがイーサリアムメインネットにデプロイされた実際のコントラクトだと仮定します：

```
600C600052602060000C
```

人間が読める形式では以下のようになります：

```
[00]            PUSH1           0C
[02]            PUSH1           00
[04]            MSTORE
[05]            PUSH1           20
[07]            PUSH1           00
[09]            NOT-EXISTING
```

この小さなバイトコードをEVM内で実行すると、存在しないオペコードに到達した時点で失敗します。

将来のアップグレードで新しいオペコード`MAX`が追加され、スタックから2つのアイテムを取り出して大きい方の値を返すとします。割り当てられるバイトは`0x0C`です。先ほどのバイトコードをもう一度考えてみましょう（メインネットにデプロイされた実際のコントラクトと仮定しています）：

```
[00]            PUSH1           0C
[02]            PUSH1           00
[04]            MSTORE
[05]            PUSH1           20
[07]            PUSH1           00
[09]            MAX
```

このバイトコードの結果はまったく異なるものになります。失敗せずに正常にリターンするようになるのです。（以前は）存在しないオペコードのためにこのコントラクトが常に失敗するという前提に依存していたコントラクトがあれば、問題が発生するでしょう。

機能の廃止はさらに困難です。EVMにはバージョニングシステムがないためです。オペコードを削除したり動作を変更したりすると、そのオペコードを使用していた古いコントラクトが壊れ

る可能性があり、新しいコントラクトを作成するという手動の介入以外に修正方法はありません。

EIP-2929がステートアクセスオペコードのガス計量を変更するために導入された際、使用するガス量や期待するガス量をハードコーディングしていた一部のコントラクトが壊れました。それらを再び動作させるための解決策が*アクセスリスト*の導入でした。アクセスリストでは、トランザクションによってアクセスされることが分かっているアカウントやストレージスロットを事前にロードしてガスコストを低減できます。

レガシーEVMのアップグレードを困難にする他の重要な側面として、*コードイントロスペクション*（コードの自己参照）と*ガスオブザーバビリティ*（ガス残量の参照）があります。ガスオブザーバビリティは`GAS`やガスを入力として取るすべての`*CALL`オペコードを通じて行うことができ、コードイントロスペクションは`CODESIZE`、`CODECOPY`、`EXTCODESIZE`、`EXTCODECOPY`、`EXTCODEHASH`などのオペコードを通じて行うことができます。

問題は、EVMが実行の特定の時点で残りのガスにアクセスできる場合、スマートコントラクトのロジックをそのガス量に依存させることができる点にあります。将来、一部のオペコードのガス計量に変更が加えられた場合（これはそれほど珍しいことではありません）、古いガス計量に依存していたすべてのスマートコントラクトに問題が生じる可能性があります。コードイントロスペクションについても同様です。

> **注記**
>
> コードイントロスペクションがなければ、将来のEOFバージョンで基盤となる仮想マシンをCairo VMやWasmなど別のものに完全に変更し、すべてのコントラクトのコードを同じ機能を持つ新しいVMのコードに自動変換することが可能になる。

コードとデータの分離

レガシーEVMは、オンチェーンに公開されるバイトコードに構造を強制しません。コード内で使用される実際のコードとデータの区別はありません。すべてはただのバイト列であり、実行時にEVMによってステップごとに解釈されます。

コントラクト全体を見なければ、バイトコードの特定の部分がコードなのかデータなのかを判断することは不可能です。以下の例を見てみましょう：

```
…730102030405060708090a0b0c0d0e0f101112131431…
[00]             PUSH20     0x0102030405060708090a0b0c0d0e0f1011121314
[15]             BALANCE
```

コードの中身を詳しく見なければ、`0x0102030405060708090a0b0c0d0e0f1011121314`が一連のオペコード（`0x01`：`ADD`、`0x02`：`MUL`など）として解釈されるべきものではなく、残高を照会したい

固定アドレスを表すプッシュデータであることをすぐには判断できません。これは静的解析ツールや形式検証にとって問題であり、スマートコントラクトがオンチェーンで実行可能なコードを正しく処理することを困難にしています。

Stack Too Deep

ここではEVMの最も嫌われている制限の1つである「stack too deep」エラーについて見ていきます。以下のスマートコントラクトをコンパイルしようとすると、このエラーがすぐに発生します：

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.27;

contract StackTooDeep {
    function add(
        uint256 a,uint256 b,uint256 c,uint256 d,uint256 e,uint256 f,uint256 g,uint256 h,uint256
i
    ) external pure returns(uint256) {
        return a+b+c+d+e+f+g+h+i;
    }
}
```

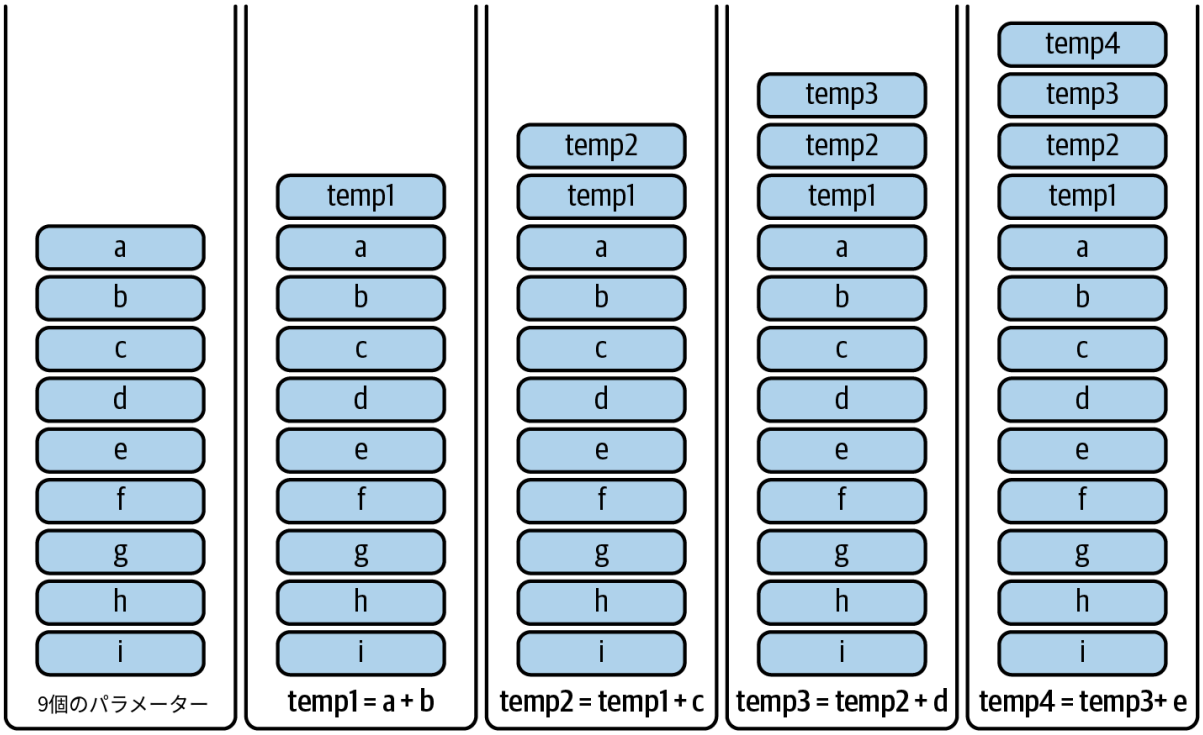
コントラクトをコンパイルしようとした際の出力は以下のとおりです：

```
solc StackTooDeep.sol --bin
Error: Stack too deep. Try compiling with `--via-ir` (cli) or the equivalent `viaIR: true`
(standard JSON) while enabling the optimizer. Otherwise, try removing local variables.
 --> StackTooDeep.sol:8:16:
  |
8 |         return a+b+c+d+e+f+g+h+i;
  |                                ^
```

このエラーは非常に微妙で、EVMの動作方法の直接的な結果です。スタックにプッシュできるアイテムの最大数は1,024ですが、EVMは `DUP1..16` や `SWAP1..16` のようなオペコードを通じて上位16要素にしかアクセスできません。

`StackTooDeep` の例では、 `add` 関数は9つの異なるパラメータを取り、それぞれをスタックに置く必要があります。その後、最終結果を得るために8回の加算を実行します。問題はこの詳細にあります。最終的な操作は1つの大きな加算に見えますが、実際にはコンパイラは各加算を分割し、中間結果を保持する一時変数を作成してスタックに保存する必要があります。結果として9個のパラメータと8個の中間結果、合計17個のスタックアイテムが必要になります。コンパイラがスタック内の17番目以降の位置にある変数を参照しようとすると、アクセスできるオペコードがないため失敗します。

この問題をさらに視覚化するために、図14-27を見てください。各ステップのスタック構成が詳細に示されています。実行の最後に変数`i`が深さ17にあることが容易に分かります。コンパイラは関数のスコープ内でローカル変数やパラメータにアクセスできない状態を許容しないため、「stack too deep」エラーをスローします。



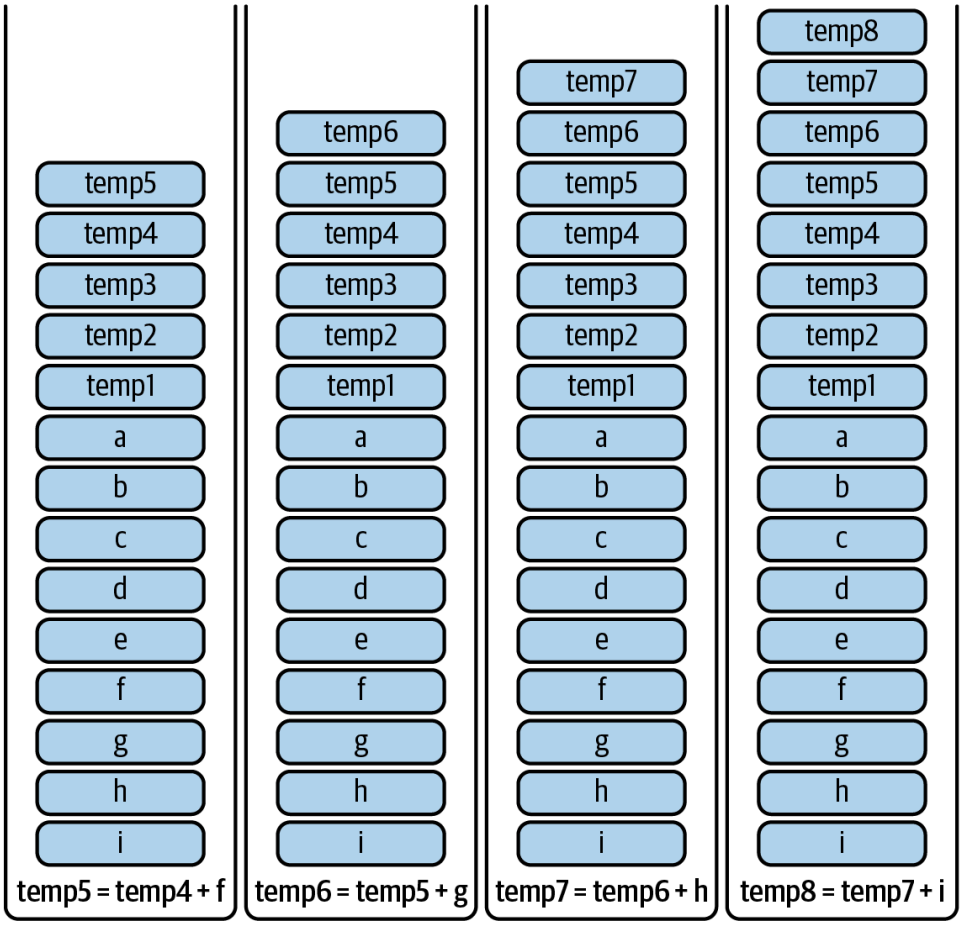


図14-27. 各ステップのスタック構成

これはスマートコントラクト開発者が日常業務で克服しなければならない問題の1つを示す非常にシンプルな例です。コンパイラはこのような状況を自動的に解析・管理する能力が向上してきていますが、EVMの現在の仕組み上、回避できないハードな制限があります。

EOF

EOFはデプロイ時にEVMバイトコードの検証を行うためのコンテナフォーマットを導入することで、これらの問題すべてに対処します。加えてバージョニングシステムやコードとデータの完全な分離も提供します。以下はEOFに準拠したバイトコードの例です：

```
EF000101000402000100080300010030040020000080000045F5FD1005FEC0000EF000101000402000100040300010014040000000080000025F5FEE00EF000101000402000100010400000000800000000000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e
```

人間が読める形式に変換すると以下のようになります：

```
Magic bytes                     EF00
EOF version                     01
Stack validation data           01 0004
Code sections                   02 0001 0008
Subcontainer section size       03 0001 0030
Data section size               04 0020
Header terminator               00
Stack: #ins, #outs, max stack   00 80 0004
Code                            5F5FD1005FEC0000
Subcontainers                   EF000101000402000100040300010014040000000080000025F5FEE00
                                EF000101000402000100010400000000800000000
Data                            000102030405060708090a0b0c0d0e0f101112131415161718191101
                                a1b1c1d1e
```

ここではまだ読み解く必要はありません。以降のセクションですべて説明します。

改善点

JUMPDEST解析とすべてのガスおよびコードイントロスペクションオペコードは完全に削除され、未定義とみなされるようになりました。コードセクションでは、すべてのオペコードが有効であること、オペコードに付随するデータ（プッシュデータなど）がコード内に存在すること、到達不能な命令がないことが求められます。スタック検証も作成時に実行されます。これにより実行中のスタックアンダーフローとオーバーフローが不可能になり、デプロイ時に検証済みのコントラクトはすべてのルールを遵守していると仮定できるため、実行時にこれらのチェックを行う必要がなくなります。

ヒント

これらの検証保証により、EVMバイトコードからマシンネイティブコードへのAOTまたはJITコンパイルの実現可能性が向上する。

動的ジャンプも削除され、3つの新しいオペコード`RJUMP`、`RJUMPI`、`RJUMPV`による静的相対ジャンプが導入されました。これらは`PUSH`オペコードと同様に付随するデータを取ります。EOFではJUMPDEST解析がなくなり、すべてが実行時ではなく作成時にチェックされるため、付随するデータを取る新しい命令を追加できるようになった点に注目してください。

前のセクションのEOFコンテナから分かるように、EOFは*関数*（すなわちコードセクション）の概念を導入し、それぞれのスタックを分離しています。レガシーEVMは動的ジャンプに頼ることでのみこの動作を模倣できます。SolidityやVyperの関数は内部表現にすぎません。

そのために3つのオペコード（`CALLF`、`RETF`、`JUMPF`）と*リターンスタック*（EVMの通常のオペランドスタックとは完全に分離されたもの）が追加されました。リターンスタックは特に、関数にジャンプする前に実行情報を保存し、データを失わずに呼び出し元に戻れるようにするために必要です。図14-28はこの仕組みの視覚化に役立ちます。

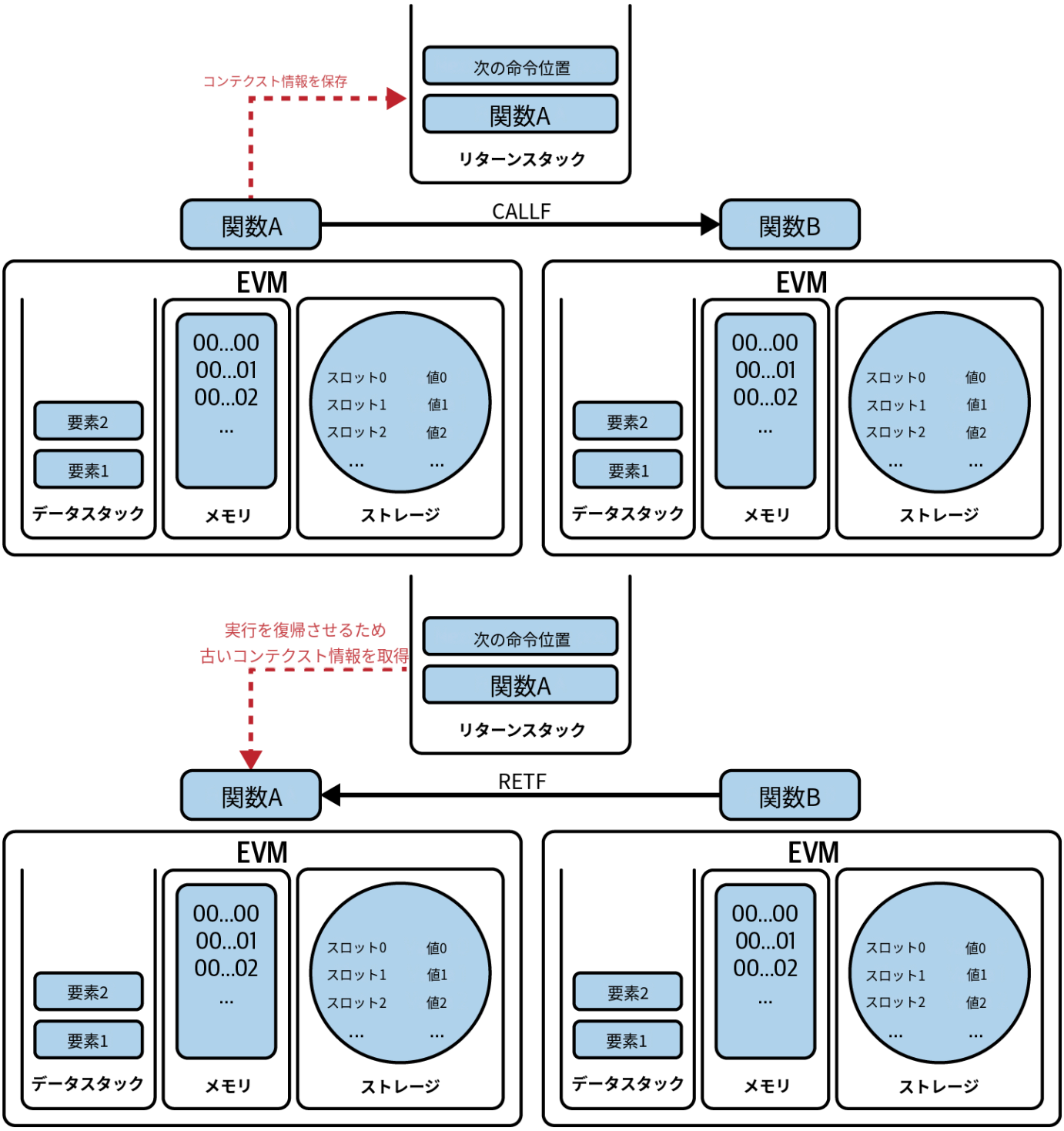


図14-28. EOFは関数（コードセクション）の概念を導入する

「stack too deep」問題は3つの新しいオペコード`DUPN`、`SWAPN`、`EXCHANGE`の追加により解決されます。最初の2つの命令は従来の`DUP1..16`と`SWAP1..16`に類似していますが、唯一の違いは複製またはスワップするアイテムのインデックスを表す付随するデータを1つ取ることです。そのインデックスは最大256まで指定でき、従来のハードリミットである16よりもはるかに大きくなっています。

`EXCHANGE`オペコードはスタック内の2つの異なるアイテムをスワップできる新しいオペコードです（`SWAPN`は常にn番目のアイテムと先頭のアイテムをスワップしますが、`EXCHANGE`は任意のアイテム同士をスワップできます）。これは変数のセットと使用分析に基づいてスタックトラフ

ィックを最小化しようとするスタックスケジューリングアルゴリズムを実装するコンパイラにとって特に有用です。

EOFコントラクトでは`CREATE`と`CREATE2`は廃止されます。代わりに`EOFCREATE`と`RETURNCONTRACT`が導入され、新しいEOFコントラクトを作成する手段を提供します。

最後に、削除された古い`CALL`命令を置き換える新しい命令が追加されます：`EXTCALL`、`EXTDELEGATECALL`、`EXTSTATICCALL`、`RETURNDATALOAD`です。これらはガスリミットの指定を許可せず（ガスオブザーバビリティなし）、EVMがサブコールの戻りデータを保存する出力バッファも必要としません。`RETURNDATALOAD`は、最後に実行されたサブコールの戻りデータを呼び出し元のスタックに取得するために導入されました。

EOFの実例

このセクションの冒頭で示したEOFコントラクトを解析してみましょう：

```
EF000101000402000100080300010003004002000008000045F5FD1005FEC0000EF000101000402000100040300010014040000000008000025F5FEE00EF000101000402000100010400000000800000000001020304050607080090a0b0c0d0e0f101112131415161718191a1b1c1d1e
```

まずEOFヘッダーがあります：

```
Magic bytes                   EF00
```

マジックバイトはEOFコントラクトをレガシーコントラクトと区別するために必要です（EIP-3541のおかげでイーサリアムメインネットでは`EF00`で始まるコントラクトは他にありません）。

次にEOFバージョンがあります。将来のEOFフォーマットのアップグレードをスムーズに行えるようにするためのものです：

```
EOF version                   01
```

スタック検証データは常に識別子`01`で始まり、その後に「Stack」部分のサイズが続きます。この例では4バイト長の「Stack」セクションがあります：

```
Stack validation data           01 0004
```

コードセクションは識別子`02`で始まり、コードセクション（関数とも呼ばれる）の数とそのサイズが続きます。ここでは8バイト長のコードセクションが1つあります：

```
Code sections               02 0001 0008
```

サブコンテナセクションサイズは識別子`03`で始まり、サブコンテナの数とそのサイズが続きます。ここでは48バイト長のサブコンテナが1つあります（常に16進形式であることに注意してください）：

```
Subcontainer section size        03 0001 0030
```

データセクションサイズは識別子`04`で始まり、データセクションのサイズが続きます。ここではデータセクションのサイズは32バイトです：

```
Data section size                04 0020
```

最後にヘッダーターミネータ`00`があります。これはEOFヘッダーの終端を表します：

```
Header terminator                00
```

Stackセクションは各コードセクション（この例では1つのみ）の入力数、出力数、最大スタック数を示します。入力はゼロで、非リターン関数（`80`は非リターン関数を表す特殊バイト）であり、スタックの最大数は4です：

```
Stack: #ins, #outs, max stack        00 80 0004
```

以下はEVMが実行するEVMバイトコードを含むすべてのコードセクションです：

```
Code                         5F5FD1005FEC0000
```

関数（コードセクション）は1つだけで、以下のオペコードを含んでいます：

```
PUSH0 PUSH0 DATALOADN 0 PUSH0 EOFCREATE 0 STOP
```

このコードはまず最初の2つの`PUSH0`オペコードによって`0-0`をスタックにプッシュします。次にデータセクションの32バイトをオフセットゼロ（コード内の次の引数値）から読み取り、スタックにプッシュします。その後、最後の`PUSH0`によってもう1つの`0`がプッシュされます。最後に`EOFCREATE`が引数`0`で呼び出され、最初のサブコンテナの最初のコードセクションを呼び出します：

```
Subcontainers                EF000101000402000100040300010014040000000080000025F5FEE00EF0001
                             010004020001000104000000080000000
```

サブコンテナセクションにはEOFバイトコードに含まれるすべてのEOFサブコンテナが含まれています。この例ではサブコンテナが1つあります。サブコンテナはEOFフォーマットのバイトコードです。EOFコンテナの中にEOFコンテナをネストできるという考え方で、ファクトリコントラクトに有用です。

最後にコントラクトの実行に必要なすべてのデータを含むデータセクションがあります。ここでは32バイトのデータがあります：

```
Data                    000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e
```

この解析はEOFバイトコードのサブコンテナに再帰的に適用して、何をしているかを完全に理解することもできます。EVMの深淵を覗きたい方は、ぜひ挑戦してみてください。

EVMの将来

EOFはさておき、EVMの将来は不確実であり、開発者やさまざまなプロジェクトによってEVMがどのように使用されるかに依存します。ただし、EVMが拡張できる興味深い領域がいくつかあります。たとえば、各ブロックの正しい実行を証明するゼロ知識証明を提供する*zk-EVM*などです。また、*EVMMAX*と*SIMD*はEVMにより多くの演算能力をもたらし、暗号処理をはるかに高速に行えるようにします。これは、プライバシープロトコルやL2などの暗号技術に依存するアプリケーションに特に有益です。

結論

本章では、EVMを探索し、さまざまなスマートコントラクトの実行をトレースし、EVMがバイトコードをどのように実行するかを見ました。また、EVMの会計メカニズムであるガスが、停止性問題をどのように解決し、イーサリアムをDoS攻撃からどのように保護するかについても確認しました。さらに、EOFを分析し、レガシーEVMのさまざまな課題をどのように修正しようとしているかを見ました。

次の第15章では、イーサリアムが分散コンセンサスを達成するために使用するメカニズムを探ります。

第15章　コンセンサス

本書を通じて*コンセンサスルール*について説明してきました。コンセンサスルールとは、分散化されながらも決定論的にシステムを運用するために全員が同意しなければならないルールのことです。コンピュータサイエンスにおいて*コンセンサス*という用語はブロックチェーンよりも前から存在し、分散システムにおける状態の同期というより広範な問題に関連しています。分散システムの異なる参加者全員が最終的にシステム全体で単一の状態に同意することを*コンセンサスの達成*と呼びます。

分散化された記録管理と検証の中核機能では、情報の正確性を保証するために信頼だけに頼ることは問題になり得ます。特に分散型ネットワークでは、何が真実かを判断する中央のエンティティが存在しないため、この課題は顕著です。中央の意思決定エンティティが存在しないことは、ブロックチェーンプラットフォームの主要な魅力の一つです。検閲耐性を持ち、情報へのアクセスに許可を必要としないためです。しかしこの利点にはコストが伴います。信頼できる仲裁者がいない場合、不一致や欺瞞、相違は他の手段で解決しなければなりません。コンセンサスアルゴリズムはセキュリティと分散化を両立させるためのメカニズムです。

ブロックチェーンにおいてコンセンサスはシステムの重要な特性です。端的に言えばお金がかかっているのです。したがってブロックチェーンの文脈でのコンセンサスとは、分散化を維持しながら共通の状態に到達できることを意味します。言い換えれば、コンセンサスは支配者なき厳格なルールのシステムを生み出すことを目的としています。特定の個人や組織、グループが「管理者」となるのではなく、権力と制御は広範な参加者ネットワーク全体に分散されます。参加者にとって、ルールに従い正直に行動することが自己利益にかなう仕組みです。

敵対的な条件下で中央集権的な制御なしに分散ネットワーク全体でコンセンサスに到達する能力は、すべてのオープンなパブリックブロックチェーンの核心的な原則です。この課題に対処し分散化という貴重な特性を維持するために、コミュニティはさまざまなコンセンサスモデルを実験し続けています。本章ではこれらのコンセンサスモデルと、イーサリアムのようなスマートコントラクトブロックチェーンへの予想される影響について探ります。

> **注記**
>
> コンセンサスアルゴリズムはブロックチェーンの仕組みの重要な部分だが、スマートコントラクトの抽象化よりもはるかに下位の基盤層で動作する。言い換えればコンセンサスの詳細のほとんどはスマートコントラクトの開発者からは隠されている。インターネットを使うためにルーティングの仕組みを知る必要がないのと同様にイーサリアムを使うためにコンセンサスの仕組みを知る必要はない。

コンセンサスの原則

ブロックチェーン技術、特にイーサリアムにおいてコンセンサスの原則を理解すると、ネットワークがどのように整合性を維持し効果的に運用されるかが分かります。より明確に把握するために、核心的なアイデアを順に見ていきましょう。

セーフティ

コンセンサスメカニズムの文脈において*セーフティ*とは、ネットワークがエラーなくブロックチェーンの現在の状態について一貫して合意することを保証するものです。二重支払いやトランザクションの衝突といった問題を回避し、ネットワーク全体の一貫性を維持します。安全なシステムではすべてのノードがチェーンの履歴について同一のビューを持ち、操作を一度に1つずつアトミックに実行する中央集権的な実装のように振る舞います。

ファイナリティ

*ファイナリティ*はイーサリアムのコンセンサスメカニズムにおける最も重要なセーフティ機能の一つです。ファイナリティとは、トランザクションが完了し不可逆になったとみなされる時点を指します。一度ブロックチェーンに追加されたトランザクションは変更も削除もできません。この不可逆性はシステムに高い信頼をもたらし、トランザクションが永久に記録されるという確実性をユーザーに提供します。

基本的にファイナリティはセーフティの概念を実際の行動に移すものであり、単なるコンセプトから実用的なものへと変換します。ネットワークの異なる部分がそれぞれのローカルなビューを持っていたとしても、チェーンの履歴を固定し変更不可能にする取り消し不能な合意のポイントが存在することを保証します。

ライブネス

セーフティがネットワーク上で「悪いことが起きない」ことを保証する一方、*ライブネス*は「良いことが最終的に必ず起きる」ことを保証します。言い換えれば、イーサリアムネットワークは何が起ころうとも、トランザクションの処理と新しいブロックの追加を継続します。

別の観点から見るとライブネスは可用性の面でも理解できます。実用的には、ネットワーク内で正直に行動しているノードに有効なトランザクションを送信するたびに、ブロックチェーンの延長に貢献する次のブロックにそのトランザクションが含まれると期待できます。トランザクションが確実に処理されるという期待は、ユーザーの信頼とイーサリアムプラットフォーム全体の有効性を維持するために不可欠です。

ブロックツリーとフォーク

あらゆる状況下でセーフティとライブネスの両方を満たすコンセンサスプロトコルを設計することは不可能であり、どちらか一方を優先する必要があります。イーサリアムのコンセンサスプロトコルは良好なネットワーク条件下ではセーフティとライブネスの両方を提供しますが、状況が混乱した際にはライブネスを優先します。フォークの概念を通じてライブネスを実現しています。

ブロックチェーンではすべてのブロック（特別なジェネシスブロックを除く）は親ブロックの上に構築され、親ブロックを指し示します。こうしてブロックのチェーン、すなわちブロックチェーンが形成されます。フォーク型コンセンサスプロトコルを実装するチェーンではこの直線的な条件は実際にはしばしば当てはまりません。現実の条件では図15-1に示すようにブロックチェーンというよりもブロックツリーのようなものが生じることがあり、コンセンサスプロトコルの目標はネットワーク上のすべてのノードが同一の直線的なブロックの順序に同意することです。

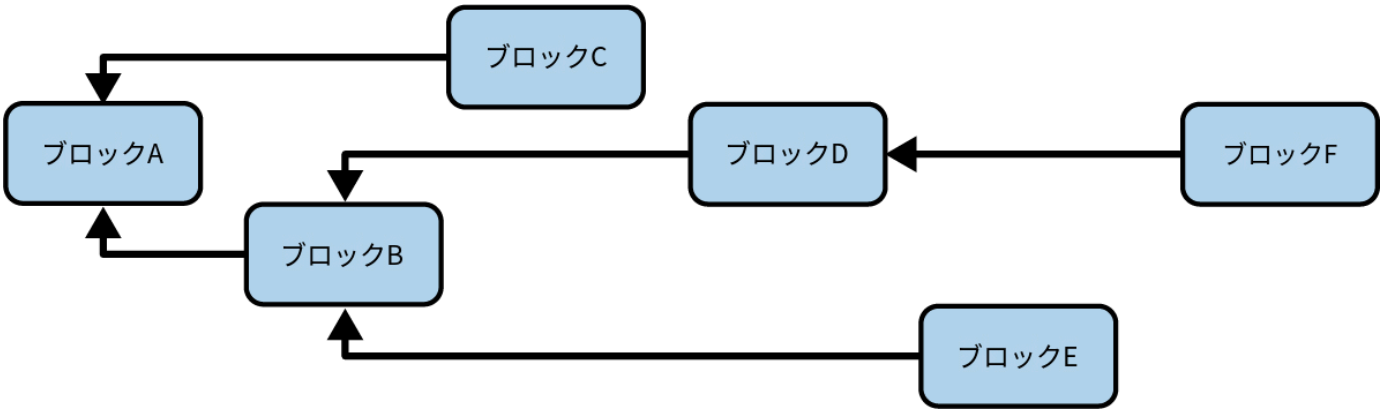


図15-1 ブロックツリー構造

ブロックツリーのさまざまな枝は*フォーク*と呼ばれます。フォークはネットワークおよび処理の遅延、クライアントの障害、悪意あるクライアントの行動の結果として自然に発生します。

異なるフォークを追従しているノードに問い合わせると、システムの状態について異なる回答が返ってきます。ここにフォーク型コンセンサスプロトコルの回復力があります。不利な条件下で完全に停止するのではなくフォークするのです。最終的にはネットワーク上のすべての正しいノードが同一の直線的な履歴のビューに同意し、システム状態の共通のビューを持つことが目標です。この合意をもたらすのがプロトコルの*フォーク選択ルール*です。ブロックツリーといくつかの判断基準が与えられた場合、フォーク選択ルールは利用可能なすべてのブランチの中から、最終的にカノニカルチェーンに到達する可能性が最も高いものを選択します。フォークプロトコルの欠点は、カノニカルチェーンに含まれないブランチを追従しているノードが、正しいブランチに移行するために最近処理したトランザクションを取り消してビューを巻き戻す必要がある点です。この巻き戻しは*リオーグ*または*リバージョン*と呼ばれ、混乱を引き起こすためプロトコルはリオーグをできるだけ最小限に抑えることを目指します。

これでイーサリアムのコンセンサスプロトコルにおけるファイナリティの力を理解できます。フォークはライブネスへの強力な手段ですが、ネットワークを利用可能にするにはこの柔軟性をある程度のセーフティの保証でバランスを取る必要があります。

proof of workによるコンセンサス

オリジナルのブロックチェーンであるビットコインの創始者は*proof of work*（PoW）に基づくコンセンサスアルゴリズムを発明しました。間違いなくPoWはビットコインを支える最も重要な発明です。PoWの一般的な呼称はマイニングであり、コンセンサスの主な目的について誤解を生んでいます。現実世界のマイニングの目的が貴金属やその他の資源を採掘することであるため、マイニングの目的は新しい通貨を作ることだと思われがちです。しかしマイニング（およびすべての他のコンセンサスモデル）の真の目的は、システムに対する制御をできるだけ多くの参加者に分散し、その分散状態でブロックチェーンを安全にすることです。新たに発行される通貨の報酬はシステムのセキュリティに貢献する者へのインセンティブにすぎません。報酬は手段であり、分散化されたセキュリティが目的です。

PoWコンセンサスには対応する「罰則」も存在し、マイニングに参加するために必要なエネルギーのコストがそれにあたります。この大量のエネルギー消費は欠陥ではなく、意図的なセキュリティとインセンティブ設計の鍵です。参加者がルールに従わず報酬を得られなかった場合、マイニングのためにすでに費やした電気代を失うリスクがあります。このようにPoWコンセンサスはリスクと報酬を慎重にバランスさせた仕組みであり、参加者が自己利益のために正直に行動するよう動機づけます。

イーサリアムはビットコインの例に倣いPoWブロックチェーンとして開始されました。同じ基本的なインセンティブシステムを使用し、同じ基本的な目標を持っていました。すなわち制御を分散化しながらブロックチェーンを安全にすることです。イーサリアムのPoWアルゴリズムはビットコインのものとはわずかに異なり*Ethash*と呼ばれていました。

Ethash：イーサリアムのPoWアルゴリズム

イーサリアムがPoSに移行する前はEthashと呼ばれるPoWアルゴリズムに依存していました。EthashはDagger-Hashimotoアルゴリズムの進化版を使用しており、ヴィタリック・ブテリンのDaggerアルゴリズムとサデウス・ドライヤのHashimotoアルゴリズムの組み合わせです。Ethashは*DAG*（有向非巡回グラフ）として知られる大規模なデータセットの生成と分析に依存しています。DAGの初期サイズは約1 GBで、ゆっくりと線形的に成長を続け、1エポック（30,000ブロック、およそ125時間）ごとに更新されていました。

DAGの目的は、EthashのPoWアルゴリズムが大規模で頻繁にアクセスされるデータ構造の維持に依存するようにすることでした。これによりEthashにASIC耐性を持たせることが意図されていました。つまり、高速なGPUよりも桁違いに速い特定用途向け集積回路（ASIC）マイニング機器の製造をより困難にするということです。イーサリアムの創設者たちはPoWマイニングにおける中央集権化を避けたいと考えていました。専門的なシリコン製造工場と大きな予算にアクセスできる者がマイニングインフラを支配し、コンセンサスアルゴリズムのセキュリティを損なうことを防ぐためです。

コンシューマーレベルのGPUを使用してイーサリアムネットワーク上でPoWを実行することで、世界中のより多くの人々がマイニングプロセスに参加できました。独立したマイナーが増え

ることでマイニングパワーがより分散化され、ビットコインのように少数の大規模な産業用マイニング事業者の手にマイニングパワーが集中する状況を回避できました。GPUをマイニングに使用することの欠点は2017年に世界的なGPU不足を引き起こし、価格が急騰してゲーマーからの怒りを招いたことです。小売業者では顧客1人あたり1〜2台のGPUに購入制限が設けられました。

2017年までイーサリアムネットワーク上のASICマイナーの脅威はほとんど存在しませんでした。イーサリアム向けのASICを使用するには高度にカスタマイズされたハードウェアの設計、製造、流通が必要でした。製造にはかなりの時間と資金の投資が必要でした。イーサリアムの開発者たちがPoSコンセンサスアルゴリズムへの移行計画を長年にわたって表明していたことが、ASIC供給業者がイーサリアムネットワークをターゲットにすることを長い間思いとどまらせていたと考えられます。

proof of stakeによるコンセンサス

歴史的に見るとPoWは最初に提案されたコンセンサスアルゴリズムではありませんでした。PoWの前に多くの研究者が金銭的または評判的なステークに基づくアイデアを探求していました。さらにそれ以前には初期のコンセンサスモデルの一部はパーミッション型でした。バリデータはオープンな競争ではなく権限やアイデンティティによって選ばれていたのです。*PBFT*（実用的ビザンチン障害耐性）は例えば固定またはキュレーションされたバリデータの集合を必要とするモデルであり、今日でも多くの従来の分散システムの基盤となっています。

PoWベースのコンセンサスの主要なブレークスルーの一つは参加をパーミッションレスにしたことです。利用可能な計算能力があれば参加の承認を必要とせずに誰でも貢献し報酬を得ることができました。これは分散化にとって大きな勝利でした。ある意味でPoWはより閉鎖的なモデルに対するパーミッションレスな代替手段として発明されたのです。

ビットコインの成功に続いて多くのブロックチェーンがPoWを採用しました。しかしコンセンサスに関する研究の爆発的な増加によりPoSへの関心が再燃し、大きな進歩につながりました。当初からイーサリアムの創設者たちは最終的にPoSに移行することを望んでいました。実際、イーサリアムの元のPoWチェーンにはいわゆる*ディフィカルティボム*という組み込みのハンディキャップが含まれていました。時間の経過とともにマイニングを徐々に困難にし、最終的な移行へとシステムを押し進めるよう設計されたものです。イーサリアムのPoSはパーミッションレスでありながら、初期の権限ベースのシステムにいくつかの概念的なルーツを持っています。ここではアイデンティティと参加はステークを預けることで得られます。しかしETHを持つ人なら誰でもそうすることができるため、ナカモトの設計のオープンアクセスの精神を保持しています。

一般的にPoSアルゴリズムは以下のように機能します。ブロックチェーンはバリデータの集合を追跡し、ブロックチェーンの基本暗号通貨（イーサリアムの場合はEther）を保有する者は誰でもEtherをデポジットとしてロックアップする特別なトランザクションを送信してバリデータになれます。バリデータは交代で次の有効なブロックを提案し投票します。各バリデータの投票

の重みはデポジット（すなわちステーク）のサイズに依存します。バリデータがプロトコルに正しく参加するとステークに比例した小さな報酬を得ます。不正確なアテステーションや遅延したアテステーションを公開するなどのミスを犯すと、報酬とほぼ同じ規模の小さなペナルティを受ける可能性があります。しかし*スラッシング*と呼ばれるはるかに深刻なペナルティもあり、バリデータが証明可能な悪意ある行為を行った場合にのみ発生します。たとえば矛盾するアテステーションを公開したり、同じスロットで矛盾するブロックを発行したりする場合です。したがってPoSは報酬と罰則のシステムを通じてバリデータが正直に行動しコンセンサスルールに従うことを強制します。PoSとPoWの主な違いは、PoSにおける罰則がブロックチェーンに内在する（たとえばステークされたEtherの喪失）のに対し、PoWにおける罰則は外在的である（たとえば電気代に費やした資金の喪失）ことです。

イーサリアムが2015年に立ち上げられて以来PoSコンセンサスプロトコルへの移行が意図されていました。その方向への最初の具体的なステップは2020年12月1日のビーコンチェーンの立ち上げで実現しました。当初ビーコンチェーンは空のブロックチェーンであり、特定のデポジットコントラクトに32 ETHを預けることで誰でもバリデータになれました。バリデータとそれぞれの残高に関する内部コンセンサスのみを処理していました。当時イーサリアムブロックチェーンはまだEthashをコンセンサスプロトコルとして使用していました。

2022年9月15日にマージハードフォークが発生し、独自のバリデータのセットを持つビーコンチェーンがPoSベースのコンセンサスプロトコルをイーサリアムのメインブロックチェーンに拡張しました。これによりEthashの使用は事実上終了しました。しかしバリデータが資本を引き出してバリデータセットを離れることができないなど、いくつかの制限が残っていました。これらの問題は2023年4月12日のShapellaアップデートにより完全に解決され、イーサリアムのPoWからPoSコンセンサスプロトコルへの移行が完了しました。

イーサリアムが使用するPoSコンセンサスプロトコルは*Gasper*と呼ばれています。以下のセクションでは基本的な用語から始めてフォーク選択ルール（LMD-GHOST）とファイナリティガジェット（Casper FFG）へと進み、Gasperの仕組みを探ります。最後に議論した理論的な概念を明確にするための例を示します。

PoSの用語

このセクションでは、イーサリアムのPoSコンセンサスのコンポーネントと用語に焦点を当てます。

ノードとバリデータ

*ノード*はイーサリアムネットワークの基盤を形成します。ノードは互いに通信しコンセンサスへの準拠を検証する責任を負います。*バリデータ*は新しいブロックの提案と投票を担当しノードに接続されていますが、名前が示すにもかかわらず実際にはブロック自体を検証するわけではありません。ブロックとトランザクションがプロトコルルールに従っているかどうかをチェックす

るのはノードソフトウェアです。1つのノードが複数のバリデータをホストでき、バリデータの職務は実行クライアントとコンセンサスクライアントの両方を実行することで遂行されます。次のセクションではこれらの職務が具体的に何であるかを見ていきます。

PoSの注目すべき特性の一つはアクティブなバリデータのセットが既知であることです。参加者の過半数投票に達したかどうかを特定できるためファイナリティを達成する上で鍵となります。

ブロックとアテステーション

厳格な時間管理は、イーサリアムのPoSの重要な特性です。PoSにおける2つの主要な時間間隔は、正確に12秒の*スロット*と、32スロットにわたる*エポック*です。

すべてのスロットで正確に1人のバリデータがブロックを提案するために選ばれます。すべてのエポックの間にすべてのバリデータは*アテステーション*の形で自分のネットワークビューを正確に1回共有します。アテステーションにはLMD-GHOSTプロトコルで使用されるチェーンのヘッドへの投票と、Casper FFGプロトコルで使用されるチェックポイントへの投票が含まれています。FFGは「Friendly Finality Gadget」の略です。アテステーションの共有は帯域幅を多く消費するため、ワークロードを分散して管理可能に保つためにすべてのブロックではなくエポック全体に分散されています。

プロトコルはバリデータに対する報酬とペナルティのシステムを通じてブロックとアテステーションの生成と正確性にインセンティブを与えますが、空のスロットやアテステーションも許容します。これらは有機的な理由（たとえばノードがオフラインになった場合）と利益追求の理由の両方で発生する可能性があります。後述の「タイミングゲーム」のセクションで詳しく説明します。

LMD-GHOST

LMD-GHOSTはイーサリアムコンセンサスプロトコルの主要部分であり、フォーク選択ルールアルゴリズムです。ノードがローカルなブロックチェーンのビューにおいて有効とみなすべき最新のブロックを選択します。このブロックは*チェーンのヘッド*とも呼ばれます。

LMD-GHOSTの仕組みを完全に理解するにはイーサリアムのPoSプロトコルの基本概念をいくつか知っておく必要があります。従来のPoWベースのコンセンサスプロトコルでは、新しいブロックを作成してチェーンに追加する責任のあるエンティティ（すなわちマイナー）は特別な要件に従う必要がありません。PoWを満たすブロックを公開すればネットワーク全体に受け入れられます。イーサリアムのPoSベースのコンセンサスプロトコルではバリデータはバリデータセットに参加するだけで、担保として多額のETH（現在少なくとも32 ETH）をステークする必要があります。

前述のとおりバリデータには2つの主要な職務があります。

ブロック提案

各スロットで1人のバリデータが擬似ランダムに選ばれ、チェーンの次のブロックを作成して提案する。

アテステーションの作成

各スロットでバリデータの一部が選ばれ、チェーンの最良のヘッドだと考えるブロックに投票する。この投票はアテステーションの形で全バリデータに共有される。

バリデータがアテステーション内で特定のブロックに投票すると、実際にはそのブロックにスコアを割り当てています。このスコアはアテステーションを公開した時点でバリデータがステークしているETHの量に正確に等しいです。しかしそれだけではありません。この投票はそのブロックだけでなく、図15-2に示すように選択されたブロックと同じフォーク上に存在するすべての祖先ブロックへの投票でもあります。

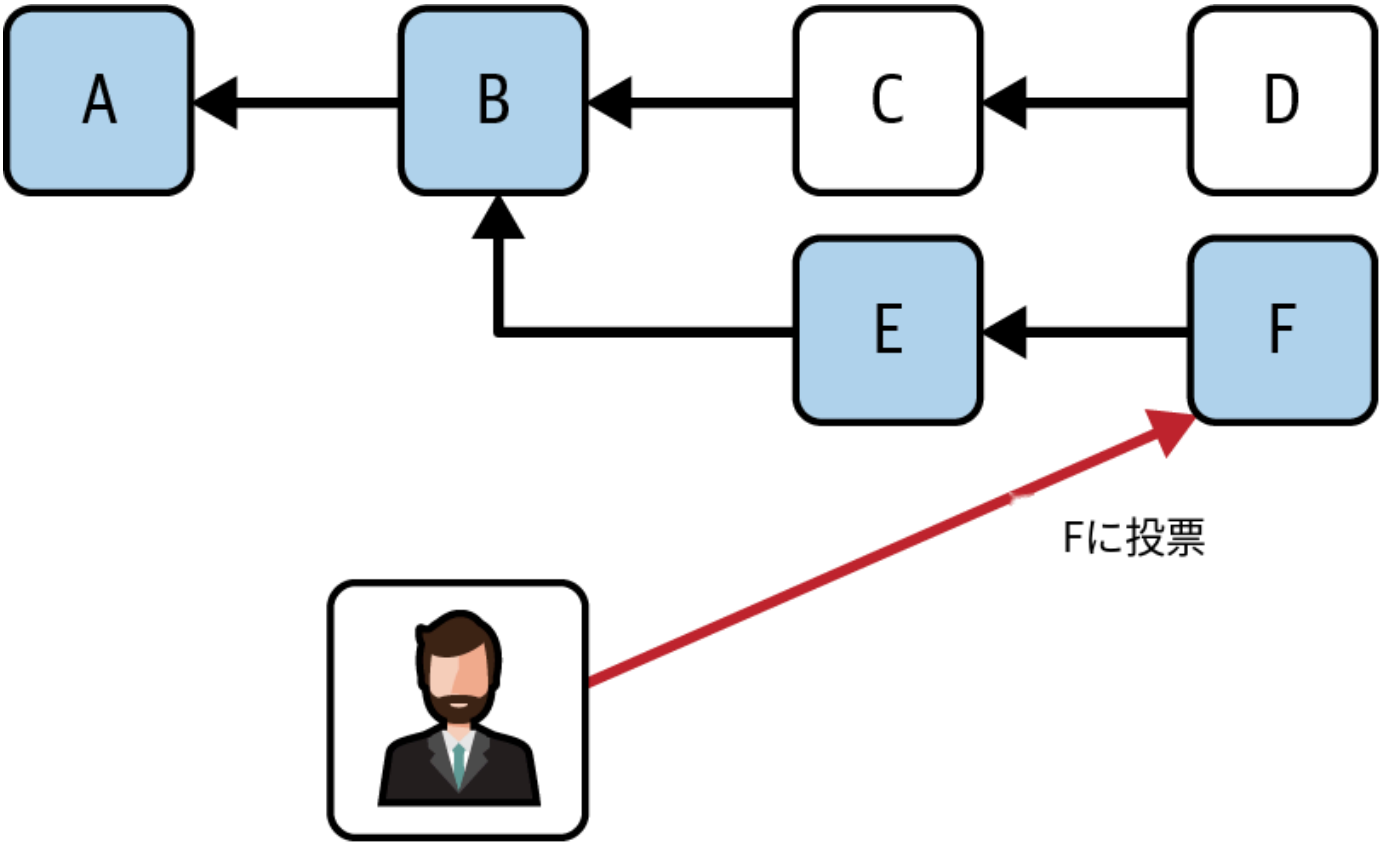


図15-2 祖先への投票の伝播

ブロックへの投票はすべての祖先に遡って伝播されると言えます。この概念をさらに明確にするためにすべてのブランチにスコアを割り当てることができます。図15-3に示すようにブランチはブロックとその親を接続するリンクです。

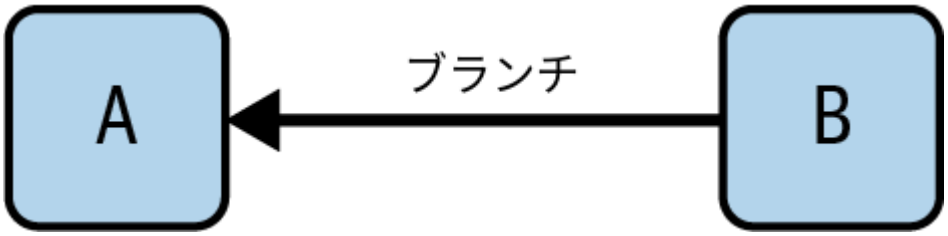


図15-3 ブランチの定義

ブランチのスコアはそのブランチの根となるブロック（図15-3のブロックB）のスコアにすべての直接の子孫ブランチのスコアを加えたものと定義されます。

図15-4は各ブロックのスコアが1に等しいブロックのチェーンを示しています。EからDへのブランチのスコアは子孫ブロックがないためブロックEのスコアに正確に等しくなります。ブランチD→Cのスコアを計算するにはブロックDのスコアにすべての子孫ブランチのスコアを加える必要があります。この場合、子孫ブランチは1つだけでブランチE→Dです。したがって1（ブロックDのスコア）＋1（ブランチE→Dのスコア）＝2です。次にブランチC→Bがあり、そのスコアは1（ブロックCのスコア）＋2（ブランチD→Cのスコア）＝3です。ここでブロックC'との小さなフォークがあります。ブランチC'→Bにスコアを割り当てる必要があります。ブロックC'には直接の子孫がないためスコアは1（ブロックC'のスコア）です。最後にブランチB→Aがあります。スコアを計算するには1（ブロックBのスコア）＋1（ブランチC'→Bのスコア）＋3（ブランチC→Bのスコア）＝5を合計する必要があります。

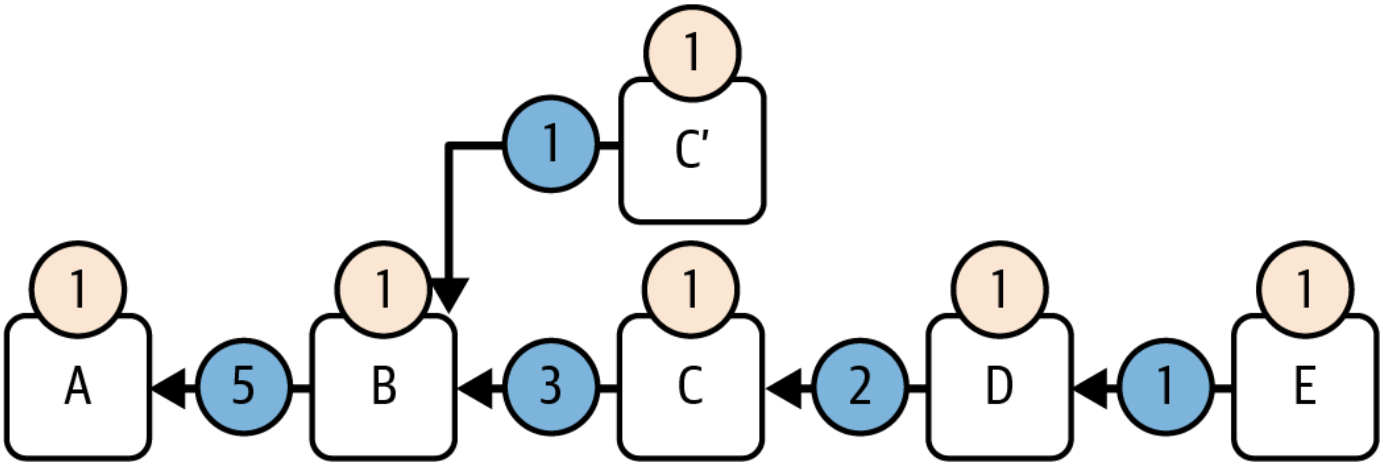


図15-4 ブランチスコア計算例

図15-5は各ブロックが異なるスコアを持つ、いくつかのフォークがあるより複雑なシナリオを含んでいます。確認して各ブランチのスコアがどのように計算されるか理解できることを確かめてください。

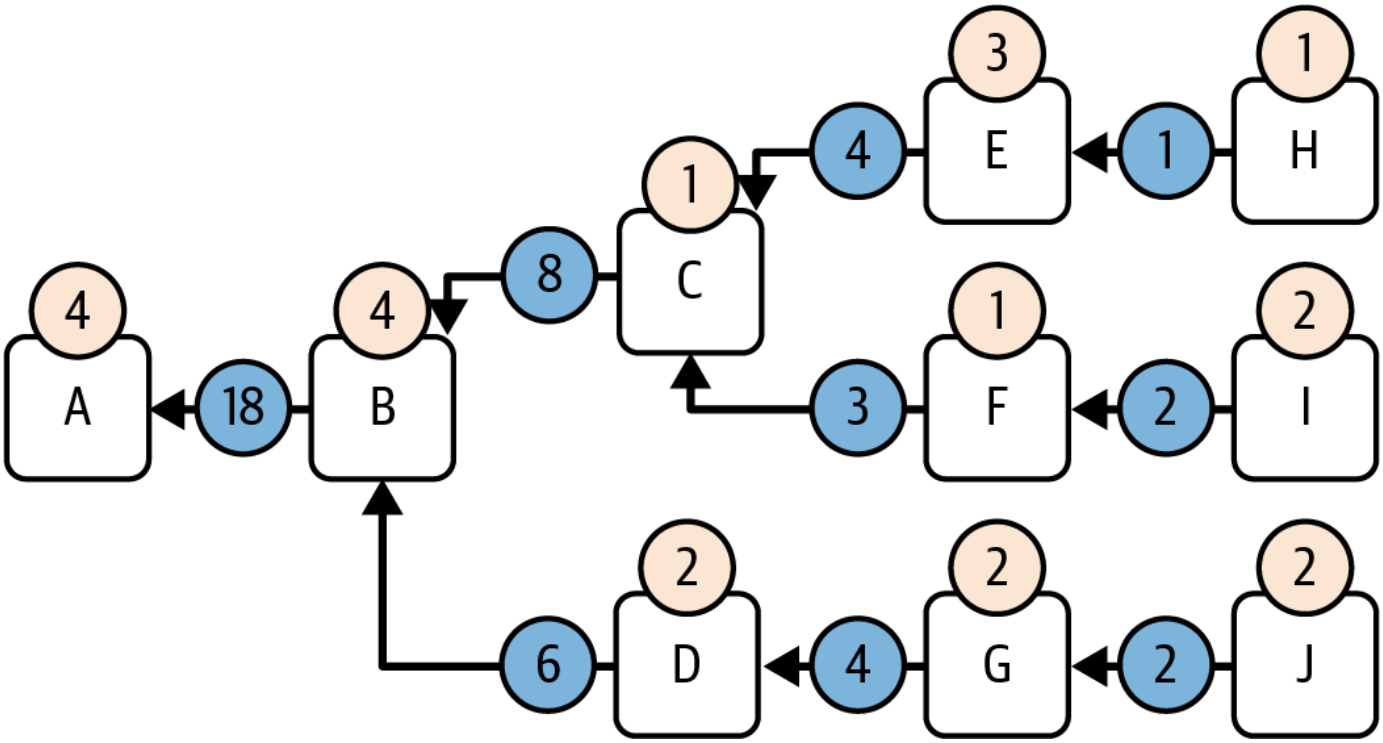


図15-5 複雑なブランチスコアリング

最後の例の後、ブロックのスコアは単一のブロックだけでなくすべての以前のブランチに遡って伝播されるため、すべての祖先にも影響を与えることが明らかになったはずです。バリデータがアテステーションの形でブロックをチェーンのヘッドとして投票する場合、そのブロックのすべての祖先も有効で正しいチェーンの一部であるとみなしているというのがその考え方です。

ここでLMD-GHOSTが実際にどのように機能しチェーンのヘッドとみなされるブロックをどのように選択するかの詳細に踏み込みましょう。まず名前を分析します。LMD-GHOSTは2つの頭字語で構成されています。*Latest Message Driven*（最新メッセージ駆動）と*Greediest Heaviest Observed Subtree*（最も貪欲で最も重い観測されたサブツリー）です。

Latest Message Driven

各ブロックとブランチにスコアを割り当てるには各バリデータの最新のアテステーションのみを考慮する必要があります。つまりバリデータVから2つのアテステーションを受け取った場合、2回カウントする必要はありません。どちらが最新であるかを確認しもう一方を破棄する必要があります。

図15-6はブロックBでアテステーションを公開しブロックBがチェーンのヘッドであると共有するバリデータを示しています。その後ブロックFの時点でそのバリデータが再び選ばれ、新しいヘッドとしてブロックFを支持する新しいアテステーションを投稿します。バリデータがブロックFで新しいアテステーションを投稿すると、他のバリデータは古いもの（ブロックBの時点で公開されたもの）を破棄し最新のもののみを考慮する必要があります。

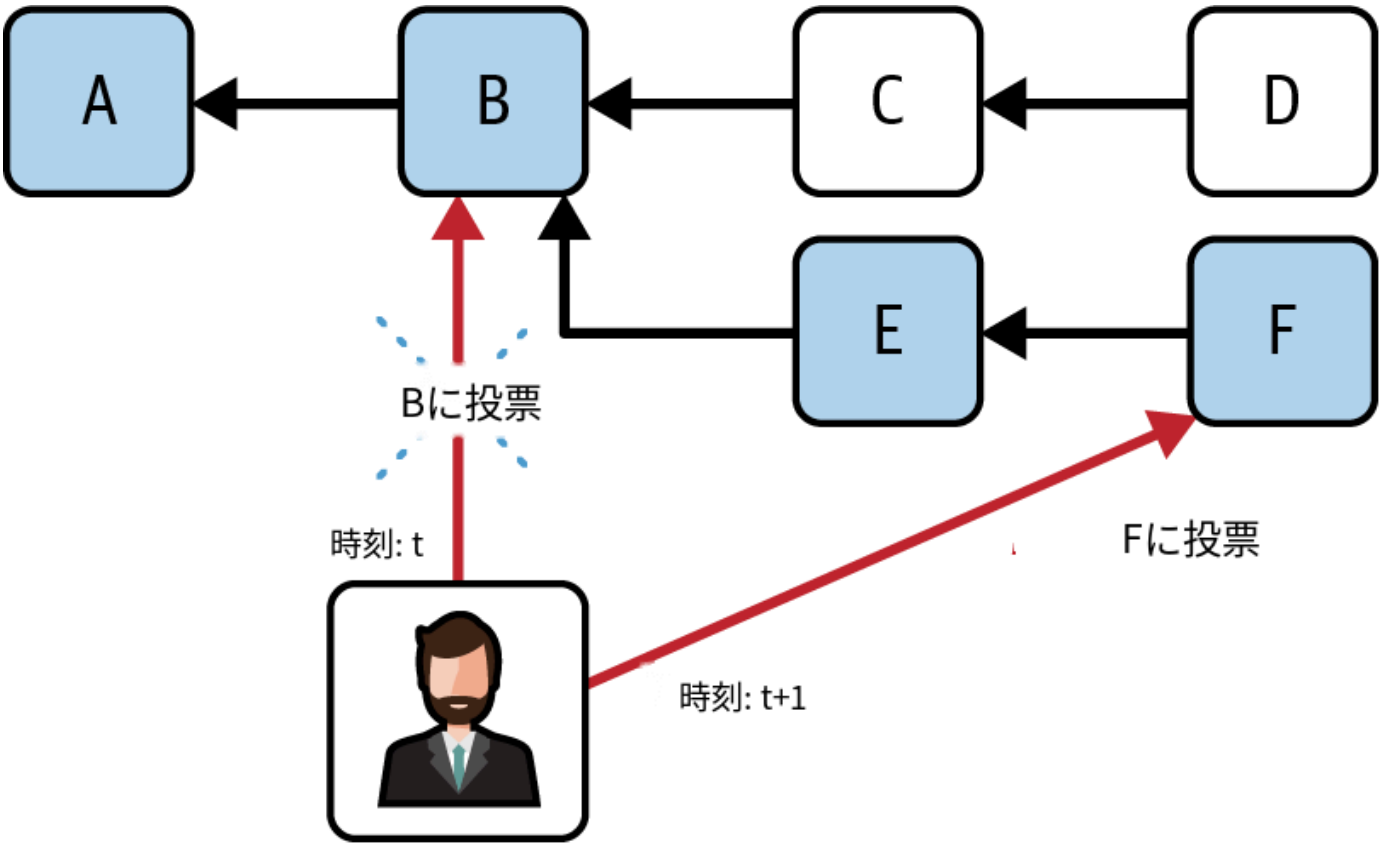


図15-6 Latest Message Drivenの例

Greediest Heaviest Observed Subtree

GHOSTはフォーク選択ルールの主要な側面です。ヘッドブロックは最も多くの投票を集めたフォークに属するさらなる子孫を持たないブロックです。

LMD-GHOSTが実際のシナリオでどのように機能するかをより良く理解するために実際に見てみましょう。図15-7は以前使用したのと同じシナリオを表しています。

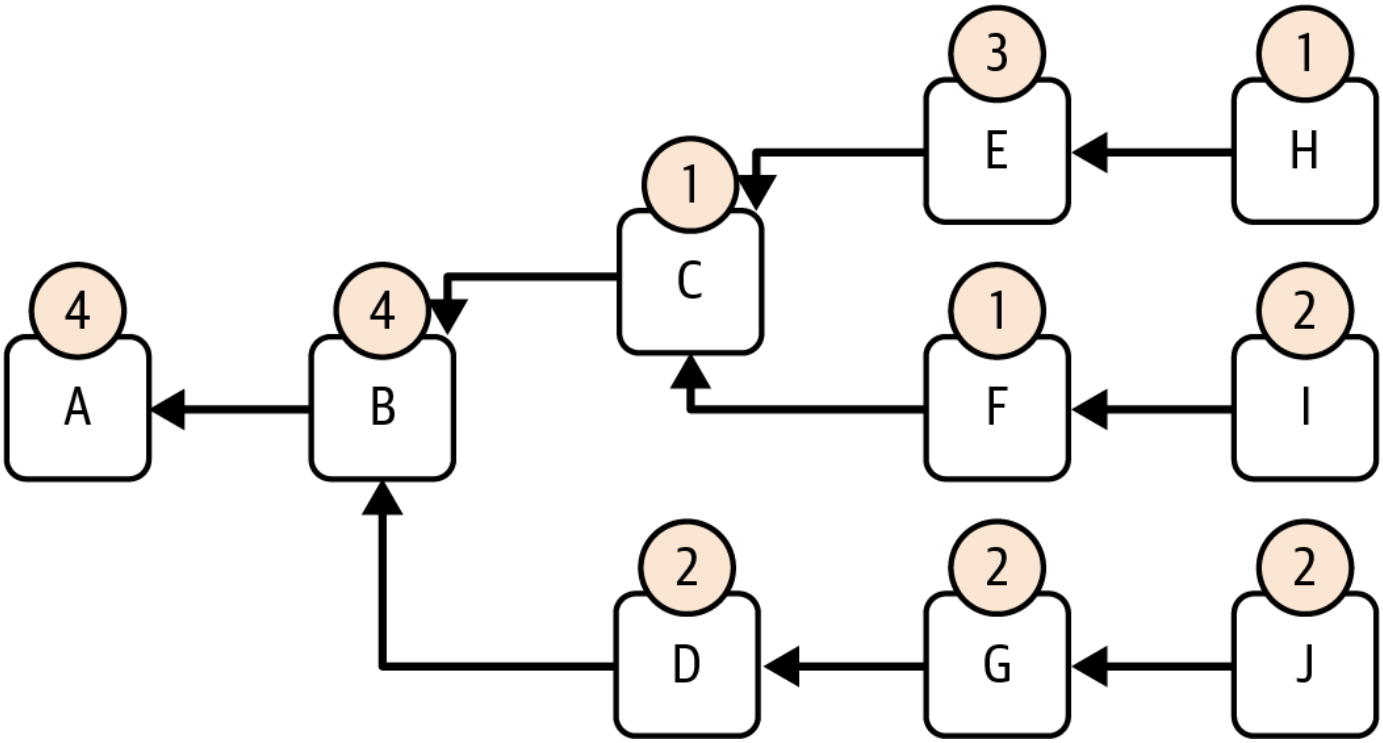


図15-7 LMD-GHOSTのシナリオ

LMD-GHOSTは常に確定されたチェーンの一部とみなされる初期ブロックから開始されます。当初はジェネシスブロックですが、完全なPoSベースのイーサリアムコンセンサスプロトコルでは最後に正当化されたチェックポイントブロックで更新され続けます。これは「Casper FFG：ファイナリティガジェット」で探求する用語です。正当化されたチェックポイントが何かをまだ知らなくても大丈夫です。常に開始ブロックが存在することだけ知っておいてください。

この例ではブロックAが初期ブロックです。自分がアテステーションを投じる必要があるバリデータ、または次のブロックを提案するために選ばれたバリデータだと仮定しましょう。チェーンのヘッドがどのブロックであるかを知るためにLMD-GHOSTを実行し、それに応じてアテステーションを公開するか正しい前のヘッドブロックの上に次のブロックを構築する必要があります。プロトコルのLMDルールに従い各バリデータの最新のアテステーションのみを考慮してこれまでの他のバリデータのアテステーションをすでに収集しています。したがってすべてのブロックのスコアがあり、各バリデータがそれぞれに投じたスコアの合計で構成されています。

この時点でLMD-GHOSTは2つのステップで機能します。

1. 各ブロックのスコアを以前に説明したのと同じ方法ですべての以前のブランチに遡って伝播させ、すべてのブランチにスコアを割り当てます。図15-8はすべてのブランチの最終スコアを示しています。

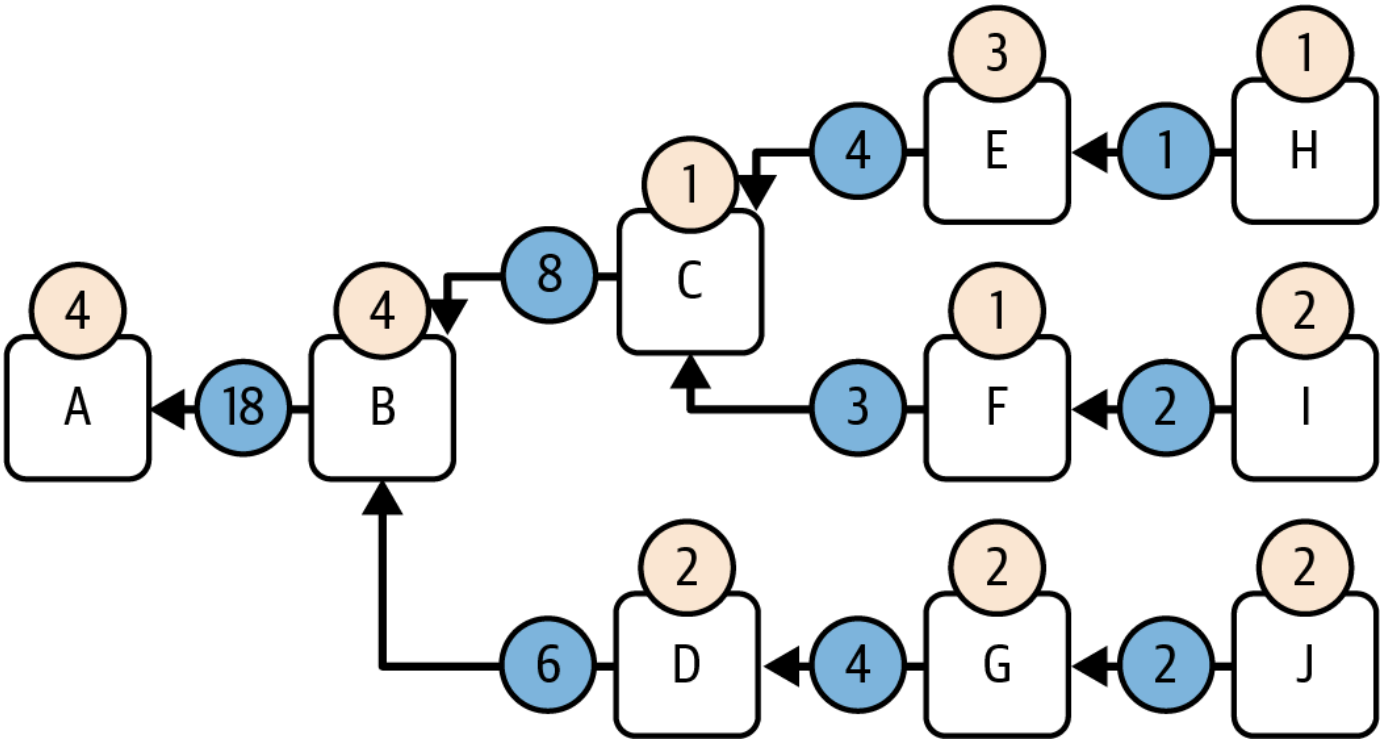


図15-8 計算されたブランチスコア

2. 次に初期ブロックから始めてプロトコルのGHOST部分が貪欲に最高スコアのブランチを選択し、子孫のないブロックに到達するまで進みます。それがLMD-GHOSTフォーク選択ルールによって返されるヘッドブロックです。

この例でステップバイステップで実行してみましょう。LMD-GHOSTは初期ブロックAから開始し、図15-9に示すように選択可能な代替ブランチがないためブランチB→Aを辿ってすぐにブロックBに進みます。

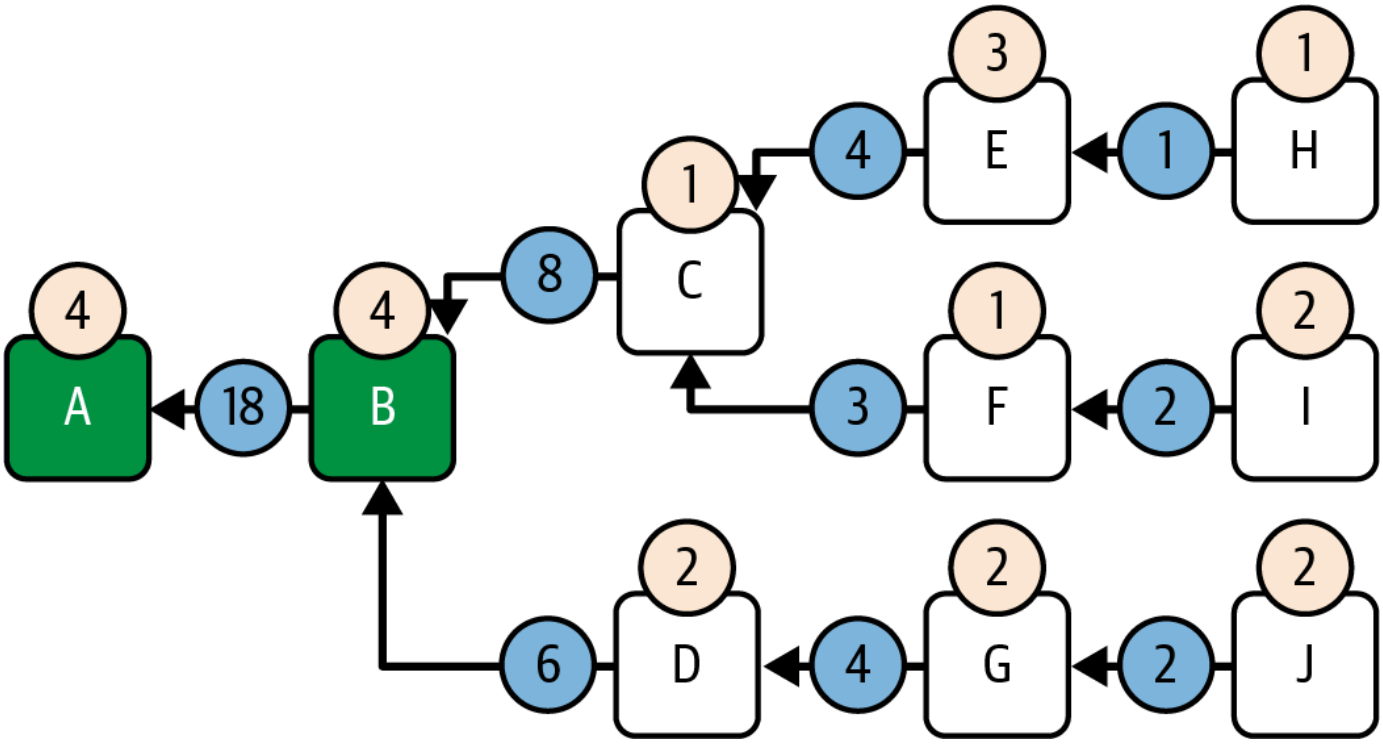


図15-9 LMD-GHOSTステップ1

ここで、2つのブランチから選択できます。

- スコア8のブランチC→B
- スコア6のブランチD→B

図15-10に示すようにGHOSTは最高スコアのブランチC→Bを貪欲に選択します。ブロックDがブロックCよりも高いスコアを持っていても関係ありません。LMD-GHOSTは単一のブロックのスコアではなくそのブロックが属するフォーク全体のスコアを考慮するためです。

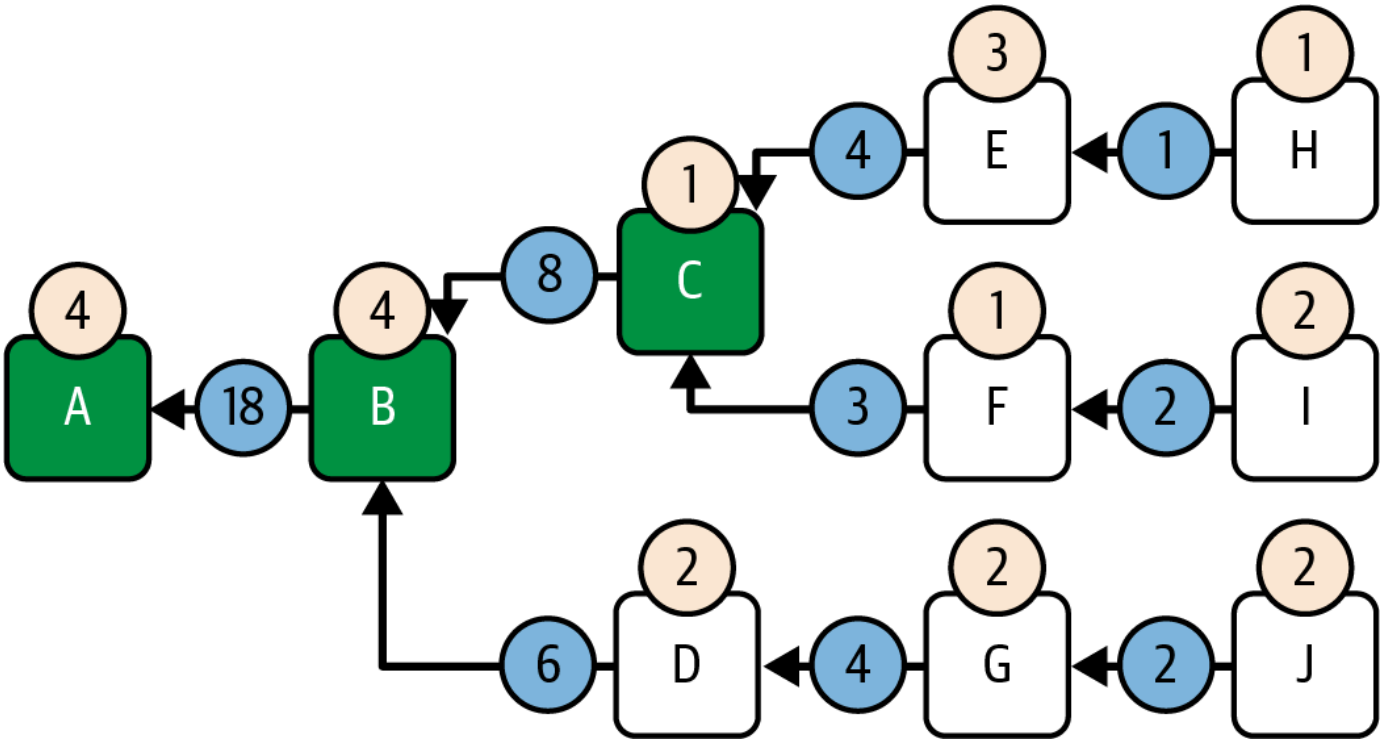


図15-10 LMD-GHOSTステップ2

ここで、2つの異なるブランチから選択できます。

- スコア4のブランチE→C
- スコア3のブランチF→C

図15-11に示すようにGHOSTはブランチE→Cを選択します。

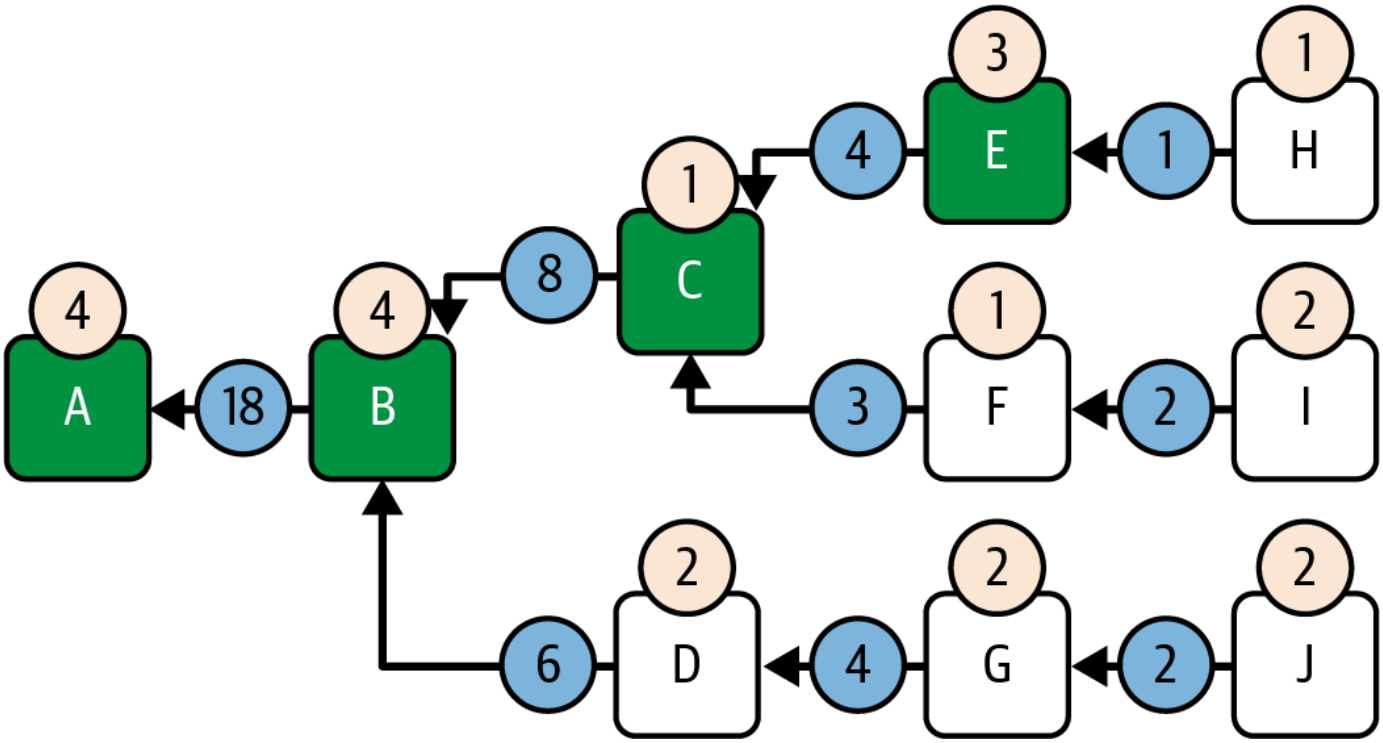


図15-11 LMD-GHOSTステップ3

　この時点で選択できるブランチは1つだけでブランチH→Eなので、図15-12に示すようにGHOSTはそれを選択します。

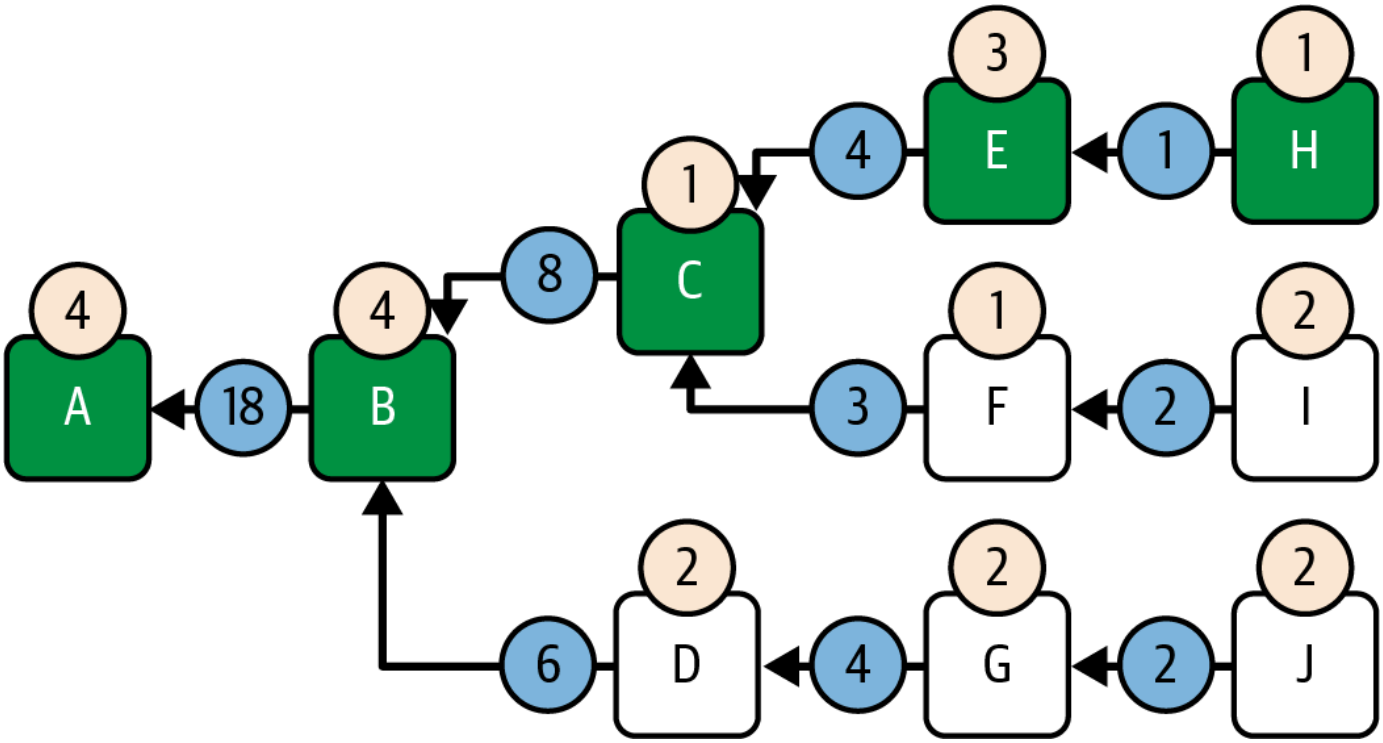


図15-12 LMD-GHOSTステップ4

　ブロックHには子孫がありません。LMD-GHOSTは停止し、チェーンの新しいヘッドブロックとして返します。

インセンティブ

LMD-GHOSTはルールに厳密に従うバリデータに対するさまざまな明示的なインセンティブと、悪意を持って行動する者への罰則を提供します。このセクションでは悪意ある行為者がルールを破ることを防ぎ、善意ある行為者に報いる方法を探ります。

バリデータは2つの異なる職務を遂行する必要があります。

- ブロックの提案
- アテステーションの作成

これらのそれぞれについてLMD-GHOSTは全員がルールに従って行動することを確実にするためのいくつかの方法を含んでいます。

ブロックの提案

バリデータがチェーンに新しいブロックを提案するために選ばれた場合、有効なブロックを1つだけ作成しなければなりません。そうすることで図15-13に示すようにバリデータは作成したブロックに含まれるすべてのトランザクションの優先手数料の合計に加えて新たに発行されたETHを獲得します。

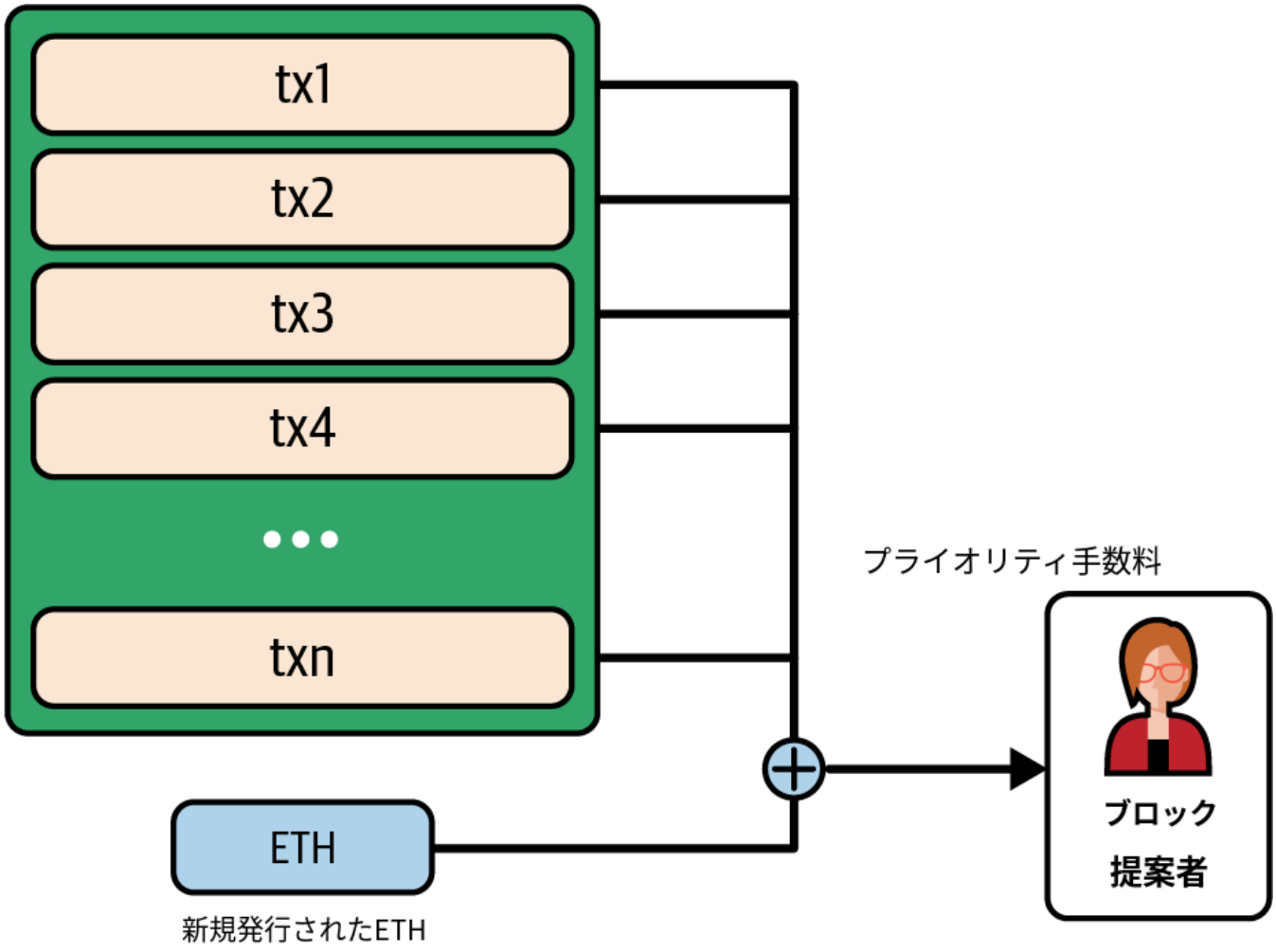


図15-13 ブロック提案の報酬

バリデータが複数のブロックを作成して不正を試みた場合、プロトコルはステークの一部をスラッシングして明示的に罰します。実際、バリデータセットの一員になるには担保として一定量のETH（少なくとも32 ETH）をステークする必要があります。図15-14に示すようにこのステークはプロトコルがステークからETHをスラッシング（つまり除去）して罰するためにも必要です。

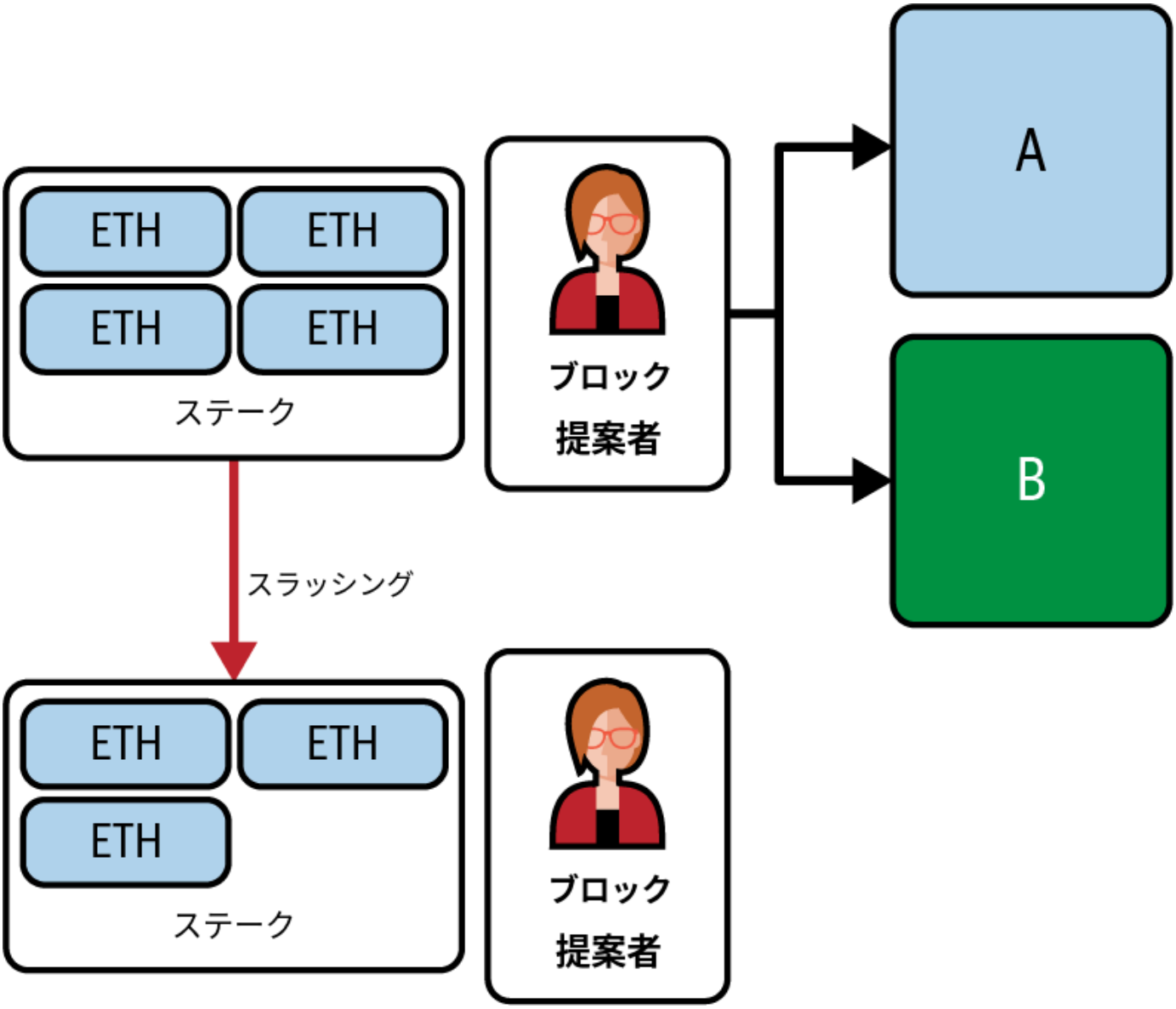


図15-14 ブロック提案のスラッシング

注記

ここで注目すべきは明示的な罰則がイーサリアムのPoSコンセンサスプロトコルとビットコインのPoWとの大きな違いであることだ。ビットコインのマイナーは自身のブロックが最も重いチェーンの一部になった場合に報酬を得る。同一のブロック番号に対して複数のブロックを作成しても明示的な罰則はない。

なぜか。PoWベースのシステムでは有効なブロックを作成するためにある程度の作業（PoW自体）が必要である。マイナーが複数のブロックを作成した場合、最終的に最も重いチェーンに含まれるのは1つのブロックだけであるため単に時間とお金を無駄にしているだけで、報酬を得られるのは1つだけである。

イーサリアムのPoSプロトコルでは有効なブロックを作成するためにPoWを実行する必要がない。つまり複数のブロックを作成することはバリデータにとってほぼ無料である。そのためこの方法で不正を試みる者に対する明示的な罰則が必要になる。

アテステーションの作成

バリデータがアテステーションの形でネットワークのビューを共有するために選ばれた場合、有効なアテステーションを1つだけ公開しなければなりません。そうすることで図15-15に示すように小さな手数料（ブロック提案者が得るものよりもはるかに少ない）を獲得します。

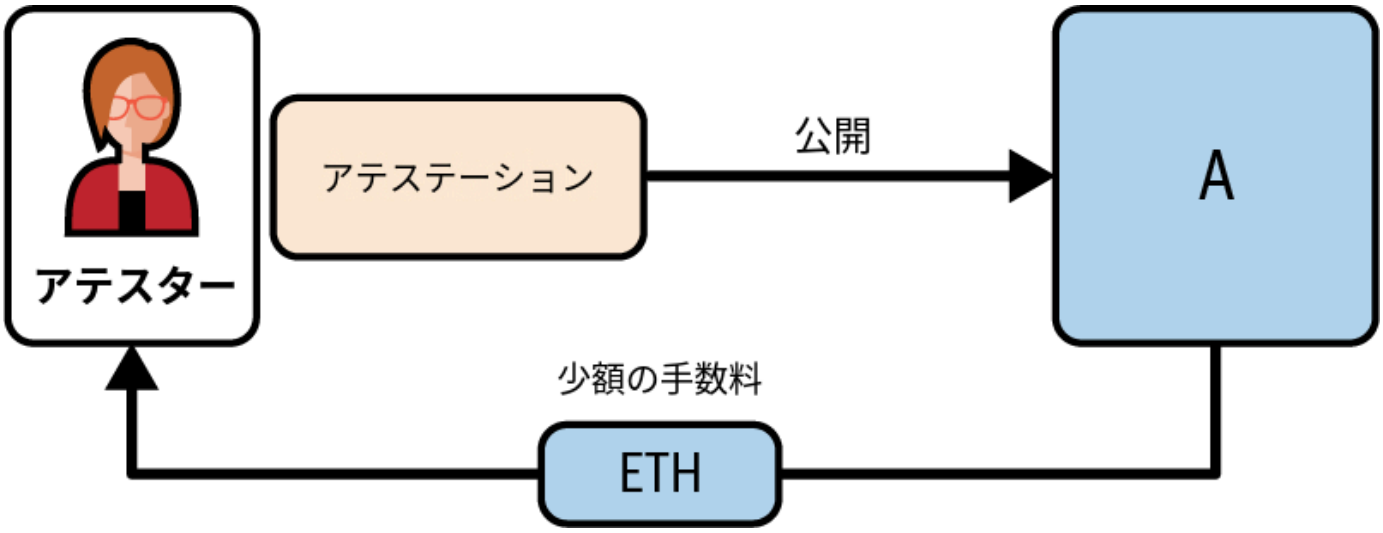


図15-15 アテステーションの報酬

バリデータが複数のアテステーションや矛盾するアテステーションを作成して不正を試みた場合、図15-16に示すようにプロトコルはステークの一部をスラッシングして明示的に罰します。

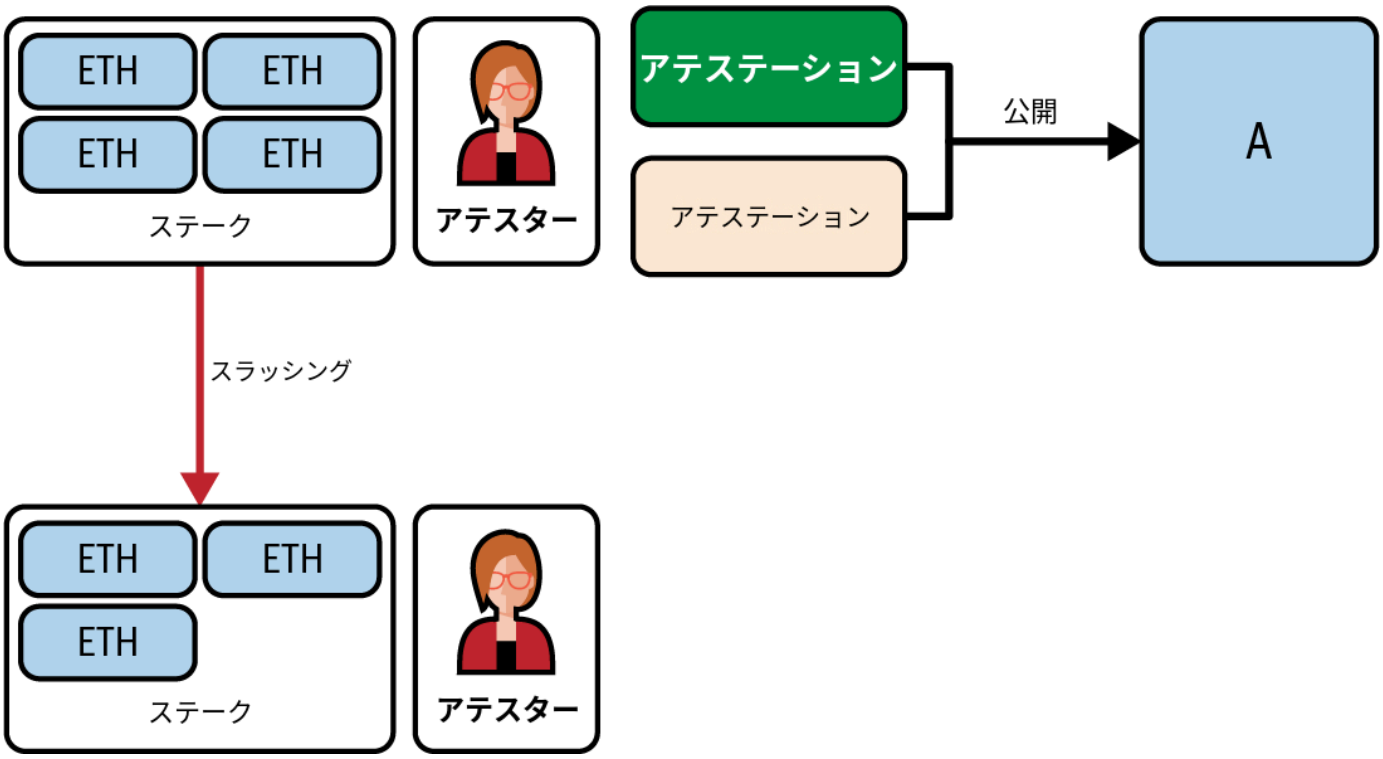


図15-16 アテステーションのスラッシング

バリデータが長期間にわたって悪意ある行動を続けた場合、プロトコルはバリデータセットから強制的に退去させることができます。

Casper FFG：ファイナリティガジェット

Casper FFGは一種のメタコンセンサスプロトコルです。基盤となるコンセンサスプロトコルの上にオーバーレイとして実行されファイナリティを追加できます。

イーサリアムのPoSコンセンサスでは基盤となるプロトコルはLMD-GHOSTでありファイナリティを提供しません。ファイナリティはブロックがチェーンで確認されると不可逆になることを保証します。確認されたブロックは永遠にチェーンの一部となります。つまり本質的にCasper FFGはファイナリティガジェットとして機能し、LMD-GHOSTにファイナリティを追加するために使用されます。

Casper FFGはPoSプロトコルでは参加者が誰であるかが既知であるという事実を活用します。ステークされたEtherを管理するバリデータです。これにより正直なバリデータの過半数の投票、より正確にはステークの過半数を管理するバリデータの投票を見たかどうかを判断するために投票の集計を使用できます。以下の説明ではすべてのバリデータの投票は管理するステークの値で重み付けされますが、簡潔にするため毎回これを明記することはしません。

Casper FFGはすべての古典的なビザンチン障害耐性（BFT）プロトコルと同様にバリデータの3分の1未満が障害を起こしているか敵対的である限りファイナリティを保証できます。正直なバリデータの過半数がブロックを確定と宣言するとすべての正直なバリデータが同意し、そのブロ

ックは不可逆になります。正直なバリデータが全体の3分の2以上を構成することを要求することでコンセンサスが正直な多数派のビューを正確に反映することが保証されます。注目すべきことにCasper FFGは従来のBFTプロトコルとは異なり、バリデータの3分の1以上が侵害された場合でも経済的ファイナリティ（詳細は「説明責任のあるセーフティともっともらしいライブネス」を参照）を提供します。

エポックとチェックポイント

Casper FFGは図15-17に示すように大規模なバリデータセットを効率的に管理するために投票を32スロットに分散し、エポック内のバリデータの3分の2以上の投票を要求することでコンセンサスを確保します。エポックは32のスロットに分割され各スロットには通常ブロックが含まれます。エポックの最初のスロットが*チェックポイント*です。

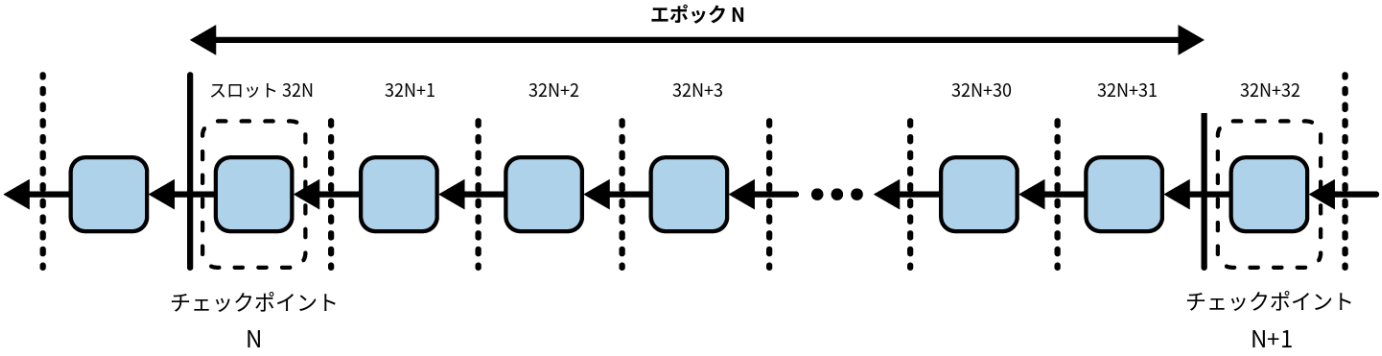


図15-17 エポックとチェックポイント

バリデータは統一された投票の焦点を維持するためにエポックごとに1回、最初のスロットであるチェックポイントに投票します。このプロセスは効率のためにCasper FFGとLMD-GHOSTの両方の投票を組み込んでおり、Casper FFGの文脈ではエポック全体ではなくチェックポイントの確定を目指しています。ファイナリティはチェックポイントとその前のコンテンツに及びます。

ジャスティフィケーションとファイナライゼーション

Casper FFGは従来のBFTプロトコルと同様に2つの段階でネットワークの合意を確保します。まずバリデータは提案されたチェックポイントに関する自身のビューをネットワークにブロードキャストし収集します。大多数が同意すればチェックポイントは*正当化*され、暫定的な合意が示されます。次のラウンドでバリデータが正当化されたチェックポイントに対する広範な支持を確認すると*確定*が達成されます。つまり全員が同意し不可逆であることが確認されます。このプロセスはネットワークの一貫性とセキュリティを確保するための協力的な取り組みであり、チェックポイントが特定の時間枠内で正当化されその後確定されることを目指してコンセンサスメカニズムの信頼性を向上させます。

ソースとターゲット、リンクと競合

Casper FFGでは投票はソースとターゲットのチェックポイントで構成され、ブロックチェーンの異なる時点での状態に対するバリデータのコミットメントを表します。これらの投票はリンクされたペアとして投じられ、バリデータの現在のコンセンサスポイントと提案されたコンセンサスポイントを示します。ソース投票はチェックポイントに対する広範な支持のバリデータによる承認を反映し、ターゲット投票は他のバリデータからの同様の支持を条件とする新しいチェックポイントへの条件付きコミットメントを表します。この二重投票システムはブロックの確定に向けた構造化された進行を促進し、ネットワークの整合性と継続性を確保します。

スーパーマジョリティリンク

Casper FFGにおいてソースとターゲットのチェックポイント間の*スーパーマジョリティリンク*（$s \rightarrow t$）は、バリデータの3分の2以上がステーク量の重み付けで同じリンクを承認しその投票がブロックチェーンに含まれた時に確立されます。このメカニズムは広範なバリデータの合意を通じてチェックポイントのシーケンスを検証することでコンセンサスとセキュリティを確保します。

ジャスティフィケーション

Casper FFGではノードがバリデータの過半数があるチェックポイントから別のチェックポイントへの遷移に同意していることを確認すると古いチェックポイントを正当化します。図15-18に示すようにノードがバリデータセットのかなりの部分からのコンセンサスの証拠を見たことを意味し、代替パスに対する圧倒的なコンセンサスが示されない限り以前の状態に戻らないというコミットメントを行います。

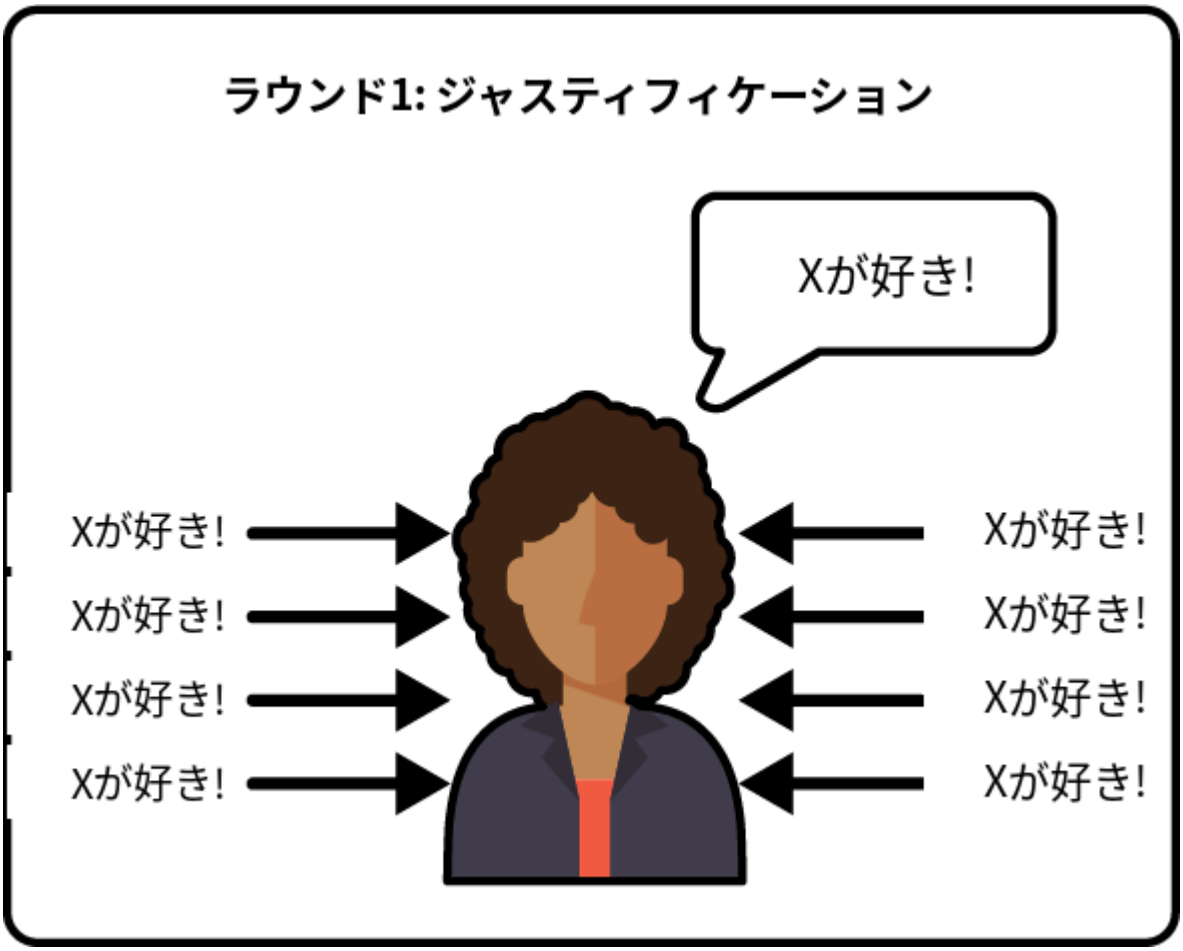


図15-18 ジャスティフィケーションプロセス[1]

[1] ノードはスーパーマジョリティリンクCN → CN + 1を確認したため、CN + 1を正当化されたとマークする。CN + 1はチェックポイントツリーにおけるCNの直接の子であるため、CNも確定されたとマークする。確定されたチェックポイントはクロスハッチングされ、Fとマークされている。

ファイナライゼーション

図15-19に示すようにノードが正当化されたチェックポイントからその直接の子へのコンセンサス（スーパーマジョリティリンク）を確認すると親チェックポイントを確定します。これはバリデータの強力な過半数の支持に裏付けられた、この時点から戻らないというネットワーク全体のコミットメントを示します。ファイナライゼーションはバリデータにとって重大な結果なしにはリバーサルを防ぎ、そのチェックポイント以降のブロックチェーンの履歴を不変にすることでネットワークの安定性とセキュリティを確保します。

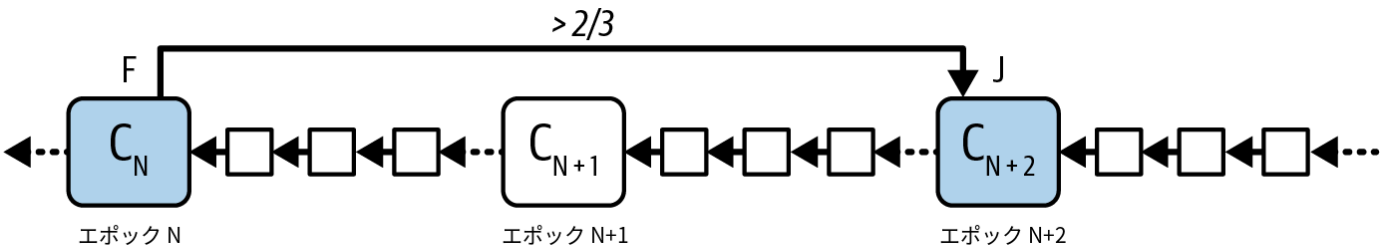


図15-19 ファイナライゼーションプロセス

スラッシング

Casper FFGはプロトコルルールの違反に対してバリデータにペナルティを課すスラッシングメカニズムを実装しています。この強制措置は、矛盾するチェックポイントの確定などブロックチェーンの整合性を損なう行動を抑止します。違反の検出、特にサラウンド投票（「フォーク選択ルール」を参照）のような複雑なシナリオの検出は技術的に困難であるため、外部の専門サービスに依存する場合があります。スラッシングの結果は不正行為の深刻さとネットワーク全体の健全性に比例し、ペナルティは特定の時間枠内での集団的な行動に基づいてスケーリングされバリデータの行動における公平性と説明責任を確保します。

フォーク選択ルール

Casper FFGは従来のLMD-GHOSTフォーク選択ルールを変更しノードが最も高い正当化されたチェックポイントを持つチェーンを優先することを義務付けます。このチェックポイントが事実上LMD-GHOSTプロトコルの開始ブロックとなります。LMD-GHOSTプロトコルのアプローチの進化であるこの適応はバリデータのスーパーマジョリティによって合意されたチェックポイントにコミットすることでネットワークがファイナリティを達成することを保証します。チェックポイントが正当化されるとネットワークはそれを超えて巻き戻せなくなり、ブロックチェーンのセキュリティと安定性を強化します。このルールはネットワークのライブネスを維持するようにも設計されておりCasperの基本的な目標と一致しています。

Casperの戒律

Casper FFGではチェックポイントがネットワークのコンセンサスとセキュリティを確保するための中心的な役割を果たしています。チェックポイントはブロックチェーンの進行とともに増加するエポック番号でマークされます。バリデータは厳格な投票ルールに従わなければなりません。同一のチェックポイントに対して異なる結果に投票することはできません。つまり図15-20に示すように二重投票は禁止されています。この投票ルールがなければリオーグの可能性がはるかに高くなりチェーンは非常に不安定になります。

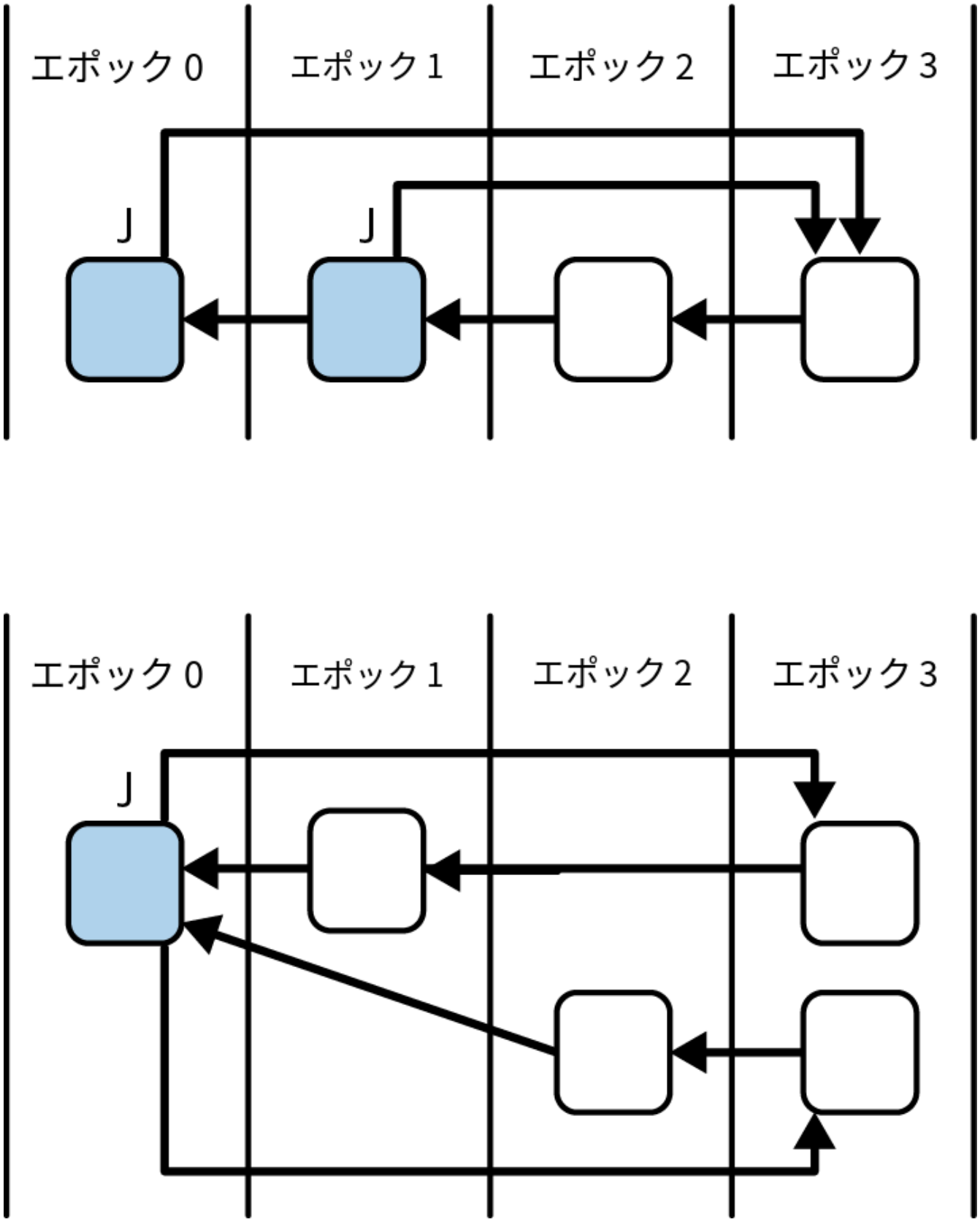


図15-20 二重投票禁止ルール

バリデータはまた以前のコミットメントと矛盾すると解釈される可能性のある投票の作成を避けなければなりません（サラウンド投票の禁止）。図15-21に示すようにこれらの原則に違反するとスラッシングが適用されます。これはコンセンサスメカニズムの整合性と説明責任を維持するために設計されたペナルティです。

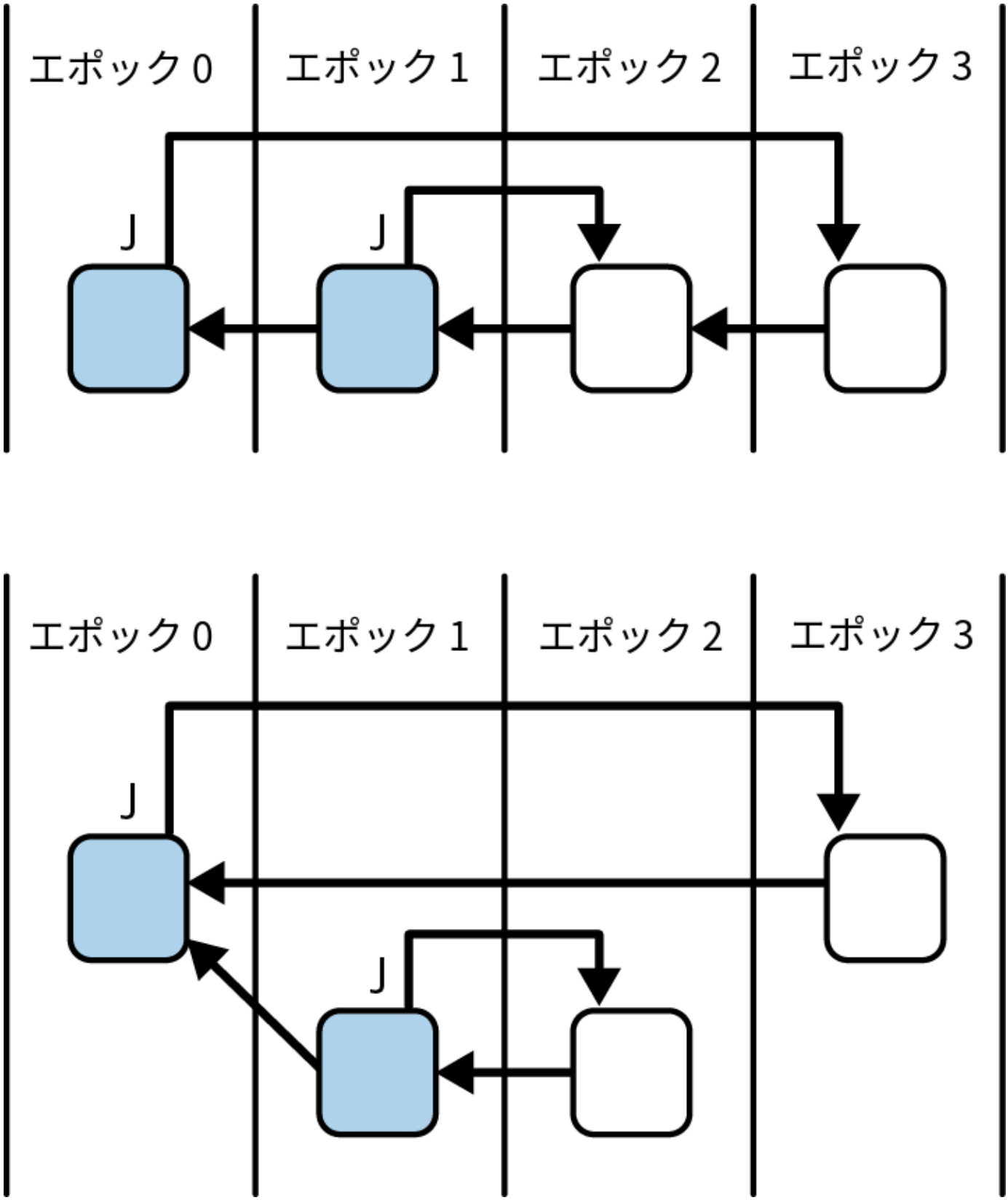


図15-21 サラウンド投票禁止ルール

説明責任のあるセーフティともっともらしいライブネス

Casper FFGコンセンサスプロトコルは、古典的なコンセンサスにおけるセーフティとライブネスに似てはいるものの異なる2つの保証を提供します。*説明責任のあるセーフティともっとも*

らしいライブネスです。

説明責任のあるセーフティと経済的ファイナリティ

Casper FFGの説明責任のあるセーフティの証明は、バリデータの3分の1以上がプロトコルルールに違反しない限り、矛盾するチェックポイントが確定されることはないことを示しています。敵対的なバリデータが3分の1未満であれば確定されたチェックポイントは不可逆であり、ネットワークセキュリティと不正行為に対する経済的ペナルティの両方が機能します。

Casper FFGにおける経済的ファイナリティは潜在的な攻撃者にコストを課し、プロトコルルールだけでなく経済的な抑止力を通じてもセキュリティを強制します。矛盾するチェックポイントを確定してネットワークを弱体化しようとするバリデータはステークのかなりの部分を失う厳しいペナルティに直面します。このアプローチは経済的な結果の層を追加することで従来のコンセンサスメカニズムとは異なり、ブロックの確定が悪意ある行為者にとって大きなコストを伴うことを保証し攻撃に対するブロックチェーンの整合性と回復力を強化します。

もっともらしいライブネス

Casper FFGはネットワークがアクティブなまま維持され正直なバリデータがペナルティを受けることなく常にコンセンサスに到達できることを保証し、もっともらしいライブネスの概念を体現しています。バリデータのスーパーマジョリティが正直であればプロトコルは新しいチェックポイントの正当化と確定を継続でき、スラッシングへの恐れから進行が停止するデッドロックシナリオを回避します。この原則はネットワークの回復力と継続的な運用を保証し、困難な条件下でもコンセンサスを維持するCasperの適応性を示しています。

実践的な例：チェックポイントのライフサイクル

イーサリアムのCasper FFGメカニズムにおけるチェックポイントのライフサイクルを一緒にたどりましょう。

イーサリアムのバリデータコミュニティは非常に大きく、潜在的に数十万人が参加しています。これらの投票をすべて一度に処理することは現実的ではありません。では、どのように管理するのでしょうか。

投票はエポックと呼ばれる期間に分散され各エポックは32のスロットに分割され各スロットは12秒間続きます。こうして各バリデータはエポックごとに正確に1回投票し、バリデータセットの約32分の1が各スロットで投票します。図15-22はバリデータのプールを示しています。

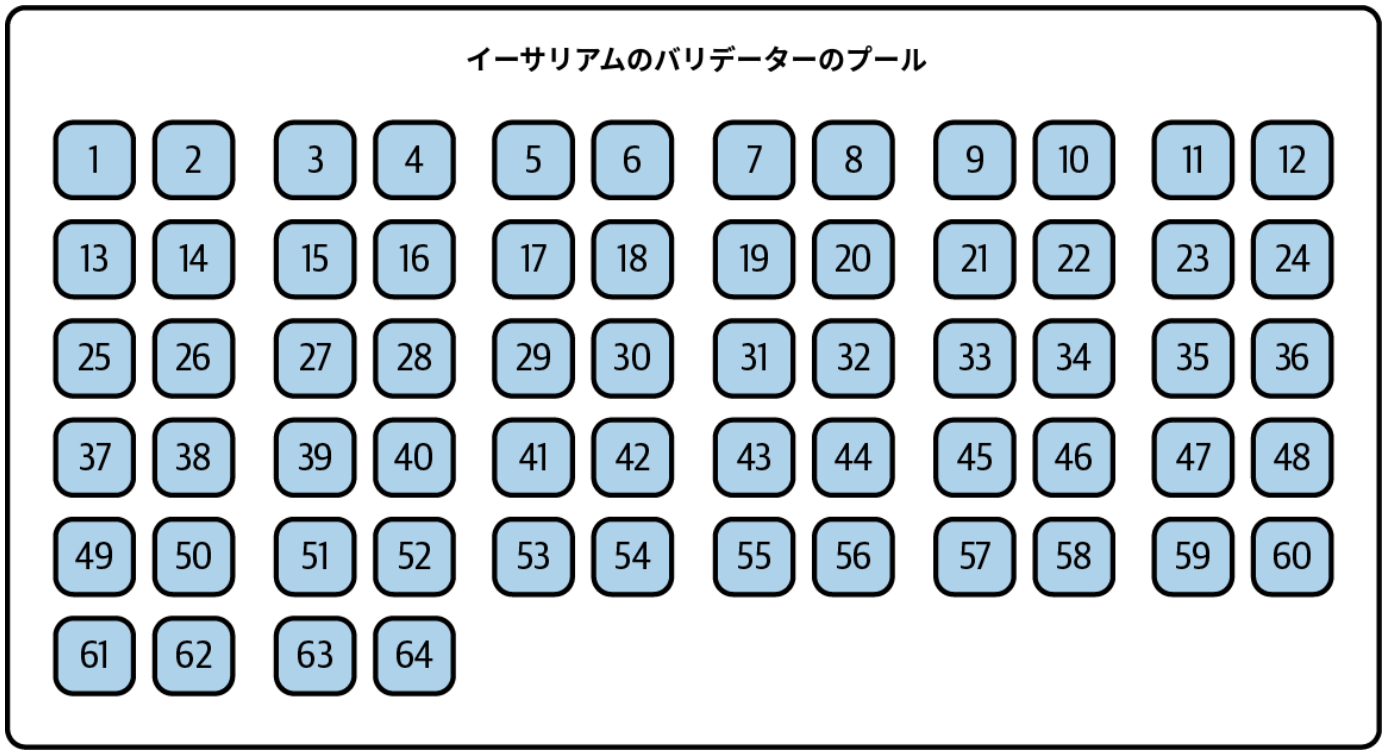


図15-22 バリデータプール

この例ではバリデータの数は実際のイーサリアムネットワークよりもはるかに限られていますが、64のノードがあり32のグループに分割されています。図15-23に示すように各グループがエポック内の1つのスロットに投票します。

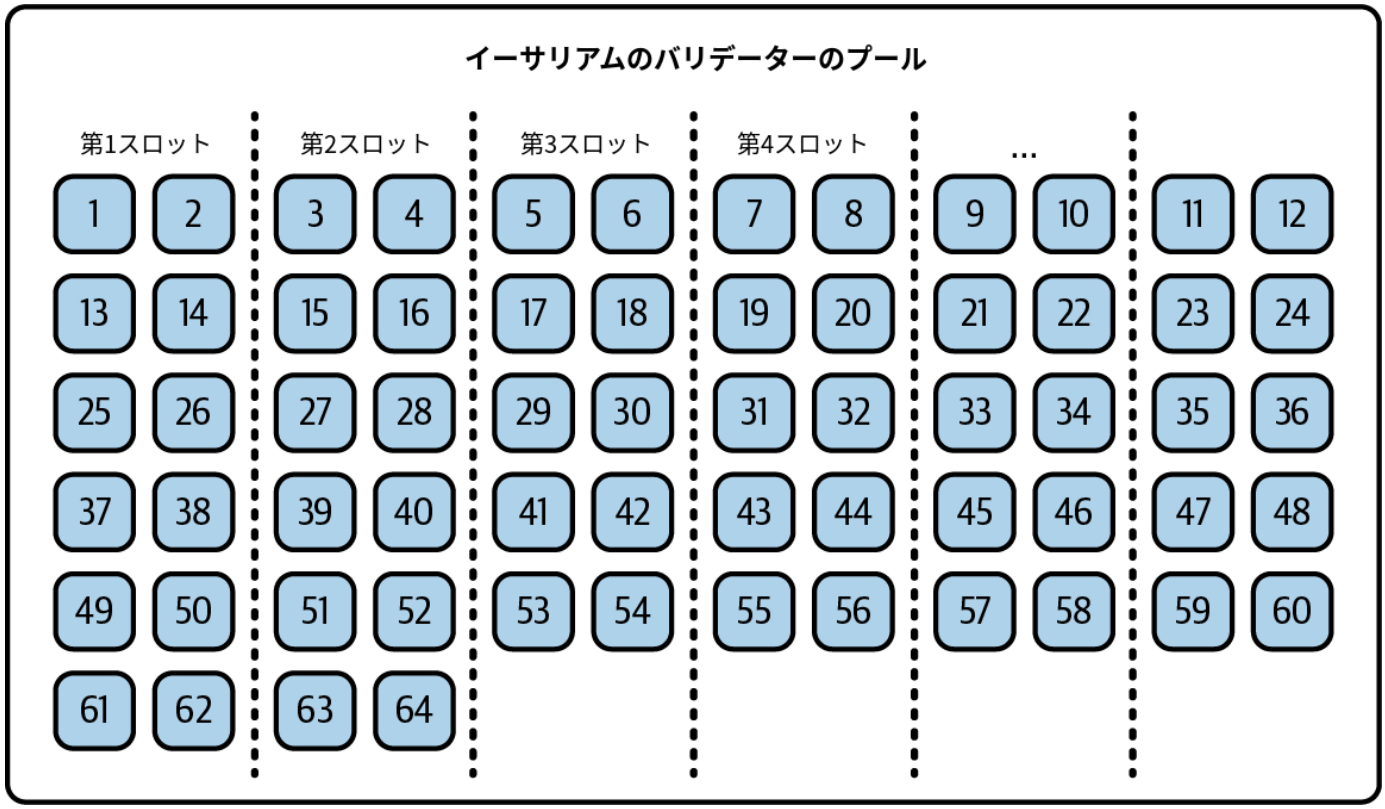


図15-23 グループに分割されたバリデータ

では何に投票するのでしょうか。チェックポイント、具体的にはエポックの最初のスロットに投票します。[2]このチェックポイントは異なる時点で投票するバリデータの共通の目標として機能します。

[2] チェックポイントは常にエポックの最初のスロットであるが、チェックポイント自身のブロックが欠落している場合、そのブロックハッシュはより以前のブロックのものになる可能性がある。

> **注記**
>
> ここで重要な点を明確にしておく。エポックの確定について語ることが多いが、技術的にはチェックポイントつまりこれらの最初のスロットを確定しているのである。チェックポイントが確定されるとそのスロットまで（そのスロットを含めて）のすべてが確定され安全で変更不可能になる。

エポック（この場合はエポックN）の表現を図15-24に示します。チェックポイントNはスロット32Nです。そのチェックポイントが確定されるとスロット32N-1とそれ以前のすべてのスロットが確定されたとみなされます。

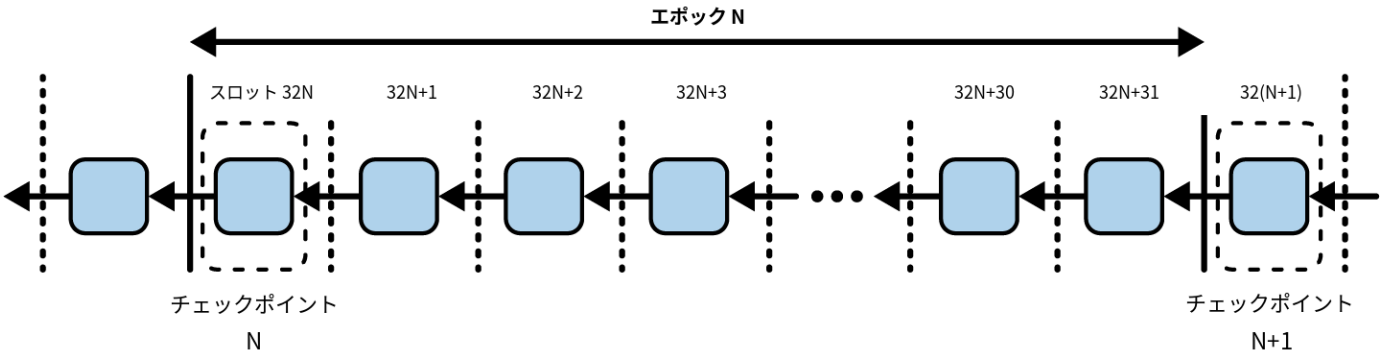


図15-24 エポックの表現

このセキュリティを達成するプロセスは厳格であり従来のBFTコンセンサスメカニズムに似ています。次のセクションではその仕組みを説明します。

第1ラウンド：ジャスティフィケーション

バリデータはそれぞれ現在のエポックのチェックポイントに関する自身のビューをネットワークにブロードキャストします。次にネットワークのスーパーマジョリティが自分の見方に同意しているかどうかを確認します。同意している場合このチェックポイントは「正当化」されます。この段階でバリデータはネットワークの大多数がこのチェックポイントの確定を支持していると信じていますが、全員が同意しているかどうかはまだ完全には確信していません。

重要な問題はバリデータがネットワーク上の悪意ある行為者がネットワークの状態について虚偽の情報を提供していないかどうかをまだ確信できないことです。ある者にはあることを言い、別の者には別のことを言う可能性があります。これは見落とされがちな非常に重要なポイントです。すべての参加者が常に正直であればジャスティフィケーションはファイナライゼーションを意味し、2ラウンドのプロセス全体を回避できるでしょう。

バリデータがチェックポイントを正当化する際、図15-25に示すようにその特定のチェックポイントに対してネットワークの3分の2からの承認を受けています。しかしこの最初のラウンドの承認はバリデータ自身にとってのローカルなものにすぎません。特に敵対的な条件下では十分なバリデータがコンセンサスに達していない可能性があります。Algorand、Dfinity、Cosmosで使用されているような従来のPBFTスタイルのコンセンサスメカニズムは、この段階で停止しライブネスを失います。一方イーサリアムは継続します。チェックポイントを正当化できない場合でも問題ありません。単に次のチェックポイントの正当化を試みるだけです。これが機能するのはイーサリアムがライブネスのためにLMD-GHOSTに依存しているからであり、Casper FFGは単なるオーバーレイつまり「あれば良い」ものだからです。したがってファイナリティが一時的に停止しても重大な問題にはなりません。

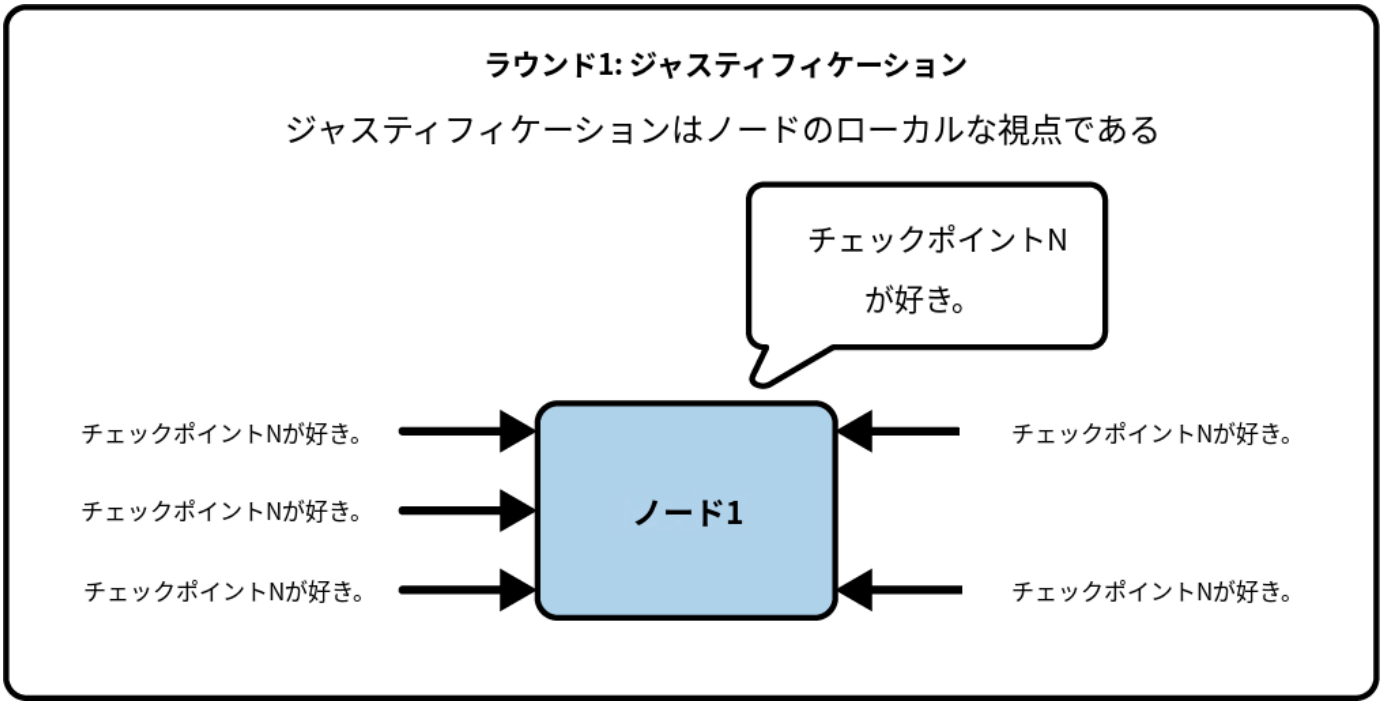


図15-25 ジャスティフィケーションラウンド

第2ラウンド：ファイナライゼーション

バリデータはスーパーマジョリティからこのチェックポイントも支持しているという報告を受けたことを発表します。ネットワークの残りの部分がこのスーパーマジョリティが確かに存在することを確認しているかどうかを再度チェックします。確認された場合、図15-26に示すようにバリデータはチェックポイントを「確定」できます。ファイナライゼーションは強力なステップです。正直なバリデータがこのチェックポイントを巻き戻すことは二度とないことを意味しま

す。ローカルなビューではまだ確定としてマークしていないかもしれませんが、少なくとも正当化としてマークしており罰則のある行為なしには取り消せません。

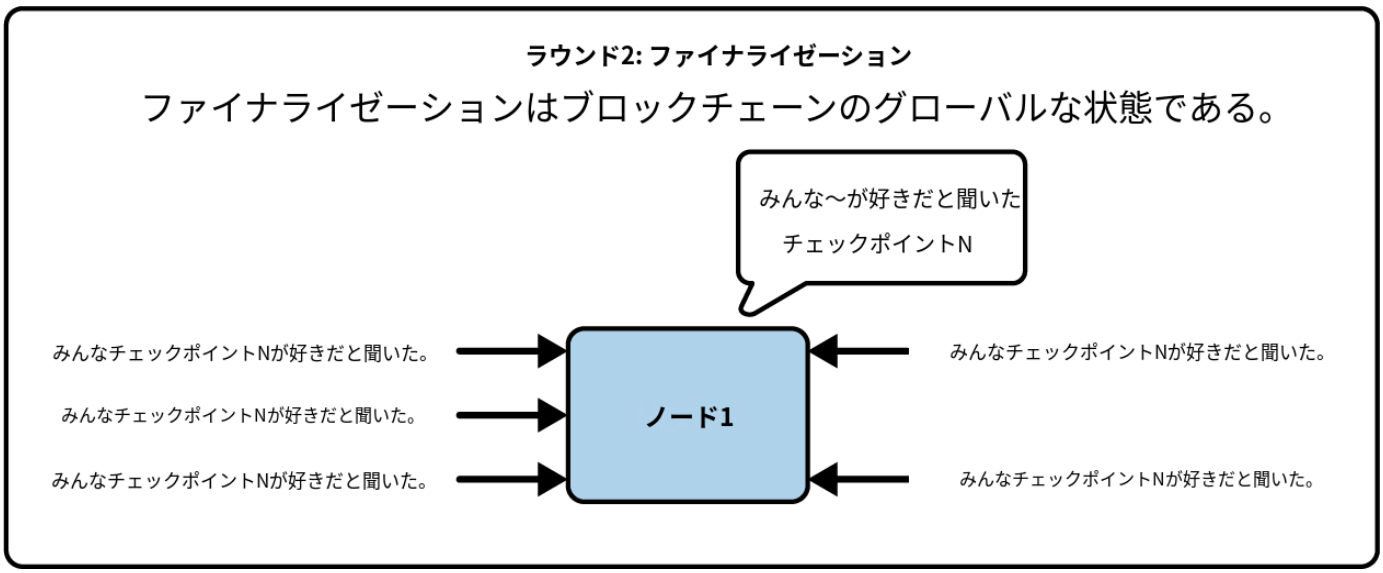


図15-26 ファイナライゼーションラウンド

実際には各ラウンドは理想的には1エポックにわたり、チェックポイントを正当化するのに1エポック、確定するのにもう1エポックかかります。合計で約12.8分です。しかしCasper FFGのパイプライン設計のおかげで1エポックごとつまり6.4分ごとにチェックポイントを確定できます。

> **注記**
>
> 外部の視点からは投票がエポックの進行とともに徐々に蓄積されるため、重大なチェーンのリオーグがないと仮定すれば12.8分が経過する前にチェックポイントが確定される可能性の兆候を見ることができる場合もある。しかしジャスティフィケーションとファイナライゼーションの公式なプロトコル内のアクションはエポックの終わりにのみ発生する。

チェックポイントのジャスティフィケーションとファイナライゼーションの過程では多くのことがうまくいかない可能性があります。2つの重要なケースとこのフレンドリーなファイナリティガジェットがそれらをどのように処理するかを分析しましょう。

矛盾するジャスティフィケーション

チェックポイントに「正当化」と「確定」の両方のステータスが必要な理由を考えることは有益です。3分の2のスーパーマジョリティが賛成票を投じた時点で、チェックポイントを直接確定するだけでは十分ではないのでしょうか。

その区別は次のとおりです。ジャスティフィケーションはローカルな合意に関するものでありファイナリティはグローバルなコンセンサスに関するものです。

チェックポイントを正当化するとはバリデータとしての自分がバリデータの3分の2からそのチェックポイントを承認する確認を受けたことを意味します。しかしこの承認は自分のローカルな視点のみを表します。他のバリデータが異なる情報を持っている可能性があり確信は持てません。この不確実性にもかかわらず正直なバリデータとして自分のローカルなデータに基づいて正当化したチェックポイントを決して取り消さないことにコミットします。

一方チェックポイントの確定はこれをさらに一歩進めます。バリデータの3分の2から、自身もまたピアの3分の2からチェックポイントの有効性を確認する報告を受けた保証を得た時に発生します。これは自分のローカルなビューだけでなくネットワークのスーパーマジョリティがこのチェックポイントを承認しコミットしていることを意味します。この広範なコンセンサスがチェックポイントをグローバルな取り消しから保護します。したがって確定されたチェックポイントはローカルに認識されるだけでなくグローバルに保護されています。

コンセンサスプロセスをより良く理解するために極端なシナリオを探りましょう。図15-27に示すように4人のバリデータA、B、C、Dがいると仮定します。全員が正直ですが運用するネットワークは無期限の遅延が発生する可能性があります。この例のために各ブロックの高さにチェックポイントがあると想像してください。

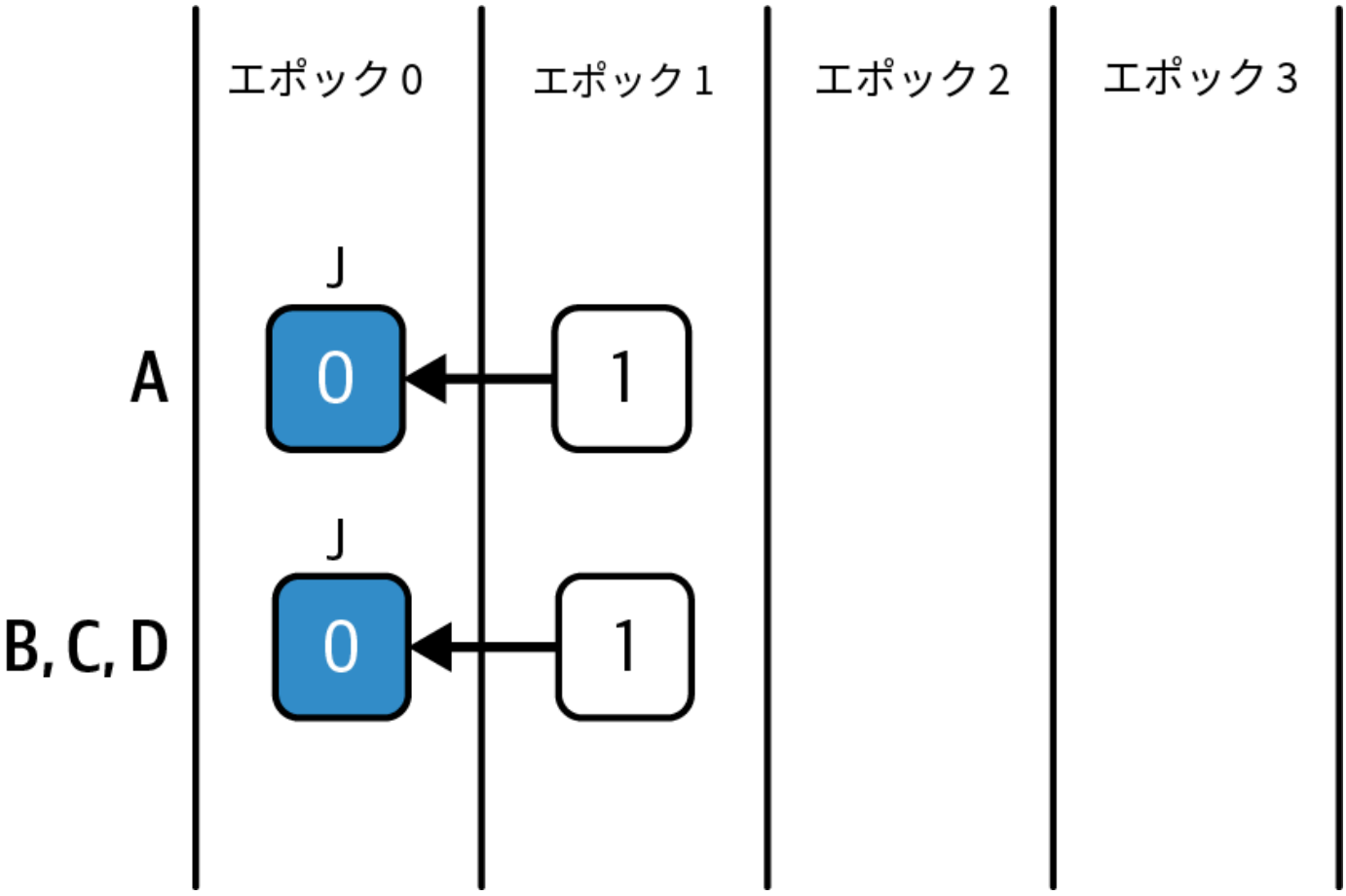


図15-27 4人のバリデータのシナリオ

シナリオのすべてのバリデータはブロック0を持っているためチェックポイント0を正当化できます。したがってソースである0はすべての4人のバリデータにとってローカルに正当化されており1がターゲットです（「ソースとターゲット、リンクと競合」を参照）。

ここでAのネットワーク接続がひどく遅延しておりブロックの提案者としても選ばれたと想像しましょう。Aはエポック2でブロックを提案します。このブロックにはチェックポイント1を正当化するための4つの投票すべてが含まれていますが、ネットワーク接続がひどく遅延しているため他のバリデータはそれを見ることがありません。

Aはソース0とターゲット1の間にスーパーマジョリティリンク（「スーパーマジョリティリンク」を参照）を持っているためチェックポイント0を確定しチェックポイント1を正当化します。一方B、C、Dは現在のエポックで投票を見ていないためチェックポイント0のみが正当化されたままです。図15-28に示すようにB、C、Dはこのエポックで空のチェックポイント（チェックポイントX）にも投票します。

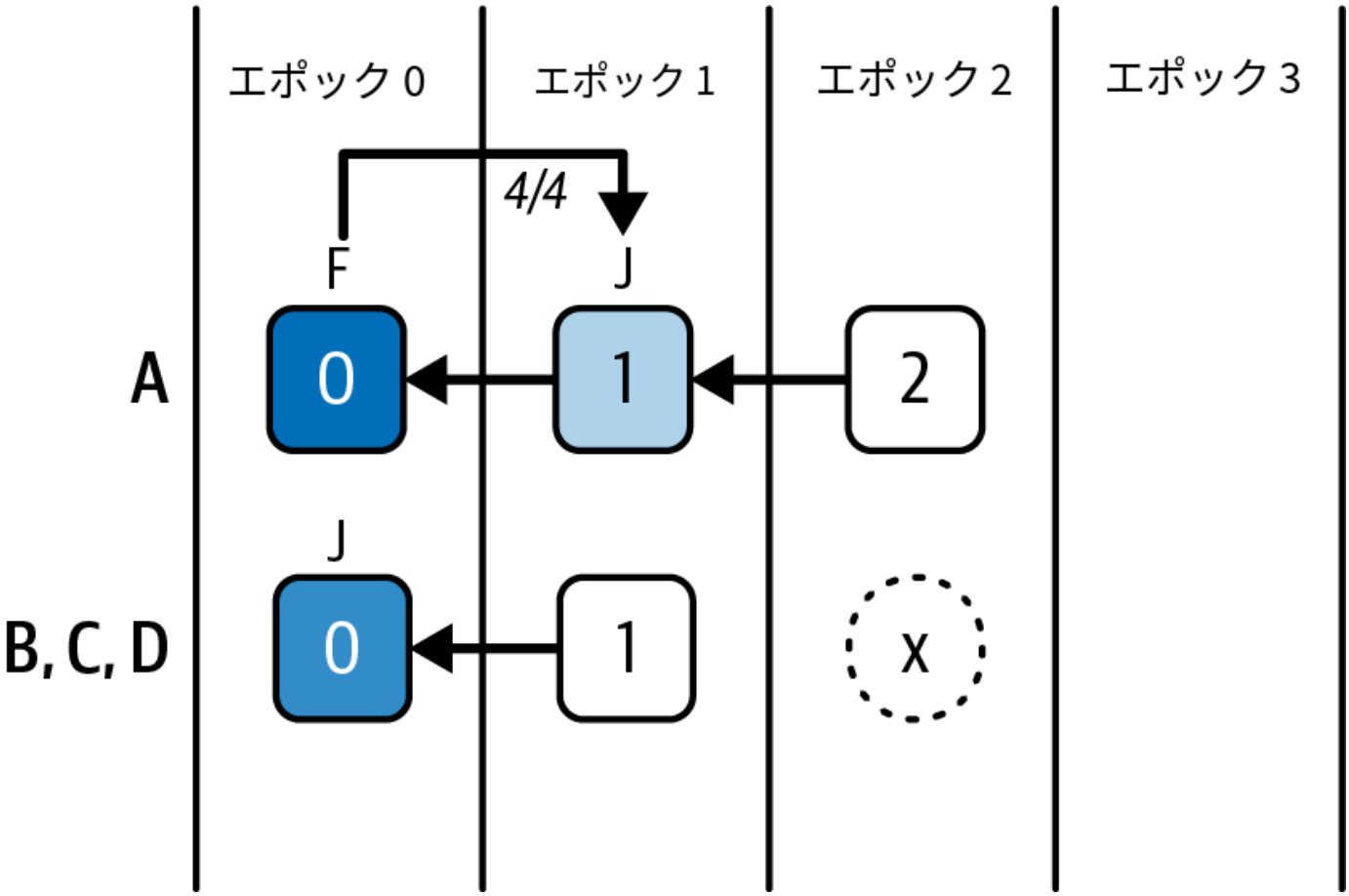


図15-28 ネットワーク遅延シナリオステップ1

エポック3ではB、C、Dの中から1人のバリデータがブロックを提案するために選ばれます。

このブロックにはソースがチェックポイント0、ターゲットがチェックポイントXの3つの投票が含まれています。したがって0とXの間にスーパーマジョリティリンクがあり、バリデータB、C、Dはチェックポイント0を確定しチェックポイントXを正当化できます（図15-29を参照）。一方Aは自身のローカルビューでは1が正当化されており取り消せないためこのブロックを無効とみなします。バリデータAのチェーンが継続するための唯一の解決策はメモリを削除してネットワークの残りの部分と再同期することです。

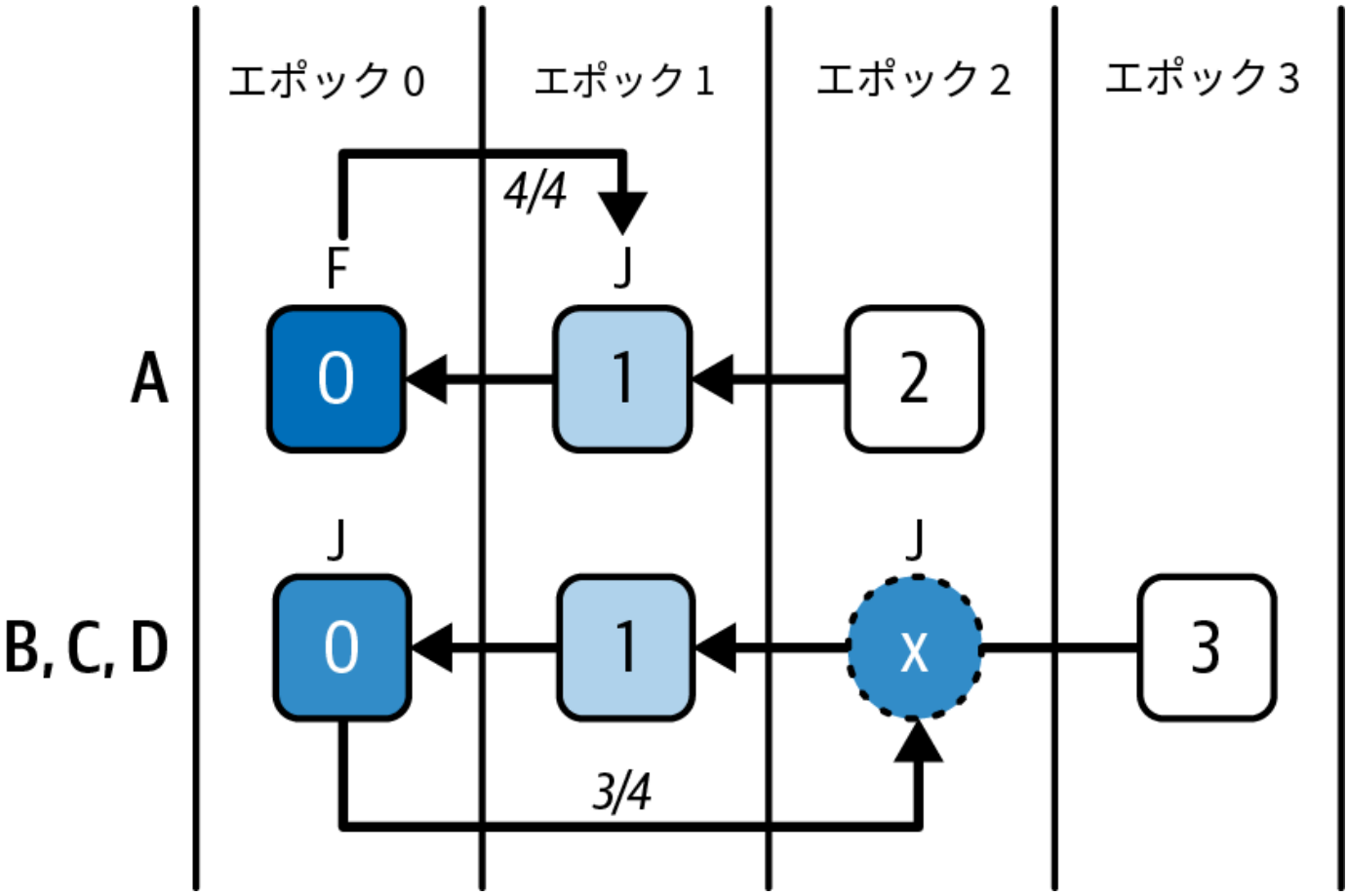


図15-29 ネットワーク遅延シナリオステップ2

注記

この例ではバリデータB、C、DがAの提案したブロックを見ていないことを覚えておくことが重要である。チェックポイント1を正当化するはずだったブロック2を確認していないためブロック1に同意できずチェックポイント1を正当化できない。その結果代わりに空のチェックポイントに投票する。

この例は単純なネットワーク遅延でさえノードがジャスティフィケーションとファイナライゼーションについて異なるビューを持つ原因になり得ることを示しています。しかしこれだけではジャスティフィケーションとファイナライゼーションの2つの別々のフェーズが必要な理由の説明にはなりません。2つのフェーズの背後にある理由は非常にシンプルです。ジャスティフィケーションのステップがなければAはチェックポイント1を確定していたでしょうが、図15-30に示すように残りのバリデータからは無効とみなされていたでしょう。

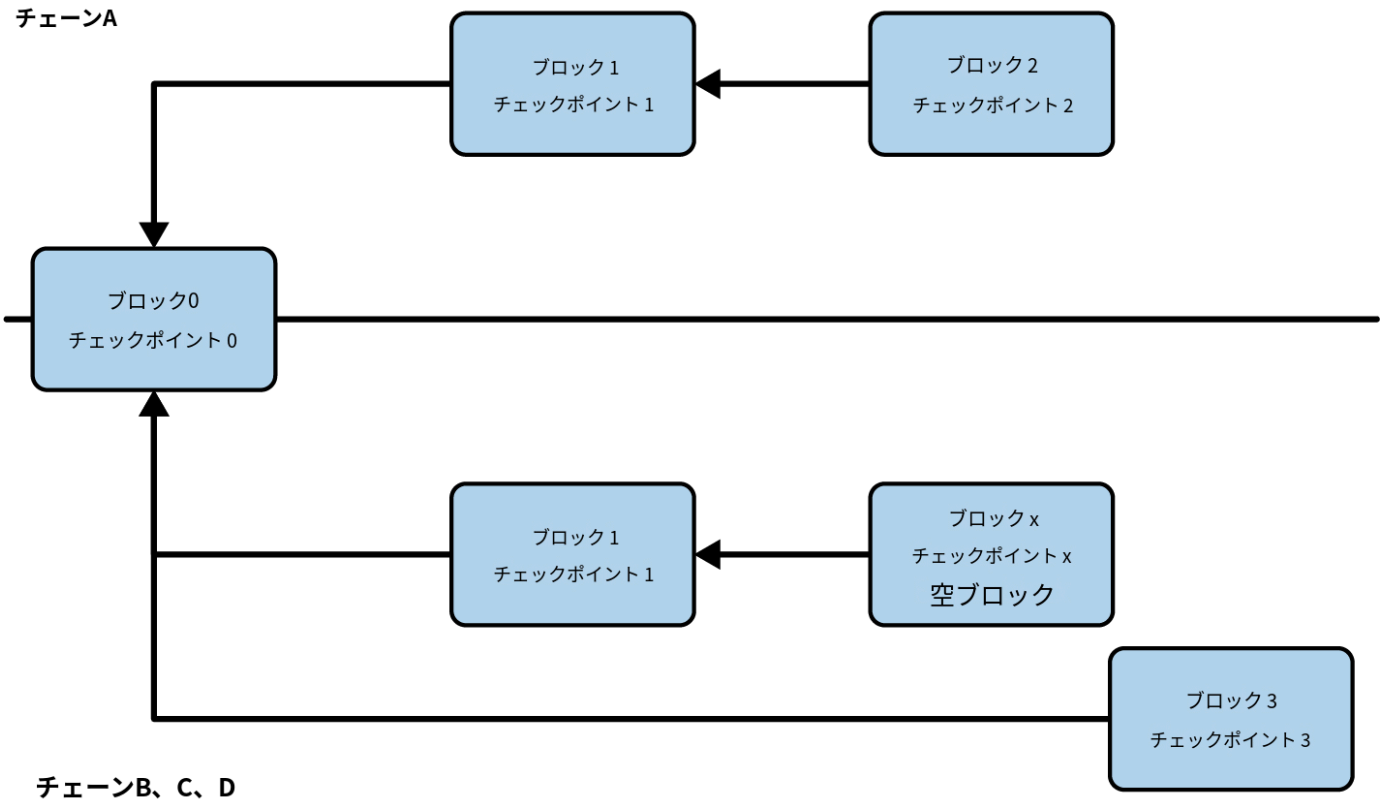


図15-30 ジャスティフィケーションステップなしの場合

前の例ではAはメモリを削除してネットワークの残りの部分と再同期する必要がありました。ジャスティフィケーションは前述のとおりローカルなステップに似ており取り消すことができるからです。しかしAがチェックポイント1を直接確定していた場合これは不可能だったでしょう。2つのフェーズがなければAは確定されたブロックが取り消され、B、C、Dはスラッシングされることなくブロック1を孤立させることができたでしょう。セーフティを保証する唯一の方法はジャスティフィケーションとファイナライゼーションの2段階コミットです。

Gasper：実際の例

ここまでCasper FFGとLMD-GHOSTがそれぞれ単独でどのように機能するかを見てきました。次にこれらがGasperに統合されイーサリアムのPoSコンセンサスプロトコル内でどのように使用されるかを見ていきましょう。

Gasperの仕組みを完全に理解する最良の方法はGasperを使用してブロックの履歴に対するコンセンサスを得るブロックチェーンの実例を追うことです。この例ではイーサリアムのメインネットは使用しません。代わりに図15-31に示すようにコンセンサスプロトコルの各フェーズで何が起こっているかをより良く説明するために3人のバリデータを持つモックアップネットワークを作成します。

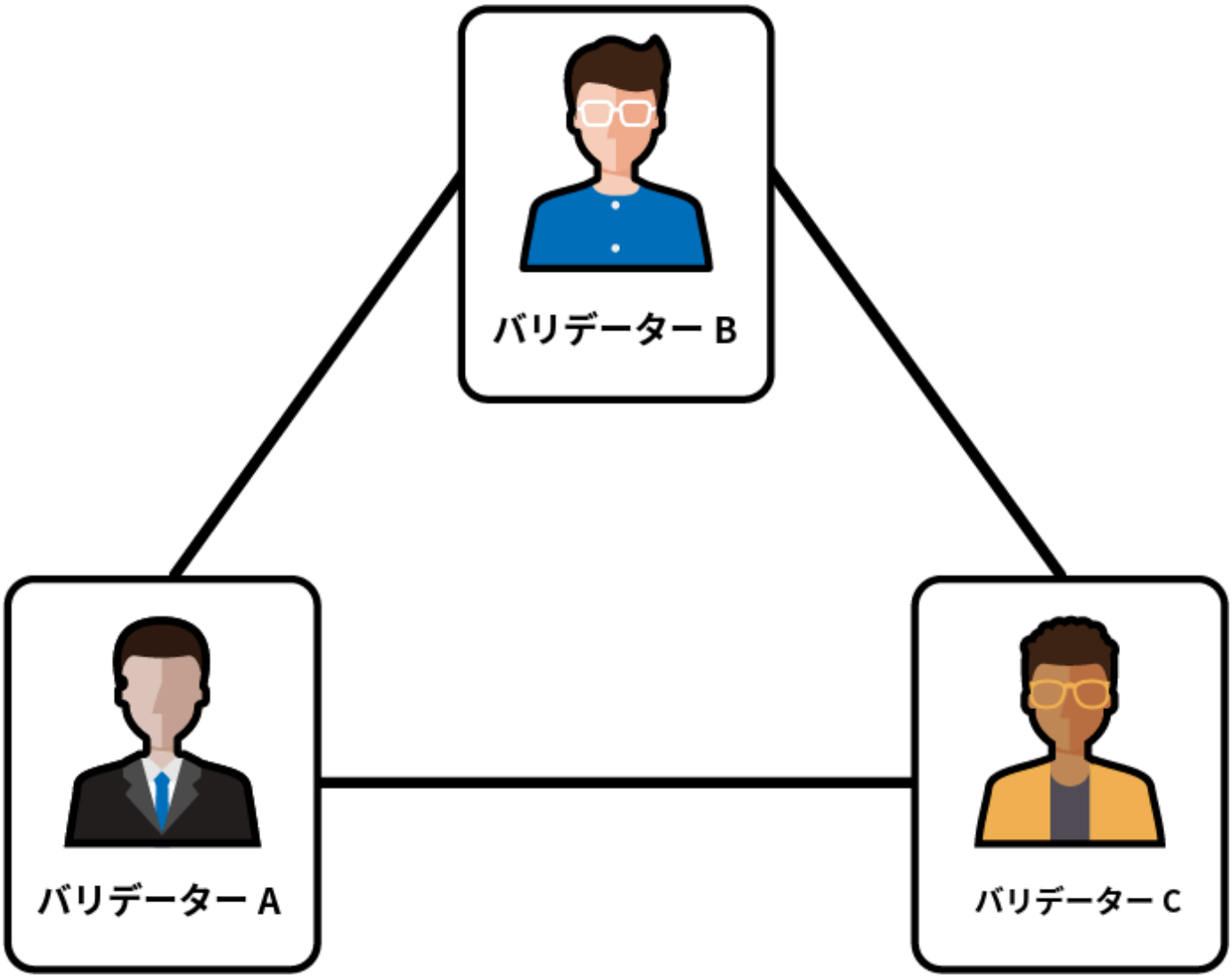


図15-31 Gasperモックネットワーク

3人のバリデータは同量のETHをステークしているため投票力（つまり投票する各ブロックのスコアへの貢献度）は同じです。またイーサリアムのメインネットのようにエポックごとに1回ではなくすべてのバリデータがブロックごとにアテステーションを公開します。

目標はブロックが公開されてからまず正当化され次に確定されるまでのライフを見ることです。

簡略化されたネットワークでは図15-32に示すように各エポックは3つのスロットで構成されています。

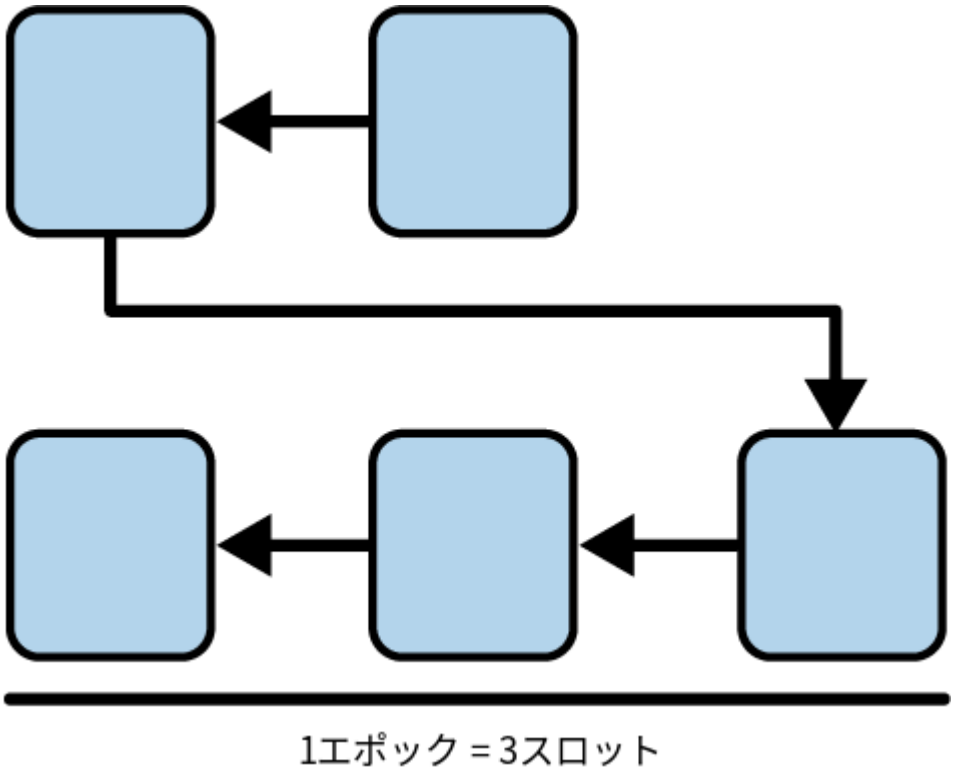


図15-32 簡略化されたエポック構造

　例をエポック番号1から始めましょう。これは実際の最初のエポックではありません。簡便のために「エポック1」と呼んでいるだけです。バリデータAが最初のブロックを提案するために選ばれます。図15-33に示すように提案するブロックをブロック1と呼びます。

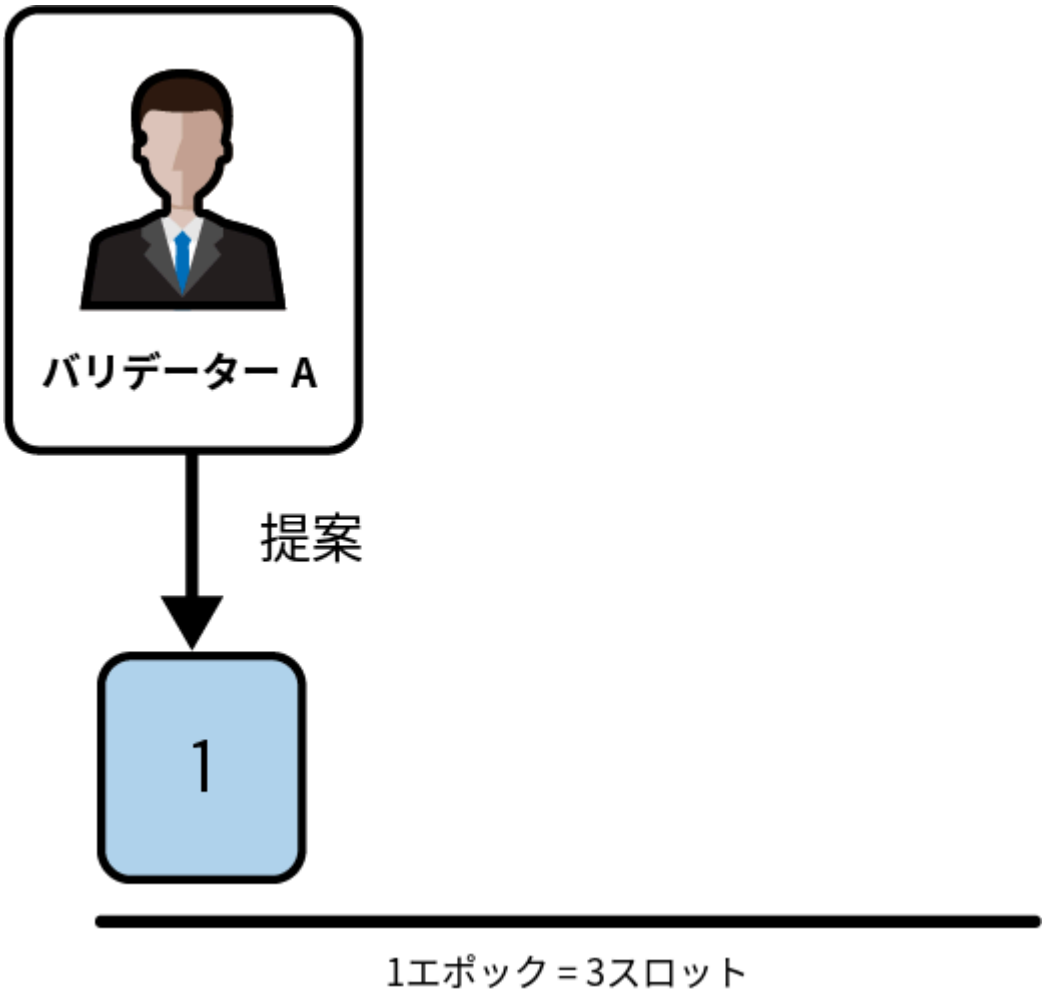


図15-33 ブロック1の提案

バリデータAがブロック1を公開するとすぐにそのブロックはネットワーク内で伝播を開始します。公開直後にバリデータBとCがブロック1を受信し自身のネットワークビューに保存します。

次にすべてのバリデータがチェーンの最後のヘッドブロックだと考えるものに投票してアテステーションを作成します。そのためにローカルビューでLMD-GHOSTを実行する必要があります。結果はブロック1です。これらのアテステーションは公開されすべてのバリデータと共有されます。

次に図15-34に示すようにバリデータBが次のスロット（スロット2）でブロックを提案するために選ばれます。そのためにバリデータはネットワークのローカルビューでLMD-GHOSTを実行して構築すべき最後のヘッドブロックを取得する必要があります。

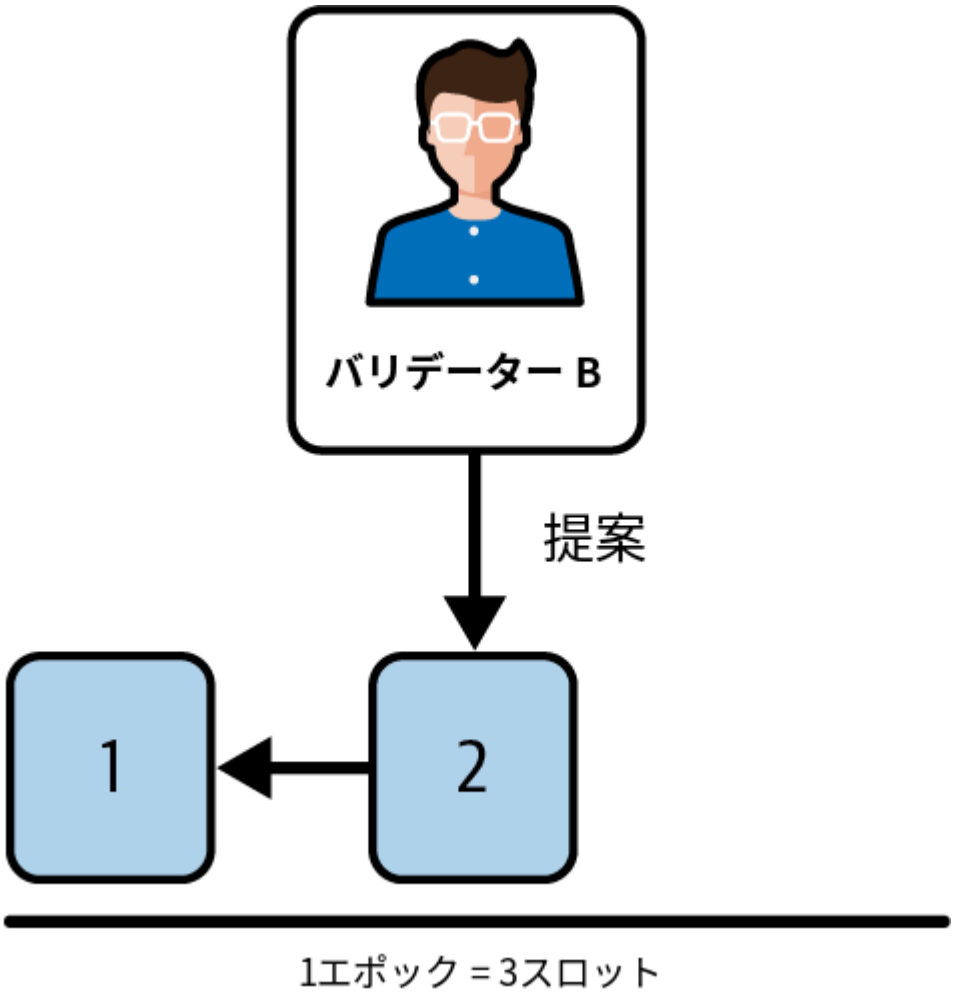


図15-34 ブロック2の提案

以下はバリデータBのネットワークビューです。

- バリデータAのアテステーション：ヘッドブロック：ブロック1
- バリデータBのアテステーション：ヘッドブロック：ブロック1
- バリデータCのアテステーション：ヘッドブロック：ブロック1

したがってバリデータBのLMD-GHOSTの結果はブロック1です。ブロック1の上にブロック2を公開できます。ブロック2の中にバリデータBは以前のブロックに含まれていなかったこれまでに確認したすべてのアテステーションも保存します。つまりブロック1への投票を含む3つのアテステーションを保存します。

ブロック2がネットワーク内で伝播し公開直後にバリデータAとCが受信します。LMD-GHOSTはP2Pを通じて共有された投票とブロックに含まれた投票の両方を使用しますが、Casper FFGはブロックに含まれた投票のみを考慮することを覚えておいてください。したがってバリデータがすでにP2Pネットワークを通じて共有している場合LMD-GHOSTの投票をブロックに含めてもLMD-GHOSTの結果には影響しませんが、バリデータが投票を知る唯一の方法であるためCasper FFGにとっては基本的に重要です。

再びすべてのバリデータがチェーンの最後のヘッドブロックだと考えるものに投票してアテステーションを作成します。全員がローカルビューにブロック2を持っているため全員がチェーン

のヘッドとしてブロック2に投票します。次にすべてのバリデータが確認できるようにこれらのアテステーションを公開します。

次にバリデータCがスロット3で次のブロックを提案するために選ばれます。ローカルビューでLMD-GHOSTを実行してヘッドブロックを取得します。以下はバリデータCのネットワークビューです。

- バリデータAのアテステーション：ヘッドブロック：ブロック2
- バリデータBのアテステーション：ヘッドブロック：ブロック2
- バリデータCのアテステーション：ヘッドブロック：ブロック2

結果はブロック2なので図15-35に示すようにバリデータCはブロック2の上にブロック3を公開します。

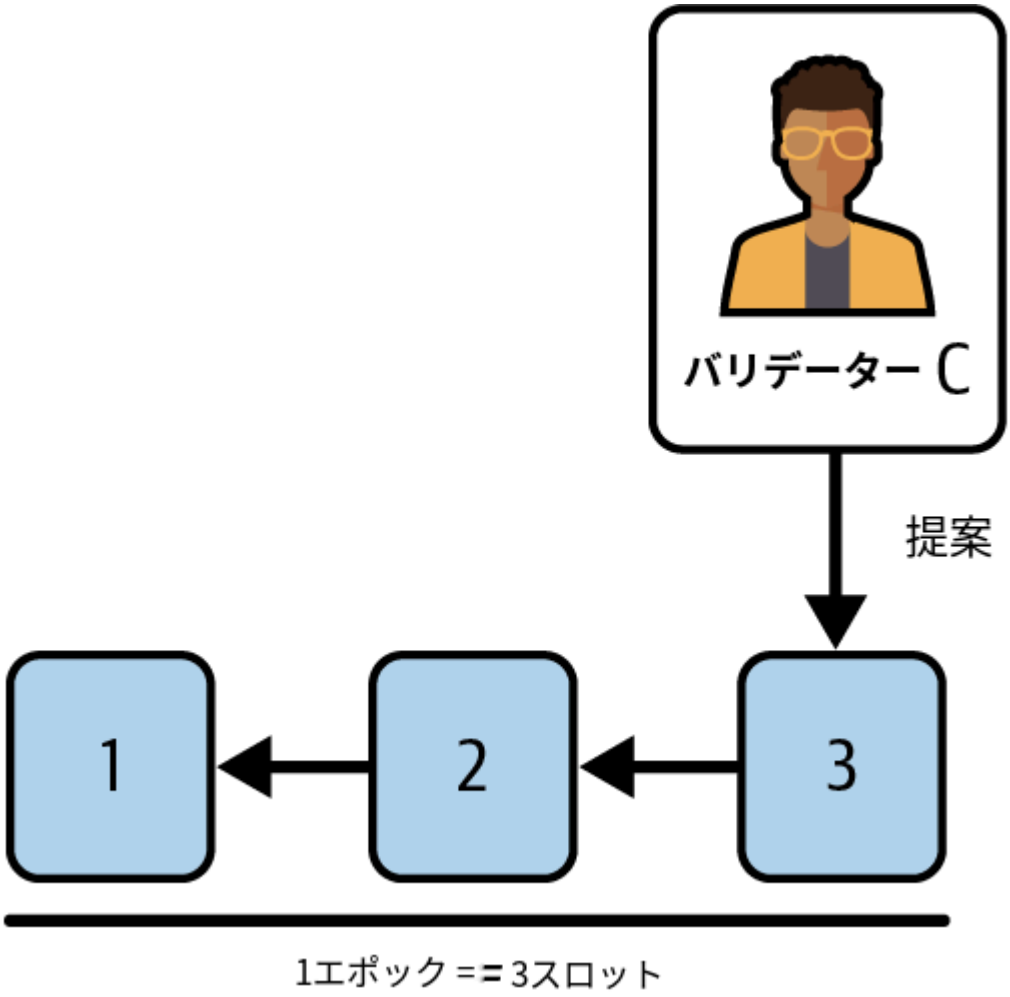


図15-35 ブロック3の提案

ブロック3の中にバリデータCは以前のブロックに含まれていなかったブロック2への投票を含む3つのアテステーションを保存します。

ブロック3がネットワーク内で伝播し公開直後にバリデータAとBが受信します。バリデータはブロック3に投票するアテステーションを作成します。

次にバリデータAに戻ります。図15-36に示すように次のブロック（スロット4）を提案する必要があり、これは新しいエポック（エポック2）の最初のブロックでもあります。

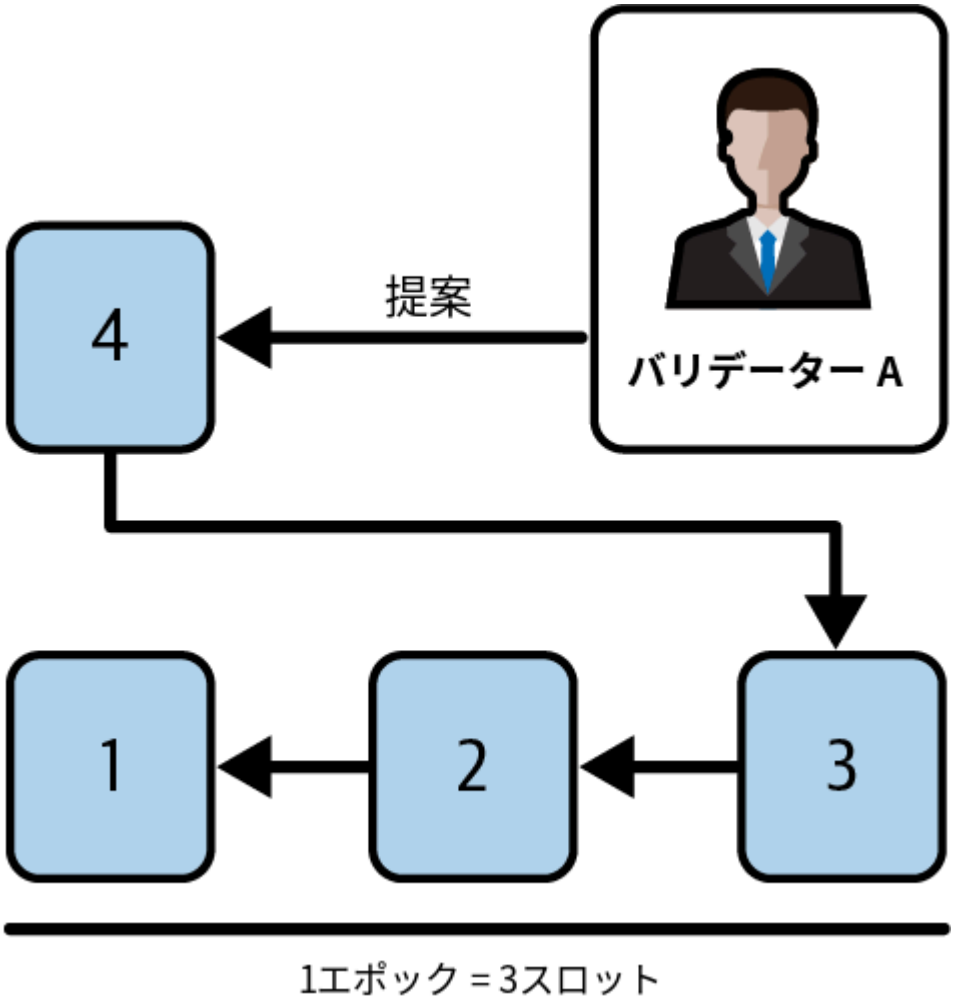


図15-36 ブロック4の提案 - 新しいエポック

バリデータAはローカルビューでLMD-GHOSTを実行してチェーンのヘッドを取得します。以下はバリデータAのネットワークビューです。

- バリデータAのアテステーション：ヘッドブロック：ブロック3
- バリデータBのアテステーション：ヘッドブロック：ブロック3
- バリデータCのアテステーション：ヘッドブロック：ブロック3

結果はブロック3なのでバリデータAはブロック3の上にブロック4を公開します。

ブロック4の中にバリデータAは以前のブロックに含まれていなかったブロック3への投票を含む3つのアテステーションを保存します。

ブロック4がネットワーク内で伝播し公開直後にバリデータBとCが受信します。バリデータはチェーンのヘッドとして投票する新しいアテステーションを作成する必要があります。しかし今回は何かが変わります。

新しいエポックに入ったためバリデータはアテステーションのCasper FFG部分を更新する必要があります。実際これまで無視していましたがアテステーションにはチェーンの最後のヘッド

ブロックへの投票（コンセンサスプロトコルのLMD-GHOST部分）だけでなくCasper FFGチェックポイントへの投票も含まれています。特にすべてのアテステーションにはソースとターゲットの投票が含まれます。ソース投票はバリデータが知っている最後に正当化されたチェックポイントでありターゲット投票はバリデータが次に正当化されるべきだと考えるブロックを表します。

バリデータA、B、Cが作成するアテステーションは以下のとおりです。

アテステーション

- ヘッドブロック：ブロック4
- ソースブロック：ブロック1
- ターゲットブロック：ブロック4

ターゲットブロックはエポックの最初のブロックであるため選択は簡単です（ターゲットブロックがエポックの最初のブロックではないエッジケースもありますが簡便のため無視します）。ソースブロックはバリデータが持っているアテステーションを調べバリデータの3分の2以上からターゲットブロックとして投票されたブロックがあるかどうかを確認して計算されます。以前のアテステーションにソースとターゲットブロックを含めていませんでしたが、すべてブロック1をターゲットブロックとして含んでいたとしましょう。したがってブロック1からブロック4へのスーパーマジョリティリンクがあるためブロック1を正当化しました。図15-37を参照してください。

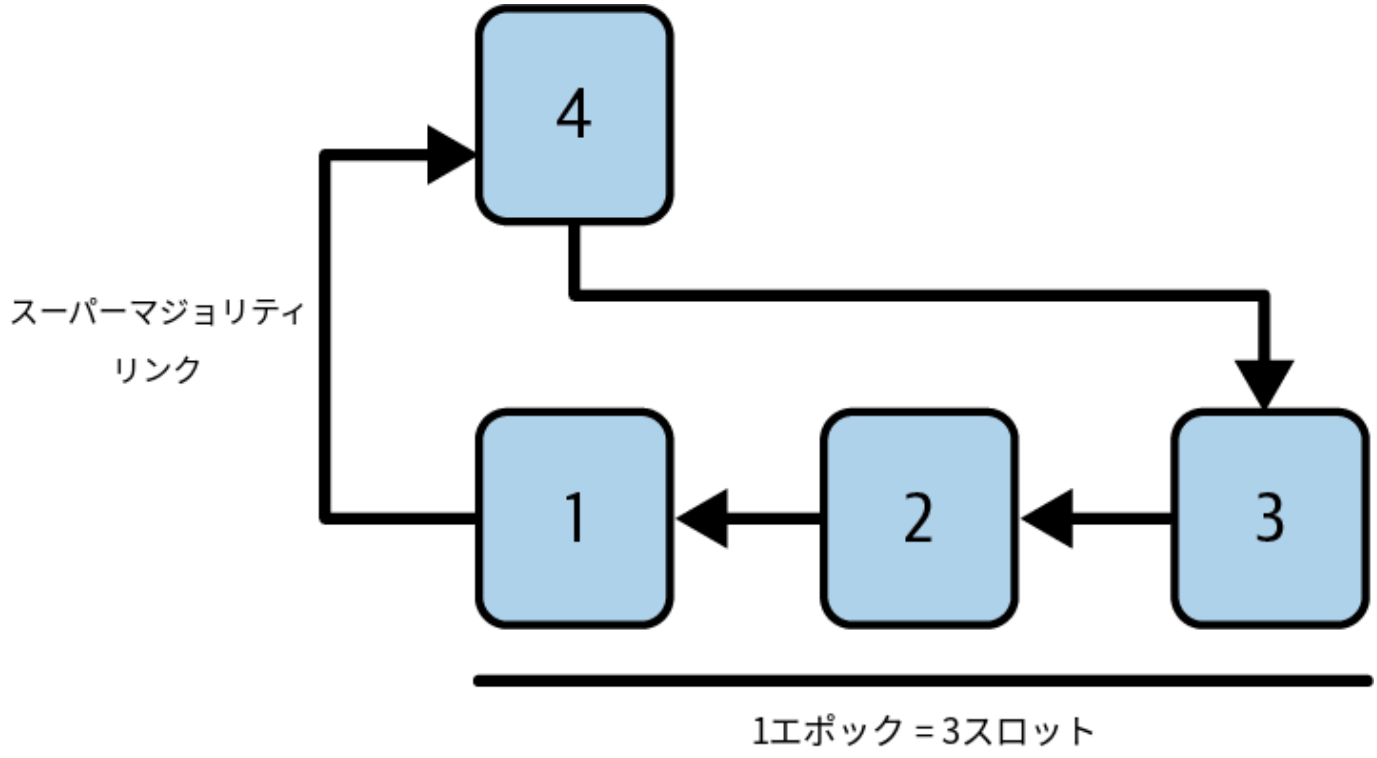


図15-37 ブロック1が正当化

これらのアテステーションは公開されすべてのバリデータと共有され、次のスロット（通常は直後のスロット）に含まれます。ブロック5と6をスキップして図15-38に示す次のエポック（エポック3）の最初のブロックであるブロック7に直接進みましょう。

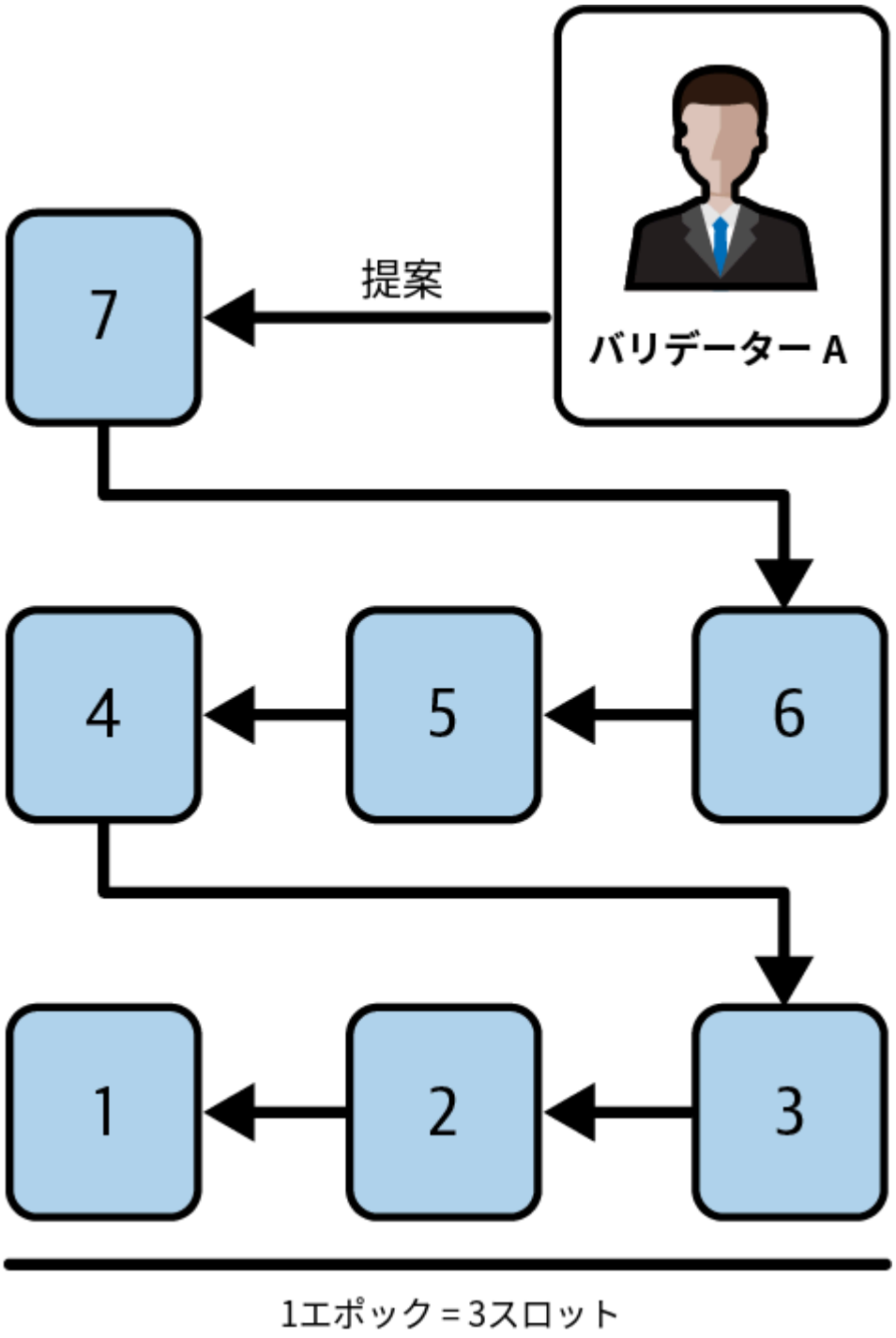


図15-38 ブロック7の提案 - エポック3

再びバリデータAがブロックを提案するために選ばれます。ローカルビューでLMD-GHOSTを実行します。以下はバリデータAのネットワークビューです。

- バリデータAのアテステーション：ヘッドブロック：ブロック6、ソースブロック：1、ターゲットブロック：4
- バリデータBのアテステーション：ヘッドブロック：ブロック6、ソースブロック：1、ターゲットブロック：4
- バリデータCのアテステーション：ヘッドブロック：ブロック6、ソースブロック：1、ターゲットブロック：4

結果はブロック6なのでブロック6の上にブロック7を公開します。

ブロック7がネットワーク内で伝播し公開直後にバリデータBとCが受信します。バリデータはチェーンのヘッドとして投票する新しいアテステーションを作成する必要があります。ここでも再び何かが変わります。

次のエポック（エポック3）に入ったためアテステーションのCasper FFG部分が再び変わります。バリデータA、B、Cが作成するアテステーションは以下のとおりです。

アテステーション

- ヘッドブロック：ブロック7
- ソースブロック：ブロック4
- ターゲットブロック：ブロック7

ご覧のとおりターゲットブロックはブロック7になりソースブロックはブロック4です。バリデータA、B、Cが前のアテステーションでターゲットブロックとしてブロック4に投票したためです。ブロック4からブロック7への新しいスーパーマジョリティリンクがあるため図15-39に示すようにブロック4を正当化しました。

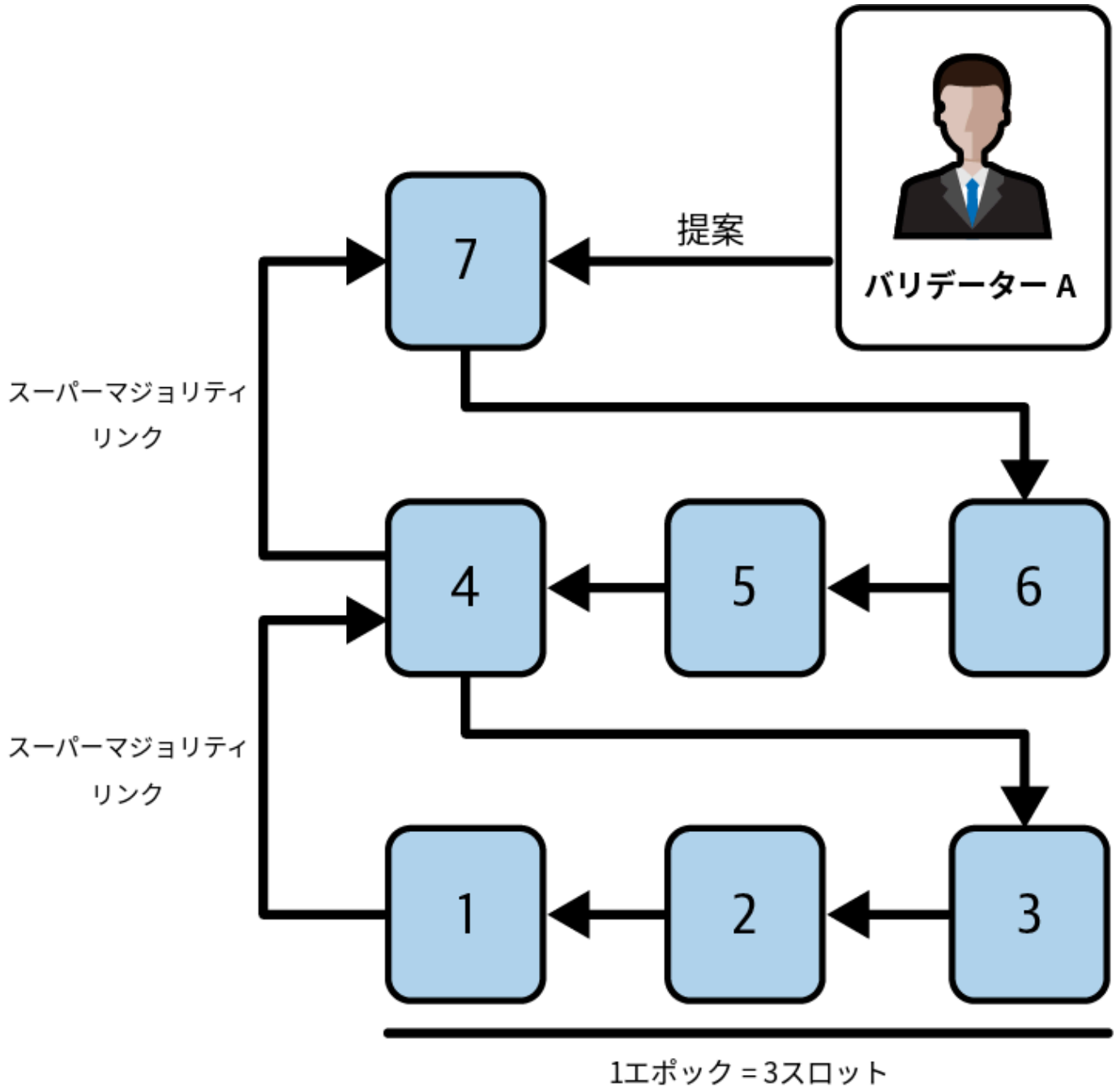


図15-39 ブロック4が正当化、ブロック1が確定

　ブロック1も確定されました。正当化されたチェックポイントでありその直接の子であるブロック4も正当化されているためです。バリデータがブロックを確定されたとみなすときバリデータの3分の2以上から全員がそのブロックが正当化されていることを確認したという確認を見たことを意味します。実際バリデータAを例にとると（バリデータBとCにも当てはまります）、BとCがソースブロックとしてブロック1に投票したアテステーションを確認しています。ソースブロックとしてブロック1に投票するということはBとCが以前にターゲットブロックとしてブロック1に3分の2以上の投票があることを確認していたことを意味します。したがってブロック1を取り消すにはバリデータの少なくとも3分の1が二重投票によりスラッシングされなければならないことを確信できます。

論争と競争

この時点でなぜこれほど多くの異なるコンセンサスアルゴリズムが必要なのか疑問に思うかもしれません。どれがより良く機能するのでしょうか。この質問への答えは過去10年間の分散システムにおける最もエキサイティングな研究分野の中心にあります。すべては何を「より良い」と考えるかに帰着します。コンピュータサイエンスの文脈では仮定、目標、そして避けられないトレードオフの問題です。

分散コンセンサスの問題のすべての側面を最適化できるアルゴリズムはおそらく存在しないでしょう。あるコンセンサスアルゴリズムが他のものよりも「優れている」と誰かが提案した場合、何がより優れているのかを明確にする質問を始めるべきです。不変性でしょうか。ファイナリティでしょうか。分散化でしょうか。コストでしょうか。少なくともまだ明確な答えはありません。さらにコンセンサスアルゴリズムの設計は数十億ドル規模の産業の中心にあり大きな論争と激しい議論を生んでいます。最終的には「正しい」答えがないかもしれませんし異なるアプリケーションに対して異なる答えがあるかもしれません。

ブロックチェーン業界全体が敵対的な条件下で巨額の金銭的価値を賭けてこれらの問いが検証される壮大な実験です。最終的に歴史がこの論争に答えを出すでしょう。

コンセンサスプロトコルにおける論争は数多くネットワークの調整によって解決することは困難です。インセンティブの整合は重要ですが常に可能というわけではありません。イーサリアムのコンセンサスアルゴリズムの2つの現在の問題を検証しましょう。

タイミングゲーム

イーサリアムのプロトコルでは時間はスロットと呼ばれる12秒の単位で構成されています。各スロットは開始時（$t = 0$）にバリデータにブロックの提案の役割を割り当てます。次にアテスターの委員会がこのブロックの検証を担当し、スロット開始から4秒後（$t = 4$）のアテステーション期限までに検証を行うことを目指します。

*タイミングゲーム*は図15-40に示すようにバリデータがMEV報酬を最大化するためにブロックの提案をできるだけ遅らせる戦略です。この戦略はより多くの価値を獲得するために提案を遅らせつつ、カノニカルチェーンに残るためにアテスティング委員会の十分な部分から支持されることを確保するという微妙なバランスを必要とします。

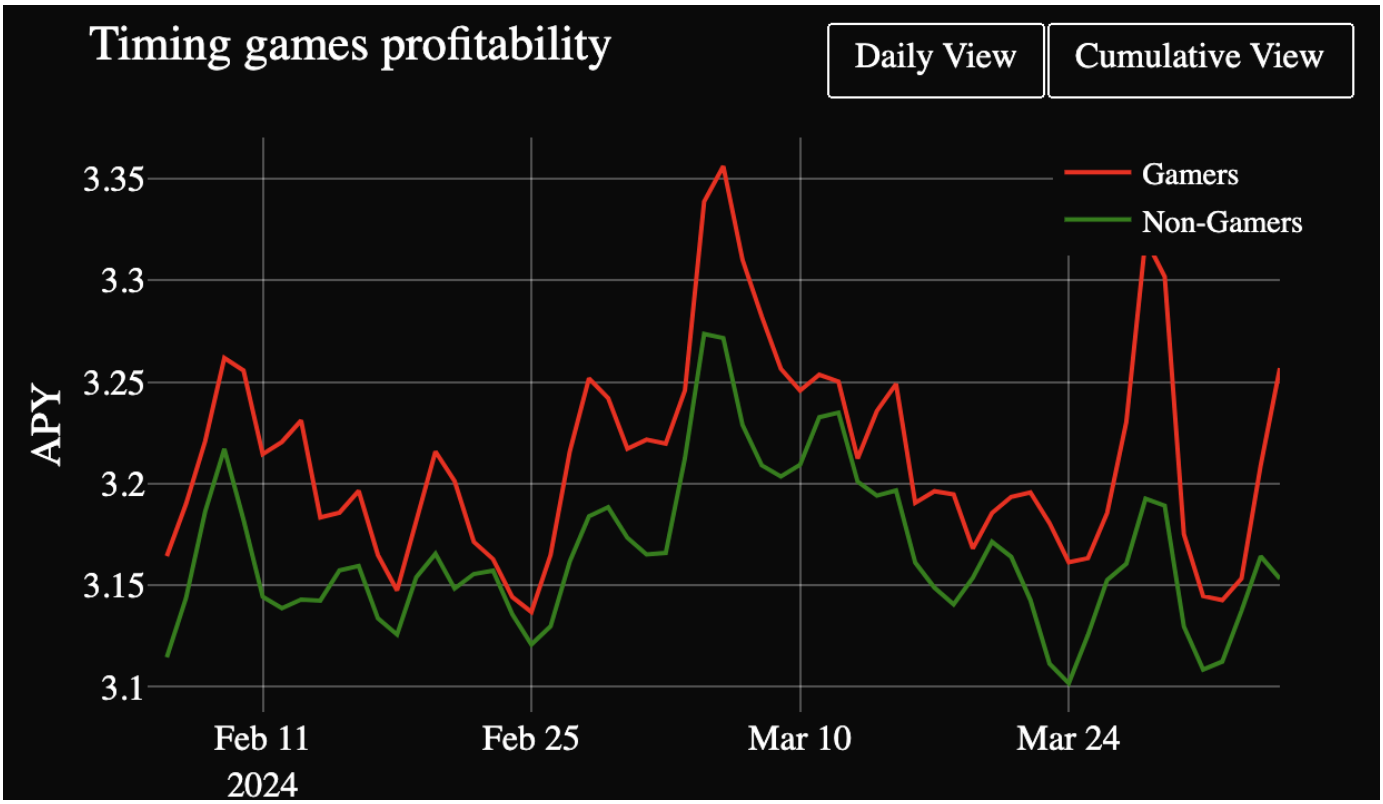


図15-40 タイミングゲーム戦略

イーサリアムにおけるタイミングゲームはあるバリデータのMEVからの利益が他のバリデータの不利になる可能性がある競争的な環境を作り出します。この競争はミスしたスロットの数の増加と潜在的なブロックリオーグを増やすことでコンセンサスを混乱させる可能性があります。さらにアテスターが検証を延期する動機にもなりプロセスに複雑さの層を追加します。

「コンセンサスの原則」ではイーサリアムのコンセンサスプロセスにおけるライブネスの重要性を指摘しました。しかしタイミングゲームはネットワークの信頼性を損なうことでこの重要な機能に対する脅威となります。

タイミングゲームを分かりやすく言えば、最大限の利益を得るために完璧なタイミングを待つ行為です。ネットワークを維持している一部のバリデータがまさにこれを行おうとしています。最大の報酬を得るために行動の適切なタイミングを待っているのです。しかしこの待ちのゲームはリスクを伴います。インターネットが遅い場合や経験が少ない場合、役割を果たすチャンスを逃す可能性があります。あまりに多くのチャンスを逃すとネットワークの信頼性が低下する可能性があります。

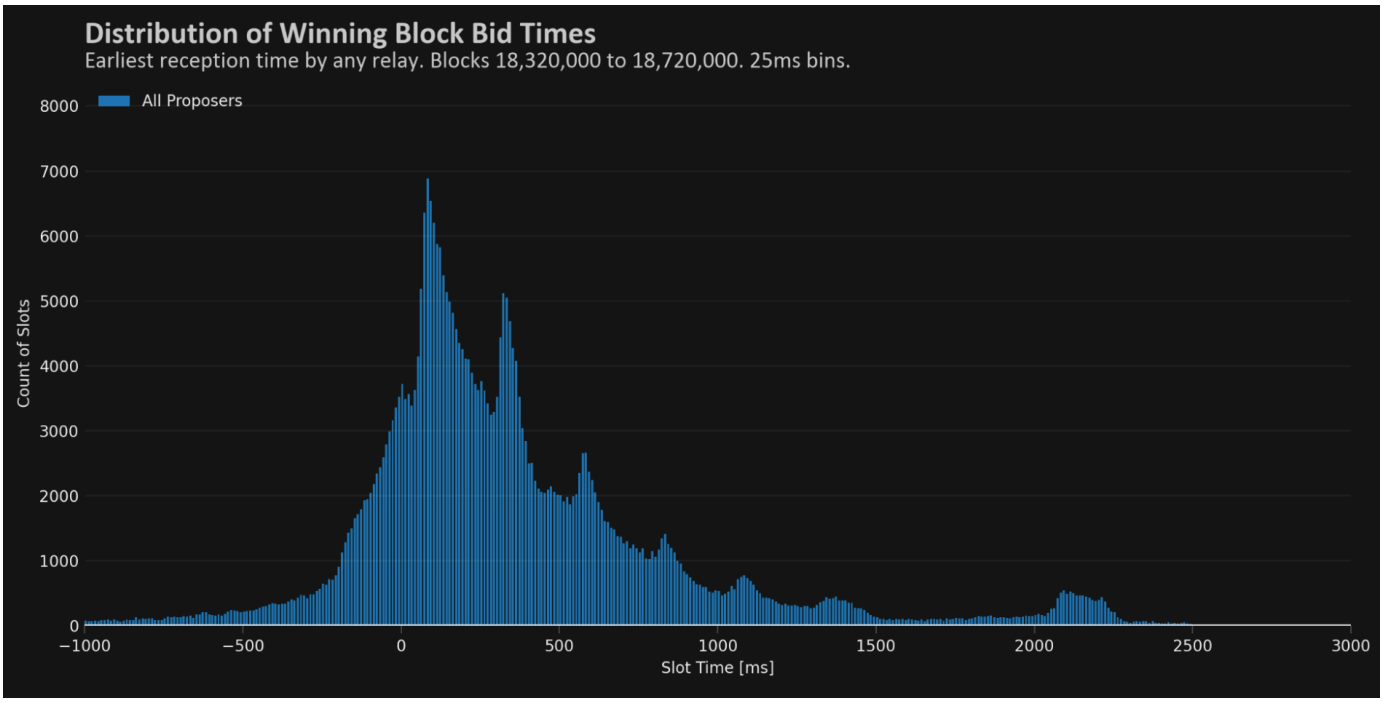


図15-41 タイミングゲームのリスク可視化

現時点では大きな問題ではありません。図15-41に示すようにバリデータとして活動しているエントリーのほとんどはこれらのタイミングゲームにあまり関与していないか、まったくプレイしていません。

注記

2024年にこの章を書いて以来状況は少し変わった。現在Dencunハードフォークで「proposer boost」と呼ばれるソリューションが追加されており遅延したブロック提案者にペナルティを課している。proposer boostはブロックが提案されたスロットでブロック提案者のアテステーションに重みを追加する。

スーパーマジョリティの中央集権化

イーサリアムにおけるスーパーマジョリティクライアントリスクとは、ネットワークの健全性とセキュリティのバランスに関する問題です。イーサリアムは単一障害点を防ぐために複数のクライアントを使用する方針を採用しました。すべてのソフトウェアにはバグが存在する可能性があるためです。本当に問題になるのはコンセンサスバグが発生した場合です。たとえばEtherを無限に生成するような深刻な事態につながりかねません。1つのクライアントだけがすべてを運用し、そのクライアントがバグに見舞われた場合、修復は悪夢のようなものになります。バグがアクティブな状態でネットワークが稼働し続け、攻撃者が不可逆的な損害を引き起こすおそれがあります。

図15-42に示すように多数派クライアントにバグがあった場合に何が起こり得るかの簡単な例を分析しましょう。図15-42のすべてのブロックはブロックチェーンのブロックではなくチェッ

クポイントであることに注意してください。

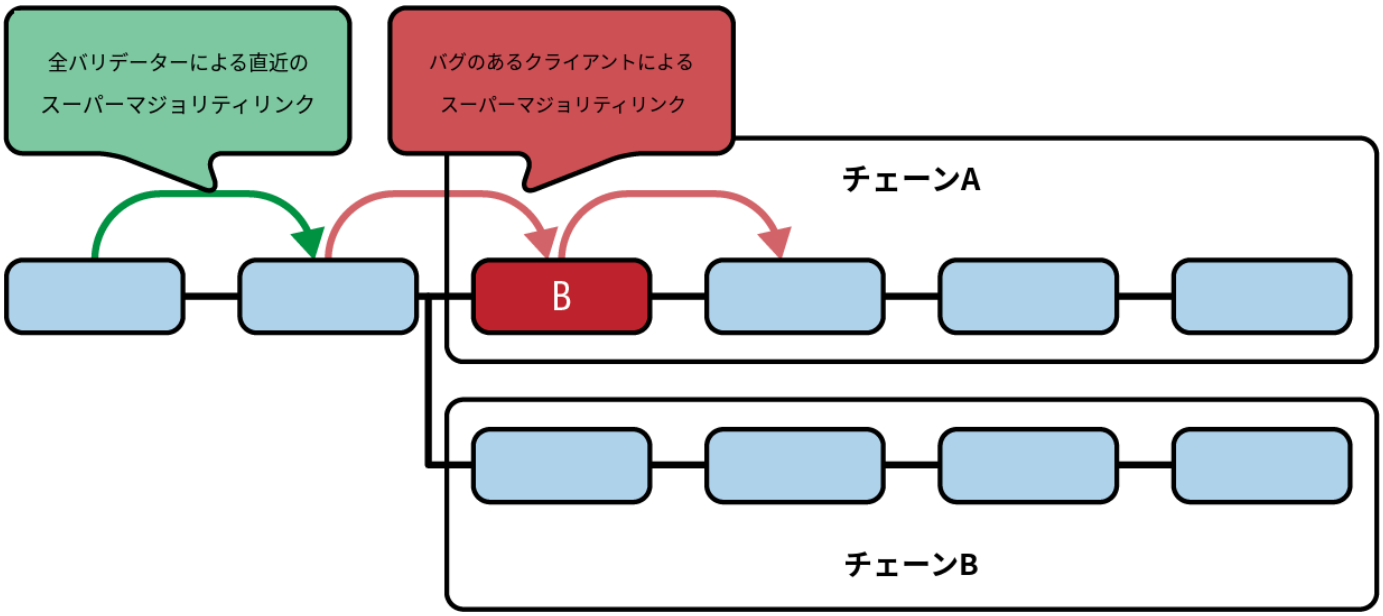


図15-42 多数派クライアントのバグシナリオ

正常に動作しているクライアントは無効なブロック（「B」とラベル付けされた）を含むエポックを無視します。ブロックBを指す矢印は無効なエポックを正当化するために機能しブロックBから出る矢印はそのエポックを確定します。

バグが解決され無効なエポックを確定したバリデータが正しいチェーンBに戻りたい場合、図15-43に示すように必要な予備的なアクションはエポックXのジャスティフィケーションです。

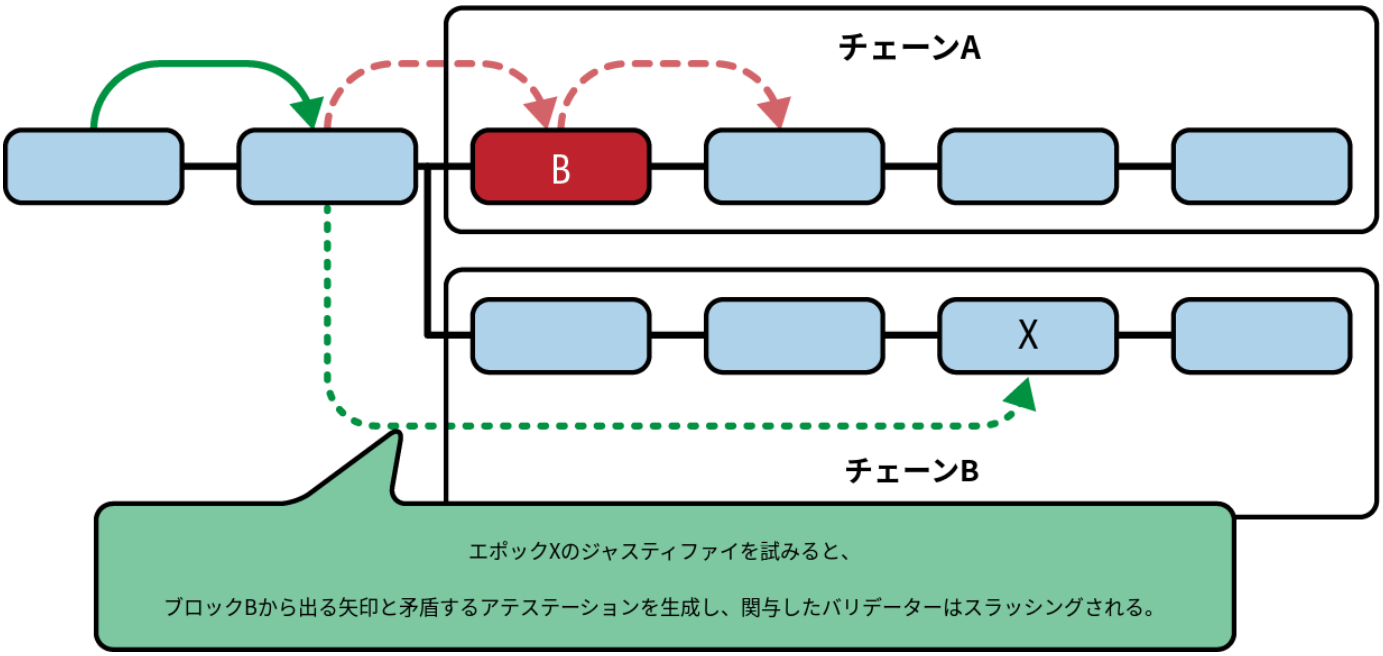


図15-43 バグからの回復にはジャスティフィケーションが必要

破線の矢印で示されるスーパーマジョリティリンクを必要とするエポックXのジャスティフィケーションに参加するにはバリデータはブロックBから出る矢印（無効なエポックの確定を表

す）をバイパスする必要があります。両方のリンクに投票するとこれらのバリデータにはペナルティが課される可能性があります。

マルチクライアントアプローチはセーフティネットを提供します。ネットワークの半分未満が使用しているクライアントにバグが発生した場合、他のクライアントを実行しているネットワークの残りの部分はバグのあるブロックを単に無視します。これによりネットワークは軌道に乗り続け混乱は最小限に抑えられます。しかし多数派クライアント（特にバリデータの3分の2以上が使用しているもの）がバグを導入した場合、チェーンが不正に確定され潜在的な分裂につながる可能性があります。

イーサリアムはクライアントの多様化を奨励しています。全員が同じクライアントを使用しそれが故障した場合、ネットワーク全体が危険にさらされるからです。多数派に逆らうクライアントを実行することに対するペナルティはすべてを一つのかごに入れることを抑止するためにあります。こうすることでクライアントにバグがあった場合でも損害は限定的で影響を受けるユーザーは少なくなります。少数派クライアントが問題を引き起こした場合、多数派がパスを修正してチェーンの確定を継続できるためそれほど大きな問題にはなりません。

異なるクライアント間で選択をより広く分散させればさせるほど、イーサリアムはより安全になります。単に技術的な障害を避けるだけでなく、単一障害点からイーサリアムの将来を守ることにつながります。クライアントの多様性はネットワーク全体の危機に対する最良の防御であり、何が起ころうともイーサリアムが堅牢で回復力を維持することを保証します。

この章を最初に書いた時点ではGethの使用率は63%であり前述のとおり問題でした。現在の状況はより健全ですが将来さらに改善する必要があります。図15-44に示すように現時点では実行クライアントの41%がGethを使用し38%がNethermindを使用しています。

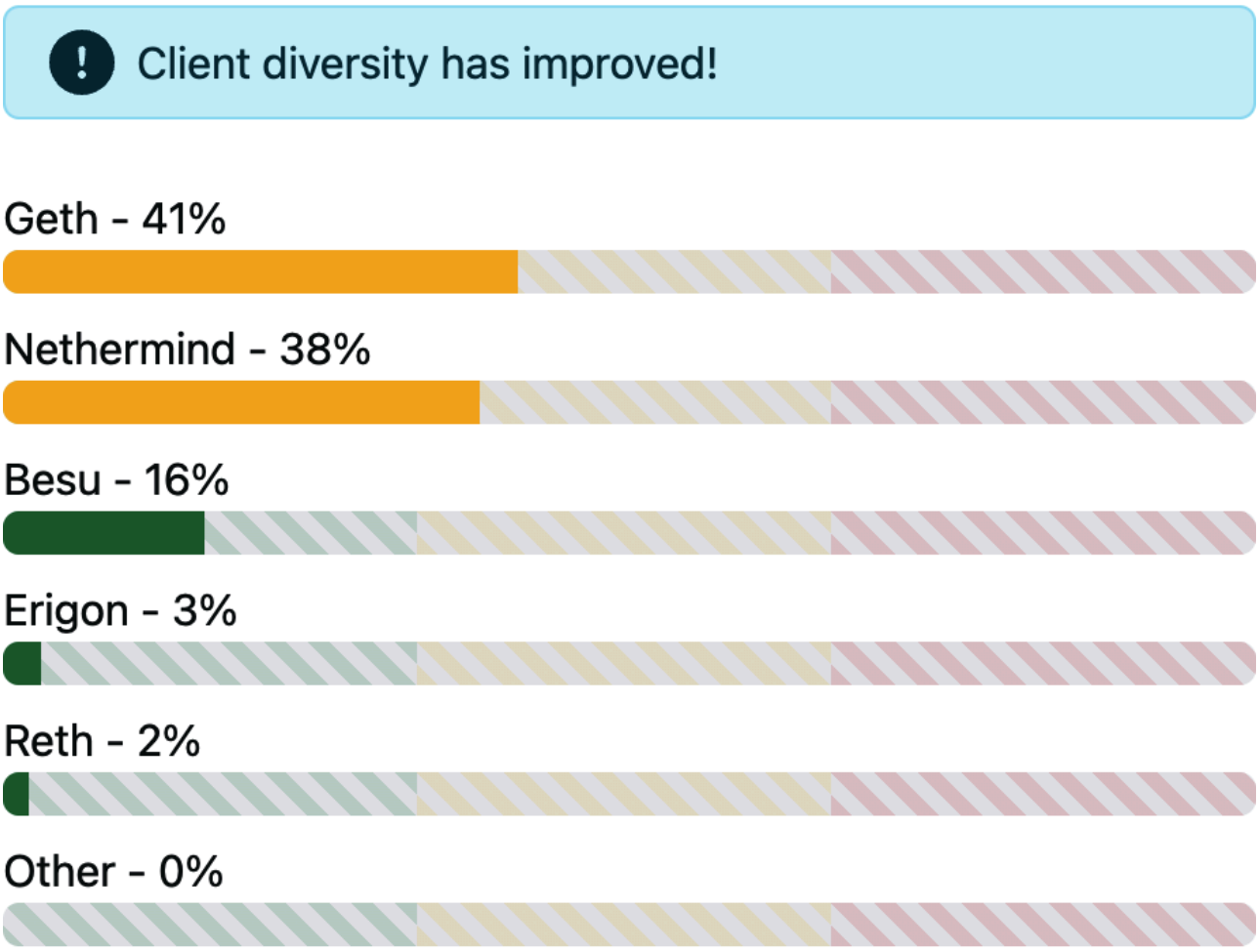


図15-44 現在の実行クライアント分布

　この問題は実行クライアントだけでなくコンセンサスクライアントにも影響しますが、コンセンサス側の問題は迅速に対処され図15-45に示すように状況は現在比較的健全で安定しています。

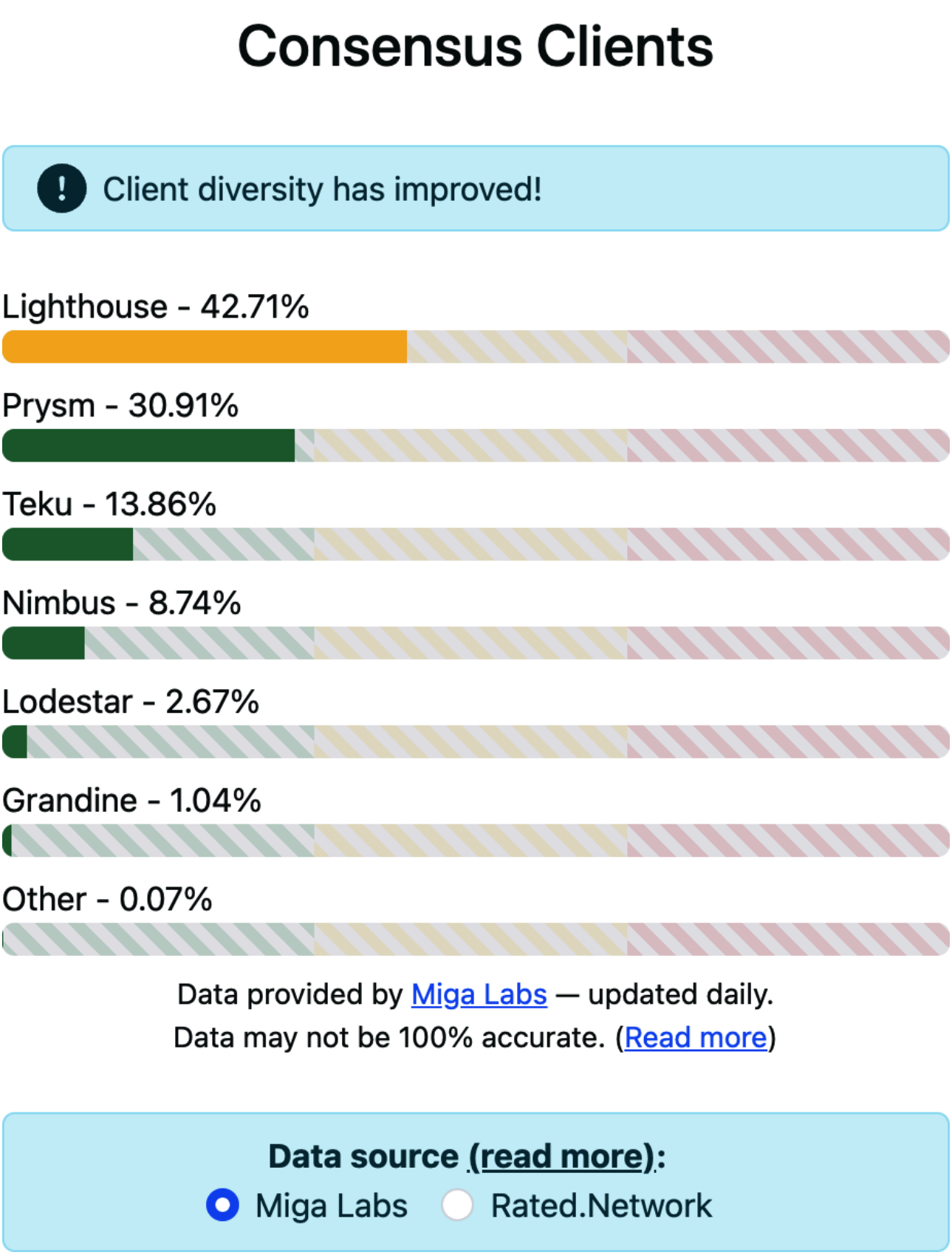


図15-45 現在のコンセンサスクライアント分布

結論

コンセンサスアルゴリズムはイーサリアムにおいて最も複雑で繊細な要素の一つです。機能性と効率性を向上させるための提案が絶えず行われており、革新と改善の終わりなき旅を表しています。single-slot finalityやバリデータの最大有効残高の増加などの機能はシステムを精緻化し最適化するための継続的な取り組みを示しています。

コンセンサスの核心的な原則を理解することはこれらの進歩とイーサリアムネットワークの堅牢性およびスケーラビリティへの影響を評価するための確固たる基盤を提供します。イーサリアムが進化するにつれてこのパイオニア的なブロックチェーン技術の能力を前進させるコンセンサスメカニズムも進化していきます。

さらなる読み物として、以下を推奨します。

- Gasper論文
- ベン・エジントン著Upgrading Ethereum
- 「Decentralization Is Good or Not? Defending Consensus in Ethereum 2.0」

第16章 イーサリアムのスケーリング

イーサリアムは最も強力かつ広く利用されているブロックチェーンプラットフォームの1つですが、これまで何度も見てきたように、成功には成長痛が伴います。利用者の急増に伴い、ベースレイヤーが需要に追いつけなくなっています。ガス代はしばしば高騰してトランザクションが高額になりすぎ、処理すべきデータの増大によりシステムの負担が増しています。イーサリアムの開発者たちはEIP-1559（第6章で説明）、The Merge（コンセンサスプロトコルをPoWからPoSに変更した2022年のハードフォーク）、EIP-4844（プロトダンクシャーディング、本章で後述）などのアップグレードを展開してきました。しかしL1の根本的な制約は大規模な普及を阻むボトルネックであり続けています。これらの改善は有用ですが、L2ロールアップのような追加のスケーリングソリューションは依然として必要です。

注記

L2ロールアップは、トランザクションをオフチェーンで処理し、その要約（証明やデータバッチなど）をイーサリアムのLayer 1に投稿するスケーリングソリューションである。イーサリアムのセキュリティに依存しながら、混雑と手数料を削減する。主に2つのタイプがある。オプティミスティックロールアップ（有効と仮定し、問題があれば異議を申し立てる）とゼロ知識ロールアップ（暗号技術で正当性を証明する）である。イーサリアムは本質的に決済レイヤーとなり、証明の検証、データ可用性の確保、L2トランザクションへの最終的なセキュリティ保証の提供へと主な役割がシフトしている。ロールアップについては、本章の後半で詳しく解説する。

イーサリアムLayer 1の問題

イーサリアムのスケーリング課題を十分に理解するには、4つの主要な問題に分解する必要があります。スケーラビリティトリレンマ、ガスコストとネットワーク混雑、ステートの増大とストレージ、ブロック伝播とMEVです。これらの問題はイーサリアム固有のものではなく、他のチェーンも形を変えて同じ問題に直面しています。しかしイーサリアムは利用者が多い分、問題がより顕著に表れています。それぞれ詳しく見ていきましょう。

スケーラビリティトリレンマ

イーサリアムは、あらゆるパーミッションレスブロックチェーンと同様に、3つの基本的な目標を持っています。*分散化*、*セキュリティ*、*スケーラビリティ*です。しかし1つを改善すると他の1つが犠牲になることが多いという問題があります。ヴィタリック・ブテリンはこれを*スケーラビリティトリレンマ*と呼んでいます。

それぞれを分解して説明します。

分散化

分散化は、イーサリアムに検閲耐性とトラストレス性をもたらす要素です。誰でもノードを運用し、トランザクションを検証し、中央機関の許可なくネットワークに参加できます。

セキュリティ

セキュリティは、イーサリアムが攻撃に対する回復力を保つことを保証します。トランザクションは不可逆でなければならず、スマートコントラクトは不変でなければなりません。悪意ある行為者がシステムを容易に操作できてはなりません。

スケーラビリティ

スケーラビリティは、ネットワークが1秒あたり数千から数百万のトランザクション（TPS）を処理し、グローバルな普及を実現するためのものです。

課題は、イーサリアムのような従来のブロックチェーンが、スケーラビリティを犠牲にして完全な分散化とセキュリティを実現するよう設計されている点です。すべてのトランザクションをすべてのノードが処理するため、正確性は保証されますがボトルネックも生じます。

なぜスループットを上げるだけではだめなのでしょうか。ブロックを大きくして処理速度を上げようとすると、ハードウェア要件が厳しくなりすぎてフルノードを運用できる人が少なくなります。その結果、少数の強力な組織だけがネットワークを制御する中央集権化が進む可能性があります。これはまさにブロックチェーンが避けるべき状態です。

Solanaのような他のブロックチェーンは異なるアプローチを取っています。速度とスケーラビリティに最適化しましたが分散化を犠牲にしており、ノードの運用にはより高性能なハードウェアが必要です。イーサリアムは分散化を妥協しない方針を選択しており、スケーリングの課題はさらに困難になっています。

The Mergeで導入されたPoSは、エネルギー集約型のPoWシステムに取って代わりました。イーサリアムのエネルギー消費を劇的に削減しただけでなく、大規模なマイニング設備を持つ大手プレイヤーの優位性も縮小しました。よりアクセスしやすく効率的なネットワークへの一歩ですが、PoSは大規模なステーキングプールによる中央集権化リスクなどの新たな課題ももたらしています。リキッドステーキングソリューションはステーキングされたETHの流動性を維持するのに役立ちますが、少数のプラットフォームに制御が集中するという側面もあります。適切なバランスを見つけることは現在も進行中の課題です。

ガスコストとネットワーク混雑

イーサリアムユーザーにとって最も不満の大きい体験の1つは、ネットワーク需要に応じて大きく変動するガス代の高騰です。なぜこのような変動が起こり、ピーク時に手数料がこれほど高くなるのでしょうか。

イーサリアムのトランザクションには*ガス*が必要です。ガスは、送金やスマートコントラクトとのやり取りなど、操作の実行に必要な計算量を測定する単位です。操作が複雑になるほどより多くのガスが必要になります。各ブロックに含められるガスの量には制限があるため、ブロックスペースの競争が生じます。需要が高いときユーザーは互いに入札し合い、手数料が上昇します。

ガス代の高騰が劇的な形で表れた事例があります。

CryptoKittiesブーム（2017年）

イーサリアムの限界が初めて本格的に試されたケースです。デジタル猫を繁殖させたり取引したりするシンプルなゲームに、多くのユーザーが殺到しネットワークが著しく詰まりました。トランザクション時間は大幅に遅延し、ガス代は急騰しました。

DeFiサマー（2020年）

UniswapやCompoundのようなDeFiアプリの爆発的な普及により、イーサリアム上のトランザクション量が急増しました。トレーダーたちはメモリプールで優先的に処理してもらうため、時には数百ドルのガス代を支払ってトランザクションを急ぎました。

NFTブーム（2021年）

NFTのドロップはガスウォーズとなり、人々は新しいデジタルコレクティブルを他の誰よりも先にミントするためだけに数千ドルを支払いました。法外なガス代を費やしても失敗するトランザクションもありました。

イーサリアムのLayer 1は、このレベルの需要を効率的に処理するようには設計されていませんでした。しかし2021年に導入されたEIP-1559は、可変ブロックサイズと新しいガス価格メカニズムを導入し、ネットワーク活動が活発な時期のガス代の急騰を緩和しました。L2ソリューションの普及に伴い、計算負担のかなりの部分がオフロードされ、ガス代がさらに削減されています。さらに最近ではEIP-4844（プロトダンクシャーディング）が展開され、特にL2ロールアップの手数料が大幅に低下し、ユーザーにとってイーサリアムのトランザクションがより手頃になりました。しかしこれらの改善を経てもなお、イーサリアムのトランザクションコストは他のほとんどのブロックチェーンより依然として高い状態です（図16-1参照）。

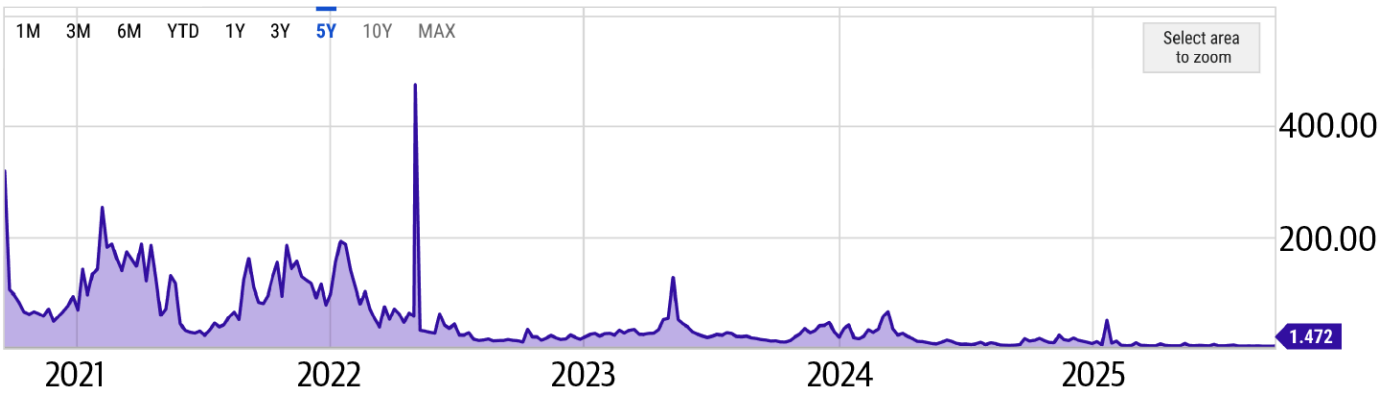


図16-1 ブロックチェーン間のガスコスト比較

ステートの増大とストレージの問題

イーサリアムはトランザクションだけに使われているわけではありません。イーサリアムは*グローバルステートマシン*であり、アカウント残高、スマートコントラクトデータ、DeFiポジション、NFTなどのオンチェーンアクティビティを継続的に追跡しています。従来のデータベースは古い記録をアーカイブまたは削除できますが、イーサリアムは透明性と検証可能性を確保するために完全な履歴を保持するよう設計されています。イーサリアムが成長するにつれて保存すべきデータが増え、ストレージ負担が増し続けていることが問題です。

現在、イーサリアムのステート、つまりすべてのアクティブなアカウント、コントラクト残高、ストレージスロットの集合は、驚くべき速度で増大しています。新しいスマートコントラクトが追加されるたびにステートが増え、すべてのトランザクションがステートを変更します。ネットワークの整合性を検証するうえで重要な役割を果たすフルノードは、このデータを保存し常に更新し続けなければなりません。ステートが増大するにつれフルノードの運用には高価なハードウェアが必要となり、分散化への懸念が生じています。

アーカイバルノードはさらに大きな課題に直面しています。アーカイバルノードは現在のステートだけでなく、過去のすべてのトランザクションやコントラクト実行を含むイーサリアムの完全な履歴も保存します。このデータの量はテラバイト規模に達し、大容量のストレージと帯域幅が必要です。アーカイバルノードを運用できる人の数は減少しており、誰がイーサリアムの長期的な履歴を保存するのかという疑問が生じています。

イーサリアムのPoSシステムでブロックの提案と証明を担当するバリデータも、ステート増大の影響を受けています。トランザクションを効率的に検証するには最新のブロックチェーンステートへの迅速なアクセスが必要です。しかしステートが拡大するにつれ、情報へのアクセスと処理は遅くなりコストも増大します。この傾向が抑制されないまま続けば、高性能なハードウェアを持つ参加者だけがバリデーションに参加できる環境となり、中央集権化に向かうリスクがあります。

イーサリアムの開発者たちは、ステートの肥大化を抑制するために履歴有効期限やステートレントなどのソリューションを検討してきました。これらについては本章の後半「L1のスケーリング」で詳しく説明します。

クライアント多様性も役立ちます。Gethは歴史的にイーサリアムの主要クライアントでしたが（図16-2参照）、Nethermind、Erigon、Besuなどの代替クライアントがストレージ効率を改善する最適化を導入しています。たとえばErigonは履歴データの効率的な処理に特化しており、フルノードの負担を軽減しています。

図16-2 イーサリアムクライアントの分布

ブロック伝播とMEV

イーサリアムがより高いトランザクションスループットを処理できたとしても、もう1つの根本的なボトルネックがあります。新しいブロックがネットワーク全体に伝播するのにかかる時間です。バリデータが新しいブロックを生成すると、そのブロックは世界中の数千のノードにブロードキャストされなければなりません。ブロックが大きいほど伝播に時間がかかり、伝播に時間がかかるほどネットワークの不一致、つまり一時的なフォーク（ネットワークの異なる部分が一時的に分岐する現象）が発生する可能性が高まります。

PoSはこれらのリスクを軽減するのに役立っていますが、ブロック伝播の遅延は依然としてパフォーマンスに影響しています。クライアントチームは処理を高速化するためのネットワーク最適化に取り組んでいますが、イーサリアムのスケーリングを進めるにあたり引き続き改善が必要な課題です。

水面下に潜むもう1つの問題がMEVです。イーサリアムはPoSに移行しましたが「miner extractable value」という名称が定着しています。今日ではより正確に*最大抽出可能価値（maximal extractable value）*と呼ばれています。MEVとは、バリデータやサーチャーがブロック内のトランザクションを戦略的に並べ替えたり含めたり除外したりして得られる利益を指します。

MEVが発生するのは、トランザクションが送信された順序で処理されるとは限らないためです。バリデータは自身の利益動機に基づいてトランザクションに優先順位をつけることができ、高度な技術を持つ行為者が一般ユーザーに不利な方法で価値を抽出する機会が生まれます。高頻度取引ボットはメモリプール（ブロックに含まれるのを待つトランザクションの待機場所）をスキャンし、利益を得る機会を探しています。たとえばあるユーザーがUniswapのような分散型取引所で大口取引を送信した場合、ボットはその前に割り込んで資産を先に購入し、より高い価格で売り戻すことができます。これは*サンドイッチ攻撃*として知られ、フロントランニングの一形態であり、MEVの中でも最も悪名高い手法の1つです。

数年前から運用されている緩和策として、Flashbotsが開発したMEV-Boostプロトコルがあり、MEVをより民主的かつ透明にしています。しかし長期的な解決策はより根本的な再設計、すなわちネイティブな*提案者-構築者分離*（PBS）の実装です。PBSについては、次のセクションでさらに詳しく解説します。

注記

ほとんどのL2チェーンでは集中型のトランザクションシーケンサーに依存しているため、MEVはほぼ存在しない。単一の集中型シーケンサーは通常、受信した順序でトランザクションを処理するため、並べ替えやフロントランニングの機会がなくなる。この集中型アプローチはMEVを大幅に削減するが、分散化と検閲耐性に関するトレードオフが生じる。今後のL2開発では分散型シーケンシングメカニズムの導入により、これらのトレードオフのバランスを取ることを目指している。

ソリューション

イーサリアムのスケーリング課題を克服するために何が行われているのでしょうか。単一の簡単な解決策はありませんが、開発者たちはネットワークの高速化、手数料の低減、大規模な普及への対応に向けてさまざまな改善に積極的に取り組んでいます。主要な戦略を詳しく見ていきましょう。

L1のスケーリング

イーサリアムのベースレイヤーであるLayer 1のスケーリングは、初日から直面している最も困難な課題の1つです。すべてを解決する単一の修正はなく、スケーリングはスイッチのように切り替えられるものではありません。分散化やセキュリティを犠牲にすることなくイーサリアムを段階的に効率化していく、長期的な最適化の積み重ねです。L2ロールアップがトランザクションの大部分を処理する最善の手段ですが、イーサリアムのベースレイヤーの改善も依然として重要です。L1でスループットと効率を向上させることができればロールアップはさらに強力になり、ガス代は下がり、中央集権化に頼ることなくイーサリアムは競争力を維持できます。イーサリアムのベースレイヤーを改善する主要な方法を見ていきましょう。

ガスリミットの引き上げ

イーサリアムのブロックは、格納できるトランザクションの数ではなく、各ブロックに収まるガスの量で制約されています。これが*ガスリミット*です。予算のようなものと考えてください。すべてのトランザクションはその複雑さに応じてガスを消費し、ブロック内の利用可能なガスはEIP-1559のメカニズムで定義されます。ガスリミットを引き上げれば各ブロックにより多くのトランザクションを格納でき、イーサリアムのスループットを実質的に向上させられます。

しかしガスリミットを単純に引き上げればよいというものではありません。ブロックが大きくなるとネットワーク全体への伝播に時間がかかり、チェーンスプリットの影響を受けやすくなります。また、フルノードのハードウェア要件も増加し、中央集権化に近づいてしまいます。そのためガスリミットの引き上げは、スループットの改善とネットワークの健全性のバランスを取りながら段階的かつ慎重に行われます（図16-3参照）。

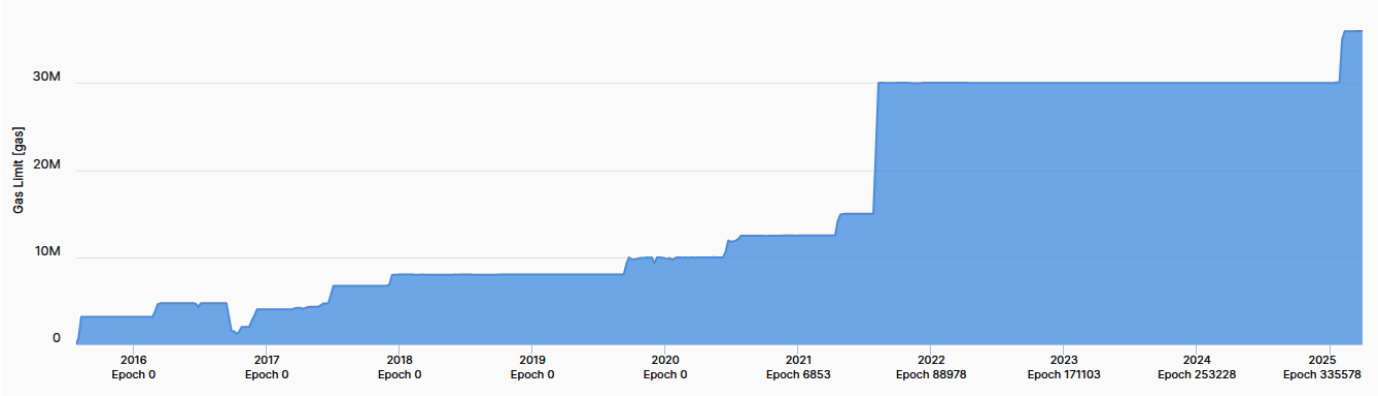


図16-3 ガスリミットの歴史的な変化

イーサリアムにおける並列実行の将来

イーサリアムは共有ステートモデルのためにトランザクションを逐次的に処理しています。セキュリティと一貫性は確保されますが、トランザクションを並列に実行できないためスケーラビリティが制限されます。一方、SolanaやAptosのような新しいブロックチェーンは並列実行を採用していますが、より中央集権的なアーキテクチャに依存しておりバリデータには高性能なハードウェアが必要です。

イーサリアムにおける課題は、トランザクションが同じステートと頻繁にやり取りすることです。たとえば2つのDeFi取引が同じ流動性プールを変更するケースです。堅牢な依存関係管理システムなしにトランザクションの順序を変更するとスマートコントラクトのロジックが壊れる可能性があります。さらに、フルノードがすべてのトランザクションを検証しなければならず、並列実行を導入するにはスレッド間の慎重な同期が必要です。

イーサリアムの研究者たちは、分散化を維持しながら部分的な並列実行を導入するためのソリューションを積極的に検討しています。1つのアプローチは*ステートレス実行*で、フルノードストレージへの依存を減らし、トランザクション処理をより効率的にします。もう1つは*オプティミスティック並行性*で、トランザクションは独立していると仮定し、競合が発生した場合にのみロールバックする方式です。これらの概念については、本章の後半で詳しく説明します。

並列EVMのいくつかの実験的実装が、Monad、Polygon PoS、ShardeumなどのEVM互換チェーンで登場しています。たとえばMonadは10,000TPSを超えるオプティミスティック並列実行モデルを実装しています。Polygon PoSはBlock-STMアプローチにより1.6倍のガススループット向上を達成し、トランザクションの部分的な並列化を可能にしています。これらの進歩は貴重な洞察を提供しますが、イーサリアムは分散化を保ちながら並列化を実装しなければならず、このバランスが依然として重要な課題です。最近の研究ではイーサリアムのトランザクションの約64.85%が並列化可能であることが示されており、パフォーマンス向上の大きな可能性が示唆さ

れています。しかし2025年3月時点で、並列実行をイーサリアムのメインネットに統合する具体的な計画はありません。ブロック末尾の仮想トランザクションを通じたEVMの並列化に関する議論は進行中ですが、イーサリアムの実行モデルの複雑さが実装を困難にしています。イーサリアムのスケーラビリティに向けたロードマップには、トランザクション依存関係の解決、代替実行モデル、EVMの効率の段階的改善に関する継続的な研究が含まれています。

ステートの増大と有効期限

イーサリアムのステート（すべてのアカウント残高、スマートコントラクトストレージなどのオンチェーンデータの集合）は増大し続けています。新しいコントラクトが追加されるたびにデータが増え、一度イーサリアムのステートに書き込まれたデータは永遠にそこにとどまります。分散化と検証可能性の観点では望ましいことですが、スケーラビリティにとっては問題です。フルノードはこのすべてのデータを保存し処理しなければならず、ステートが大きくなるにつれてノード運用コストは増加します。

現在、イーサリアムのステートサイズはフルノードで約1.2TBですが、完全な履歴ステートとトランザクションデータを保存するアーカイバルノードは21TB以上のストレージが必要です。この膨大なデータフットプリントにより、個人がアーカイバルノードを運用することは困難になりつつあり、この役割は資金力のある少数のエンティティに集中し始めています。なお、ErigonとRethの実行クライアントはストレージの使用量が少なく最適化されており、アーカイブノードでも約2TBで済みます。

ステート有効期限と履歴有効期限は混同されがちですが、異なる問題に対処しています。*ステート有効期限*は、スマートコントラクトが消費するストレージに対して定期的にレントを支払うことを要求し、イーサリアムのアクティブに維持されるステートのサイズを削減するものです。コントラクトが支払いを行わない場合、誰かが明示的に復活のための支払いを行うまでアクセスできなくなります。これによりステートの増大が大幅に抑制され、フルノードの運用が容易になります。

一方、*履歴有効期限*は過去のトランザクションデータの膨大なサイズに対処します。すべてのノードにすべての履歴トランザクションの保存を強制する代わりに、イーサリアムは古いデータをプルーニングし外部ストレージソリューションにオフロードできるようになります。ライブステートには影響しませんが、履歴クエリはサードパーティのデータプロバイダへの依存が強まります。両方のアプローチにはトレードオフがあり、効率性とアクセシビリティの最適なバランスを見極めるための研究が進行中です。

このトピックをさらに深く知りたい方は、履歴有効期限を扱うEIP-4444を参照してください。ステート有効期限についてはまだ研究段階で明確な戦略はありませんが、イーサリアムのロードマップで詳しい情報を確認できます。

提案者-構築者分離

MEVはPoSへの移行後もイーサリアムで根強い問題です。バリデータや専門のサーチャーが一般ユーザーを犠牲にしてトランザクションの並べ替え、フロントランニングなどの戦略で利益を

抽出しています。これは経済的な問題だけでなくネットワークの健全性にも影響し、混雑の増加やガス代の予測困難を招いています。

PBSはMEVを緩和するための最も有望なソリューションの1つです。現在、バリデータはブロックの提案と構築の両方を行っておりトランザクションの順序を完全に制御しています。PBSはこれらの役割を分離します。バリデータは引き続きブロックを提案しますが、実際のブロック構築は競争入札を通じて専門のビルダーに委託されます。これによりバリデータがMEV抽出に関与する直接的なインセンティブが取り除かれ、トランザクションの組み込みがより透明になります。

PBSは、バリデータがブロック構築を最高入札者に委託できるMEV-Boostの形ですでにテストされています。ただしMEV-BoostとPBSはMEVを排除するわけではなく、より透明で公平なものにするだけである点を理解することが重要です。裁定取引、フロントランニング、サンドイッチ攻撃を駆動する根本的なインセンティブがなくならないためMEVは引き続き存在します。PBSが保証するのは、少数のインサイダーが不透明なMEV戦略から利益を得るのではなく、MEVの獲得プロセスがよりオープンで公平かつ競争的になることです。長期的にはオーダーフローオークション、暗号化メモリプールなどのMEV緩和技術がPBSと並行して統合され、MEVの悪影響をさらに低減する必要があります。

ロールアップ

イーサリアムはスケーラビリティ、トランザクションスループット（TPSで測定）、高い手数料に関する課題に継続的に直面してきました。これらの問題に対処するためにロールアップの概念が登場しました。

*ロールアップ*は、専用のL2上で「オフチェーン」でトランザクションを実行し、集約（「ロールアップ」）されたデータまたは証明をL1ブロックチェーンに戻して投稿するメカニズムです。計算とステート更新の重い処理がL1から離れた場所で行われるため、通常のスループットボトルネックを回避しトランザクション速度が向上して手数料が低下します。ロールアップ自体の実行環境が署名チェック、コントラクト実行、ステート遷移をより効率的に処理し、L1は権威ある「決済レイヤー」として機能し続けます。

ロールアップはL1のセキュリティをできる限り維持することを目指しています。セキュリティの目標はデータ可用性の確保、正しいステート遷移の検証、検閲耐性の提供です。データ可用性は、紛争が発生した場合に参加者が独立してチェーンのステートを検証したり、L1に投稿された（またはL1によって保証された）データに基づいて安全に資金を引き出したりできるよう、すべてのトランザクションとステート情報へのアクセスを要求します。ステート遷移の整合性は、L2での変更がネットワークルールに従って有効であることを保証し、通常は有効性証明（ゼロ知識証明など）または不正証明を通じて実現されます。検閲耐性は、単一のエンティティまたは少数の参加者グループがユーザーのトランザクションを無期限にブロックまたは保留できないことを保証します。

注記

データ可用性と有効性は安全なロールアップ実装の不可欠な要素であり、悪意ある行為者がトランザクションを偽造したり資金を盗んだり残高を不正に膨らませたりすることを防ぐ。データの有効性は通常、2つの主要なアプローチのいずれかに依存する。最初のアプローチはゼロ知識証明を使用するもので、L2トランザクションの各バッチに正しい実行を証明する暗号学的証明が付属する。この証明がL1に提出されるとスマートコントラクトが新しいステートルートを受け入れる前にその正当性を検証する。2番目のアプローチはオプティミスティックな仮定のもとで不正証明を使用する。L2オペレーターが新しいステートをL1に投稿し、誰でも不正行為の証拠を提示して異議を申し立てることができる。不正証明が認められた場合、無効なバッチは取り消され悪意ある行為者にはペナルティが科せられる。

有効性に加えて、データ可用性はL2オペレーターが消失したり不正な行動を取ったりした場合でもユーザーが常にチェーンを再構築できることを保証する。実現方法はさまざまである。一部のシステムではすべてのトランザクションデータをL1に直接保存する。多くの場合イーサリアムのcalldataとして、または最近ではblobとして保存し、ブロックチェーンのログに透明に記録する。他の設計ではオフチェーンのデータ可用性レイヤー、特殊なネットワーク、またはデータの維持と提供に暗号経済的インセンティブを提供する外部ストレージソリューションに依存する。ハイブリッドアプローチでは両方の方法を組み合わせることもある。重要な情報はオンチェーンに配置し、それほど重要でないデータはオフチェーンに保存する。

ロールアップはL1にデプロイされた専用のスマートコントラクトに依存しており、すべてのL2アカウントの正規ステートを維持しています。このコントラクトは現在のL2ステートを表すルート（一般的にマークルルート）を保存し、指定されたアクター（シーケンサー、アグリゲーター、またはオペレーターと呼ばれることもある）から送信された新しいトランザクションのバッチを受け入れ、その有効性を検証します。ロールアップのタイプに応じてゼロ知識証明のチェックや不正証明の処理を行い、ステート更新が正当であることを確認します。違反が検出された場合（たとえば不正証明が成功した場合）、コントラクトは無効なバッチを取り消せます。

ロールアップの要件（ボンドのステーキングなど）を満たす人は誰でもL2トランザクションのステート更新をスマートコントラクトに送信できます。ただし常にそうとは限りません。現時点ではトランザクション量とTotal Value Lockedが最も多いロールアップのほとんどがシーケンサーレベルで中央集権化されています。一部のロールアップは分散化を達成していますが、大多数にとってはまだ最終目標の段階です。各ステート更新には、前のステートルート（連続性を示すため）、新しく提案されたステートルート（送信されたトランザクションの結果を反映）、および圧縮されたトランザクションデータまたはそのデータへの参照が含まれます。ロールアップのルールが満たされた場合（オプティミスティックロールアップの場合は有効な異議申し立てがない場合）、コントラクトは保存されたステートルートを新しいものに更新し正規のL2ステートとします。

ロールアップの種類によって、不正または無効なステート更新への対処方法が異なります。オプティミスティックロールアップは新しいステート更新がデフォルトで正しいと仮定しますが、誰でも不正証明を提出できるチャレンジウィンドウ（多くの場合数日間）を設けています。証明

が検証されると無効なステート更新はロールバックされ、悪意ある提出者のステークはスラッシュされます。一方、ZKロールアップは新しいステートルートごとにゼロ知識証明の添付を要求します。この証明がオンチェーンで検証されるため、チェーンは更新の有効性を即座に判断できます。証明が正しければ長い異議申し立て期間は不要です。

ロールアップのステージ

初期段階ではほとんどのロールアップは部分的な中央集権的制御を保持しています。これは*トレーニングホイール*と呼ばれることが多く、バグや重要な更新が発生した場合にオペレーターが迅速に介入できるようにするものです。新しいシステムにとっては実用的ですが、真の分散化にはこのトレーニングホイールを段階的に取り除く必要があります。

この移行を示すために、ヴィタリック・ブテリンの最初のマイルストーンに基づくフレームワークが提案されており、ロールアップを3つの成熟度ステージに分類しています。各ステージは、中央集権的な手に残されている権限の量と、ロールアップがイーサリアムのベースレイヤーセキュリティをどの程度継承しているかを示しています。

*ステージ0*では、ロールアップはロールアップを自称していますが、依然としてオペレーターが大きく制御しています。ステートルートをL1に投稿しL1上でデータ可用性を提供することで、問題が発生した場合にL2ステートの再構築を可能にしています。しかしこの時点では、システムの「証明メカニズム」（不正証明または有効性証明）がオンチェーンのスマートコントラクトで完全に強制されていない場合があります。エラー発生時の主なフォールバックはオペレーターの介入です。ステージ0はロールアップの基本要素（オンチェーンデータ、ステートルート、ユーザー向けノードソフトウェア）が整っていることを保証しますが、ガバナンスは中央集権的なままです。

> **注記**
>
> この3段階フレームワークの要件は変更される可能性がある。たとえば、本稿執筆時点（2025年2月）では、Arbitrumはステージ1のロールアップだが、新しい要件では、ネットワークを期限内にアップグレードしなければステージ0のロールアップに戻る可能性がある。

*ステージ1*のロールアップは、適切な証明システム（オプティミスティックロールアップの場合は不正証明、ZKロールアップの場合は有効性証明）を備えている必要があり、これらの証明を提出できる外部アクターが少なくとも5人いなければなりません。ユーザーはオペレーターの協力なしにシステムから資金を引き出したり「退出」したりでき、検閲から保護されます。もう1つの基準は、ユーザーがシステムアップグレードの提案に同意しない場合に最低7日間の退出ウィンドウを設けることです。ただし「セキュリティカウンシル」は重大なバグが発生した場合により迅速に介入できます。このカウンシルは8人以上の署名者のうち少なくとも50%の承認を必要とするマルチシグで構成されなければならず[1]、その半数はロールアップの主要組織の外部

の者でなければなりません。このカウンシルはバグの修正や悪意あるトランザクションの取り消しを行えますが、依然として単一障害点となる可能性があります。

[1] これは最近変更され、要件が少し変わりました。本章ではこの変更を詳しく分析しません。詳しくはルカ・ドンノのMedium記事を参照してください。

*ステージ2*は、ロールアップが真に分散化されておりパーミッションレスな証明と堅牢なユーザー保護に依存していることを意味します[2]。不正証明または有効性証明システムは誰にでも開かれていなければならず、許可リストは不可です。ガバナンス提案やアップグレードが導入された場合、ユーザーには退出までに少なくとも30日間の猶予が与えられ、変更を強制されません。セキュリティカウンシルの役割は矛盾する証明の提出などのオンチェーンの健全性関連エラーに厳密に限定され、広範なガバナンスや裁量権は持ちません。この最終ステージでは人間の介入は狭い範囲に限定され、ロールアップはほぼスマートコントラクトとコミュニティのコンセンサスで統治されます。最小限の信頼というイーサリアムの精神に近づいています。

[2] ステージ2はより良いUXやより多くの採用を示すものではなく、より多くの分散化を示すだけである。

このフレームワークの開発と更新に関わったすべての方々、そしてロールアップのステージに関する情報の分析と公開に携わったすべての方々に感謝します。皆さまの活動は非常に高く評価されており、また必要不可欠なものです。

オプティミスティックロールアップ

オプティミスティックロールアップは不正証明に依存しています。オペレーターはステートルートが有効であるという前提でL1に投稿します。ただしオブザーバーは不正と思われるバッチに異議を申し立てる権利を保持しています。異議が正しいと証明された場合、無効なバッチは取り消されオペレーターにペナルティが科せられます。有効性が即座に確認されないため、ユーザーは資金の引き出しやファイナリティの達成前にチャレンジウィンドウ（場合によっては1週間以上）を待つ必要があります。この設計により引き出し時間は長くなりますが、EVMとの容易な互換性と低い証明の複雑さを実現しています。オペレーターはゼロ知識回路を構築する必要がないためシステム運用の一部が簡素化されます。ただし引き出しの遅延はユーザー体験に影響する可能性があり、オペレーターのボンドがロックされた総額より小さい場合には経済的攻撃の可能性も存在します。オプティミスティックロールアップの例としてはArbitrum、Optimism、Baseなどがあります。

ゼロ知識ロールアップ

ZKロールアップは有効性証明に基づいて動作します。プロバイダがL2上でトランザクションをバンドルする際、ステート遷移の正当性を証明する暗号学的証明（多くの場合SNARKまたはSTARK）を生成します。これらの証明はオンチェーンで検証され、長い異議申し立て期間が不要なためほぼ即時のファイナリティを提供します。正当性の確認に待機期間が不要なためユーザー

は高速な引き出しの恩恵を受けます。高いセキュリティは証明の直接検証から生まれ、ウォッチャーや大規模なオペレーターのステークへの依存を軽減します。オンチェーンでの証明の検証は各トランザクションを個別に処理するよりも通常は効率的です。しかし汎用EVMの計算に対するゼロ知識証明の実装は計算コストが高く、専用ハードウェアが必要になる可能性があります。

ゼロ知識証明のリスク

最先端の暗号技術にはリスクが伴うのが現実です。Zcashを例に取りましょう。Zcashは「Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture」という論文で提示された実装の一部を使用しました。この論文はZcashの最初のローンチで使用されたzk-SNARK構造を記述したものです。

2018年、この論文の公開と数十回のピアレビューから数年後、当時Zcashに雇用されていた暗号学者アリエル・ガビゾンが、偽造脆弱性（本質的には二重支払い攻撃）を可能にする微妙な暗号学的欠陥を発見しました。この脆弱性はZcash Saplingアップグレードで修正され誰にも悪用されなかったようですが、広く公開され引用されている論文の中で誰も気づかないまま何年も潜んでいたのです。

本章ではゼロ知識証明を高セキュリティで信頼できるものとして言及しています。一般的にはそのとおりですが、無条件に受け入れることは危険な仮定です。

特定のゼロ知識システムは、初期パラメータを生成するための*トラステッドセットアップ*[3]（SNARKでは一般的）も必要とし、独自のセキュリティ上の考慮事項があります。主要なZKロールアッププロジェクトには、ZKsync、Starknet、Scroll、Aztecがあります。Aztecは「ZK-ZKロールアップ」というラベルでプライバシー機能も組み込んでいます。

[3] 第4章で説明した通り。

ZKロールアップはもともとトークン送金やスワップなどの単純なタスクに適していましたが、より複雑な機能には苦戦していました。これは*zk-EVM*の登場で変わりました。zk-EVMはEVM全体をオフチェーンで再現することを目的とした開発です。チューリング完全な計算（EVMバイトコードの実行を含む）に対する証明を生成することで、zk-EVMはZKロールアップの範囲を拡大し、幅広いDAppがスケーラビリティとゼロ知識レベルのセキュリティの両方の恩恵を受けられるようにしています。

zk-EVM機能を実現するためにプロジェクトはさまざまな方法を採用しています。1つはトランスパイラを使用する方法で、Solidity（または他のEVM高水準言語）をCairo（StarkWareが使用）のような回路に適した言語に変換します。もう1つは標準的なEVMバイトコードをオペコードごとに直接解釈し、各命令を反映する回路を構築するアプローチです。ハイブリッドまたはマルチタイプのソリューションはEVMの一部（データ構造やハッシュアルゴリズムなど）を証明に適したものに調整しつつ、イーサリアムとのほぼ完全な互換性を維持しようとしています。zk-EVMについては本章ではこれ以上詳しく展開しません。第17章で詳述します。

その他のスケーリングソリューション

オプティミスティックロールアップとZKロールアップだけがスケーリングソリューションではありません。現時点では主に採用されている2つですが、将来的には変わる可能性があります。古くて今後関連性が低いものについては簡潔に、今後有望なソリューションについてはより詳しく分析します。

Validium

*Validium*はトランザクションデータをイーサリアム上に保存しません。代わりにL2チェーンのステートを検証する証明をイーサリアムに投稿します（図16-4参照）。本質的にvalidiumは代替的なデータ可用性ソリューション（Celestia、Avail、EigenLayerなど）を使用するロールアップです。

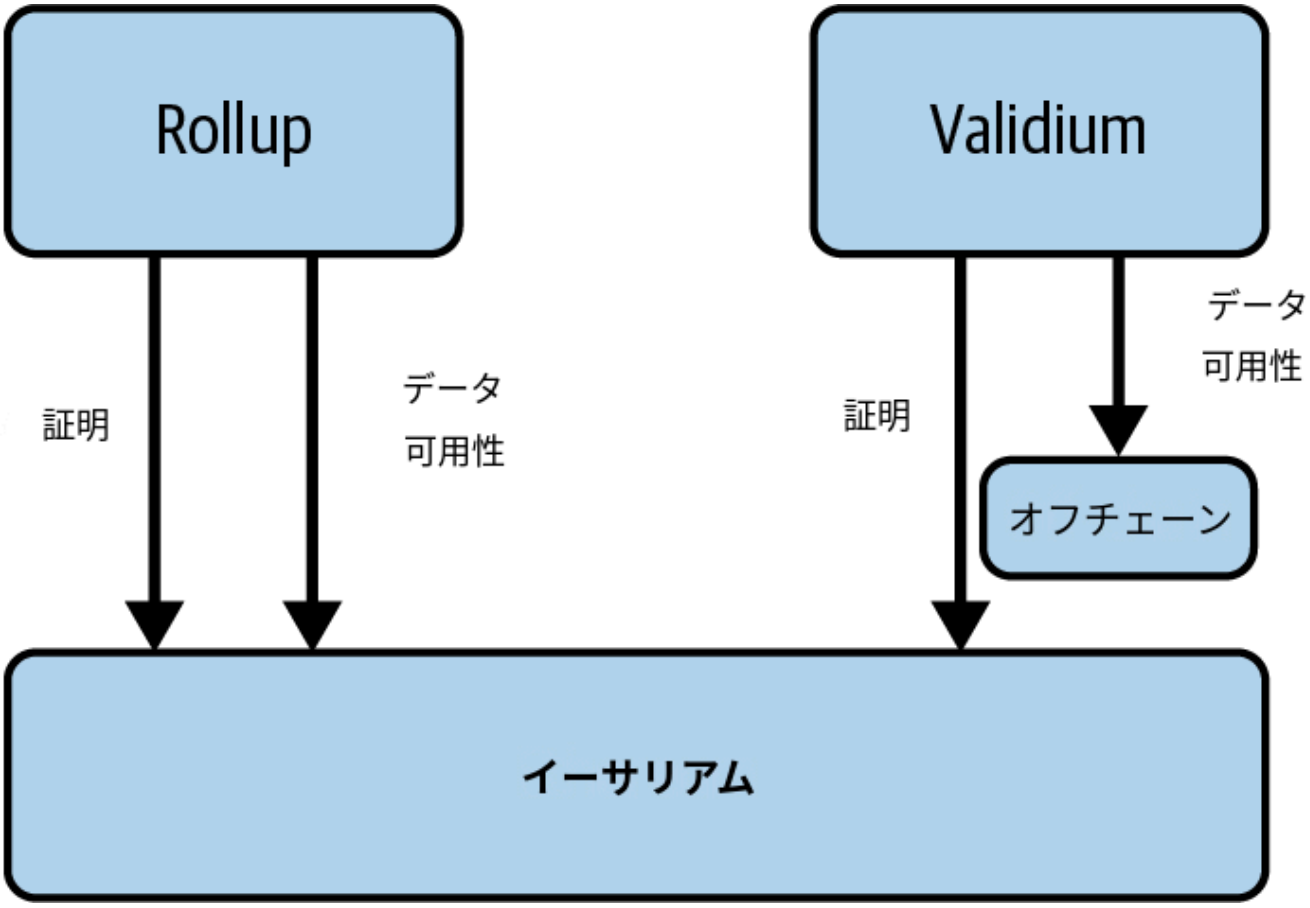


図16-4 Validiumアーキテクチャ

L2としてvalidiumはイーサリアム上にデータを保存する際の高いガス代を支払いません。このアプローチはロールアップよりコスト効率が高くユーザーのガス代は大幅に低くなります。しかしvalidiumはデータ可用性委員会や代替的なデータ可用性ソリューションなどを使用してイーサリアム外にトランザクションデータを保存するため、他のロールアップよりセキュリティが低いと一般的に考えられています。

サイドチェーン

*サイドチェーン*は、別のブロックチェーン（メインチェーン）と並行して動作するブロックチェーンネットワークです。通常サイドチェーンは双方向ブリッジを介してメインチェーンと接続し、資産やコントラクトのステート、マークル証明、特定のトランザクションの結果などのデータを2つのネットワーク間で転送できます。

ほとんどのサイドチェーンはメインチェーンとは別のコンセンサスメカニズムとバリデータセットを持っています。これによりサイドチェーンは他のブロックチェーンに依存せずにトランザクションの決済とファイナリティを実現できます。しかし同時にサイドチェーンにブリッジされた資金のセキュリティは、バリデータ間の悪意ある行動を抑止する強力な暗号経済的インセンティブの存在に依存します。

Basedロールアップ

*Basedロールアップ*は、L1ブロックチェーンのネイティブシーケンシング機能に依存しています。この設計により、L1の分散化、ライブネス、セキュリティ特性を活用したシームレスな統合が可能になります。

basedロールアップは従来のロールアップと比較してよりシンプルなシーケンシングアプローチを使用しています。ほとんどのロールアップは独自のシーケンサーを実装していますが、basedロールアップは基盤となるLayer 1のシーケンサーを利用します。基本的な違いは、オプティミスティックロールアップのように外部のシーケンサーを持つのではなくL1のバリデータがロールアップのシーケンサーを務める点です。

コンセンサス、データ可用性、決済レイヤーはすべてイーサリアムそのものです。ロールアップ自体に組み込まれている唯一のコンポーネントは実行レイヤーであり、トランザクションの実行とステートの更新を担当します。この設計によりL1ブロックプロポーザーはL2ブロックビルダーやサーチャーと直接連携して、次のロールアップブロックをL1ブロックに組み込めます。basedシーケンシングは既存のイーサリアムの検証アプローチのみに依存するため外部のコンセンサスは不要です。

> **注記**
>
> basedロールアップはおそらく最も有望なソリューションであり、利用においてオプティミスティックロールアップを超える可能性がある。ファイナリティがイーサリアム自体と同じであるため、チャレンジウィンドウなしでbasedロールアップ間のアトミックトランザクションが可能になるからである。たとえば、DeFiの流動性をbasedロールアップ間で集約できるようになる。以前は、これはZKロールアップでのみ可能だった。

ブースターロールアップ

*ブースターロールアップ*は、トランザクションがL1上で直接実行されたかのように処理しL1ステートへの完全なアクセスを付与するロールアップです。同時に独自の別個のストレージを維持します。実行とストレージの両方がL2レベルでスケールし、L1フレームワークは共有ベースとして機能します。別の見方をすると、各L2がL1を反映しトランザクション実行とストレージをシャーディングすることで、L1にデプロイされたすべてのアプリに新しいブロックスペースを効果的に追加しています。

イーサリアムの将来が高いスケーラビリティ需要を処理するために何百、何千ものロールアップの使用を求めるなら、各ロールアップが独自のルールセットとコントラクトを持つ独立したチェーンとして機能するのは理想的ではありません。開発者は各ロールアップにコードを複製することになってしまいます。

ブースターロールアップは、L1上で稼働しているあらゆるDAppに直接追加のブロックスペースを提供するという別のアプローチを取ります。ブースターロールアップの展開は、コンピュータにCPUコアやハードドライブ容量を追加するようなものです。アプリケーションがマルチスレッド（ブロックチェーンの文脈ではマルチロールアップ）の活用方法を知っている場合、拡張された容量を自動的に利用できます。開発者はその追加環境を最適に活用する方法を検討するだけで済みます。

ネイティブロールアップ

ネイティブロールアップの提案は、ロールアップのステート遷移の検証器として機能する`EXECUTE`プリコンパイルを導入するものです。これにより不正証明ゲーム、SNARK回路、セキュリティカウンシルなどの複雑なインフラストラクチャが不要になり、EVM等価なロールアップの開発が大幅に簡素化されます。`EXECUTE`を使えばわずか数行のSolidityコードで最小限のネイティブかつbasedなロールアップをデプロイできます。

この新しいプリコンパイルは「EVM in EVM」のアイデアに密接に類似しているため、社会的コンセンサスで統治されるイーサリアムの通常のハードフォークプロセスを通じてアップグレードされます。この整合性によりEVMへの更新がプリコンパイルにも適用されるため、ロールアップはセキュリティカウンシルやマルチシグなどのガバナンス構造なしにイーサリアムの検証ルールを継承でき、エンドユーザーにとってより安全なものとなります。

`EXECUTE`プリコンパイルはEVMのステート遷移を検証し、ロールアップがアプリケーションレイヤーでイーサリアムのネイティブインフラストラクチャを活用できるようにします。`pre_state_root`、`post_state_root`、`trace`、`gas_used`などの入力を使用してEVMのステート遷移をチェックし、ガス価格設定にはEIP-1559のようなメカニズムを採用しています。正当性を強制するためにバリデータは各ロールアップのスケールと複雑さのバランスに応じて再実行またはSNARK証明に依存する場合があります。この設計はbasedロールアップアプローチ（シーケンシングと証明システムの両方がイーサリアム上で直接動作する）と組み合わせることで「トラストレスロールアップ」の作成を簡素化します。

ダンクシャーディング

ダンクシャーディングは、以前の設計に比べて大幅な簡素化を提供する、イーサリアムの最新のシャーディングアプローチです。

> **注記**
>
> シャーディングについて初めて言及するので、概念を簡単に紹介しておく。これはさらに深く研究できるトピックであることを念頭に置いてほしい。本書の範囲外だが、興味深いトピックである。
>
> シャーディングの基本的な概念は、ネットワークを異なる部分に分割し、各シャードがトランザクションのサブセットを処理することで、パフォーマンスとコストを改善することである。
>
> シャーディングは、他のチェーンでさまざまな形ですでに実装されている。イーサリアムでは、数年前に概念化された最初の純粋なシャーディングのアイデアには実際には到達しない。もはや必要ないからだ。ロードマップはL2中心の方法でL1をスケーリングする方向に進んでいる。

イーサリアムの最近のシャーディング提案（ダンクシャーディングとプロトダンクシャーディングの両方）と他のほとんどのシャーディングソリューションとの重要な違いは、イーサリアムのロールアップ中心戦略にあります。トランザクション容量を直接拡大するのではなく、コアプロトコルが解釈しない大きなデータの「blob」のためのスペースをより多く提供することに焦点を当てています（blobについては後のセクションでさらに詳しく解説します）。これらのデータblobを検証するための重要な要件はデータが依然としてアクセス可能であること、つまりネットワークから取得できることの確認です。L2ロールアッププロトコルはこれらのデータblobを使用して高スループットトランザクションを実現します（図16-5参照）。

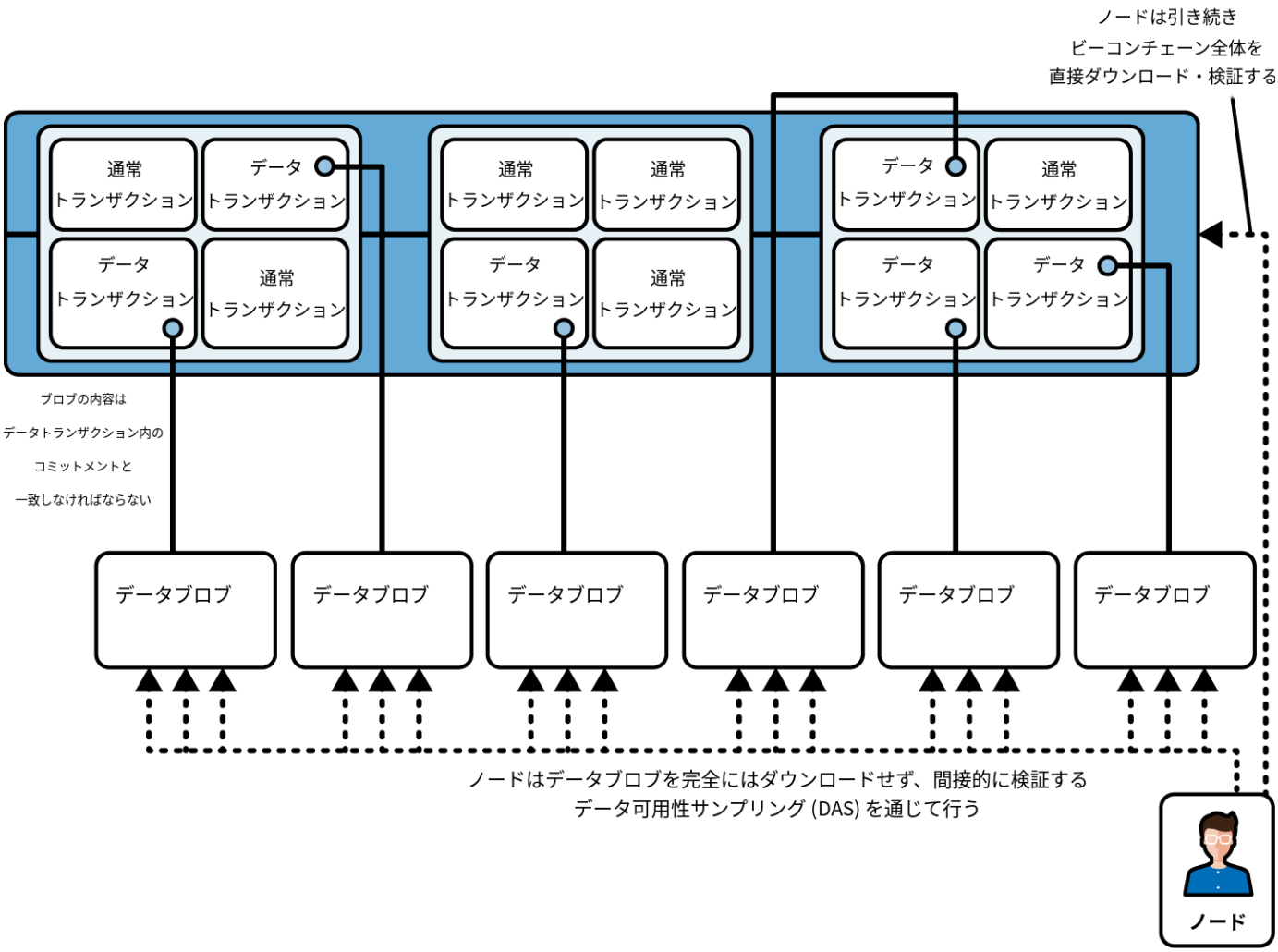


図16-5 ダンクシャーディングのアーキテクチャ

ロールアップは実行されたトランザクションをデータblobに保存し、検証のための暗号学的「コミットメント」[4]を生成します。このコミットメントを作成するために、データを多項式関数を使用してエンコードします。この多項式は特定のポイントで評価できます。

[4] 多項式コミットメントは、多項式（*f(x) = 2x − 1*のような数学的表現）の短くて安全な「要約」であり、多項式全体を公開せずに特定の値を証明できるものである。秘密の公式を書き留め、鍵のかかった金庫に入れることを想像してほしい。後で誰かが「*x = 10*にした場合の結果は何ですか」と尋ねることができる。金庫を開けたり完全な公式を見せたりすることなく、正しい答えが特定の数であることの証明を素早く簡単に提供できる。多項式コミットメントはまさにこれを数学的に行う。より大きな数学的データセット内の特定のポイント（*x = 10*など）の正確さを、すべての詳細を公開せずに安全かつ効率的に証明できる。多項式コミットメントのより詳細な説明は第4章にある。

たとえば、*f(x) = 2x − 1*のような単純な多項式関数を考えてみましょう。*x = 1, 2, 3*でこの関数を評価すると、値1, 3, 5が得られます。証明者は独立して同じ多項式を元データに適用し、これらのポイントでの値を確認します。基礎となるデータがわずかでも変更されると、多項式、したがって評価値が一致しなくなり、参加者に不整合を警告します。

ロールアップがblob内にデータを投稿する際、オンチェーンで公開されるコミットメントも提供します。このコミットメントは、データに多項式を当てはめ、KZGセレモニー（第4章で説明）で生成された乱数によって決定される特定のポイントでその多項式を評価することで作成されます。証明者は独立して同じポイントで多項式を評価します。評価がコミットメントの値と一致すれば、データは正確であることが確認されます。実際には、これらのコミットメントとその証明は暗号学的手法で保護されているため、より複雑です。

ダンクシャーディングの主な進歩は統合手数料市場の導入です。これはガス手数料市場とblob手数料市場の組み合わせを意図しています。複数のシャードがそれぞれ独自のブロックとプロポーザーを持つのではなく、ダンクシャーディングでは各スロットのすべてのトランザクションとデータを選択する単一のプロポーザーを指定します。

このアプローチがバリデータに高いハードウェア要件を課すことを防ぐためにPBSが使用されます。PBSでは専門の「ブロックビルダー」がスロットのコンテンツを組み立てる権利を得るために入札で競争し、プロポーザーは最高入札額の有効なヘッダーを選択するだけです。フルブロックを処理する必要があるのはブロックビルダーのみであり、このステップも専用のオラクルプロトコルでさらに分散化できます。一方、バリデータとユーザーはDASに依存してブロックを検証できます。ブロックの大部分はデータに過ぎないことを忘れないでください。

> **注記**
>
> *ダンクシャーディング*と*プロトダンクシャーディング*という用語には、このシャーディングの形成に貢献したイーサリアムファウンデーションの2人の主要人物の名前が含まれている。ダンクはダンクラート・ファイストを、プロトはプロトラムダ、別名ディーデリック・ローラッカーを指している。両者とも研究者であり、ファイストはイーサリアムファウンデーションで、ローラッカーは執筆時点でOP Labsで勤務している。

プロトダンクシャーディング

プロトダンクシャーディング（EIP-4844とも呼ばれる）は、完全なダンクシャーディングに必要な基礎的なロジックとインフラストラクチャ（トランザクションフォーマットや検証ルールなど）のほとんどを実装する提案ですが、実際のDASは含まれていません[5]。プロトダンクシャーディングではバリデータとユーザーが依然としてすべてのデータblobの完全な可用性を直接検証します。

[5] データ可用性サンプリングは、ブロックのすべてのデータをダウンロードすることなくデータ可用性を検証するメカニズムである。

プロトダンクシャーディングで導入された主要なイノベーションは*blob搬送トランザクション*と呼ばれる新しいトランザクションタイプです。これらのトランザクション（すでに第6章で分析済み）は通常のトランザクションと同様に機能しますが、blobと呼ばれる追加のデータコ

ンポーネントを含んでいます。blobは比較的大きく（約125KB）、同等のcalldataよりも大幅に安価です[6]。ただしblobデータはEVMから直接アクセスできません。EVMはblobへの暗号学的コミットメントにのみアクセスできます。これらのblob内のデータは一定期間（執筆時点で4,096エポック、約18日間に設定）が経過すると自動的に削除されます。

[6] EVMにおける外部呼び出し時に渡される関数引数の保存に使用される生データ。

バリデータとクライアントは各blobの完全な内容をダウンロードしなければならないため、プロトダンクシャーディングではデータ帯域幅が1スロットあたり約1MBに制限されています。この制限があってもblobデータは標準的なイーサリアムトランザクションのガスコストと直接競合しないため、プロトダンクシャーディングは依然として大きなスケーラビリティの恩恵を提供します。

ここまで読むと、いくつかの疑問が浮かぶかもしれません。

18日後にblobデータを削除しても問題ないのはなぜですか。古いblobにはどうやってアクセスするのですか

ロールアップはトランザクションデータへの暗号学的コミットメントをオンチェーンに公開し、同時に基礎となるトランザクションデータをデータblobを通じて利用可能にします。この仕組みにより独立した証明者がコミットメントの正確性を検証したり、必要に応じて不正確なデータに異議を申し立てたりできます。ネットワークレベルではコンセンサスクライアントがこれらのデータblobを一時的に保存し、データがイーサリアムネットワーク全体に伝播され利用可能になったことを証明します。ノードが過度に大きくリソース集約的にならないようこのデータは18日後に自動的にプルーニングされます。コンセンサスクライアントによる証明はこの期間中に証明者がデータを検証または異議申し立てするための十分な時間とアクセスがあったことを保証します。プルーニング後も実際のデータはロールアップオペレーター、ユーザー、その他のサードパーティがオフチェーンに保存し続けることができます。

履歴データを長期的に保存し容易にアクセスできるようにするための実用的な方法がいくつかあります。たとえばアプリケーション固有のプロトコル（個々のロールアップなど）は、自身のアプリケーションに関連する履歴データを独自のノードに保持するよう要求できます。履歴データの損失はイーサリアム自体にはリスクをもたらさず個々のアプリケーションにのみ影響するため、各アプリケーションが独立してデータストレージを管理するのは合理的です。その他のソリューションとしては、BitTorrentのような分散型システム（たとえばblobデータの日次スナップショットを定期的に生成・配布する用途）や、イーサリアムのPortal Network（履歴データストレージをサポートするよう拡張可能）の活用があります。さらにブロックエクスプローラー、APIプロバイダ、The Graphのようなサードパーティインデキシングプラットフォームなどのサービスも包括的なアーカイブを維持する可能性があります。データ分析を行う研究者、愛好家、学術機関も、データへの直接アクセスの利便性とパフォーマンスの恩恵を受けながら完全な履歴記録をローカルに保持できます。

blob用に1スロットあたり1MBの帯域幅を確保するのではなく、通常のトランザクションのcalldataコストを削減した方がよいのではないですか

ここでの問題はイーサリアムネットワークの平均負荷と最悪ケース（ピーク時）の負荷の違いに関連しています。現在、イーサリアムのブロックは平均約90KBですが、理論上の最大ブロックサイズ（ブロック内の3,600万ガスすべてがcalldataのみに使用された場合[7]に達成される）は約2MBです。イーサリアムはこの最大サイズに近いブロックを大きな問題なく処理した実績があります。しかし単純にcalldataのガスコストを10分の1に引き下げて平均ブロックサイズが管理可能なままであっても、最悪ケースのブロックサイズは約20MBに急増しイーサリアムネットワークを圧倒することになります。

[7] calldataとは、直接実行されるのではなく、主に記録保持と検証の目的でオンチェーンに投稿されるデータを含むトランザクションの部分を指す。

イーサリアムの現在のガス価格モデルでは平均負荷シナリオと最悪ケースのシナリオを個別に管理することが困難です。この2つの比率はユーザーがcalldataとその他のリソース間でガス支出をどう配分するかに依存するためです。その結果イーサリアムは最悪ケースのシナリオに基づいてガス価格を設定しなければならず、ネットワークが快適にサポートできる水準よりも人為的に平均負荷を制限しています。多次元手数料市場を導入しガス価格が異なるリソースタイプを明示的に区別すれば、平均ネットワーク使用量を実際の容量により適切に整合させ、1ブロックあたりにより多くのデータを安全に収容できるようになります。プロトダンクシャーディングとEIP-4488はイーサリアムのガス価格モデルを改善することでこの問題に対処するために設計された2つの提案です。

警告

EIP-4488をEIP-4844と混同してはいけない（これらのEIPの紛らわしい番号には感謝である）。EIP-4488は、平均ケースと最悪ケースの負荷不一致という同じ問題を解決しようとする、より初期のシンプルな試みである。現在は停滞しているため、おそらく実装されることはないだろう。一方、EIP-4844はすでに稼働している。

もう1つの隠れた動機は、最終目標としてノードがすべてのデータblobをダウンロードしなくてもよくなることです。これはblobでは可能ですが、calldataでは実現できません。

ヒント

さらに多くの疑問があるかもしれない。回答については、本章末尾の参考文献の完全なリストを参照してほしい。

ステートレスイーサリアム

控えめなハードウェアでイーサリアムノードを運用できることは真の分散化を実現するうえで極めて重要です。ノードを運用すればユーザーはサードパーティに頼ることなく暗号学的チェックを通じてブロックチェーン情報を独立して検証できます。また仲介者なしにイーサリアムのP2Pネットワークに直接トランザクションを送信することも可能です。これらの利点が高価な機器を持つユーザーだけに限定されるなら真の分散化は達成できません。イーサリアムのノードは最小限の処理能力とメモリ要件を持ち、理想的にはスマートフォン、マイクロコンピュータ、または家庭用コンピュータで目立たずに動作できるものでなければなりません。

現在、イーサリアムノードの運用への広範なアクセスを阻む主要な障害は、高いディスクスペース要件です。この主な原因は、新しいブロックとトランザクションを正しく処理するために重要な、イーサリアムの広範なステートデータを保存する必要性にあります。

より安価なハードドライブで古いデータを保存できますが、受信ブロックを効率的に管理するには通常遅すぎます。ストレージを単に安価にしたり効率的にしたりするのは一時的な緩和にしかなりません。イーサリアムのステートデータの増大は事実上無制限であり、ストレージのニーズは継続的に増加するためです。より持続可能なアプローチは、ローカルストレージからのデータ取得に依存しないブロックとトランザクションの検証のための新しいクライアント手法を開発することです。

*ステートレス性*という用語はやや誤解を招く可能性があります。ステートの概念を完全に排除するものではなく、イーサリアムノードがステートデータを管理する方法を変更するものだからです。ステートレス性には主に2つのタイプがあります。弱いステートレス性と強いステートレス性です。*弱いステートレス性*は限定された数の専門ノードにステートデータの保存責任を移すことで、ほとんどのノードがステートデータを保存せずに動作できるようにします。一方、*強いステートレス性*はすべてのノードから完全なステートデータの保存要件を取り除きます。

ヒント

以下のサブセクションでは弱いステートレス性と強いステートレス性について説明する。「ステートレスイーサリアム」を実現する方法はこれらだけではないことも述べておく。ステートレス性は基本的にイーサリアムノードがステートデータを管理する方法を変更することを意味する。もう1つの方法はHeliosのようなライトクライアントを使用することである。Heliosは信頼できない集中型RPCエンドポイントを、安全で操作不可能なローカルRPCに変換する。モバイルデバイスで実行できるほど軽量で、ストレージもほとんど必要としない。

弱いステートレス性

*弱いステートレス性*は第14章で簡単に触れましたが、イーサリアムノードがステート更新を検証する方法の変更を伴います。ただしネットワーク全体でのステートストレージの必要性を完全に排除するものではありません。代わりにブロックプロポーザーまたはビルダーと呼ばれる専

門ノードに完全なステートデータの保存責任を置きます。ネットワーク上の他のすべてのノードは完全なステートデータをローカルに維持せずにブロックを検証できるようになります。弱いステートレス性では新しいブロックの作成（提案）にはイーサリアムのステートデータへの完全なアクセスが必要ですが、ブロックの検証はステートデータを一切保存せずに行えます。

弱いステートレス性の実装は、イーサリアムのクライアントが*ヴァークルツリー*（次のセクションで詳述）と呼ばれる新しいデータ構造を採用することに依存しています。ヴァークルツリーはイーサリアムの現在のステートストレージ構造に取って代わり、ノードがローカルデータベースを参照せずにブロックを検証するために交換する小さく固定サイズのウィットネス[8]の作成を可能にします。さらにPBSも必要です。PBSによりより強力なハードウェアを持つ専門ノードであるブロックビルダーが完全なステートデータの維持という集約的なタスクを処理でき、通常のノードはステートレスに動作できます。

[8] ブロックの検証とは、ブロック内のトランザクションを再実行し、イーサリアムのステートを更新し、計算されたステートルートがブロックプロポーザーによって提供されたものと一致することを確認することである。イーサリアムのクライアントは現在、ブロックを検証するためにローカルに保存されたステートトライ全体を必要としている。ウィットネスには、ブロックのトランザクションを実行するために必要なステートデータの一部のみが含まれる。しかし、従来のマークルツリーを使用すると、ウィットネスが大きくなりすぎ、イーサリアムの12秒のスロット時間内にノードがダウンロードして処理することが困難になる。この制限は高速インターネット接続を持つノードに有利であり、中央集権化につながる。ヴァークルツリーはウィットネスのサイズを大幅に削減することでこの問題を解決し、ステートのローカルストレージを必要とせずにステートレス検証を可能にする。

ヴァークルツリー

*ヴァークルツリー*という用語は「ベクトルコミットメント（vector commitment）」と「マークルツリー（Merkle tree）」（第14章で説明）を組み合わせたものです。ヴァークルツリーはイーサリアムのステート全体をローカルに保存せずにブロックを検証するステートレスイーサリアムクライアントの実現に不可欠です。これらのクライアントは有効性を確認する暗号学的証明が付随するウィットネス[9]に依存しています。ウィットネスのサイズが小さいことは極めて重要です。ウィットネスはイーサリアムの12秒のスロット内でノードが効率的に配布・処理しなければならないからです。現在のマークルベースのステートデータ構造は過度に大きなウィットネスを生成しステートレス検証には不適切です。ヴァークルツリーはウィットネスのサイズを大幅に削減することでこの問題に対処します。

[9] ウィットネスは、ブロックのトランザクションを実行するために必要なステートデータのコンパクトな集合である。

名前が示すようにヴァークルツリーは*ベクトルコミットメント*（具体的にはKZG多項式コミットメント）を使用しています。ベクトルコミットメントはデータセット全体を公開せずに、大きなデータセット内の特定の位置にあるデータ値を効率的に証明できる暗号学的コミットメントです。図16-6に示すようにマークルツリーで現在使用されているハッシュよりもはるかに優れたス

ケーラビリティと高速な計算を実現します。マークルツリーではマークルルート（ハッシュ）のみですが、ヴァークルツリーではベクトルコミットメントも持ちます。

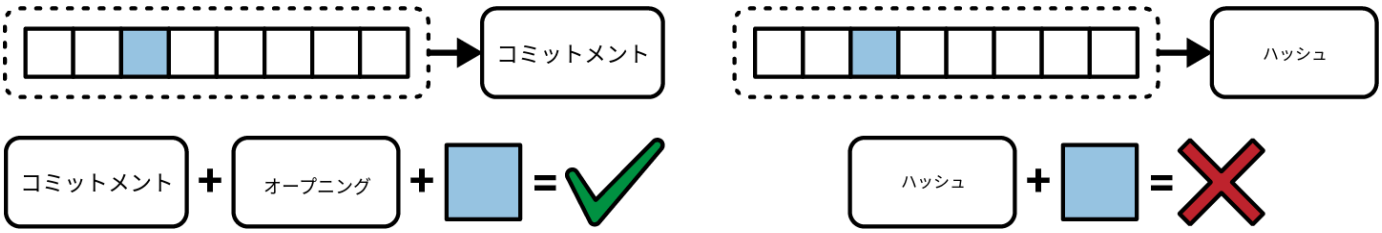


図16-6 マークルツリー対ヴァークルツリー

ハッシュのみでは、特定の値のベクトル内の特定の位置に特定の要素が存在することを証明できません。ベクトル全体を渡す必要があります。しかし、ベクトルコミットメントとオープニング（値のベクトル全体の小さな部分）があれば、特定の要素がその特定の位置に存在することを証明できます。

マークルツリーによりイーサリアムノードはブロックチェーン全体をダウンロードせずに小さなデータの一部を検証できます。しかしマークルツリーが非常に大きくなると、証明（データを検証するために必要な情報）のサイズも大幅に増加します。図16-7に示すようにこれらの大きな証明はネットワークを遅くし、イーサリアムがスケールし続ける中で効率の維持を困難にします。

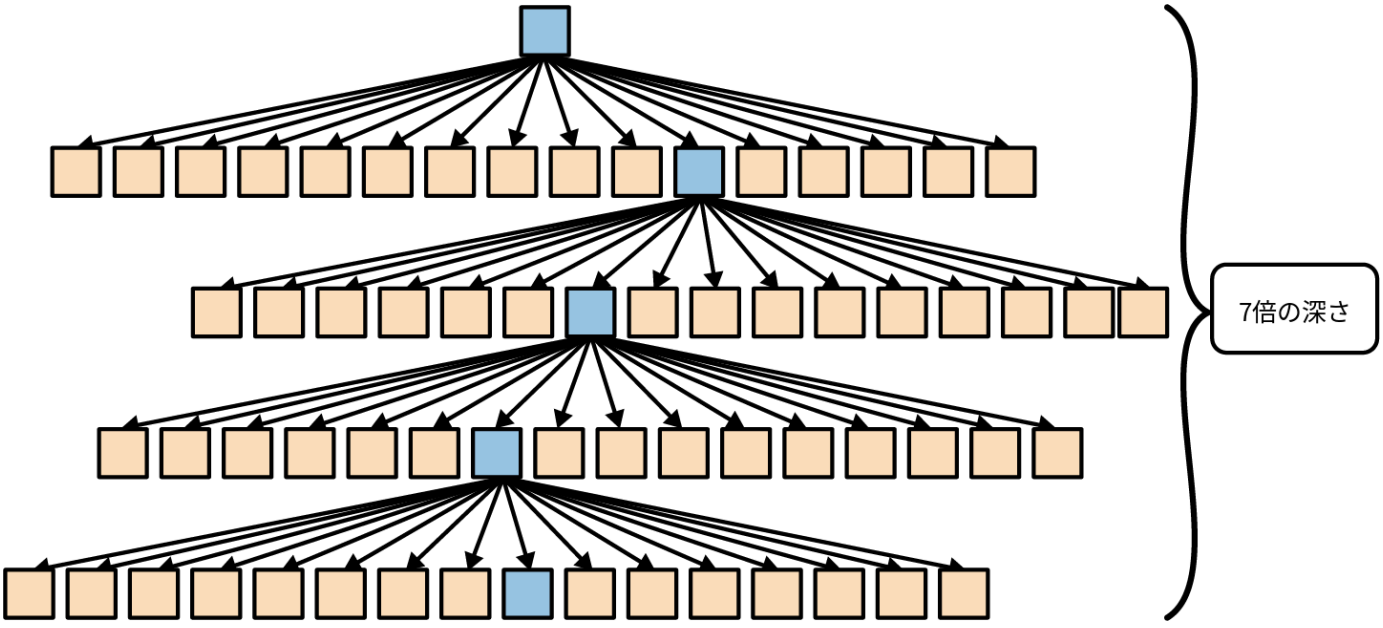


図16-7 マークルツリーの証明サイズ

特定のリーフXがこのマークルツリーに存在することを証明するには、パスに沿った各ノードのすべての兄弟ノードを渡すことが必須です。このデータがなければハッシュが意味をなさないためです。

ヴァークルツリーはこれらの証明のサイズを大幅に削減することでこの問題に対処しています。データの量が増えるにつれて証明が大きくなる代わりに、ヴァークルツリーはベクトルコミットメントと呼ばれる暗号学的手法を使用します。ベクトルコミットメントにより非常に短くコ

ンパクトな証明で大量のデータを証明できます。図16-8に示すようにイーサリアムのブロックチェーンが大きくなっても証明は小さく効率的なまま維持されます。

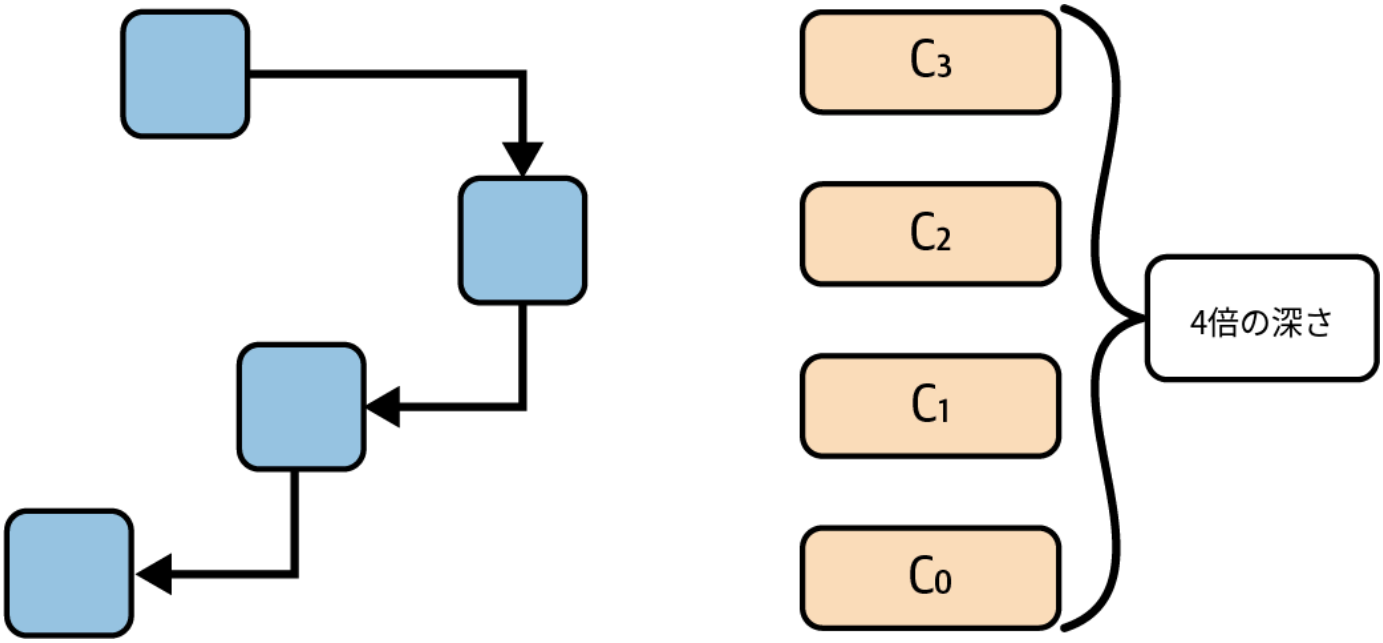


図16-8 ヴァークルツリーの証明サイズ

前述のベクトルコミットメントを活用することで、不要なデータを省略し、ヴァークルツリーの証明サイズを大幅に削減できます。

マークルツリーの証明サイズは以下の通りです（証明サイズは図16-5と図16-6に基づいて計算されています）。

- リーフデータに15の兄弟ノード（深さの各レベルで送信される不要なデータ、各兄弟ノード32バイト）を加え、7レベルの深さを掛ける = 1,000リーフで3.58MB

ヴァークルツリーの証明サイズはこれよりはるかに小さくなります。

- リーフデータにコミットメント（32バイト）と値（32バイト）とインデックス（1バイト）を加え、4レベルの深さを掛け、さらに小さな定数サイズのデータを加える = 1,000リーフで150KB

この説明の後に浮かぶかもしれない小さな疑問が1つあります。マークルツリーでは7レベルの深さを計算し、ヴァークルツリーでは4レベルしか計算しなかったのはなぜでしょうか。答えは非常にシンプルです。マークルツリーのノードは15の兄弟ノードしか持ちませんが、ヴァークルツリーのノードは255の兄弟ノードを持ちます。各レベルの幅がはるかに大きいため、同じ深さでもヴァークルツリーにはるかに多くのデータを格納できます。

ヴァークルツリーはデータを（キー、値）ペアに整理します。各キーは32バイトで、31バイトの「ステム」と1バイトの「サフィックス」で構成されます（図16-9参照）。このキースキームは近接するストレージ位置が同じステムと異なるサフィックスを持つよう設計されており、「隣接する」ストレージ位置へのアクセスをより安価にしています。

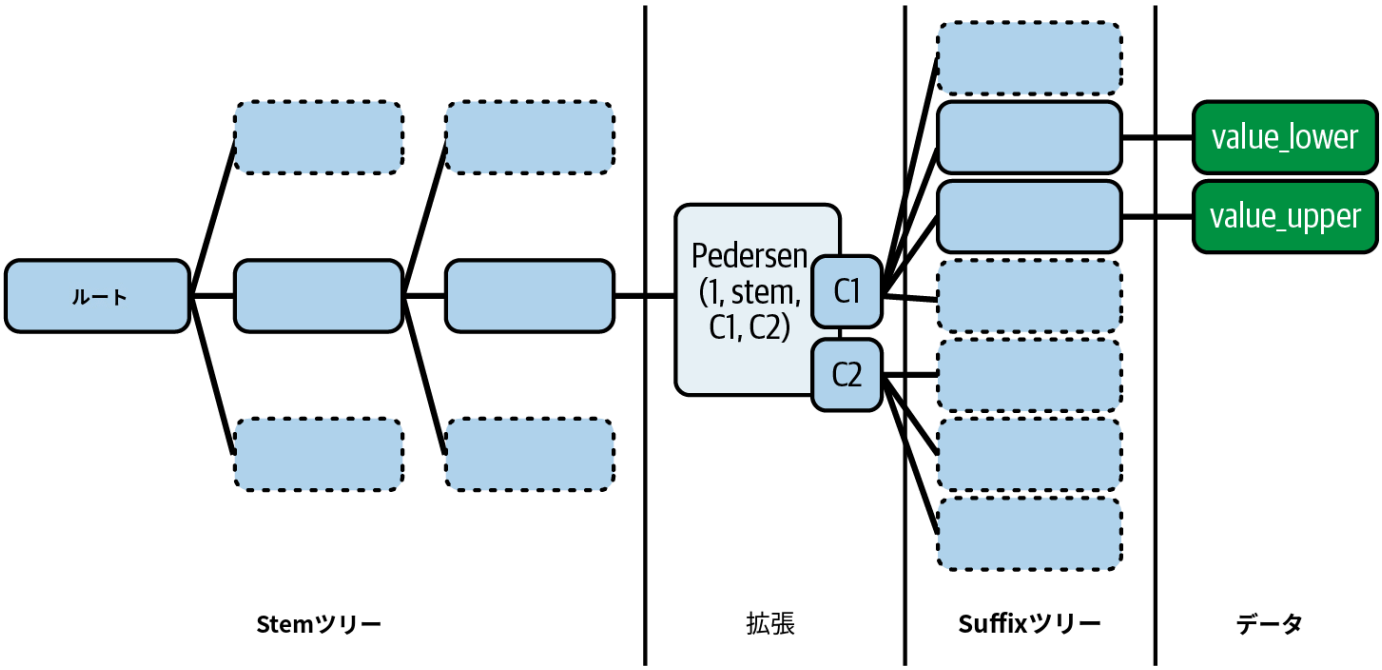


図16-9 ヴァークルツリーのキー構造

これらのキーは、3種類のノードに構造化されています。

- 拡張（またはリーフ）ノード：1つのステムに最大256の異なるサフィックスを表す
- 内部ノード：他の拡張ノードを含む可能性のある最大256の子ノードを含む
- 空ノード

完全なヴァークルツリーを構築するにはリーフノードから始めて多項式コミットメントを段階的に上方向に（ボトムアップで）計算し、トップレベルまたはルートコミットメントに到達します。このルートコミットメントはツリー全体のデータを簡潔に表現し、ノードがこの単一のルートコミットメントを参照するだけでブロックチェーンデータを迅速に検証できるようにします。ノードがユーザーのアカウント残高やトランザクションの有効性などの特定のブロックチェーンデータを検証する必要がある場合、この既知のルートコミットメントとともに通常わずか数百バイトの非常に小さな暗号学的証明を取得します。

強いステートレス性

*強いステートレス性*はノードがステートデータを一切保存する必要性を排除します。このモデルではトランザクションにブロックプロデューサーが集約できる小さなウィットネスが含まれます。ブロックプロデューサーは頻繁にアクセスされるアカウントのウィットネスを生成するために必要な最小限のステートのみを保存します。ステート管理の責任の大部分はユーザー自身に移されます。ユーザーはこれらのウィットネスを提供し、アクセスリストを通じてトランザクションがどのアカウントやストレージキーとやり取りするかを正確に指定しなければなりません。このアプローチによりノードは極めて軽量に動作できますが、スマートコントラクトとのやり取りの複雑さが増すというトレードオフが生じます。

強いステートレス性は研究で検討されてきましたが、現在イーサリアムの直近のロードマップには含まれていません。弱いステートレス性が当面のネットワークのスケーリング目標を満たす

のに十分と思われるため、イーサリアムは弱いステートレス性を追求する可能性が高いです。

まとめ

イーサリアムのスケーリングへのアプローチは単に処理を高速化することだけではありません。信頼性を支える分散化とセキュリティを犠牲にすることなくシステム全体をアップグレードすることです。場当たり的な修正ではなく、イーサリアムはロールアップとダンクシャーディングを活用しステートレス性を目指しながらLayer 1の改善を継続することで、レイヤードシステムを構築しています。このシステムではL2ネットワークが実行作業の大部分を処理しますが、セキュリティと最終的な決済についてはイーサリアムのベースレイヤーに依存し続けます。コアとなる価値観を維持しながら長期的な成長とイノベーションの基盤を築く、思慮深いモジュラーな道筋です。

さらに詳しくは、以下の参考文献を参照してください。

- ダンクシャーディングの提案
- プロトダンクシャーディングの提案
- EIP-4844
- ダンクシャーディング（誰にでもわかりやすい解説）
- EIP-4488
- ステートレス性のためのヴァークルツリー
- ヴァークル証明
- ヴァークルツリーのEIP
- ヴァークルツリーの構造
- BLS12-318曲線をよりよく理解するためのペアリングベース暗号のレポート
- BLS12-318の「簡単な」解説

第17章 ゼロ知識証明

この章では、*ゼロ知識暗号*の魅力的な世界を探求し、イーサリアムのロードマップにどのように適用されるかを見ていきます。ゼロ知識技術を活用すれば、ブロックスペースに対する大量導入の需要に対応した真のスケーリングが可能になります。ゼロ知識技術は非常に複雑なテーマであり、詳細に説明しきれない多くの数学的規則に依存しています。この章の目的は、ゼロ知識暗号がイーサリアムにとってなぜ独自の機会を提供し、スケーリングロードマップにうまく適合するのかを理解することです。この章を読み終える頃には、ゼロ知識暗号の概要、有用な特性、イーサリアムがプロトコルの改善にどのように活用しようとしているかを理解しているでしょう。

歴史

*ゼロ知識証明*は、1985年にシャフィ・ゴールドワッサー、シルビオ・ミカリ、チャールズ・ラコフの3人が発表した論文"The Knowledge Complexity of Interactive Proof-Systems"で初めて提唱されました。この論文では、あるステートメントが真であるという事実以外の情報を一切明かさずに、そのステートメントの真実性を証明する方法としてゼロ知識証明を記述しています。

ゼロ知識証明は1980年代に発見されましたが、実用的なユースケースは非常に限られていました。状況が一変したのは2011年のことです。BIT+11論文の登場により、任意の計算に対するゼロ知識証明を作成するための理論的フレームワークとして*SNARK*（*succinct noninteractive arguments of knowledge*）が導入されました。2年後の2013年には、Pinocchio PGHR13論文が汎用SNARKの最初の実用的な実装を提供し、現実世界のアプリケーションで使用可能になりました。汎用プログラムが正しく実行されたことを、再実行せずに、実際の計算の詳細を明かさずに証明できるようになったのは、このときが初めてです。

革命が始まりました。その瞬間から、ゼロ知識証明の分野は驚異的なスピードで進化しました。2016年には、Groth16アルゴリズムが証明サイズと検証時間を削減し、zk-SNARKの効率を大幅に改善しました。その卓越した簡潔性から、新しいシステムが利用可能であるにもかかわらず、Groth16は現在でも広く使われています。例えば、分散型ミキサーアプリケーションであるTornado Cashは、オンチェーンでのゼロ知識証明の実装にGroth16を使用しています。

2017年には、Bulletproofsが*トラステッドセットアップ*の必要性を排除するという画期的な進歩をもたらしました（トラステッドセットアップの実際の意味については、後のセクションで詳しく解説します）。ただし、証明サイズが大きくなるというトレードオフがありました。2018年には*zk-STARK*が登場し、トラステッドセットアップの要件を排除しただけでなく、ポスト量子セキュリティも提供しました。ポスト量子セキュリティとは、暗号基盤が量子コンピュータからの攻撃に対して耐性を持つことを意味します。Bulletproofsは現在、暗号通貨プロジェクトMoneroでトランザクション金額の秘匿化に使われており、zk-STARKはイーサリアム L2であるStarknetの暗号基盤となっています。

2019年には、PLONKとSonicがユニバーサルかつ更新可能なトラステッドセットアップを導入し、大きな貢献を果たしました。SNARKをより幅広いアプリケーションで実用化しました。これらのイノベーションは、現在のゼロ知識システムにも引き続き影響を与えています。

ゼロ知識証明は現在も活発に開発が続いています。最近の進歩では、証明時間や再帰効率、zk-EVMやモダンなzk-VMといった実用的なアプリケーションが改善されています。新しい構築法や最適化が定期的に登場し続けており、ゼロ知識技術の可能性の限界を押し広げています。

こうした進歩は主に、暗号通貨分野がもたらした資金のおかげで実現しました。イーサリアムファンデーションも、複数の助成金を提供し、このテーマを研究する専門チームを擁することで貢献してきました（そして現在も貢献し続けています）。

定義と性質

ここからは、ゼロ知識証明とは何か、どのような性質を満たす必要があるかを、より具体的に定義しましょう。すでに述べたように、ゼロ知識証明とは、一方の当事者（通常*証明者*、Pと呼ばれる）が、もう一方の当事者（通常*検証者*、Vと呼ばれる）に対して、あるステートメントが真であるという事実以外の情報を一切明かさずに、そのステートメントが真であることを証明するプロトコルです。

いくつかの重要な定義を形式化しましょう。

ステートメント

証明される主張である。ステートメントは公開されており、誰でも検証可能で、秘密情報を含まない。

ウィットネス

ステートメントが真であることを証明する秘密情報である。ウィットネスは証明者のみが知っている。

証明

ウィットネスを明かすことなく、ステートメントが真であることを検証者に納得させる暗号オブジェクトである。

すべてのゼロ知識証明システムは、以下の3つの性質に従わなければなりません。

完全性

証明者Pが真のステートメントを保持している場合、プロトコルの規則に従って有効なゼロ知識証明を計算できなければならない。すべての規則に正しく従っているにもかかわらず有効な証明を生成できないケースがあってはならない。

健全性

悪意のある証明者が偽のステートメントに対して有効な証明を偽造することは実行不可能でなければならない。ゼロ知識証明が検証者に受理された場合、証明者がその証明を作成できた唯一の方法は、真のステートメントから始めてすべてのプロトコル規則に適切に従ったことである。

ゼロ知識性

名前が示す通り、検証者はプロトコルに従う過程で、ステートメントの有効性以外の情報を知ることができてはならない。

イーサリアムにおけるゼロ知識証明の活用

この非常に新しい暗号技術がイーサリアムの将来の開発とロードマップにとってなぜ重要なのか、疑問に思うかもしれません。答えはシンプルです。

イーサリアムにおけるゼロ知識証明システムの真の力は、最終的なステートメントに到達するために必要なすべての計算を実行しなくても、そのステートメントの妥当性を検証できる点にあります。まず証明者がある計算を実行してステートメントを算出し、ゼロ知識証明を添付します。次に検証者はゼロ知識プロトコルを使用するだけで、証明者と同じ計算を行わずに、そのステートメントをトラストレスに真として受け入れることができます。

注意深い読者は、これがイーサリアムのどこに適用されるか、すでに気づいているかもしれません。ブロックの実行とステートの更新、つまり*EVMステート遷移関数*です。新しいブロックごとに、そのブロックに含まれるすべてのトランザクションを処理して現在のステートを更新します。図17-1はこの様子をよく表しています。

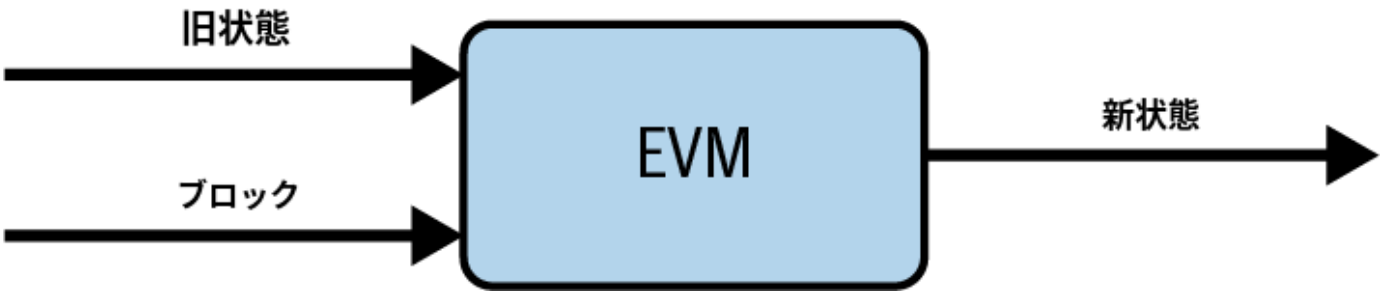


図17-1 EVMステート遷移関数

EVMは、ブロックチェーンの現在のステートとトランザクションが詰まった新しいブロックを入力として受け取り、チェーンの新しいステートを返すブラックボックスと考えることができます。現在、イーサリアムのフルノードは新しいブロックを受信すると、トラストレスにステートを更新するために、そのブロックに含まれるすべてのトランザクションを再実行しています。

このアプローチの主な問題は、ブロックの実行が非常に負荷の高い処理であり、ボトルネックになりうることです。フルノードが正常に動作しチェーンの先端に追従するために必要なハード

ウェア要件と、ネットワークで達成したい分散化のレベルとのバランスを取る必要があります。フルノードの要件を引き上げてスケーリングしようとするほど、資金的に余裕のないアクターがフルノードを維持できなくなり、分散化が損なわれます。逆に、誰でもノードを運用できるようにハードウェア要件を低く抑えると、ブロックチェーンが処理できる最大スループットが制限されます。イーサリアムは常にこのトレードオフの分散化側を選択し、ソロステーカーやフルノード運用者が参加できるようにハードウェア要件を十分に低く保ってきました。

ここでゼロ知識証明システムが活躍します。フルノードがトランザクションを一切実行せずに、つまりEVMステート遷移関数の重い計算を行わずに、トラストレスにステートを更新できるとしたらどうでしょうか。このアイデアは以下のように要約できます。

1. 一部のアクターが、新しいブロックに含まれるすべてのトランザクションを実行し、チェーンの更新されたステートを生成するEVM計算を行う。
2. これらのアクターがステートの妥当性を証明するゼロ知識証明を作成し、新しいステートとともにすべてのフルノードに共有する。
3. 他のすべてのフルノードは、チェーンの新しいステートとゼロ知識証明を受信した際、証明の妥当性を検証するだけでよい。証明が有効であれば、受け取った新しいステートで自身のチェーンのステートをトラストレスに更新できる。

この方法では、すべてのトランザクションを実行するノード（おそらく潤沢な資金を持つ企業が維持する）は依然として必要ですが、それ以外のノードは証明を検証するだけでよいため、運用コストを抑えられます。トランザクションを実行するノードのハードウェア要件を非常に高く引き上げて、はるかに大きなスループットを処理しつつも、ゼロ知識証明の「魔法」のおかげで高い分散化レベルを維持できます。

これまでの説明では、ゼロ知識証明の生成と検証は高速で簡単であると仮定してきました。しかし実際には、ゼロ知識証明の検証が平均的なイーサリアムブロックに含まれるすべてのトランザクションの再実行よりも高速である場合にのみ、前述のステップは意味をなします。現時点ではこの条件を達成するのは非常に困難です。しかしこれは最適化の問題であり、「できるかどうか」ではなく「いつできるか」の問題にすぎません。イーサリアムブロックのゼロ知識証明をリアルタイムで生成・検証できる時点に間もなく到達するでしょう。この分野の動向に興味がある方は、Ethproofsというウェブサイトをフォローすることをお勧めします。

L2もZKの恩恵を受ける

ゼロ知識証明技術の恩恵を受けるのはイーサリアムメインネットだけではありません。第16章ですでに読んだかもしれませんが、*ZKロールアップ*と呼ばれるロールアップのカテゴリが存在します。名前が示す通り、ZKロールアップはEVM実行を証明するためにゼロ知識証明システムを使用しています。チェーンのステート更新を、新しいステートが正しく計算されたことを保証するゼロ知識証明とともにポストできるようにしています。

前述のように、現在の平均的なイーサリアムブロックのリアルタイム証明が達成できないのに、ZKロールアップはどのようにゼロ知識技術を使用できるのでしょうか。答えは、リアルタ

イム証明は行っていないということです。L2の各ブロックに対してリアルタイムに証明する必要はありません。通常は1時間ほどの間隔で集約されたゼロ知識証明をポストし、前回の証明から最新のものまでの間に行われたすべてのステート更新が正しく実行されたことを証明します。

ゼロ知識証明の間隔はロールアップ自体のファイナリティまでの時間にのみ影響します。証明が1時間ごとにポストされる場合、トランザクションが（L1上で）本当にファイナライズされたと見なせるまで平均して30分待つ必要があります。

小さな例

ゼロ知識証明システムの動作をより身近に理解するために、シンプルで小さな例から始めましょう。*分割問題*です。これは「与えられた正の整数の多重集合Sを、S1とS2の2つの部分集合に分割して、両者の合計が等しくなるようにできるか」を判定するよく知られた問題です。

> **注記**
>
> 分割問題は決定問題であるのに対し、イーサリアムは計算トレースの検証にゼロ知識証明を使用する。イーサリアムにおけるゼロ知識証明の主要なユースケースは実行トレースの検証であるが、分割問題を通じて学ぶことはゼロ知識証明システムの仕組みに対する直感を養う優れた方法である。

S = [1, 1, 5, 7]の場合、S1 = [1, 1, 5]とS2 = [7]に分割すれば問題を満たせます。しかし常に正しい解が見つかるとは限りません。例えばS = [1, 1, 5, 8]の場合、同じ合計になる2つの部分集合に分割することは不可能です。

分割問題はNP完全です。つまり、多項式時間で解くアルゴリズム（解が存在する場合にそれを提示するか、解が存在しないことを短時間で証明するアルゴリズム）は知られていません。

証明してみよう

分割問題の構造は、これまで見てきたゼロ知識証明システムの性質にぴったり当てはまります。解を見つけるのが困難なため、解を探す計算能力をスーパーコンピュータを備えた証明者に委任し、ゼロ知識技術を使って有効な解を見つけたことを証明するのは興味深いアプローチです。他の全員が同じ重い計算を行う必要がなく、実際の解を明かすこともありません。

分割問題をゼロ知識証明システムに適した形にするため、少し調整を加えます。正の整数のリスト`s`があるとします。別のリスト`a`が*充足割り当て*であるとは、以下の条件を満たす場合を指します。

1. `len(s) == len(a)`

2. `a`のすべての要素が1または-1である
3. `s`と`a`の内積が0である

この構成は分割問題と完全に同等であることに注意してください。前の例の初期リストS = [1, 1, 5, 7]を取ると、充足割り当て`a`は次のようになります。

```
a = [1, 1, 1, -1]
```

3つの条件がすべて真であることを手動で確認できます。

1. `len(s) = 4 == len(a) = 4`
2. `a`のすべての要素が1または-1である
3. 内積: 1 × 1 + 1 × 1 + 1 × 5 + (–1) × 7 = 1 + 1 + 5 – 7 = 0

*内積*は、等しい長さの2つのリストについて、最初のリストの各要素と2番目のリストの対応する要素を掛け合わせ、すべての結果を合計して計算します。より技術的に言えば、等しい長さ`n`のリスト`s = [s0, s1, s2, sn]`と`a = [a0, a1, a2, an]`がある場合、内積は次のように計算します。

$$s \cdot a = \sum_{i=0}^{n} s_i \times a_i = a_0 s_0 + a_1 s_1 + a_2 s_2 + \ldots + a_n s_n$$

もちろん`a`を検証者にそのまま提供して有効な解であることを確認させることもできますが、それでは解を明かさないというゼロ知識の前提に違反してしまいます。

*部分和リスト*を作成して、別のリスト`w`を生成しましょう。`w[i]`は`s`と`a`の`i`までの部分内積に等しくなります。ゼロ知識証明システムの文脈では、`w`は*ウィットネス*とも呼ばれます。先の例に従って、`w`を計算できます。

```
w = [1, 2, 7, 0]
```

`a`が有効な充足割り当てであれば、`w`は常に0で終わることに注意してください。

証明システムをうまく機能させるために、これまで構築してきた証明`p`に小さな変更を加えましょう。具体的には、リスト`w`の最後の要素を最初の位置に移動します。先ほどの`w`は次のようになります。

```
w = [0, 1, 2, 7]
```

このリスト`w`には非常に便利な性質があります。

```
|s[i]| = |w[i + 1] - w[i]|
```

このステートメントの妥当性を自分で確認できます。

```
|s[0]| = |w[0 + 1] - w[0]| = |w[1] - w[0]| = |1 - 0| = |1| = 1
|s[1]| = |w[1 + 1] - w[1]| = |w[2] - w[1]| = |2 - 1| = |1| = 1
|s[2]| = |w[2 + 1] - w[2]| = |w[3] - w[2]| = |7 - 2| = |5| = 5
|s[3]| = |w[3 + 1] - w[3]| = |w[4] - w[3]| = |w[0] - w[3]| = |0 - 7| = |-7| = 7
```

`s[3]`を計算するには`w[4]`にアクセスする必要がありますが、`w`は4つの要素しかないため（インデックスはゼロから始まる）、`w[4]`は存在しません。しかし解決策は簡単です。`w`が巡回的であるかのように最初の要素に戻ればよいのです。

検証者は`w`の連続する2つの要素を要求し、前述の関係式が成り立つかを確認できます。検証者は`s`にアクセスできることを覚えておいてください。`s`は公開されており、この問題の入力を表しています。しかし検証者は解を表す充足割り当て`a`にはアクセスできません。等式が成り立てば、証明者がこの問題の充足割り当て`a`を知っていることを意味します。

問題点

ここまで構築してきたのは、非常に素朴な証明システムです。実際には、3つの大きな問題があります。

1. 前述のように、検証者が`w`の連続する2つの要素を要求し等式が成り立てば、証明者がこの問題の解を知っていることが検証されたと述べた。しかしこれは正確ではない。1回のクエリだけでは完全に確信を持てない。十分に高い確率で確信を得るまで、`w`の連続する要素に対して複数回のクエリを行う必要がある。
2. 検証者が証明者の解の保有を確認できることは確かだが、この証明システムは証明者の誠実性に依存している。証明者が悪意を持ち嘘をつくとどうなるか。検証者が`w`の連続する2つの要素を要求した際、証明者は等式を満たすランダムな2つの数を提供する可能性がある。
3. さらに、この証明システムはゼロ知識ではない。`w`の連続する要素を要求することで、検証者は充足割り当て`a`について多くの情報を得てしまう。検証者は`w`の構築方法を知っているため、`w`について知れば知るほど解`a`についても知ることになる。

これらの問題に対処し、より良い証明システムの構築を試みましょう。

ゼロ知識性の実現

システムにゼロ知識性を追加しましょう。これまで構築してきたプロトコルの主な問題は、`w`の実際の値を検証者に送信しているため、検証者が`a`について多くの情報を理解できてしまうことです。実際に確認するため、証明者と検証者の間で2つの簡単なインタラクションステップを行いましょう。

検証者は`w[1]`と`w[2]`を要求し、`|w[2] - w[1]| = |s[1]|`を確認します。証明者は`w[1] = 1`と`w[2] = 2`を提供し、検証者は等式が成り立つことを確認できます。

$s[1] = w[2] - w[1] = 2 - 1 = 1 = 1$

次に、検証者はもう1つのクエリを要求します。`w[2]`と`w[3]`です。証明者は`w[2] = 2`と`w[3] = 7`を提供し、検証者は等式を確認します。

$s[2] = w[3] - w[2] = 7 - 2 = 5 = 5$

この2回のインタラクションで、検証者はwの3つの要素を知ることになります。

$w[1] = 1$
$w[2] = 2$
$w[3] = 7$

検証者はすでに`s = [1, 1, 5, 7]`を知っており、`w`がどのように計算されたかも知っているため、解`a`の最初の部分が`[1, 1, 1]`であることを導出できます。`w[3] = 7`、`w[2] = 2`、`w[1] = 1`を得る唯一の方法がこれだからです。

`w`の実際の値をマスクしつつ、新しい`w`と入力リスト`s`の間の関係式を満たし続ける方法を見つける必要があります。これを実現するには、特定の方法でランダム性を追加する必要があります。

まずコインを投げます。表が出れば`a`をそのままにし、裏が出ればすべての要素の符号を変更して、すべての1を-1に、すべての-1を1にします。この変更は充足割り当て`a`の3つの主要な性質を壊さないことに注意してください。

次に、新しいランダムな整数値`r`を取得します。`w`を以前と同じ方法で計算しますが、すべての要素に`r`を加えます。この変更も、`s`と`w`の間の重要な関係式を壊しません。

検証者が新しいクエリを要求するたびに、コインを投げて異なるランダム値`r`を計算する必要があります。こうすることで検証者は等式の妥当性を検証できますが、すべての`w`の値がランダムに見えるため、`a`について何も理解できません。

小さなデモンストレーションを行いましょう。証明者（この例では私たち）は`s`と`a`の両方を持っていることを思い出してください。

$s = [1, 1, 5, 7]$
$a = [1, 1, 1, -1]$

ここで検証者が`w[1]`と`w[2]`を要求し、`|w[2] - w[1]| = |s[1]|`を確認しようとします。

まず、`a`のすべての値を変更するかどうかを決めるためにコインを投げる必要があります。裏が出たので、すべての符号を反転させ、`a`は次のようになります。

$a' = [-1, -1, -1, 1]$

次に、ランダム値`r = 4`を計算します。`w`を計算し、各要素に`r`を加えます。

$w = [0, -1, -2, -7]$
$w' = [4, 3, 2, -3]$

そして、$w'[1] = 3$と$w'[2] = 2$を提供します。

検証者は依然として等式の妥当性を確認できます。

$s[1] = w'[2] - w'[1] = 2 - 3 = -1 = 1$

ここで検証者はもう1つのクエリを実行し、`w[2]`と`w[3]`を要求します。再び`a`のすべての値を変更するかどうかを決めるためにコインを投げます。今回は表なので符号は変更しません。

次に、ランダム値`r = 1`を計算します。`w`を計算し、各要素に`r`を加えます。

$a = [1, 1, 1, -1]$

そして、$w''[2] = 3$と$w''[3] = 8$を提供します。

検証者は依然として等式の妥当性を確認できます。

$s[2] = w''[3] - w''[2] = 8 - 3 = 5 = 5$

しかし今回、検証者は充足割り当て`a`についてあまり理解できません。検証者が持っている情報は以下の通りです。

$w'[1] = 3$
$w'[2] = 2$
$w''[2] = 3$
$w''[3] = 8$

異なるクエリの`w`値は検証者にとってランダムに見え、割り当て`a`を復元することはできません。

証明システムにゼロ知識性を追加することができました。次は、悪意のある証明者の問題に対処しましょう。

証明者のコミットメント

対処すべき問題は、証明者が計算されたウィットネス`w`の実際の値ではなく、でっち上げた数値を検証者に提供して嘘をつける点です。これは非常に深刻な問題です。偽のステートメントに対して有効な証明を提供できてしまうため、証明者が実際には充足割り当て`a`を持っていなくても、検証者を騙して持っていると信じ込ませる可能性があります。

証明者が発覚せずに嘘をつけないようにする方法を見つける必要があります。本質的に、証明者は`w`のすべての値を検証者に提供する前に*コミットメント*を行う必要があります。

ここでマークルツリーが再び役立ちます。マークルツリーは第14章で紹介したので、仕組みの詳細にはここでは立ち入りません。

新しい例で、新しい構成を実際に見てみましょう。証明者（私たち）は以下を持っています。

$$s = [1, 1, 5, 7]$$
$$a = [1, 1, 1, -1]$$

コインを投げると表なので、割り当て`a`のすべての符号を変更する必要はありません。次に、`a`と`s`の内積として`w`を計算します（最初と最後の要素を入れ替えます）。

$$w = [0, 1, 2, 7]$$

ランダム値`r = 8`を計算し、`w`の各要素に加えます。

$$w' = [8, 9, 10, 15]$$

ここで`w'`のすべての値にコミットし、そのコミットメントを検証者に送信します。これにより`w'`の偽の値を提供して不正を行った場合、簡単に発見できます。図17-2は`w'`のすべての値をリーフとして使用して構築されたマークルツリーを示しています。

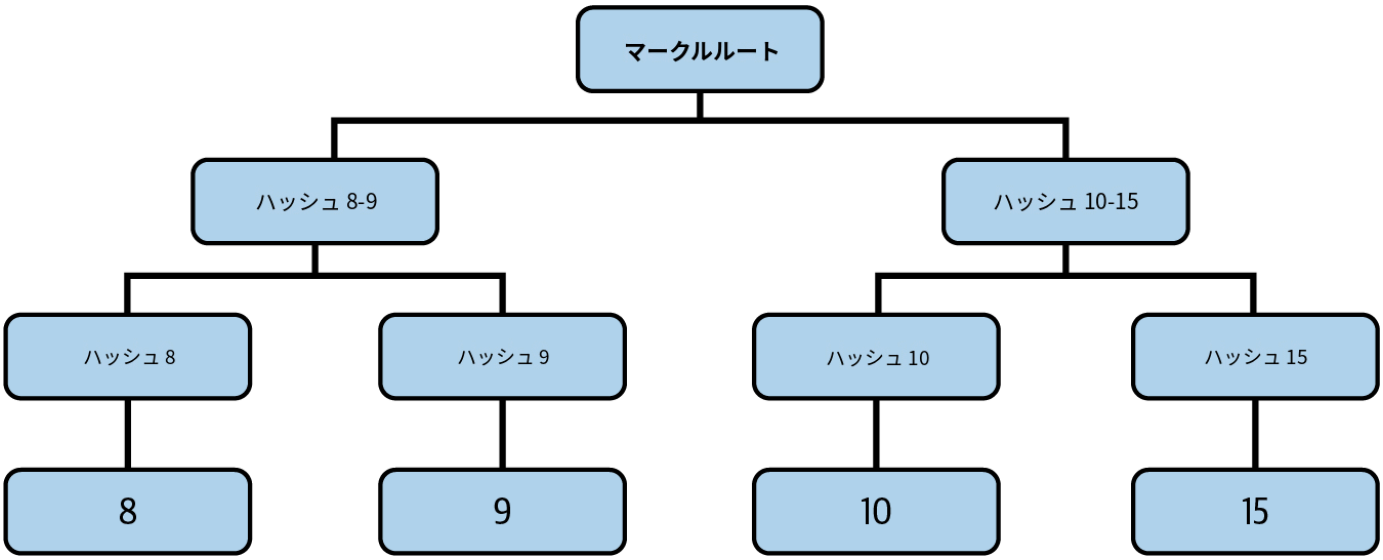


図17-2 `w'`の値を使用して構築されたマークルツリー

マークルルートは、検証者に送信される最終的なコミットメントです。

この時点で、検証者は最初のクエリを要求し始めます。$w'[1]$と$w'[2]$です。$w'[1] = 9$と$w'[2] = 10$を検証者に提供し、検証者は以下を確認できます。

$$s[1] = w'[2] - w'[1] = 10 - 9 = 1 = 1$$

ここで検証者は証明者が不正をしていないことを確認する必要があります。そのため検証者は証明者に*検証パス*も要求します。検証パスには、マークルルートが送信されたコミットメントと一致するかを確認するために、マークルツリーを再構築するのに必要なすべてのデータが含まれ

ています。一致すれば不正がなかったことが確実になり、一致しなければ不正があったことが即座にわかります。

具体的に、このクエリの検証パスには以下が含まれます。

- Hash 8
- Hash 15

こうすることで検証者はマークルルートまでマークルツリーを再構築し、コミットメントを検証できます。Hash 8とHash 15を検証者に送信し、検証者がコミットメントの妥当性を確認すれば、このクエリは終了です。

しかし新たな問題に気づいたかもしれません。検証パスの要件により、`w'`に関する追加データ（Hash 8とHash 15）を送信することになりました。ハッシュ関数を逆算すること（つまりハッシュから実際の値8と15を取得すること）は計算上実行不可能であるはずですが、悪意のある検証者がブルートフォース攻撃を試み、意図していなかったマークルツリーの一部を明らかにする可能性があります。幸い、この問題を解決するシンプルで優れた調整があります。

コミットメントへのランダム性の追加

このアイデアは、証明システムにゼロ知識性を追加するために以前行ったことと似ています。今回は検証パスで`w`（または`w'`）に関する情報を明かさないよう、コミットメントにランダム性を追加する必要があります。

マークルツリーを作成する際、ツリーのリーフに`w'`の各要素の正確な値のハッシュだけを使用するのではなく、検証者に提供しないランダムな文字列を追加します。図17-3はこの新しい方法で構築されたマークルツリーを示しています。

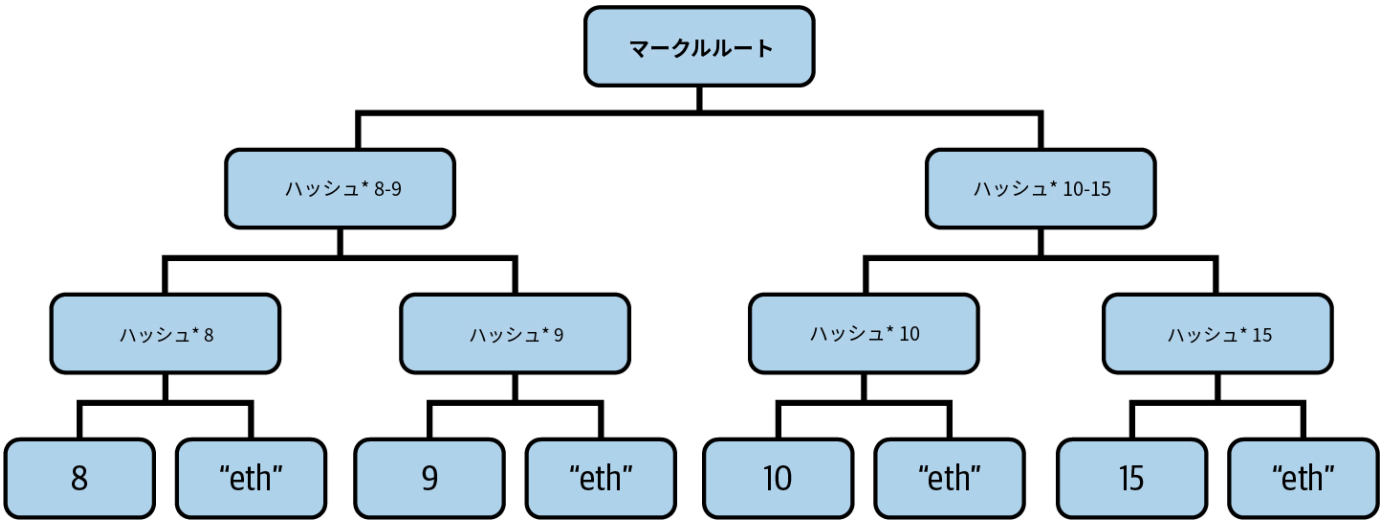


図17-3 ゼロ知識性のためにランダム性を追加したマークルツリー

この例ではウィットネス`w'`の各項目をマスクするために文字列「eth」を使用しました。この秘密の文字列がなければ実際の値をデコードすることは不可能です。検証パスでHash* 8と

Hash* 15を検証者に送信しても、検証者はハッシュの背後にある具体的な`w'`の値を知ることができません。ブルートフォース攻撃を試みても、`w'`の項目を隠すために使用した秘密の文字列を知らないためです。

結論？ それとも…

ゼロ知識証明システムの構築に成功しました。分割問題に対する（非常に素朴で基本的な）ゼロ知識証明システムを作成できたのです。

「素晴らしい証明システムだが、証明者と検証者の間にまだ多くのインタラクションが必要だ。プロトコルを成功させるには双方が同時にオンラインでなければならない。この対話型ゼロ知識証明システムを非対話型に変換する方法はないだろうか」と思われるかもしれません。

まさにその通りです。この証明システムが正しく機能するには証明者と検証者のインタラクションが必要であり、ほとんどのユースケースにとって不便です。幸い（本当に幸運が多いですね）、助けとなる科学論文があります。

Fiat-Shamirヒューリスティック

1986年、著名な暗号学者であるアモス・フィアットとアディ・シャミアが論文"How to Prove Yourself: Practical Solutions to Identification and Signature Problems"を発表しました。この論文で、現在も広く使われている変換プロトコルを発明しました。それが*Fiat-Shamirヒューリスティック*（または変換）と呼ばれるものです。

> **注記**
>
> アディ・シャミアは暗号学の真のレジェンドである。RSAアルゴリズムのSであり、最初かつ最も広く使用されている公開鍵暗号方式の1つである。アモス・フィアットはイスラエルのワイツマン科学研究所におけるシャミアの同僚であった。

Fiat-Shamirヒューリスティックは、検証者のランダムなチャレンジを暗号学的ハッシュ関数に置き換えることで、対話型ゼロ知識証明システムを非対話型に変換できるプロトコルです。

これまで構築してきたシステムで検証者が何をしているかを考えると、本質的には証明者にいくつかのランダムな数値（クエリ）を与え、証明者がそれを使って対応する証明を生成しているだけであることに気づくでしょう。

以前、「検証者が`w[1]`と`w[2]`を要求する」と述べたことを思い出してください。これは、検証者が証明者に数値1と2を与えて、`w[1]`と`w[2]`の証明を作成させることと言い換えられます。

証明者がウィットネス`w`を作成して`w'`に変換する部分を省くと、元の対話型プロトコルは以下のように要約できます。

1. 証明者が`w'`へのコミットメントを生成し、検証者に送信する。
2. 検証者がランダムなチャレンジ（クエリ）を送信する。
3. 証明者がチャレンジに対するレスポンス（証明）を送信する。
4. 検証者が証明の妥当性を検証する。　このプロセスは検証者が行う異なるクエリごとに繰り返されます。これがうまく機能するのは、証明者が検証者のクエリを事前に知らないからです。クエリは証明者にとってランダムです。したがって証明者自身がランダムだが予測可能な方法ですべてのクエリを生成できれば、プロセス全体を非対話型プロトコルに変換できます。

1986年の論文でフィアットとシャミアは、検証者のランダムなチャレンジをシミュレートする「ランダムオラクル」としてハッシュ関数を使用することを提案しました。プロトコルの2ラウンドのみの簡単な例を見てみましょう。

第1ラウンドでは以下のようになります。

1. 証明者がコミットメントを生成する。
2. 証明者がコミットメントと問題の公開入力を連結してハッシュする。結果が最初のチャレンジとなる。

```
challenge = hash(commitment || public inputs)
```

3. 証明者がチャレンジに対するレスポンス（証明）を計算する。

第2ラウンドでは以下のようになります。

1. 証明者が新しいコミットメントを生成する。
2. 証明者が新しいコミットメントと、プロトコルでこれまで計算されたすべて（古いコミットメント、証明、公開入力）を連結してハッシュする。結果が新しいチャレンジとなる。

```
new challenge = hash(commitment || proof || new_commitment || public inputs)
```

3. 証明者が新しいチャレンジに対するレスポンス（新しい証明）を計算する。

この時点で、証明者はプロトコルのトランスクリプト全体を検証者に送信できます。検証者は以下を行う必要があります。

1. すべてのチャレンジがハッシュ関数と有効な入力を使用して正しく計算されたことを確認する。これは証明者がでっち上げのチャレンジで不正を試みなかったことを保証するためである。
2. 対話型プロトコルと同じ方法で、すべての証明の妥当性を検証する。これは証明者が実際に元の問題の解を知っていることを保証するためである。

図17-4は、検証者がゼロ知識証明の妥当性を検証する前に、チャレンジが正しく計算されたことを確認するプロセスを示しています。

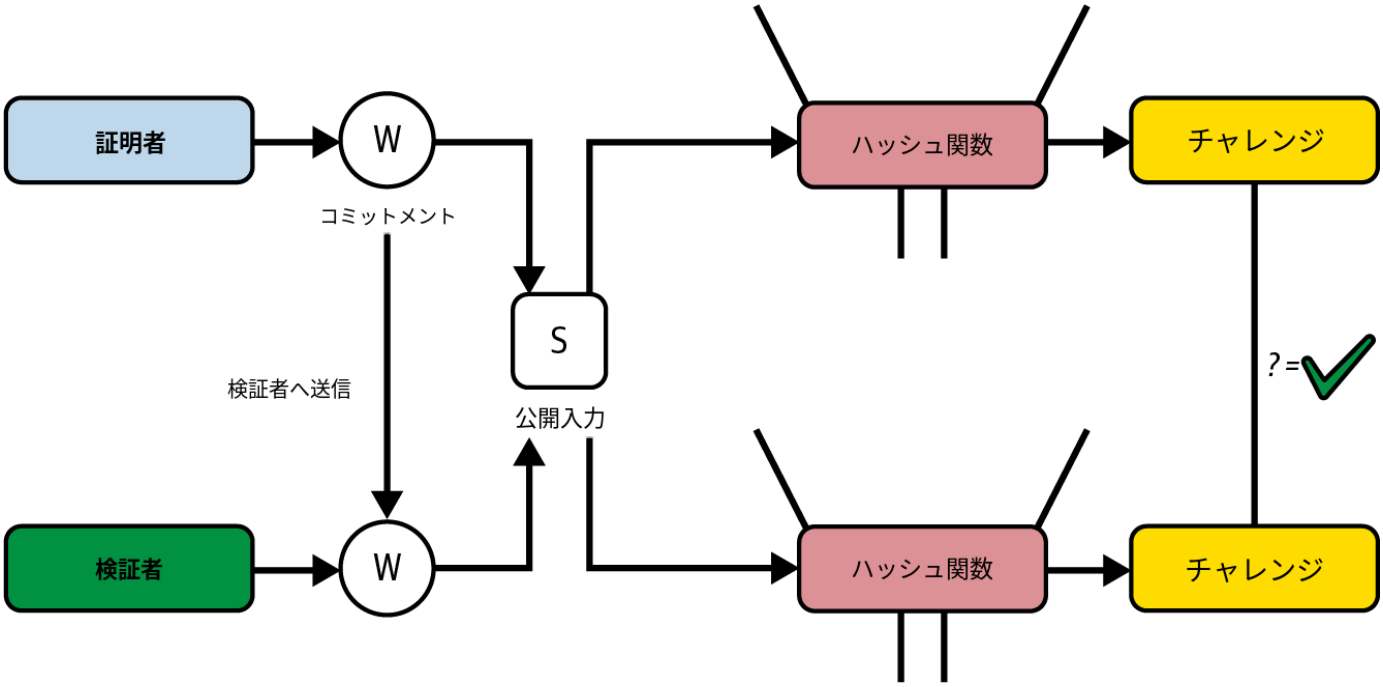


図17-4 Fiat-Shamirヒューリスティックによるプロトコルの非対話型化

以上です。Fiat-Shamirヒューリスティックのおかげで非対話型ゼロ知識証明システムが実現しました。おまけとして優れた特徴があります。証明者が誰でも検証可能な有効な証明を生成できるようになったことです。証明者が不正を試みているかを検出するには、証明を検証する正直な当事者が1人いるだけで十分です。

結論？

このセクションでは、分割問題に対する非対話型ゼロ知識証明システムを構築しました。最新のゼロ知識アーキテクチャははるかに複雑であり、本書ではそのすべてを説明できません（ただし章末にさらに読み進めるための優れた論文をいくつか紹介しています）。それでも、ここまで議論してきたことがすべての新しいゼロ知識技術の基盤となっています。これを理解していれば、ゼロ知識のラビットホールに飛び込んで背後にある細かい詳細を探求し始められるでしょう。

次のセクションでは、ゼロ知識証明システムを構築するための最も広く使用されている2つの実際のフレームワーク、SNARKとSTARKについて見ていきます。

SNARKとSTARKの比較

前のセクションでは分割問題に対するゼロ知識証明システムを作成できました。良いスタートです。では、まったく異なる計算に対してゼロ知識証明システムを作成したい場合、分割問題用

に構築したのと同じアーキテクチャを再利用できるでしょうか。

答えは「できない」です。コミットメント`w`とその性質（例えば`|s[i]| = |w[i+1] - w[i]|`）は分割問題に特化しすぎており、他の問題には適用できません。しかしこれは大きな欠点ではありません。このゼロ知識証明システムの構築から学んだことを活用して、より一般的なアプローチを開発できます。

ここでSNARKが登場します。「歴史」セクションですでに述べた通り、SNARKは2011年のBIT+11論文で任意の計算に対するゼロ知識証明を構築するための汎用フレームワークとして導入されました。わずか2年後にはPinocchio論文が現実世界のアプリケーションで使用可能な最初の実装を作成しました。

SNARKシステムは、非対話型で機能するために*トラステッドセットアップ*と呼ばれる暗号学的な秘密に依存しています。トラステッドセットアップは、暗号プロトコルのすべての参加者が共有する共通の知識基盤と捉えることができます。具体的には、いくつかの秘密を生成し、それを使ってSNARKプロトコルの正しい実行に必要な値（証明鍵と検証鍵）を計算する初期化フェーズです。

通常、トラステッドセットアップは*マルチパーティ計算*を使用して取得します。多くのアクターがそれぞれ異なる秘密（*トキシックウェイスト*とも呼ばれる）を生成し、証明鍵と検証鍵を計算した後、初期の秘密を削除します。その後これらの鍵を合算して、SNARKプロトコルで使用される2つの最終的な鍵を得ます。この方法の主な利点は、1つのエンティティがトキシックウェイストを削除するだけで、誰も不正ができずトラステッドセットアップが改ざん防止された方法で生成されたと確信できることです。第4章のKZGコミットメントの議論でこの内容を取り上げました。

SNARKプロトコルは完全に機能しますが、トラステッドセットアップの1-of-N信頼仮定と、最終的な公開鍵を生成する堅牢な初期セレモニーの作成が困難である点は、常に多くの議論を生んできました。こうした課題が、トラステッドセットアップなしで動作するトラストレスなゼロ知識証明システムを研究する動機を複数の研究者や企業に与えました。さらにSNARKシステムは耐量子性のない楕円曲線暗号に依存しており、この点も重要な課題として指摘されています。

2018年、エリ・ベンサッソン、イド・ベントフ、イノン・ホレシュ、マイケル・リアブゼフの論文"Scalable, Transparent, and Post-Quantum Secure Computational Integrity"が、汎用ゼロ知識証明システムを構築するための新しいトラストレスなフレームワークであるzk-STARKを導入しました。

STARKは「*scalable transparent argument of knowledge*」の略です。特に*transparent*はトラステッドセットアップを必要としないという画期的な特性を指します。また楕円曲線暗号の代わりに衝突耐性ハッシュ関数に依存しているため、耐量子性も備えています。

これらの利点から、現在ほとんどの人がSTARKを最もモダンなゼロ知識証明システムと見なしています。しかしSNARKは時代遅れの技術ではありません。特定のコンテクスト、主に証明サイズにおいて利点があり、ブロックチェーンのような帯域幅が制約される環境では基本的に重要です。さらに両方の長所を取り入れようとする複数の混合アプローチも開発されています。

zk-EVMとzk-VM

ゼロ知識証明システムの仕組みについて技術的に深掘りしたので、ここでイーサリアムに戻り近年の技術の進化を見ていきましょう。

ゼロ知識証明システムを使ってイーサリアムを改善する方法、つまりブロックのEVM実行を証明してフルノードがブロック内のすべてのトランザクションを再実行せずにステートをトラストレスに更新できるようにする方法については、すでに説明しました。ゼロ知識証明を検証するだけで、有効であれば新しいステートが正しく計算されたことがわかります。また特定の計算（分割問題）に対するゼロ知識証明システムの基本的な作成例を見てから、SNARKおよびSTARKフレームワークを紹介しました。これらのフレームワークにより、アーキテクチャを一般化して任意の計算に適用できるようになります。

そこですぐに疑問が湧くかもしれません。SNARKまたはSTARKのゼロ知識証明システムを使ってEVMブロック実行を正しく証明できるでしょうか。はい、確実にできますが、いくつかの選択が必要です。

分割問題の例では、最初の命題（整数の集合を2つの部分集合に分割して合計が等しくなるか）を、後に構築したシステムで証明しやすい形に少し変換する必要がありました。つまり充足割り当て`a`と、それが有効であるために満たすべき3つの制約を追加しました。この変換は問題を数学的に取り扱えるようにするために必要です。SNARKおよびSTARKフレームワークにも同様に適用されます。

SNARKまたはSTARKフレームワークを使って任意の計算をゼロ知識で証明できることは確かですが、まずフレームワークが正しく処理・実行できる形式（通常*回路*と呼ばれる）に計算を変換する必要があります。これは非常に困難なタスクです。特にEVMステート遷移関数のような複雑な計算をゼロ知識回路に変換するとエラーが起きやすく、問題がある場合のデバッグも困難です。回路が正しくないかバグがある場合、生成されるゼロ知識証明は実際の計算を反映しないため信頼できません。偽のステートメントに対して有効なゼロ知識証明を生成する（つまり正しいEVMステートが「state2」であるべきなのに「state1」であると主張する）ことすら可能になりかねません。

ゼロ知識証明生成システムを含むEVM実装は*zk-EVM*と呼ばれます。具体的なzk-EVMはZKsyncリポジトリで見ることができます。

当初、ZKロールアップを構築しようとするプロジェクトはいずれも、EVMステート遷移関数の正しいゼロ知識回路を作成するという困難な作業に取り組む必要がありました。執筆時点（2025年6月）でオプティミスティックロールアップの方がZKロールアップよりもはるかに多いのはそのためです。しかしzk-VMの登場により状況は急速に変化しています。

*zk-VM*は、ZKロールアップの構築、そしてより一般的にはEVMステート遷移関数のゼロ知識証明システムの構築に対する視点を劇的に変えます。アイデアはシンプルです。SNARKまたはSTARKフレームワークを適用するために異なる計算ごとにカスタム回路を作成するのではなく、あらゆる種類の計算に使えるユニバーサル回路を作成できるとしたらどうでしょうか。任意の計

算を処理できる一種の汎用ゼロ知識コンピュータのようなものです。この抽象化レイヤーは非常に強力です。「1つの回路がすべてを支配する」のです。

この方法により証明したい計算にのみ集中すればよく、zk-VMが証明生成の困難な作業をすべて処理します。現在最も有名なzk-VMはSP1とRISC Zeroです。

図17-5は、zk-EVMとzk-VMの両方のフレームワークの核心的な要素を捉えた簡潔な視覚化を提供しています。

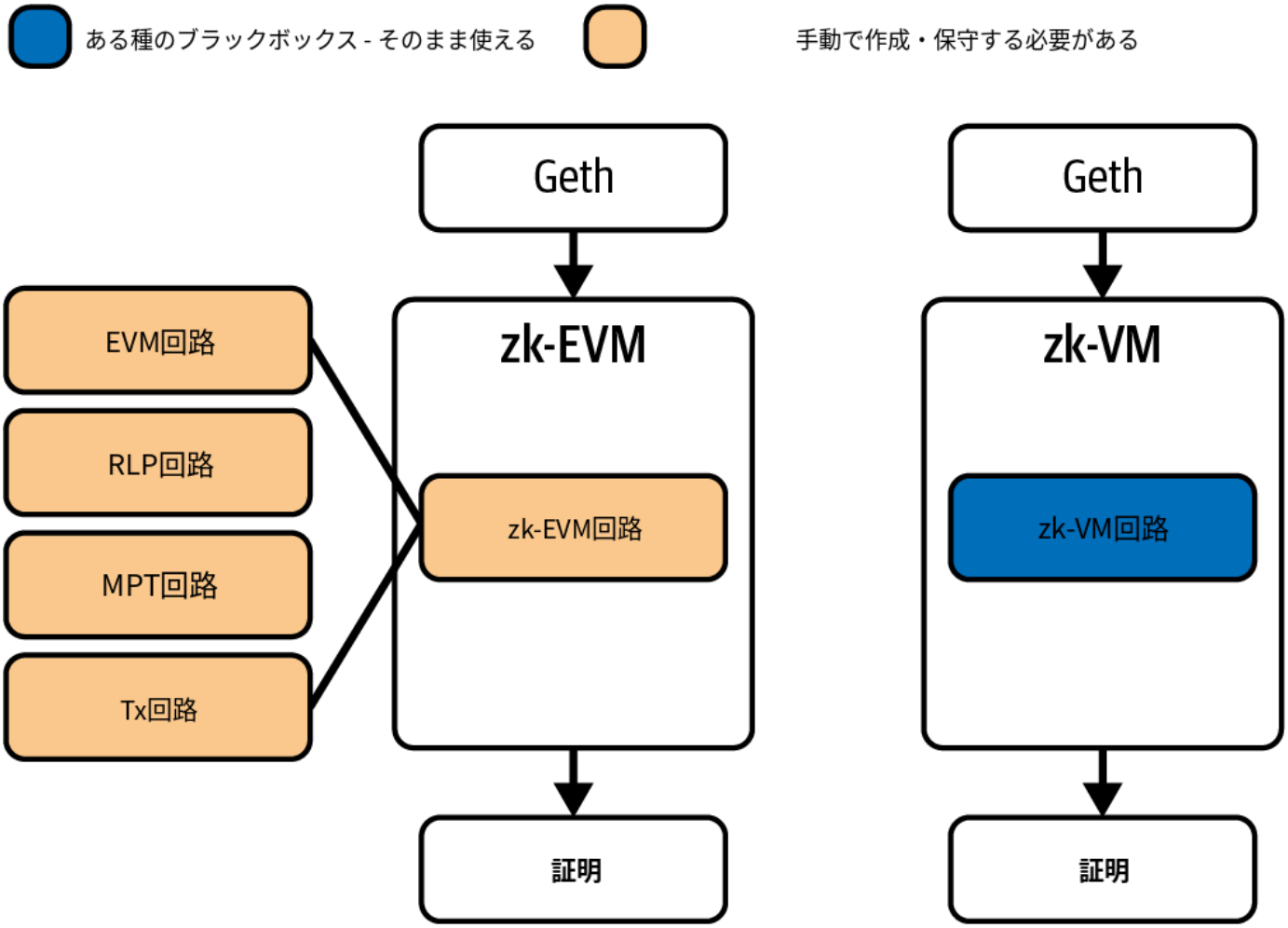


図17-5 zk-EVMとzk-VMフレームワークの比較

まとめ

この最終章ではゼロ知識技術の基本を探求しました。1985年の発明から、ゼロ知識技術を現実世界のアプリケーション、特にブロックチェーンの世界に真にもたらしたBIT+11論文とPinocchio論文の画期的なイノベーションまでを辿りました。小さな分割問題を例に用いて、ゼロ知識証明システムが内部でどのように機能するかの直感を養いました。その後、SNARKやSTARKフレームワークのようなより一般化されたアプローチを検討しました。これらのフレームワークにより、任意の計算に対してゼロ知識証明システムを構築する標準的な方法論を適用できます。最後にzk-VMを紹介しました。zk-VMは単一の回路で任意の計算を処理できるため、開発

者が特定の計算に結びついたカスタム回路を作成する必要がなくなり、この分野を急速に革新しています。

イーサリアムの未来はL2レベルでもL1上でも、ゼロ知識技術と間違いなく交差するでしょう。最終的に優位に立つ具体的なソリューションはまだ不明です。おそらくバグや障害に対するより大きな耐性を確保するため、さまざまなアプローチの組み合わせになるでしょう。時が経てばわかるはずです…

このテーマについてさらに読むには、以下を参照してください。

- A simple example
- イーサン・ブックマン著"You Could Have Invented SNARKs"
- マクシム・ペトクス著"Why and How zk-SNARK Works: Definitive Explanation"
- The RareSkills Book of Zero Knowledge